



WOLAITA SODO UNIVERSITY
SCHOOL OF INFORMATICS
DEPARTMENT OF INFORMATION
TECHNOLOGY

SYSTEM OR SOFTWARE DEVELOPMENT THEME

Theme Content

- Object oriented programming (Java)
- Project management for IT
- Advanced programming
- Event driven programming
- System analysis and design

March, 2023

Wolaita Sodo, Ethiopia

Course Name

Object Oriented Programming in Java

Compiled by:

Tadele Shashego (MSc)

Reviewed by:

Birhanesh Fikre (MSc)

Table of Contents

Object Oriented Programming in Java	I
CHAPTER ONE	1
1 Introduction to Object-Oriented Programming	1
1.1 Types of programming paradigm.....	2
1.2 Types of Structured Programming.....	3
1.3 Overview of OO principles	8
1.4 Java Language Specification and Basic Terms of OOP in Java	10
1.5 Overview of Java Programming and types of Java Program	12
1.5.1 Definition of Java Application, Java Applets	17
1.6 Chapter Review Questions.....	19
Chapter Two.....	20
2 Basics in Java Programming	20
2.1 Introduction.....	20
2.2 Variable types and identifiers	21
2.3 Number types, strings, constants.....	27
2.3.1 Number Types.....	27
2.3.2 The String Type	31
2.4 Type Conversion/ Casting.....	34
2.4.1 . Numeric Type Conversion.....	35
2.4.2 Casting between char and Numeric Types.....	35
2.4.3 Converting Strings to Numbers and Vice Versa.....	36
2.5 Decision and Repetition Statements.....	37
2.5.1 . Sequential Statements	37
2.5.2 Branching Statements	37
2.6 Arrays and working with arrays.....	45
2.7 Chapter Review Questions.....	50
Chapter Three.....	51
3 Objects and Classes	51
3.1 Introduction.....	51
3.2 Defining Classes and Objects	51

3.2.2	Declaring Object	55
3.2.3	Constructing Object using Constructor.....	58
3.2.4	Static Variables, Constants, and Methods.....	60
3.3	Accessibility Modifiers.....	61
3.3.1	Public	61
3.3.2	. Protected	61
3.3.3	Default.....	62
3.3.4	Private	62
3.4	Data Field Encapsulation	62
3.5	Passing Objects to Methods	63
3.6	Array of Objects.....	64
3.7	Chapter Review Questions.....	66
Chapter Four		67
4	Object Oriented Programming Concepts.....	67
4.1	Introduction.....	67
4.2	OOP Principles in Java	67
4.2.1	. Encapsulation.....	67
4.2.2	Inheritance.....	69
4.2.3	Method Overloading.....	74
4.2.4	Overriding Methods	75
4.2.5	Polymorphism	77
4.2.6	Abstraction.....	79
4.2.7	Abstract Class	80
4.2.8	Interfaces.....	82
4.2.9	Packages.....	86
4.2.10	Template	88
4.3	Chapter Review Questions.....	91
Chapter Five.....		93
5	Exception Handling.....	93
5.1	Introduction.....	93
5.2	Exception handling overview.....	93

5.2.1	Introduction.....	93
5.2.2	Exception Handling	93
5.2.3	Types of Exception	97
5.2.4	The finally Clause	99
5.3	Input/Output (I/O) Streams in Java	99
5.3.1	Introduction.....	99
5.3.2	Types of I/O Streams	100
5.4	Chapter Review Questions.....	107
Chapter Six.....		110
6	GUI & Java Applets	110
6.1	Introduction.....	110
6.2	Swing versus AWT	110
6.3	The Java GUI API.....	111
6.3.1	. The GUI Container Classes	111
6.3.2	The GUI Component Classes.....	118
6.3.3	The GUI Helper Classes	127
6.3.4	Overview of Graphics Class in Java	141
6.3.5	Overview of Java Applets.....	145
6.3.6	Applet Life-Cycle Methods	150
6.4	Java Beans.....	154
6.4.1	Bean Properties	155
6.4.2	Uses of Java Beans.....	157
6.5	Chapter Review Questions.....	158
I.	Module description.....	161
II.	Objectives of this Module	161
CHAPTER ONE		162
INTRODUCTION TO IT PROJECT MANAGEMENT		162
1.1	Introduction.....	162
1.2	Importance of IT project management.....	162
1.1.1.	What is a Project?	163
1.1.2.	Project Attributes:.....	163

1.1.3. Project Constraints.....	164
1.1.4. Problems with IS Project:	165
1.1.5. What is Project Management?	165
1.1.1 What is a project?	165
1.2. Stages of Project	166
1.2.1. The Feasibility Study	167
1.2.2. Planning	167
1.2.3. Project Execution.....	168
1.2.4. Project and Product Life Cycles	168
1.3. The Stakeholder of a Project.....	169
1.3.1. All Parties of a Project.....	170
1.3.2. The Role of Project Manager.....	171
1.4. Ethics in Project Management	172
1.5. Project Management Framework	172
1.6. Program and Project Portfolio Management.....	174
1.7. Software Tools for Project Management	175
1.8. Chapter one review question.....	176
CHAPTER TWO	177
PROJECT PLANNING	177
2.1. Introduction.....	177
What is project planning?	177
2.1. Integration Management	177
2.2. Integration Management	177
2.2.1. What is Integration Management?.....	177
2.2.2. Project Plan Development	178
1.2.1 What is Integration Management?	180
2.3. Scope Management	180
2.1.2. Project Plan Development	180
2.1.3. Plan Execution	180
2.2 Scope Management	180
2.2.1 What is Scope Management?	180

2.3.2. Methods for Selecting Projects.....	181
2.2.2 Methods for Selecting Project	182
2.2.3 Project Charter	183
2.2.4 Scope Statement	183
2.2.5 Work Breakdown Structure	183
2.4. Stepwise Project Planning.....	183
2.4.1. Overview of Project Planning.....	183
2.4.2. Main Steps in Project Planning	184
2.3.1 Overview of Project Planning.....	186
2.3.2 Main Steps in Project Planning	187
2.5. Chapter two review questions	187
CHAPTER THREE	188
PROJECT SCHEDULING	188
3.1. Introduction.....	188
3.2 Time Management	188
3.2.1. Importance of Project Schedules	188
3.2.2. Schedules and Activities.....	189
3.1.3. Sequencing and Scheduling Activity.....	191
3.2. Project Network Diagrams.....	191
3.4. Chapter three review questions	192
CHAPTER FOUR.....	193
PROJECT COST MANAGEMENT.....	193
4.1. Introduction, Importance and Principles of Project Cost Management	193
4.2. Planning Cost Management	196
4.3. Cost Estimating	197
4.4. Cost Budgeting.....	198
4.5 Cost Control	199
4.6. Chapter four review questions	202
CHAPTER FIVE	204
PROJECT QUALITY MANAGEMENT	204
5.1 Quality of IT Project	205

5.2 Stages of IT Quality Management	205
5.2.1 Quality Planning	206
5.2.2 Quality Assurance.....	206
5.2.3 Quality Control	206
5.3 Quality Standards.....	206
5.4.1. Quality Planning.....	206
5.4 Tools and Techniques for Quality Control.....	207
5.6. Chapter five review questions.....	208
CHPATER SIX	209
PROJECT HUMAN RESOURCES MANAGEMENT	209
6.1. What is Project Human Resources Management?	209
6.2 Managing People	210
6.3. Organizational Planning.....	211
6.4. Issues in Project Staff Acquisition and Team Development.....	212
6.6. Chapter six review questions	212
CHAPTER SEVEN	214
PROJECT COMMUNICATION MANAGEMENT	214
7.1. Communications Planning.....	214
7.2. Planning Communications Management	214
7.2. Information Distribution	214
7.3. Performance Reporting	214
7.4. Administrative Closure	217
7.5. Suggestions for Improving Project communications	218
7.7. Contract Closure:	218
6.6 Chapter seven review questions.....	219
CHAPTER EIGHT	220
PROJECT RISK MANAGEMENT	220
8.1. The Importance of Project Risk Management	220
8.2. Common Sources of Risk in IS project.....	221
8.3. Risk Identification	222
8.4. Risk Quantification	225

8.5. Risk Response Development and Control.....	226
CHAPTER NINE.....	229
PROJECT PROCUREMENT MANAGEMENT	229
9.1. Importance of Project Procurement Management.....	229
9.2. Importance of project procurement management	229
9.3. Procurement Planning.....	231
9.4. Solicitation (Procurement Document)	233
9.5. Source Selection.....	235
9.6. Contract Administration.....	236
9.7. Closing Procurement.....	237
9.8. Chapter nine review questions	238
CHAPTER TEN.....	239
PROJECT MANAGEMENT PROCESS GROUPS.....	239
10.1 Introduction to Project Management Process Groups.....	239
10.2. Phases of Project Management Process Groups	239
10.2.1. Project Initiation → Initiating Process Group	240
10.2.2. Project Planning → Planning Process Group	241
10.2.3. Project Executing → Executing Process Group	241
10.2.4. Project Monitoring and Controlling → Monitoring and Controlling Process Group	242
10.2.5. Project Closing → Closing Process Group.....	242
10.3. Chapter ten review questions	243
CHAPTER ONE	245
1 Multithreading.....	245
1.1 Introduction.....	245
1.2 What are Threads?	246
1.3 Life cycle of a Thread (Thread states)	248
Thread States	249
1.4 Creating a Thread.....	252
1.5 Thread priorities.....	256
1.6 Synchronization	256
1.6.1 Types of Synchronization	256

1.	Process Synchronization	256
2.	Thread Synchronization	257
1.7	Implementation of Threads States.....	257
1.7.1	Determining When a Thread Ends.....	263
1.8	Producer/consumer relationship without Synchronization	265
1.9	Producer/consumer relationship with Synchronization.....	269
1.10	Producer/consumer relationship: The Circular Buffer	270
1.11	Daemon Threads	271
1.12	Runnable Interfaces.....	273
1.13	Chapter Review Questions.....	276
	Chapter Two.....	277
2	Networking.....	277
2.1	Introduction.....	277
2.2	Sockets	278
2.3	Manipulating a File on a Web Server	281
2.4	Establishing a Simple Server Using Stream Sockets	284
2.5	Establishing a Simple Client Using Stream Sockets.....	285
2.6	Client/Server Interaction with Stream Socket Connection	286
2.7	Connectionless Client/Server Interaction with Datagram.....	287
2.8	Security and the Network.....	291
2.9	Chapter Review Questions.....	292
	CHAPTER THREE	292
3	Java Database connectivity with JDBC.....	292
3.1	Introduction.....	292
3.2	Relational Database Systems	296
3.3	Creating Database and Manipulating Data	297
3.3.1	SQL	297
3.3.2	Creating a User Account on MySQL.....	297
3.4	Creating Database	298
3.5	JDBC Drivers.....	300

3.6	The JDBC API	300
3.6.1	Common JDBC Components.....	301
4	Developing Database Applications Using JDBC	302
4.1.1	Creating JDBC Application	302
4.2	JDBC Drivers Types	303
4.3	Chapter Review Questions.....	305
Chapter Four		306
5	SERVLETS	306
5.1	Introduction.....	306
5.2	Servlet Overview and Architecture.....	307
5.2.1	Components of Servlet Architecture.....	307
5.2.2	How does a Servlet Request flow?	308
5.3	Life Cycle of a Servlet	309
5.3.1	Servlet Life Cycle Methods	311
	Servlet Life Cycle:	313
5.3.2	Handling Servlet Life-Cycle Events	313
5.4	Defining the Listener Class.....	313
5.4.1	Specifying Event Listener Classes.....	315
5.5	Handling Servlet Errors	315
5.6	Handling HTTP get Request	320
5.6.1	Handling HTTP get Requests Containing Data	320
5.6.2	Handling HTTP post request	324
5.7	Redirecting Requests to Other Resources	328
5.7.1	REDIRECTING REQUESTS TO OTHER RESOURCES.....	328
5.8	Multi-Tier Applications: Using JDBC from a Servlet	331
5.9	Chapter Review Questions.....	338
CHAPTER FIVE		339
6	Java Server Pages	339
6.1	Introduction.....	339
6.2	Java Server Pages Overview	339
6.2.1	Java Server Pages.....	339

6.2.2	Why Use JSP?.....	339
6.3	JSP Architecture.....	340
6.3.1	The Page-Centric Approach.....	340
	The Dispatcher Approach.....	341
6.4	JSP Processing and its flow	342
6.5	Applications and Advantages of JSP	343
6.6	Setting up the environment for JSP.....	343
6.7	LIFE CYCLE	346
6.8	Chapter Review Exercises	354
7	Chapter four.....	401
7.1.2	Brief introduction to the usages of SQL data bases.....	424
7.1.3	Database Object	424
7.1.4	RecordSet Object	425
7.1.5	Accessing records	425
7.1.6	Searching the RecordSet.....	426
7.1.7	Updating the Database	426
7.1.8	Deleting and Adding records	427
7.1.9	Deleting.....	427
7.1.10	Adding.....	427
	SYSTEM ANALYSIS AND DESIGN.....	428
	References.....	597

CHAPTER ONE

1 Introduction to Object-Oriented Programming

Object means a real-world entity such as a pen, chair, table, computer, watch, etc. Object-Oriented Programming is a methodology or paradigm to design a program using classes and objects. It simplifies software development and maintenance by providing some concepts: Object, Class, Inheritance, Polymorphism, Abstraction, and Encapsulation.

Apart from these concepts, there are some other terms which are used in Object-Oriented design: Coupling, Cohesion, Association, Aggregation, and Composition

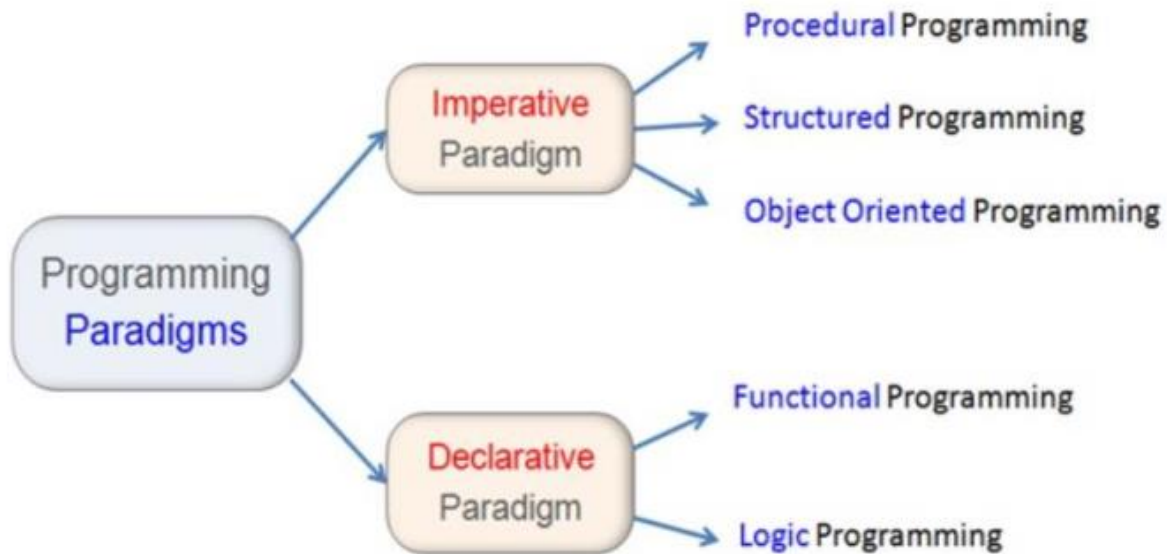
1.1 Types of programming paradigm

Programming paradigms are different ways or styles in which a given program or programming language can be organized. Each paradigm consists of certain structures, features, and opinions about how common programming problems should be tackled.

The question of why are there many different programming paradigms is similar to why are there many programming languages. Certain paradigms are better suited for certain types of problems, so it makes sense to use different paradigms for different kinds of projects.

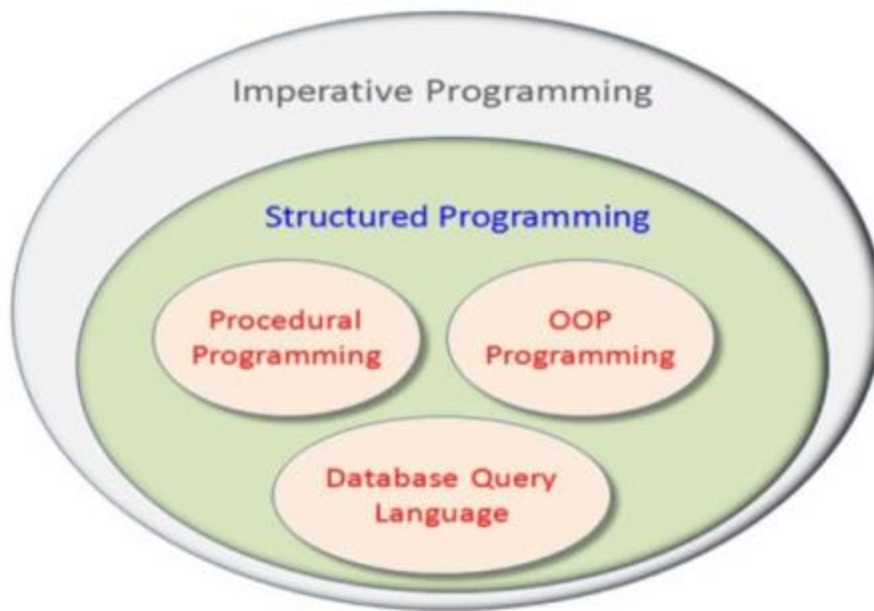
The word paradigm derives from Greek and Latin. It simply means pattern, model or a set of rules that define a standard. However, in the context of programming language, it refers to a set of design principles that defines the program structure. It is all about how we model and organize the program code.

The programming paradigms can be classified into two main types:



1. Imperative Paradigms

They are said to be command driven. This paradigm directs the written code execution as sequence of statements executed one by one. The imperative style program consists of set of program statements. Each statement directs the computer to perform specific task. In this style the programmer has to elaborate each statement in details. Each statement directs what is to be done and how it is to be done.



The execution of the program statements is decided by the control flow statements. And the program flow can be directed as per the program logic.

Control flow in imperative programming is explicit: commands show how the computation takes place, step by step. Each step affects the global state of the computation.

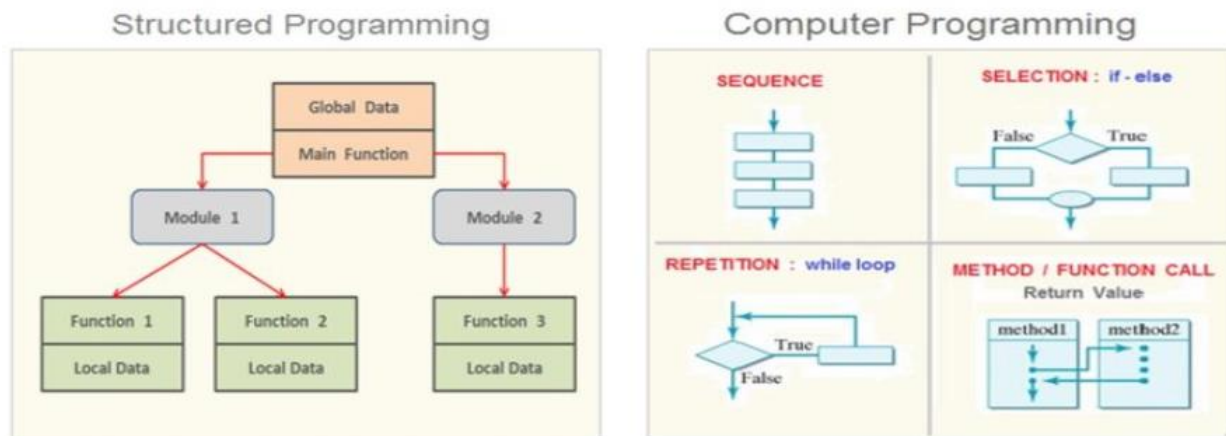
```
    result = []
    i = 0
start:
    numPeople = length(people)
    if i >= numPeople goto finished
    p = people[i]
    nameLength = length(p.name)
    if nameLength <= 5 goto nextOne
    upperName = toUpper(p.name)
    addToList(result, upperName)
nextOne:
    i = i + 1
    goto start
finished:
    return sort(result)
```

Types of imperative programming

a. Structured Programming

The structured programming intends to optimize the code by using the program control flow constructs, decision making (IF, IF THEN, ELSE) constructs and the iteration constructs (for, while loops), blocks and functions. The main intent of the structured programming is to improve the readability of the code. This facilitates the re-usability of the code by dividing the code into semi-independent modules.

It is also called **Modular Programming**.



The structured programming does not advocate the use of unconditional control statements such as goto constructs. The use of goto constructs can result into spaghetti code that is difficult to debug.

Examples of Languages that support the structural programming paradigm:- C++.

Structured programming is a kind of imperative programming where control flow is defined by nested loops, conditionals, and subroutines, rather than via gotos. Variables are generally local to blocks (have lexical scope).

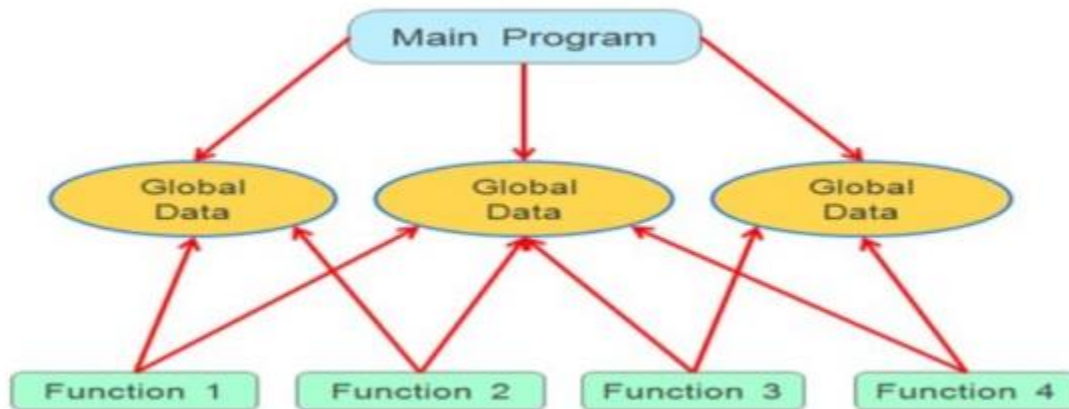
```
result = [];
for i = 0; i < length(people); i++ {
    p = people[i];
    if length(p.name) > 5 {
        addToList(result, toUpper(p.name));
    }
}
return sort(result);
```

1.2 Types of Structured Programming

A. Procedural Programming

This paradigm consists of set of procedures which is referred as function, method or subroutines. Each function performs specific operation. The function consists of group of computational steps

that directs the computer to perform specific operation. The function once defined can be called many times in the program to repeat the same operation.



The programmer can either use standard library functions or create library of user defined functions.

Examples of Languages that support the procedural programming paradigm:

FORTRAN C, C++, Java, Ruby, PHP, JavaScript, Python, Pascal, ColdFusion and COBOL.

Procedural example in C language.

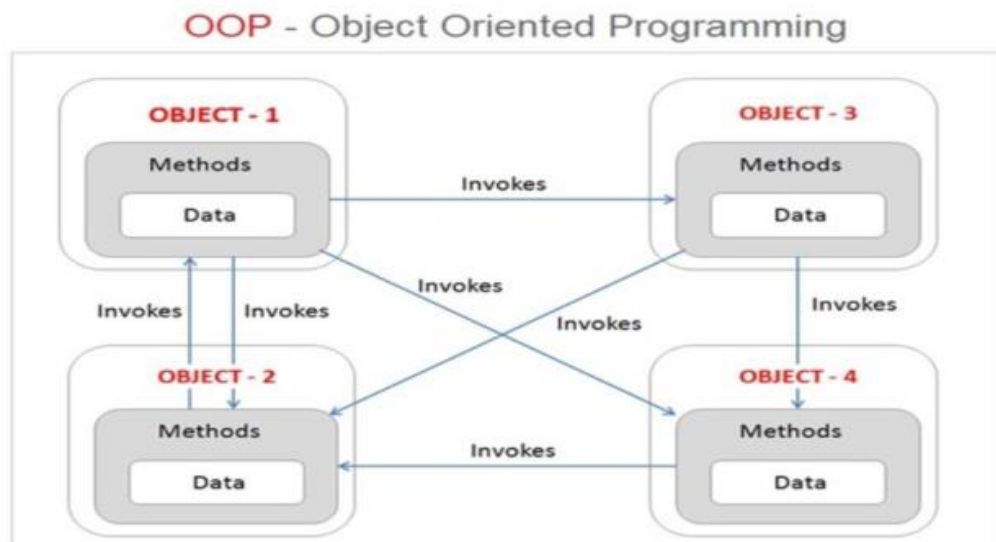
```
#include<stdio.h>

int main() {
    int result = 0;
    for (int i = 0; i < 10; i++) {
        result = result + i;
    }
    printf("Result = %d", result);
    return 0;
}
```

B. Object Oriented Programming

OOP is type of structured programming that views the program components as object. In OOP programming, all the program components are represented as objects. An object binds the data

and the associated methods together as single unit. The programmer can control the data access permissions by defining the access specifier. They robust security features.



✓ Example C++, Java, Python, C#, PHP, VB, Ruby, Smalltalk.

C. Database Query Language

Database is vital component of most software projects. Relational Database Management System (RDBMS) support the SQL for database programming. Structured Query Language (SQL) is the most commonly used example of model based language.

```
CREATE DATABASE personalDetails;
CREATE TABLE Persons (
    PersonID int,
    LastName varchar(255),
    FirstName varchar(255),
    Address varchar(255),
    City varchar(255)
);
```

```
SELECT * FROM Books
WHERE Category='Programming'
AND language='english';
```



2. Declarative Paradigm

This paradigm focuses on the logic of the program and the end result. The control flow is not the important element of the program. The main focus is achieving the end result. It is called a straight forward paradigm. The declarative paradigm focuses on what is to be solved.

Types of Declarative Programming

a. Functional Programming

As its name suggests, functional programming is based on mathematical functions. It attempts to solve the problem by composing mathematical functions as program components. The computer program is created with the application and composing the mathematical functions. It is also known as a Parallel processing approach.



The advantage of functional programming is the program can run on multicore and multithreaded environment.

Examples of Functional Programming languages: javascript, Scala, Lisp, SQL, XQuery, Clojure, ajax, c++, java and etc.

Example of Functional programming is the key feature of JavaScript.

In functional programming, the program written in programming language will always produce output exactly similar to its equivalent mathematical function.

For example, in this function: $f(x) = x + 2$ for a given value of x , the output will be the same.

If $x=4$ then the output will be $f(4) = 4 + 2 = 6$

If $x=5$ then the output will be $f(5) = 5 + 2 = 7$

The immutability is one of the most important feature of the functional programming

For example, if we create a variable `var a=100`; the variable value will remain same for life span of the variable. `Var b=a+10`; the value of `a` is the same.

b. Logic Programming

This declarative programming paradigm that is based on the logic and control. The term logic means essentially facts and rules. Whereas control means, an order of rules. The term algorithm

is defined as logic plus control (Logic + Control). The logic defines what should be solved and the control defines how it should be solved.

The program written in logic programming language consists of set of program statements in the logical form. Each statement is an expression of facts, rules and order of rules. The logic programming is an abstract model of computation. It is based on the first order predicate logic (FOL).

A logical program is divided into three sections:

1. a series of definitions/declarations that define the problem domain
2. statements of relevant facts
3. statement of goals in the form of a query

Example of Logic programming is Prolog, F-Logic and etc.

A programming language is essentially a problem solving tool. For each problem there may be many solutions. Each solution can adopt a different approach in providing the solution.

1.3 Overview of OO principles

The fundamental object-oriented principles.

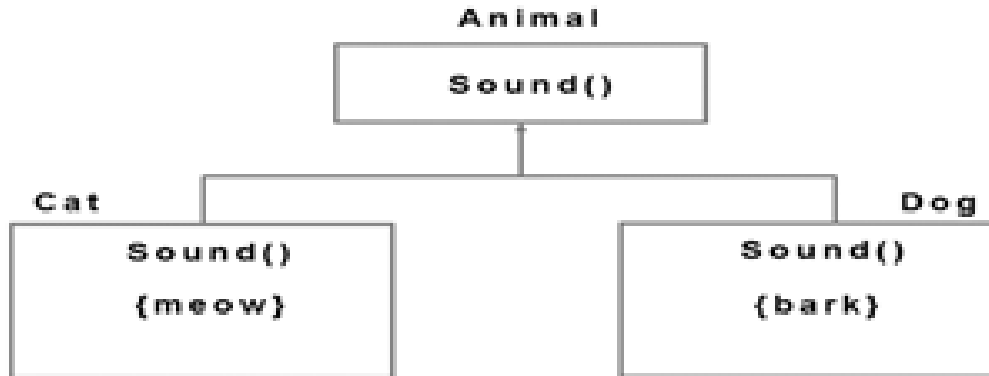
- **Abstraction** – it allows the developer to consider the complex ideas and ignores the irrelevant details. Generally, it allows the developers or users to consider the important high level of the system. **Data abstraction** is the reduction of a particular body of data to a simplified representation of the whole. Data abstraction is usually the first step in database design.
- **Encapsulation** – it allows the developers to focus on what something does without considering the complexity of how it works. Generally, it allows the developer or user how the system does, and the user operates, without needing worry about the technical details how it actually works. Here you hide unnecessary details in classes and deliver a simple and clear interface for working. It describes the idea of bundling data and methods that work on that data within one unit. **Encapsulation** is a programming mechanism that binds together code and the data it manipulates, and that keeps both safe from outside interference and misuse. In an object-oriented language, code and data can be bound together in such a way that a self-contained **black**

box is created. Within the box are all necessary data and code. When code and data are linked together in this fashion, an object is created. In other words, an object is the device that supports encapsulation.

Within an object, code, data, or both may be **private** to that object or **public**. Private code or data is known to and accessible by only another part of the object. That is, private code or data cannot be accessed by a piece of the program that exists outside the object. When code or data is public, other parts of your program can access it even though it is defined within an object. Typically, the public parts of an object are used to provide a controlled interface to the private elements of the object.

Java's basic unit of encapsulation is the class. A class defines the form of an object. It specifies both the data and the code that will operate on that data. Java uses a class specification to construct objects. Objects are instances of a class. Thus, a class is essentially a set of plans that specify how to build an object. The code and data that constitute a class are called members of the class. Specifically, the data defined by the class are referred to as member variables or instance variables. The code that operates on that data is referred to as member methods or just methods. Method is Java's term for a subroutine. If you are familiar with C/C++, it may help to know that what a Java programmer calls a method, a C/C++ programmer calls a function.

- **Generalization / Specialization** – a way to use **inheritance**. Generalization allows the developer or user to consider the general categories of objects which have a common property and then define specialized sub-classes that inherit the properties of the general categories.
- **Inheritance** – Inheritance is one of the core concepts of object-oriented programming (OOP) languages. It is a mechanism where you can derive a class from another class for a hierarchy of classes that share a set of attributes and methods. It explains how the class hierarchies develop code readability and support to the reuse of functionality.
- **Polymorphism** – is the ability to interact with an object as its generalized category regardless of its more specialized category. Polymorphism is the process of using same method name by multiple classes and redefines methods for the derived classes.



Benefits of object-oriented programming approach:

- Better abstraction
- Better maintainability

1.4 Java Language Specification and Basic Terms of OOP in Java

Computer languages have strict rules of usage. If you do not follow the rules when writing a program, the computer will be unable to understand it. The Java language specification and Java API define the Java standard. The Java language specification is a technical definition of the language that includes the syntax and semantics of the Java programming language. The Java language specification is stable, but the API is still expanding.

- **Class** – it is unit of program that houses a set of methods and attributes. **Example:** a class that represents a bank account might contain one method to deposit money to an account, another to withdraw money from an account and a third to inquire what the account's current balance is.

```

Class Account
{
    int account_number;
    int account_balance;
    public void showdata()
    {
        system.out.println("Account Number"+account_number)
        system.out.println("Account Balance"+ account_balance)
    }
}

```

Here you have a Class named as: **Account**

Attributes are: account_number, account_balance

Method/Action is: showdata()

- **Object** – it is an instance of its class. Real-world objects share two characteristics – They all have state and behavior. Let us see the following pictorial example to understand Objects.



In the above diagram, the object 'Dog' has both state and behavior. An object stores its information in attributes and discloses its behavior through methods.

- **Method** – it is a way to perform a task in a program. It houses the program statements that actually perform its tasks.

- **Message** – it is something which is send within an object or class. **Example:** When you drive a car, pressing its gas pedal sends a message to the car to perform a task, that is, to go faster.
- **Method Call** – Each message is implemented as a method call that tells a method of the object to perform its task.
- **Instantiation** – it is the process of build an object of a class before a program can perform the tasks that the class's methods define.
- **Reuse** – Reuse of existing classes when building new classes and programs saves time and effort. Reuse also helps you build more reliable and effective systems, because existing classes and components often have gone through extensive testing, debugging and performance tuning.
- **Attribute** – is a visible or an external specification of an object. **Example:** If you consider car, its attributes are like color, number of doors, speed, etc.
- **Instance Variable** – it is a name that specifies the classes attributes. Example: a bank account object has a balance attribute that represents the amount of money in the account. Each bank account object knows the balance in the account it represents, but not the balances of the other accounts in the bank.

1.5 Overview of Java Programming and types of Java Program

Java – the world's most widely used computer programming language. Java is the preferred language for meeting many organizations' enterprise programming needs. Java has also become the language of choice for implementing Internet-based applications and software for devices that communicate over a network. Java programming language was originally developed by Sun Microsystems which was initiated by James Gosling and released in 1995 as core component of Sun Microsystems' Java platform (Java 1.0 [J2SE]).

Sun Microsystems in 1991 funded an internal corporate research project led by **James Gosling**, which resulted in a C++-based object-oriented programming language Sun called Java. As of December 2008, the latest release of the Java Standard Edition is 6 (J2SE). With the advancement of Java and its widespread popularity, multiple configurations were built to suite various types of platforms. Ex: J2EE for Enterprise Applications, J2ME for Mobile Applications. Sun Microsystems has renamed the new J2 versions as Java SE, Java EE and Java ME, respectively. A key goal of Java is to be able to write programs that will run on a *great variety of*

computer systems and computer-control devices. This is sometimes called “*write once, run anywhere.*” Java is guaranteed to be Write Once, Run Anywhere.

Java Editions

There are basically four editions of java.

- **Java Standard Edition (SE)** – it is used to develop client-side standalone applications or applets. It defines everything from the basic types and objects of the Java programming language to high-level classes that are used for networking, security, database access, graphical user interface (GUI) development, and XML parsing. Consists of a virtual machine, development tools, deployment technologies, and other class libraries and toolkits commonly used in Java technology applications.
- **Java Enterprise Edition (EE)** – is geared toward developing large-scale, distributed networking applications and web-based applications. In the past, most computer applications ran on “standalone” computers (computers that were not networked together). Today’s applications can be written with the aim of communicating among the world’s computers via the Internet and the web.
- **Java Mobile Edition (ME)** – is geared toward developing applications for small, memory-constrained devices, such as BlackBerry smartphones. The Java ME platform provides an API and a small-footprint virtual machine for running Java programming language applications on small devices, like mobile phones. The API is a subset of the Java SE API, along with special class libraries useful for small device application development. Java ME applications are often clients of Java EE platform services.
- **JavaFX** – it is a platform for creating rich internet applications using a lightweight user-interface API. JavaFX applications use hardware-accelerated graphics and media engines to take advantage of higher-performance clients and a modern look-and-feel as well as high-level APIs for connecting to networked data sources. JavaFX applications may be clients of Java EE platform services.

Features of Java

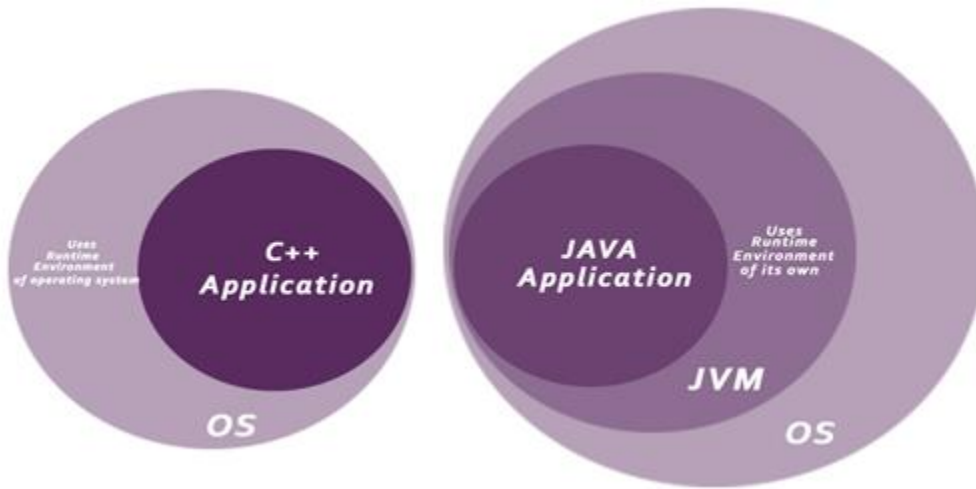
Programmers have accepted java very quickly because it provides everything that is needed in a modern day language including the following features; Java is:

- **Object Oriented** – In Java, everything is an Object. Java can be easily extended since it is based on the Object model.
- **Platform Independent** – Unlike many other programming languages including C and C++, when Java is compiled, it is not compiled into platform specific machine, rather into platform-independent byte code. This byte code is distributed over the web and interpreted by the Virtual Machine (JVM) on whichever platform it is being run on. Platform-independence is a program's capability of moving easily from one computer system to another. A platform is the hardware or software environment in which a program runs. There are two types of platforms software-based and hardware-based. Java provides a software-based platform. The Java platform differs from most other platforms in the sense that it is a software-based platform that runs on the top of other hardware-based platforms.

It has two components:

- a. Runtime Environment
 - b. API (Application Programming Interface)
- **Simple** – Java is designed to be easy to learn and understand, and its syntax is simple. If you understand the basic concept of OOP Java, it would be easy to master. According to Sun, Java language is a simple programming language because:
 - Java syntax is based on C++.
 - Java has removed many complicated and rarely-used features, for example, explicit pointers, operator overloading, etc.
 - There is no need to remove unreferenced objects because there is an Automatic Garbage Collection in Java.
 - **Secure** – With Java's secure feature it enables to develop virus-free, tamper-free systems. Authentication techniques are based on public-key encryption. Java is secured because:

- No explicit pointer
- Java Programs run inside a virtual machine sandbox



- **Class loader** – it is part of the Java Runtime Environment (JRE) which is used to load Java classes into the Java Virtual Machine dynamically. It adds security by separating the package for the classes of the local file system from those that are imported from network sources.
- **Bytecode Verifier** – it checks the code fragments for illegal code that can violate access right to objects.
- **Security Manager** – it determines what resources a class can access such as reading and writing to the local disk.
- **Architecture-neutral** – Java compiler generates an architecture-neutral object file format, which makes the compiled code executable on many processors, with the presence of Java runtime system and the size of primitive types is fixed. In C programming, int data type occupies 2 bytes of memory for 32-bit architecture and 4 bytes of memory for 64-bit architecture. However, it occupies 4 bytes of memory for both 32 and 64-bit architectures in Java.
- **Portable** – Being architecture-neutral and having no implementation dependent aspects of the specification makes Java portable. The compiler in Java is written in ANSI C with a clean portability boundary, which is a POSIX subset.
- **Robust** – Java makes an effort to eliminate error-prone situations by emphasizing mainly on compile time error checking and runtime checking. Robust simply means strong.

Java is robust because:

It uses strong memory management.

There is a lack of pointers that avoids security problems.

There is automatic garbage collection in java which runs on the Java Virtual Machine to get rid of objects which are not being used by a Java application anymore.

There are exception handling and the type checking mechanism in Java. All these points make Java robust.

- **Multithreaded** – With Java’s multithreaded feature it is possible to write programs that can perform many tasks simultaneously. This design feature allows the developers to construct interactive applications that can run smoothly. A thread is like a separate program, executing concurrently. We can write Java programs that deal with many tasks at once by defining multiple threads. It shares a common memory area.
- **Interpreted** – Java byte code is translated on the fly to native machine instructions and is not stored anywhere. The development process is more rapid and analytical since the linking is an incremental and light-weight process.
- **High Performance** – With the use of Just-In-Time compilers, Java enables high performance. Java is faster than other traditional interpreted programming languages because Java bytecode is “close” to native code. It is still a little bit slower than a compiled language (e.g., C++). Java is an interpreted language that is why it is slower than compiled languages, e.g., C, C++, etc.
- **Distributed** – Java is designed for the distributed environment of the internet. Java is distributed because it facilitates users to create distributed applications in Java. This feature of Java makes us able to access files by calling the methods from any machine on the internet.
- **Dynamic** – Java is considered to be more dynamic than C or C++ since it is designed to adapt to an evolving environment. Java programs can carry an extensive amount of run-time information that can be used to verify and resolve accesses to objects at run-time. It supports dynamic loading of classes. It means classes are loaded on demand. It also supports functions from its native languages, i.e., C and C++.

1.5.1 Definition of Java Application, Java Applets

- **Editor** – You can use any text editor or IDE to create and edit a Java source-code file.
- **Java Development Toolkit (JDK)** – it consists of a set of separate programs, each invoked from a command line, for developing and testing Java programs. Besides JDK, you can use a Java development tool (e.g., NetBeans, Eclipse, and TextPad).

It contains a private Java Virtual Machine (JVM) and a few other resources such as an interpreter/loader (java), a compiler (javac), an archiver (jar), a documentation generator (Javadoc), etc. to complete the development of a Java Application.

- **Integrated Development Environment (IDE)** – it provides tools that support the software development process, including editors for writing and editing programs and debuggers for locating **logic errors** (errors that cause programs to execute incorrectly). Popular IDEs include **Eclipse** (www.eclipse.org) and **NetBeans** (www.netbeans.org).
- **Application Programming Interface (API)** – contains predefined classes and interfaces for developing Java programs.
- **Java Runtime Environment (JRE)** – it is a set of software tools which are used for developing Java applications. It is used to provide the runtime environment. It is the implementation of JVM. It physically exists. It contains a set of libraries + other files that JVM uses at runtime.
- **Java Virtual Machine (JVM)** – is an abstract machine. It is called a virtual machine because it doesn't physically exist. It is a specification that provides a runtime environment in which Java bytecode can be executed. It can also run those programs which are written in other languages and compiled to Java bytecode. JVMs are available for many hardware and software platforms. JVM, JRE, and JDK are platform dependent because the configuration of each OS is different from each other. However, Java is platform independent.

The JVM performs the following main tasks:

Loads code

Verifies code

Executes code

Provides runtime environment

Generally, in java the following are key points.

Development Environment:

The place where software is being developed. Example: Software Company. The required tools to develop a software are

Compiler (javac), debugger (jdb), Java Virtual Machine or Interpreter (java), packaging tool (.jar), running Applet (appletviewer), etc.

JDK

It contains the software development tools listed above.

JDK = JRE + Development tools

IDE

It is the entire JDK tool integrated and accessible from a single window. Example: NetBeans, Eclipse, etc.

Production Environment:

It is a place where a software is being used at real-environment (client environment)

JRE

JRE = JVM + Java libraries

1.6 Chapter Review Questions

1. What is the difference between procedural and object-oriented programs?
2. Explain JDK, JRE and JVM?
3. What is OOP and understand the concepts of OOP?
4. What is a class, superclass, subclass and what is the difference between class and structure?
5. What is an object?
6. What is the difference between a class and an object?
7. Define abstraction in software in general.
8. Define data abstraction.
9. Define process abstraction.
10. What is encapsulation?
11. What is information hiding?
12. Define inheritance and list different types of inheritance.
13. Define polymorphism
14. What is public interface?
15. Define instantiation.
16. Write a java program to display any message.
17. Explain ***public static void main(String args[])*** in Java.
18. Why is main method declared as static?
19. Why Java is platform independent?
20. Is JDK required on each machine to run a java program?

Chapter Two

2 Basics in Java Programming

2.1 Introduction

In this Chapter you learned how to create, compile, and run a Java program. Now you will learn how to solve practical problems programmatically. Through these problems, you will learn elementary programming using primitive data types, variables, constants, operators, expressions, and input and output.

Writing Simple Programs

Writing a program involves designing algorithms and translating algorithms into code.

Problem:

It an environmental situation that needs to be solved.

Algorithm:

It describes how a problem is solved in terms of the actions to be executed and the order of their execution. It can help the programmer plan a program before writing it in a programming language. It can be described in natural languages or in pseudocode (i.e., natural language mixed with programming code).

Example-1-:

Write a simple java code for computing the area of a circle. The algorithm for this program can be described as follows:

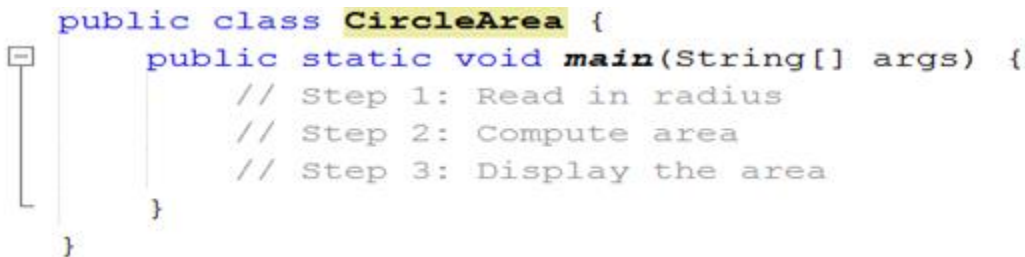
1. Read in the radius.
2. Compute the area using the following formula: $\text{area} = \text{radius} * \text{radius} * p$
3. Display the area.

When you code, you translate an algorithm into a program.

Syntax:

```
public class ClassName {  
    // statements or codes here.  
}
```

As you know, every Java program must have a main method where program execution begins.



```
1 public class CircleArea {  
2     public static void main(String[] args) {  
3         // Step 1: Read in radius  
4         // Step 2: Compute area  
5         // Step 3: Display the area  
6     }  
7 }
```

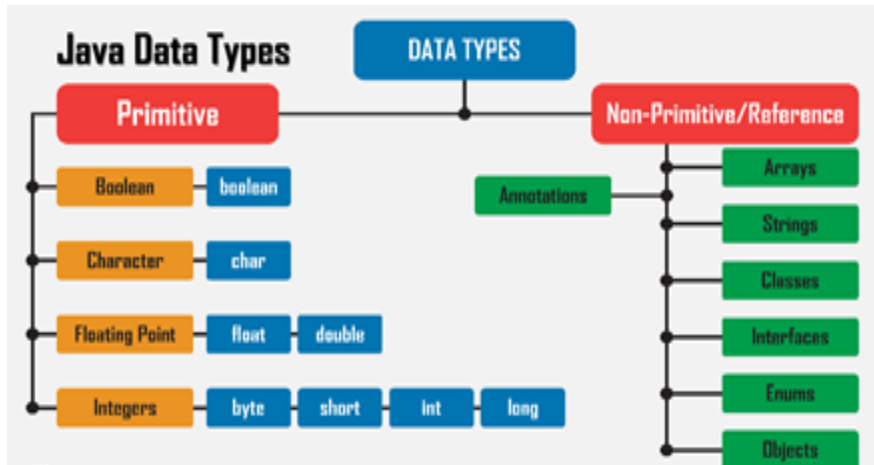
2.2 Variable types and identifiers

A variable designates a location in memory for storing data and computational results in the program. A variable has a name that can be used to access the memory location. Variable names should descriptive names.

A programming language is called a strong data typing language if the language needs the instructions in the programs to clearly state the types of all data unless the data type can be unambiguously implied. Java as well as languages in the C-family (E.g. C, C++, and C #) are strong data typing languages, while some examples of popular programming languages that are not strong data typing are Visual Basic and PHP.

There are 2 main types of data in Java.

1. Primitive data types
2. Classes



Classes are more complex data types. There are classes that are standard to Java such as String, Rectangle, etc. Also, new classes can be defined by programmers when needed. There are four primitive data types for integer values, including byte, short, int, and long. They are different in the sizes of the space they occupied in memory. The reason why Java has multiple integer (as well as, floating-point) types is to allow programmer to access all the types support natively by various computer hardware and let the program works efficiently.

Variable Declaration

It tells the compiler to allocate appropriate memory space for the variable based on its data type. Every variable has a name, a type, a size, and a value.

The syntax for declaring a variable is

```
datatype variableName;
```

Example:

```
int count;           // Declare count to be an integer variable;
double radius;       // Declare radius to be a double variable;
double interestRate; // Declare interestRate to be a double variable;
```

If variables are of the same type, they can be declared together, as follows:

```
datatype variable1, variable2, ..., variablen;
```

The variables are separated by commas.

Example:

```
int oop, it;
```

Naming Variables

By convention, variable names are in lowercase. If a name consists of several words, concatenate all of them and capitalize the first letter of each word except the first. Examples of variables are `radius` and `interestRate`.

Initializing Variables

Variables often have initial values. You can declare a variable and initialize it in one step. Consider, for instance, the following code:

```
intcount = 1;
```

This is equivalent to the next two statements:

```
intcount;
```

```
x = 1;
```

You can also use a shorthand form to declare and initialize variables of the same type together.

Example:

```
inti = 1, j = 2;
```

Data Types

int, double, and char, byte, short, long, float, char, and boolean.

Reading Input from the Console

Scanner class is used to obtain input from console. Java uses **System.out** to refer to the standard output device and **System.in** to the standard input device. By default, the output device is the display monitor, and the input device is the keyboard. To perform console output, you simply use the `println` method to display a primitive value or a string to the console. Console input is not directly supported in Java, but you can use the `Scanner` class to create an object to read input from `System.in`, as follows:

```
Scanner input = new Scanner(System.in);
```

The syntax `new Scanner (System.in)` creates an object of the `Scanner` type. The syntax `Scanner input` declares that `input` is a variable whose type is `Scanner`. The whole line `Scanner input = new Scanner (System.in)` creates a `Scanner` object and assigns its reference to the variable `input`. An object may invoke its methods. To invoke a method on an object is to ask the object to perform a task.

Methods for Scanner Objects

Method	Description
<code>nextByte()</code>	reads an integer of the <code>byte</code> type.
<code>nextShort()</code>	reads an integer of the <code>short</code> type.
<code>nextInt()</code>	reads an integer of the <code>int</code> type.
<code>nextLong()</code>	reads an integer of the <code>long</code> type.
<code>nextFloat()</code>	reads a number of the <code>float</code> type.
<code>nextDouble()</code>	reads a number of the <code>double</code> type.
<code>next()</code>	reads a string that ends before a whitespace character.
<code>nextLine()</code>	reads a line of text (i.e., a string ending with the <i>Enter</i> key pressed).
<code>next().charAt(0)</code>	reads a character of the <code>char</code> type.

Example:

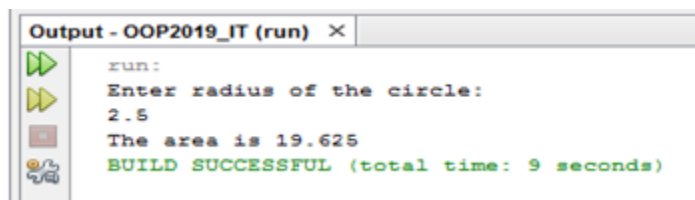
Write a java program that accepts a radius from the console to compute area of the circle.

```

1  package oopit;
2  import java.util.Scanner;
3  public class UserCircleArea {
4      public static void main(String[] args) {
5          double radius;
6          double Area;
7          Scanner input = new Scanner(System.in);
8          System.out.println("Enter radius of the circle: ");
9          radius = input.nextDouble();
10         Area = radius * radius * 3.14;
11         System.out.println("The area is " + Area);
12     }
13 }

```

Output:



```

Output - OOP2019_IT (run) x
run:
Enter radius of the circle:
2.5
The area is 19.625
BUILD SUCCESSFUL (total time: 9 seconds)

```

The **Scanner** class is in the **java.util** package. It is imported in line 2. Line 7 creates a Scanner object. The statement in line 8 displays a message to prompt the user for input.

```
System.out.println("Enter radius of the circle: ");
```

print vs. println vs. printf vs. format

These methods for printing strings to standard output (**System.out**). The **print method** is identical to the **println method** except that **println** moves the cursor to the next line after displaying the string, but **print** does not advance the cursor to the next line when completed.

Format specifiers begin with a percent sign (%) and end with a conversion. The conversion is a character indicating the type of argument to be formatted.

Example:

```
int i = 461012;
```

```
System.out.format("The value of i is: %d%n", i);
```

The %d specifies that the single variable is a decimal integer. The %n is a platform-independent newline character. The output is:

The value of i is: 461012

The printf() and format() methods are overloaded.

Identifiers

Identifiers are names that programmers give to classes, methods, and variables. Note that Java is a case-sensitive language. Uppercase and lowercase letters are not considered the same.

All identifiers must obey the following rules:

- An identifier is a sequence of characters that consists of letters, digits, underscores (_), and dollar signs (\$).
- An identifier must start with a letter, an underscore (_), or a dollar sign (\$). It cannot start with a digit.
- An identifier cannot be a reserved word. (Example: class, double, float ...)
- An identifier cannot be true, false, or null.
- An identifier can be of any length.

Example: \$2, ComputeArea, area, radius, and showMessageDialog are legal identifiers, whereas 2A and d+4 are not because they do not follow the rules.

Comments and Comment Styles in Java

It is used to introduce each major step and explain anything that is difficult to read. There are three types comments in java.

- **Line comment** – is represented as `//`. It is used for commenting on steps inside a method.
- **Block comment** – is represented as `/* ... */`
- **Javadoc comment** – is represented as `/** ... */`. They can be extracted into an HTML file using JDK's javadoc command. It is used for commenting on an entire class or an entire method. These comments must precede the class or the method header in order to be extracted in a javadoc HTML file.

2.3 Number types, strings, constants

2.3.1 Number Types

Every data type has a range of values. The compiler allocates memory space for each variable or constant according to its data type. Java provides eight primitive data types for numeric values, characters, and Boolean values.

The six numeric data types, their ranges, and their storage sizes.

<i>Name</i>	<i>Range</i>	<i>Storage Size</i>
byte	-2^7 (-128) to 2^7-1 (127)	8-bit signed
short	-2^{15} (-32768) to $2^{15}-1$ (32767)	16-bit signed
int	-2^{31} (-2147483648) to $2^{31}-1$ (2147483647)	32-bit signed
long	-2^{63} to $2^{63}-1$ (i.e., -9223372036854775808 to 9223372036854775807)	64-bit signed
float	Negative range: $-3.4028235E + 38$ to $-1.4E-45$ Positive range: $1.4E-45$ to $3.4028235E + 38$	32-bit IEEE 754
double	Negative range: $-1.7976931348623157E + 308$ to $-4.9E-324$ Positive range: $4.9E - 324$ to $1.7976931348623157E + 308$	64-bit IEEE 754

Java uses four types for **integers**: *byte*, *short*, *int*, and *long*. Choose the type that is most appropriate for your variable. Java uses two types for **floating-point numbers**: *float* and *double*. The double type is twice as big as float. So, the double is known as double precision, float as single precision. Normally you should use the **double** type, because it is more accurate than the float type.

Numeric Literals

A literal is a constant value that appears directly in a program.

Example: 34 and 0.305 are literals in the following statements:

- **Integer Literal:**

An integer literal can be assigned to an integer variable as long as it can fit into the variable.

Example:

```
byte b = 128;
```

- **Floating-Point Literals**

It is written with a decimal point. By default, a floating-point literal is treated as a double type value. You can make a number a float by appending the letter f or F, and you can make a number a double by appending the letter d or D.

Example:

You can use **100.2f** or **100.2F** for a float number, and **100.2d** or **100.2D** for a double number.

```
System.out.println("1.0 / 3.0 is " + 1.0 / 3.0);  
displays 1.0 / 3.0 is 0.3333333333333333.
```

Scientific Notation

Floating-point literals can also be specified in scientific notation; for example, **1.23456e+2**, the same as **1.23456e2**, is equivalent to $1.23456 \times 10^2 = 123.456$, and **1.23456e-2** is equivalent to $1.23456 \times 10^{-2} = 0.0123456$. **E** (or **e**) represents an exponent and can be in either lowercase or uppercase.

Characters

The character data type, **char**, is used to represent a single character. A character literal is enclosed in single quotation marks. Consider the following code:

```
char letter = 'A';  
char numChar = '4';
```

The first statement assigns character A to the char variable letter. The second statement assigns digit character 4 to the char variable numChar.

Unicode and ASCII Code

Computers use binary numbers internally. A **character** is stored in a computer as a sequence of 0s and 1s. Mapping a character to its binary representation is called **encoding**. There are different

ways to encode a character. How characters are encoded is defined by an **encoding scheme**.

Java supports **Unicode**, an encoding scheme established by the Unicode Consortium to support the interchange, processing, and display of written texts in the world's diverse languages. Unicode was originally designed as a 16-bit character encoding. The primitive data type **char** was intended to take advantage of this design by providing a simple data type that could hold any character. However, it turned out that the 65,536 characters possible in a 16-bit encoding. These characters can be stored in a **char** type variable.

A 16-bit Unicode takes two bytes, preceded by **\u**, expressed in four hexadecimal digits that run from **'\u0000'** to **'\uFFFF'**.

Example:

The Unicode's for the Greek letters is **\u03b1**, **\u03b2**, **\u03b3**, respectively.

```
1 package oopit;  
2 public class DataTypes {  
3     public static void main(String[] args) {  
4         char symbol = '\u03b1';  
5         char letter = '\u0061';  
6         System.out.println("Result is: " + symbol + " and " + letter);  
7     }  
8 }
```

Output:

```
Output - OOP2019_IT (run) X
run:
Result is: α and a
BUILD SUCCESSFUL (total time: 0 seconds)
```

Most computers use ASCII (American Standard Code for Information Interchange), a 7-bit encoding scheme for representing all uppercase and lowercase letters, digits, punctuation marks, and control characters. Unicode includes ASCII code, with ‘\u0000’ to ‘\u007F’ corresponding to the 128 ASCII characters.

Example:

```
char letter = '\u0041'; // Character A's Unicode is 0041
```



Note

The increment and decrement operators can also be used on `char` variables to get the next or preceding Unicode character. For example, the following statements display character `b`.

```
char ch = 'a';
System.out.println(++ch);
```

Escape Sequences for Special Characters

Java defines escape sequences to represent special characters. An escape sequence begins with the backslash character (`\`) followed by a character that has a special meaning to the compiler. It is shown as follows.

Character	Escape Sequence	Name	Unicode Code
	<code>\b</code>	Backspace	<code>\u0008</code>
	<code>\t</code>	Tab	<code>\u0009</code>
	<code>\n</code>	Linefeed	<code>\u000A</code>
	<code>\f</code>	Formfeed	<code>\u000C</code>
	<code>\r</code>	Carriage Return	<code>\u000D</code>
	<code>\\</code>	Backslash	<code>\u005C</code>
	<code>\'</code>	Single Quote	<code>\u0027</code>
	<code>\"</code>	Double Quote	<code>\u0022</code>

Example:

So, now you can print the quoted message using the following statement:

```
System.out.println("He said \"Java is fun\"");
```

The output is

```
He said "Java is fun"
```

2.3.2 The String Type

The char type represents only one character. To represent a string of characters, use the data type called String.

Example:

```
String message = "Welcome to Java";
```

String is actually a predefined class in the Java library just like the classes **System**, **JOptionPane**, and **Scanner**. The **String** type is not a primitive type. It is known as a *reference type*. Any Java class can be used as a reference type for a variable. The plus (+) sign is used for concatenation.

Concatenating Strings

It combines two strings if two operands are strings.

Example:

```
System.out.println("The area for the circle of radius " +  
    radius + " is " + area);
```

Concatenating Strings and Numbers

If one of the operands is a non-string (e.g., a number), the non-string value is converted into a string and concatenated with the other string.

Example:

```
String s = "Chapter" + 2; // s becomes Chapter2
```


Concatenating Strings and Characters

Example:

```
String s1 = "Supplement" + 'B'; // s1 becomes SupplementB
```

Breaking a Long String

A string constant cannot cross lines in the source code. Thus the following statement would result in a compile error:

```
System.out.println("Introduction to Java Programming,  
by Y. Daniel Liang");
```

To fix the error, break the string into separate substrings, and use the concatenation operator (+) to combine them:

```
System.out.println("Introduction to Java Programming, " +  
"by Y. Daniel Liang");
```

Getting Characters and Substrings by Index

You can get the character at a particular index within a string by invoking the **charAt()** accessor method. The index of the first character is **0**, while the index of the last character is `length()-1`.

Example:

The following code gets the character at index 9 in a string:

```
String today = "Niagara. O roar again!";
```

```
System.out.println(today.charAt(9));
```

Indices begin at 0, so the character at index 9 is "O". The following figures shows the use of `charAt` method to get a character at a particular index.

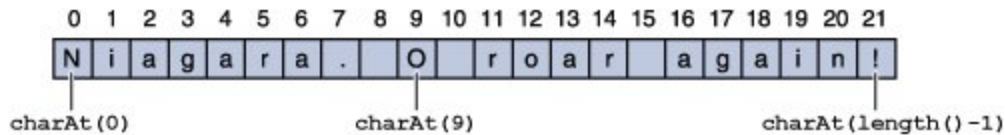


Fig. 2.1. Displaying an index of a character from a string.

If you want to get more than one consecutive character from a string, you can use the **substring** method. The substring method has two versions. Here is the syntax and description

String substring(int beginIndex, int endIndex)

Returns a new string that is a substring of this string. The first integer argument specifies the index of the first character. The second integer argument is the index of the last character + 1.

String substring(int beginIndex)

Returns a new string that is a substring of this string. The integer argument specifies the index of the first character. Here, the returned substring extends to the end of the original string.

Example:

```
String today = "Niagara. O roar again!";
```

```
System.out.println(today.substring(11, 15));
```

Output:

roar

It is shown in the figure below.

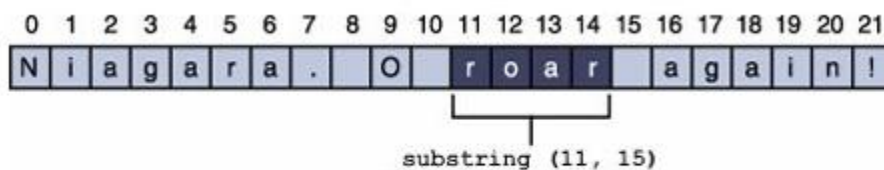


Fig. 2.2. Displaying consecutive characters from a string.

Constants

The value of a variable may change during the execution of a program, but a named constant or simply constant represents permanent data that never changes.

Syntax of writing constant values in java.

A constant must be declared and initialized in the same statement. The word final is a Java keyword for declaring a constant.

Example:

```
final double PI = 3.14159; // Declare a constant
```

Boolean

A variable that holds a Boolean value is known as a **Boolean variable**. The boolean data type is used to declare Boolean variables. A boolean variable can hold one of the two values: true and false.

Example:

The following statement assigns true to the variable **lightsOn**:

```
boolean lightsOn = true;
```

2.4 Type Conversion/ Casting

Casting is an operation that converts a value of one data type into a value of another data type. Casting a variable of a type with a small range to a variable of a type with a larger range is known as **widening a type**. Casting a variable of a type with a large range to a variable of a type with a smaller range is known as **narrowing a type**. Widening a type can be performed

automatically **without explicit casting**. Narrowing a type must be performed explicitly. The syntax is the target type in parentheses, followed by the variable's name or the value to be cast.

2.4.1 . Numeric Type Conversion

Example: The following statement displays 1

```
System.out.println((int)1.7);
```

When a double value is cast into an int value, the fractional part is truncated. The following statement displays 0.5, because 1 is cast to 1.0 first, then 1.0 is divided by 2

```
System.out.println((double)1 / 2);
```

Example:

```
double d = 4.5;  
int i = (int)d; // i becomes 4, but d is not changed, still 4.5
```

2.4.2 Casting between char and Numeric Types

A **char** can be cast into any numeric type, and vice versa. When an integer is cast into a **char**, only its lower 16 bits of data are used; the other part is ignored.

Example:

```
char ch = (char)0XAB0041; // the lower 16 bits hex code 0041 is assigned to ch
```

```
System.out.println(ch); // ch is character A
```

When a floating-point value is cast into a **char**, the floating-point value is first cast into an **int**, which is then cast into a **char**.

```
char ch = (char)65.25; // decimal 65 is assigned to ch
```

```
System.out.println(ch); // ch is character A
```

When a **char** is cast into a numeric type, the character's Unicode is cast into the specified numeric type.

```
int i = (int)'A'; // the Unicode of character A is assigned to i
```

```
System.out.println(i); // i is 65
```

Implicit casting can be used if the result of a casting fits into the target variable. Otherwise, **explicit casting** must be used.

Example:

The Unicode of **'a'** is **97**, which is within the range of a byte, these implicit castings are fine:

```
byte b = 'a';
```

```
int i = 'a';
```

But the following casting is incorrect, because the Unicode **\uFFFF4** cannot fit into a byte:

```
byte b = '\uFFFF4';
```

To force assignment, use explicit casting, as follows:

```
byte b = (byte)\uFFFF4';
```

Any positive integer between **0** and **FFFF** in hexadecimal can be cast into a character implicitly. Any number not in this range must be cast into a **char** explicitly.

2.4.3 Converting Strings to Numbers and Vice Versa

The input returned from the input dialog box is a **string**. If you enter a numeric value such as **123**, it returns **"123"**. You have to convert a string into a number to obtain the input as a number. To convert a string into an **int** value, use the **parseInt** method in the Integer class, as follows:

```
int intValue = Integer.parseInt(intString);
```

where **intString** is a numeric string such as “123”.

To convert a string into a double value, use the **parseDouble** method in the Double class, as follows:

```
double doubleValue = Double.parseDouble(doubleString);
```

where **doubleString** is a numeric string such as “123.45”.

The **Integer** and **Double** classes are both included in the **java.lang** package, and thus they are automatically imported

A **valueOf** method is used to convert a string to an object of numeric type and vice versa.

Example:

```
String a = “123”;
```

```
System.out.println(“Result of valueOf is: “+Integer.valueOf(a));
```

Output:

```
Result of valueOf is: 123
```

2.5 Decision and Repetition Statements

Java statements are classified into three main parts. They are sequential statement, branch statement and looping statements.

2.5.1 . Sequential Statements

The program execution that was performed from top-down following each line of code.

2.5.2 Branching Statements

2.5.2.1 If statement

1. One-Way if Statements

It executes an action if and only if the condition is true. The syntax

```
if (boolean-expression) {  
    statement(s);  
}
```

The execution flow chart is shown in figure below.

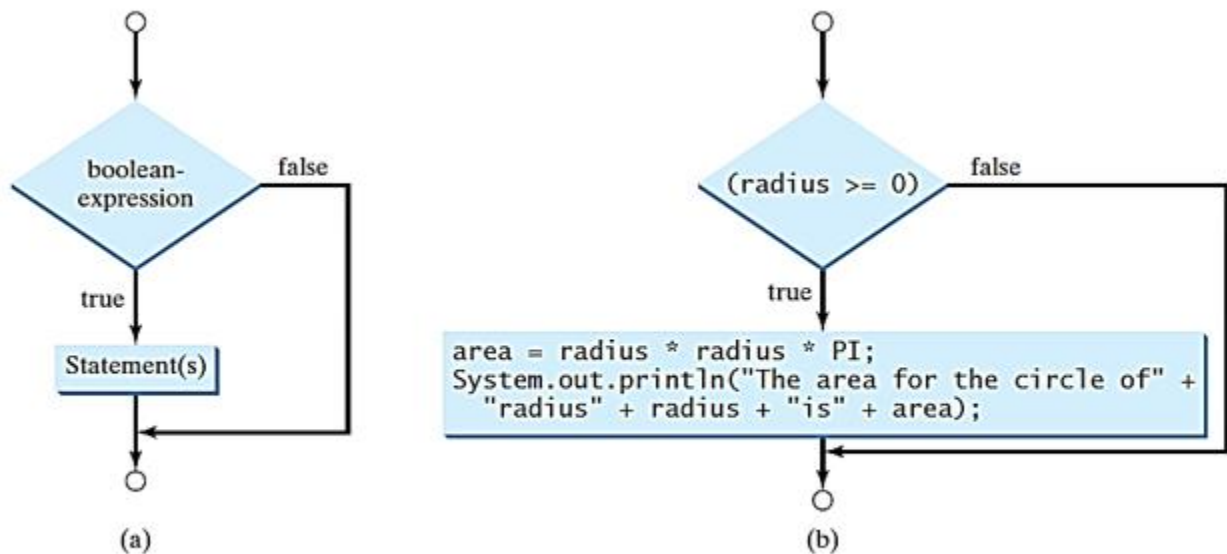


Fig. 2.3. Flowchart of one-way if statement

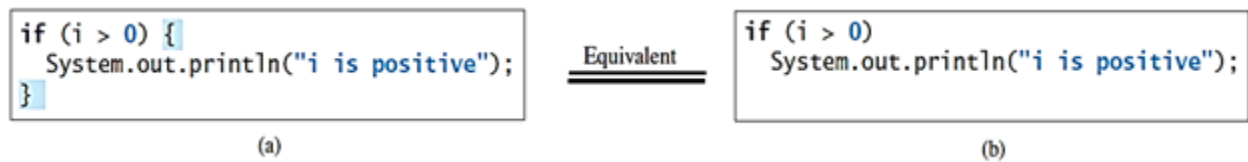
If the boolean-expression evaluates to true, the statements in the block are executed, and if the condition is false, nothing is done. If the value of radius is greater than or equal to 0, then the area is computed and the result is displayed; otherwise, the two statements in the block will not be executed. The boolean-expression is enclosed in parentheses.

Example:

<pre>if i > 0 { System.out.println("i is positive"); }</pre>	<pre>if (i > 0) { System.out.println("i is positive"); }</pre>
(a) Wrong	(b) Correct

The block braces can be omitted if they enclose a single statement.

Example: The following statements are equivalent.



2. Two-Way if Statements

if ... else Statement

It specifies an action that differ based on whether the condition is true or false. An if ... else statement executes statements for the true case if the boolean-expression evaluates to true; otherwise, statements for the false case are executed. Syntax:

```
if (boolean-expression) {  
    statement(s)-for-the-true-case;  
}  
else {  
    statement(s)-for-the-false-case;  
}
```

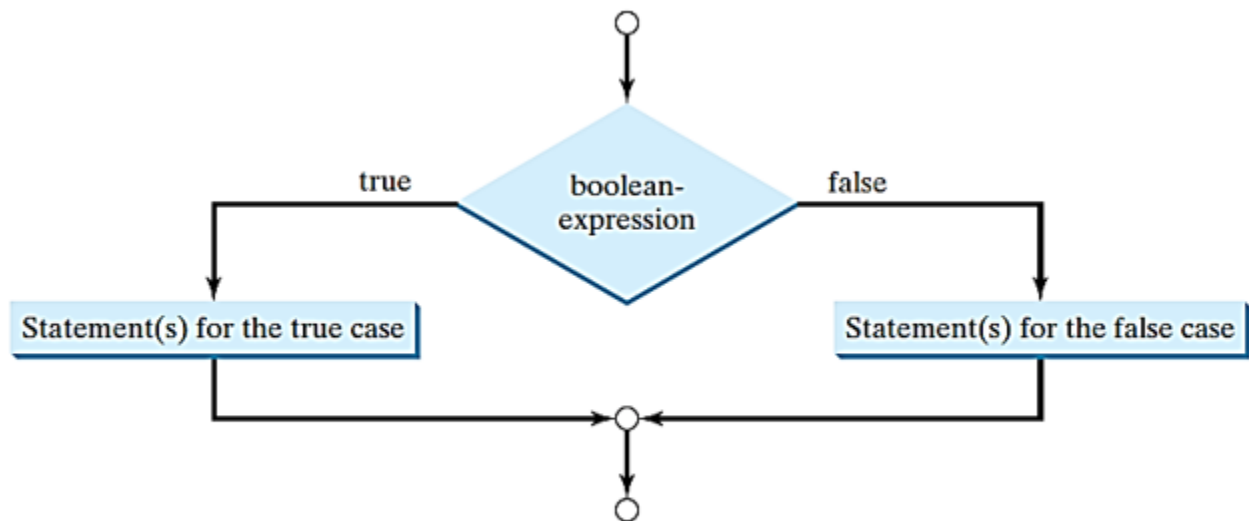


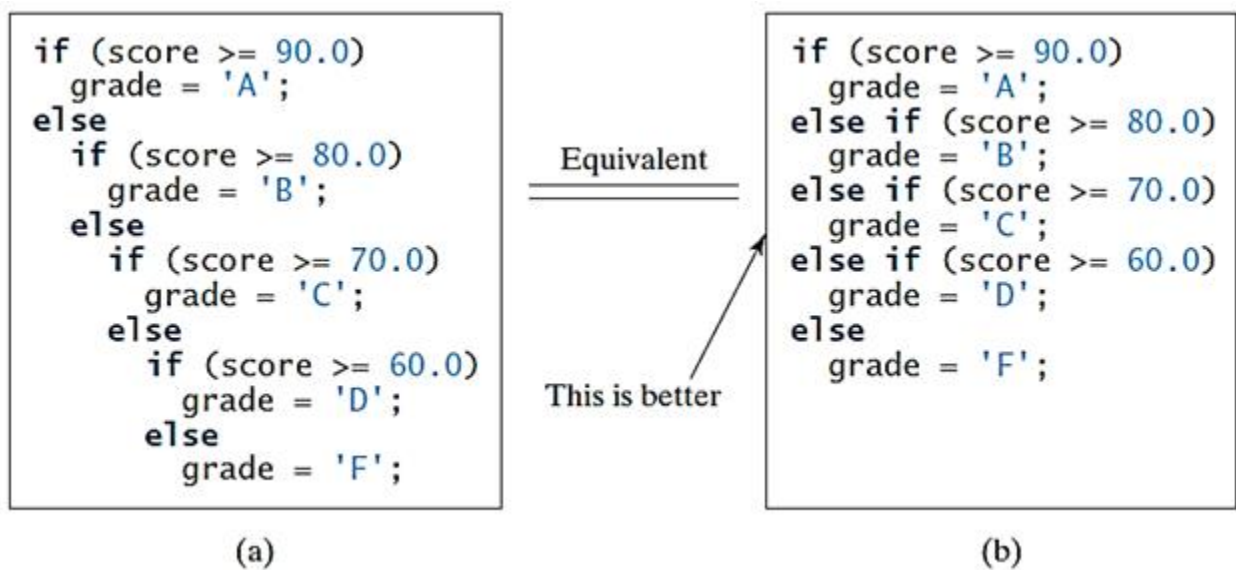
Fig. 2.4. Flowchart of two-way if statement

If the boolean-expression evaluates to true, the statement(s) for the true case are executed; otherwise, the statement(s) for the false case are executed.

3. Nested if Statements

The statement in an if or if ... else statement can be any legal Java statement, including another if or if ... else statement. The inner if statement is said to be nested inside the outer if statement. The inner if statement can contain another if statement; in fact, there is no limit to the depth of the nesting. The nested if statement can be used to implement multiple alternatives.

Example: Assigning a letter grade to the variable grade according to the score, with multiple alternatives.



4. Switch statement

It is used to replace the nested if statement. The switch statement checks all cases and executes the statements in the matched case. Syntax:

```

switch (switch-expression) {
    case value1: statement(s)1;
                break;

    case value2: statement(s)2;
                break;

    ...
    case valueN: statement(s)N;
                break;
    default:    statement(s)-for-default;
}

```

The switch statement observes the following rules:

- The **switch-expression** must yield a value of char, byte, short, or int type and must always be enclosed in parentheses.
- The **value1**, and **valueN** must have the same data type as the value of the switch-expression. Note that value1, and valueN are constant expressions, meaning that they cannot contain variables, such as $1 + x$.
- When the value in a **case** statement matches the value of the switch-expression, the statements *starting from this case* are executed until either a **break** statement or the end of the switch statement is reached.
- The keyword **break** is optional. The break statement immediately ends the **switch** statement.
- The **default** case, which is optional, can be used to perform actions when none of the specified cases matches the **switch-expression**.
- The **case** statements are checked in sequential order, but the order of the cases (including the default case) does not matter. However, it is good programming style to follow the logical sequence of the cases and place the default case at the end.

It summarized in figure below.

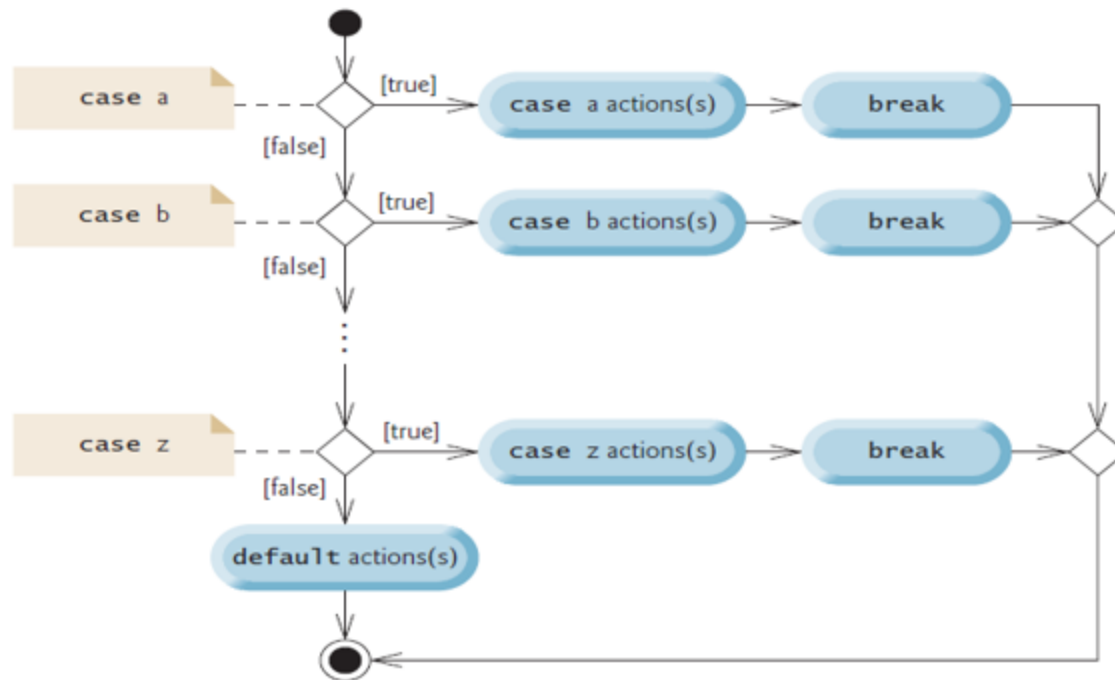


Fig. 2.5. Flowchart of switch statement

2.5.2.2 Iterations Statements (Looping)

Suppose that you need to print a string (e.g., **“Welcome to Java!”**) a hundred times. It would be tedious to have to write the following statement a hundred times:

100 times { `System.out.println("Welcome to Java!");`
`System.out.println("Welcome to Java!");`
`...`
`System.out.println("Welcome to Java!");`

To solve such kind of problems the concepts of looping is used. Java provides a powerful construct called a loop that controls how many times an operation or a sequence of operations is performed in succession. Loops are constructs that control repeated executions of a block of statements. There are basically three types of looping.

1 While Loop

The syntax for the while loop is as follows:

```

while (loop-continuation-condition) {
    // Loop body
    Statement(s);
}

i = initialValue; // Initialize loop control variable
while (i < endValue) {
    // Loop body
    ...
    i++; // Adjust loop control variable
}

```

The part of the loop that contains the statements to be repeated is called the loop body. A one-time execution of a loop body is referred to as an iteration of the loop. Each loop contains a loop-continuation-condition, a Boolean expression that controls the execution of the body. It is evaluated each time to determine if the loop body is executed. If its evaluation is true, the loop body is executed; if its evaluation is false, the entire loop terminates and the program control turns to the statement that follows the while loop.

The flow-chart of while is shown as below figure.

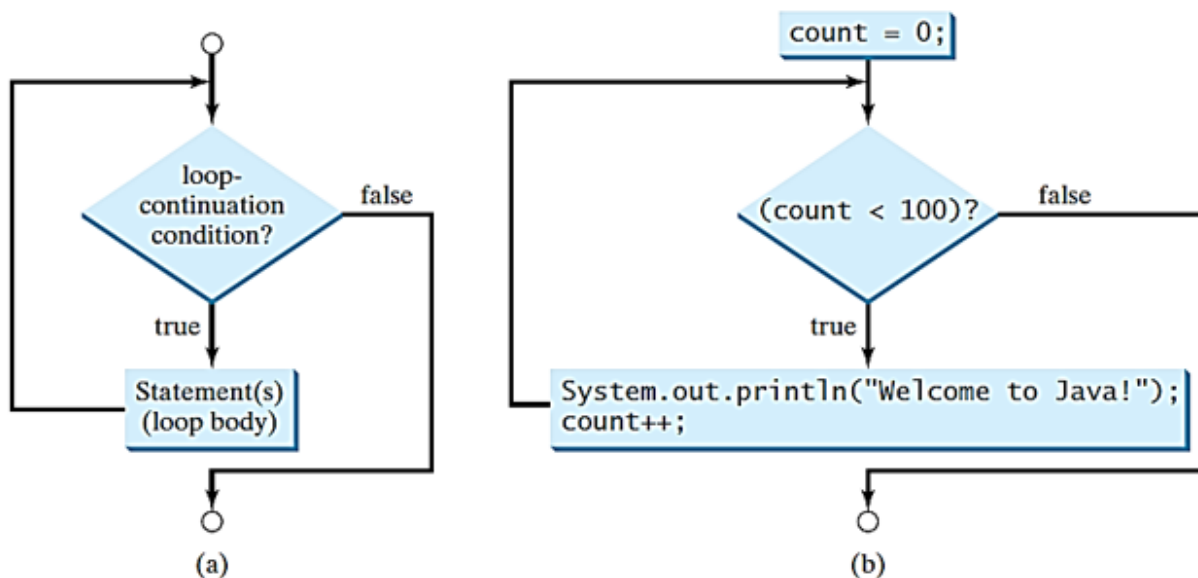


Fig. 2.6. flowchart of while loop

2. Do while loop

The do-while loop is a variation of the while loop.

Syntax:

```
do {  
    // Loop body;  
    Statement(s);  
} while (loop-continuation-condition);
```

Its execution flow chart is shown in figure below.

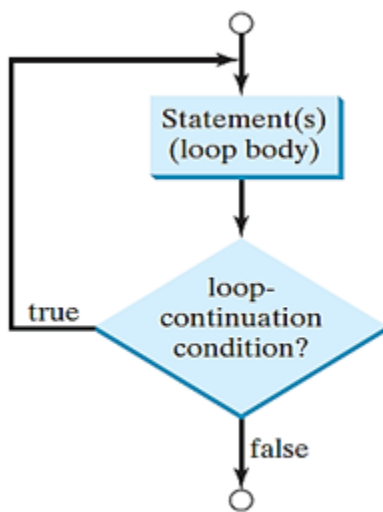


Fig. 2.7. Flowchart of do-while loop

The do-while loop executes the loop body first, then checks the loop-continuation-condition to determine whether to continue or terminate the loop. The loop body is executed first. Then the loop-continuation-condition is evaluated. If the evaluation is true, the loop body is executed again; if it is false, the do-while loop terminates.

3. for Loop

Syntax:

```
for (initial-action; loop-continuation-condition;
    action-after-each-iteration) {
    // Loop body;
    Statement(s);
}
```

4. Nested Loop

Nested loops consist of an outer loop and one or more inner loops. Each time the outer loop is repeated, the inner loops are reentered, and started a new

2.6 Arrays and working with arrays

Arrays are used to store multiple values in a single variable, instead of declaring separate variables for each value. An *array* is a container object that holds a fixed number of values of a single type. The length of an array is established when the array is created. After creation, its length is fixed.

An array in Java is a group of like-typed variables referred to by a common name. Arrays in Java work differently than they do in C/C++. Following are some important points about Java arrays.

- In Java, all arrays are dynamically allocated. (discussed below)
- Since arrays are objects in Java, we can find their length using the object property *length*. This is different from C/C++, where we find length using `sizeof`.
- A Java array variable can also be declared like other variables with `[]` after the data type.
- The variables in the array are ordered, and each has an index beginning from 0.
- Java array can be also be used as a static field, a local variable, or a method parameter.
- The **size** of an array must be specified by int or short value and not long.
- The direct superclass of an array type is `Object`.
- Every array type implements the interfaces `Cloneable` and `java.io.Serializable`.

An array can contain primitives (int, char, etc.) and object (or non-primitive) references of a class depending on the definition of the array. In the case of primitive data types, the actual values are stored in contiguous memory locations.

To declare an array, define the variable type with **square brackets**:

```
String[] cars;
```

We have now declared a variable that holds an array of strings. To insert values to it, we can use an array literal – place the values in a comma-separated list, inside curly braces:

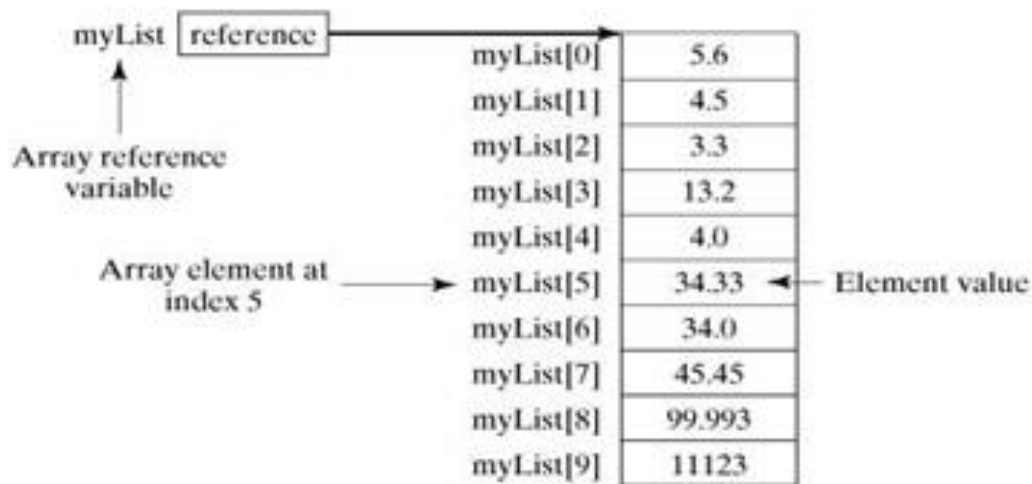
```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
```

To create an array of double, you could write:

```
double[] myList = new double[10];
```

The array elements are accessed through the **index**. Array indices are 0-based; that is, they start from 0 to **arrayRefVar.length-1**.

Following picture represents array myList. Here, myList holds ten double values and the indices are from 0 to 9.



Access the Elements of an Array

You access an array element by referring to the index number.

This statement accesses the value of the first element in cars:

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
System.out.println(cars[0]);
// Outputs Volvo
```

40	55	63	17	22	68	89	97	89
0	1	2	3	4	5	6	7	8

<- Array Indices

Array Length = 9

First Index = 0

Last Index = 8

Creating, Initializing, and Accessing an Array

One-Dimensional Arrays:

The general form of a one-dimensional array declaration is


```
type var-name[]; OR type[] var-name;
```

An array declaration has two components: the type and the name. *type* declares the element type of the array. The element type determines the data type of each element that comprises the array. Like an array of integers, we can also create an array of other primitive data types like char, float, double, etc., or user-defined data types (objects of a class). Thus, the element type for the array determines what type of data the array will hold.

Instantiating an Array in Java

When an array is declared, only a reference of an array is created. To create or give memory to the array, you create an array like this: The general form of *new* as it applies to one-dimensional arrays appears as follows:

```
var-name = new type [size];
```

Here, *type* specifies the type of data being allocated, *size* determines the number of elements in the array, and *var-name* is the name of the array variable that is linked to the array. To use *new* to allocate an array, **you must specify the type and number of elements to allocate.**

Example:

```
int intArray[]; //declaring array
```

```
intArray = new int[20]; // allocating memory to array
```

OR

```
int[] intArray = new int[20]; // combining both statements in one
```

Accessing Java Array Elements using for Loop

Each element in the array is accessed via its index. The index begins with 0 and ends at (total array size)-1. All the elements of array can be accessed using Java for Loop.

```
// accessing the elements of the specified array
```

```
for (int i = 0; i < arr.length; i++)
```

```
System.out.println("Element at index " + i +
```

```
" : "+ arr[i]);
```

Multidimensional Arrays

Multidimensional arrays are **arrays of arrays** with each element of the array holding the reference of other arrays. These are also known as Jagged Arrays. A multidimensional array is created by appending one set of square brackets ([]) per dimension. Examples:

```
int[][] intArray = new int[10][20]; //a 2D array or matrix
```

```
int[][][] intArray = new int[10][20][10]; //a 3D array
```

2.7 Chapter Review Questions

1. Define variable?
2. What is instance variable?
3. Define class variable?
4. What is an expression?
5. Write a Java program to display default value of all primitive data types of java.
6. Write a program check two strings are equal or not.
7. Write a Program to print the area of triangle. Save it with name **Area.java** in your folder.
8. Write a program to give the examples of operators.
 - a. Increment and decrement operators.
 - b. Bitwise Complement Operator.
 - c. Arithmetic operator.
 - d. Relational Operator
 - e. Bitwise operator.
 - f. Conditional Operator.
9. Write a program to give the example of control statements.
 - a. If statements.
 - b. Switch Statements.
 - c. For loop.
 - d. While Statements.
 - e. Do While statements
10. What is Unicode?
11. Write a java program that creates a user interface to perform integer divisions. The user enters two numbers in the text fields, Num1 and Num2. The division of Num1 and Num2 is displayed in the Result field when the Divide button is clicked. If Num1 or Num2 were not an integer, the program would throw a NumberFormatException. If Num2 were Zero, the program would throw an Arithmetic Exception Display the exception in a message dialog box.
12. Write a Java program that checks whether a given string is a palindrome or not. Ex: MADAM is a palindrome?
13. Write a java program to multiply two given matrices.
14. Write a java program to find all even and odd integers up to a given integer.
15. Write a Java program that prints all real solutions to the quadratic equation $ax^2+bx+c = 0$. Read in a, b, c and use the quadratic formula. If the discriminate b^2-4ac is negative, display a message stating that there are no real solutions?
16. Write a Java program that prompts the user for an integer and then prints out all the prime numbers up to that Integer?

Chapter Three

3 Objects and Classes

3.1 Introduction

When you think in an object-oriented manner, everything is an object, and every object is a member of a class. A circle is a member of the class that called geometry. An object-oriented programmer would say that circle is an instance of the geometry class. These statements represent **is-a** relationships, that is, relationships in which the object “**is a**” concrete example of the class

3.2 Defining Classes and Objects

Object-oriented programming (OOP) involves programming using objects. An object represents an entity in the real world that can be distinctly identified.

Example:

A student, a desk, a circle, a button, and even a loan can all be viewed as objects. An object has a unique identity, state, and behavior.

- The **state** of an object (also known as its properties or attributes) is represented by data fields with their current values.

Example:

- A circle object has a data field **radius**, which is the property that characterizes a circle.
- A rectangle object has data fields **width** and **height**, which are the properties that characterize a rectangle.
- The **behavior** of an object (also known as its actions) is defined by **methods**. To invoke a method on an object is to ask the object to perform an action.

Example:

- A circle object has a method named **getArea()**. A circle object may invoke **getArea()** to return its area.

Objects of the same type are defined using a common class. A **class** is a template, blueprint, or contract that defines what an object's data fields and methods will be. An **object** is an instance of a class. You can create many instances of a class. Creating an instance is referred to as *instantiation*

Example:

A class named Circle and its three objects.

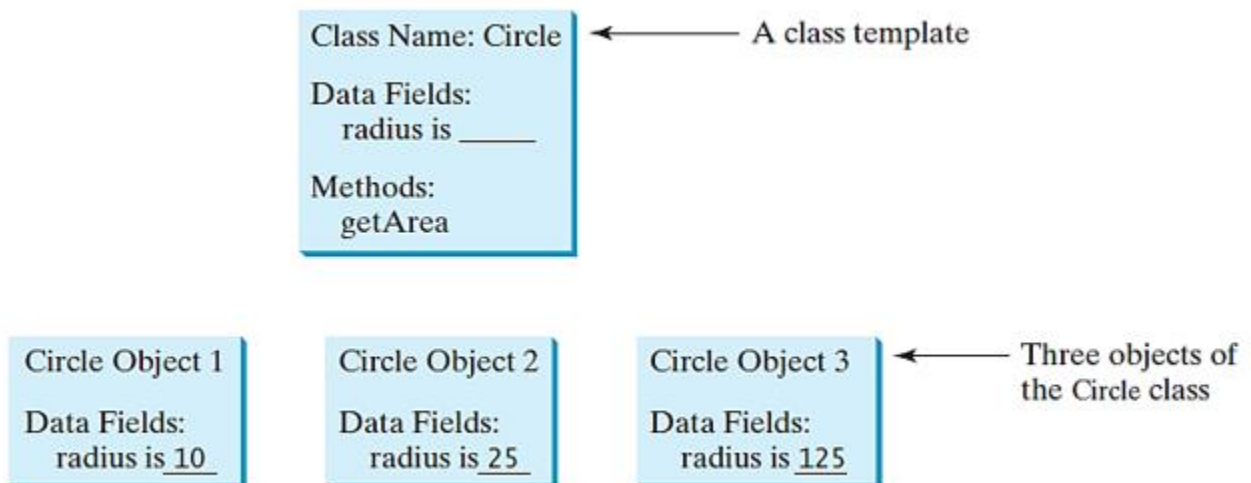


Fig. 3.1. A class is a template for creating objects.

A Java class uses variables to define data fields and methods to define actions. Additionally, a class provides methods of a special type, known as **constructors**, which are invoked to create a new object. A **constructor** can perform any action, but constructors are designed to perform initializing actions, such as initializing the data fields of objects.

3.2.1. Creating a Class

When you create a class, you must assign a name to the class, and you must determine what data and methods will be part of the class. To begin, you create a class header with three parts:

- a. An optional **access specifier**;
- b. The keyword **class**;
- c. Any **legal identifier** you choose for the name of your class – starting with an uppercase letter is conventional.

Syntax:

AccessModifiers class ClassName

Example:

```
public class Circle
```

After writing the class header **public class Circle**, you write the body of the **Circle** class between a set of curly braces. The body contains the **data** and **methods** for the class. The data components of a class are often referred to as **data fields** to help distinguish them from other variables you might use. Data fields are variables you declare within a class but outside of any method. A class's data fields are most often **private** and **not static**. The exception occurs when you want to use a non-changing value without being required to create an object – in that case you make the field both static and final.

Example:

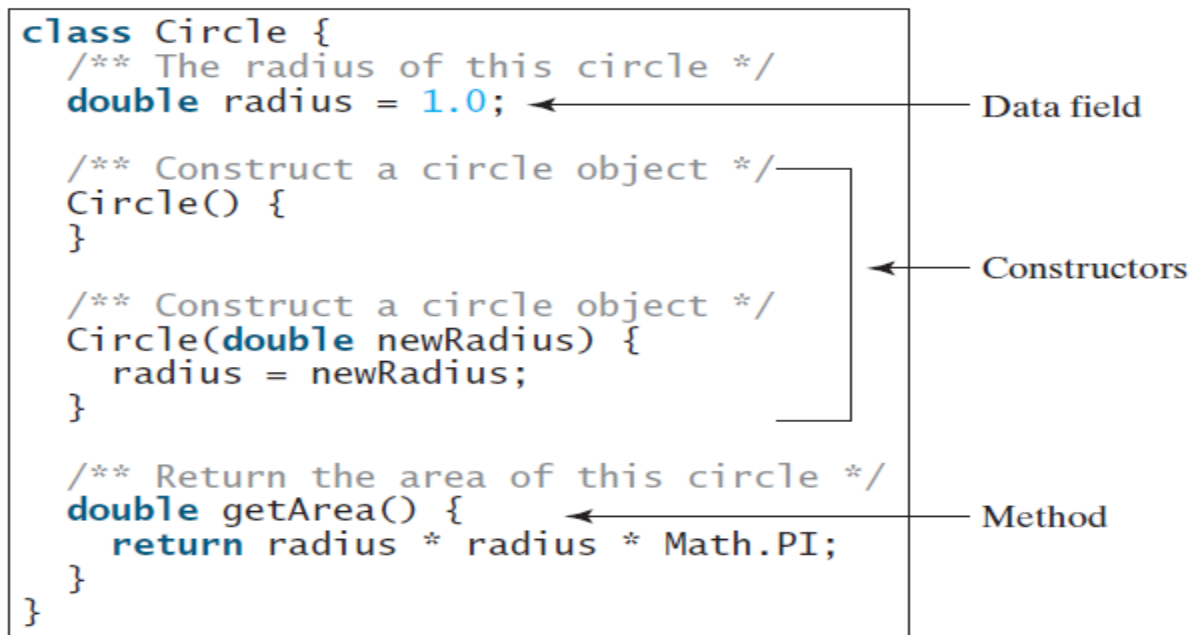
The Java Math class contains a **final, public, static** field named *PI* that you can use without instantiating a Math object.

Besides data, classes contain **methods**. Methods that set or change field values are called **mutator** methods; methods that retrieve values are called **accessor** methods. In Java, mutator methods conventionally start with the prefix **set**, and accessor methods conventionally start with the prefix **get**. **Non-static** methods, those methods used with object instantiations, are called **instance methods**. You can use either a static or non-static method with an object, but only non-static methods behave uniquely for each object. You cannot use a non-static method without an object.

Understanding when to declare fields and methods as **static** and **non-static** is a challenge for new programmers. To help you determine whether a data field should be static or not, you can ask yourself how many times it occurs. If it occurs **once per class**, it is **static**, but if it occurs once per object, it is not static.

Example:

Sample class declaration



3.2.1.1 Class Diagram

In the class diagram *data fields*, *constructor* and *methods* are denoted as follows, respectively.

dataFieldName: dataFieldType

ClassName(parameterName: parameterType)

methodName(parameterName: parameterType): returnType

The UML diagram and UML notation for objects of the above class is:

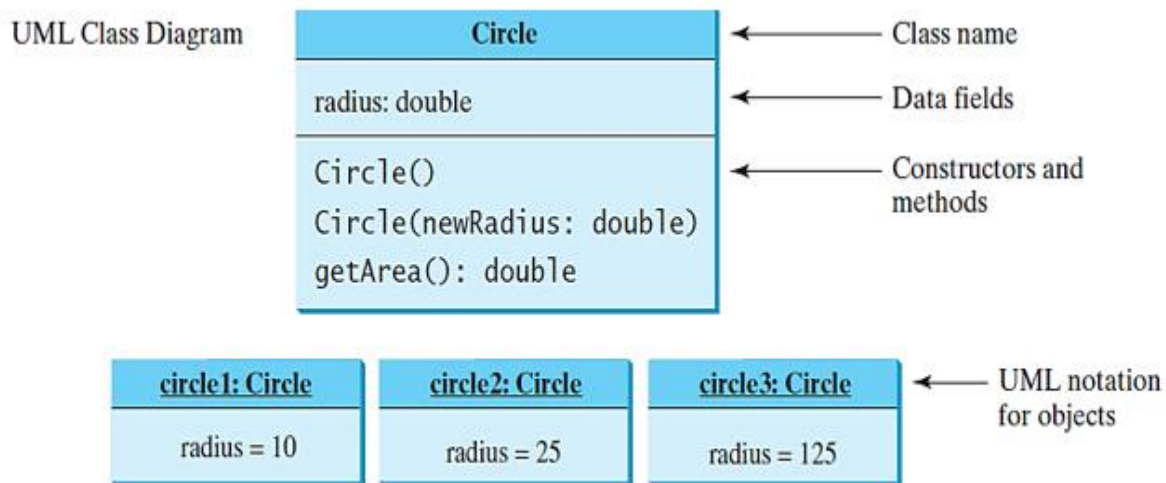


Fig. 3.2. Classes and objects can be represented using UML notations.

3.2.2 Declaring Object

Declaring a class does not create any actual objects. A class is just an abstract description of what an object will be like if any objects are ever actually instantiated. A two-step process creates an object that is an instance of a class.

1. Write a **type** and an **identifier** – just as when you declare any variable – and then you allocate computer memory for that object. Objects (Object name) conventionally start with a lowercase letter. Syntax for creating an object of a class:

ClassName objectRefVar;

Example:

```
Circle circle1;
```

When you declare the **circle1** instance of the **Circle** class, you are notifying the compiler that you will use the identifier **circle1**. However, you are not yet setting aside computer memory in which the **Circle** named **circle1** might be stored – that is done automatically only for primitive type variables.

2. Allocating memory for object name. To allocate the needed memory for an object, you must use the **new** operator (used to create an object from the constructor). The syntax is:

```
ClassName objectRefVar = new ClassName();
```

Example: `Circle circle1 = new Circle();`

In this statement, **Circle** is the object's type (its class), and **circle1** is the name of the object. Also, **circle1** becomes a **reference to the object** – the name for a memory address where the object is held. Every object name is also a **reference** – that is, a computer memory location. The **new** operator is allocating a new, unused portion of computer memory for **circle1**. The value that the statement is assigning to **circle1** is a memory address at which **circle1** is to be located. You do not need to be concerned with what the actual memory address is – when you refer to **circle1**, the compiler locates it at the appropriate address for you.

The statement after the new operator, **Circle()**, with its parentheses, looks suspiciously like a method name that constructs a **Circle** object. The **Circle()** method is a constructor, a special type of method that creates and initializes objects.

3.2.2.1 Accessing Objects

Newly created objects are allocated in the memory. They can be accessed via *reference variables* (objects). The objects of the classes are independent. A class is essentially a programmer-defined type. A class is a *reference type*, which means that a variable of the class type can reference an instance of the class.

After an object is created, its data can be accessed and its methods invoked using the **dot operator** (**.**), also known as the **object member access operator**.

A. Accessing a Data Fields

Syntax: (it is by reference of object).

```
objectRefVar.dataField;
```

Example:

```
circle1.radius; //References the radius in circle1.
```

B. Accessing a Methods

Syntax: (it is by reference of object).

```
objectRefVar.method(arguments);
```

Example:

```
circle1.getArea(); //Invokes the getArea method on circle1. Methods are invoked as operations on objects.
```

3.2.2.2 3.2.2.2. Anonymous Object

It is a technique of creating an object without explicitly assigning it to a variable.

Example:

```
System.out.println("Area is " + new Circle(5).getArea());
```

Example:

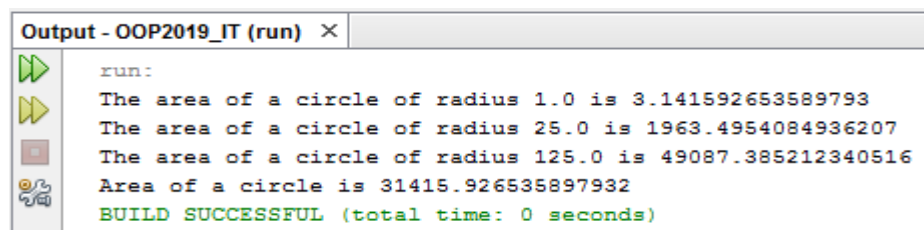
Write a java program using class of circle with radius data field and getArea method.

```

13 public static void main(String[] args) {
14     Circle circle1 = new Circle();
15     System.out.println("The area of a circle of radius "
16         + circle1.radius + " is " + circle1.getArea());
17     Circle circle2 = new Circle(25);
18     System.out.println("The area of a circle of radius "
19         + circle2.radius + " is " + circle2.getArea());
20     Circle circle3 = new Circle(125);
21     System.out.println("The area of a circle of radius "
22         + circle3.radius + " is " + circle3.getArea());
23     //Anonymous Object
24     System.out.println("Area of a circle is " + new Circle(100).getArea());
25 }
26

```

Output:



```

Output - OOP2019_IT (run) ×
run:
The area of a circle of radius 1.0 is 3.141592653589793
The area of a circle of radius 25.0 is 1963.4954084936207
The area of a circle of radius 125.0 is 49087.385212340516
Area of a circle is 31415.926535897932
BUILD SUCCESSFUL (total time: 0 seconds)

```

3.2.3 Constructing Object using Constructor

Constructors are a special kind of method. They have three peculiarities:

- A constructor must have the same name as the **class** itself.
- Constructors do not have a return type – not even void.
- Constructors are invoked using the **new** operator when an object is created. Constructors play the role of initializing objects.

A **constructor** can be **overloaded** (i.e., multiple constructors can have the same name but different signatures), making it easy to construct objects with different initial data values. Constructors are used to construct objects. To construct an object from a class, invoke a constructor of the class using the **new** operator, as follows:

```
new ClassName(arguments);
```

A class normally provides a constructor without arguments (e.g., Circle()). Such a constructor is referred to as a **no-arg** or **no-argument constructor**. A class may be defined *without*

constructors. In this case, a no-arg constructor with an empty body is implicitly defined in the class. This constructor, called a **default constructor**, is provided automatically only if no constructors are explicitly defined in the class. The automatically supplied default constructor provides the following specific initial values to an object's data fields:

- Numeric fields are set to 0 (zero).
- Character fields are set to Unicode '\u0000'.
- Boolean fields are set to false.
- Fields that are object references (for example, String fields) are set to null (or empty).

Example:

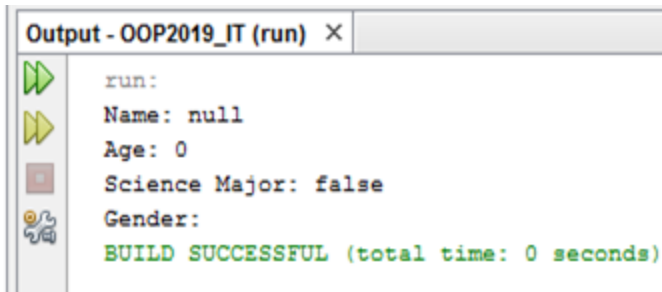
The following **Student** class contains a data field name of the **String** type. **String** is a predefined Java class.

```
class Student {  
    String name; // name has default value null  
    int age; // age has default value 0  
    boolean isScienceMajor; // isScienceMajor has default value false  
    char gender; // c has default value '\u0000'  
}
```

The java program for default values of student, name of String type, age of integer type, major types of boolean type and gender of character.

```
1  package Chapter3;  
2  public class DefaultValue {  
3      String name;  
4      int age;  
5      boolean isScienceMajor;  
6      char gender;  
7      public static void main(String[] args) {  
8          DefaultValue student = new DefaultValue();  
9          System.out.println("Name: " + student.name);  
10         System.out.println("Age: " + student.age);  
11         System.out.println("Science Major: " + student.isScienceMajor);  
12         System.out.println("Gender: " + student.gender);  
13     }  
14 }
```

Output:



3.2.4 Static Variables, Constants, and Methods

The data field of a class is an instance variable. An instance variable is tied to a specific instance of the class; it is not shared among objects of the same class.

Example: Suppose that you create the following objects:

```
Circle circle1 = new Circle();  
Circle circle2 = new Circle(5);
```

The radius in **circle1** is independent of the radius in **circle2** and is stored in a different memory location. Changes made to **circle1**'s radius do not affect **circle2**'s radius, and vice versa.

If you want all the instances of a class to share data, use **static variables**, also known as **class variables**. Static variables store values for the variables in a common memory location. Because of this common location, if one object changes the value of a static variable, all objects of the same class are affected. Java supports static methods as well as static variables. Static methods can be called without creating an instance of the class.

Example:

Let us modify the **Circle** class by adding a **static variable numberOfObjects** to count the number of circle objects created. When the first object of this class is created, **numberOfObjects** is **1**. When the second object is created, **numberOfObjects** becomes **2**.

The UML of the new circle class which defines the instance variable **radius** and the static variable **numberOfObjects**, the instance methods **getRadius**, **setRadius**, and **getArea**, and the static method **getNumberOfObjects**. (Note that static variables and methods are underlined in the UML class diagram.)

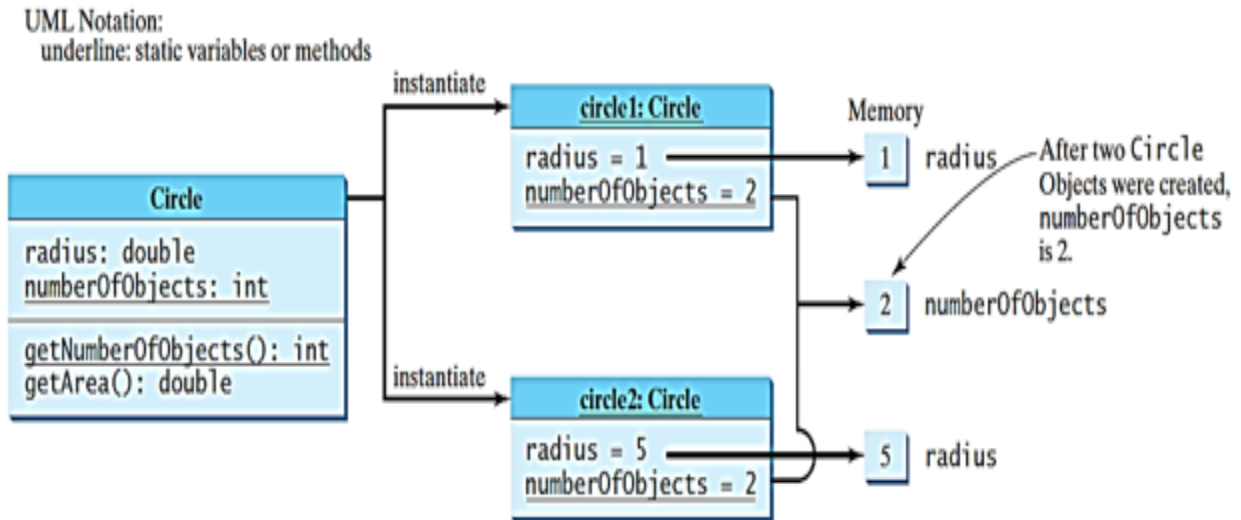


Fig. 3.4. Instance variable and static variable in a Circle class.

3.3 Accessibility Modifiers

It specifies whether data fields and methods can be accessed from the outside of the class or specify how class and class members are accessed. There is no restriction on accessing data fields and methods from inside the class. The visibility of these modifiers increases in this order:

Visibility increases
 →
 private, none (if no modifier is used), protected, public

There are basically four types of visibility specifiers. They are:

3.3.1 Public

It enables the members of the class to be accessed by any class. Public members can be accessed from any other classes. It is used for classes, methods, and data fields.

3.3.2 . Protected

It enables the members of the class to be accessed by the subclasses in any package or classes in the same package. A protected data field or method in a superclass can be accessed in its subclasses. It is used for members of the class (methods, and data fields).

3.3.3 Default

The default or no modifiers in order to allow the members of the class to be accessed directly from any class within the same package but not from other packages. This is known as *package-private* or *package-access*. It is used for classes, methods, and data fields.

3.3.4 Private

It hides the members of the class completely so that they cannot be accessed directly from outside the class. Private members can be accessed only from the inside of the class. It is used for members of the class (methods, and data fields).

Generally, class can be used in two ways:

- For creating instances of the class, and
- For defining subclasses by extending the class.

The following table summarizes the visibility modifiers.

<i>Modifier on members in a class</i>	<i>Accessed from the same class</i>	<i>Accessed from the same package</i>	<i>Accessed from a subclass</i>	<i>Accessed from a different package</i>
public	✓	✓	✓	✓
protected	✓	✓	✓	–
(default)	✓	✓	–	–
private	✓	–	–	–

Table 3.1. Data and Methods visibility

3.4 Data Field Encapsulation

o prevent direct modifications of **data fields**, you should declare the data fields **private**, using the private modifier. This is known as **data field encapsulation**.

A private data field cannot be accessed by an object from outside the class that defines the private field. But often a client needs to retrieve and modify a data field. To make a private data field accessible, provide a **get** method to return its value. To enable a private data field to be updated, provide a **set** method to set a new value.

A get method has the following signature:

```
public returnType getPropertyname()
```

If the **returnType** is **boolean**, the **get** method should be defined as follows by convention:

```
public boolean isPropertyName()
```

A set method has the following signature:

```
public void setPropertyName(dataType propertyValue)
```

Example:

Let us create a new circle class with a private data-field radius and its associated accessor and mutator methods, its UML notation is as follows.

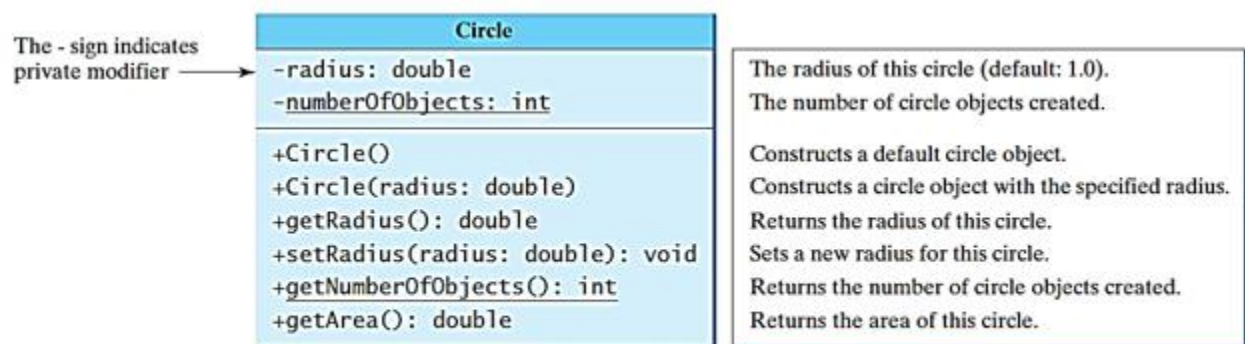


Fig. 3.5. Data field encapsulation

3.5 Passing Objects to Methods

You can pass objects to methods. Like passing an array, passing an object is actually passing the reference of the object.

Example:

```
1 package Chapter3;
2 public class PassObject {
3     public static void main(String[] args) {
4         Circle passObject = new Circle();
5         printCircle(passObject);
6     }
7     private static void printCircle(Circle passedValue) {
8         System.out.println("The area of Circle of radius " +
9             passedValue.radius + " is " + passedValue.getArea());
10    }
11 }
```

Output:

```
Output - OOP2019_IT (run) X
run:
The area of Circle of radius 1.0 is 3.141592653589793
BUILD SUCCESSFUL (total time: 0 seconds)
```

3.6 Array of Objects

The syntax to declare and create an array of Circle objects:

```
Circle[] circleArray = new Circle[arraySize];
```

To initialize the circleArray, you can use a for loop like this one:

```
for (int i = 0; i < circleArray.length; i++) {
    circleArray[i] = new Circle();
}
```

An array of objects is actually an array of reference variables. So, invoking **circleArray[1].getArea()** involves two levels of referencing. circleArray references the entire array. **circleArray[1]** references a Circle object.

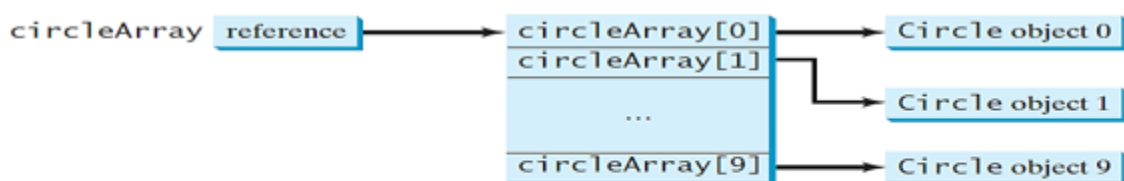


Fig. 3.6. Representation of an array of objects.

Example:

Write java program that computes an area of five circle. Display each area of the circle and total area of five circles.

```
1 package Chapter3;
2 public class CircleArray {
3     public static void main(String[] args) {
4         CircleArray[] circleArray = new CircleArray[5];
5         for(int i = 0; i < circleArray.length; i++){
6             circleArray[i] = new CircleArray(Math.random() * 20);
7         }
8         printCircleArray(circleArray);
9     }
10    double radius;
11    CircleArray(double r) {
12        this.radius = r;
13    }
14    double getArea(){
15        return radius * radius * Math.PI;
16    }
17    private static void printCircleArray(CircleArray[] ca) {
18        System.out.println("\tArea of a Circle");
19        System.out.println("-----");
20        System.out.printf("%-30s%-15s\n", "Radius", "Area");
21        for (int i = 0; i < ca.length; i++) {
22            System.out.printf("%-30f%-15f\n", ca[i].radius, ca[i].getArea());
23        }
24        System.out.println("-----");
25        System.out.printf("%-30s%-15f\n", "The total area of circles is",
26            sum(ca));
27    }
28    public static double sum(CircleArray[] circleArray){
29        double sum = 0;
30        for (int i = 0; i < circleArray.length; i++)
31            sum += circleArray[i].getArea();
32        return sum;
33    }
34 }
```

Output:

```
Output - OOP2019_IT (run) X
run:
    Area of a Circle
-----
Radius                               Area
11.847077                            440.932704
1.288510                             5.215855
13.962018                            612.415595
0.840704                             2.220425
3.042593                             29.082891
-----
The total area of circles is 1089.867469
BUILD SUCCESSFUL (total time: 0 seconds)
```

3.7 Chapter Review Questions

1. What are 'access specifiers'?
2. What is the difference between public, private and protected access modifiers?
3. What is garbage collector?
4. Define final variable?
5. Define finalize method?
6. Define inheritance?
7. What is a Constructor?
8. Can we have a class with no Constructor in it? What will happen during object creation?
9. Write a program to create a class **Student2** along with two method **getData(),printData()** to get the value through argument and display the data in printData. Create the two objects **s1, s2** to declare and access the values from class **STtest**.
10. What is the difference between public, private and protected access modifiers?
11. Develop a Java application to generate Electricity bill. Create a class with the following members: Consumer no., consumer name, previous month reading, current month reading, and type of EB connection (i.e domestic or commercial). Compute the bill amount using the following tariff. If the type of the EB connection is domestic, calculate the amount to be paid as follows:
 - ➔ First 100 units - Rs. 1 per unit
 - ➔ 101-200 units - Rs. 2.50 per unit
 - ➔ 201 -500 units - Rs. 4 per unit
 - ➔ > 501 units - Rs. 6 per unit

If the type of the EB connection is commercial, calculate the amount to be paid as follows:

- ➔ First 100 units - Rs. 2 per unit
 - ➔ 101-200 units - Rs. 4.50 per unit
 - ➔ 201 -500 units - Rs. 6 per unit
 - ➔ > 501 units - Rs. 7 per unit
12. Develop a java application with Employee class with Emp_name, Emp_id, Address, Mail_id, Mobile_no as members. Inherit the classes, Programmer, Assistant Professor, Associate Professor and Professor from employee class. Add Basic Pay (BP) as the member of all the inherited classes with 97% of BP as DA, 10 % of BP as HRA, 12% of BP as PF, 0.1% of BP for staff club fund. Generate pay slips for the employees with their gross and net salary.
 13. Design a class to represent a Student details include the Student ID, Name of the Student, Branch, year, location and college. Assign initial values using constructor. Calculate average of marks of 6 subjects and calculate attendance percentage.
 14. Write a program to demonstrate static variables, methods, and blocks.

Chapter Four

4 Object Oriented Programming Concepts

4.1 Introduction

Object-oriented programming allows you to derive new classes from existing classes. This is called inheritance. Inheritance is an important and powerful feature in Java for reusing software. Polymorphism enables you to “*program in the general*” rather than “*program in the specific*.” Abstract classes cannot be used to instantiate objects, because they are incomplete. Subclasses must declare the “missing pieces” to become “concrete” classes, from which you can instantiate objects. In the Java programming language, an interface is a reference type that can contain only constants, method signatures, and nested types. There are no method bodies. The Java API provides several predefined data structures, called collections, used to store groups of related objects.

4.2 OOP Principles in Java

4.2.1 . Encapsulation

Encapsulation simply means binding object state (data fields) and behavior (methods) together. It hides the implementation details from users. We can create a fully encapsulated class in Java by making all the *data members* of the class **private**. Now we can use setter and getter methods to set and get the data in it.

Example: A capsule which is mixed of several medicines.



Advantages of Encapsulation:

- By providing only a setter or getter method, you can make the class read-only or write-only.
- It provides you the control over the data. Suppose you want to set the value of ID which should be greater than 100 only, you can write the logic inside the **setter** method.
- It is a way to achieve **data hiding** in Java because other class will not be able to access the data through the private data members.

Example:

Write a java program that computes the area of a circle using encapsulation.

```
1  package Chapter4;
2  public class EncCircle {
3      private double radius;
4      public double setArea(double r){
5          return this.radius = r;
6      }
7      public double getArea(){
8          return Math.pow(radius, 2) * Math.PI;
9      }
10     public static void main(String[] args) {
11         EncCircle encapsulate = new EncCircle();
12         encapsulate.setArea(5);
13         System.out.println("Area is: " + encapsulate.getArea());
14     }
15 }
```

Output:

```
run:
Area is: 78.53981633974483
BUILD SUCCESSFUL (total time: 0 seconds)
```

Note:

setter is used only for write-only and **getter** is used only for read-only!

4.2.2 Inheritance

You use a class to model objects of the same type. Different classes may have some common properties and behaviors, which can be generalized in a class that can be shared by other classes. Inheritance enables you to define a general class and later extend it to more specialized classes. The specialized classes inherit the properties and methods from the general class.

Consider Geometric Objects. Suppose you want to design the classes to model Geometric Objects such as circles and rectangles. Geometric Objects have many common properties and behaviors. Assume that this class also contains the **dateCreated** property, **getDateCreated()** method and **displayDate()** methods. The **displayDate()** method returns a string representation for the object. Since a Circle is a special type of Geometric Object, it shares common properties and methods with other Geometric Objects. Thus, it makes sense to define the Circle class that extends the **GeometricObject** class. Likewise, Rectangle can also be declared as a **subclass** of GeometricObject.

Therefore, the GeometricObject is called *a super-class* or *parent class* or *base-class*, while Circle and Rectangle are *called sub-class* or *derived class* or *extended class* or *child-class*. A subclass inherits accessible data fields and methods from its superclass and may also add new data fields and methods.

The **Circle** and **Rectangle** class inherits all accessible data fields and methods from the **GeometricObject** class. In addition, it has a new data field, radius and also contains the **getArea()**.

Likewise, rectangle has the data fields, height and width and method **getArea()**.

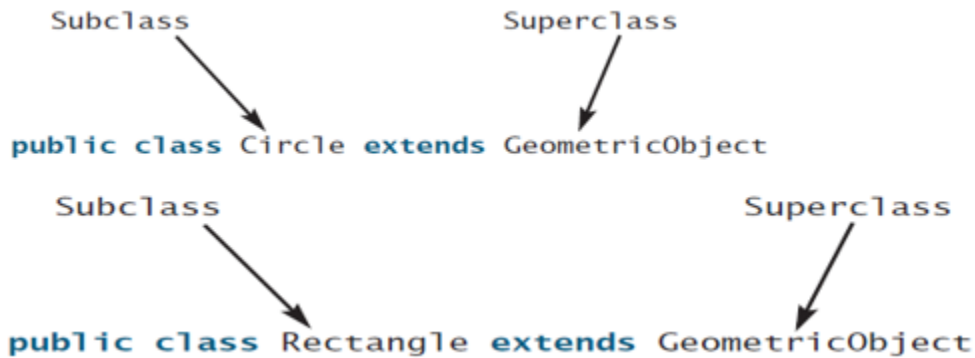
Syntax of inheritance:

```
public class Subclass extends Superclass
```

Note:

- The keyword **extend** tells the compiler that the subclass extends the common features from the superclass, thus inheriting the data fields and methods.

Example:



Features of inheritance:

- A subclass is not a subset of its superclass. In fact, a subclass usually contains more information and methods than its superclass.
- Private data fields in a superclass are not accessible outside the class. Therefore, they cannot be used directly in a subclass. They can, however, be accessed/mutated through public accessor/mutator if defined in the superclass.
- Inheritance is used to model the **is-a** relationship. A subclass and its superclass must have the **is-a** relationship.
- Some programming languages allow you to derive a subclass from several classes. This capability is known as **multiple inheritance**. Java, however, does not allow multiple inheritance. A Java class may inherit directly from only one superclass. This restriction is known as **single inheritance**. If you use the extends keyword to define a subclass, it allows only one parent class. Nevertheless, multiple inheritance can be achieved through **interfaces**.
- The inheritance relationship enables a subclass to inherit features from its superclass with additional new features. A subclass is a specialization of its superclass; every instance of a subclass is also an instance of its superclass, but not vice versa.

Example:

Write a java program with two subclasses, Circle and Rectangle that inherit the date created from the Geometry with their any respective coordinates.

Superclass Inheritance Geometry (InGeometry):

```
1  package Chapter4;
2  import java.util.Date;
3  @
4  public class InGeometry {
5      Date dateCreated = new Date();
6      double x, y;
7      public InGeometry(int xCoordinate, int yCoordinate) {
8          this.x = xCoordinate;
9          this.y = yCoordinate;
10     }
11     public Date getDateCreated(){
12         return dateCreated;
13     }
14     public String displayDate(){
15         return "Created on: " + getDateCreated() + " and has [" +
16             x + ", " + y + "]" + " axis.";
17     }
18 }
```

First subclass – Circle (InCircle):


```

1 package Chapter4;
2 public class InCircle extends InGeometry{
3     private double radius;
4     protected InCircle(double r){
5         super(1, 3);
6         this.radius = r;
7     }
8     public double getArea(){
9         return radius * radius * Math.PI;
10    }
11 }

1 package Chapter4;
2 public class InRectangle extends InGeometry{
3     private final double width;
4     private final double height;
5     public InRectangle(double W, double H) {
6         super(3, 4);
7         this.width = W;
8         this.height = H;
9     }
10    public double getArea(){
11        return height * width;
12    }
13 }

```

Main method:

```

1 package Chapter4;
2 public class InDemo {
3     public static void main(String[] args) {
4         InCircle circle = new InCircle(2);
5         System.out.println("A Circle " + circle.displayDate());
6         System.out.println("Circle area is " + circle.getArea());
7
8         InRectangle rectangle = new InRectangle(2, 4);
9         System.out.println("\nA Rectangle " + rectangle.displayDate());
10        System.out.println("Rectangle Area is " + rectangle.getArea());
11    }
12 }

```

Output:



```
run:
A Circle Created on: Wed May 08 09:27:22 EAT 2019 and has [1.0, 3.0] axis.
Circle area is 12.566370614359172

A Rectangle Created on: Wed May 08 09:27:22 EAT 2019 and has [3.0, 4.0] axis.
Rectangle Area is 8.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

4.2.2.1 The super Keyword

A subclass inherits accessible data fields and methods from its superclass. It refers to the superclass of the class in which super appears. It can be used:

- To access the data members of parent class when both parent and child class have member with same name.
- To explicitly call the no-arg and parameterized constructor of parent class
- To access the method of parent class when child class has overridden that method.

The syntax to call a superclass constructor and super-class methods are, respectively:

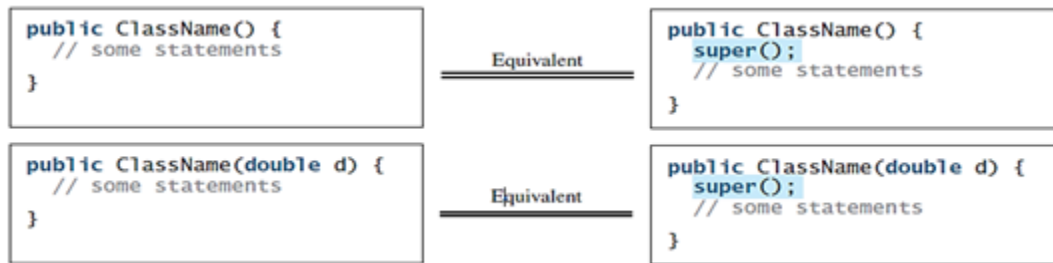
super(), or **super(parameters); super.method(parameters);**

The statement **super()** or **super(arguments)** must appear in the first line of the subclass constructor; this is the only way to explicitly invoke a superclass constructor.

4.2.2.2 Constructor Chaining

A constructor may invoke an overloaded constructor or its superclass constructor. If neither is invoked explicitly, the compiler automatically puts **super()** as the first statement in the constructor.

Example:



In any case, constructing an instance of a class invokes the constructors of all the superclasses along the inheritance chain. When constructing an object of a subclass, the subclass constructor first invokes its superclass constructor before performing its own tasks. If the superclass is derived from another class, the superclass constructor invokes its parent-class constructor before performing its own tasks. This process continues until the last constructor along the inheritance hierarchy is called. This is **constructor chaining**.

Note:

- *If a class is designed to be extended, it is better to provide a no-arg constructor to avoid programming errors.*
- *It is better to provide a no-arg constructor (if desirable) for every class to make the class easy to extend and to avoid errors.*

4.2.3 Method Overloading

It is a method that define multiple methods with the same name but **different signatures** with parameter list within one class. The Java compiler determines which method is used based on the method signature.

Example:

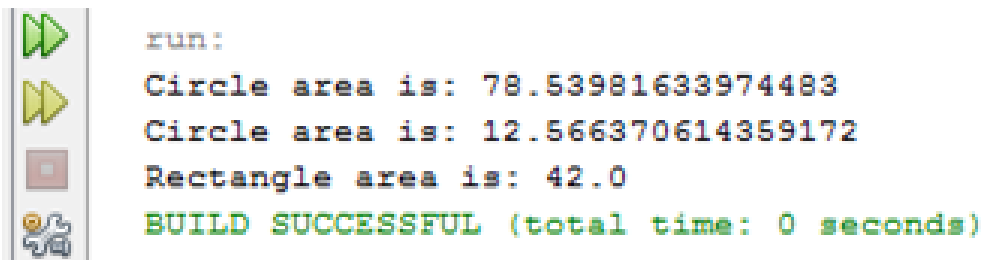
Write a java program that computes area of a Circle and Rectangle and make the Rectangle static.

```

1  package Chapter4;
2  public class MethodOverloading {
3      public double area(int radius){
4          return Math.pow(radius, 2) * Math.PI;
5      }
6      public double area(double radius){
7          return Math.pow(radius, 2) * Math.PI;
8      }
9      public static double area(int height, double width){
10         return height * width;
11     }
12     public static void main(String[] args) {
13         MethodOverloading overload = new MethodOverloading();
14         System.out.println("Circle area is: " + overload.area(5));
15         System.out.println("Circle area is: " + overload.area(2));
16         System.out.println("Rectangle area is: " + area(6, 7));
17     }
18 }

```

Output:



```

run:
Circle area is: 78.53981633974483
Circle area is: 12.566370614359172
Rectangle area is: 42.0
BUILD SUCCESSFUL (total time: 0 seconds)

```

Note: Overloaded methods must have different parameter lists. You cannot overload methods based on different modifiers or return types.

4.2.4 Overriding Methods

Overriding means to provide a new implementation for a method in the subclass. The method is already defined in the superclass. To override a method, the method must be defined in the subclass using the same signature and the same return type.

A subclass inherits methods from a superclass. Sometimes it is necessary for the subclass to modify the implementation of a method defined in the superclass.

Features of Overriding Methods:

- An instance method can be overridden only if it is accessible. Thus, a **private method** cannot be overridden, because it is not accessible outside its own class. If a method defined in a subclass is private as in its superclass, the two methods are completely unrelated.
- A static method can be inherited. However, a *static method* cannot be *overridden*. If a static method defined in the superclass is redefined in a subclass, the method defined in the *superclass* is hidden. The hidden static methods can be invoked using the syntax ***SuperClassName.staticMethodName***.

Example:

Write a java program that override the circle perimeter computation.

The super-class to be extended:

```
1 package Chapter4;
2 import static java.lang.Math.PI;
3 import static java.lang.Math.ceil;
4 public class OverrideSuper {
5     public void perimeter(double radius) {
6         System.out.println("Superclass perimeter: " + ceil(2*PI*radius));
7     }
8 }
```

The sub-class that extends:

```
1 package Chapter4;
2 import static java.lang.Math.*;
3 public class OverrideSub extends OverrideSuper{
4     @Override
5     public void perimeter(double radius) {
6         //This method overrides the method in parent class
7         System.out.println("Subclass perimeter: " + ceil(2*PI*radius));
8     }
9 }
```

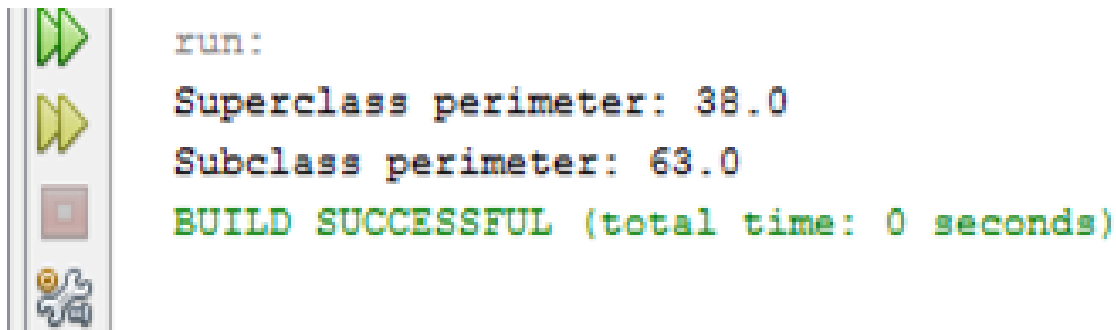
The main method for creating objects:

```

2 public class OverDemo{
3     public static void main(String[] args) {
4         OverrideSub subObject = new OverrideSub();
5         OverrideSuper superObject = new OverrideSuper();
6         superObject.perimeter(6);
7         subObject.perimeter(10.0D); //Overriding
8     }
9 }

```

Output:



```

run:
Superclass perimeter: 38.0
Subclass perimeter: 63.0
BUILD SUCCESSFUL (total time: 0 seconds)

```

Note:

- Every class in Java is descended from the **java.lang.Object** class. If no inheritance is specified when a class is defined, the superclass of the class is **Object** by default.

Example:

<pre>public class ClassName { ... }</pre>	Equivalent	<pre>public class ClassName extends Object { ... }</pre>	
---	--	--	--

4.2.5 Polymorphism

Polymorphism is derived from two Greek words: poly and morphs. The word “poly” means many and “morphs” means forms. So, polymorphism means many forms. It allows us to perform a single action in different ways or it is the capability of a method to do different things based on the object that it is acting upon. In other words, polymorphism allows you define one interface and have multiple implementations.

A class defines a **type**. A type defined by a subclass is called a **subtype** and a type defined by its superclass is called a **supertype**.

Suppose we have a class **Geometry** that has a method **area()**. The action for the different geometry like circle, rectangle, square, etc., are different. Now consider two subclasses of Geometry as **Circle** and **Rectangle** that extends from Geometry. All the Geometry have an area, but there were different ways to do the same action. This feature allows us to perform a single action in different ways. It would not make any sense to just call the generic area() method as each Geometry has a different area. Thus, we can say that the action this method performs is based on the type of object.

So, the method area() have multiple implementations in the different-2 subclasses. Which area() method will be called is determined at **runtime**. Therefore, it is a *runtime polymorphism*.

Example:

Write a java program that computes the area of Circle and Rectangle using polymorphism.

Superclass (PolyGeometry):

```
1 package Chapter4;
2 public class PolyGeometry {
3     public void area() {
4         System.out.println("Geometry has an area");
5     }
6 }
```

Firs subclass, Circle (PolyCircle):

```
1 package Chapter4;
2 public class PolyCircle extends PolyGeometry{
3     public double area(double radius){
4         return Math.pow(radius, 2) * Math.PI;
5     }
6 }
```

Second subclass, Rectangle (PolyRectangle):


```

1  package Chapter4;
2  public class PolyRectangle extends PolyGeometry{
3      public double area(double Height, double Width){
4          return Height * Width;
5      }
6  }

```

Main method:

```

1  package Chapter4;
2  public class PolyDemo {
3      public static void main(String[] args) {
4          PolyCircle obj = new PolyCircle();
5          PolyRectangle obj1 = new PolyRectangle();
6          System.out.println("Circle area is: "+obj.area(4)); //Compile time
7          System.out.println("Rectangle area is: "+obj1.area(5, 4)); //Compile time
8          obj.area(); //Runtime
9          obj1.area(); //Runtime
10     }
11 }

```

Output:

```

run:
Circle area is: 50.26548245743669
Rectangle area is: 20.0
Geometry has an area
Geometry has an area
BUILD SUCCESSFUL (total time: 0 seconds)

```

Note:

- *Method Overloading in Java – compile time (or static polymorphism)*
- *Method Overriding in Java – runtime time (or dynamic polymorphism)*

4.2.6 Abstraction

It is a process of hiding the implementation details from the user. Only the functionality will be provided to the user. In Java, abstraction is achieved using *abstract classes* and *interfaces*

4.2.7 Abstract Class

In the inheritance hierarchy, classes become more *specific* and *concrete* with each *new subclass*. If you move from a **subclass** back up to a **superclass**, the classes become more **general** and less specific. Class design should ensure that a superclass contains common features of its subclasses. Sometimes a superclass is so **abstract** that it cannot have any specific instances. Such a class is referred to as an **abstract class**.

- **Abstract Methods** – it is a method that didn't implemented (i.e. defined without implementation) in the superclass, its implementation is provided by the subclasses and are denoted using the **abstract** modifier in the method header. A class that contains abstract methods must be defined abstract

Abstract classes are like regular classes, but you cannot create instances of abstract classes using the **new** operator. The constructor in the abstract class is defined *protected*, because it is used only by subclasses. When you create an instance of a concrete subclass, its superclass's constructor is invoked to initialize data fields defined in the superclass.

Features of Abstract Classes:

- An abstract method cannot be contained in a non-abstract class. If a subclass of an abstract superclass doesn't implement all the abstract methods, the subclass must be defined abstract. In other words, in a non-abstract subclass extended from an abstract class, all the abstract methods must be implemented. Also note that abstract methods are non-static.
- An abstract class cannot be instantiated using the **new** operator, but you can still define its constructors, which are invoked in the constructors of its subclasses.
- A subclass can be abstract even if its superclass is concrete. **Example:** The `Object` class is concrete, but its subclasses, such as **`GeometricObject`**, may be abstract.
- A subclass can override a method from its superclass to define it abstract. This is very unusual but is useful when the implementation of the method in the superclass becomes invalid in the subclass. In this case, the subclass must be defined abstract.
- You cannot create an instance from an abstract class using the **new** operator, but an abstract class can be used as a **data type**. Therefore, the following statement, which creates an array whose elements are of the `GeometricObject` type, is correct.

```
AbstGeometry[] circle = new AbstGeometry[10];
```

You can then create an instance of GeometricObject and assign its reference to the array like this:

```
AbstGeometry[] circle = new AbstCircle[10];
```

Example:

Write a java program that has a concrete subclasses (AbstCircle) and one abstract superclass (AbstGeometry). The superclass has contained two abstract methods that computes the area (getArea (parameter list)) of circle with parameter.

The superclass (AbstGeometry):

```
1 package Chapter4;
2 public abstract class AbstGeometry {
3     public abstract double getCircleArea(int radius);
4     public abstract double getRectangleArea(int height, int width);
5 }
```

Circle concrete class (AbstCircle):

```
1 package Chapter4;
2 public class AbstCircle extends AbstGeometry{
3     @Override
4     public double getCircleArea(int r) {
5         return Math.pow(r, 2) * Math.PI;
6     }
7     @Override
8     public double getRectangleArea(int h, int w) {
9         return h * w;
10    }
11 }
```

Main method (AbstDemo):

```

1 package Chapter4;
2 public class AbstDemo {
3     public static void main(String[] args) {
4         AbstGeometry circle = new AbstCircle();
5         System.out.println("Circle area is: "+circle.getCircleArea(5));
6         System.out.println("Rectangle area is: "+circle.getRectangleArea(7, 4));
7     }
8 }

```

Output:

```

run:
Circle area is: 78.53981633974483
Rectangle area is: 28.0
BUILD SUCCESSFUL (total time: 0 seconds)

```

4.2.8 Interfaces

An interface is for defining common behavior for classes. It is a reference type, similar to a class, that can contain only *constants*, *method signatures*, and *nested types*. There are no method bodies, i.e. the method signatures have no braces and are terminated with a semicolon. Interfaces cannot be instantiated they can only be implemented by classes or extended by other interfaces.

Declaring Interface:

Syntax:

```

modifier interface InterfaceName {
    /** Constant declarations */
    /** Method signatures */
}

```

Example:

```

public interface IntCircle {
    final static double PI = Math.PI;
    double getArea(int radius);
}

```

It is a special class in Java. Each interface is compiled into a separate bytecode file, just like a regular class. The class for the object implements this **interface** using the **implements** keyword. When a class implements an interface, it implements all the methods defined in the interface with

the exact signature and return type. The relationship between the class and the interface is known as **interface inheritance**.



Note

Since all data fields are *public final static* and all methods are *public abstract* in an interface, Java allows these modifiers to be omitted. Therefore the following interface definitions are equivalent:

```
public interface T {  
    public static final int K = 1;  
    public abstract void p();  
}
```

Equivalent

```
public interface T {  
    int K = 1;  
    void p();  
}
```

An interface with an empty body is referred to as a **marker interface**. A marker interface does not contain constants or methods. It is used to denote that a class possesses certain desirable properties.

Extending Interface

An interface can inherit other interfaces using the `extends` keyword. Such an interface is called a **subinterface**.

Syntax:

```
Modifiers interface subInterfaceName extends superInterfaceName {  
    // Constant declarations, Method signatures, ...  
}
```

Multiple Interface-inheritance:

NewInterface in the following code is a subinterface of Interface1, and InterfaceN.

```
public interface NewInterface extends Interface1, ..., InterfaceN {  
    // constants and abstract methods  
}
```

Implementing an Interface

To use an interface, you write a class that implements the interface. When an instantiable class implements an interface, it provides a method body for each of the methods declared in the interface. To declare a class that implements an interface, you include an **implements** clause in the class declaration.

Syntax:

```
Modifiers class ClassName implements InterfaceName {
```

```
// Method signatures, Methods body
```

```
}
```

Your class can implement more than one interface, so the **implements** keyword is followed by a comma-separated list of the interfaces implemented by the class. By convention, the implements clause follows the extends clause, if there is one.

Syntax:

```
Modifiers class ClassName implements InterfaceName1, InterfaceName2 ... {
```

```
/**Constant declarations, method signatures
```

```
}
```

Combining inheritance and interface implementation together:

```
Modifiers class NewClass extends BaseClass implements InterfaceName1, ...InterfaceNameN {
```

```
/**Constant declarations, method signatures*/
```

```
}
```

Generally, in Java, a class can inherit from only one class but it can implement more than one interface. Java allows only single inheritance for class extension but allows multiple extensions

for interfaces. An interface can extend other interfaces but not classes. A class can extend its superclass and implements multiple interfaces.

Example:

Write a java program that computes the area and perimeter of circle using interface.

Circle Interface (Circle):

```
1 package Chapter4;
2 public interface Circle {
3     final static double PI = Math.PI;
4     double getCircleArea(int radius);
5     double getCirclePerimeter(int radius);
6 }
```

Circle class that implements the interface (CircleClass):

```
1 package Chapter4;
2 public class CircleClass implements Circle{
3     @Override
4     public double getCircleArea(int radius) {
5         return radius * radius * PI;
6     }
7     @Override
8     public double getCirclePerimeter(int radius) {
9         return 2 * PI * radius;
10    }
11 }
```

The main method class (IntDemo):

```
1 package Chapter4;
2 public class IntDemo{
3     public static void main(String[] args) {
4         CircleClass circle = new CircleClass();
5         System.out.println("Circle area is: " + circle.getCircleArea(5));
6         System.out.println("Circle Perimeter is: " +
7             circle.getCirclePerimeter(5));
8     }
9 }
```

Output:

```
run:
Circle area is: 78.53981633974483
Circle Perimeter is: 31.41592653589793
BUILD SUCCESSFUL (total time: 0 seconds)
```

All classes share a single root, the Object class, but there is no single root for interfaces. Like a class, an interface also defines a type. A variable of an interface type can reference any instance of the class that implements the interface. If a class implements an interface, the interface is like a superclass for the class. You can use an interface as a data type and cast a variable of an interface type to its subclass, and vice versa.

In general, interfaces are preferred over abstract classes because an interface can define a common super-type for unrelated classes. Interfaces are more flexible than classes.

4.2.9 Packages

It is a grouping of related types providing access protection and name space management. The types refer to classes, interfaces, enumerations, and annotation types. Enumerations and annotation types are special kinds of classes and interfaces, respectively.

The types that are part of the Java platform are members of various packages that bundle classes by function: Fundamental classes are in ***java.lang***, classes for reading and writing (input and output) are in ***java.io***, and so on. You can put your types in packages too.

Creating a Packages

To create a package, you choose a name for the package and put a package statement with that name at the top of every source file that contains the types (classes, interfaces, enumerations, and annotation types) that you want to include in the package. The package statement must be the first line in the source file. There can be only one package statement in each source file, and it applies to all types in the file.

Naming Conventions

- Package names are written in all lowercase to avoid conflict with the names of classes or interfaces.
- Packages in the Java language itself begin with ***java.*** or ***javax.***

The types that comprise a package are known as the package members. To use a public package member from outside its package (*now consider you are currently in package Chapter4*), you must do one of the following:

- a. Refer to the member by its fully qualified name. **Example:** Chapter3.Circle

You could use this qualified name to create an instance of Chapter3.Circle:

```
Chapter3.Circle myRect = new Chapter3.Circle ()
```

2. Import the package member. **Example:** import Chapter3.Circle;

Now you can refer to the Rectangle class by its simple name:

```
Circle c = new Circle();
```

3. Import the member's entire package. **Example:** import Chapter3.*;

To import all the types contained in a particular package, use the import statement with the asterisk (*) wildcard character. Now you can refer to any class or interface in the graphics package by its simple name:

```
Circle c = new Circle();
```

```
PassObject p = new PassObject();
```

The asterisk in the import statement can be used only to specify all the classes within a package. It cannot be used to match a subset of the classes in a package.

For convenience, the Java compiler automatically imports three entire packages for each source file:

1. The package with no name,
2. The **java.lang** package, and
3. The current package (the package for the current file).

4.2.10 Template

It makes easier to implement complex algorithms by encapsulating logic in a single method. **Examples:** A ruler is a template when used to draw a straight line.

Template method is used to manage algorithms, relationships and responsibilities between objects. In the template, each step of the algorithm is represented by a method. The template method defines the steps of an algorithm and allows subclasses to provide the implementation for one or more steps. Thus, template method defines algorithm but exact steps can be defined in subclasses.

Example:

The process of making a circle. So, how circle is created?

- a. Prepare instrument (pair of compasses)
- b. Identifying the central point.
- c. Specify the radius
- d. Create circle.

The process of making a rectangle. So, how rectangle is created?

- a. Prepare instrument (ruler)
- b. Identifying the central point.
- c. Specify the height and width
- d. Create rectangle.

The UML notation of the above examples are

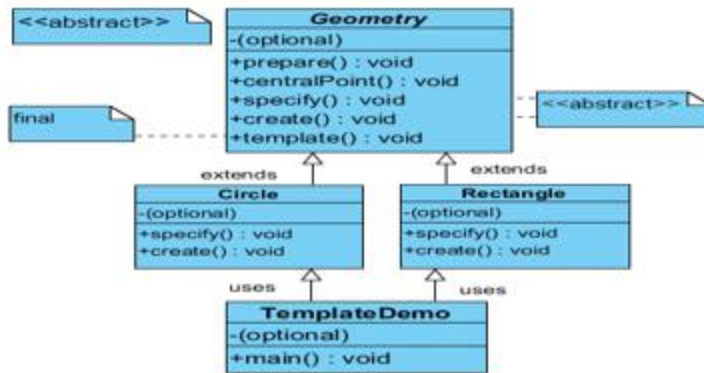


Fig. 4. The template class diagram.

Note:

- Template method should consist of certain steps whose order is fixed and for some of the methods, implementation differs from base class to subclass. Template method should be **final**.
- Most of the times, subclasses call methods from super class but in template, superclass template method calls methods from subclasses, this is known as **Hollywood Principle** – “don’t call us, we’ll call you.”.
- Methods in base class with default implementation are referred as **Hooks** and they are intended to be overridden by subclasses, if you want some of the methods to be not overridden, you can make them final, for example in our case we can make template() method final because if we don’t want subclasses to override it.

Write a java program of the above UML notation.

The abstract class (TempGeometry):

```

1 package Chapter4;
2 public abstract class TempGeometry {
3     final void template() {
4         prepare();
5         centralPoint();
6         specify();
7         create();
8     }
9     private void prepare() {
10        System.out.println("Prepare instrument.");
11    }
12    private void centralPoint() {
13        System.out.println("Identify central point.");
14    }
15    abstract void specify();
16    abstract void create();
17 }
  
```

Concrete class-1- (TempCircle):

```

1  package Chapter4;
2  public class TempCircle extends TempGeometry{
3      @Override
4      void specify() {
5          System.out.println("Radius is used.");
6      }
7      @Override
8      void create() {
9          System.out.println("Circle is created by RADIUS");
10     }
11 }

```

Concrete class-2- (TempRectangle):

```

1  package Chapter4;
2  public class TempRectangle extends TempGeometry{
3      @Override
4      void specify() {
5          System.out.println("Height and width are used.");
6      }
7      @Override
8      void create() {
9          System.out.println("Rectangle is created using height & width");
10     }
11 }

```

The main method:

```

1  package Chapter4;
2  public class TempDemo {
3      public static void main(String[] args) {
4          TempCircle circle = new TempCircle();
5          TempRectangle rectangle = new TempRectangle();
6          System.out.println("\tCreating a CIRCLE");
7          circle.template();
8          System.out.println("=====");
9          System.out.println("\tCreating a RECTANGLE");
10         rectangle.template();
11     }
12 }

```

Output:

```

run:
    Creating a CIRCLE
Prepare instrument.
Identify central point.
Radius is used.
Circle is created by RADIUS
=====
    Creating a RECTANGLE
Prepare instrument.
Identify central point.
Height and width are used.
Rectangle is created using height & width
BUILD SUCCESSFUL (total time: 0 seconds)

```

Benefits of template:

- It reuses of the code because it uses inheritance. Only a few methods to override.
- There is no duplication of code.
- Subclasses can decide how to introduce the steps in an algorithm with mobility.
- The duplicate code can be pulled into a superclass

4.3 Chapter Review Questions

1. Describe how subclass methods override super class?
2. What is super keyword?
3. Which object is used to call super class members?
4. What are Encapsulation, Inheritance and Polymorphism?
5. What is polymorphism?
6. What is static polymorphism?
7. What is dynamic polymorphism?
8. What is method overloading?
9. What is method overriding and when we use it?
10. What is operator overloading?
11. State the differentiate between overloading and overriding.
12. What is data abstraction?
13. How to achieve data abstraction?
14. What is an abstract class?
15. What are destructors?
16. Define function templates
17. Define class templates
18. Can A Class Extends More Than One Class In Java?
19. What is Inheritance in Java and how to use it?
20. How many types of constructors are used in Java?
21. Is constructor inherited?
22. What are the differences between the constructors and methods?

23. Write a program in JAVA to create a class Bird also declares the different parameterized constructor to display the name of Birds.
24. What is No-arg constructor?
25. Write a program in JAVA to demonstrate the method and constructor overloading.
26. Write a java program to create an abstract class named Shape that contains two integers and an empty method named print Area (). Provide three classes named Rectangle, Triangle and Circle such that each one of the classes extends the class Shape. Each one of the classes contains only the method print Area () that prints the area of the given shape
27. Write a program to give the example for method overriding concepts.
28. Develop with suitable hierarchy, classes for Point, Shape, Rectangle, Square, Circle, Ellipse, Triangle, Polygon, etc. Design a simple test application to demonstrate dynamic polymorphism.
29. Write a program to create interface A in this interface we have two method meth1 and meth2. Implements this interface in another class named MyClass.
30. Write a program to create interface named test. In this interface the member function is square. Implement this interface in arithmetic class. Create one new class called ToTestInt in this class use the object of arithmetic class.

Chapter Five

5 Exception Handling

5.1 Introduction

This chapter contains two parts. The first part is about exception handling that deals with handling of unexpected runtime errors in java program. The second ideas are about Input/ Output streams that deals with the concepts in which the user can able view the existing file, create new file, write to the file and read from the in either text I/O handling way or binary I/O handling ways. The types of each parts in details.

5.2 Exception handling overview

5.2.1 Introduction

Runtime errors occur while a program is running if the environment detects an operation that is impossible to carry out.

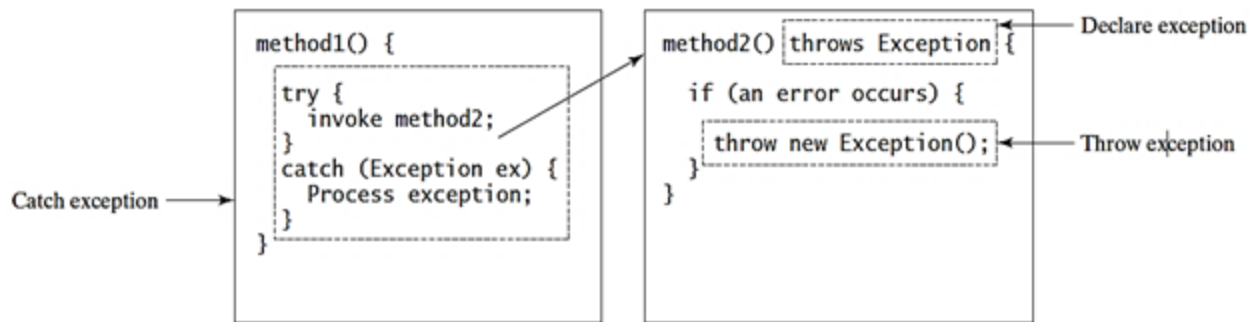
Example:

If you access an array using an index out of bounds, your program will get a runtime error with an **ArrayIndexOutOfBoundsException**. To read data from a file, you need to create a Scanner object using **new Scanner (new File(filename))**. If the file does not exist, your program will get a runtime error with a **FileNotFoundException**.

In Java, runtime errors are caused by **exceptions**. An **exception** is an object that represents an error or a condition that prevents execution from proceeding normally. If the exception is not handled, the program will terminate abnormally.

5.2.2 Exception Handling

Java's exception-handling model is based on three operations as *declare exception*, *throw exception*, and *catch exception*.



The exception-handling contains a **try block** and a **catch block**. The **try block** contains the code that is executed in normal circumstances. **The catch block** contains the code that is executed when the statements in the **try block** is not evaluated. The try block contain a **throw method** that takes the statements in the try block to catch block.

5.2.2.1 Declaring Exceptions

In Java, this is declaring an exception explicitly in the method header so that the caller of the method is informed of the exception. To declare an exception in a method, use the **throws** keyword in the method header.

Example:

```
public void myMethod() throws IOException
```

The **throws** keyword indicates that myMethod might throw an IOException. If the method might throw multiple exceptions, add a list of the exceptions, separated by commas, after throws:

```
public void myMethod()
```

```
throws Exception1, Exception2, ..., ExceptionN
```

5.2.2.2 Throwing Exceptions

It is an exception that detects an error to create an instance of an appropriate exception type and throw it.

Example:

Suppose the program that detects an error when a number is divided by zero and throw it.

```
throw new ArithmeticException ("Divisor cannot be zero.");
```

5.2.2.3 Catching Exceptions

When an exception is thrown, it can be caught and handled in a try-catch block, as follows:

```
try {  
  
    statements; // Statements that may throw exceptions  
  
} catch (Exception1 exVar1) {  
  
    handler for exception1;  
  
} .....  
  
catch (ExceptionN exVarN) {  
  
    handler for exceptionN;  
  
}
```

If no exceptions arise during the execution of the try block, the catch blocks are skipped, unless it skips the try-block and starts the process of finding the code to handle the exception. The code that handles the exception is called the **exception handler**. Each catch block is examined in turn, from first to last. If so, the exception object is assigned to the variable declared, and the code in the catch block is executed. If no handler is found, Java exits this method, passes the exception to the method that invoked the method, and continues the same process to find a handler. If no handler is found in the chain of methods being invoked, the program terminates and prints an error message on the console. The process of finding a handler is called **catching an exception**.

Example:

Write a java program that displays the quotient of any integer.

```
1 package Chapter5;
2 public class Quotient {
3     public static void main(String[] args) {
4         int num1 = 2, num2 = 0;
5         try {
6             if(num2 == 0)
7                 throw new ArithmeticException("Divisor cannot be zero");
8             System.out.println("Quotient of "+num1+" / "+num2+" is " +
9                 num1 / num2);
10        } catch (ArithmeticException ex) {
11            System.out.println("Dividing by zero is impossible");
12        }
13    }
14 }
```

Output:

```
run:
Dividing by zero is impossible
BUILD SUCCESSFUL (total time: 0 seconds)
```

In general, a template for a try-throw-catch block may look like this:

```
try {
    Code to try;
    Throw an exception with a throw statement or
    from method if necessary;
    More code to try;
}
catch (type ex) {
    Code to process the exception;
}
```

Advantages of Exception Handling

- It enables a method to throw an exception to its caller. The caller can handle this exception. Without this capability, the called method itself must handle the exception or terminate the program. Often the called method does not know what to do in case of error. This is typically the case for the library methods. The library method can detect the error, but only the caller knows what needs to be done when an error occurs.

- To separate the detection of an error (done in a called method) from the handling of an error (done in the calling method).

5.2.3 Types of Exception

There are many predefined exception classes in the Java API. They are shown in figure below.

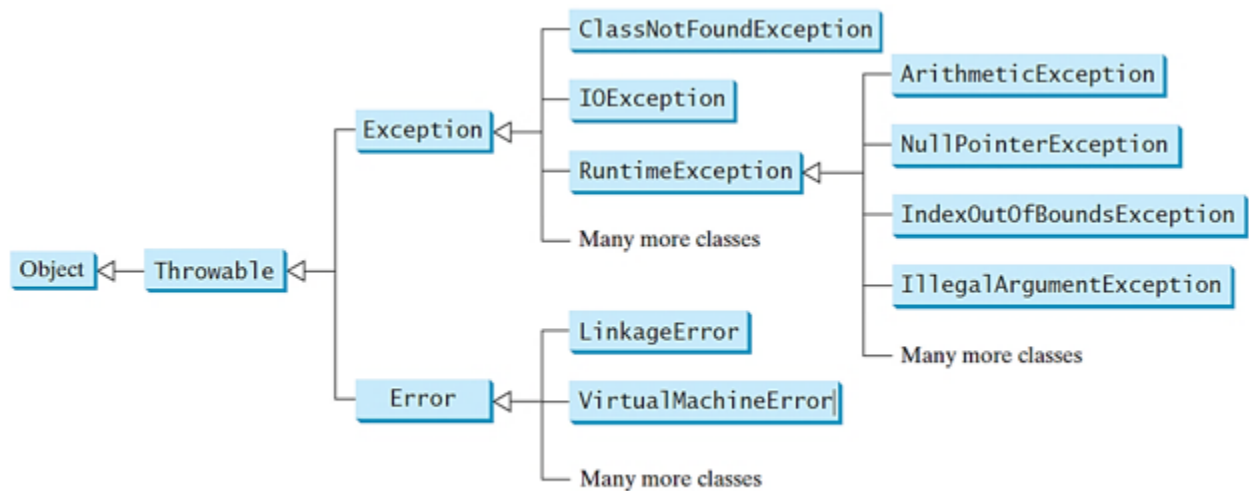


Fig. 5.1 - Exceptions thrown are instances of the classes, or subclasses.

The **Throwable** class is the root of exception classes. All Java exception classes inherit directly or indirectly from **Throwable**. You can create your own exception classes by extending **Exception** or a subclass of **Exception**. The exception classes are classified into three major types. They are

System Errors – are thrown by the JVM and represented in the **Error** class. The **Error** class describes internal system errors. Such errors rarely occur. If one does, there is little you can do beyond notifying the user and trying to terminate the program gracefully.

Class	Possible Reason for Exception
LinkageError	A class has some dependency on another class, but the latter class has changed incompatibly after the compilation of the former class.
VirtualMachineError	The JVM is broken or has run out of the resources it needs in order to continue operating.

Exceptions – are represented in the **Exception** class, which describes errors caused by your program and by external circumstances. These errors can be caught and handled by your program.

<i>Class</i>	<i>Possible Reason for Exception</i>
<code>ClassNotFoundException</code>	Attempt to use a class that does not exist. This exception would occur, for example, if you tried to run a nonexistent class using the <code>java</code> command, or if your program were composed of, say, three class files, only two of which could be found.
<code>IOException</code>	Related to input/output operations, such as invalid input, reading past the end of a file, and opening a nonexistent file. Examples of subclasses of <code>IOException</code> are <code>InterruptedException</code> , <code>EOFException</code> (EOF is short for End Of File), and <code>FileNotFoundException</code> .

Runtime Exceptions – are represented in the `RuntimeException` class, which describes programming errors, such as bad casting, accessing an out-of-bounds array, and numeric errors. Runtime exceptions are generally thrown by the JVM.

<i>Class</i>	<i>Possible Reason for Exception</i>
<code>ArithmeticException</code>	Dividing an integer by zero. Note that floating-point arithmetic does not throw exceptions.
<code>NullPointerException</code>	Attempt to access an object through a <code>null</code> reference variable.
<code>IndexOutOfBoundsException</code>	Index to an array is out of range.
<code>IllegalArgumentException</code>	A method is passed an argument that is illegal or inappropriate.

`RuntimeException`, `Error`, and their subclasses are known as **unchecked exceptions**. All other exceptions are known as **checked exceptions**, meaning that the compiler forces the programmer to check and deal with them.

In most cases, unchecked exceptions reflect programming logic errors that are **unrecoverable**.

Example:

- A `NullPointerException` is thrown if you access an object through a reference variable before an object is assigned to it;
- An `IndexOutOfBoundsException` is thrown if you access an element in an array outside the bounds of the array. These are logic errors that should be corrected in the program. Unchecked exceptions can occur anywhere in a program. To avoid cumbersome overuse of try-catch blocks, Java does not mandate that you write code to catch or declare unchecked exceptions.

5.2.4 The finally Clause

Occasionally, you may want some code to be executed regardless of whether an exception occurs or is caught. Java has a finally clause that can be used to accomplish this objective.

The syntax:

```
try {  
    statements;  
}  
catch (TheException ex) {  
    handling ex;  
}  
finally {  
    finalStatements;  
}
```

The code in the finally block is executed under all circumstances, regardless of whether an exception occurs in the try block or is caught. Consider three possible cases:

- If no exception arises in the try block, **finalStatements** is executed, and the next statement after the try statement is executed.
- If a statement causes an exception in the try block that is caught in a catch block, the rest of statements in the try block are skipped, the catch block is executed, and the **finally** clause is executed. The next statement after the try statement is executed.
- If one of the statements causes an exception that is not caught in any catch block, the other statements in the try block are skipped, the **finally** clause is executed, and the exception is passed to the caller of this method.

The finally block executes even if there is a return statement prior to reaching the **finally** block.

5.3 Input/Output (I/O) Streams in Java

5.3.1 Introduction

An I/O stream represents an input source or an output destination. A **stream** can represent many different kinds of sources and destinations, including disk files, devices, other programs, and

memory arrays. Streams support many different kinds of data, including simple bytes, primitive data types, localized characters, and objects. Some streams simply pass on data; others manipulate and transform the data in useful ways. *A stream is a sequence of data.*

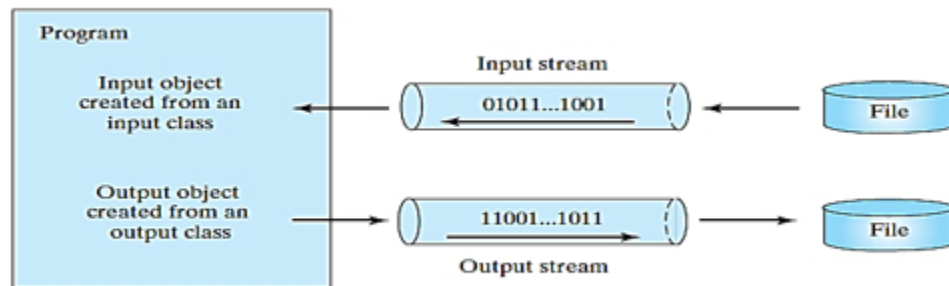


Fig. 5.2 - The program receives data through an input object and sends data through an output object.

Data stored in a text file are represented in human-readable form. Data stored in a binary file are represented in binary form. You cannot read binary files. They are designed to be read by programs.

5.3.2 Types of I/O Streams

Java offers many classes for performing file input and output. Java source programs are stored in text files and can be read by a text editor, but Java classes are stored in binary files and are read by the JVM. The advantage of binary files is that they are more efficient to process than text files. There are two types of I/O streams in java. They are “Text (File) I/O Streams” and “Binary I/O Streams.”

5.3.2.1 File I/O Streams

Computers do not differentiate binary files and text files. All files are stored in binary format, and thus all files are essentially binary files. Text I/O is built upon binary I/O to provide a level of abstraction for character encoding and decoding, as shown in Figure below.

Encoding and decoding are automatically performed for text I/O. The JVM converts a Unicode to a file-specific encoding when writing a character and converts a file-specific encoding to a Unicode when reading a character.

Example:

Suppose you write string “199” using text I/O to a file. Each character is written to the file. Since the Unicode for character ‘1’ is **0x0031**, the Unicode **0x0031** is converted to a code that depends on the encoding scheme for the file. (Note that the prefix **0x** denotes a **hex number**.) In the United States, the default encoding for text files on Windows is **ASCII**. The ASCII code for character ‘1’ is **49 (0x31 in hex)** and for character ‘9’ is **57 (0x39 in hex)**. So, to write the characters “199”, three bytes – 0x31, 0x39, and 0x39 – are sent to the output, as shown in Figure below.

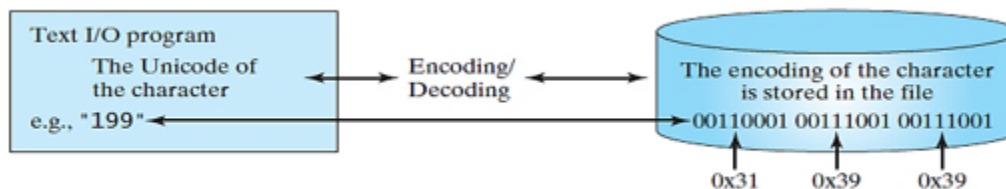


Fig. 5.3 - Text I/O requires encoding and decoding.

1. The File Class

Data stored in variables, arrays, and objects are **temporary**; they are lost when the program terminates. To permanently store the data created in a program, you need to save them in a file on a disk or a CD. Every file is placed in a directory in the file system. An absolute file name contains a file name with its complete path and drive letter.

Example:

- **C:\OOP\Welcome.java** is the absolute file name for the file **Welcome.java** on the Windows operating system. Here **C:\OOP** is referred to as the **directory path** for the file. Absolute file names are **machine dependent**.

The **File** class is intended to provide an abstraction that deals with most of the machine dependent

complexities of files and path names in a machine-independent fashion. It contains the methods for *obtaining file properties* and for *renaming* and *deleting* files, as shown in the figure below. It doesn't contain the methods for **reading** and **writing** file contents.

java.io.File	
+File(pathname: String)	Creates a File object for the specified path name. The path name may be a directory or a file.
+File(parent: String, child: String)	Creates a File object for the child under the directory parent. The child may be a file name or a subdirectory.
+File(parent: File, child: String)	Creates a File object for the child under the directory parent. The parent is a File object. In the preceding constructor, the parent is a string.
+exists(): boolean	Returns true if the file or the directory represented by the File object exists.
+canRead(): boolean	Returns true if the file represented by the File object exists and can be read.
+canWrite(): boolean	Returns true if the file represented by the File object exists and can be written.
+isDirectory(): boolean	Returns true if the File object represents a directory.
+isFile(): boolean	Returns true if the File object represents a file.
+isAbsolute(): boolean	Returns true if the File object is created using an absolute path name.
+isHidden(): boolean	Returns true if the file represented in the File object is hidden. The exact definition of <i>hidden</i> is system dependent. On Windows, you can mark a file hidden in the File Properties dialog box. On Unix systems, a file is hidden if its name begins with a period character '.'.
+getAbsolutePath(): String	Returns the complete absolute file or directory name represented by the File object.
+getCanonicalPath(): String	Returns the same as getAbsolutePath() except that it removes redundant names, such as "." and "..", from the path name, resolves symbolic links (on Unix platforms), and converts drive letters to standard uppercase (on Win32 platforms).
+getName(): String	Returns the last name of the complete directory and file name represented by the File object. For example, new File("c:\\book\\test.dat").getName() returns test.dat.
+getPath(): String	Returns the complete directory and file name represented by the File object. For example, new File("c:\\book\\test.dat").getPath() returns c:\\book\\test.dat.
+getParent(): String	Returns the complete parent directory of the current directory or the file represented by the File object. For example, new File("c:\\book\\test.dat").getParent() returns c:\\book.
+lastModified(): long	Returns the time that the file was last modified.
+length(): long	Returns the size of the file, or 0 if it does not exist or if it is a directory.
+listFile(): File[]	Returns the files under the directory for a directory File object.
+delete(): boolean	Deletes this file. The method returns true if the deletion succeeds.
+renameTo(dest: File): boolean	Renames this file. The method returns true if the operation succeeds.

Fig. 5.4 - The File class methods and properties.

The file name is a **string**. The **File** class is a wrapper class for the file name and its directory path.

Syntax:

```
File FileObjectName = new File("FileDirectory");
```

Example:

```
File file = new File("C:\\OOP\\Welcome.java");
```

In this, **new File("C:\\OOP ")** creates a **File** object for the directory **C:\\OOP**, and **new File("C:\\OOP\\Welcome.java")** creates a **File** object for the file **C:\\OOP\\Welcome.java**, both on Windows.

Example:

Create a **File** object and use the methods in the **File** class to obtain its properties. The program creates a **File** object for the file **Java.jpg**. This file is stored under the **Pictures** directory in the current directory.

```
4 public class FileClass {  
5     public static void main(String[] args) {  
6         File file = new File("C:\\Users\\AyuBest\\Pictures\\Java.jpg");  
7         System.out.println("Does file exist? " + file.exists());  
8         System.out.println("The file has " + file.length() + " bytes");  
9         System.out.println("Absolute path is " + file.getAbsolutePath());  
10        System.out.println("Last modified on " + new Date(file.lastModified()));  
11    }  
12 }
```

Output:

```
run:  
Does file exist? true  
The file has 3050 bytes  
Absolute path is C:\Users\AyuBest\Pictures\Java.jpg  
Last modified on Tue Apr 09 14:51:54 EAT 2019  
BUILD SUCCESSFUL (total time: 0 seconds)
```

2. File I/O Handling

A **File** object encapsulates the properties of a file or a path but does not contain the methods for creating a file or for reading/writing data from/to a file. In order to perform I/O, you need to create objects using appropriate Java I/O classes. The objects contain the methods for reading/writing data from/to a file. The file I/O classes used to read/write strings and numeric values from/to a text file are called the **Scanner** and **PrintWriter**, respectively.

A. **PrintWriter** Class

This is used to create and write data to a text file. The **java.io.PrintWriter** package is imported to perform the activity.

Syntax:

```
PrintWriter ObjectName = new PrintWriter (FileName);
```

Example:

```
File file = new File("OOP.txt");
```

```
PrintWriter write = new PrintWriter(file);
```

The frequently used methods to write data to a text file in **PrintWriter** are shown below.

java.io.PrintWriter	
+PrintWriter(file: File)	Creates a PrintWriter object for the specified file object.
+PrintWriter(filename: String)	Creates a PrintWriter object for the specified file-name string.
+print(s: String): void	Writes a string to the file.
+print(c: char): void	Writes a character to the file.
+print(cArray: char[]): void	Writes an array of characters to the file.
+print(i: int): void	Writes an int value to the file.
+print(l: long): void	Writes a long value to the file.
+print(f: float): void	Writes a float value to the file.
+print(d: double): void	Writes a double value to the file.
+print(b: boolean): void	Writes a boolean value to the file.

Invoking the constructor of **PrintWriter** will create a new file if the file does not exist. If the file already exists, the current content in the file will be discarded. Invoking the constructor of **PrintWriter** may throw an I/O exception. Java forces you to write the code to deal with this type of exception. You have used the **System.out.print** and **System.out.println** methods to write text to the console. **System.out** is a **standard** Java object for the console. You can create objects for writing text to any file using **print**, **println**, and **printf**. The **close()** method must be used to close the file. If this method is not invoked, the data may not be saved properly in the file.

B. Scanner Class

This is used to read primitive values and String from the console. The **java.util.Scanner** package imported to perform the activities. A Scanner breaks its input into tokens delimited by whitespace characters.

Syntax:

```
Scanner ObjectName = new Scanner(System.in);
```

Example:

```
File file = new File("OOP.txt");
```

```
Scanner input = new Scanner(file);
```

The frequently used methods in Scanner are summarized as follows:

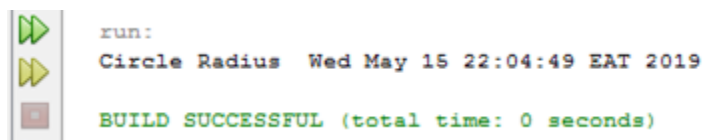
java.util.Scanner	
+Scanner(source: File)	Creates a scanner that produces values scanned from the specified file.
+Scanner(source: String)	Creates a scanner that produces values scanned from the specified string.
+close()	Closes this scanner.
+hasNext(): boolean	Returns true if this scanner has more data to be read.
+next(): String	Returns next token as a string from this scanner.
+nextLine(): String	Returns a line ending with the line separator from this scanner.
+nextByte(): byte	Returns next token as a byte from this scanner.
+nextShort(): short	Returns next token as a short from this scanner.
+nextInt(): int	Returns next token as an int from this scanner.
+nextLong(): long	Returns next token as a long from this scanner.
+nextFloat(): float	Returns next token as a float from this scanner.
+nextDouble(): double	Returns next token as a double from this scanner.
+useDelimiter(pattern: String): Scanner	Sets this scanner's delimiting pattern and returns this scanner.

Example:

Write a java program that creates and writes any geometry name (of String), attribute value (of int) and date (of Date) on “OOP.txt” using a PrintWriter concept on a separate line and consequently read its contents using Scanner class and display in a single line.

```
7 public class WriteReadData {
8     public static void main(String[] args) throws IOException {
9         File file = new File("ReadWriteFile.txt");
10        PrintWriter write = new PrintWriter(file);
11        write.println("Circle");
12        write.printf("Radius ");
13        write.println(new Date());
14        write.close();
15        Scanner input = new Scanner(file);
16        while (input.hasNext()) {
17            System.out.println(input.next() + " " + input.next() +
18                               " " + input.nextLine());
19        }
20        System.out.println();
21        input.close();
22    }
23 }
```

Output:



```
run:
Circle Radius Wed May 15 22:04:49 EAT 2019

BUILD SUCCESSFUL (total time: 0 seconds)
```

5.3.2.2 Binary I/O Handling

Binary I/O does not require conversions. If you write a numeric value to a file using binary I/O, the exact value in the memory is copied into the file.

Example:

A byte-type value **199** is represented as **0xC7** ($199 = 12 * 16^1 + 72$) in the memory and appears exactly as **0xC7** in the file, as shown in Figure below.

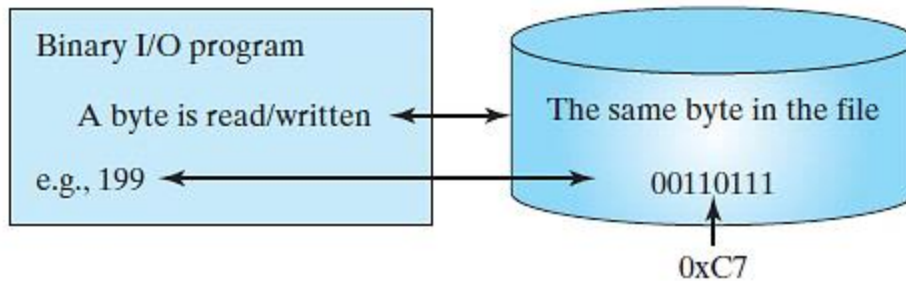


Fig. 5.5 - The way binary I/O is performed.

When you read a byte using binary I/O, one-byte value is read from the input. In general, you should use text input to read a file created by a text editor or a text output program, and use binary input to read a file created by a Java binary output program.

Binary I/O is more efficient than **text I/O**, because binary I/O does not require encoding and decoding. **Binary files** are independent of the encoding scheme on the host machine and thus are portable. Java programs on any machine can read a binary file created by a Java program. This is why Java class files are **binary files**. Java class files can run on a JVM on any machine.

5.4 Chapter Review Questions

1. Exceptions are defined in which java package?
2. How are the exceptions handled in java?
3. What are the Exception Handling Keywords in Java?
4. What is Runtime Exception or unchecked exception?
5. What is the difference between exception and error class?
6. What is difference between throw and throws keyword in Java?
7. What happens when exception is thrown by main method?
8. What is checked exception?
9. What is an event and what are the models available for event handling?
10. What is difference between **ClassNotFoundException** and **NoClassDefFoundError**?
11. What is use of throws keyword?
12. Write a program for example of try and catch block. In this check whether the given array size is negative or not.

13. Write a program for example of multiple catch statements occurring in a program.
14. Write a program to illustrate usage of try/catch with finally clause.
15. Write a program for creation of user defined exception
16. Write a program in java if number is less than 10 and greater than 50 it generate the exception out of range. Else it displays the square of number.
17. What is file?
18. What are the types of I / O streams?
19. How to create file input stream?
20. What is character stream?
21. What is byte stream?
22. How to convert the string data into long data?
23. Which input stream is used for primitive data?
24. What are the file access modes?
25. What is reader class?
26. Define file attributes and its methods?
27. What is File Input and output stream?
28. How to extract tokens?
29. What is a string class?
30. What is stringbuffer class?
31. What is the difference between reader class and writer class?
32. What are input stream classes?
33. What are output stream classes?
34. What is buffered reader class?
35. How to file object?
36. Suppose that a table named Table.txt is stored in a text file. The first line in the file header and the remaining lines correspond to row in the table. The elements are separated by commas. Write a Java program to display the table using labels in grid layout.
37. Write a Java program that reads on file name from the user, then displays information about whether the file exists, whether the file is readable, whether the file is writable, the type of file and the length of the file in bytes?

38. Write a Java program that reads a file and displays the file on the screen, with a line number before each line?
39. Write a program to create a text file in the path c:\java\abc.txt and check whether that file is exists. Using the command exists(), isDirectory(), isFile(), getName() and getAbsolutePath().
40. Write a program to create a directory and check whether the directory is created.
41. Write a program to create a file and write data into it using the methods OutputStream class.
42. Write a program to accept specified number of characters as input and converts them into uppercase characters.
43. Write a program to get the input from the user and store it into file. Using Reader and Writer file.

Chapter Six

6 GUI & Java Applets

6.1 Introduction

A graphical user interface (GUI) makes a system user friendly and easy to use. Creating a GUI requires creativity and knowledge of how GUI components work. Since the GUI components in Java are very *flexible* and *versatile*, you can create a wide assortment of useful user interfaces. Many Java IDEs provide tools for visually designing and developing GUI interfaces. This enables you to rapidly assemble the elements of a user interface (UI) for a Java application or applet with minimum coding. This chapter serves two purposes. **First**, it introduces the basics of Java GUI programming. **Second**, it uses GUI to demonstrate OOP.

6.2 Swing versus AWT

When Java was introduced, the GUI classes were bundled in a library known as the **Abstract Windows Toolkit (AWT)**. AWT is fine for developing simple graphical user interfaces, but not for developing comprehensive GUI projects. Besides, AWT is prone to platform-specific bugs. The AWT user-interface components were replaced by a more robust, versatile, and flexible library

known as **Swing components**. Swing components are painted directly on canvases using Java code, except for components that are subclasses of **java.awt.Window** or **java.awt.Panel**, which must be drawn using native GUI on a specific platform. Swing components depend less on the target platform and use less of the native GUI resource. For this reason, Swing components that don't rely on native GUI are referred to as **lightweight components**, and AWT components are referred to as **heavyweight components**. To distinguish new Swing component classes from their AWT counterparts, the Swing GUI component classes are named with a prefixed **J**. Although AWT components are still supported in Java, it is better to learn to how program using Swing components, because the AWT user-interface components will eventually fade away.

6.3 The Java GUI API

The GUI API contains classes that can be classified into three groups: container classes, component classes, and helper classes.

6.3.1 . The GUI Container Classes

It is used to contain other components. An instance of Container can hold instances of Component. Container classes are GUI components that are used to contain other GUI components. Window, Panel, Applet, Frame, and Dialog are the container classes for AWT components. To work with Swing components, use Container, JFrame, JDialog, JApplet, and JPanel. It is summarized in the following table.

<i>Container Class</i>	<i>Description</i>
<code>java.awt.Container</code>	is used to group components. Frames, panels, and applets are its subclasses.
<code>javax.swing.JFrame</code>	is a window not contained inside another window. It is used to hold other Swing user-interface components in Java GUI applications.
<code>javax.swing.JPanel</code>	is an invisible container that holds user-interface components. Panels can be nested. You can place panels inside a container that includes a panel. <code>JPanel</code> is also often used as a canvas to draw graphics.
<code>javax.swing.JApplet</code>	is a subclass of <code>Applet</code> . You must extend <code>JApplet</code> to create a Swing-based Java applet.
<code>javax.swing.JDialog</code>	is a popup window or message box generally used as a temporary window to receive additional information from the user or to provide notification that an event has occurred.

The Java GUI API is shown graphically as follows.

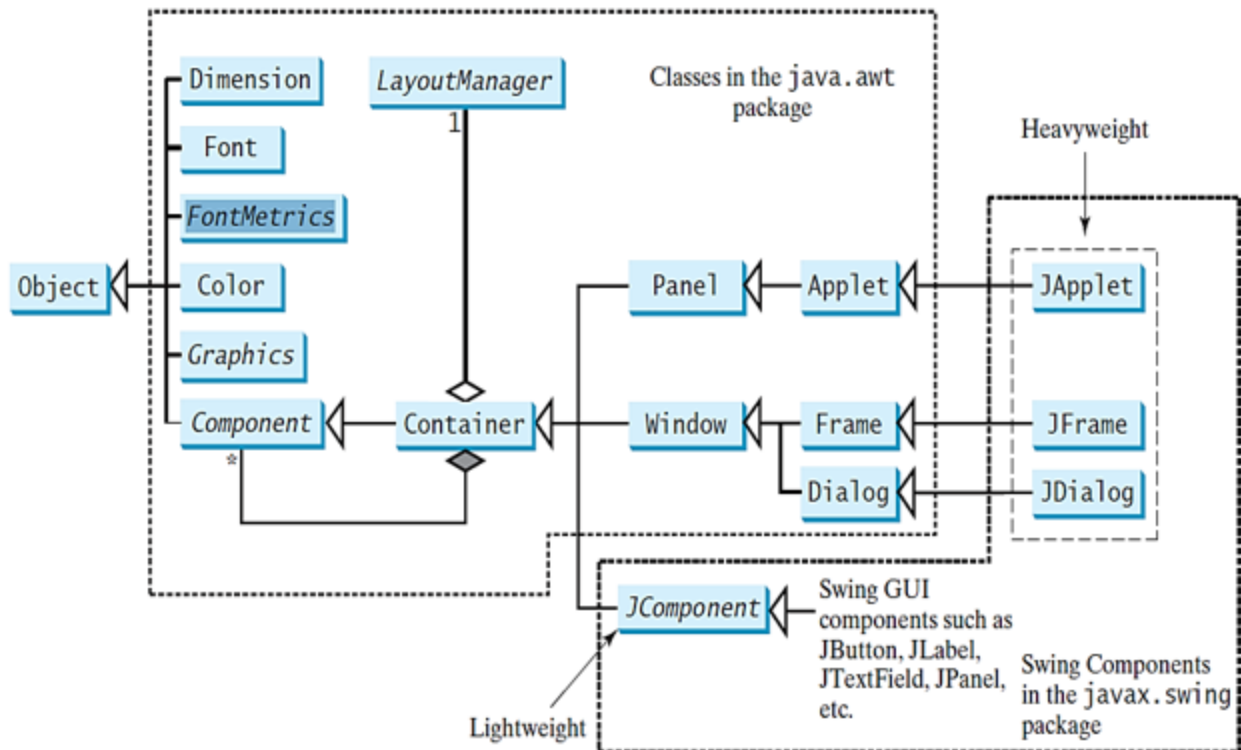
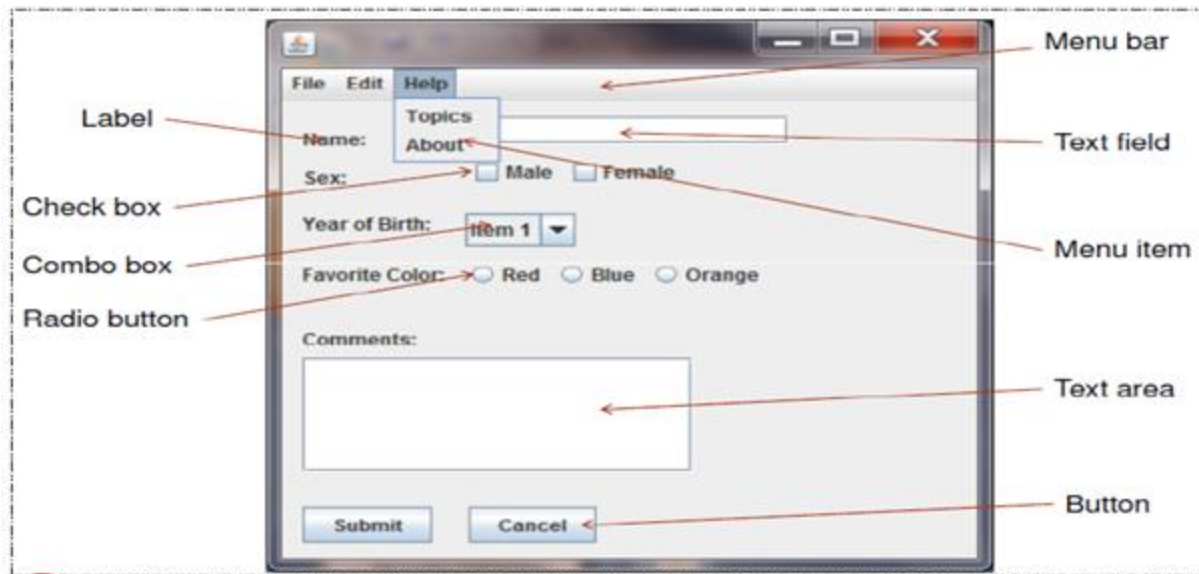


Fig. 6.1. Java GUI programming utilizes the classes shown in this hierarchical diagram.

The basics of the Container classes are:

A. Frame

It is one type of container which is used to create a user interface that holds the other components. Frame is the AWT format, while its Swing format is JFrame. When Frame is used, the package imported are **java.awt.Frame** and when JFrame is used **javax.swing.JFrame** package is imported. It is a graphical window that can be used to hold other components



Creating JFrame:

To create a frame, use the **JFrame** class.

Example:

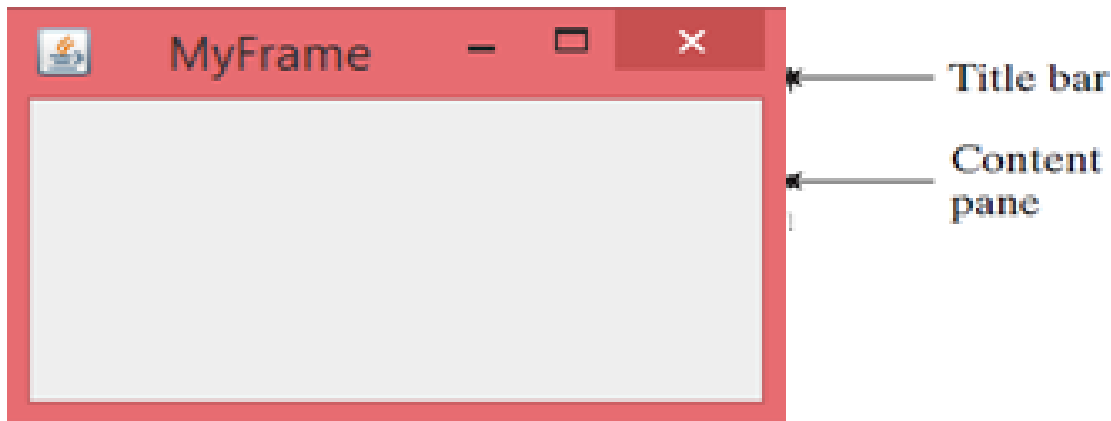
Create a Java GUI program using JFrame that display “MyFrame” as a title bar with an empty body.

```

1  package Chapter6;
2  import javax.swing.JFrame;
3  public class MyFrame {
4      public static void main(String[] args) {
5          JFrame frame = new JFrame("MyFrame"); //Create a frame
6          frame.setSize(400, 300); //Set the frame size
7          frame.setLocationRelativeTo(null); //Center a frame
8          frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
9          //Close upon exit.
10         frame.setVisible(true); //Display the frame
11     }
12 }

```

Output:



Description of the above sample code of frame:

javax.swing.JFrame	
<pre> +JFrame() +JFrame(title: String) +setSize(width: int, height: int): void +setLocation(x: int, y: int): void +setVisible(visible: boolean): void +setDefaultCloseOperation(mode: int): void +setLocationRelativeTo(c: Component): void +pack(): void </pre>	Creates a default frame with no title. Creates a frame with the specified title. Sets the size of the frame. Sets the upper-left-corner location of the frame. Sets true to display the frame. Specifies the operation when the frame is closed. Sets the location of the frame relative to the specified component. If the component is null, the frame is centered on the screen. Automatically sets the frame size to hold the components in the frame.

Fig. 6.2. JFrame is a top-level container to hold GUI components.

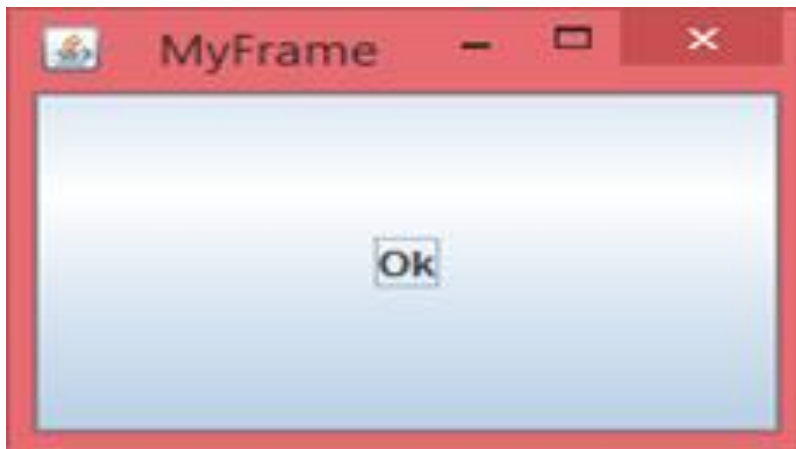
The frame is not displayed until the ***frame.setVisible(true)*** (line 10) method is invoked. The ***frame.setSize(400, 300)*** (line 6) specifies that the frame is **400** pixels wide and **300** pixels high. If the **setSize** method is not used, the frame will be sized to display just the title bar. Since the **setSize** and **setVisible** methods are both defined in the **Component** class, they are inherited by the **JFrame** class. Invoking **setLocationRelativeTo(null)** (line 7) centers the frame on the screen. Invoking **setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)** (line 8) tells the program to terminate when the frame is closed. If this statement is not used, the program does not terminate when the frame is closed. In that case, you have to stop the program by pressing Ctrl+C at the DOS prompt window in Windows or stop the process by using the kill command in Unix. If you run the program from an IDE such as Eclipse or NetBeans, you need to click the red Terminate button in the Console pane to stop the program.

Note:

- *A pixel is the smallest unit of space available for drawing on the screen. The resolution specifies the number of pixels per square inch. The more pixels the screen has, the higher the screen's resolution. The higher the resolution, the finer the detail you can see.*
- *You should invoke the **setSize(w, h)** method before invoking **setLocationRelativeTo(null)** to center the frame.*

The above created frame is called **empty frame**. If we add one component (JButton – between line code 5 & 6), using the **add(objectName) method**, the following lines of codes you will get the output is:

```
JButton button = new JButton("Ok");  
frame.add(button);
```

Output:

Each JFrame contains a content pane. A content pane is an instance of **java.awt.Container**. The GUI components such as JButton, JLabel, etc. are placed in the content pane in a JFrame. These components are placed into the content pane by invoking a frame's add method. This feature is called *content-pane delegation*. An object of JButton was created using **new JButton("OK")**, and this object was added to the content pane of the frame.

The **add(Component comp)** method defined in the Container class adds an instance of Component to the container. Since **JButton** is a subclass of Component, an instance of **JButton** is also an instance of Component. To remove a component from a container, use the remove method. The following statement removes the button from the container:

```
frameObject.remove(objectName);
```

Example:

```
frame.remove(button);
```

Note:

- *The default layout manager for the content pane places the button in the center.*

B. Panel

Panel act as sub-containers to group user-interface components. It is a simplest Container class. Its Swing version is **JPanel**. You can use **new JPanel()** to create a JPanel with a default FlowLayout manager or **new JPanel(LayoutManager)** to create a JPanel with the specified layout manager. Use the **add (Component comp)** method to add a component to the panel. the following code creates a panel and adds a button to it:

```
JPanel p = new JPanel();  
p.add(new JButton("OK"));
```

JPanels can be placed inside a JFrame or inside another JPanel. The following statement places

JPanel **p** into JFrame **f**:

```
f.add(p);
```

Example:

Write a Java GUI program using JPanel that add ten button and one text field within a JFrame.

```
1  package Chapter6;
2  import java.awt.*;
3  import javax.swing.*;
4  public class JPanel extends JFrame{
5      public JPanel() {
6          JPanel jp = new JPanel();
7          jp.setLayout(new GridLayout(3, 4));
8          for(int i = 1; i <= 7; i++){
9              jp.add(new JButton(" " + i));
10         }
11         jp.add(new JButton("Start"));
12         jp.add(new JButton("Stop"));
13
14         JPanel jp2 = new JPanel(new BorderLayout());
15         jp2.add(new JTextField("Displayed Food"), BorderLayout.NORTH);
16         jp2.add(jp, BorderLayout.CENTER);
17         add(jp2, BorderLayout.EAST);
18         add(new JButton("Displayed Time"), BorderLayout.CENTER);
19     }
20     public static void main(String[] args) {
21         JPanel panel = new JPanel();
22         panel.setTitle("Java Panel");
23         panel.setSize(350, 200);
24         panel.setLocationRelativeTo(null);
25         panel.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
26         panel.setVisible(true);
27     }
28 }
```

Output:



The `setLayout` method is defined in **java.awt.Container**. Since **JPanel** is a subclass of **Container**, you can use `setLayout` to set a new layout manager in the **JPanel**. To achieve the desired layout, the program uses **JPanel** `p1` of **GridLayout** to group the number buttons, the **Stop** button, and the **Start** button, and **JPanel** `p2` of **BorderLayout** to hold a text field in the north and `p1` in the center. The button representing the food is placed in the center of the **JFrame**, and `p2` is placed in the east of the **JFrame**.



Note

It is worthwhile to note that the **Container** class is the superclass for GUI component classes, such as **JButton**. Every GUI component is a container. In theory, you could use the `setLayout` method to set the layout in a button and add components into a button, because all the public methods in the **Container** class are inherited into **JButton**, but for practical reasons you should not use buttons as containers.

6.3.2 The GUI Component Classes

It is used for creating the user interface (UI). An instance of **Component** can be displayed on the **screen**. **Component** is the root class of all the user-interface classes including container classes, and **JComponent** is the root class of all the lightweight Swing components. Both **Component** and **JComponent** are abstract classes. The AWT **Component** are like **Button**, **Label**, **TextField**, **TextArea**, etc. and the Swing **Component** are like **JButton**, **JLabel**, **JTextField**, **JTextArea**, etc.

The basics of **Component** classes are:

A. Button

It is a component that triggers an action event when clicked. Its Swing part is JButton and it provides regular JButton, JToggleButton, JCheckBox button, and JRadioButton. The common features of these JButtons are defined in *javax.swing.AbstractButton*.

- **Icons, Pressed Icons, and Rollover Icons**

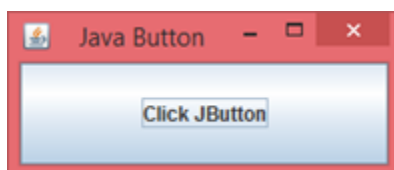
A regular JButton has a default icon, a pressed icon, and a rollover icon. Normally you use the default icon. The other icons are for special effects. A **pressed icon** is displayed when a JButton is pressed, and a **rollover icon** is displayed when the mouse is over the JButton but not pressed.

Example:

Create a Java GUI program using JButton with a name “Click JButton” that extends the properties of JFrame and displayed within a JFrame.

```
3 public class JButton extends JFrame{
4     public JButton(){
5         JButton jb = new JButton("Click JButton");
6         add(jb);
7     }
8     public static void main(String[] args) {
9         JButton button = new JButton();
10        button.setTitle("Java Button");
11        button.setSize(240, 100);
12        button.setLocationRelativeTo(null);
13        button.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
14        button.setVisible(true);
15    }
16 }
```

Output:



B. CheckBox

It is used to enable the user to toggle a choice ON or OFF. It is a sub-class of a `ToggleButton` which is a two-state button like a light switch. `ToggleButton` inherits `AbstractButton` and implements a `ToggleButton`. Their Swing are `JToggleButton` subclasses `JCheckBox`. `JCheckBox` inherits all the properties from ***AbstractButton***, such as text, icon, mnemonic, `verticalAlignment`, `horizontalAlignment`, `horizontalTextPosition`, `verticalTextPosition`, and `selected`, and provides several constructors to create check boxes

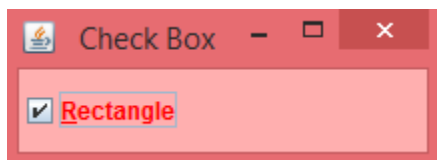
Example:

Create a Java GUI program using `JCheckBox` that display text `Rectangle` with a foreground red, background pink, mnemonic key 'R', and initially selected within a `JFrame`.

```
5 public class JavaCheckbox extends JFrame{
6     JCheckBox jcb = new JCheckBox("Rectangle", true);
7     public JavaCheckbox() {
8         jcb.setForeground(Color.RED);
9         jcb.setBackground(Color.pink);
10        jcb.setMnemonic('R');
11        add(jcb);
12    }
```

//Add main method here.

Output:



C. RadioButton

It is also known as option Button. It is used enable to choose a single item from a group of choices. In appearance `RadioButton` resemble `CheckBox`, but `CheckBox` display a **square** that is either checked or blank, whereas `RadioButton` display a **circle** that is either filled (if selected) or blank (if not selected). Its Swing is `JRadioButton` that inherits `AbstractButton` and provides several constructors to create `JRadioButton`.

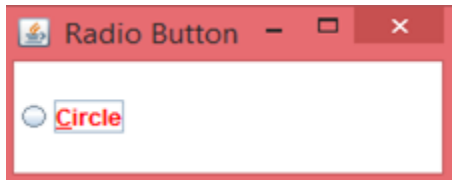
Example:

Create a Java GUI program using JRadioButton that display text Circle with red foreground, white background, mnemonic key C, and initially deselected within a JFrame.

```
5 public class JavaRadioButton extends JFrame{
6     JRadioButton jr = new JRadioButton("Circle",false);
7     public JavaRadioButton() {
8         jr.setForeground(Color.RED);
9         jr.setBackground(Color.WHITE);
10        jr.setMnemonic('C');
11        add(jr);
12    }
```

//Add a main method here.

Output:



When both a JCheckBox and JRadioButton is clicked (checked or unchecked), it fires an ItemEvent and then an ActionEvent. To see if a JCheckBox is selected, use the *isSelected()* method.

D. ComboBox

It is also known as a choice list or drop-down list, contains a list of items from which the user can choose. It is useful in limiting a user's range of choices and avoids the cumbersome validation of data input. Its Swing JComboBox.

Example:

Create a Java GUI program using JComboBox that provide a list of six geometry names with blue foreground, and pink background within the JFrame.

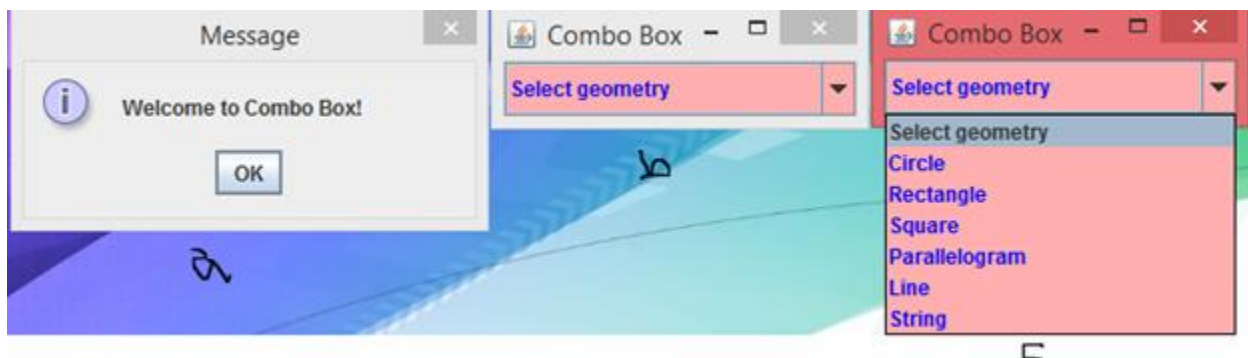
```

4 public class JavaComboBox extends JFrame{
5     private String[] geometry ={
6         "Select geometry", "Circle", "Rectangle", "Square",
7         "Parallelogram", "Line", "String"
8     };
9
10    private JComboBox jcb = new JComboBox(geometry);
11    public JavaComboBox() {
12        JOptionPane.showMessageDialog(null, "Welcome to Combo Box!");
13        add(jcb);
14        jcb.setForeground(Color.BLUE);
15        jcb.setBackground(Color.pink);
16    }

```

//Add a main method here.

Output:



E. List

It is a component that basically performs the same function as a ComboBox but enables the user to choose a single value or multiple values. The Swing **JList** is very versatile. The `selectionMode` is one of the **three** values (*SINGLE_SELECTION*, *SINGLE_INTERVAL_SELECTION*, *MULTIPLE_INTERVAL_SELECTION*) defined in *javax.swing.ListSelectionModel* that indicate whether a single item, single-interval item, or multiple-interval item can be selected.

- **Single selection** allows only one item to be selected.
- **Single-interval selection** allows multiple selections, but the selected items must be contiguous.
- **Multiple-interval selection** allows selections of multiple contiguous items without restrictions.

The default value is **MULTIPLE_INTERVAL_SELECTION**.

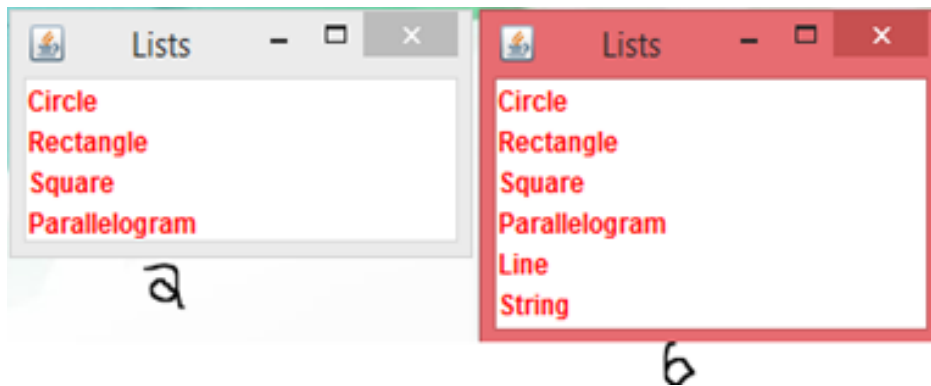
Example:

Create a Java GUI program using JList that display six geometry names with red foreground, white background, pink selection foreground, black selection background, and visible row count 4 within the JFrame.

```
4 public class JavaList extends JFrame{
    private String[] geometry ={
6         "Circle", "Rectangle", "Square",
7         "Parallelogram", "Line", "String"
8     };
    private JList jl = new JList(geometry);
10 public JavaList(){
    add(jl);
12     jl.setForeground(Color.RED);
13     jl.setBackground(Color.WHITE);
14     jl.setSelectionForeground(Color.PINK);
15     jl.setSelectionBackground(Color.BLACK);
16     jl.setVisibleRowCount(4);
17 }
```

//add main method here

Output:



F. **TextComponent**

i. **TextField**

It is used to enter or display a string in a single line. Its Swing is **JTextField** and is a subclass of **JTextComponent**. It inherits the properties of **JTextComponent**, which then inherits the properties of **JComponent**.

Example:

Create a Java GUI program that display “It is a JTextField” using JTextField with red foreground color and LEFT horizontal alignment within a JFrame.

```

6      public class JavaTextField extends JFrame{
7          private JTextField jtf = new JTextField("It is a JTextField");
8          public JavaTextField() {
9              jtf.setForeground(Color.RED);
10             jtf.setHorizontalAlignment(SwingConstants.LEFT);
11             add(jtf);
12         }
    
```

//Add main method here.

Output:



ii. **TextArea**

It is Component type that used to enter or display multiple lines of text. It is a several instance of **JTextField**. Its Swing is **JTextArea** that inherits the properties of **JTextComponent**, which contains the methods **getText**, **setText**, **isEditable**, and **setEditable**.

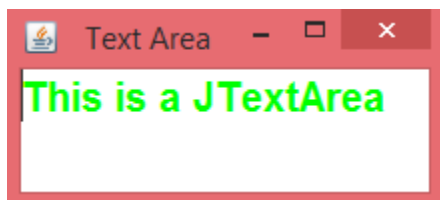
Example:

Create a Java GUI program that display “It is JTextArea” using a JTextArea with 5 rows and 20 columns, line-wrapped on words with GREEN foreground color, and Courier font, style BOLD, and 20 pixels size within a JFrame.

```
7 public class JavaTextArea extends JFrame{
8     JTextArea jta = new JTextArea("This is a JTextArea", 5, 20);
9     public JavaTextArea() {
10         jta.setLineWrap(true);
11         jta.setWrapStyleWord(true);
12         jta.setForeground(Color.GREEN);
13         jta.setFont(new Font("Courier", Font.BOLD, 20));
14         add(jta);
15     }
```

//Add main methods here.

Output:



iii. Label

A Label is a display area for an uneditable short text, an image, or both. It is often used to label other components. Its Swing is **JLabel** and it inherits all the properties from JComponent and has many properties similar to the ones in JButton, such as text, icon, horizontalAlignment, verticalAlignment, horizontalTextPosition, verticalTextPosition, and iconTextGap.

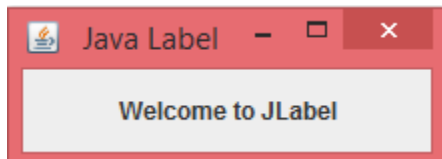
Example:

Create a Java GUI program that display “Welcome to JLabel” using JLabel in the center of JFrame.

```
4 public class JavaLabel extends JFrame{  
5     JLabel jb = new JLabel("Welcome to JLabel", SwingConstants.CENTER);  
6     public JavaLabel() {  
7         add(jb);  
8     }  
}
```

//Add main method here

Output:



G. ScrollBar

It is a component that enables the user to select from a range of values. It is shown as follows:

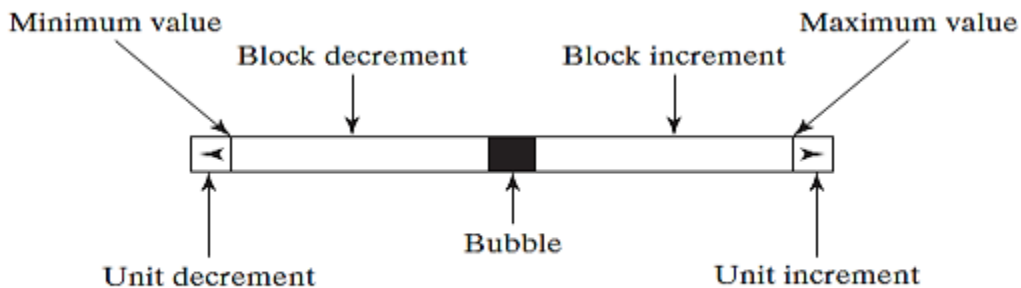


Fig. 6.6. A scroll bar represents a range of values graphically.

Normally, the user changes the value of the scroll bar by making a gesture with the mouse. The user can drag the **ScrollBar's** bubble up and down, or click in the **ScrollBar's** unit-increment or block-increment areas. Keyboard gestures can also be mapped to the **ScrollBar**. By convention, the Page Up and Page Down keys are equivalent to clicking in the ScrollBar's block-increment and block-decrement areas. Its Swing component is JScrollBar that inherits the properties

JComponent. It scrolls the bubble either vertical or horizontal. The horizontal JScrollBar is used to move the message to the left or the right, and the vertical is used to move it up and down.



Note

The width of the scroll bar's track corresponds to `maximum + visibleAmount`. When a scroll bar is set to its maximum value, the left side of the bubble is at `maximum`, and the right side is at `maximum + visibleAmount`.

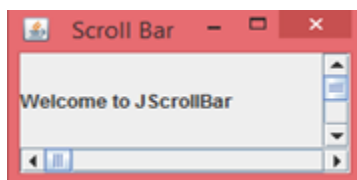
Example:

Write a java GUI program that display “Welcome to JScrollBar” using JScrollBar. Apply the horizontal and vertical scrolls to control a message displayed within JFrame.

```
4 public class JavaScrollBar extends JFrame{
    private JScrollBar jsbh = new JScrollBar(JScrollBar.HORIZONTAL);
    private JScrollBar jsbv = new JScrollBar(JScrollBar.VERTICAL);
    private JLabel jl = new JLabel("Welcome to JScrollBar");
    public JavaScrollBar(){
        setLayout(new BorderLayout());
        add(jl, BorderLayout.CENTER);
        add(jsbh, BorderLayout.SOUTH);
        add(jsbv, BorderLayout.EAST);
    }
13 }
```

//Add the main methods here.

Output:



6.3.3 The GUI Helper Classes

It is used to support GUI Components. **Example:** Graphics, Color, Font, FontMetrics, LayoutManager, Dimension etc. It is not sub-classes of Component. They are used to describe the properties of GUI components. It is summarized in the following table.

<i>Helper Class</i>	<i>Description</i>
<code>java.awt.Graphics</code>	is an abstract class that provides the methods for drawing strings, lines, and simple shapes.
<code>java.awt.Color</code>	deals with the colors of GUI components. For example, you can specify background or foreground colors in components like <code>JFrame</code> and <code>JPanel</code> , or you can specify colors of lines, shapes, and strings in drawings.
<code>java.awt.Font</code>	specifies fonts for the text and drawings on GUI components. For example, you can specify the font type (e.g., <code>SansSerif</code>), style (e.g., <code>bold</code>), and size (e.g., 24 points) for the text on a button.
<code>java.awt.FontMetrics</code>	is an abstract class used to get the properties of the fonts.
<code>java.awt.Dimension</code>	encapsulates the width and height of a component (in integer precision) in a single object.
<code>java.awt.LayoutManager</code>	specifies how components are arranged in a container.



Note

The helper classes are in the `java.awt` package. The Swing components do not replace all the classes in AWT, only the AWT GUI component classes (e.g., `Button`, `TextField`, `TextArea`). The AWT helper classes are still useful in GUI programming.

The basics of GUI helper classes are:

A. Color Class

The GUI components can be set by colors using the `java.awt.Color` package class. Colors are made of **red**, **green**, and **blue** components, each represented by an `int` value that describes its intensity, ranging from 0 (darkest shade) to 255 (lightest shade). This is known as the **RGB** model.

You can create a color using the following constructor:

```
public Color(int r, int g, int b);
```

in which `r`, `g`, and `b` specify a color by its red, green, and blue components.

Example:

```
Color color = new Color(128, 100, 100);
```

You can use the `setBackground(Color c)` and `setForeground(Color c)` methods defined in the `java.awt.Component` class to set a component's background and foreground colors.

Example: Setting the background and foreground of a button:

```
JButton jbtOK = new JButton("OK");  
  
jbtOK.setBackground(color);  
  
jbtOK.setForeground(new Color(100, 1, 1));
```

Alternatively, you can use one of the 13 standard colors (BLACK, BLUE, CYAN, DARK_GRAY, GRAY, GREEN, LIGHT_GRAY, MAGENTA, ORANGE, PINK, RED, WHITE, and YELLOW) defined as constants in **java.awt.Color**. The following code, for instance, sets the foreground color of a button to red:

```
jbtOK.setForeground(Color.RED);
```

B. Font Class

The font can be created using the **java.awt.Font** class and set fonts for the components using the `setFont` method in the `Component` class. The constructor for `Font` is:

```
public Font(String name, int style, int size);
```

You can choose a font name from `SansSerif`, `Serif`, `Monospaced`, `Dialog`, or `DialogInput`, choose a style from `Font.PLAIN` (0), `Font.BOLD` (1), `Font.ITALIC` (2), and `Font.BOLD + Font.ITALIC` (3), and specify a font size of any positive integer.

Example:

The following statements create two fonts and set one font to a button.

```
Font font1 = new Font("SansSerif", Font.BOLD, 16);  
  
Font font2 = new Font("Serif", Font.BOLD + Font.ITALIC, 12);  
  
JButton jbtOK = new JButton("OK");
```

```
jbtOK.setFont(font1);
```

C. ImageIcon Class

An icon is a fixed-size picture. It is small and used to decorate components. Images are normally stored in image files. Java currently supports three image formats: **GIF** (Graphics Interchange Format), **JPEG** (Joint Photographic Experts Group), and **PNG** (Portable Network Graphics). The image file names for these types end with .gif, .jpg, and .png, respectively. If you have a bitmap file or image files in other formats, you can use image-processing utilities to convert them into GIF, JPEG, or PNG format for use in Java. To display an image icon, first create an **ImageIcon** object using **new javax.swing.ImageIcon(filename)**.

Example:

The following statement creates an icon from an image file us.gif in the image directory under the current class path:

```
ImageIcon icon = new ImageIcon("image/us.gif");
```

“image/us.gif” is located in “c:\book\image\us.gif.” The back slash (\) is the Windows file path notation.



Tip

File names are not case sensitive in Windows but are case sensitive in Unix. To enable your programs to run on all platforms, name all the image files consistently, using lowercase.

An image icon can be displayed in a label or a button using **new JLabel(imageIcon)** or **new JButton(imageIcon)**.

Example:

Write a Java GUI program that applies ImageIcon, Font and Color on JLabel in a JFrame.

```

9   public class ColorFontImage extends JFrame{
10       ImageIcon imgIcon = new ImageIcon(
11           "C:\\Users\\AyuBest\\Pictures\\login.jpg");
12       Image icon = imgIcon.getImage().getScaledInstance(50, 50, 10);
13       JLabel jl = new JLabel("It GUI Helper Class.");
14       public ColorFontImage() {
15           imgIcon = new ImageIcon(icon);
16           setLayout(new GridLayout(1, 5));
17           add(new JLabel(imgIcon));
18           add(jl);
19           jl.setForeground(Color.BLUE);
20           jl.setFont(new Font("Sans_Serif", Font.ITALIC, 20));
21       }

```

Output:



D. Layout Manager

The Java GUI components are placed in containers, where they are arranged by the container's layout manager. A **Layout Manager** is created using a **LayoutManager** interface. Layout Managers are set in containers using the **setLayout(new LayoutManagerName)** method. It is implemented by all classes of layout managers.

Example:

The following statements to create an instance of **XLayout** and set it in a container:

```

LayoutManager layoutManager = new XLayout();
container.setLayout(layoutManager);

```

There are basically three types of Java Layout Manager. They are:

i. FlowLayout Manager

It is the simplest LayoutManager. The components are arranged in the container from *left to right* in the order in which they were added. When one row is filled, a new row is started. You can specify the way the components are aligned by using one of **three** constants:

- FlowLayout.RIGHT,
- FlowLayout.CENTER, or
- FlowLayout.LEFT.

You can also specify the gap between components in pixels. The class diagram for FlowLayout is shown as follows:

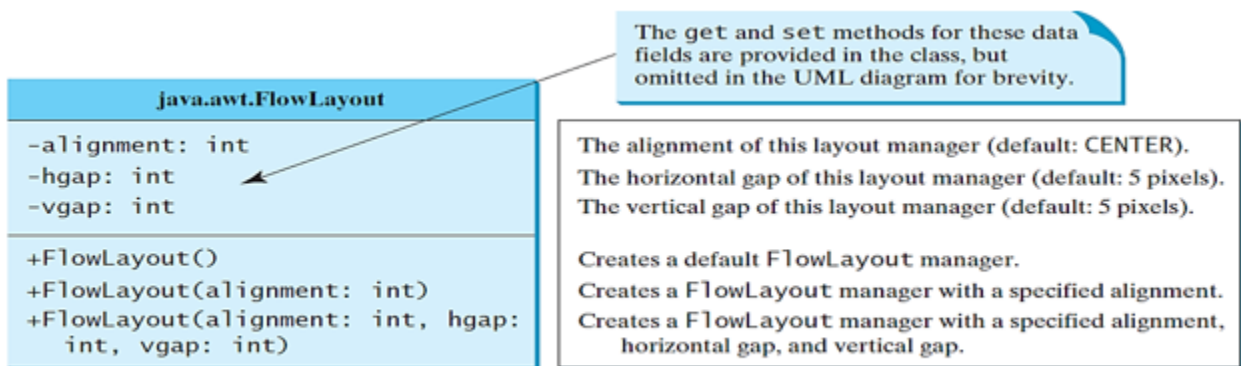


Fig. 6.2. FlowLayout lays out components row by row.

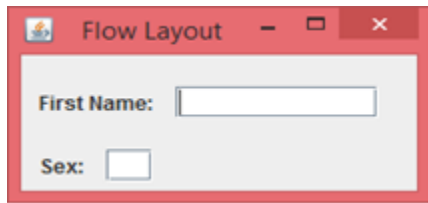
Example:

Write a Java GUI program that adds two labels using JLabel and two text fields using JTextField into to the JFrame using FlowLayout.

```
4 public class FlowLayoutManager extends JFrame{
5     public FlowLayoutManager() {
6         setLayout(new FlowLayout(FlowLayout.LEFT, 10, 20));
7         add(new JLabel("First Name: "));
8         add(new JTextField(10));
9         add(new JLabel("Sex: "));
10        add(new JTextField(2));
11    }
```

//Add the main methods here.

Output:



Using one style consistently makes programs easy to read. From now on, most of the GUI main classes will extend the **JFrame** class. The constructor of the main class constructs the user interface. The main method creates an instance of the main class and then displays the JFrame. The FlowLayout LayoutManager is used to place components in a JFrame. If you resize the frame, the components are automatically rearranged to fit.

An anonymous **FlowLayout** object:

```
setLayout(new FlowLayout(FlowLayout.LEFT, 10, 20));
```

which is equivalent to:

```
FlowLayout layout = new FlowLayout(FlowLayout.LEFT, 10, 20);  
setLayout(layout);
```



Caution

Do not forget to put the **new** operator before a layout manager class when setting a layout style—for example, **setLayout(new FlowLayout())**.

ii. GridLayout Manager

It arranges components in a grid (matrix) formation. The components are placed in the grid from left to right, starting with the first row, then the second, and so on, in the order in which they are added. The class diagram for GridLayout is shown as follows:

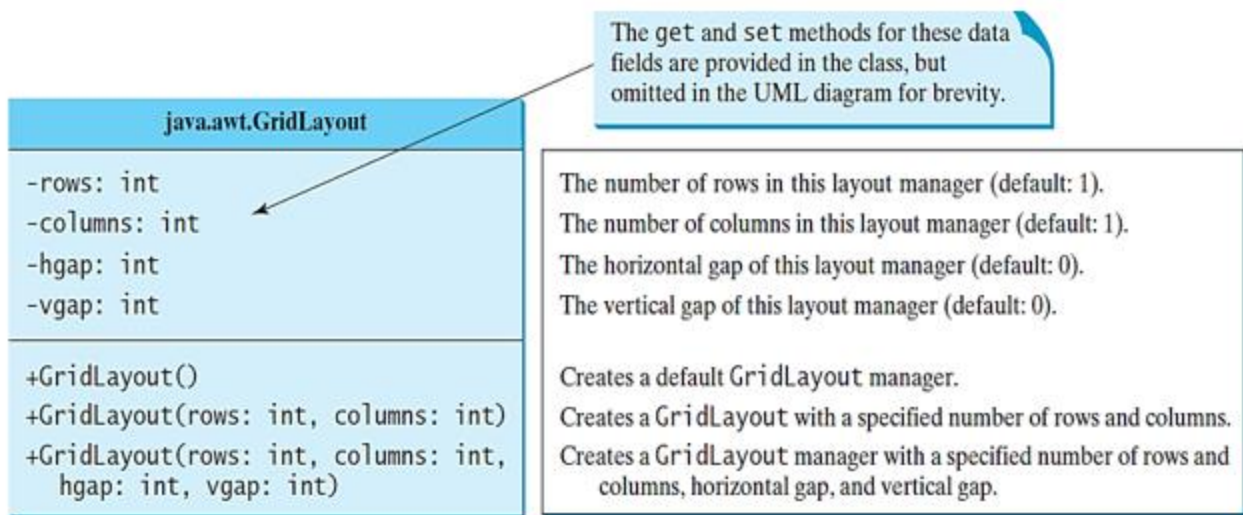


Fig. 6.3. GridLayout lays out components in equal-sized cells on a grid.

You can specify the number of rows and columns in the grid. The basic rule is as follows:

- The number of rows or the number of columns can be zero, but not both. If one is zero and the other is nonzero, the nonzero dimension is fixed, while the zero dimension is determined dynamically by the layout manager.
- If both the number of rows and the number of columns are nonzero, the number of rows is the dominating parameter; that is, the number of rows is fixed, and the layout manager dynamically calculates the number of columns.

Example:

Write a Java GUI program that adds two labels and two text fields to the JFrame using GridLayout.

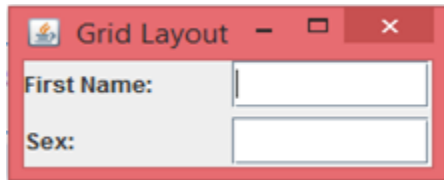
```

4 public class GridLayoutManager extends JFrame {
5     public GridLayoutManager() {
        setLayout(new GridLayout(2, 2, 5, 5));
        add(new JLabel("First Name: "));
        add(new JTextField(10));
        add(new JLabel("Sex: "));
        add(new JTextField(2));
11    }

```

//Add main methods statements here.

Output:



If you resize the JFrame, the layout of the labels remains unchanged (i.e., the number of rows and columns does not change, and the gaps doesn't change either). All components are given equal size in the container of GridLayout.

Note:

- The GridLayout manager divides the container into grids; then the components are added to fill in the cells row by row.



Note

In **FlowLayout** and **GridLayout**, the order in which the components are added to the container is important. It determines the location of the components in the container.

iii. BorderLayout Manager

It divides a container into five areas: **East**, **South**, **West**, **North**, and **Center**. Components are added to a BorderLayout by using **add(Component, index)**, where **index** is a constant **BorderLayout.EAST**, **BorderLayout.SOUTH**, **BorderLayout.WEST**, **BorderLayout.NORTH**, or **BorderLayout.CENTER**. The class diagram for BorderLayout is shown as follows:

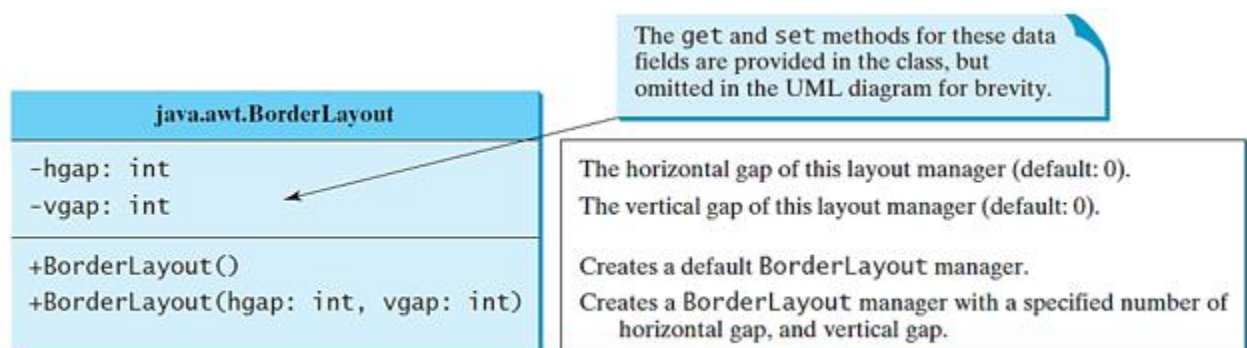


Fig. 6.4. BorderLayout lays out components in five areas.

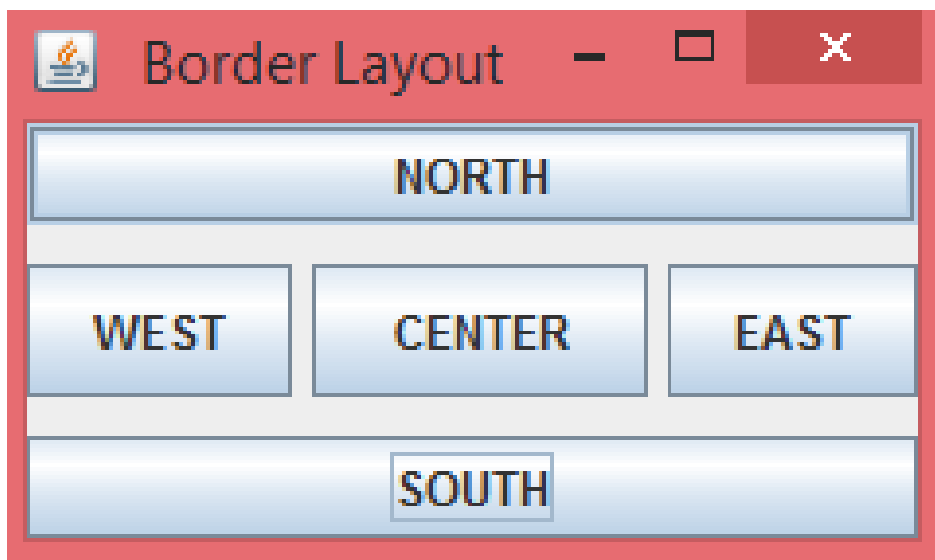
The components are laid out according to their preferred sizes and their placement in the container. The **North** and **South** components can stretch horizontally; the **East** and **West** components can stretch vertically; the **Center** component can stretch both horizontally and vertically to fill any empty space.

Example:

Write a Java GUI program using BorderLayout that adds five buttons into the JFrame as EAST, SOUTH, WEST, NORTH and CENTER.

```
5 public class BorderLayoutManager extends JFrame{
6     public BorderLayoutManager(){
            setLayout(new BorderLayout(5, 10)); //hgap, vgap
            add(new JButton("EAST"), BorderLayout.EAST);
            add(new JButton("SOUTH"), BorderLayout.SOUTH);
            add(new JButton("WEST"), BorderLayout.WEST);
            add(new JButton("NORTH"), BorderLayout.NORTH);
            add(new JButton("CENTER"), BorderLayout.CENTER);
13 }
```

Output:



Note:

- *BorderLayout* divides the container into five areas, each of which can hold a component. The **add()** method for *BorderLayout* is different from the one for *FlowLayout* and *GridLayout*. With *BorderLayout* you specify where to put the components.

Slider

JSlider is similar to *JScrollBar*, but *JSlider* has more properties and can appear in many forms. The following figure shows two sliders.



Fig. 6.7. The sliders move the message on a panel horizontally and vertically.

JSlider lets the user graphically select a value by sliding a knob within a bounded interval. The slider can show both major tick marks and minor tick marks between them. The number of pixels between the tick marks is controlled by the **majorTickSpacing** and **minorTickSpacing** properties. Sliders can be displayed horizontally or vertically, with or without ticks, and with or without labels.



Note

The values of a vertical scroll bar increase from top to bottom, but the values of a vertical slider decrease from top to bottom by default.



Note

All the properties listed in Figure 17.30 have the associated **get** and **set** methods, which are omitted for brevity. By convention, the **get** method for a Boolean property is named **is<PropertyName>()**. In the *JSlider* class, the **get** methods for **paintLabels**, **paintTicks**, **paintTrack**, and **inverted** are **getPaintLabels()**, **getPaintTicks()**, **getPaintTrack()**, and **getInverted()**, which violate the naming convention.

Example:

Writes a java GUI program that uses the sliders to control a message displayed on a panel.

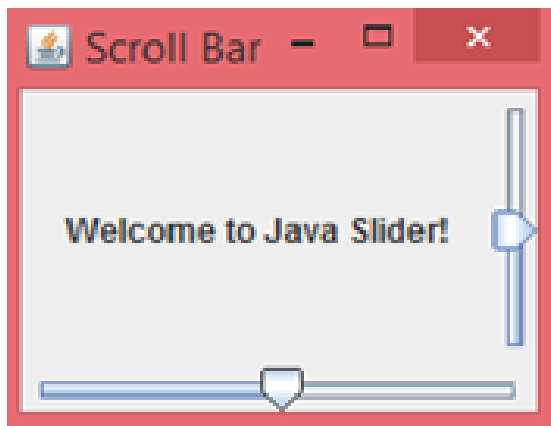
```

4  public class JavaSlider extends JFrame{
    private JSlider jsh = new JSlider(JSlider.HORIZONTAL);
    private JSlider jsv = new JSlider(JSlider.VERTICAL);
    private JLabel jl = new JLabel("Welcome to Java Slider!");
8  public JavaSlider(){
    setLayout(new BorderLayout());
10     jl.setHorizontalAlignment(SwingConstants.CENTER);
    add(jl, BorderLayout.CENTER);
    add(jsh, BorderLayout.SOUTH);
    add(jsv, BorderLayout.EAST);
14 }

```

//add main method here.

Output:



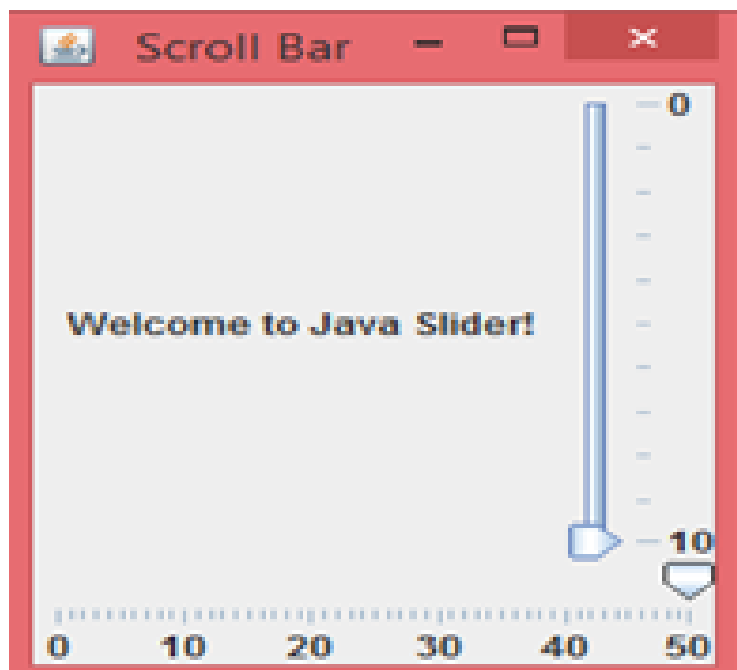
If you include the following in the constructor, the corresponding output is:

```

8      public JavaSlider() {
9          //add layout statements
10         jsh.setMaximum(50);
11         jsh.setPaintLabels(true);
12         jsh.setPaintTicks(true);
13         jsh.setMajorTickSpacing(10);
14         jsh.setMinorTickSpacing(1);
15         jsh.setPaintTrack(false);
16         jsv.setInverted(true);
17         jsv.setMaximum(10);
18         jsv.setPaintLabels(true);
19         jsv.setPaintTicks(true);
20         jsv.setMajorTickSpacing(10);
21         jsv.setMinorTickSpacing(1);
22     }

```

Output:



Menu

Menus make selection easier and are widely used in window applications. Its Swing is **JMenu**. Java provides five classes that implement menus: **JMenuBar**, **JMenu**, **JMenuItem**, **JCheckBoxMenuItem**, and **JRadioButtonMenuItem**.

JMenuBar is a top-level menu component used to hold the menus. A menu consists of menu items that the user can select (or toggle on or off). A menu item can be an instance of **JMenuItem**, **JCheckBoxMenuItem**, or **JRadioButtonMenuItem**. Menu items can be associated with icons, keyboard mnemonics, and keyboard accelerators. Menu items can be separated using separators.

Creating Menus

The sequence of implementing menus in Java is as follows:

1. Create a menu bar and associate it with a frame or an applet by using the **setJMenuBar** method. For example, the following code creates a frame and a menu bar, and sets the menu bar in the frame:

```
JFrame frame = new JFrame();
frame.setSize(300, 200);
frame.setVisible(true);
JMenuBar jmb = new JMenuBar();
frame.setJMenuBar(jmb); // Attach a menu bar to a frame
```

2. Create menus and associate them with the menu bar. You can use the following constructor to create a menu:

```
public JMenu(String label)
```

Here is an example of creating menus:

```
JMenu fileMenu = new JMenu("File");
JMenu helpMenu = new JMenu("Help");
```

This creates two menus labeled File and Help, as shown in Figure below (a). The menus will not be seen until they are added to an instance of **JMenuBar**, as follows:

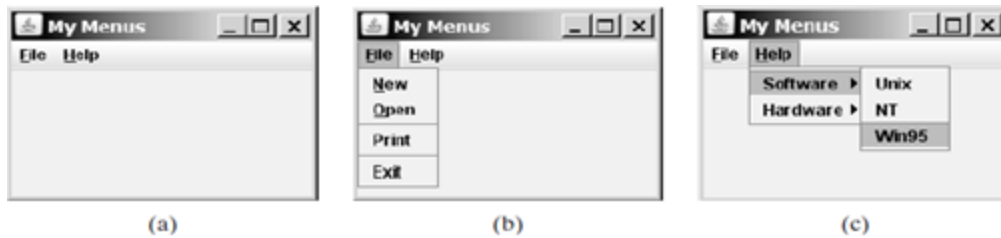


Fig. (a) The menu bar appears below the title bar on the frame. (b) Clicking a menu on the menu bar reveals the items under the menu. (c) Clicking a menu item reveals the submenu items under the menu item.

3. Create menu items and add them to the menus.

```
fileMenu.add(new JMenuItem("New"));
fileMenu.add(new JMenuItem("Open"));
fileMenu.addSeparator();
fileMenu.add(new JMenuItem("Print"));
fileMenu.addSeparator();
fileMenu.add(new JMenuItem("Exit"));
```

This code adds the menu items New, Open, a separator bar, Print, another separator bar, and Exit, in this order, to the File menu, as shown in Figure above (b). The ***addSeparator()*** method adds a separator bar in the menu.

6.4. Java Applets and Java Beans

When browsing the Web, often the graphical user interface and animation you see are developed by the use of Java. The programs used are called **Java applets**. In this chapter, you will learn how to write Java applets, explore the relationship between applets and the Web browser, and discover the similarities and differences between applications and applets. You will also learn how to create multimedia Java applications and applets with images and audio.

6.3.4 Overview of Graphics Class in Java

Graphical Coordinate Systems

To paint, you need to specify where to paint. Each component has its own coordinate system with the **origin (0, 0)** at the **upper-left corner**. The **x-coordinate** increases to the **right**, and the **y-coordinate** increases **downward**. Note that the Java coordinate system differs from the conventional coordinate system, as shown in figure below.

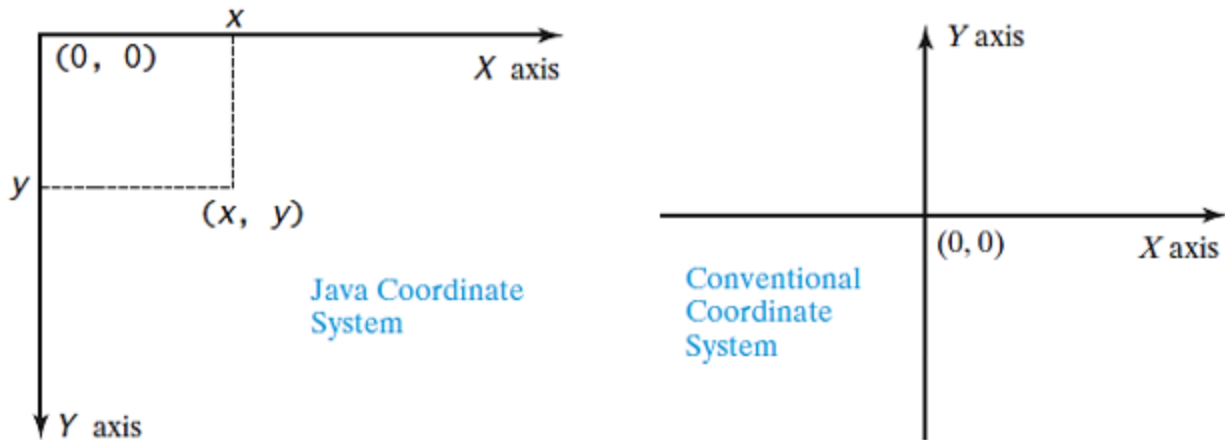


Fig. 8.8 - The Java coordinate system is measured in pixels, with (0, 0) at its upper left corner.

The location of the upper-left corner of a component **c1** (e.g., a button) inside its parent component **c2** (e.g., a panel) can be located using **c1.getX()** and **c1.getY()**. As shown in Figure below, $(x_1, y_1) = (c1.getX(), c1.getY())$, $(x_2, y_2) = (c2.getX(), c2.getY())$, and $(x_3, y_3) = (c3.getX(), c3.getY())$.

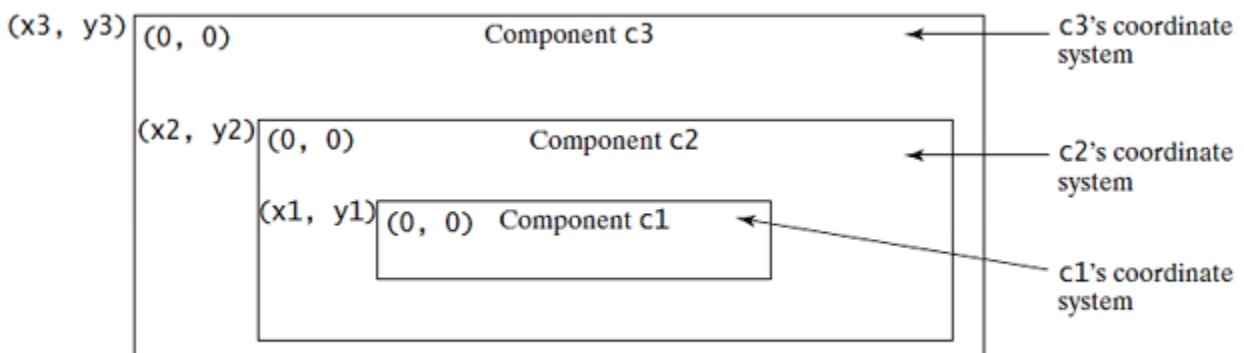


Fig. 8.9 - Each GUI component has its own coordinate system.

The **Graphics** class provides the methods for drawing strings, lines, rectangles, ovals, arcs, polygons, and polylines with the reusable GUI components, as shown in figure below. Think of a GUI component as a piece of paper and the Graphics object as a pencil or paintbrush. You can apply the methods in the Graphics class to draw things on a GUI component.

<i>java.awt.Graphics</i>	
+setColor(color: Color): void	Sets a new color for subsequent drawings.
+setFont(font: Font): void	Sets a new font for subsequent drawings.
+drawString(s: String, x: int, y: int): void	Draws a string starting at point (x, y).
+drawLine(x1: int, y1: int, x2: int, y2: int): void	Draws a line from (x1, y1) to (x2, y2).
+drawRect(x: int, y: int, w: int, h: int): void	Draws a rectangle with specified upper-left corner point at (x, y) and width w and height h.
+fillRect(x: int, y: int, w: int, h: int): void	Draws a filled rectangle with specified upper-left corner point at (x, y) and width w and height h.
+drawRoundRect(x: int, y: int, w: int, h: int, aw: int, ah: int): void	Draws a round-cornered rectangle with specified arc width aw and arc height ah.
+fillRoundRect(x: int, y: int, w: int, h: int, aw: int, ah: int): void	Draws a filled round-cornered rectangle with specified arc width aw and arc height ah.
+draw3DRect(x: int, y: int, w: int, h: int, raised: boolean): void	Draws a 3-D rectangle raised above the surface or sunk into the surface.
+fill3DRect(x: int, y: int, w: int, h: int, raised: boolean): void	Draws a filled 3-D rectangle raised above the surface or sunk into the surface.
+drawOval(x: int, y: int, w: int, h: int): void	Draws an oval bounded by the rectangle specified by the parameters x, y, w, and h.
+fillOval(x: int, y: int, w: int, h: int): void	Draws a filled oval bounded by the rectangle specified by the parameters x, y, w, and h.
+drawArc(x: int, y: int, w: int, h: int, startAngle: int, arcAngle: int): void	Draws an arc conceived as part of an oval bounded by the rectangle specified by the parameters x, y, w, and h.
+fillArc(x: int, y: int, w: int, h: int, startAngle: int, arcAngle: int): void	Draws a filled arc conceived as part of an oval bounded by the rectangle specified by the parameters x, y, w, and h.
+drawPolygon(xPoints: int[], yPoints: int[], nPoints: int): void	Draws a closed polygon defined by arrays of x- and y-coordinates. Each pair of (x[i], y[i])-coordinates is a point.
+fillPolygon(xPoints: int[], yPoints: int[], nPoints: int): void	Draws a filled polygon defined by arrays of x- and y-coordinates. Each pair of (x[i], y[i])-coordinates is a point.
+drawPolygon(g: Polygon): void	Draws a closed polygon defined by a Polygon object.
+fillPolygon(g: Polygon): void	Draws a filled polygon defined by a Polygon object.
+drawPolyline(xPoints: int[], yPoints: int[], nPoints: int): void	Draws a polyline defined by arrays of x- and y-coordinates. Each pair of (x[i], y[i])-coordinates is a point.

Fig. 8.10 - The Graphics class contains the methods for drawing strings and shapes.

The **Graphics** class – is an abstract class – provides a device-independent graphics interface for displaying figures and images on the screen on different platforms. Whenever a component (e.g., a button, a label, a panel) is displayed, the **JVM** automatically creates a **Graphics** object for the component on the native platform and passes this object to invoke the **paintComponent** method to display the drawings. The signature of the **paintComponent** method is as follows:

protected void **paintComponent** (Graphics g)

This method, defined in the **JComponent** class, is invoked whenever a component is first displayed or redisplayed.

In order to draw things on a component, you need to define a class that extends **JPanel** and overrides its **paintComponent** method to specify what to draw.

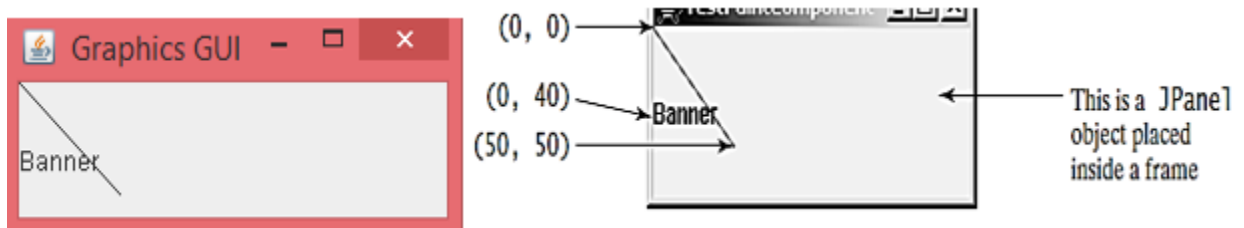
Example -1-:

Write a Java GUI program that draws a line and a string named as “Banner” using Graphics class.

```
5 public class GraphicsGUI extends JFrame {
6     public GraphicsGUI() {
7         add(new JPanel());
8     }
9     class JPanel extends JPanel {
10        @Override
11        protected void paintComponent(Graphics g) {
12            super.paintComponent(g);
13            g.drawLine(0, 0, 50, 50);
14            g.drawString("Banner", 0, 40);
15        }
16    }
```

//Add the main method here.

Output:



The **paintComponent** method is automatically invoked to paint graphics when the component is first displayed or whenever the component needs to be redisplayed. Invoking **super.paintComponent(g)** (line 12) invokes the **paintComponent** method defined in the superclass. This is necessary to ensure that the viewing area is cleared before a new drawing is displayed. Line 13 invokes the **drawLine** method to draw a line from (0, 0) to (50, 50). Line 14 invokes the **drawString** method to draw a string.

All the drawing methods have parameters that specify the locations of the subjects to be drawn. All measurements in Java are made in pixels. The string “**Banner**” is drawn at location (0, 40). The JVM invokes **paintComponent** to draw things on a component. The user should never invoke **paintComponent** directly. For this reason, the protected visibility is sufficient for **paintComponent**.

Panels are invisible and are used as small containers that group components to achieve a desired layout. Another important use of **JPanel** is for drawing. You can draw things on any Swing GUI component, but normally you should use a **JPanel** as a canvas upon which to draw things.

6.3.5 Overview of Java Applets

The **Applet** class provides the essential framework that enables applets to be run from a Web browser. Every applet is a subclass of **java.applet.Applet**. The Applet class is an AWT class and is not designed to work with Swing components. To use **Swing** components in Java applets, you need to create a Java applet that extends **javax.swing.JApplet**, which is a subclass of **java.applet.Applet**. Every Java GUI program you have developed can be converted into an applet by replacing **JFrame** with **JApplet** and deleting the main method.

Suppose that, the following (a) shows a Java GUI application program, which can be converted into a Java applet as in the (b), shows their basic difference.

```
import javax.swing.*;

public class DisplayLabel extends JFrame {
    public DisplayLabel() {
        add(new JLabel("Great!", JLabel.CENTER));
    }

    public static void main(String[] args) {
        JFrame frame = new DisplayLabel();
        frame.setTitle("DisplayLabel");
        frame.setSize(200, 100);
        frame.setLocationRelativeTo(null);
        frame.setDefaultCloseOperation(
            JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

(a) GUI application

```
import javax.swing.*;

public class DisplayLabel extends JApplet {
    public DisplayLabel() {
        add(new JLabel("Great!", JLabel.CENTER));
    }

    public static void main(String[] args) {
    JFrame frame = new DisplayLabel();
    frame.setTitle("DisplayLabel");
    frame.setSize(200, 100);
    frame.setLocationRelativeTo(null);
    frame.setDefaultCloseOperation(
        JFrame.EXIT_ON_CLOSE);
    frame.setVisible(true);
}
}
```

(b) Applet

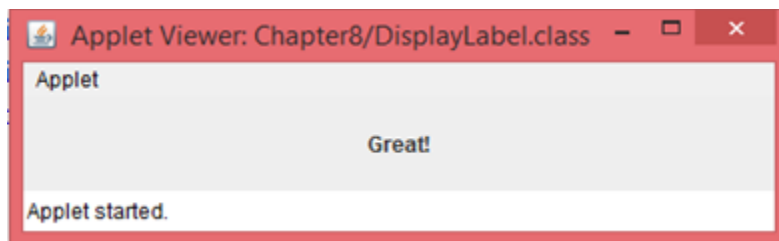
Like **JFrame**, **JApplet** is a container that can contain other GUI components.

Example -2- :

Write a Java Program using JApplet that display “Great” message.

```
2  import javax.swing.JApplet;
3  import javax.swing.JLabel;
4  public class DisplayLabel extends JApplet {
5      public DisplayLabel() {
6          add(new JLabel("Great!", JLabel.CENTER));
7      }
8  }
```

Output:



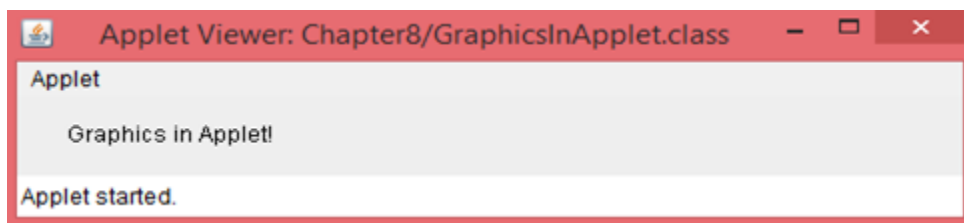
6.4.2.1. Displaying Graphics in Applet

Example -3-:

Write an JApplet program that contain graphics that display a message “Graphics in Applet”.

```
4  public class GraphicsInApplet extends JApplet {
5      public void paint(Graphics g) {
6          super.paint(g);
7          g.drawString("Graphics in Applet!", 25, 25);
8      }
9  }
```

Output:



6.4.2.2. Java Application vs Java Applet

So far, you have used only Java applications. Everything you have learned about writing applications, however, applies also to writing applets. Applications and applets share many common programming features, although they differ slightly in some aspects. Every Java application must have a **main** method, which is invoked by the Java interpreter. Java applets, on the other hand, do not need a **main** method. They run in the *Web browser environment*. Because applets are invoked from a Web page, Java provides special features that enable applets to run from a Web browser.

6.3.5.1 The HTML File and the <applet> Tag

To run an applet from a browser, you need to create an HTML file with the **<applet>** tag. HTML is a markup language that presents static documents on the Web. It uses tags to instruct the Web browser how to render a Web page and contains a tag called **<applet>** that incorporates applets into a Web page.

A **tag** is an instruction to the Web browser. The browser interprets the tag and decides how to display or otherwise treat the subsequent contents of the HTML document. **Tags** are enclosed inside **brackets**. The first word in a tag, called the **tag name**, describes **tag functions**. A tag can have additional attributes, sometimes with values after an equal's sign, which further define the tag's action. For example, in the preceding HTML file, **<applet>** is the tag name, and code, width, and height are attributes. The width and height attributes specify the rectangular viewing area of the applet.

Most tags have a **start tag** and a corresponding **end tag**. The **tag** has a specific effect on the region between the start tag and the end tag.

Example:

- **<applet...>...</applet>** tells the browser to display an applet. An end tag is always the start tag's name preceded by a slash.

An **HTML** document begins with the **<html>** tag, which declares that the document is written in **HTML**. Each document has **two** parts, a **head** and a **body**, defined by **<head>** and **<body>**

tags, respectively. The head part contains the document title, including the <title> tag and other information the browser can use when rendering the document, and the body part holds the actual contents of the document. The header is optional. The complete syntax of the <applet> tag is as follows:

```
<applet
  [codebase = applet_url]
  code = classfilename.class
  width = applet_viewing_width_in_pixels
  height = applet_viewing_height_in_pixels
  [archive = archivefile]
  [vspace = vertical_margin]
  [hspace = horizontal_margin]
  [align = applet_alignment]
  [alt = alternative_text]
>
```

<param> tag

```
<param name = param_name1 value = param_value1>
<param name = param_name2 value = param_value2>
...
<param name = param_name3 value = param_value3>
</applet>
```

The **code**, **width**, and **height** attributes are required; all the others are optional.

- **codebase** specifies a base where your classes are **loaded**. If this attribute is not used, the Web browser loads the applet from the directory in which the HTML page is located. If your applet is located in a different directory from the HTML page, you must specify the `applet_url` for the browser to load the applet. This attribute enables you to load the class from anywhere on the Internet. The classes used by the applet are dynamically loaded when needed.
- **archive** instructs the browser to load an archive file that contains all the class files needed to run the applet. Archiving allows the Web browser to load all the classes from a single compressed file at one time, thus reducing loading time and improving performance.
- **vspace** and **hspace** specify the size, in pixels, of the blank margin to pad around the applet vertically and horizontally.
- **align** specifies how the applet will be aligned in the browser. One of nine values is used: left, right, top, texttop, middle, absmiddle, baseline, bottom, or absbottom.
- **alt** specifies the text to be displayed in case the browser cannot run Java.

Example -4-:

Write an HTML file to display the JApplet by invoking the Example -2- above class. The html file is saved as Applet.html as follows.

```
1 <HTML>
2 <title> Hello Applet </title>
3 <applet code = "DisplayLabel.class" width = 300 height = 150>
4 </applet>
```

There are two ways of viewing the output of the HTML file that contains applet. They are

6.4.2.4. Viewing Applets from a Web Browser

To display an applet from a Web browser, open the applet's HTML file (e.g., Applet.html). To make your applet accessible on the Web, you need to store the DisplayLabel.class and Applet.html on a **Web server**, as shown below. You can view the applet from an appropriate URL.



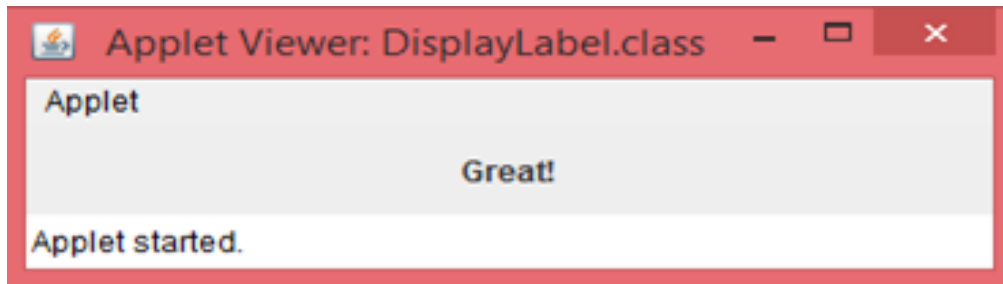
Fig. 8.4 - A Web browser requests an HTML page from a Web server.

6.3.5.2 Viewing Applets using the Applet Viewer Utility

The applet viewer utility used to view an HTML file that contains applet by invoking DOS is called **appletviewer**. The appletviewer functions as a **browser**. It is convenient for testing applets during development without launching a web-browser. It is shown as follows.

```
Command Prompt - appletviewer Applet.html
C:\OOP>javac DisplayLabel.java
C:\OOP>appletviewer Applet.html
```

Output:



6.3.5.3 Applet Security Restrictions

Java uses the so-called “**sandbox security model**” for executing applets to prevent destructive programs from damaging the system on which the browser is running. Applets are not allowed to use resources outside the “**sandbox**”. Specifically, the sandbox restricts the following activities:

- Applets are not allowed to read from, or write to, the file system of the computer. Otherwise, they could damage the files and spread viruses.
- Applets are not allowed to run programs on the browser’s computer. Otherwise, they might call destructive local programs and damage the local system on the user’s computer.
- Applets are not allowed to establish connections between the user’s computer and any other computer, except for the server where the applets are stored. This restriction prevents the applet from connecting the user’s computer to another computer without the user’s knowledge.

6.3.6 Applet Life-Cycle Methods

Applets are actually run from the applet container, which is a plug-in of a Web browser. The Applet class contains the **init()**, **start()**, **stop()**, and **destroy()** methods, known as the **applet life-cycle methods**. These methods are called by the **applet container** to control the execution of an applet. They are implemented with an empty body in the Applet class. So, they do nothing by default. You may override them in a subclass of Applet to perform desired operations. The following figure shows how the applet container calls these methods.

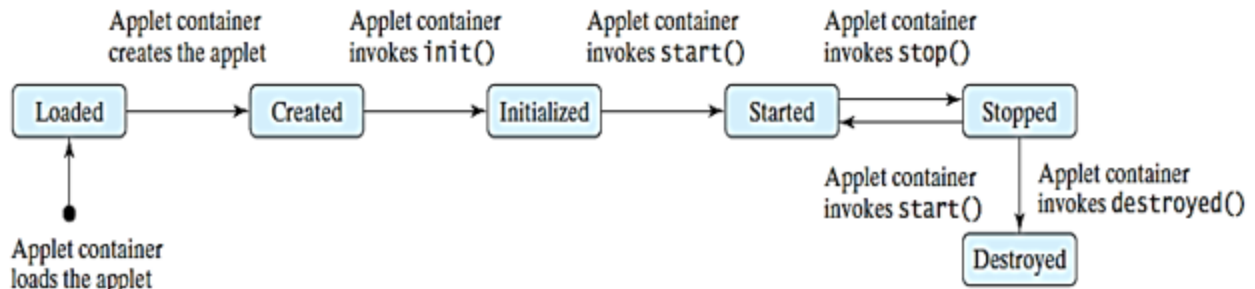


Fig. 6.5 - The applet container uses the init, start, stop, and destroy methods to control the applet.

6.3.6.1 The init Method – init()

It is invoked after the applet is created. If a subclass of **Applet** has an initialization to perform, it should override this method. The functions usually implemented in this method include getting string parameter values from the `<applet>` tag in the HTML page.

6.3.6.2 The start Method – start ()

It is invoked after the init method. It is also called when the user returns to the Web page containing the applet after surfing other pages. A subclass of **Applet** overrides this method if it has any operation that needs to be performed whenever the Web page containing the applet is visited. An applet with animation, for example, might start the timer to resume animation.

6.3.6.3 The stop Method – stop ()

It is invoked when the user leaves the page. A subclass of **Applet** overrides this method if it has any operation to be performed each time the Web page containing the applet is no longer visible. An applet with animation, for example, might stop the timer to pause animation.

6.3.6.4 The destroy Method – destroy ()

It is invoked when the browser exits normally to inform the applet that it is no longer needed and should release any resources it has allocated. The stop method is always called before the destroy method. A subclass of **Applet** overrides this method if it has any operation to be performed

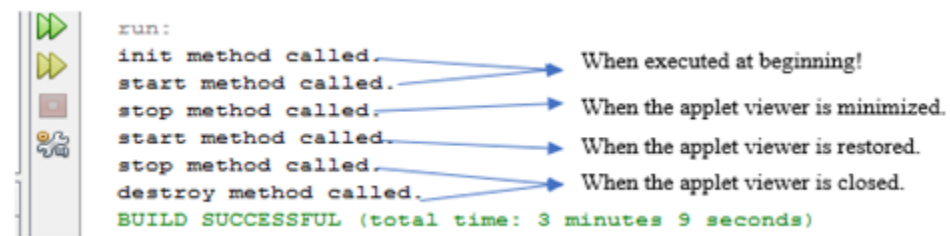
before it is destroyed. Usually, you won't need to override this method unless you wish to release specific resources that the applet created.

Example – 5-:

Write a Java program that implement the applet life cycle.

```
2  import javax.swing.JApplet;
3  public class AppletLifeCycle extends JApplet {
4      public void init() {
5          System.out.println("init method called.");
6      }
7      public void start() {
8          System.out.println("start method called.");
9      }
10     public void stop() {
11         System.out.println("stop method called.");
12     }
13     public void destroy() {
14         System.out.println("destroy method called.");
15     }
16 }
```

Output – Using NetBeans



```
run:
init method called.
start method called.
stop method called.
start method called.
stop method called.
destroy method called.
BUILD SUCCESSFUL (total time: 3 minutes 9 seconds)
```

When executed at beginning!

When the applet viewer is minimized.

When the applet viewer is restored.

When the applet viewer is closed.

Output – Using command prompt. (Write an HTML file with a name “Cycle.html or write in comment-wise in the code).

```
Command Prompt

C:\OOP>javac AppletLifeCycle.java
C:\OOP>appletviewer AppletLifeCycle.java
init method called.
start method called.
stop method called.
start method called.
stop method called.
start method called.
stop method called.
destroy method called.

C:\OOP>appletviewer Cycle.html
init method called.
start method called.
stop method called.
start method called.
stop method called.
destroy method called.

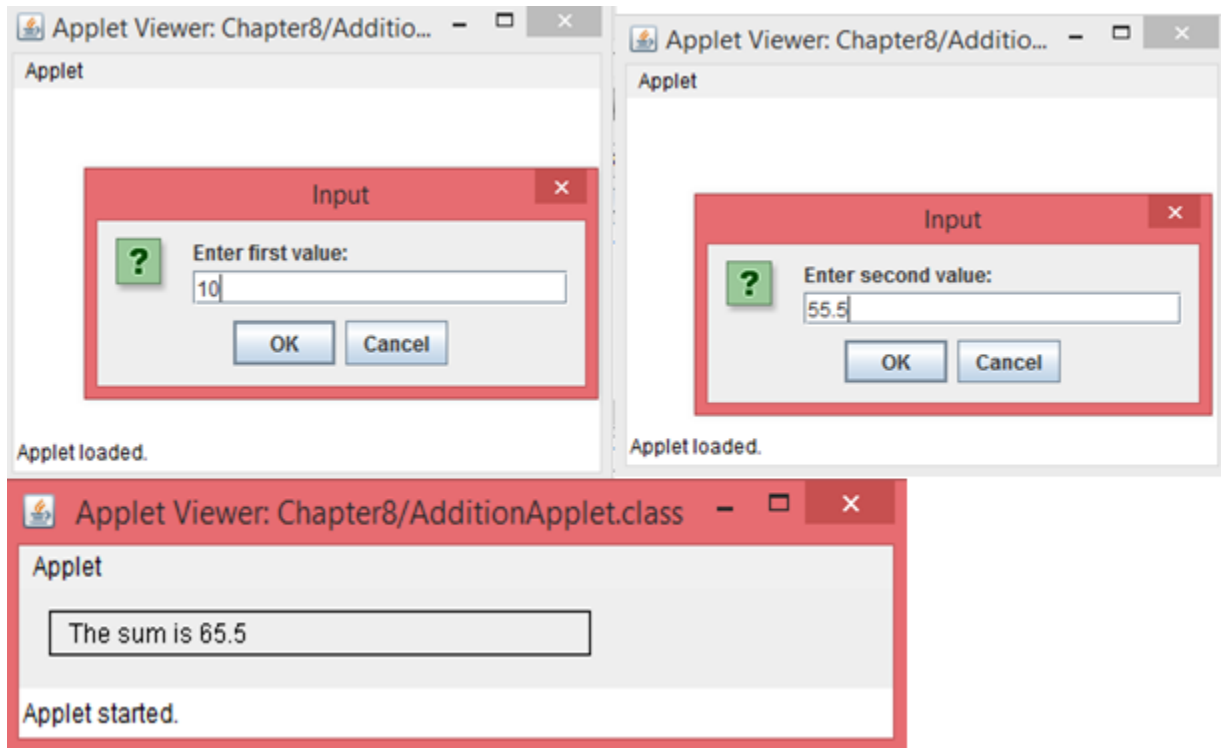
C:\OOP>
```

Example -6-:

Computes the sum of two values entered into input dialogs by the user and displays the result by drawing a String inside a rectangle on the applet. The sum is stored in an instance variable of the **Addition Applet** class, so it can be used in both the init method and the paint method.

```
2  import java.awt.Graphics;
3  import javax.swing.JApplet;
4  import javax.swing.JOptionPane;
5  public class AdditionApplet extends JApplet{
6      private double sum;
7      public void init(){
8          //obtain first number from user
9          String firstNumber = JOptionPane.showInputDialog("Enter first value: ");
10         // obtain second number from user
11         String secondNumber = JOptionPane.showInputDialog("Enter second value: ");
12         // convert numbers from type String to type double
13         double number1 = Double.parseDouble( firstNumber );
14         double number2 = Double.parseDouble( secondNumber );
15         sum = number1 + number2; // add numbers
16     }
17     public void paint( Graphics g ){
18         super.paint(g);
19         g.drawRect( 15, 10, 270, 20 );
20         g.drawString( "The sum is " + sum, 25, 25 );
21     }
22 }
```

Output:



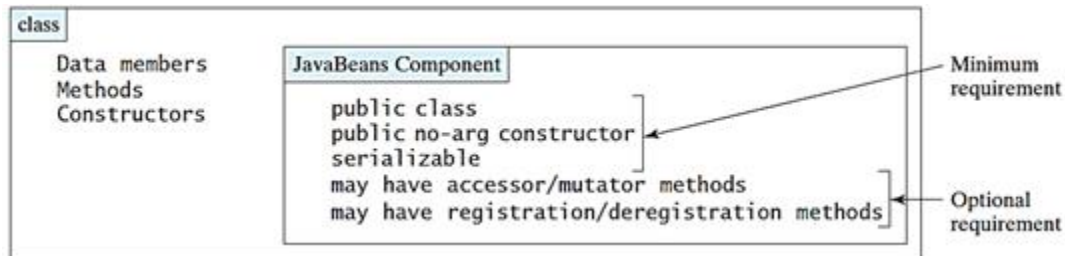
6.4 Java Beans

Every Java user interface class is a **JavaBeans** component. **JavaBeans** is a software component architecture that extends the power of the Java language by enabling well-formed objects to be manipulated visually at design time in a pure Java builder tool, such as NetBeans and Eclipse. Such well-formed objects are referred to as **JavaBeans** or simply **beans**. The classes that define the beans, referred to as JavaBeans components or bean components conform to the JavaBeans component model with the following requirements:

1. A bean must be a public class.
2. A bean must have a public no-arg constructor.
3. It is an extension of **java.awt.Component** which must implement the **java.io.Serializable** interface to ensure a persistent state.

4. A bean usually has properties with correctly constructed public accessor (get) methods and mutator (set) methods that enable the properties to be seen and updated visually by a builder tool.

The first three requirements must be observed, and therefore are referred to as minimum Java Beans component requirements. The last (4) requirement depends on implementations. It is possible to write a bean component without get/set methods.



6.4.1 Bean Properties

Properties are discrete, named attributes of a Java bean that can affect its appearance or behavior. They are often data fields of a bean. For example, the JButton component has a property named text that represents the text to be displayed on the button. Private data fields are often used to hide specific implementations from the user and prevent the user from accidentally corrupting the properties. Accessor (getter) and mutator (setter) methods are provided instead to let the user read and write the properties. There are basically two bean properties in Java. They are

6.4.1.1 Property-Naming Patterns

The bean property-naming pattern is a convention of the JavaBeans component model that simplifies the bean developer's task of presenting properties. A property can be a primitive data type or an object type. The property type dictates the signature of the accessor and mutator methods.

- The accessor method is named **get<PropertyName>()**, which takes no parameters and returns a primitive type value or an object of a type identical to the property type.

Example:

```
public String getMessage()  
public int getXCoordinate()  
public int getYCoordinate()
```

For a property of boolean type, the accessor method should be named **is<PropertyName>()**, which returns a boolean value.

```
public boolean isCentered()
```

Generally, the syntax for getter methods:

- It should be public in nature.
- The return type should not be void i.e. according to our requirement we have to give return-type.
- It should be prefixed with get.
- It should not take any argument.
- The mutator method should be named **set<PropertyName> (dataType p)**, which takes a single parameter identical to the property type and returns **void**.

Example:

```
public void setMessage(String s)  
public void setXCoordinate(int x)  
public void setYCoordinate(int y)  
public void setCentered(boolean centered)
```

Generally, syntax for setter methods:

- It should be public in nature.
- The return-type should be **void**.
- The setter method should be prefixed with set.
- It should take some argument i.e. it should not be no-arg method.



Note

You may have multiple get and set methods, but there must be one get or set method with a signature conforming to the naming patterns.

6.4.1.2 Properties and Data Fields

Properties describe the state of the bean. Naturally, data fields are used to store properties. However, a bean property is not necessarily a data field.



Note

A property may be *read-only* with a `get` method but no `set` method, or *write-only* with a `set` method but no `get` method.

6.4.2 Uses of Java Beans

- A Bean obtains all the benefits of Java's "write-once, run-anywhere" paradigm.
- A Bean may be designed to operate correctly in different locales, which makes it useful in global markets.

Example:

Write a JavaBean program that contain ID of type int and Type of type string geometry.

```
2 import java.io.Serializable;
3 import java.util.Scanner;
4 public class JavaBeanGeometry implements Serializable{
5     private int ID;
6     private String Type;
7     public JavaBeanGeometry() {
8     }
9     public void setID(int id){
10         this.ID = id;
11     }
12     public int getID(){
13         return ID;
14     }
15     public void setType(String type){
16         this.Type = type;
17     }
18     public String getType(){
19         return Type;
20     }
21 }
```

Output:

```
run:
Enter ID: 100
Enter type: Circle
ID      Type
-----
100     Circle
BUILD SUCCESSFUL (total time: 8 seconds)
```

6.5 Chapter Review Questions

1. What is GUI in Java?
2. What is the difference between Java awt and Java Swing?
3. Fill in the blanks in each of the following statements:
 1. a. The JTextField class directly extends class _____.
 2. b. Container method _____ attaches a GUI component to a container.
 3. c. Method _____ is called when a mouse button is released (without moving the mouse).
 4. d. The _____ class is used to create a group of JRadioButton
4. What is a layout manager and what are different types of layout managers available in Java Swing?
5. Why swing is not thread safe?
6. What Is an Event and Event handler in Swing?
7. What is the purpose of the enableEvents() method?
8. What is the difference between a Choice and a List?
9. What interface is extended by AWT event listeners?
10. What is a layout manager?
11. Which Component subclass is used for drawing and painting?
12. What is the difference between a Scrollbar and a ScrollPane?
13. Write a program to check the font class method as follows: Create a font TimesRoman bold and Italic size 12. In this object use the font methods. Then display the attributes of the font.
14. Write a program that displays the menu bar and when you click the options it has to display a dialog box stating which option has been clicked.
15. Write a program to create a dialogbox and menu.
16. Write a program to create a border layout control.
17. Write a program to create a padding layout control.
18. Write a program to give the example for button control.
19. Write a program that will display check boxes and option buttons they are numbered from 1 to. Use a textbox to display the number those corresponding boxes or button checked.
20. Write a java GUI program to create a simple calculator.
21. Write a program that displays the x and y position of the cursor movement using Mouse.
22. Write a program that displays the x and y position of the cursor movement using Keyboard.

23. Write a program to create a analog clock using GUI.
24. Write a java program to simulate a traffic light. The program lets the user select one of the three lights: red, yellow or green. On selecting a button, an appropriate message with "Stop" or "Ready" or " Go" should appear above the buttons selected color. Initially, there is no message shown.
25. Design a scientific calculator using event driven programming paradigm of Java.
26. Write a java program that works as a simple calculator. Use a Grid Layout to arrange Buttons for digits and for the + – * % operations. Add a text field to display the result.
27. Write a Java program for display the exception in a message dialog box.
28. What Is An Applet? Should Applets Have Constructors?
29. What Is The Order Of Method Invocation In An Applet?
30. How Do Applets Differ From Applications?
31. Can We Pass Parameters To An Applet From Html Page To An Applet? How?
32. How Will You Communicate Between Two Applets?
33. How Do You Communicate In Between Applets And Servlets?
34. What Is The Base Class For All Swing Components?
35. What Are The Attributes Of Applet Tags?
36. Is every applet an instance of java.applet.Applet? Is every applet an instance of javax.swing.JApplet?
37. Describe the init (), start(), stop(), and destroy() methods in the Applet class.
38. How do you add components to a JApplet? What is the default layout manager of the content pane of JApplet?
39. Describe the <applet> HTML tag. How do you pass parameters to an applet?
40. Where is the getParameter method defined?
41. What are the differences between applications and applets? How do you run an application, and how do you run an applet? Is the compilation process different for applications and applets? List some security restrictions on applets.
42. Which classes can an applet extend?

**COURSE TITLE:
INFORMATION TECHNOLOGY PROJECT
MANAGEMENT**

COURSE CODE: - ITec3062

Complied by

**Aklilu Elias
Tigist Simon**

**Reviwed by
Nuhamin Nigussie**

I. Module description

This course will introduce the area of Information Technology project management, presenting basic techniques and approaches and aiming to develop a critical awareness of the challenges and shortcomings of the area. IT Project Management is an important area of study since most non-trivial software development efforts will be make use of some type of project management approach in an aim to manage the development process in such a way that the software meets its requirements and is on-time and within budget.

II. Objectives of this Module

At the end of the course students will be able to:

- ✓ Understand the issues involved in IT project management and the factors that affect software quality;
- ✓ Familiar with a range of standards, techniques and tools developed to support IT project management and the production of high quality software;
- ✓ Develop IT project plans, supporting software quality plans and risk management plans.
- ✓ Capable of actively participating or successfully managing a software development project by applying project management concepts.
- ✓ Demonstrate knowledge of project management terms and techniques

CHAPTER ONE

INTRODUCTION TO IT PROJECT MANAGEMENT

1.1 Introduction

Activity 1

A. What is project management?

A. Briefly describe the project management framework, providing examples of stakeholders, knowledge areas, tools and techniques, and project success factors.

C. Why is there a new or renewed interest in the field of the project?

Many people and organizations today have a new – or renewed – interest in project management. Until the 1980s, project management primarily focused on providing schedule and resource data to top management in the military, computer, and construction industries. Today's project management involves much more, and people in every industry and every country manage projects. Project management is a distinct profession with degree programs, certifications, and excellent career opportunities.

Today's companies, governments, and nonprofit organizations are recognizing that to be successful, they need to use modern project management techniques, especially for IT projects. Individuals are realizing that to remain competitive in the workplace, they must develop skills to become good project team members and project managers. They also realize that many of the concepts of project management will help them in their everyday lives as they work with people and technology on a day-to-day basis.

1.2 Importance of IT project management

- Better control of financial, physical, and human resources
- Improved customer relations
- Shorter development times
- Lower costs and improved productivity
- Higher quality and increased reliability
- Higher profit margins
- Better internal coordination
- Positive impact on meeting strategic goals
- Higher worker morale

1.1.1. What is a Project?

A project is “a temporary endeavor undertaken to create a unique product, service, or result.” Operations are work done in organizations to sustain the business. Projects are different from operations in that they end when their objectives have been reached or the project has been terminated.

Examples of IT Projects:

Projects can be large or small and involve one person or thousands of people. They can be done in one day or take years to complete. IT projects involve using hardware, software, and networks to create a product, service, or result. Examples of IT projects include the following:

- A team of students creates a smartphone application and sells it online.
- A company is developing a driverless car.
- A small software development team adds a new feature to an internal software application for the finance department.
- A college upgrades its technology infrastructure to provide wireless Internet access across campus.
- A company develops a new system to increase sales force productivity and customer relationship management that will work on various laptops, smartphones, and tablets.
- A television network implements a system to allow viewers to vote for contestants and provide other feedback on programs via social media sites.
- A government group develops a system to track child immunizations.
- A large group of volunteers from organizations throughout the world develops standards for environmentally friendly or green IT.
- A global bank acquires a smaller financial institution and needs to reconcile systems and procedures into a common entity.
- Government regulations require new reporting of commercial business data for a manufacturing company.
- A multinational firm decides to consolidate its information systems into an integrated enterprise resource management approach.

1.1.2. Project Attributes:

Projects come in all shapes and sizes. The following attributes help to define a project further:

- A project is temporary. A project has a definite beginning and end.

- A project is developed using progressive elaboration. Projects are often defined broadly when they begin, and as time passes, the specific details of the project become clearer. Therefore, projects should be developed in increments. A project team should develop initial plans and then update them with more detail based on new information.
- A project requires resources, often from various areas. Resources include people, hardware, software, and other assets. Many projects cross-departmental or other boundaries to achieve their unique purposes. Once the project team has selected key projects for implementation, they will probably require additional resources. Resources, however, are limited and must be used effectively to meet project and other corporate goals.
- A project should have a primary customer or sponsor. Most projects have many interested parties or stakeholders, but someone must take the primary role of sponsorship. The project sponsor usually provides the direction and funding for the project.
- A project involves uncertainty. Because every project is unique, it is sometimes difficult to define its objectives clearly, estimate how long it will take to complete, or determine how much it will cost. Besides, external factors.

An effective project manager is crucial to a project's success. Project managers work with the project sponsors, the project team, and the other people involved to meet project goals.

1.1.3. Project Constraints

Every project is constrained in different ways, often by its scope, time, and cost goals. These limitations are sometimes referred to in project management as the triple constraint. To create a successful project, a project manager must consider the scope, time, and cost and balance these three often-competing goals:

- Scope – What work will be done as part of the project? What unique product, service, or result does the customer or sponsor expect from the project? How will the scope be verified?
- Time – How long should it take to complete the project? What is the project's schedule? How will the team track actual schedule performance? Who can approve changes to the schedule?
- Cost – What should it cost to complete the project? What is the project's budget? How will costs be tracked? Who can authorize changes to the budget?

The three dimensions of the triple constraint are summarized in the following below

Example:

The IT collaboration project might have an initial scope of producing a 40- to 50-page report and a one-hour presentation on about 30 potential IT projects. The initial time estimate for this project might be one month, and the cost estimate might be \$45,000–\$50,000. These expectations provide the targets for the scope, time, and cost dimensions of the project.

Quality is often a key factor in projects, as is customer or sponsor satisfaction. Some people, in fact, refer to the quadruple constraint of project management, which includes quality as well as scope, time, and cost. A project team may meet scope, time, and cost goals but might fail to meet quality standards and satisfy the sponsor.

Risk can also affect major project decisions. A company might wait to start a project until the risks are at an acceptable level.

1.1.4. Problems with IS Project:

Every software project has to face some problems depending on the domain and scope. The scale, complexity, heterogeneity, different stakeholders with different viewpoints, volatile requirements, and other business-related problems.

- Vague and ever-changing requirements.
- The client is slow with communication.
- The project does not start on time.
- You try to manage every project the same way.
- The client does not like what you created.
- Your point of contact does not seem to care about your project.
- Too much time is spent solving problems after projects are “Live”.

1.1.5. What is Project Management?

It is “the application of knowledge, skills, tools, and techniques to project activities to meet project requirements.” Project managers must strive not only to meet specific scope, time, cost, and quality goals of projects, they must also facilitate the entire process to meet the needs and expectations of people involved in project activities or affected by them.

1.1.1 What is a project?

A project is a temporary endeavor undertaken to accomplish a unique purpose (4).

Project management: - is the application of knowledge, skills, tools, and techniques to project activities in order to meet or exceed stakeholder needs and expectations from a project (6).

Attributes of a Project: - Projects can also be viewed in terms of their attributes: time frame, purpose, ownership, resources, roles, risks and assumptions, interdependent tasks, organizational change, and operating in an environment larger than the project itself.

Time Frame: - Because a project is a temporary endeavor, it must have a definite beginning and end. Many projects begin on a specific date and the date of completion is estimated. Some projects, on the other hand, have an immovable date when the project must be completed. In this case, it is necessary to work backwards to determine the date when the project must start. Keep in mind that your career should not consist of a single project, but a number of projects.

Purpose: - Projects are undertaken to accomplish something. An IT project can produce any number of results—a system, a software package, or a recommendation based on study. Therefore, a project's goal must be to produce something tangible and of value to the organization. A project must have a goal to drive the project in terms of defining the work to be done, its schedule, and its budget, and to provide the project team with a clear direction. Because it sets expectations that will directly influence the client's level of satisfaction, the project's goal must be clearly defined and agreed upon. The definition for project management suggests that project activities must meet or exceed stakeholder needs and expectations. Expectations and needs, however, cannot be met if the project's goal is not achieved. It is, therefore, important to keep in mind that a project should only be undertaken to provide some kind of value to the organization. Moreover, a specific and measurable project goal can be evaluated after the project is completed.

Ownership: - The project must provide something of value to an individual or group who will own the project's product after it is completed. Determining who owns this product is not always easy. For example, different groups may fight over who does or does not own the system, the data, the support, and the final cost of implementing and maintaining the system. Although a project may have many stakeholders (i.e., people or groups who have a vested interest in the project's outcome), a project should have a clearly defined sponsor. The sponsor may be the end user, customer, or the client who has the ability and desire to provide direction, funding, and other resources to the project.

1.2. Stages of Project

A software project undergoes a series of identifiable stages during its lifetime. Following are the various stages of a project:

1.2.1. The Feasibility Study

It is used to determine whether developing the product is financially and technically feasible. The data collected in this phase are analyzed to arrive at the following:

- An abstract definition of the problem.
- Formulation of the different solution strategies.
- Examination of alternative solution strategies and their benefits, indicating resources required, development, cost, and time in respect of each of the alternative solutions.
- A cost/benefit analysis is performed to determine which solution is the best. At this stage, it may also be determined whether any of the solutions is not feasible due to high cost, resource constraints, or extraordinary technical reasons.

1.2.1.1.The Cost-Benefit Analysis

It estimates and totals up the equivalent money value of the benefits and costs to the community of projects to establish whether they are worthwhile. These projects may be dams and highways or can be training programs and health care systems. Following are the principles of cost-benefit analysis in any project:

- There must be a common unit of measurement;
- CBA valuations should represent consumers' or producers' valuations as revealed by their actual behavior;
- Benefits are usually measured by market choices;
- The analysis of a project should involve a With versus Without comparison;
- The cost-benefit analysis involves a particular study area
- Double counting of benefits or costs must be avoided

1.2.2. Planning

Project planning is a discipline for stating how to complete a project within a certain timeframe, usually with defined stages, and with designated resources. The main purpose of Project Planning is to guide the execution of projects. The key outputs of Project Planning include:

- Team contract
- Scope statement
- Work breakdown structure
- Project schedule in the form of a Gantt chart with all dependencies and resources entered

- List of prioritized risks

1.2.3. Project Execution

The Project execution takes most of the time and resources in Project Management. Project Managers must use their leadership skills to handle the various challenges that occur during project execution. Project Sponsors and customers focus on the deliverables related to providing the products, services, or results desired from the project. A milestone report will help the Project Manager to keep the focus on completing major milestones.

1.2.4. Project and Product Life Cycles

A life cycle model prescribes the different activities that need to be carried out to develop a software product and the sequencing of these activities. A software life cycle model is a descriptive and diagrammatic representation of the software life cycle. Following are the various stages of a software project:

Requirement Analysis and Specification: The aim of the requirement analysis and specification phase is to understand the exact requirements of the customer and to document them properly. This phase consists of two distinct activities:

- Requirement analysis
- Requirement specification

The goal of the requirements analysis is to collect and analyze all related data and information with a view to understanding the customer requirements clearly and weeding out inconsistencies and incompleteness in these requirements.

During requirements specification, the user requirements are properly organized and documented in an SRS document. The SRS document addresses the functional requirements, the nonfunctional requirements, and the special requirements for the maintenance and development of the software product if any.

B. Design: The goal of the design phase is to transform the requirements specified in the SRS document into a structure that is suitable for implementation in some programming language. In technical terms, during the design phase, the software architecture is derived from the SRS document. Two distinctly different approaches are available:

- **Traditional Design Approach:** it consists of two different activities; first a structured analysis of the requirements specification is carried out where the detailed structure of the problem is examined. This is followed by a structured design activity. During structured design, the results of the structured analysis are transformed into software design.
- **Object-Oriented Design Approach:** In this technique, various objects that occur in the problem domain and the solution domain are first identified, and the different relationships that exist among these objects are identified. The object structure is further refined to obtain a detailed design.

C. Coding and Unit Testing: The purpose of this phase (also called the implementation phase) of software development is to translate the software design into source code. The end product of the implementation phase is a set of program modules that have been individually tested.

D. Integration and System Testing: During this phase, the different modules are integrated in a planned manner. The different modules making up a system are almost never integrated into a single shot. The goal of system testing is to ensure that the developed system functions according to its requirements as specified in the SRS document.

E. Maintenance: Maintenance involves performing any one or more of the following three kinds of activities:

1. Correcting errors that were not discovered during the product development phase. This is called corrective maintenance.
2. Improving the implementation of the system and enhancing the functionalities of the system according to the customer's requirements. This is called perfective maintenance.
3. Porting the software to a new environment, e.g. to a new computer or to a new operating system. This is called adaptive maintenance.

1.3. The Stakeholder of a Project

Stakeholders are the people involved in or affected by project activities, and include the project sponsor, project team, support staff, customers, users, suppliers, and even opponents of the project. These stakeholders often have very different needs and expectations.

1.3.1. All Parties of a Project

Example:

Building a new house is a well-known example of a project. There are several stakeholders involved in a home construction project.

a. The project sponsors would be the potential new homeowners. They would be the people paying for the house and they could be on a very tight budget, so they would expect the contractor to provide accurate estimates for the costs of building the house. They would also need a realistic idea of when they could move in and what type of home, they could afford given their budget constraints. The new homeowners would have to make important decisions to keep the costs of the house within their budget. Can they afford to finish the basement right away? If they can afford to finish the basement, will it affect the projected move-in date? In this example, the project sponsors are also the customers and users of the product, which is the house.

b. The house may require financing by a bank or other financial institution like a credit union, which will secure a legal interest (lien) in the property and the finished home. This institution is an example of a legal stakeholder who must be informed of any changes to the plans or schedule because the project is part of a legal contract.

c. The project manager would normally be the general contractor responsible for building the house. The project manager needs to work with all the project stakeholders to meet their needs and expectations.

d. The project team for building the house would include several construction workers, electricians, and carpenters. These stakeholders would need to know exactly what work they must do and when they need to do it. They would need to know if the required materials and equipment will be at the construction site or if they are expected to provide the materials and equipment. Their work would need to be coordinated because many interrelated factors are involved.

Example: The carpenter cannot put in kitchen cabinets until the walls are completed.

e. Support staff might include the buyers' employers, the general contractor's administrative assistant, and people who support other stakeholders. The buyers' employers might expect their employees to complete their work but allow some flexibility so they can visit the building site or take phone calls related to building the house. The contractor's administrative assistant would support the project by coordinating meetings between the buyers, the contractor, suppliers, and other parties.

f. Building a house requires many suppliers. The suppliers would provide the wood, windows, flooring, appliances, and other materials. Suppliers would expect exact details on the items they need to provide, and where and when to deliver those items.

g. A project might have opponents. In this example, a neighbor might oppose the project because the workers make so much noise that she cannot concentrate on her work at home, or the noise might wake her sleeping children. Even without such complaints, the home must comply with certain building codes and other restrictions; these considerations may also result in changes to the project's requirements, making the local government a stakeholder in the project.

1.3.2. The Role of Project Manager

The project managers must work closely with the other stakeholders on a project, especially a sponsor and a project team. What do project managers do exactly? What skills do they really need to do a good job? Here below are some of the job descriptions of a project manager:

- Project manager for a consulting firm: Plans, schedules, and controls activities to fulfill identified objectives applying technical, theoretical, and managerial skills to satisfy project requirements. Coordinates and integrates team and individual efforts and builds positive professional relationships with clients and associates.
- IT project manager for a financial services firm: Manages, prioritizes, develops, and implements IT solutions to meet business needs. Prepares and executes project plans using project management software following a standard methodology. Establishes cross-functional end-user teams defining and implementing projects on time and within budget. Acts as a liaison between third-party service providers and end-users to develop and implement technology solutions. Participates in vendor contract development and budget management. Provides post-implementation support.
- IT project manager for a nonprofit consulting firm: Responsibilities include business analysis, requirements gathering, project planning, budget estimating, development, testing, and implementation. Responsible for working with various resource providers to ensure development is completed in a timely, high-quality, and cost-effective manner.

Ten most important skills and competencies for project managers:

1. People skills
2. Leadership
3. Listening
4. Integrity, ethical behavior, consistency
5. Strength at building trust
6. Verbal communication
7. Strength at building teams
8. Conflict resolution, conflict management
9. Critical thinking, problem solving
10. Understanding and balancing of priorities

Figure 1-2 skills for project managers

Leadership and management are terms often used interchangeably, although there are differences. Generally, a leader focuses on long-term goals and big-picture objectives while inspiring people to reach those goals. A manager often deals with the day-to-day details of meeting specific goals. Some people say: “Managers do things right, and leaders do the right things.” “Leaders determine the vision, and managers achieve the vision.” “You lead people and manage things.”

1.4. Ethics in Project Management

Ethics, loosely defined, is a set of principles that guides decision-making based on personal values of what is considered right and wrong. Making ethical decisions is an important part of project managers’ personal and professional lives because it generates trust and respect with other people. Project managers often face ethical dilemmas.

Example:

Several projects might involve different payment methods. If project managers can make more money by doing their jobs poorly, should they? No!

1.5. Project Management Framework

It is the project management knowledge areas that describe the key competencies that project managers must develop. It is shown in the following figure.

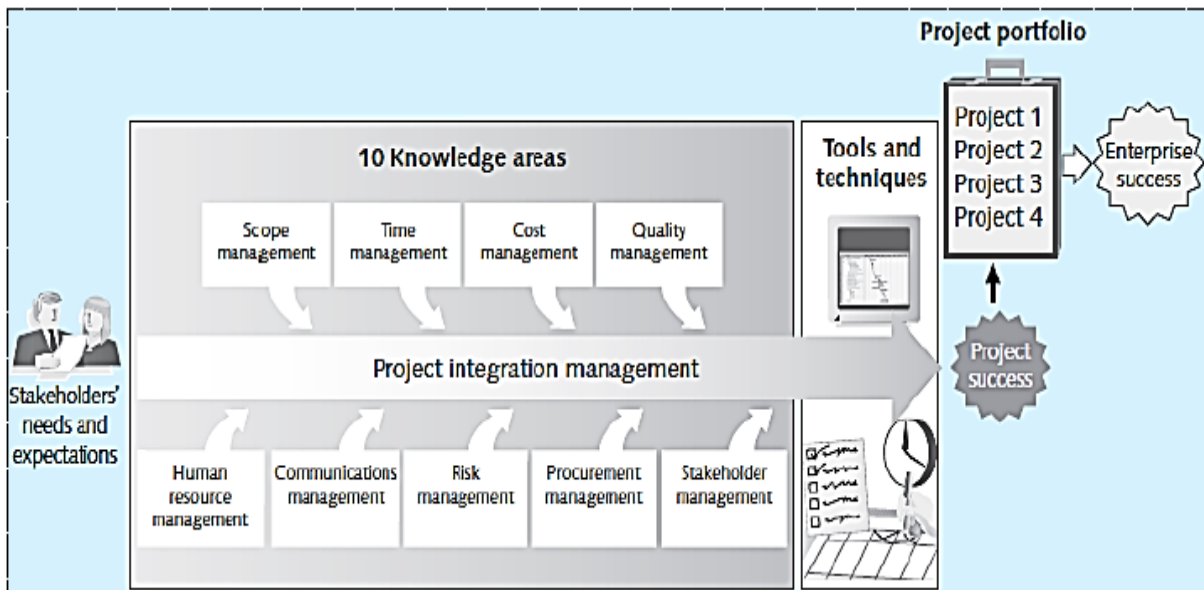


Figure 1-3 Project management framework

The 10 knowledge areas of project management are described as:

Project scope management involves defining and managing all the work required to complete the project successfully.

1. *Project scope management* involves defining and managing all the work required to complete the project successfully.
2. *Project time management* includes estimating how long it will take to complete the work, developing an acceptable project schedule, and ensuring timely completion of the project.
3. *Project cost management* consists of preparing and managing the budget for the project.
4. *Project quality management* ensures that the project will satisfy the stated or implied needs for which it was undertaken.
5. *Project human resource management* is concerned with making effective use of the people involved with the project.
6. *Project communications management* involves generating, collecting, disseminating, and storing project information.
7. *Project risk management* includes identifying, analyzing, and responding to risks related to the project.
8. *Project procurement management* involves acquiring or procuring goods and services for a project from outside the performing organization.
9. *Project stakeholder management* includes identifying and analyzing stakeholder needs while managing and controlling their engagement throughout the life of the project.
10. *Project integration management* is an overarching function that affects and is affected by all of the other knowledge areas.

1.6. Program and Project Portfolio Management

Programs:

It is “a group of related projects managed in a coordinated way to obtain benefits and control not available from managing them individually.” It is often more economical to group projects together to help streamline management, staffing, purchasing, and other work. The following are examples of common programs in the IT field.

- **Infrastructure** – An IT department often has a program for IT infrastructure projects. This program could encompass several projects, such as providing more wireless Internet access, upgrading hardware and software, and developing and maintaining corporate standards for IT.
- **Applications Development** – this program could include several projects, such as updating an enterprise resource planning (ERP) system, purchasing a new off-the-shelf billing system, or developing a new capability for a customer relationship management system.
- **User Support** –in addition to the many operational tasks related to user support, many IT departments have several projects to support users. For example, a project might provide a better e-mail system or develop technical training for users.

Project Portfolio Management:

In many organizations, project managers also support an emerging business strategy of project portfolio management, in which organizations group and manage projects and programs as a portfolio of investments that contribute to the entire enterprise’s success. Portfolio managers help their organizations make wise investment decisions by helping to select and analyze projects from a strategic perspective. Portfolio managers may or may not have previous experience as project or program managers. It is most important that they have strong financial and analytical skills and understand how projects and programs can contribute to meeting strategic goals.

Project Management vs Project Portfolio Management

They are mainly distinguished by tactical and strategic. Tactical goals are generally more specific and short-term than strategic goals, which emphasize long-term goals for an organization.

Individual projects often address tactical goals, whereas portfolio management addresses strategic goals. Project management and Project Portfolio management addresses questions which shown in the following Fig.



Figure 1-4 The comparison of project management and project portfolio management

1.7. Software Tools for Project Management

“Man is a tool-using animal. Without tools, he is nothing, with tools he is all.” It assists project managers and their teams in carrying out work in all 10 knowledge areas.

Example:

Some popular time-management tools and techniques include Gantt charts, project network diagrams, and critical path analysis.

Hundreds of project management software tools provide specific functionality for managing projects and performing portfolio management. These software tools can be divided into three general categories based on functionality and price:

1. **Low-end Tools** – these tools provide basic project management features for Smartphone and tablet apps. It is often recommended for small projects and single users.
Example: Gantt Charts
2. **Midrange Tools** – it is designed to handle larger projects, multiple users, and multiple projects. All of these tools can produce Gantt charts and network diagrams and can assist in critical path analysis, resource allocation, project tracking, and status reporting.
Example: Microsoft Project
3. **High-end Tools** – are sometimes referred to as enterprise project management software. These tools provide robust capabilities to handle very large projects and dispersed workgroups, and they have enterprise and portfolio management functions that summarize and combine individual project information to provide an enterprise view of all projects. These products are generally licensed on a

per-user basis, can be integrated with enterprise database management software, and are accessible via the Internet.

Several free or open-source tools are also available.

Example:

dotProject (www.dotproject.net), Achievo (www.achievo.org), and Ganttproject (www.ganttproject.biz) are all free online project management tools.

Generally, how do you define the success or failure of a project? The measuring criteria for the success of a project are

1. The project met scope, time, and cost goals.
2. The project satisfied the customer/sponsor.
3. The results of the project met its main objective, such as making or saving a certain amount of money, providing a good return on investment, or simply making the sponsors happy.

1.8. Chapter one review question

1. What is a project?
2. What is the project aiming to achieve? Why is it important to achieve it?
3. Who is going to be involved and what are their responsibilities? How and when is it all going to happen?
4. How is a project different from what most people do in their day-to-day jobs?
5. What is the triple constraint?
6. Discuss the relationship between project, program, and portfolio management and their contribution to enterprise success.
7. What are the roles of the project, program, and portfolio managers?
8. What are the suggested skills for project managers?
9. What additional skills do program and portfolio managers need?

CHAPTER TWO

PROJECT PLANNING

2.1. Introduction

Activity

- Explain the strategic planning. What do you know about SWOT?
- What do you call the area or boundary of a project?
- Discuss the scope planning process?

All the projects that should be managed by a project manager should have a project plan. The project plan details many aspects of the project to be executed. First of all, it details the project scope. Then, it describes the approach or strategy used for addressing the project scope and project objectives. The strategy is the core of the project plan and it could vary depending on the project purpose and specific project requirements. The resource allocation and delivery schedule are the other two main components of the project plan. These details each activity involved in the project as well as the information such as who executes them and when. This is important information for the project manager as well as all the other stakeholders of the project.

What is project planning?

Project planning is a discipline addressing how to complete a project in a certain timeframe, usually with defined stages and designated resources. One view of project planning divides the activity into these steps:

2.1. Integration Management

Integration management means taking an overview across all project teams and analyzing where you can make efficiencies. Making the most of the resources at your disposal could help save time on tasks and improve productivity, ensuring team members always have something to do.

2.2. Integration Management

2.2.1. What is Integration Management?

Project integration management involves coordinating all of the other project management knowledge areas throughout a project's life cycle. This integration ensures that all the elements of a

project come together at the right times to complete the project successfully. There are six main processes involved in project integration management:

1. Developing the project charter involves working with stakeholders to create the document that formally authorizes a project charter.
2. Developing the project management plan involves coordinating all planning efforts to create a consistent, coherent document for the project management plan.
3. Directing and managing project execution involves carrying out the project management plan by performing the activities included in it. The outputs of this process are deliverables, work performance information, change requests, project management plan updates, and project document updates.
4. Monitoring and controlling project work involve overseeing activities to meet the performance objectives of the project. The outputs of this process are change requests, project management plan updates, and project document updates.
5. Performing integrated change control involves identifying, evaluating, and managing changes throughout the project life cycle. The outputs of this process include change request status updates, project management plan updates, and project document updates.
6. Closing the project or phase involves finalizing all activities to formally close the project or phase. Outputs of this process include final product, service, or result transition and organizational process assets updates.

Therefore, project integration management follows initiating → planning → executing → monitoring and controlling → closing.

Project integration management includes interface management. Interface management involves identifying and managing the points of interaction between various elements of the project. The number of interfaces can increase exponentially as the number of people involved in a project increases. Thus, one of the most important jobs of a project manager is to establish and maintain good communication and relationships across organizational interfaces. The project manager must communicate well with all project stakeholders, including customers, the project team, top management, other project managers, and opponents of the project.

2.2.2. Project Plan Development

Successful leaders look at the big picture or strategic plan of the organization to determine what types of projects will provide the most value. Some may argue that project managers should not be involved in strategic planning and project selection because top management is usually responsible for these types of business decisions. But successful organizations know that project managers can provide valuable insight into the project selection process.

Strategic Planning

It involves determining long-term objectives by analyzing the strengths and weaknesses of an organization, studying opportunities and threats in the business environment, predicting future trends, and projecting the need for new products and services. Strategic planning provides important information to help organizations identify and then select potential projects. Many people are familiar with SWOT analysis analyzing Strengths, Weaknesses, Opportunities, and Threats that are used to aid in strategic planning.

Example:

A group of four people who want to start a new business in the film industry could perform a SWOT analysis to help identify potential projects. They might determine the following based on a SWOT analysis:

Strengths:

- As experienced professionals, we have numerous contacts in the film industry.
- Two of us have strong sales and interpersonal skills.
- Two of us have strong technical skills and are familiar with several filmmaking software tools.
- We all have impressive samples of completed projects.

Weaknesses:

- None of us have accounting/financial experience.
- We have no clear marketing strategy for products and services.
- We have little money to invest in new projects.
- We have no company Web site and limited use of technology to run the business.

Opportunities:

- A current client has mentioned a large project she would like us to bid on.
- The film industry continues to grow.
- There are two major conferences this year where we could promote our company.

Threats:

- Other individuals or companies can provide the services we can.
- Customers might prefer working with more established individuals/ organizations.

- There is a high risk in the film business.

1.2.1 What is Integration Management?

What is project integration management? Project integration management is a project management knowledge area that helps teams work together more seamlessly. Integration management brings together various processes, systems, and methodologies to form a cohesive strategy. To accomplish this, trade-offs must be made.

2.3. Scope Management

Project scope management defines and outlines all work included within a project, such as objectives, tasks, outputs, and deadlines. Project scope management identifies and documents all project objectives, goals, deliverables, deadlines, and budgets during the planning process.

2.1.2. Project Plan Development

Project development involves organizing and planning all the variables that go into a project, including deciding materials, creating a budget and creating tasks for team members

2.1.3. Plan Execution

What is project execution? Project execution is the stage of the project where everything your team has planned is put into action. Your team does everything it can to get projects off on the right foot.

2.2 Scope Management

2.2.1 What is Scope Management?

Scope refers to all the work involved in creating the products of the project and the processes used to create them. Deliverable describes a product created as part of a project. Deliverables can be product-related, such as a piece of hardware or software, or process-related, such as a planning document or meeting minutes. Project stakeholders must agree on what the products of the project are and, to some extent, how they should be produced to define all of the deliverables.

Project scope management includes the processes involved in defining and controlling what work is or is not included in a project. It ensures that the project team and stakeholders have the same understanding of what products the project will produce and what processes the project team will use to produce them. Six main processes are involved in project scope management:

1. Planning scope management involves determining how the project's scope and requirements will be managed. The project team works with appropriate stakeholders to create a scope management plan and requirements management plan.
2. Collecting requirements involves defining and documenting the features and functions of the products for the project as well as the processes used for creating them. The project team creates requirements documentation and a requirements traceability matrix as outputs of the requirements collection process.
3. Defining scope involves reviewing the scope management plan, project charter, requirements documents, and organizational process assets to create a scope statement, adding more information as requirements are developed and change requests are approved. The main outputs of scope definition are the project scope statement and updates to project documents.
4. Creating the WBS involves subdividing the major project deliverables into smaller, more manageable components. It includes a scope baseline and updates to project documents.
5. Validating scope involves formalizing acceptance of the project deliverables. Key project stakeholders, such as the customer and sponsor for the project, inspect and then formally accept the deliverables during this process. If the deliverables are not acceptable, the customer or sponsor usually requests changes. The main outputs of this process, therefore, are accepted deliverables, change requests, work performance information, and updates to project documents.
6. Controlling scope involves controlling changes to project scope throughout the life of the project – a challenge in many IT projects. Scope changes often influence the team's ability to meet project time and cost goals, so project managers must carefully weigh the costs and benefits of scope changes. The main outputs of this process are work performance information, change requests, and updates to the project management plan, project documents, and organizational process assets.

2.3.2. Methods for Selecting Projects

Organizations identify many potential projects as part of their strategic planning processes, and they often rely on experienced project managers to help them make project selection decisions. However, organizations need to narrow down the list of potential projects to those projects that will be of most benefit. Selecting projects is not an exact science, but it is a necessary part of project management. Many methods exist for selecting from among possible projects. Five common techniques are:

- a. Focusing on broad organizational needs. (Need → Funding → Will)
- b. Categorizing information technology projects. (Problems → Opportunities → Directives)
- c. Performing net present value or other financial analyses.

Net Present Value (NPV) analysis is a method of calculating the expected net monetary gain or loss from a project by discounting all expected future cash inflows and outflows to the present point in time. An organization should consider only projects with a positive NPV if the financial value is a key criterion for project selection.

Return on Investment (ROI) – is the result of subtracting the project costs from the benefits and then dividing them by the costs.

$$ROI = \frac{(Current\ Cost\ (Benefit\ Cost) - previous\ cost\ ((Invested\ Cost))}{Previous\ cost} * 100$$

Example:

If you invest Rs.100 today and next year it is worth Rs.110, your ROI is (Rs. 110 – 100)/100 or 0.10 (10 percent). Note that the ROI is always a percentage. It can be positive or negative. It is best to consider discounted costs and benefits for multi-year projects when calculating ROI.

Payback Analysis – payback period is the amount of time it will take to recoup, in the form of net cash inflows, the total dollars invested in a project. In other words, a payback analysis determines how much time will lapse before accrued benefits overtake accrued and continuing costs. Payback occurs when the net cumulative benefits equal the net cumulative costs, or when the net cumulative benefits minus costs equal zero.

d. Using a weighted scoring model.

a weighted scoring model is to identify criteria important to the project selection process

e. Implementing a balanced scorecard.

A balanced scorecard is a methodology that converts an organization's value driver, such as customer service, innovation, operational efficiency, and financial performance, to a series of defined metrics. Organizations record and analyze these metrics to determine how well projects help them achieve strategic goals.

2.2.2 Methods for Selecting Project

Read the following Various Project Selection Methods

- Benefit Measurement Methods.
- Benefit/Cost Ratio.
- Economic Model. .
- Scoring Model in Project Management.

- Payback Period.
- Net Present Value.
- Discounted Cash Flow.
- Internal Rate Of Return.

2.2.3 Project Charter

A project charter is a short document used in project planning to outline the key aims and benefits of a project. It's an at-a-glance guide to why a project is taking place. It's used both as a marketing tool, useful to get buy-in from stakeholders, and a reference point to keep the project on track.

2.2.4 Scope Statement

A scope statement is a document that defines all the elements of the project scope as well as assumptions, project requirements and acceptance criteria. Your project scope statement will act as the primary tool for stakeholders and teammates to refer back to and use as a guideline to accurately measure project success.

2.2.5 Work Breakdown Structure

A work breakdown structure (WBS) is a project management tool that takes a step-by-step approach to complete large projects with several moving pieces. By breaking down the project into smaller components, a WBS can integrate scope, cost and deliverables into a single tool.

2.4. Stepwise Project Planning

2.4.1. Overview of Project Planning

A project management plan is a document used to coordinate all project planning documents and help guide a project's execution and control. Plans created in the other knowledge areas are considered subsidiary parts of the overall project management plan. Project management plans also, document project planning assumptions and decisions regarding choices, facilitate communication among stakeholders, define the content, extent, and timing of key management reviews, and provide a baseline for progress measurement and project control. Project management plans should be dynamic, flexible, and subject to change when the environment or project changes. These plans should greatly assist the project manager in leading the project team and assessing the project status.

The main inputs for developing a project management plan include the project charter, outputs from planning processes, enterprise environmental factors, and organizational process assets. The main tool and technique are expert judgment, and the output is a project management plan.

2.4.2. Main Steps in Project Planning

1. Introduction or overview of the project.

- The Project Name: Every project should have a unique name. Why?
- A brief description of the project and the need it addresses: It should clearly outline the goals of the project and the reason for the project. It should be written in layperson's terms, avoid technical jargon, and include a rough time and cost estimate.
- The sponsors' name: Every project needs a sponsor. Include the name, title, and contact information of the sponsor in the introduction.
- The names of the project manager and key team members: The project manager should always be the contact for project information. Depending on the size and nature of the project, names of key team members may also be included.
- Deliverables of the project: It should briefly list and describe the products that will be produced as part of the project. Example: Software, hardware, technical reports, and training materials.
- A list of important reference materials: Many projects have a history preceding them. Listing important documents or meetings related to a project helps project stakeholders understand the history that was produced for other knowledge areas.
- A list of definitions and acronyms, if appropriate: Many projects involve terminology unique to a particular industry or technology. Providing a list of definitions and acronyms will help avoid confusion.

2. A description of how the project is organized.

- Organizational charts: It is used to show the lines of authority, responsibilities, and communication for the project.
- Project responsibilities: It should describe the major project functions and activities and identify those individuals who are responsible for them.
- Other organizational or process-related information: Depending on the nature of the project, there may be a need to document major processes followed on the project.

Example:

If the project involves releasing a major software upgrade, it might help everyone involved in the project to see a diagram or timeline of the major steps.

3. The management and technical processes used in the project

- Management objectives: It is important to understand top management's view of the project, what the priorities are for the project, and any major assumptions or constraints.
- Project controls: It describes how to monitor project progress and handles changes. Will there be monthly status reviews and quarterly progress reviews? Will there be specific forms or charts to monitor progress? Will the project use earned value management to assess and track performance? What is the process for change control? What level of management is required to approve different types of changes?
- Risk management: It addresses how the project team will identify, manage, and control risks.
- Project staffing: It describes the number and types of people required for the project.
- Technical processes: It describes specific methodologies a project might use and explains how to document information.

Example:

Software development methodologies or Computer-Aided Software Engineering (CASE) tools.

4. Sections describing the work to be performed

- Major work packages: It should briefly summarize the main work packages for the project like WBS.
- Key deliverables: It lists and describes the key products produced as part of the project. It should also describe the quality expectations for the product deliverables.
- Other work-related information: It highlights key information related to the work performed on the project like a list of specific hardware or software to use on the project.

5. The schedule

- Summary schedule: It is helpful to see a one-page summary of the overall project schedule. Depending on the size and complexity of the project, the summary schedule might list only key deliverables and their planned completion dates. For smaller projects, it might include all of the work and associated dates for the entire project in a Gantt chart.
- Detailed schedule: It provides information on the project schedule that is more detailed.

Example:

It might explain that a major part of the work cannot start until an external agency provides funding. A network diagram can show these dependencies.

- Other schedule-related information: Many assumptions are often made in preparing project schedules.

6. The budget.

- **Summary budget:** It includes the total estimate of the overall project's budget, and the budget estimate for each month or year by certain budget categories.
- **Detailed budget:** It summarizes what is in the cost management plan and includes more detailed budget information.

Example:

What are the fixed and recurring cost estimates for the project each year? What are the projected financial benefits of the project? What types of people are needed to do the work, and how are the labor costs calculated?

- **Other budget-related information:** It documents major assumptions and highlights other important information related to the financial aspects of the project.

7. Closing projects or phases.

- In order to close a project or phase, you must finalize all activities and transfer the completed or canceled work to the appropriate people. The main inputs to this process are the project management plan, accepted deliverables, and organizational process assets.

Generally, the main steps of project planning are

Major section headings	Section topics
Overview	Purpose, scope, and objectives; assumptions and constraints; project deliverables; schedule and budget summary; evolution of the plan
Project Organization	External interfaces; internal structure; roles and responsibilities
Managerial Process Plan	Start-up plans (estimation, staffing, resource acquisition, and project staff training plans); work plan (work activities, schedule, resource, and budget allocation); control plan; risk management plan; closeout plan
Technical Process Plans	Process model; methods, tools, and techniques; infrastructure plan; product acceptance plan
Supporting Process Plans	Configuration management plan; verification and validation plan; documentation plan; quality assurance plan; reviews and audits; problem resolution plan; subcontractor management plan; process improvement plan

Figure 2-3 steps of

project planning

Note:

└ All the main steps of project planning will undertake after you select the Project Title!!

2.3.1 Overview of Project Planning

Project planning is a discipline addressing how to complete a project in a certain timeframe, usually with defined stages and designated resources. One view of project planning divides the activity into these steps: setting measurable objectives identifying deliverable and scheduling.

2.3.2 Main Steps in Project Planning

Project planning includes the following 10 steps:

- Define stakeholders. Stakeholders include anyone with an interest in the project.
- Define roles. Each stakeholder's role should be clearly defined.
- Introduce stakeholders.
- Set goals.
- Prioritize tasks.
- Create a schedule.
- Assess risks.
- Communicate.

2.5. Chapter two review questions

1. Discuss the elements involved in risk management planning and the contents of a risk management plan.
2. List common sources of risks in information technology projects
3. describe the risk identification process, tools, and techniques to help identify project risks and the main output of the risk identification-risk register
4. Discuss the qualitative risk analysis process and explain how to calculate risk factors, create probability impact matrixes, apply the top ten risk item tracking techniques, and use expert judgment to rank risks
5. Explain the quantitative risk analysis process and how to apply decision trees, simulation, and sensitivity analysis to quantify risks
6. Provide examples of using different risk response planning strategies to address both negative and positive risks

CHAPTER THREE

PROJECT SCHEDULING

3.1. Introduction

Activity

- How do you estimate project duration?
- What kind of mechanisms are there to predict the time it takes to finish a project?

Managers often cite the need to deliver projects on time as one of their biggest challenges and the main cause of conflict. Perhaps part of the reason that schedule problems are so common is that time is easily measured and remembered. You can debate scope and cost overruns and make actual numbers appear closer to estimates, but once a project schedule is set, people remember the projected completion date, and anyone can quickly estimate schedule performance by subtracting the original time estimate from how long it really took to complete the project. People often compare planned and actual project completion times without taking into account the approved changes in the project. Time is the variable that has the least amount of flexibility. Time passes no matter what happens on a project.

Individual work styles and cultural differences may also cause schedule conflicts. With all these possibilities for schedule conflicts, it is important for project managers to use good project time management.

3.2 Time Management

Project time management involves the processes required to ensure timely completion of a project.

3.2.1. Importance of Project Schedules

Seven main processes are involved in project time management:

1. Planning schedule management involves determining the policies, procedures, and documentation that will be used for planning, executing, and controlling the project schedule. The main output of this process is a schedule management plan.

2. Defining activities involves identifying the specific activities that the project team members and stakeholders must perform to produce the project deliverables. An activity or task is an element of work normally found on the work breakdown structure (WBS) that has expected duration, cost, and resource requirements. The main outputs of this process are an activity list, activity attributes, a milestone list, and project management plan updates.
3. Sequencing activities involves identifying and documenting the relationships between project activities. The main outputs of this process include project schedule network diagrams and project documents updates.
4. Estimating activity resources involves estimating how many resources – people, equipment, and materials – a project team should use to perform project activities. The main outputs of this process are activity resource requirements, a resource breakdown structure, and project documents updates.
5. Estimating activity durations involves estimating the number of work periods that are needed to complete individual activities. Outputs include activity duration estimates and project documents updates.
6. Developing the schedule involves analyzing activity sequences, activity resource estimates, and activity duration estimates to create the project schedule. Outputs include a schedule baseline, project schedule, schedule data, project calendars, project management plan updates, and project documents updates.
7. Controlling the schedule involves controlling and managing changes to the project schedule. Outputs include work performance information, schedule forecasts, change requests, project management plan updates, project documents updates, and organizational process assets updates

3.2.2. Schedules and Activities

a. Planning Schedule Management

It is planning how the schedule will be managed throughout the life of the project. Project schedules grow out of the basic documents that initiate a project. The project charter often mentions planned project start and end dates, which serve as the starting points for a more detailed schedule. After reviewing the project management plan, project charter, enterprise environmental factors, and organizational process assets, the project team uses expert judgment, analytical techniques, and meetings to develop the schedule management plan. It includes the following information:

- Project schedule model development: Many projects include a schedule model, which contains project activities with estimated durations, dependencies, and other planning information that can be used to produce a project schedule.
- The scheduling methodology and the scheduling tool to use when developing the project schedule model: Some projects will use critical path or critical chain methodologies, for example, while simpler projects might focus only on milestones.
- Level of accuracy and units of measure: This section discusses how accurate schedule estimates should be and determines whether time is measured in hours, days, or another unit.
- Control thresholds: Variance thresholds, such as $\pm 10\%$, are established for monitoring schedule performance.
- Rules of performance measurement: For example, if earned value management (EVM) will be used, this section specifies how.
- Reporting formats: This section describes the format and frequency of schedule reports required for the project.
- Process descriptions: The schedule management plan also describes how all of the schedule management processes will be performed.

b. Defining Activities

Defining activities involves identifying the specific actions that will produce the project deliverables in enough detail to determine resource and schedule estimates. The project team reviews the schedule management plan, scope baseline, enterprise environmental factors, and organizational process assets to begin defining activities. Outputs of this process include:

- The activity list is a tabulation of activities to be included on a project schedule. The list should include the activity name, an activity identifier or number, and a brief description of the activity.
- The activity attributes provide more schedule-related information about each activity, such as predecessors, successors, logical relationships, leads and lags, resource requirements, constraints, imposed dates, and assumptions related to the activity. The activity list and activity attributes should be in agreement with the WBS and WBS dictionary.
- A milestone on a project is a significant event that normally has no duration. It often takes several activities and a lot of work to complete a milestone, but the milestone itself is like a marker to help in identifying necessary activities for setting schedule goals and monitoring progress.
- Activity information is a required input to the other time management processes. You cannot determine activity sequencing, resources, or durations, develop the schedule, or control the schedule until you have a good understanding of project activities.

3.1.3. Sequencing and Scheduling Activity

There are a few different tools you can use to sequence project activities, including Gantt charts, the PERT charts, and the precedence diagramming method (PDM). Since we've already covered the first two in-depth, we'll focus on the PDM today. PDM is a graphical representation of the relationships between tasks.

3.2. Project Network Diagrams

It is the preferred technique for showing activity sequencing. It is a schematic display of the logical relationships among project activities and their sequencing.

Note the main elements on this network diagram.

- The letters A through J represent activities with dependencies that are required to complete the project. These activities come from the WBS and activity definition process.
- The arrows represent the activity sequencing or relationships between tasks.

Example – Activity A must be done before Activity D, and Activity D must be done before Activity H.

The format of this network diagram uses the **activity-on-arrow (AOA)** approach or the **arrow diagramming method (ADM)** – a network diagramming technique in which activities are represented by arrows and connected at points called nodes to illustrate the sequence of activities. Node is simply the starting and ending point of an activity. The first node signifies the start of a project, and the last node represents the end.

Assuming that you have a list of the project activities and their start and finish nodes, follow these steps to create an AOA network diagram:

1. Find all of the activities that start at Node 1. Draw their finish nodes, and draw arrows between Node 1 and each of the finish nodes. Put the activity letter or name on the associated arrow. If you have a duration estimate, write it next to the activity letter or name, as shown in Fig. 3.3.

Example:

| A = 1 means that the duration of Activity A is one day, week, or other standard unit of time. Be sure to put arrowheads on all arrows to signify the direction of the relationships.

2. Continue drawing the network diagram, working from left to right. Look for bursts and merges. Bursts occur when two or more activities follow a single node. A merge occurs when two or more nodes precede a single node.

Example:

⌋ In Fig. 3.3, Node 1 is a burst because it goes into Nodes 2, 3, and 4. Node 5 is a merge preceded by Nodes 2 and 3.

3. Continue drawing the AOA network diagram until all activities are included.
4. As a rule of thumb, all arrowheads should face toward the right, and no arrows should cross on an AOA network diagram. You may need to redraw the diagram to make it look presentable.

3.4. Chapter three review questions

1. What are some advantages project network diagrams have over traditional Gantt charts?
2. Define predecessor, successor, and parallel activities. Give a real world example of each.
3. How can parallel activities help shorten the project schedule? Are there any trade-offs?
4. What is meant by slack (or float)?
5. What is the difference between crashing and fast tracking a project's schedule?
6. Why should assumptions used in estimating be documented?
7. What is a baseline plan?
8. When does the project manager or team have the authority to begin executing the project plan?
9. Why should the project manager ensure that the project resources are leveled?

CHAPTER FOUR

PROJECT COST MANAGEMENT

4.1. Introduction, Importance and Principles of Project Cost Management

Activity

- Explain what happens during project budgeting?
- Why is it important to estimate cost and manage it?

IT projects have room for improvement in meeting cost goals. In project cost management, particularly planning cost management, creating good estimates, and using earned value management (EVM) to assist in cost control.

What is the Cost?

- “Accountants usually define cost as a resource sacrificed or foregone to achieve a specific objective.”
- Webster’s dictionary defines cost as “something given up in exchange.” Costs are often measured in monetary amounts, such as dollars, that must be paid to acquire goods and services.

What is Project Cost Management?

Project cost management includes the processes required to ensure that a project team completes a project within an approved budget. Notice two crucial phrases in this definition: “a project” and “approved budget.” Project managers must make sure their projects are well defined, have accurate time and cost estimates, and have a realistic budget that they were involved in approving. It is the project manager’s job to satisfy project stakeholders while continuously striving to reduce and control costs. There are four processes for project cost management:

1. Planning cost management involves determining the policies, procedures, and documentation that will be used for planning, executing, and controlling project costs. The main output of this process is a cost management plan.
2. Estimating costs involves developing an approximation or estimate of the costs of the resources needed to complete a project. The main outputs of the cost estimating process are activity cost estimates, the basis of estimates, and project documents updates.

3. Determining the budget involves allocating the overall cost estimate to individual work items to establish a baseline for measuring performance. The main outputs of the cost budgeting process are a cost baseline, project funding requirements, and project documents updates.
4. Controlling costs involves controlling changes to the project budget. The main outputs of the cost control process are work performance information, cost forecasts, change requests, project management plan updates, project documents updates, and organizational process assets updates

Basic Principles of Cost Management

- **Profits** – are revenues minus expenditures. To increase profits, a company can increase revenues, decrease expenses, or try to do both. Most executives are more concerned with profits than with other issues. When justifying investments in new information systems and technology, it is important to focus on the impact on profits, not just revenues or expenses. You cannot measure the potential benefits of the application without knowing the profit margin. Profit margin is the ratio of revenues to profits. If revenues of \$100 generate \$2 in profits, there is a 2 percent profit margin. If the company loses \$2 for every \$100 in revenue, there is a -2 percent profit margin.
- **Life Cycle Costing** – allows you to see a big-picture view of the cost of a project throughout its life cycle. This helps you develop an accurate projection of a project's financial costs and benefits. It considers the total cost of ownership, or development plus support costs, for a project.

Example:

A company might complete a project to develop and implement a new customer service system in one or two years, but the new system could be in place for 10 years.

- **Cash Flow Analysis** – is a method for determining the estimated annual costs and benefits of a project and the resulting annual cash flow. Project managers must conduct cash flow analysis to determine net present value. Most consumers understand the basic concept of cash flow: If they do not have enough money in their wallets or bank accounts, they cannot purchase something. Top management must consider cash flow concerns when selecting projects in which to invest. If top management selects too many projects that have high cash flow needs in the same year, the company will not be able to support all of its projects and maintain its profitability. It is also important to clarify the year used to analyze dollar amounts.
- **Tangible Costs or Benefits** – can easily be measured in dollars.
- **Intangible Costs or Benefits** – are difficult to measure in monetary terms. Intangible benefits for projects often include items like goodwill, prestige, and general statements of improved

productivity that an organization cannot easily translate into dollar amounts. Because intangible costs and benefits are difficult to quantify, they are often harder to justify.

- **Direct Costs** – can be directly related to creating the products and services of the project. You can attribute direct costs to a certain project.

Example:

↳ It includes the salaries of people working full time on the project and the cost of hardware and software purchased specifically for the project. Project managers should focus on direct costs because they can be controlled.

- **Indirect Costs** – are not directly related to the products or services of the project, but are indirectly related to performing the project.

Example:

↳ It would include the cost of electricity, paper towels, and other necessities in a large building that houses 1,000 employees who work on many projects. Indirect costs are allocated to projects, and project managers have very little control over them.

- **Sunk Cost** – is money that has been spent in the past. Consider it gone, like a sunken ship that can never be raised. When deciding what projects to invest in or continue, you should not include sunk costs. It should be forgotten, even though it is often difficult to think that way.
- **Learning Curve Theory** – states that when many items are produced repetitively, the unit cost of those items decreases in a regular pattern as more units are produced.

Example:

↳ Suppose that the Surveyor Pro project would potentially produce 1,000 handheld devices that could run the new software and access information via satellite. The cost of the first handheld unit would be much higher than the cost of the thousandth unit.

- **Reserves** – are dollar amounts included in a cost estimate to mitigate cost risk by allowing for future situations that are difficult to predict.

Contingency reserves allow for future situations that may be partially planned for (sometimes called known unknowns) and are included in the project cost baseline.

Example: – If an organization knows it has a 20 percent rate of turnover for IT personnel, it should include contingency reserves to pay for recruiting and training costs of IT personnel.

Management reserves allow for future situations that are unpredictable (sometimes called unknown unknowns). Example: – If a project manager gets sick for two weeks or an important supplier goes out of business, management reserve could be set aside to cover the resulting costs. Management reserves are not included in a cost baseline.

4.2. Planning Cost Management

It is the first step that helps how the costs will be managed throughout the life of the project. Project costs, like project schedules, grow out of the basic documents that initiate a project, like a project charter. The project manager and other stakeholders use expert judgment, analytical techniques, and meetings to produce the cost management plan.

A cost management plan includes the following information:

- Level of Accuracy – activity cost estimates normally have rounding guidelines, such as rounding to the nearest \$100. There may also be guidelines for the number of contingency funds to include, such as 10 or 20 percent.
- Units of Measure – each unit used in cost measurements, such as labor hours or days, should be defined.
- Organizational Procedures Links – many organizations refer to the WBS component used for project cost accounting as the control account (CA). Each control account is often assigned a unique code that is used in the organization's accounting system. Project teams must understand and use these codes properly.
- Control Thresholds – similar to schedule variance, costs often have a specified amount of variation allowed before action needs to be taken, such as ± 10 percent of the baseline cost.
- Rules of Performance Measurement – if the project uses earned value management (EVM) the cost management plan would define measurement rules, such as how often actual costs will be tracked and to what level of detail.
- Reporting Formats – it would describe the format and frequency of cost reports required for the project.
- Process Descriptions – the cost management plan would also describe how to perform all of the cost management processes.

4.3. Cost Estimating

Project managers must take cost estimates seriously if they want to complete projects within budget constraints. After developing a good resource requirements list, project managers and their project teams must develop several estimates of the costs for these resources.

Types of Cost Estimates

One of the main outputs of project cost management is a cost estimate. Project managers normally prepare several types of cost estimates for most projects. Three basic types of estimates include the following:

A. A rough order of magnitude (ROM) estimate provides an estimate of what a project will cost. A ROM estimate can also be referred to as a ballpark estimate, a guesstimate, a swag, or a broad gauge. This type of estimate is done very early in a project or even before a project is officially started. Project managers and top management use this estimate to help make project selection decisions. The timeframe for this type of estimate is often three or more years prior to project completion. A ROM estimate's accuracy is typically -50 percent to +100 percent, meaning the project's actual costs could be 50 percent below the ROM estimate or 100 percent above.

Example:

↓ The actual cost for a project with a ROM estimate of \$100,000 could range from \$50,000 to \$200,000. Many IT professionals automatically double estimates for software development because of the history of cost overruns on IT projects.

B. A budgetary estimate is used to allocate money into an organization's budget. Many organizations develop budgets at least two years into the future. Budgetary estimates are made one to two years prior to project completion. The accuracy of budgetary estimates is typically -10 percent to +25 percent, meaning the actual costs could be 10 percent less or 25 percent more than the budgetary estimate.

Example:

↓ The actual cost for a project with a budgetary estimate of \$100,000 could range from \$90,000 to \$125,000.

C. A definitive estimate provides an accurate estimate of project costs. Definitive estimates are used for making many purchasing decisions for which accurate estimates are required and for estimating final project costs.

Example:

If a project involves purchasing 1,000 personal computers from an outside supplier in the next three months, a definitive estimate would be required to aid in evaluating supplier proposals and allocating the funds to pay the chosen supplier.

Definitive estimates are made one year or less prior to project completion. A definitive estimate should be the most accurate of the three types of estimates. The accuracy of this type of estimate is normally -5 percent to +10 percent, meaning the actual costs could be 5 percent less or 10 percent more than the definitive estimate.

Example:

The actual cost for a project with a definitive estimate of \$100,000 could range from \$95,000 to \$110,000.

The following summarizes the three basic types of cost estimates.

Type of Estimate	When Done	Why Done	How Accurate
Rough order of magnitude (ROM)	Very early in the project life cycle, often 3–5 years before project completion	Provides estimate of cost for selection decisions	–50% to +100%
Budgetary	Early, 1–2 years out	Puts dollars in the budget plans	–10% to +25%
Definitive	Later in the project, less than 1 year out	Provides details for purchases, estimates actual costs	–5% to +10%

Figure

4-1 cost estimates

4.4. Cost Budgeting

Determining the budget involves allocating the project cost estimate to individual material resources or work items over time. These material resources or work items are based on the activities in the work breakdown structure for the project. The cost management plan, scope baseline, activity cost estimates, basis of estimates, project schedule, resource calendars, risk register, agreements, and organizational process assets are all inputs for determining the budget. The main goal of the cost budgeting process is to produce a cost baseline for measuring project performance and determining project funding requirements. The process may also result in project document updates, such as items being added, removed, or modified in the scope statement or project schedule.

A cost baseline is a time-phased budget that project managers use to measure and monitor cost performance.

4.5 Cost Control

Controlling project costs include monitoring cost performance, ensuring that only appropriate project changes are included in a revised cost baseline, and informing project stakeholders of authorized changes to the project that will affect costs. The project management plan, project funding requirements, work performance data, and organizational process assets are inputs for controlling costs. Outputs of this process are work performance information, cost forecasts, change requests, project management plan updates, project document updates, and organizational process asset updates.

Many general accounting approaches are available for measuring cost performance, earned value management (EVM) is a powerful cost control technique that is unique to the field of project management.

Earned Value Management (EVM)

It is a project performance measurement technique that integrates scope, time, and cost data. Given a cost performance baseline, project managers and their teams can determine how well the project is meeting scope, time, and cost goals by entering actual information and then comparing it to the baseline. **A baseline** is a figure in the original project plan plus approved changes. **Actual information** includes whether or not a WBS item was completed, approximately how much of the work was completed, when the work actually started and ended, and how much the completed work actually cost. The accuracy of calculating earned value is simply by multiplying the planned value to date by a percentage complete value.

Earned value management involves calculating three values for each activity or summary activity from a project's WBS.

- a. **The Planned Value (PV)**, also called the budget, is the portion of the approved total cost estimate planned to be spent on an activity during a given period. Table 4.2. shows an example of earned value calculations. Suppose that a project included a summary activity of purchasing and installing a new Web server. Suppose further that, according to the plan, it would take one week and cost a total of \$10,000 for the labor hours, hardware, and software. Therefore, the planned value (PV) for the activity that week is \$10,000.

b. The Actual Cost (AC) is the total direct and indirect costs incurred in accomplishing work on an activity during a given period.

Example:

Suppose that it actually took two weeks and cost \$20,000 to purchase and install the new Web server. Assume that \$15,000 of these actual costs were incurred during Week 1 and \$5,000 was incurred during Week 2. These amounts are the actual cost (AC) for the activity each week.

c. The Earned Value (EV) is an estimate of the value of the physical work actually completed. EV is based on the original planned costs for the project or activity and the rate at which the team is completing work on the project or activity to date. The rate of performance (RP) is the ratio of actual work completed to the percentage of work planned to have been completed at any given time during the life of the project or activity.

Example:

Suppose that the server installation was halfway completed by the end of Week 1. The rate of performance would be 50 percent because, by the end of Week 1, the planned schedule reflects that the task should be complete but only 50 percent of the work has been completed. In Table 4.2., the earned value estimate after one week is therefore \$5,000.

Figure 4-2 earned value calculations for one activity after Week 1

The earned value calculations above are carried out as follows:

$$EV = 10,000 * 50\% = 5,000$$

$$CV = 5,000 - 15,000 = -10,000$$

$$SV = 5,000 - 10,000 = -5,000$$

$$CPI = 5,000 / 15,000 = 33\%$$

$$SPI = 5,000 / 10,000 = 50\%$$

Formula	
	$EV = PV * SPI$
	$CV = EV - AC$
	$SV = EV - PV$
	$CPI = \frac{EV}{AC} * 100$
	$SPI = \frac{EV}{PV} * 100$
	$EAC = \frac{BAC}{CPI}$
	$ETTC = \frac{OTE}{SPI}$

The formulas above summarize the formulas used in earned value management. Note that the formulas for variances and indexes start with EV, the earned value. **Variances** are calculated by subtracting the actual cost or planned value from EV, and **indexes** are calculated by dividing EV by the actual cost or planned value. After you total the EV, AC, and PV data for all activities on a project, you can use the CPI and SPI to project how much it will cost and how long it will take to finish the project based on performance to date. Given the budget at completion and original time estimate, you can divide by the appropriate index to calculate the estimate at completion (EAC) and

estimated time to complete, assuming that performance remains the same. There are no standard acronyms for the terms estimated time to complete (ETTC) or original time estimate (OTE).

Cost Variance (CV) is the earned value minus the actual cost. If cost variance is a negative number, it means that performing the work cost more than planned. If the cost variance is a positive number, performing the work cost less than planned.

Schedule Variance (SV) is the earned value minus the planned value. A negative schedule variance means that it took longer than planned to perform the work, and a positive schedule variance means that the work took less time than planned to perform.

The Cost Performance Index (CPI) is the ratio of earned value to actual cost; it can be used to estimate the projected cost of completing the project. If the CPI is equal to one, or 100 percent, then the planned and actual costs are equal – the costs are exactly as budgeted. If the CPI is less than one or less than 100 percent, the project is over budget. If the CPI is greater than one or more than 100 percent, the project is under budget.

The Schedule Performance Index (SPI) is the ratio of earned value to planned value; it can be used to estimate the projected time to complete the project. Similar to the cost performance index, an SPI of one, or 100 percent, means the project is on schedule. If the SPI is greater than one or 100 percent, then the project is ahead of schedule. If the SPI is less than one or 100 percent, the project is behind schedule.

Note that in general, negative numbers for cost and schedule variance indicate problems in those areas. Negative numbers mean the project is costing more than planned or taking longer than planned. Likewise, a CPI and SPI of less than one or less than 100 percent also indicate problems.

The Cost Performance Index (CPI) can be used to calculate the *estimate at completion (EAC)* – an estimated cost of completing a project based on performance to date. Similarly, the **schedule performance index (SPI)** can be used to calculate an estimated time to complete the project.

Budget at Completion (BAC) is the overall original total budget for the project. Example: \$100,000. The BAC point is plotted on the chart at the original time estimate of 12 months. The earned value management is summarized in the following figure.

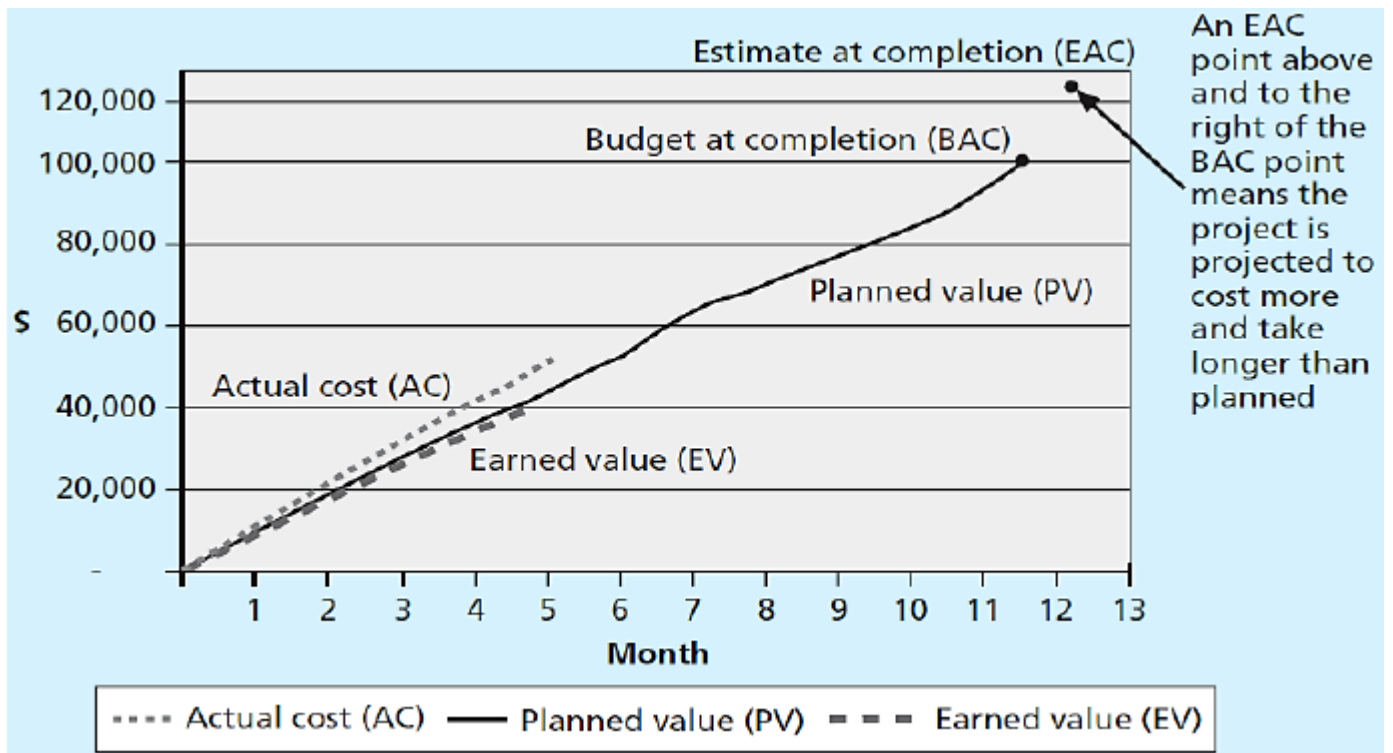


Figure 4-3 Earned value chart for the project after five months

The estimate at completion (EAC), is estimated to be \$122,308 in the figure above. This number is calculated by taking the BAC, or \$100,000 in this case, and dividing it by the CPI, which was 81.761 percent. This EAC point is plotted on the chart at the estimated time to complete 12.74 months. This number is calculated by taking the original time estimate, or 12 months in this case, and dividing it by the SPI, which in this example was 94.203 percent.

Software to Assist in Project Cost Management

- Spreadsheets
- Microsoft Project 2010

4.6. Chapter four review questions

1. Describe the steps necessary for estimating the cost of a particular activity or task that has an estimated duration.
2. What does prorating the cost of a resource mean?
3. What is EVM? define its terms(planned value, actual value, earned value).
4. Discuss why many IT professionals may overlook project cost management and how this might affect the ability to complete projects within budget.
5. Explain some of the basic principles of cost management, such as profits, life cycle costs, tangible and intangible costs and benefits, direct and indirect costs, and reserves.

6. Give examples of when you would prepare rough order of magnitude (ROM), budgetary, and definitive cost estimates for an IT project. Give an example of how you would use each of the following techniques for creating a cost estimate: analogous, parametric, and bottom-up.

CHAPTER FIVE

PROJECT QUALITY MANAGEMENT

Activity

- What do you think are the standards of a project quality?
- What things should be considered for a project to be qualified?

What is Project quality management?

Project quality management is a difficult knowledge area to define. ISO defines quality as “the totality of characteristics of an entity that bear on its ability to satisfy stated or implied needs” (ISO8042:1994) or “the degree to which a set of inherent characteristics fulfills requirements” (ISO9000:2000). Other experts define quality based on conformance to requirements and fitness for use. **Conformance to requirements** means that the project’s processes and products meet written specifications.

Example: If the project scope statement requires the delivery of 100 computers with specific processors and memory, you could easily check whether suitable computers had been delivered.

Fitness for use means that a product can be used as it was intended. If these computers were delivered without monitors or keyboards and were left in boxes on the customer’s shipping dock, the customer might not be satisfied because the computers would not be fit for use. The customer may have assumed that the delivery included monitors and keyboards, unpacking the computers, and installation so they would be ready to use.

The purpose of project quality management is to ensure that the project will satisfy the needs for which it was undertaken. Recall that project management involves meeting or exceeding stakeholder needs and expectations. The project team must develop good relationships with key stakeholders, especially the main customer for the project, to understand what quality means to them. After all, the customer ultimately decides if the quality is acceptable. **Quality**, therefore, must be on an equal level with project scope, time, and cost. If a project’s stakeholders are not satisfied with the quality of the project management or the resulting products of the project, the project team will need to adjust scope, time, and cost to satisfy the stakeholder.

Meeting only written requirements for scope, time, and cost is not sufficient. To achieve stakeholder satisfaction, the project team must develop a good working relationship with all stakeholders and understand their stated or implied needs.

5.1 Quality of IT Project

To have a good quality of IT projects:

- i. Strong leadership.
- ii. Understanding the cost of quality. The cost of quality is the cost of conformance plus the cost of nonconformance. Conformance means delivering products that meet requirements and are fitness for use. Examples include the costs associated with developing a quality plan, costs for analyzing and managing product requirements, and costs for testing. The cost of nonconformance means taking responsibility for failures or not meeting quality expectations.
- iii. Providing a good workplace to enhance quality, and
- iv. Working toward improving the organization's overall maturity level

5.2 Stages of IT Quality Management

Modern quality management requires customer satisfaction, prefers prevention to inspection, and recognizes management responsibility for quality.

Six Sigma is – a comprehensive and flexible system for achieving, sustaining, and maximizing business success. Six Sigma is uniquely driven by a close understanding of customer needs, disciplined use of facts, data, and statistical analysis, and diligent attention to managing, improving, and reinventing business processes.

Projects that use Six Sigma principles for quality control normally follow a five-phase improvement process called **DMAIC** (pronounced de-MAY-ick), which stands for Define, Measure, Analyze, Improve, and Control. DMAIC is a systematic, closed-loop process for continuous improvement that is scientific and fact-based.

1. **Define** – define the problem/opportunity, process, and customer requirements. Important tools used in this phase include a project charter, a description of customer requirements, process maps, and Voice of the Customer (VOC) data. Examples of VOC data include complaints, surveys, comments, and market research that represent the views and needs of the organization's customers.
2. **Measure** – define measures, then collect, compile, and display data. Measures are defined in terms of defects per opportunity.
3. **Analyze** – scrutinize process details to find improvement opportunities. A project team working on a Six Sigma project, normally referred to as a Six Sigma team, investigates and verifies data to prove the suspected root causes of quality problems and substantiates the problem statement.
4. **Improve** – generate solutions and ideas for improving the problem. A final solution is verified with the project sponsor, and the Six Sigma team develops a plan to pilot test the solution. The Six

Sigma team reviews the results of the pilot test to refine the solution, if needed, and then implements the solution where appropriate.

5. **Control** – track and verify the stability of the improvements and the predictability of the solution. Control charts are one tool used in the control phase.

5.2.1 Quality Planning

The role of quality planning is to design a process that will be able to meet established goals under operating conditions. Quality planning is a methodology which can be used when a situation exhibits one or more of the following characteristics: A service has never existed before. Customer requirements are not known.

5.2.2 Quality Assurance

What is quality assurance in project management? Quality assurance in project management is a series of actions designed to ensure the final outcomes of a project meet the expected requirements. It focuses on the processes teams use to maintain standards and produce quality deliverables.

5.2.3 Quality Control

Quality control consists of inspection, measurement and testing to verify that the project outputs meet acceptance criteria defined during quality planning. It is focused on preventing problems being passed on to the internal or external customer.

5.3 Quality Standards

Quality Standards in Project Management (PM) refer to the documentation that proscribes the requirements, guidelines, and specifications for delivering consistent quality across the projects from initiation through all phases until completion.

5.4.1. Quality Planning

Planning quality management implies the ability to anticipate situations and prepare actions that bring about the desired outcome. The current thrust in modern quality management is the prevention of defects through a program of selecting the proper materials, training and indoctrinating people in quality, and planning a process that ensures the appropriate outcome. It includes identifying which quality requirements and standards are relevant to the project and how to satisfy them. ***Design of Experiments*** is a technique that helps identify which variables have the most influence on the overall outcome of a process. Understanding which variables affect the outcome is a very important part of quality planning.

Quality planning also involves communicating the correct actions for ensuring quality in a format that is understandable and complete. Organizational policies related to quality, the particular project's scope statement and product descriptions, and related standards and regulations are all important input to the quality planning process. For an IT project, quality standards might include allowing for system growth, planning a reasonable response time for a system, or ensuring that the system produces consistent and accurate information. Quality standards can also apply to IT services.

Example: You can set standards for how long it should take to get a reply from a help desk or how long it should take to ship a replacement part for a hardware item under warranty.

The main outputs of planning quality management are a quality management plan, a process improvement plan, quality metrics, quality checklists, and project document updates. A metric is a standard of measurement.

Examples of common metrics include failure rates of products, availability of goods and services, and customer satisfaction ratings.

Important scope aspects of IT projects that affect quality include functionality and features, system outputs, performance, and reliability and maintainability.

- **Functionality** is the degree to which a system performs its intended function. Features are the system's special characteristics that appeal to users.
- **System outputs** are the screens and report the system generates. Can the users easily interpret these outputs? Can users get all of the reports they need in a suitable format?
- **Performance** addresses how well a product or service performs the customer's intended use. To design a system with high-quality performance, project stakeholders must address many issues. What volumes of data and transactions should the system be capable of handling? How many simultaneous users should the system be designed to handle?
- **Reliability** is the ability of a product or service to perform as expected under normal conditions.
- **Maintainability** addresses the ease of performing maintenance on a product.

5.4 Tools and Techniques for Quality Control

There are basically seven tools and techniques for quality control.

1. Cause-effect diagrams trace complaints about quality problems back to the responsible production operations.
2. A control chart is a graphic display of data that illustrates the results of a process over time. Control charts allow you to determine whether a process is in control or out of control.
3. A checksheet is used to collect and analyze data. It is sometimes called a tally sheet or checklist, depending on its format.
4. A scatter diagram helps to show if there is a relationship between two variables. The closer data points are to a diagonal line, the more closely the two variables are related.
5. A histogram is a bar graph of a distribution of variables. Each bar represents an attribute or characteristic of a problem or situation, and the height of the bar represents its frequency.
6. A Pareto chart is a histogram that can help you identify and prioritize problem areas. The variables described by the histogram are ordered by frequency of occurrence. Pareto charts help you identify the vital few contributors that account for most quality problems in a system. Pareto analysis is sometimes referred to as the 80-20 rule, meaning that 80 percent of problems are often due to 20 percent of the causes.
7. Flowcharts are graphic displays of the logic and flow of processes that help you analyze how problems occur and how processes can be improved. They show activities, decision points, and the order in which information is processed.

Using Software to Assist in Project Quality Management

- Spreadsheet
- Charting software.

5.6. Chapter five review questions

1. Define quality in your own words. How would you define quality in a word processing, spreadsheet, or presentation software package
2. What is a flow chart? How can it be used as a quality control tool for an IT project?
3. What is a standard? What role do standards play in developing an information system?
4. Briefly describe Six Sigma and its objectives.
5. How does achieving a Six Sigma objective improve quality?
6. What is a control chart? When is a process said to be in statistical control? How would you know if it was not?

CHAPTER SIX

PROJECT HUMAN RESOURCES MANAGEMENT

Activity

- Why is managing humans so difficult task in a project?
- Discuss if encouragement is important in project management?

Many corporate executives have said, “People are our most important asset.” People determine the success and failure of organizations and projects. Most project managers agree that managing human resources effectively is one of the toughest challenges they face.

6.1. What is Project Human Resources Management?

Project human resource management includes the processes required to make the most effective use of the people involved with a project. Human resource management includes all project stakeholders: sponsors, customers, project team members, support staff, suppliers supporting the project, and so on. Human resource management includes the following four processes:

1. ***Planning human resource management*** involves identifying and documenting project roles, responsibilities, and reporting relationships. The main output of this process is a human resource plan.
2. ***Acquiring the project team*** involves assigning the needed personnel to work on the project. Resource assignment and resource loading is an essential points. Resource loading refers to the number of individual resources that an existing schedule requires during specific time periods. Resource loading helps project managers understand the demands of a project on the organization’s resources and on individual people’s schedules. Key outputs of this process are project staff assignments, resource calendars, and project management plan updates.
3. ***Developing the project team*** involves building individual and group skills to enhance project performance. Team-building skills are often a challenge for many project managers. The main outputs of this process are team performance assessments and enterprise environmental factors updates.
4. ***Managing the project team*** involves tracking team member performance, motivating team members, providing timely feedback, resolving issues and conflicts, and coordinating changes to help enhance project performance. Outputs of this process include change requests, project management plan updates, project document updates, enterprise environmental factors updates, and organizational process assets updates.

6.2 Managing People

Industrial-organizational psychologists and management theorists have devoted extensive research and thought to the field of managing people at work. Psychosocial issues that affect how people work and how well they work include motivation, influence and power, and effectiveness.

A. Motivational Theory

Psychologists, managers, coworkers, teachers, parents, and most people in general still struggle to understand what motivates people to do what they do. Intrinsic motivation causes people to participate in an activity for their own enjoyment.

Example:

Some people love to read, write, or play an instrument because it makes them feel good.

Extrinsic motivation causes people to do something for a reward or to avoid a penalty.

Example:

Some young children would prefer not to play an instrument, but they do because they receive a reward or avoid punishment for doing so. Why do some people require no external motivation to produce high-quality work while others require significant external motivation to perform routine tasks? Why can't you get someone who is extremely productive at work to do simple tasks at home?

B. Motivation-Hygiene Theory

This considers the hygiene that causes motivations in It Projects.

Hygiene Factors	Motivators
Larger salaries	Achievement
More supervision	Recognition
More attractive work environment	Work itself
Computer or other required equipment	Responsibility
Health benefits	Advancement
Training	Growth

Figure 6-1 motivation-hygiene theory

C. Acquired-Needs Theory

A person's specific needs are acquired or learned over time and shaped by life experiences. The

main categories of acquired needs include achievement, affiliation, and power. Normally, one or two of these needs are dominant in people.

- i. Achievement** – people who have a high need for achievement seek to excel, and tend to avoid both low-risk and high-risk situations to improve their chances of achieving something worthwhile. Achievers need regular feedback and often prefer to work alone or with other high achievers.
- ii. Affiliation** – people with a high need for affiliation desire harmonious relationships with other people and need to feel accepted by others. They tend to conform to the norms of their work group and prefer work that involves significant personal interaction. Managers should try to create a cooperative work environment to meet the needs of people with a high need for affiliation.
- iii. Power** – people with a need for power desire either personal power or institutional power. People who need personal power want to direct others and can be seen as bossy. People who need institutional power or social power want to organize others to further the goals of the organization. Management should provide such employees with the opportunity to manage others, emphasizing the importance of meeting organizational goals.

6.3. Organizational Planning

An organizational plan is basically a “to-do” list for an organization. It lists out the plan of work, programs, and organizational growth over a period of time – six months, a year, five years. They can be pretty simple to create and use. Writing a plan can just mean getting a clear list of the types of work that need to be done, the tasks involved, who is responsible for them, and when they’ll be done.

- Preparing for organizational planning.
- Identifying the project interfaces (organizational interfaces, technical interfaces, and interpersonal interfaces).
- Identifying the staffing requirements.
- Identifying the project constraints.
- Completing organizational planning.
- Relying on templates
- Applying human resource practices
- Examining organizational planning results
- Creating the role and responsibility assignments
- Creating a staffing management plan

6.4. Issues in Project Staff Acquisition and Team Development

A group is – a collection of people who come together because they share something in common. What they share could be as insignificant as a desire to get on the next bus that will arrive at a particular stop. A team, however, is – a group of people who share a common name, mission, history, set of goals or objectives, and expectations. A strategy that can help groups develop into real teams is teambuilding, – the process needed to create, maintain, and enrich the development of a group of people into a cohesive unit. There are five critical issues identified in this outline.

i. Cohesiveness

This term refers to the attractiveness of group membership. Groups are cohesive to the extent that membership in them is positively valued, and members are drawn toward the group. In task-oriented (e.g., learning or project) groups, the concept can be differentiated into two subconcepts: social cohesiveness and task cohesiveness. Social cohesiveness refers to the bonds of interpersonal attraction that link group members. Task cohesiveness refers to the way in which skills and abilities of the group members mesh to allow optimal performance.

ii. Roles and Norms

All groups develop a set of roles and norms over time, whether or not these are explicitly discussed. Norms are the rules governing the behavior of group members. The use of explicitly defined roles enables the group to cope effectively with the requirements of the task. The roles and norms that govern cooperative learning groups are often imposed by the instructor, but that does not preclude a teambuilding exercise in which those roles and norms, as well as some that are specific to a group, are discussed and accepted.

iii. Communication

Effective interpersonal communication is vital to the smooth functioning of any task group.

iv. Goal Specification

It is very important for group members to have common goals for group achievement, as well as to communicate clearly about individual goals they may have.

v. Interdependence

This is the issue of how each team member's success is determined, at least in part, by the success of the other members.

6.6. Chapter six review questions

1. Discuss changes in the job market for IT workers. How do the job market and current state of the economy affect human resource management?

2. Summarize the processes involved in project human resource management.
3. Briefly summarize the works of Maslow, Herzberg, McClelland, McGregor, Ouchi, Thamhain and Wilemon, and Covey. How do their theories relate to project management?
4. Describe situations in which it would be appropriate to create a project organizational chart, a responsibility assignment matrix, a RACI chart, and a resource histogram. Describe what these charts or matrices look like.
5. Discuss the difference between resource loading and resource leveling, and provide an example of when you would use each technique.
6. Explain two types of team-building activities described in this chapter and discuss their advantages and disadvantages.
7. Summarize the different ways that project managers can address conflicts to help them manage project teams. What can they do to manage virtual team members successfully?

CHAPTER SEVEN

PROJECT COMMUNICATION MANAGEMENT

Activity

- What is there to communicate in a project?
- What are the parties involved in project communication?
- How do you encourage the project team as a manager?

Many experts agree that the greatest threat to the success of any project, especially IT projects, is a failure to communicate. Many problems in other knowledge areas, such as an unclear scope or unrealistic schedules, indicate problems with communications. It is crucial for project managers and their teams to make good communications a priority, especially with top management and other key stakeholders.

The goal of project communications management is to ensure timely and appropriate generation, collection, dissemination, storage, and disposition of project information.

7.1. Communications Planning

7.2. Planning Communications Management

It involves determining the information and communications needs of the stakeholders. Who needs what information? When will they need it? How will the information be given to them? The outputs of this process include a communications management plan and project document updates.

7.2. Information Distribution

It involves creating, distributing, storing, retrieving, and disposing of project communications based on the communications management plan. The main outputs of this process are project communications, project document updates, project management plan updates, and organizational process assets updates.

7.3. Performance Reporting

Another important tool for managing project communications is performance reporting. Performance reporting keeps stakeholders informed about how resources are being used to achieve project objectives. It also motivates workers to have some progress to report. Performance reports are normally provided as status reports or progress reports. Many people use the two terms interchangeably, but some people distinguish between them as follows:

- Status reports describe where the project stands at a specific point in time. It addresses where the project stands in terms of meeting scope, time, and cost goals. How much money has been spent to date? How long did it take to do certain tasks? Is work being accomplished as planned? Status reports can take various formats depending on the stakeholders' needs.
- Progress reports describe what the project team has accomplished during a certain period. Many projects have each team member prepare a monthly report or sometimes a weekly progress report. Team leaders often create consolidated progress reports based on the information received from team members.

Selecting the Appropriate Communication Methods and Media

There are three broad classifications for communication methods:

i. Interactive Communication – as the name implies, two or more people interact to exchange information via meetings, phone calls, or video conferencing. This method is usually the most effective way to ensure common understanding.

ii. Push Communication – information is sent or pushed to recipients without their request via reports, e-mails, faxes, voice mails, and other means. This method ensures that the information is distributed, but does not ensure that it was received or understood.

iii. Pull Communication – information is sent to recipients at their request via Web sites, bulletin boards, e-learning, knowledge repositories like blogs, and other means.

Controlling Communications

It involves monitoring and controlling project communications to ensure that stakeholder communication needs are met.

Keys to Good Communications:

Project managers say they spend as much as 90 percent of their time communicating. The key important concepts are:

- a. Focusing on individual and group communication needs
- b. Using formal and informal methods for communicating

Oral communication also helps build stronger relationships among project personnel and project stakeholders. People like to interact with each other to get a true feeling for how a project is going. Research has indicated that in face-to-face interaction, 58 percent of communication is through

body language, 35 percent is through how the words are said, and a mere 7 percent is through the actual spoken content.

- c. Providing important information in an effective and timely manner
- d. Setting the stage for communicating bad news, and
- e. Understanding communication channels.

Another important aspect of communications is the number of people involved in a project. As the number increases, the complexity of communication increases because there are more channels or pathways through which people can communicate. You can use the following simple formula to determine the number of communication channels as the number of people involved in a project increases:

$$\text{number of communication channels} = \frac{n(n-1)}{2} \quad \text{where } n \text{ is the number of people}$$

involved.

Example:

$$\frac{(2(2-1))}{2} = 1.$$

Two people have one communication channel:

Three people have three channels:

$$\frac{(3(3-1))}{2} = 3.$$

. Four people have six channels, five people have 10, and so on. The following Fig. illustrates this concept. You can see that as the number of people increases above three, the number of communication channels increases rapidly. Project managers should try to limit the size of teams or sub-teams to avoid making communications too complex. For example, if three people are working together on a particular project task, they have three communication channels. If you added two more people to their team, you would have 10 communication channels, an increase of seven. If you added three more people instead of two, you'd have 12 communication channels. You can see how quickly communication becomes more complex as you increase team size.

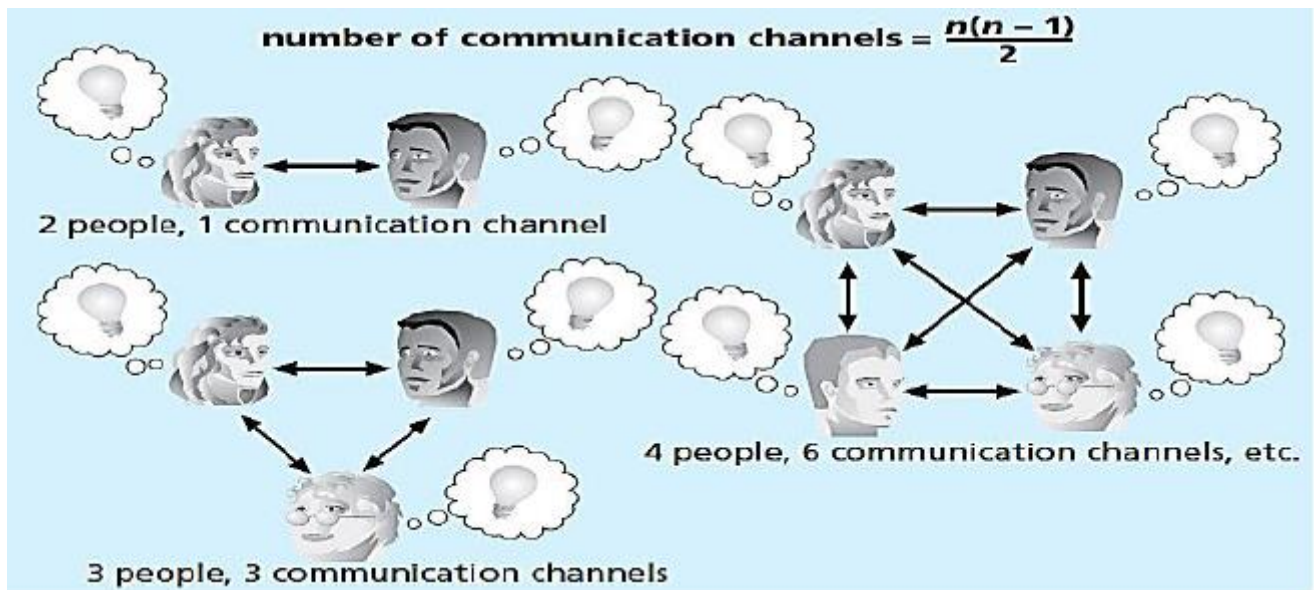


Figure 7-1 The impact of the number of people on communication channels

Good communicators consider many factors before deciding how to distribute information, including the size of the group, the type of information, and the appropriate communication medium. People often send e-mail messages that are quickly written and therefore not as carefully planned as they should be.

Communication is the oil that keeps everything working properly.

7.4. Administrative Closure

It involves bringing to completion all internal aspects of the project concerning team members, management, other stakeholders, financials, and equipment. The following processes ensure that no loose ends are left dangling:

- *Deliverable Turnover-Verification and Acceptance* – in this step, deliverables are reviewed and tested against previously determined requirements and are accepted by the customer with a formal sign-off.
- *Post completion Data* – in this step, you determine any variances in the schedule, cost, and scope.
- *Follow-up Maintenance and Warranties* – if applicable, hand off any hardware, software, or other equipment and review the coverage on warranties and the maintenance requirements.
- *Team Member Performance Reporting* – the project manager provides information to functional management on the performance of project team members during the life of the project.
- *Financials* – ensure that all expenses are paid and project budgets are closed. Generate the necessary financial reports.
- *Releasing Staff* – ensure a smooth transition for all staff to new assignments. Notify functional managers with sufficient lead time so that meaningful work assignments can be made.

- *Formal Closing Report* – prepare a summary of the information above, including any open issues, and distribute it to appropriate stakeholders.

7.5. Suggestions for Improving Project communications

You have seen that good communication is vital to the management and success of IT projects; you have also learned that project communications management can ensure that essential information reaches the right people at the right time, that feedback and reports are appropriate and useful, and that there are formalized processes for planning, managing, and controlling communications. This section highlights a few areas that all project managers and project team members should consider in their quests to improve project communications. The guidelines for improving project communication are:

- Developing better communication skills,
- Running effective meetings,
- Using e-mail, instant messaging, texting, and collaborative tools effectively,
- Using templates for project communications

7.7. Contract Closure:

A contract is closed upon reaching the end of the contract, or when a contract is terminated before his work is completed (usually by the buyer – if the work is no longer required, or if the work performed is not acceptable due to quality or other reasons). The seller may still need to be compensated for the work completed, as governed by the clauses in the contract. In multi-phase projects, Close Procurements closes the appropriate contract(s) for that phase of the project. Thus, the Close Procurements process may be repeated many times in a project.

Unresolved claims may be subject to litigation after Close Procurements is completed. A complete set of indexed contract documentation, including the closed contract, is prepared for inclusion in the final project files – Output of Close Procurements.

Key Difference between Administrative and Contract Closure:

- Procurement Closure or Contract Closure is done before the project can be closed completely.
- Procurement Closure may be done multiple times (once for each contract) during the lifecycle of a project.
- Administrative Closure is only done once per phase, or for the entire project. Project closure is not complete without procurement closure.
- Product acceptance is carried out using the Close Project or Phase process.

- The contracts are closed using the Close Procurement process. **A. Suggestions for Improving Project communications**

Guidelines for improving project communications are

- a. Developing better communication skills,
- b. Running effective meetings,
- c. Using e-mail, instant messaging, texting, and collaborative tools effectively, and
- d. Using templates for project communications.

6.6 Chapter seven review questions

1. What courses in your educational program address the development of communications skills?
What skills do you think employers are looking for? Do you think there should be more emphasis on communications skills in your degree program?
2. Discuss the importance of understanding tone of voice and body language in comprehending the meaning of what people say. Give examples of how the same words said in different ways have totally different meanings.
3. What items should a communications management plan address? How can a stakeholder analysis assist in preparing and implementing parts of this plan?
4. Discuss the advantages and disadvantages of different ways of distributing project information.
5. What are some of the ways to create and distribute project performance information?
6. Discuss the advantages and disadvantages of telecommuting.
7. Explain why you agree or disagree with some of the suggestions in this chapter for improving project communications, such as creating a communications management plan, stakeholder analysis, or performance reports. What other suggestions do you have?

CHAPTER EIGHT

PROJECT RISK MANAGEMENT

Activity

- how do you respond to risk in a project?
- Do you think all risks have a negative impact on a project?

8.1. The Importance of Project Risk Management

Project risk management is the art and science of identifying, analyzing, and responding to risk throughout the life of a project and in the best interests of meeting project objectives. A frequently overlooked aspect of project management, risk management can often result in significant improvements in the ultimate success of projects. Risk management can have a positive impact on selecting projects, determining their scope, and developing realistic schedules and cost estimates. It helps project stakeholders understand the nature of the project, involves team members in defining strengths and weaknesses and helps to integrate the other project management knowledge areas.

Good project risk management often goes unnoticed, unlike crisis management, which indicates an obvious danger to the success of a project. The crisis, in turn, receives the intense interest of the entire project team. Resolving a crisis has much greater visibility, often accompanied by rewards from management, than successful risk management. In contrast, when risk management is effective, it results in fewer problems, and for the few problems that exist, it results in more expeditious resolutions. It may be difficult for outside observers to tell whether risk management or luck was responsible for the smooth development of a new system, but project teams always know that their projects worked out better because of good risk management. Managing project risks takes dedicated, talented professionals.

Risk is “the possibility of loss or injury.” This definition highlights the negativity often associated with risk and points out that uncertainty is involved. Project risk management involves understanding potential problems that might occur on the project and how they might impede project success. This definition refers to this type of risk as a negative risk or threat. However, there are also positive risks or opportunities, which can result in good outcomes for a project. A general definition of project risk, therefore, is an uncertainty that can have a negative or positive effect on meeting project objectives

Several risk experts suggest that organizations and individuals should strive to find a balance between risks and opportunities in all aspects of projects and their personal lives. The idea of striving for balance suggests that different organizations and people have different attitudes toward risk. These attitudes are based on two themes:

a. Risk Appetite is the degree of uncertainty an entity is willing to take on, in anticipation of a reward.

b. Risk Tolerance is the maximum acceptable deviation an entity is willing to accept on the project or business objectives as the potential impact. ... The project may be accepted if the risks are within tolerances and are in balance with the rewards that may be gained by taking the risks.” Some organizations or people have a neutral tolerance for risk, some have an aversion to risk, and others are risk-seeking. These three preferences are part of the utility theory of risk.

Risk utility is the amount of satisfaction or pleasure received from a potential payoff. The following Fig. shows the basic difference between risk-averse, risk-neutral, and risk-seeking preferences.

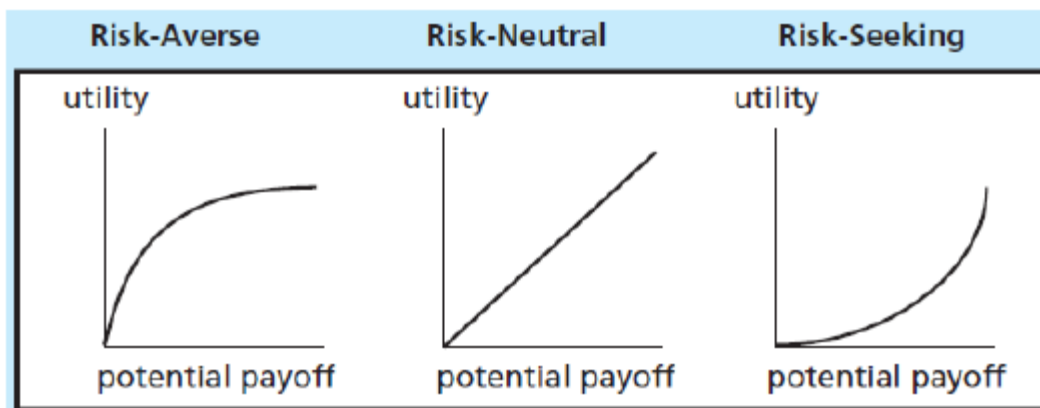


Figure 8-1 risk-

averse, risk-neutral and risk-seeking

8.2. Common Sources of Risk in IS project

- **Market Risk** – if the IT project will create a new product or service, will it be useful to the organization or marketable to others? Will users accept and use the product or service? Will someone else create a better product or service faster, making the project a waste of time and money?
- **Financial Risk** – can the organization afford to undertake the project? How confident are stakeholders in the financial projections? Will the project meet NPV, ROI, and payback estimates? If not, can the organization afford to continue the project? Is this project the best way to use the organization's financial resources?

- **Technology Risk** – is the project technically feasible? Will it use mature, leading-edge, or bleeding-edge technologies? When will decisions be made on which technology to use? Will hardware, software, and networks function properly? Will the technology be available in time to meet project objectives? Could the technology be obsolete before a useful product can be created? You can also break down the technology risk category into hardware, software, and network technology if desired.
- **People Risk** – does the organization have people with the appropriate skills to complete the project successfully? If not, can the organization find such people? Do people have the proper managerial and technical skills? Do they have enough experience? Does senior management support the project? Is there a project champion? Is the organization familiar with the sponsor or customer for the project? How good is the relationship with the sponsor or customer?
- **Structure/Process Risk** – what degree of change will the new project introduce into user areas and business procedures? How many distinct user groups does the project need to satisfy? How many other systems does the new project or system need to interact with? Does the organization have processes in place to complete the project successfully?

Review the work breakdown structure (WBS) for a project to see if there might be specific risks by WBS categories. A risk breakdown structure is a hierarchy of potential risk categories for a project.

8.3. Risk Identification

Identifying risks is the process of understanding what potential events might hurt or enhance a particular project. It is important to identify potential risks early, but you must also continue to identify risks based on the changing project environment. Also, remember that you cannot manage risks if you do not identify them first. By understanding common sources of risks and reviewing a project's planning documents (for risk, cost, schedule, quality, and human resource management), activity cost and duration estimates, the scope baseline, stakeholder register, project documents, procurement documents, enterprise environmental factors, and organizational process assets, project managers and their teams can identify any potential risks.

Another consideration for identifying risks is the likelihood of advanced discovery, which is often viewed at a program level rather than a project level.

Suggestions for Identifying Risks

Four common techniques:

- **Brainstorming** – is a technique by which a group attempts to generate ideas or find a solution for a specific problem by amassing ideas spontaneously and without judgment.
- **The Delphi Technique** – is an approach to gathering information that helps prevent some of the negative group effects found in brainstorming. The basic concept of the Delphi technique is to derive a consensus among a panel of experts who make predictions about future developments. It is a systematic, interactive forecasting procedure based on independent and anonymous input regarding future events. The Delphi technique uses repeated rounds of questioning and written responses, including feedback to responses in earlier rounds, to take advantage of group input while avoiding the possible biasing effects of oral panel deliberations. To use the Delphi technique, you must select a panel of experts for the particular area in question.
- **Interviewing** – is a fact-finding technique for collecting information in face-to-face, phone, e-mail, or instant-messaging discussions. Interviewing people with similar project experiences is an important tool for identifying potential risks.
- **Root Cause Analysis** – often results in identifying even more potential risks for a project.

The Risk Register

The main output of risk identification is a list of identified risks and other information needed to begin creating a risk register. A risk register is a document that contains the results of various risk management processes; it is often displayed in a table or spreadsheet format. A risk register is a tool for documenting potential risk events and related information. Risk events refer to specific, uncertain events that may occur to the detriment or enhancement of the project.

Example:

Negative risk events might include the performance failure of a product created as part of a project, delays in completing work as scheduled, increases in estimated costs, supply shortages, litigation against the company, and strikes.

Positive risk events include completing work sooner or cheaper than planned, collaborating with suppliers to produce better products, and good publicity resulting from the project.

The sample risk register:

No.	Rank	Risk Description	Category	Root	Potential	Risk
				Cause	Triggers	Responses
Owner	Probability	Impact	Status			
R44	1					
R21	2					
R7	3					

Figure 8-2 sample risk register

The main headings often included in the register are

- An identification number for each risk event: The project team may want to sort by risk events or quickly search for specific risk events, so they need to identify each risk with a unique descriptor, such as an identification number.
- A rank for each risk event: The rank is usually a number, with 1 representing the highest risk.
- The name of the risk event: Example names include defective server, late completion of testing, reduced consulting costs, and good publicity.
- A description of the risk event: Because the name of a risk event is often abbreviated, it helps to provide a more detailed description. For example, reduced consulting costs might be expanded in the description to explain that the organization might be able to negotiate a lower cost for a particular consultant because she enjoys working with your company.
- The category under which the risk event falls: For example, defective server might fall under the broader category of technology or hardware technology.
- The root cause of the risk: The root cause of the defective server might be a defective power supply.
- Triggers for each risk: Triggers are indicators or symptoms of actual risk events. For example, cost overruns on early activities may be symptoms of poor cost estimates. Defective products may be symptoms of a low-quality supplier. Documenting potential risk symptoms for projects also helps the project team identify more potential risk events.
- Potential responses to each risk: A potential response to the defective server might be to include a clause in the supplier's contract to replace the server within a certain time period at a negotiated cost.
- The risk owner or person who will take responsibility for the risk: For example, a certain person might be in charge of any server-related risk events and managing response strategies.
- The probability of the risk occurring: There might be a high, medium, or low probability of a certain risk event.

- The impact to the project if the risk occurs: There might be a high, medium, or low impact to project success if the risk event actually occurs. A defective server might have a high impact on successfully completing a project on time.
- The status of the risk: Did the risk event occur? Was the response strategy completed? Is the risk no longer relevant to the project?

Example:

- *No.:* R44
- *Rank:* 1
- *Risk:* New customer
- *Description:* We have never done a project for this organization before and don't know too much about them. One of our company's strengths is building good customer relationships, which often leads to further projects with that customer. We might have trouble working with this customer because they are new to us.
- *Category:* People risk
- *Root cause:* We won a contract to work on a project without really getting to know the customer.
- *Triggers:* The project manager and other senior managers realize that we don't know much about this customer and could easily misunderstand their needs or expectations.
- *Potential responses:* Make sure the project manager is sensitive to the fact that this is a new customer and takes the time to understand them. Have the PM set up a meeting to get to know the customer and clarify their expectations. Have Cliff attend the meeting, too.
- *Risk owner:* Our project manager
- *Probability:* Medium
- *Impact:* High
- *Status:* PM will set up the meeting within the week.

After identifying risks, the next step is to understand which risks are most important by performing qualitative risk analysis.

8.4. Risk Quantification

There are two types of risk quantification.

a. Performing Qualitative Risk Analysis

Qualitative risk analysis involves assessing the likelihood and impact of identified risks to determine their magnitude and priority. It uses a probability/ impact matrix to produce a prioritized list of risks, the Top Ten Risk Item Tracking technique to produce an overall ranking for project risks and to track trends in qualitative risk analysis and expert judgment in performing risk analysis.

b. Performing Quantitative Risk Analysis

Quantitative risk analysis often follows qualitative risk analysis, yet both processes can be done together or separately. On some projects, the team may only perform qualitative risk analysis. The nature of the project and availability of time and money affect which risk analysis techniques are used. Large, complex projects involving leading-edge technologies often require extensive quantitative risk analysis. The main techniques for quantitative risk analysis include data gathering, analysis and modeling techniques, and expert judgment. Data gathering often involves interviewing experts and collecting probability distribution information.

8.5. Risk Response Development and Control

After an organization identifies and quantifies risks, it must develop an appropriate response to them. Developing a response to risks involves developing options and defining strategies for reducing negative risks and enhancing positive risks. The four basic response strategies for negative risks are:

- ***Risk avoidance or eliminating*** a specific threat, usually by eliminating its causes. Of course, not all risks can be eliminated, but specific risk events can be. For example, a project team may decide to continue using a specific piece of hardware or software on a project because the team knows it works. Other products that could be used on the project may be available, but if the project team is unfamiliar with them, they could cause significant risk. Using familiar hardware or software eliminates this risk.
- ***Risk acceptance or accepting*** the consequences if a risk occurs. For example, a project team planning a big project review meeting could take an active approach to risk by having a contingency or backup plan and contingency reserves if the team cannot get approval for a specific meeting site. On the other hand, the team could take a passive approach and accept whatever facility the organization provides.
- ***Risk transference or shifting*** the consequence of a risk and responsibility for its management to a third party. For example, risk transference is often used in dealing with financial risk exposure. A project team may purchase special insurance or warranty protection for specific hardware needed for a project. If the hardware fails, the insurer must replace it within a specified period of time.
- ***Risk mitigation or reducing*** the impact of a risk event by reducing the probability of its occurrence. Suggestions for reducing common sources of risk on IT projects were provided at the beginning of this chapter. Other examples of risk mitigation include using proven technology, having competent

project personnel, using various analysis and validation techniques, and buying maintenance or service agreements from subcontractors.

The four basic response strategies for positive risks are:

- **Risk exploitation or doing** whatever you can to make sure the positive risk happens. For example, suppose that Cliff's company funded a project to provide new computer classrooms for a nearby school in need. They might select one of their top project managers to organize news coverage of the project, write a press release, or hold some other public event to ensure that the project produces good public relations for the company, which could lead to more business.
- **Risk sharing or allocating** ownership of the risk to another party. Using the same example of implementing new computer classrooms, the project manager could form a partnership with the school's principal, school board, or parent-teacher organization to share responsibility for achieving good public relations for the project. On the other hand, the company might partner with a local training firm that agrees to provide free training for all of the teachers on how to use the new computer classrooms.
- **Risk enhancement or changing** the size of the opportunity by identifying and maximizing key drivers of the positive risk.
- **Risk acceptance** also applies to positive risks when the project team does not take any actions toward a risk.

Risk response strategies often include identification of residual and secondary risks as well as contingency plans and reserves, as described earlier. Residual risks are risks that remain after all of the response strategies have been implemented. For example, even though a stable hardware product may have been used on a project, there may still be a risk that it fails to function properly. Secondary risks are a direct result of implementing a risk response. For example, using the more stable hardware may have caused a risk of peripheral devices failing to function properly.

Controlling Risks

Controlling risks involves executing the risk management processes to respond to risk events and ensuring that risk awareness is an ongoing activity performed by the entire project team throughout the entire project. Project risk management does not stop with the initial risk analysis. Identified risks may not materialize, or their probabilities of occurrence or loss may diminish. Previously

identified risks may be determined to have a greater probability of occurrence or a higher estimated loss value. Similarly, new risks will be identified as the project progresses. Newly identified risks need to go through the same process as those identified during the initial risk assessment. A redistribution of resources devoted to risk management may be necessary because of relative changes in risk exposure.

CHAPTER NINE

PROJECT PROCUREMENT MANAGEMENT

9.1. Importance of Project Procurement Management

Activity

- What things should be considered in procurement management as a whole?
- Describe types of contracts.

Procurement means acquiring goods and services from an outside source. The term procurement is widely used in government; many private companies use the terms purchasing and outsourcing. Organizations or individuals who provide procurement services are referred to as suppliers, vendors, contractors, subcontractors, or sellers; of these terms, suppliers is the most widely used. Many IT projects involve the use of goods and services from outside the organization. The Project Management Institute defines an outside source as a source outside the project team, so the same organization can be a supplier to the project team, or the project team can be a supplier to another group in the organization.

In fact, many IT departments in organizations are in direct competition with outside vendors, and they are subject to the same kind of requirements definition, statements of work, and bids. The rules and methods of sound project procurement practices are good to follow regardless of who provides the services to whom.

9.2. Importance of project procurement management

Outsourcing has become a hot topic for research and debate, especially the implications of outsourcing to other countries, which is called offshoring. Deciding whether to outsource, what to outsource, and how to outsource are important topics for many organizations throughout the world. Many organizations are turning to outsourcing to accomplish the following:

- **Access skills and technologies.** Organizations can gain access to specific skills and technologies when they are required by using outside resources.
- **Reduce both fixed and recurrent costs.** Outsourcing suppliers often can use economies of scale that may not be available to the client alone, especially for hardware and software.
- **Allow the client organization to focus on its core business.** Most organizations are not in business to provide IT services, yet many have spent valuable time and resources on IT functions

when they should have focused on core competencies such as marketing, customer service, and new product design.

- **Provide flexibility.** Outsourcing to provide extra staff during periods of peak workloads can be much more economical than trying to staff entire projects with internal resources. Many companies cite better flexibility in staffing as a key reason for outsourcing.
- **Increase accountability.** A well-written contract – a mutually binding agreement that obligates the seller to provide specified products or services and obligates the buyer to pay for them – can clarify responsibilities and sharpen focus on key deliverables of a project. Because contracts are legally binding, there is more accountability for delivering the work as stated in the contract.

Outsourcing can also cause problems in other areas for companies and nations as a whole.

Example:

] Many people in Australia are concerned about outsourcing software development. The Australian Computer Society says sending work offshore may lower the number of students entering IT courses, deplete the number of skilled IT professionals, and diminish the nation's strategic technology capability. Another issue is security, which encompasses the protection of intellectual property, integrity of data, and the reliability of infrastructure in offshore locations.”

The success of many IT projects that use outside resources is often due to good project procurement management. Project procurement management includes the processes required to acquire goods and services for a project from outside the performing organization. Organizations can be either the buyer or seller of products or services under a contract or other agreement. There are four main processes in project procurement management:

- i. Planning procurement management** involves determining what to procure and when and how to do it. In procurement planning, one must decide what to outsource, determine the type of contract, and describe the work for potential sellers. Sellers are providers, contractors, or suppliers who provide goods and services to other organizations. Outputs of this process include a procurement management plan, procurement statements of work, procurement documents, source selection criteria, make-or-buy decisions, change requests, and project documents updates.
- ii. Conducting procurements** involves obtaining seller responses, selecting sellers, and awarding contracts. Outputs include selected sellers, agreements, resource calendars, change requests, and updates to the project management plan and other project documents.
- iii. Controlling procurements** involves managing relationships with sellers, monitoring contract performance, and making changes as needed. The main outputs of this process include work

performance information, change requests, and updates to the project management plan, project documents, and organizational process assets.

iv. Closing procurements involves completion and settlement of each contract or agreement, including resolution of any open items. Outputs include closed procurements and organizational process assets updates.

9.3. Procurement Planning

Planning procurements involves identifying which project needs can best be met by using products or services outside the organization. It involves deciding whether to procure, how to procure, what to procure, how much to procure, and when to procure. An important output of this process is the make-or-buy decision, in which an organization decides whether it should make certain products and perform certain services inside the organization, or if it is better to buy those products and services from an outside organization. If there is no need to buy products or services from outside the organization, then further procurement management is not needed.

Inputs needed for planning procurements include the project management plan, requirements documentation, the risk register, activity resource requirements, the project schedule, activity cost estimates, the stakeholder register, enterprise environmental factors, and organizational process assets, such as types of contracts.

Types of Contracts

There are basically three broad categories of contracts are fixed price or lump sum, cost reimbursable, and time and material. A single contract can actually include all three of these categories if it makes sense for a particular procurement.

Example:

] You could have a contract with a seller that includes purchasing specific hardware for a fixed price or lump sum, some services that are provided on a cost reimbursable basis, and other services that are provided on a time-and-material basis. Project managers and their teams must understand and decide which approaches to use to meet their project needs. It is also important to understand when and how to take advantage of unit pricing in contracts.

A. Fixed-Price or Lump-Sum contracts involve a fixed total price for a well-defined product or service. The buyer incurs little risk in this situation because the price is predetermined. The sellers often pad their estimate to reduce their risk, although they realize their price must still be competitive.

Example:

A company could award a fixed-price contract to purchase 100 laser printers with a certain print resolution and print speed to be delivered to one location within two months. In this example, the product and delivery date are well defined. Several sellers could create fixed-price estimates for completing the job.

B. Cost-reimbursable contracts involve payment to the supplier for direct and indirect actual costs. Direct costs can be directly related to producing a project's products and services. Normally, these costs can be traced back to a project in a cost-effective way. Indirect costs are not directly related to the products or services of the project, but they are indirectly related to performing the project. Normally, these costs cannot be traced back to the project in a cost-effective way.

Example:

Direct costs include the salaries for people working directly on a project and hardware or software purchased for a specific project. Indirect costs include the cost of providing a workspace with electricity and an employee cafeteria. Indirect costs are often calculated as a percentage of direct costs.

Cost-reimbursable contracts often include fees, such as a profit percentage or incentives for meeting or exceeding selected project objectives. These contracts are often used for projects that include providing goods and services that involve new technologies. The buyer absorbs more of the risk with cost-reimbursable contracts than with fixed-price contracts.

C. Time and Material (T&M) contracts are a hybrid of fixed-price and cost-reimbursable contracts.

Example:

An independent computer consultant might have a contract with a company based on a fee of \$80 per hour for services, plus a fixed price of \$10,000 for providing specific project materials. The materials fee might also be based on approved receipts for purchasing items, with a ceiling of \$10,000. The consultant would send an invoice to the company each week or month; the invoice

would list the materials fee, the number of hours worked, and a description of the work produced. This type of contract is often used for required services when the work cannot be specified clearly and total costs cannot be estimated in a contract.

Unit pricing can also be used in various types of contracts to require the buyer to pay the supplier a predetermined amount per unit of product or service. The total value of the contract is a function of the quantities needed to complete the work.

9.4. Solicitation (Procurement Document)

It involves preparing the documents needed for potential sellers to provide their responses and determining the evaluation criteria for the contract award. The project team often uses standard forms and expert judgment as tools to help create relevant procurement documents and evaluation criteria. There are two common procurement documents. They are

1) A Request for Proposal (RFP) is a document used to solicit proposals from prospective suppliers. A proposal is a document prepared by a seller when there are different approaches for meeting buyer needs.

Example:

If an organization wants to automate its work practices or solve a business problem, it can write and issue an RFP so suppliers can respond with proposals. Suppliers might propose various hardware, software, and networking solutions to meet the organization's needs. Selections of winning sellers are often made on a variety of criteria, not just the lowest price. Developing an RFP is often a time-consuming process. Organizations must plan properly to ensure that they adequately describe what they want to procure, what sellers to include in their proposals, and how they will evaluate proposals.

2) A Request for Quote (RFQ) is a document used to solicit quotes or bids from prospective suppliers. A bid, also called a tender or quote (short for quotation), is a document prepared by sellers to provide pricing for standard items that the buyer has clearly defined. Organizations often use an RFQ for solicitations that involve specific items.

Example:

If a company wanted to purchase 100 personal computers with specific features, it might issue an RFQ to potential suppliers. RFQs usually do not take nearly as long to prepare as RFPs, nor do responses to RFQs. Selections are often based on the lowest bid.

An RFP usually includes sections on the purpose of the RFP, the organization's background, requirements, environments, statement of work (SOW) with schedule, and required deliverables with the schedule. Almost all mutually binding agreements or contracts include an SOW. Solicitation is the process of preparing to solicit sellers to provide products the project needs. There are three inputs to the solicitation.

a) Procurement Management Plan

It is a document that describes how the procurement processes will be managed, from developing documentation for making outside purchases or acquisitions to contract closure. Like other project plans, the contents of the procurement management plan will vary with project needs.

b) Statement of Work (SOW)

It is a description of the work required for procurement. It should be clear, concise, and as complete as possible. It should describe all services required and include performance reporting. Many organizations use samples and templates to generate SOW. Fig. provides a basic outline or template for a contract SOW.

Statement of Work (SOW)	
I.	Scope of Work: Describe the work to be done in detail. Specify the hardware and software involved and the exact nature of the work.
II.	Location of Work: Describe where the work must be performed. Specify the location of hardware and software and where the people must perform the work.
III.	Period of Performance: Specify when the work is expected to start and end, working hours, number of hours that can be billed per week, where the work must be performed, and related schedule information.
IV.	Deliverables Schedule: List specific deliverables, describe them in detail, and specify when they are due.
V.	Applicable Standards: Specify any company or industry-specific standards that are relevant to performing the work.
VI.	Acceptance Criteria: Describe how the buyer organization will determine if the work is acceptable.
VII.	Special Requirements: Specify any special requirements such as hardware or software certifications, minimum degree or experience level of personnel, travel requirements, and so on.

Figure 9-1 Statement of Work (SOW) template

c) Other planning outputs – other details within the project plan, such as the schedules, estimates, constraints, and assumptions, are referenced as their values may have a direct influence on the solicitation process.

9.5. Source Selection

Source selection criteria are used to score seller proposals as part of the procurement process. The criteria could be subjective or objective and are generally specified as part of the procurement documents. Examples of source selection criteria include life cycle cost, the technical capability of the seller, past performance of sellers, references, and intellectual property rights.

Inputs to Source Selection

1. **Evaluation criteria.** Evaluation criteria may include samples of the supplier's previously produced products/services for the purpose of providing a way to evaluate their capabilities and quality of products. They also may include a review of the supplier's history with the contracting organization.

Tools and Techniques for Source Selection

1. Contract negotiation. Contract negotiation involves clarification and mutual agreement on the structure and requirements of the contract prior to the signing of the contract. To the extent possible, the final contract language should reflect all agreements reached. Subjects covered generally include, but are not limited to, responsibilities and authorities, applicable terms and law, technical and business management approaches, contract financing, and price. For complex procurement items, contract negotiation may be an independent process with inputs (e.g., an issues or open items list) and outputs (e.g., memorandum of understanding) of its own.

2. Weighting system. A weighting system is a method for quantifying qualitative data to minimize the effect of personal prejudice on source selection. Most such systems involve

- Assigning a numerical weight to each of the evaluation criteria,
- Rating the prospective sellers on each criterion,
- Multiplying the weight by the rating, and
- Totaling the resultant products to compute an overall score.

3. Screening system. A screening system involves establishing minimum requirements of performance for one or more of the evaluation criteria. For example, a prospective seller might be required to propose a project manager who has specific qualifications—for example, a PMP®—before the remainder of the proposal would be considered.

4. Independent estimates. For many procurement items, the procuring organization may prepare its own independent estimates as a check on proposed pricing. Significant differences from these estimates may be an indication that the SOW was not adequate, or that the prospective seller either misunderstood or failed to respond fully to the SOW. Independent estimates are often referred to as should cost estimates.

Outputs from Source Selection

1. **Contract.** A contract is a mutually binding agreement that obligates the seller to provide the specified product and obligates the buyer to pay for it. A contract is a legal relationship subject to remedy in the courts. The agreement may be simple or complex, usually (but not always) reflecting the simplicity or complexity of the product. Contracts may be called, among other names, a contract, an agreement, a subcontract, a purchase order, or a memorandum of understanding. Most organizations have documented policies and procedures specifically defining who can sign such agreements on behalf of the organization, typically called a delegation of procurement authority.

9.6. Contract Administration

This contract consists of two parts. They are

i. Conducting Procurement

It involves deciding whom to ask to do the work, sending appropriate documentation to potential sellers, obtaining proposals or bids, selecting a seller, and awarding a contract. Prospective sellers do some of the work in this process, normally at no cost to the buyer or project. The buying organization is responsible for advertising the work, and for large procurements, the organization often holds some sort of bidders' conference to answer questions about the job. Two of the main outputs of this process are a selected seller and procurement contract award. Organizations can advertise to procure outside goods and services in many different ways.

ii. Controlling Procurement

It ensures that the seller's performance meets contractual requirements. The contractual relationship is a legal relationship, which means it is subject to state and federal contract laws. It is very important that appropriate legal and contracting professionals be involved in writing and administering contracts.

It is critical that project managers and team members watch for constructive change orders. Constructive change orders are oral or written acts or omissions by someone with actual or apparent authority that can be construed to have the same effect as a written change order.

9.7. Closing Procurement

The final process in project procurement management is closing procurements, which is sometimes referred to as contract closure (close-out). Contract closure involves the completion and settlement of contracts and resolution of any open items. The project team should determine if all work required in each contract was completed correctly and satisfactorily. The team should also update records to reflect final results and archive information for future use.

Tools to assist in contract closure include procurement audits, negotiated settlements, and a record management system. Procurement audits are often done during contract closure to identify lessons learned in the entire procurement process. Organizations should strive to improve all of their business processes, including procurement management. Ideally, all procurements should end in a negotiated settlement between the buyer and seller. If negotiation is not possible, then some type of alternate dispute resolution such as mediation or arbitration can be used; if all else fails, litigation in courts can be used to settle contracts. A records management system provides the ability to easily organize, find, and archive procurement-related documents. It is often an automated system, or at least partially automated because a large amount of information can be related to project procurement.

Outputs from contract closure include closed procurements and updates to organizational process assets. The buying organization often provides the seller with formal written notice that the contract has been completed. The contract itself should include requirements for formal acceptance and closure.

9.8. Chapter nine review questions

1. List five reasons why organizations outsource. When should an organization choose not to outsource? Why are some organizations beginning to the rural sources?
2. Explain the make-or-buy decision process and describe how to perform the financial calculations in the simple lease-or-buy example provided in this chapter. What are the main types of contracts if you decide to outsource? What are the advantages and disadvantages of each?
3. Do you think many IT professionals have experience writing RFPs and evaluating proposals for IT projects? What skills would be useful for these tasks?
4. How do organizations decide whom to send RFPs or RFQs?
5. How can organizations use a weighted scoring model to evaluate proposals as part of seller selection?
6. List two suggestions for ensuring adequate change control on projects that involve outside contracts.
7. What is the main purpose of a procurement audit?

CHAPTER TEN

PROJECT MANAGEMENT PROCESS GROUPS

10.1 Introduction to Project Management Process Groups

Activity

- Discuss the project management process.
- What should be done at the first and last stage of a project management process?

There are five fundamental process groups. These five Process Groups are the building blocks of every project management life-cycle (PMLC). In the simplest of cases, the Process Groups will each be completed once and, in the sequence, listed below. In more complex situations, some or all of them might be repeated a number of times.

10.2. Phases of Project Management Process Groups

The project management process groups include:

- Initiating processes
- Planning processes
- Executing processes
- Monitoring and controlling processes
- Closing processes

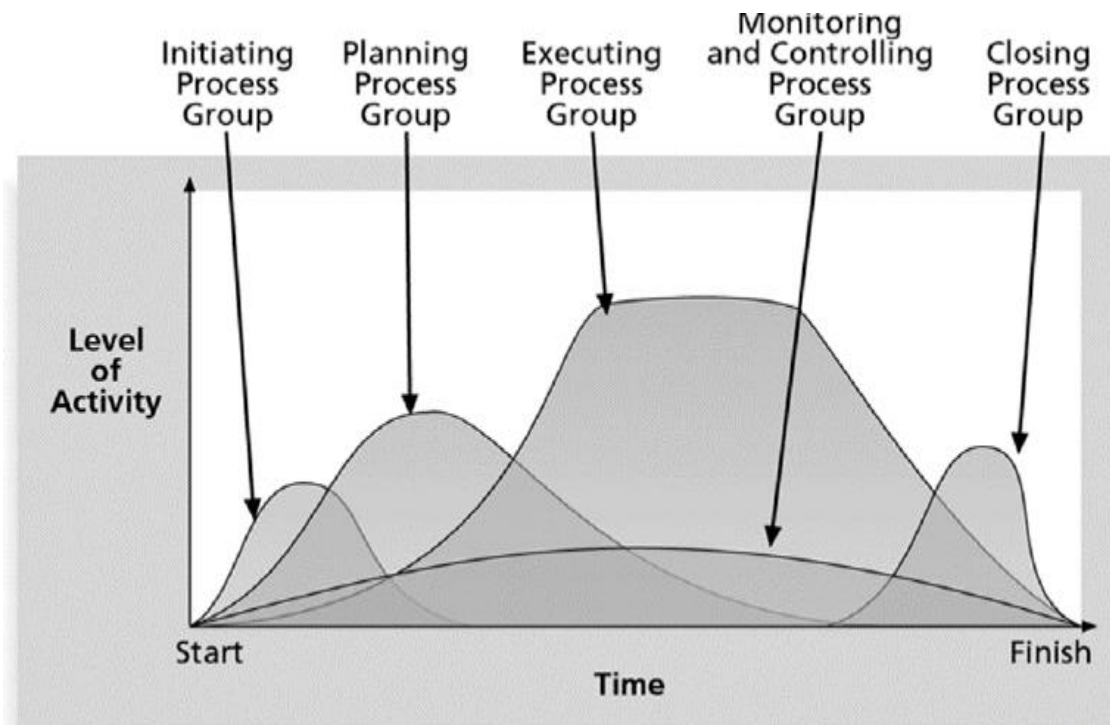


Figure 10-1 Phases of Project Management Process Groups

10.2.1. Project Initiation → Initiating Process Group

The purpose of Project Initiation is to evaluate proposed projects and to reach a consensus on the projects to be selected. During this, the Project Charter is presented, and the strength of projects Business Case and the viability of the proposed solution are evaluated. A determination is made as to whether the project is consistent with the institutions business and/or strategic plan, and if the Project Planning (High Level) budget is affordable.

Project Initiation Phase

It is the critical phase within the project life-cycle. It is also called the project pre-planning phase and about stating the basic characteristics of the project. To successfully initiate a project, you need to which basics steps are required to carry out to develop a business case, undertake a feasibility study, develop a project charter and others. The basic steps of the project initiation phase:

a. Create a Business Case – A business case document is the formal start of the project when the project sponsor (or the project initiator) gives a description of the business problem/opportunity. The project is to be initiated to address the problem or provide alternative solutions. The business case document will include the business problem and potential costs associated with the project implementation.

b. Make a Feasibility Study – A feasibility study is a research conducted to determine whether the

project can be accomplished and to identify the likelihood of the alternative solutions. The feasibility study investigates whether the benefits stated in the business case can be delivered. It also depicts relationship of business costs with the project solutions.

c. Develop a Project Charter – Once the business problem/opportunity and possible solutions have been identified, your next step is to develop a project charter which is the critical document of the initiation phase. The project charter essentially describes what your project sets out to solve the business problem and what the boundaries of the project will be. It specifies the project vision, goals & objectives, scope & boundaries, deliverables & expectations, project organization and an implementation plan.

d. Assign Project Management Team. The project charter is developed so you can proceed with identifying human resources required for delivering the project and achieving its goals. You will need to appoint the management team which will take the primary responsibility for the planning and implementation of your project. The Project Committee should be established and the project manager should be assigned. Then the project manager will work on recruiting the project team and making project assignments. Once the team is recruited and assignments are made, the project initiation phase is almost finished; only one step is ahead.

e. Perform Project Review. Your last step to take through the project initiation phase is about reviewing your project. A review stage should be conducted to ensure that the project is successfully initiated – this means all of the initiation activities are completed. Once the project is reviewed, the project planning phase will be.

10.2.2. Project Planning → Planning Process Group

Planning is often the most difficult and unappreciated process in project management. Because planning is not always used to facilitate action, many people view planning negatively. The main purpose of project plans, however, is to guide project execution. To guide execution, plans must be realistic and useful, so a fair amount of time and effort must go into the planning process; people who are knowledgeable with the work need to plan the work.

10.2.3. Project Executing → Executing Process Group

Executing the project involves taking the actions necessary to ensure that activities in the project plan are completed. It also includes work required to introduce any new hardware, software, and procedures into normal operations. The products of the project are created during project execution, and it usually takes the most resources to accomplish this process. Many project sponsors and customers focus on deliverables related to providing the products, services, or results desired from

the project. It is also important to document change requests and prepare updates to planning documents as part of execution.

10.2.4. Project Monitoring and Controlling → Monitoring and Controlling Process Group

Monitoring and controlling are the process of measuring progress toward project objectives, monitoring deviation from the plan, and taking corrective action to match progress with the plan. It is done throughout the life of a project. It also involves nine of the 10 project management knowledge areas.

On the project management intranet site project, there were several updates to the project management plan to reflect changes made to the project scope, schedule, and budget. Erica and other project team members took corrective action when necessary. For example, when they were not getting many responses to their survey, Erica asked Joe for help. When Cindy had trouble negotiating with a supplier, she got help from another senior consultant who had worked with that supplier in the past. Erica also had to request more funds for that part of the project. Project team members submitted a brief progress report every Friday to show work performance information.

10.2.5. Project Closing → Closing Process Group

The closing process involves gaining stakeholder and customer acceptance of the final products and services and then bringing the project or project phase to an orderly end. It includes verifying that all of the deliverables are complete, and it often includes a final project report and presentation. Even though many IT projects are canceled before completion, it is still important to formally close any project and reflect on what can be learned to improve future projects. It is also important to plan for and execute a smooth transition of the project into normal operations of the company. Most projects produce results that are integrated into the existing organizational structure.

10.3. Chapter ten review questions

1. Briefly describe what happens in each of the five project management process groups (initiating, planning, executing, monitoring and controlling, and closing). What types of activities occur before initiating a project?
2. Approximately how much time do good project managers spend on each process group, and why?
3. Why do organizations need to tailor project management concepts, such as those found in the PMBOK® Guide, to create their own methodologies?
4. What are some of the key outputs of each process group?
5. What are some of the typical challenges project teams face during each of the five process groups?
You can frame your discussion based on a project described in one of the feature boxes in this chapter (for example, the What Went Right? or What Went Wrong? feature). You can also frame your discussion on one of PMI's Project of the Year Award winners, or on a well-known project failure like the Denver International Airport baggage handling system.



Wolaita Sodo University
School Of Informatics
Department of Information Technology

Course Title: Advanced programming

Course code: ITec3058

Comiled by Nuhamin Nigussie

Reviewd by Tadele Shasheho

CHAPTER ONE

1 Multithreading

1.1 Introduction

Performing only one task at a time usually difficult to do in a complex world. It would be nice if we could “**do one thing at a time**” and “**do it well**”, but that is simply not how the world works. The human body performs a great variety of operations **in parallel**, or as we will say throughout this chapter, concurrently. Respiration, blood circulation and digestion, for example, can occur **concurrently**. All of the senses—seeing, touching, smelling, tasting and hearing—can all occur concurrently. An automobile can be accelerating, turning, air conditioning and playing music concurrently. Computers, too, perform operations concurrently.

For example, it’s common for personal computers to compile a program, send a file to a printer, and receive electronic mail messages over a network and more, concurrently. Ironically, though, most programming languages do not enable programmers to specify concurrent activities. Rather, programming languages generally provide only a simple set of control structures that enable programmers to perform one action at a time then proceed to the next action after the previous one is finished. Ada programming language is one best example of single task programming.

The programmer specifies that applications contain threads of execution, each thread designating a portion of a program that may execute concurrently with other threads. This capability is called multithreading, gives the Java programmer powerful capabilities not available in C and C++, the languages on which Java is based. C and C++ are called single-threaded languages.

Java is unique among popular general-purpose programming languages in that it makes concurrency primitives available to the applications programmer. Java’s capabilities for developing programs that create and manage multiple tasks improve program performance. When we say that two tasks are operating **in parallel (concurrently)**, we mean that they’re executing **simultaneously**. Operating systems on such computers execute tasks concurrently by rapidly switching between them, doing a small portion of each before moving on to the next, so that all tasks keep progressing. In this sense, parallelism is a subset of concurrency.

Another example of multithreading is Java’s automatic garbage collection. In C and C++, the programmer is responsible for reclaiming dynamically allocated memory. Java provides a garbage collector thread that reclaims dynamically allocated memory that the program no longer needs. If anyone attempts tell you that C++ is better than Java, you only need to mention one word: **Threading**.

Multithreading in Java is a process of executing multiple threads simultaneously.

In C and C++, programmers must provide explicit statements that reclaim dynamically allocated memory. When memory is not reclaimed (because a programmer forgets to do so, because of a logic error or because an exception diverts program control), this results in an all-too-common error called a memory leak that can eventually exhaust the supply of free memory and may cause premature program termination. Java's automatic garbage collection eliminates the vast majority of memory leaks, that is, those that are due to orphaned (unreferenced) objects.

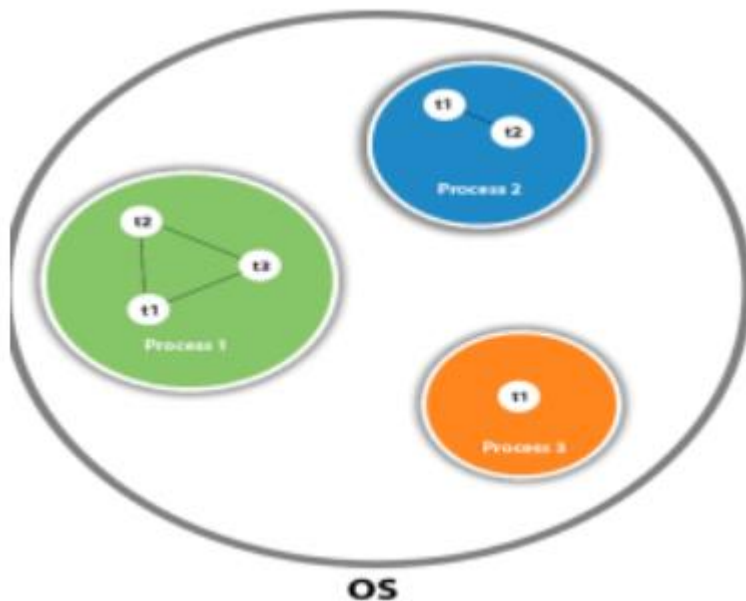
Java's garbage collector runs as a low-priority thread. When Java determines that there are no longer any references to an object, it marks the object for eventual garbage collection. The garbage-collector thread runs when processor time is available and when there are no higher priority runnable threads. However, the garbage collector will run immediately when the system is out of memory. Setting an object reference to null marks that object for eventual garbage collection (if there are no other references to the object). This can help conserve memory in a system in which a local variable that refers to an object does not go out of scope because the method in which it appears executes for a lengthy period.

Nearly every operating system supports the concept of processes –independently running programs that are isolated from each other to some degree. Threading is a facility to allow multiple activities to coexist within a single process. Most modern operating systems support threads, and the concept of threads has been around in various forms for many years. Java is the first mainstream programming language to explicitly include threading within the language itself, rather than treating threading as a facility of the underlying operating system.

1.2 What are Threads?

Threads are sometimes referred to as lightweight sub-processes, the smallest unit of processing.

- It is a separate path of execution.
- Threads are the basic unit of program execution.
- A thread is a single sequential flow of control within a program that allow multiple activities to coexist within a single process.



Why use threads?

- If you use Swing, Servlets, RMI, etc technology, you may already be using threads without realizing it.
- Some of the reasons for using threads are that they can help to:
 - Make the UI more responsive
 - Take advantage of multiprocessor systems
 - Simplify modeling

Benefits of Multithreading

- Enable programmers to do multiple things @ one time.
- Programmers can divide a long program into threads and execute them in a parallel which eventually increase the speed of program execution.
- Improved performance and concurrency.
- Simultaneous access to multiple applications.
- Enhanced performance by decreased development time
- Simplified and streamlined program coding
- Improvised GUI responsiveness
- Simultaneous and parallelized occurrence of tasks
- Decreased cost of maintenance
- Better use of CPU resource

Disadvantages of Multithreading

Multithreading does not only provide you with benefits, it has its disadvantages too. Let us go through some common disadvantages:

- *Difficulty of managing concurrency*
- Complex debugging and testing processes
- Overhead switching of context
- Increased potential for deadlock occurrence
- Increased difficulty level in writing a program
- Unpredictable results

Difference between Multithreading and Multiprocessing

Multiprocessing and Multithreading both adds performance to the system.

Multiprocessing is adding more number of or CPUs/processors to the system which increases the computing speed of the system.

Multithreading is allowing a process to create more threads which increase the responsiveness of the system.

1.3 Life cycle of a Thread (Thread states)

A thread goes through various stages in its life cycle. For example, a thread is born, started, runs, and then dies. The following diagram shows the complete life cycle of a thread.

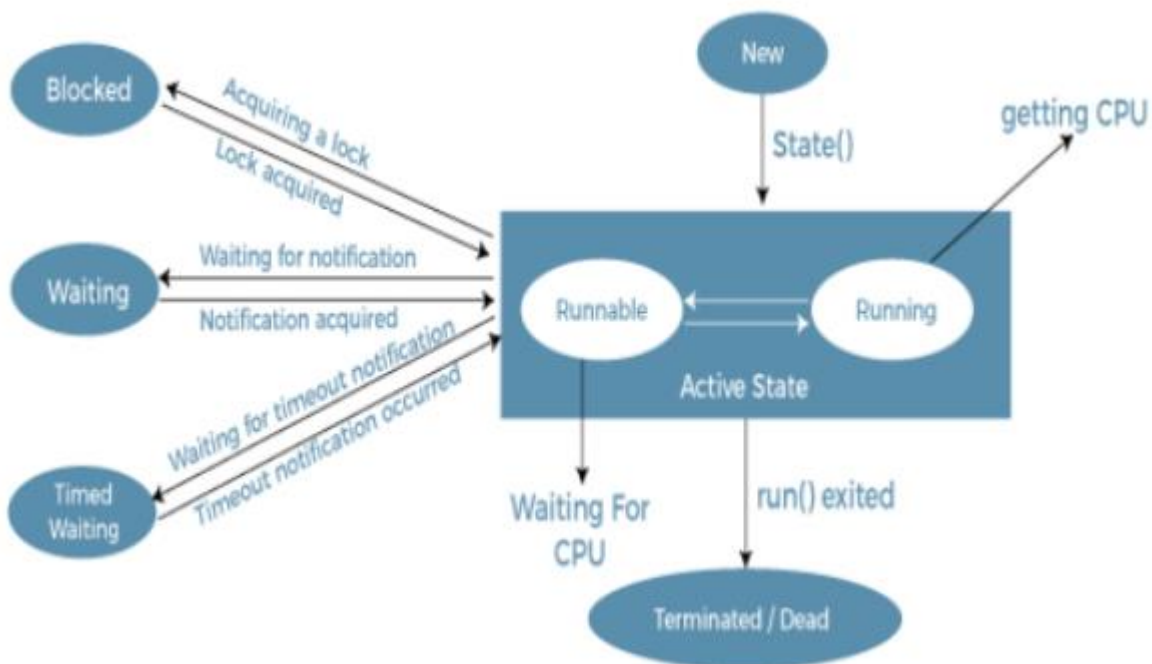


Figure 1 Life Cycle of Thread

Thread States

Threads exist in several states.

- A thread can be running. It can be ready to run as soon as it gets CPU time.
- A running thread can be suspended, which temporarily suspends its activity.
- A suspended thread can then be resumed, allowing it to pick up where it left off.
- A thread can be blocked when waiting for a resource.
- At any time, a thread can be terminated, which halts its execution immediately.
- Once terminated, a thread cannot be resumed

Every thread after creation and before destruction can have any one of four states:

- Newly created (New)
- Runnable state (Activate)
- Blocked/Waiting
- Dead (Terminated)

1. **New Thread:** Whenever new thread is created, it lies in the new state, its code is yet to be run and hasn't started to execute.

A new thread begins its life cycle in the new state. It remains in this state until the program starts the thread, which places it in the runnable state. A thread in the runnable state is considered to be executing its task.

- A thread enters the newly created by using a new operator.
- It is new state or born state immediately after creation. i.e. when a constructor is called the Thread is created but is not yet to run() method will not begin until it start() method is called.
- After the start() method is called, the thread will go to the next state, Runnable state.
- Note : in the above cycle stop(), resume() and suspend are deprecated methods. Java 2 strongly discourage their usage in the program.

2. Runnable/Running: When a thread invokes the start() method, it moves from the new state to the active state. The active state contains two states within it: one is runnable, and the other is running. In this state, a thread might actually be running or it might be ready run at any instant of time. Sometimes a runnable thread transitions to the waiting state while it waits for another thread to perform a task. A waiting thread transitions back to the runnable state only when another thread notifies it to continue executing.

- **Runnable/Waiting:** A thread that is ready to run is then moved to the runnable state. In the runnable state, the thread may be running or may be ready to run at any given instant of time. It is the duty of the thread scheduler to provide the thread time to run, i.e., moving the thread the running state. A program implementing multithreading acquires a fixed slice of time to each individual thread. Each and every thread runs for a short span of time and when that allocated time slice is over, the thread voluntarily gives up the CPU to the other thread, so that the other threads can also run for their slice of time. Whenever such a scenario occurs, all those threads that are willing to run, waiting for their turn to run, lie in the runnable state. In the runnable state, there is a queue where the threads lie.
- **Running:** When the thread gets the CPU, it moves from the runnable to the running state. Generally, the most common change in the state of a thread is from runnable to running and again back to runnable.

3. Blocked/Waiting state:

Whenever a thread is inactive for a span of time (not permanently) then, either the thread is in the blocked state or is in the waiting state. A thread might be temporarily inactive, then it's in one of the following states:

- Blocked
- Waiting

This is happen when a thread is waiting for I/O to complete, it lies in the blocked state. For example, a thread (let's say its name is A) may want to print some data from the printer. However, at the same time, the other thread (let's say its name is B) is using the printer to print some data. Therefore, thread A has to wait for thread B to use the printer. Thus, thread A is in the blocked state. A thread in the blocked state is unable to perform any execution and thus never consume any cycle of the CPU. Hence, we can say that thread A remains idle until the thread scheduler reactivates thread A, which is in the waiting or blocked state.

It's the responsibility of the thread scheduler to reactivate (time to run) and schedule a blocked/waiting thread. A thread in this state cannot continue its execution any further until it is moved to runnable state. Any thread in these states does not consume any CPU cycle.

When the main thread invokes the **join()** method then, it is said that the main thread is in the waiting state. The main thread then waits for the child threads to complete their tasks. When the

child threads complete their job, a notification is sent to the main thread, which again moves the thread from waiting to the active state.

A thread can be **blocked** when waiting for a resource. A thread is in the blocked state when it tries to access a protected section of code that is currently locked by some other thread. When the protected section is unlocked, the scheduler picks one of the thread which is blocked for that section and moves it to the runnable state. Whereas, a thread is in the waiting state when it waits for another thread on a condition. When this condition is fulfilled, the scheduler is notified and the waiting thread is moved to runnable state. If a currently running thread is moved to blocked/waiting state, another thread in the runnable state is scheduled by the thread scheduler to run. It is the responsibility of thread scheduler to determine which thread to run. If there are a lot of threads in the waiting or blocked state, then it is the duty of the thread scheduler to determine which thread to choose and which one to reject, and the chosen thread is then given the opportunity to run.

*Note: A running thread can be **suspended**, which is a temporary halt to its execution. It can later be **resumed to get CPU time**.*

Timed Waiting: Sometimes, waiting for leads to starvation. A thread lies in timed waiting state when it calls a method with a time out parameter. A thread lies in this state until the timeout is completed or until a notification is received. For example, a thread (its name is A) has entered the critical section of a code and is not willing to leave that critical section. In such a scenario, another thread (its name is B) has to wait forever, which leads to starvation. To avoid such scenario, a timed waiting state is given to thread B. Thus, thread lies in the waiting state for a specific span of time, and not forever. A real example of timed waiting is when we invoke the sleep() method on a specific thread. The sleep() method puts the thread in the timed wait state. After the time runs out, the thread wakes up and start its execution from when it has left earlier. A multi-threaded program allocates a fixed amount of time to each individual thread. Each and every thread runs for a short while and then

pauses and relinquishes the CPU to another thread, so that other threads can get a chance to run. When this happens, all such threads that are ready to run, waiting for the CPU and the currently running thread lies in runnable state.

4. Terminated (Dead) State: A thread terminates because of either of the following reasons:

- **Because it exists normally.** This happens when the code of thread has entirely executed by the program. When a thread has finished its job, then it exists or terminates normally.

- **Abnormal termination:** Because there occurred some unusual erroneous event, like segmentation fault or an unhandled exception. It occurs when some unusual events such as an unhandled exception or segmentation fault.

A terminated thread does no longer consumes any cycles of CPU. In other words, the thread is dead, and there is no way one can respawn (active after kill) the dead thread. Along with threadbased multitasking comes the need for a special type of feature called synchronization, which allows the execution of threads to be coordinated in certain well-defined ways.

BASIS FOR COMPARISON	MULTIPROCESSING	MULTITHREADING
Basic	Multiprocessing adds CPUs to increase computing power.	Multithreading creates multiple threads of a single process to increase computing power.
Execution	Multiple processes are executed concurrently.	Multiple threads of a single process are executed concurrently.
Creation	Creation of a process is time-consuming and resource intensive.	Creation of a thread is economical in both sense time and resource.
Classification	Multiprocessing can be symmetric or asymmetric.	Multithreading is not classified.

1.4 Creating a Thread

A thread is created by instantiating an object of type **Thread**. Java defines two ways in which this can be accomplished:

1. By implementing the **Runnable** interface.

2. By extending the **Thread** class.

The easiest way to create a thread, create a class that implements Runnable interface. To implement Runnable interface, a class need only implement a single method called **run()**, which is declared like: **public void run()**. After the new Thread is created, it will not start running until you call its **start()** method, which is declared within thread.

The start() method is: **void start();**

The Thread Class methods

- The Thread class has three primary methods that are used to control a thread:
 - **public void start():** prepares a thread to be run;
 - **public void run():** actually performs the work of the thread;
 - **public final void stop():** halts the thread;
- The thread *dies* when the run() method terminates or when the thread's stop() method is invoked.
- NB!!!: You never call run() explicitly. It is called automatically by the runtime as necessary once you've called start().

Create a thread using Runnable Interface:

```
public class Begin implements Runnable
{
    public void run()
    {
        System.out.println("Thread is Running");
    }
}

public static void main(String[] args)
{
    Begin b=new Begin();
    Thread t=new Thread(b);
    t.start();
}
}
```

Output: Thread is Running

Create a Thread by extending Thread class:

The second way to create a Thread is to create a new class that extends Thread, and then to create the instance of that class. **The extending class must override the run() method.**

Example:

```
public class Begin2 extends Thread
{
    public void run()
    {
        System.out.println("The Thread is Running");
    }
    public static void main(String [] args)
    {
        Begin2 b=new Begin2();
        b.start();
    }
}
```

Output: The Thread is Running

Example:

```
class Hi extends Thread
{
//      public void show()    //the method
      @Override
      public void run()
      {
          for(int i=1; i<=5; i++){
              System.out.println("Hi");
              try{
                  Thread.sleep(1000);    //to suspend a thread for one second(1000 milli second)
              }
              catch(Exception e){}
          }
      }
}

class Hello extends Thread
{
```

```

//      public void show()    //the method
      public void run()
    {
      for(int i=1; i<=5; i++){
        System.out.println("Hello");
        try{Thread.sleep(1000);    //to suspend a thread for one second(1000 milli second)
        }
        catch(Exception e){}
      }
    }
}

public class second {
public static void main(String []args)
{
  Hi obj1=new Hi();    // to call the method
  Hello obj2=new Hello();

  obj1.start(); //used to call run method
  //try{Thread.sleep(1000);}catch(Exception e){}    //to continue Hi, Hello up to end!!
  obj2.start();
  //  obj1.show();    //used to call show method
  //  obj2.show();
}
}

```


1.5 Thread priorities

When a Java thread is created, it inherits its priority from the thread that created it. You can also modify a thread's priority at any time after its creation using the `setPriority` method.

1.6 Synchronization

Synchronization in java: is the capability to control the access of multiple threads to any shared resource. It is the process of allowing only one thread at a time to complete the task entirely.

- The problem occurs, when the two or more threads are trying to perform their task. This kind of problem technically called **race condition**.

Thus, Java Synchronization is better option where we want to allow only one thread to access the shared resource.

Why use Synchronization?

The synchronization is mainly used to:

- ✓ To prevent thread interference
- ✓ To avoid and solve data inconsistency problem

Problems of Synchronization

- ✓ Increases waiting time
- ✓ Effects on performance of the system
- ✓ Less or poor performance

1.6.1 Types of Synchronization

There are two types of synchronization

1. Process Synchronization
2. Thread Synchronization

1. Process Synchronization

Process Synchronization is a way to coordinate processes that use shared data. It occurs in an operating system among cooperating processes.

Process Synchronization means sharing system resources by processes in a way that, Concurrent access to shared data is handled thereby minimizing the chance of inconsistent data. Maintaining data consistency demands mechanisms to ensure synchronized execution of cooperating processes. Process synchronization problem arises in the case of Cooperative process also because resources are shared in Cooperative processes.

Process Synchronization was introduced to handle problems that arose while multiple process executions.

On the basis of synchronization, processes are categorized as one of the following two types:

- **Independent Process:** Execution of one process does not affect the execution of other processes.
- **Cooperative Process:** Execution of one process affects the execution of other processes.

2. Thread Synchronization

Thread synchronization is defined as a mechanism which ensures that two or more concurrent processes or threads do not simultaneously execute some particular program segment known as critical section. In concurrent programming, concurrent accesses to shared resources can lead to unexpected or erroneous behavior, so parts of the program where the shared resource is accessed need to be protected in ways that avoid the concurrent access. This protected section is the **critical section** or **critical region**.

There are two types of thread synchronization:

1. Mutual Exclusive
2. Cooperation (Inter-thread communication in java)

Mutual Exclusive

Mutual Exclusive helps keep threads from interfering with one another while sharing data. This can be done by three ways in java:

1. by synchronized method
2. by synchronized block
3. by static synchronization

Inter-thread communication or **Co-operation** is all about allowing synchronized threads to communicate with each other.

Cooperation (Inter-thread communication) is a mechanism in which a thread is paused running in its critical section and another thread is allowed to enter (or lock) in the same critical section to be executed. It is implemented by following methods of **Object class**:

- wait()
- notify()
- notifyAll()

1.7 Implementation of Threads States

In Java, one can get the current state of a thread using the Thread.getState() method. The java.lang.Thread.State class of Java provides the constants ENUM to represent the state of a thread. These constants are:

1. public static final Thread.State NEW: It represents the first state of a thread that is the NEW state.
2. public static final Thread.State RUNNABLE: It represents the runnable state. It means a thread is waiting in the queue to run.

3. `public static final Thread.State BLOCKED`: It represents the blocked state. In this state, the thread is waiting to acquire a lock.
4. `public static final Thread.State WAITING`: It represents the waiting state. A thread will go to this state when it invokes the `Object.wait()` method, or `Thread.join()` method with no timeout. A thread in the waiting state is waiting for another thread to complete its task.
5. `public static final Thread.State TIMED_WAITING`: It represents the timed waiting state. The main difference between waiting and timed waiting is the time constraint. Waiting has no time constraint, whereas timed waiting has the time constraint. A thread invoking the following method reaches the timed waiting state.

Each Java Run time thread is encapsulated in a `java.lang.Thread` instance class. Java's multithreading system is built upon the **Thread** class and its companion interface, **Runnable**. **Thread** encapsulates a thread of execution.

To create a new thread, your program can be declared in one of two ways:

1. **by extending the Thread class**
2. **by implementing the Runnable interface**

The **Thread** class (package **java.lang**) defines several methods or constructors that help manage threads. Here are some of the more commonly used ones (we will be looking at these more closely as they are used):

```
public Thread( String threadName )
```

constructs a **Thread** object whose name is **threadName**. The constructor

```
public Thread()
```

constructs a **Thread** whose name is **"Thread-"** concatenated with a number, like **Thread-1**, **Thread-2**, and so on.

Method	Meaning
<code>final String getName()</code>	Obtains a thread's name.
<code>final int getPriority()</code>	Obtains a thread's priority.
<code>final boolean isAlive()</code>	Determines whether a thread is still running.
<code>final void join()</code>	Waits for a thread to terminate.
<code>void run()</code>	Entry point for the thread.
<code>static void sleep(long milliseconds)</code>	Suspends a thread for a specified period of milliseconds.
<code>void start()</code>	Starts a thread by calling its run() method.

All processes have at least one thread of execution, which is usually called the main thread, because it is the one that is executed when your program begins. **Thus, the main thread is the thread that all of the preceding example programs in the book have been using.** From the main thread, you can create other threads.

You create a thread by instantiating an object of type `Thread`. The `Thread` class encapsulates an object that is runnable.

As mentioned, Java defines two ways in which you can create a runnable object:

- You can implement the `Runnable` interface.
- You can extend the `Thread` class.

Remember: Both approaches use the `Thread` class to instantiate, access, and control the thread. The only difference is how a thread-enabled class is created.

1. Implementing the `Runnable` interface

```
public class MyRunnable extends AnotherClass
                        implements Runnable
{
    public void run() {
        // the thread body
    }

    // other methods and fields
}
```

Creating and starting a thread:

```
new Thread(new MyRunnable()).start();
```

The **`Runnable`** interface abstracts a unit of executable code. You can construct a thread on any object that implements the **`Runnable`** interface. **`Runnable`** defines only one method called **`run ()`**, which is declared like this:

```
public void run( )
```

The code that “does the real work” of a thread is placed in its `run` method. The `run` method can be overridden in a subclass of `Thread` or it may be implemented in a `Runnable` object. Inside `run ( )`,

you will define the code that constitutes the new thread. It is important to understand that `run()` can call other methods, use other classes, and declare variables just like the main thread.

The only difference is that **`run( )`** establishes the entry point for another, concurrent thread of execution within your program. This thread will end when **`run( )`** returns. After you have created a class that implements **`Runnable`**, you will instantiate an object of type `Thread` on an object of that class. `Thread` defines several constructors. The one that we will use first is shown here:

`Thread(Runnable threadOb)`

In this constructor, **`threadOb`** is an instance of a class that implements the `Runnable` interface. This defines where execution of the thread will begin. Once created, the new thread will not **`start`** running until you call its `start( )` method, which is declared within `Thread`. A program launches a thread's execution by calling the thread's **`start`** method, which, in turn, calls method `run`. After `start` launches the thread, **`start`** returns to its caller immediately. The caller then executes concurrently with the launched thread. The **`start`** method throws an **`IllegalThreadStateException`** if the thread it is trying to start has already been started. In essence, `start( )` executes a call to `run()`. The `start( )` method is shown here:

`void start( )`

Here is an example that creates a new thread and starts it running:

```

package threads;
// Create a thread by implementing Runnable.
public class MyThread implements Runnable {
    int count;
    String thrdName;
    MyThread(String name) {
        count = 0;
        thrdName = name;
    }
    // Entry point of thread.
    public void run() {
        System.out.println(thrdName + " starting.");
        try {
            do {
                Thread.sleep(500);
                System.out.println("In " + thrdName
                    + ", count is " + count);
                count++;
            } while (count < 10);
        } catch (InterruptedException exc) {
            System.out.println(thrdName + " interrupted.");
        }
        System.out.println(thrdName + " terminating.");
    }
}

```

Objects of MyThread can be run in their own threads because MyThread implements Runnable.

Threads start executing here.

```

class UseThreads {
    public static void main(String args[]) {
        System.out.println("Main thread starting.");
        // First, construct a MyThread object.
        MyThread mt = new MyThread("Child #1");
        // Next, construct a thread from that object.
        Thread newThrd = new Thread(mt);
        // Finally, start execution of the thread.
        newThrd.start();
        do {
            System.out.print(".");
            try {
                Thread.sleep(100);
            } catch (InterruptedException exc) {
                System.out.println("Main thread interrupted.");
            }
        } while (mt.count != 10);
        System.out.println("Main thread ending.");
    }
}

```

Create a runnable object.

Construct a thread on that object.

Start running the thread.

Let's look closely at this program. First, MyThread implements Runnable. This means that an object of type MyThread is suitable for use as a thread and can be passed to the Thread constructor. Inside run(), a loop is established that counts from 0 to 9. Notice the call to sleep().

The static method `sleep` is called with an argument specifying how long the currently executing thread should sleep (in milliseconds); while a thread sleeps, it does not contend for the processor,

so other threads can execute. This can give lower priority threads a chance to run. The `sleep( )` method causes the thread from which it is called to suspend execution for the specified period of milliseconds. Its general form is shown here: `static void sleep(long milliseconds)` throws `InterruptedException`. The number of milliseconds to suspend is specified in milliseconds. This method can throw an `InterruptedException`. Thus, calls to it must be wrapped in a try/catch block. The `sleep( )` method also has a second form, which allows you to specify the period in terms of milliseconds and nanoseconds if you need that level of precision. Inside `main( )`, a new `Thread` object is created by the following sequence of statements:

```
// First, construct a MyThread object.
MyThread mt = new MyThread("Child #1");

// Next, construct a thread from that object.
Thread newThrd = new Thread(mt);

// Finally, start execution of the thread.
newThrd.start();
```

As the comments suggest, first an object of `MyThread` is created. This object is then used to construct a `Thread` object. This is possible because `MyThread` implements `Runnable`. Finally, execution of the new thread is started by calling `start( )`. This causes the child thread's `run( )` method to begin. After calling `start( )`, execution returns to `main( )`, and it enters `main( )`'s do loop. Both threads continue running, sharing the CPU, until their loops finish. The output produced by this program is as follows. Because of differences between computing environments, the precise output that you see may differ from that shown here.


```
Output - Threads (run) x
run:
Main thread starting.
Child #1 starting.
....In Child #1, count is 0
.....In Child #1, count is 1
.....In Child #1, count is 2
.....In Child #1, count is 3
.....In Child #1, count is 4
.....In Child #1, count is 5
.....In Child #1, count is 6
.....In Child #1, count is 7
.....In Child #1, count is 8
.....In Child #1, count is 9
Child #1 terminating.
Main thread ending.
BUILD SUCCESSFUL (total time: 5 seconds)
```

Progress Check

1. In what two ways can you create a class that can act as a thread?
2. What is the purpose of the **run()** method defined by **Runnable**?
3. What does the **start()** method defined by **Thread** do?

2. Extending Threads

```
public class MyThread extends Thread {
    public void run() {
        // the thread body
    }
    // other methods and fields
}
```

Creating and starting a thread:

```
new MyThread().start();
```

Implementing **Runnable** is one way to create a class that can instantiate thread objects. Extending **Thread** is the other. In this project, you will see how to extend **Thread** by creating a program functionally identical to the **UseThreadsImproved** program. When a class extends **Thread**, it must override the **run()** method, which is the entry point for the new thread. It must also call **start()** to begin the execution of the new thread. It is possible to override other **Thread** methods, but doing so is not required.

1.7.1 Determining When a Thread Ends

It is often useful to know when a thread has ended. In the preceding examples, we accomplished this by watching the count variable, but this is, of course, hardly a satisfactory or generalizable solution. Fortunately, **Thread** provides two means by which you can determine if a thread has ended. First, you can call **isAlive()** on the thread. Its general form is shown here:

final boolean isAlive()

The **isAlive()** method returns **true** if the thread upon which it is called is still running. It returns **false** otherwise. To try **isAlive()**, substitute this version of **MoreThreads** for the one shown in the preceding program.

```
class MoreThreads {
    public static void main(String args[]) {
        System.out.println("Main thread starting.");

        MyThread mt1 = new MyThread("Child #1");
        MyThread mt2 = new MyThread("Child #2");
        MyThread mt3 = new MyThread("Child #3");

        do {
            System.out.print(".");
            try {
                Thread.sleep(100);
            }
            catch (InterruptedException exc) {
                System.out.println("Main thread interrupted.");
            }
        } while (mt1.thrd.isAlive() ||
                mt2.thrd.isAlive() || ← This waits until all threads terminate.
                mt3.thrd.isAlive());

        System.out.println("Main thread ending.");
    }
}
```

Progress Check

1. What are the two ways in which you can determine whether a thread has finished?
2. Explain `join()`.

1. To determine whether a thread has ended, you can call `isAlive()` or use `join()` to wait for the thread to join the calling thread.
2. The `join()` method suspends execution of the calling thread until the thread on which `join()` is called, ends.

Thread Methods: Following is the list of important methods available in the Thread class.

SN	Methods with Description
1.	public void start : Starts the thread in a separate path of execution, then invokes the run method on this Thread object.

2.	public void run: If this Thread object was instantiated using a separate Runnable target, the run method is invoked on that Runnable object.
3.	public final void setNameStringname : Changes the name of the Thread object. There is also a getName method for retrieving the name.
4.	public final void setPriorityintpriority : Sets the priority of this Thread object. The possible values are between 1 and 10.
5.	public final void setDaemonbooleanon : A parameter of true denotes this Thread as a daemon thread.
6.	public final void joinlongmillisec: The current thread invokes this method on a second thread, causing the current thread to block until the second thread terminates or the specified number of milliseconds passes.
7.	public void interrupt : Interrupts this thread, causing it to continue execution if it was blocked for any reason.
8.	public final boolean isAlive : Returns true if the thread is alive, which is any time after the thread has been started but before it runs to completion.

9.	public static void yield : Causes the currently running thread to yield to any other threads of the same priority that are waiting to be scheduled.
10.	public static void sleep(longmillisec: Causes the currently running thread to block for at least the specified number of milliseconds.
11.	public static boolean holdsLockObjectx: Returns true if the current thread holds the lock on the given Object.
12.	public static Thread currentThread: Returns a reference to the currently running thread, which is the thread that invokes this method.
13.	public static void dumpStack: Prints the stack trace for the currently running thread, which is useful when debugging a multithreaded application.

1.8 Producer/consumer relationship without Synchronization

In a producer/consumer relationship, a **producer thread** calling a **produce** method may see that the consumer thread has not read the last message from a shared region of memory called a **buffer**, so the producer thread will call **wait**. When a **consumer thread** reads the message, it will

call **notify** to allow a waiting producer to proceed. When a consumer thread enters the monitor and finds the buffer empty, it calls **wait**. A producer finding the buffer empty writes to the buffer, then calls **notify** so a waiting consumer can proceed.

Shared data can get corrupted if we do not synchronize access among multiple threads. Consider a producer/consumer relationship in which a producer thread deposits a sequence of numbers (we use 1, 2, 3 ...) into a slot of shared memory. The consumer thread reads this data from the shared memory and prints it. We print what the producer produces as it produces it and what the consumer consumes as it consumes it.

The program below demonstrates a producer and a consumer accessing a single shared cell of memory (**int** variable **sharedInt**) without any synchronization. The threads are not synchronized, so data can be lost if the producer places new data into the slot before the consumer consumes the previous data. Also, data can be “doubled” if the consumer consumes data again before the producer produces the next item. To show these possibilities, the consumer thread in this example keeps a total of all the values it reads. The producer thread produces values from 1 to 10. If the consumer reads each value produced only once, the total would be 55. However, if you execute this program several times, you will see that the total is rarely, if ever, 55.

The program consists of four classes—

ProduceInteger (ProducerInteger.java), **ConsumeInteger** (ConsumeInteger.java), **HoldIntegerUnsynchronized** (HoldIntegerUnsynchronized.java) and **SharedCell** (SharedCell.java). Class **ProduceInteger** (ProducerInteger.java)—a subclass of **Thread**—consists of instance variable **sharedObject** (line 3), a constructor (lines 5–8), and a **run** method (lines 15–37). The constructor initializes the instance variable **sharedObject** (line 10) to refer

to the **HoldIntegerUnsynchronized** object **shared**, which was passed as an argument. Class **ProduceInteger**’s **run** method (lines 11–27) consists of a **for** structure that loops 10 times. Each iteration of the loop first invokes method **sleep** to put the **ProduceInteger** object into the sleeping state for a random time interval between 0 and 3 seconds.

When the thread awakens, line 22 calls class **HoldIntegerUnsynchronized**’s **setSharedInt** method and passes the value of control variable **count** to set the shared object’s instance variable **sharedInt**. When the loop completes, lines 24–26 display a line of text in the command window indicating that the thread finished producing data and that the thread is terminating, then the thread terminates (i.e., the thread dies).

```

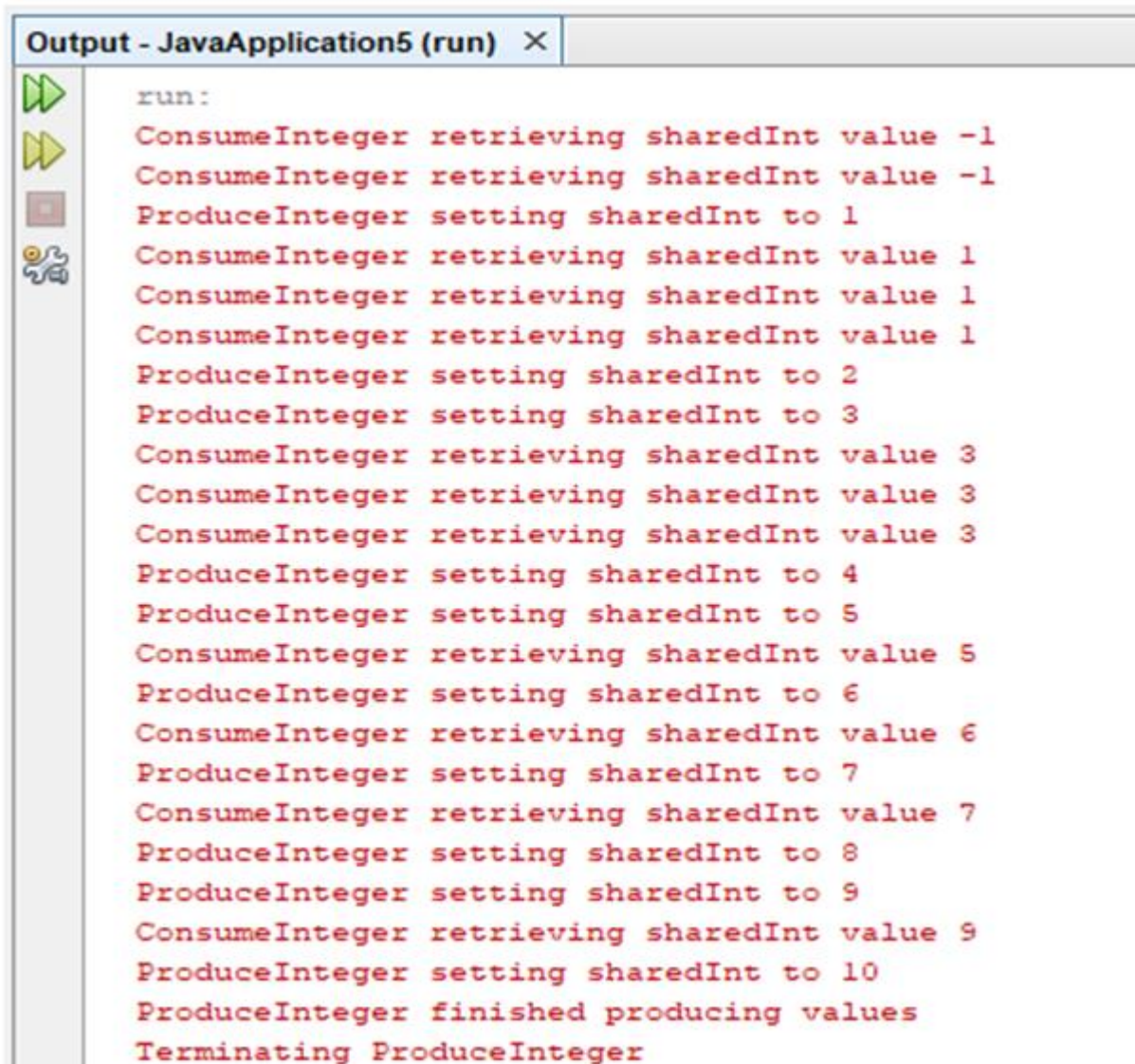
1  package ThreadImplementation;
2  public class ProduceInteger extends Thread {
3      private HoldIntegerUnsynchronized sharedObject;
4      // initialize ProduceInteger thread object
5      public ProduceInteger(HoldIntegerUnsynchronized shared) {
6          super("ProduceInteger");
7          sharedObject = shared;
8      }
9      // ProduceInteger thread loops 10 times and calls
10     // sharedObject's setSharedInt method each time
11     public void run() {
12         for (int count = 1; count <= 10; count++) {
13             // sleep for a random interval
14             try {
15                 Thread.sleep((int) (Math.random() * 3000));
16             } // process InterruptedException during sleep
17             catch (InterruptedException exception) {
18                 System.err.println(exception.toString());
19             }
20             // call sharedObject method from this
21             // thread of execution
22             sharedObject.setSharedInt(count);
23         }
24         System.err.println(
25             getName() + " finished producing values"
26             + "\nTerminating " + getName());
27     }
28 }

```

Class **HoldIntegerUnsynchronized** consists of instance variable **sharedInt** (line 3), method **setSharedInt** (lines 5–9) and method **getSharedInt** (lines 11–15). Methods **setSharedInt** and **getSharedInt** do not synchronize access to instance variable **sharedInt**. Note that each method uses **static Thread** method **currentThread** to obtain a reference to the currently executing thread,

then use **Thread** method **getName** to obtain the thread's name. Class **SharedCell**'s **main** method (lines 4–15) instantiates the shared **HoldIntegerUnsynchronized** object **sharedObject** and uses it as the argument to the constructors for the **ProduceInteger** object **producer** and the **ConsumeInteger** object **consumer**. The **sharedObject** contains the data that will be shared between the two threads. Next, method **main** invokes the **Thread** class **start** method on the **producer** and **consumer** threads to place them in the **ready** state. This launches these threads.

```
1 package ThreadImplementation;
2 public class HoldIntegerUnsynchronized {
3     private int sharedInt = -1;
4     // unsynchronized method to place value in sharedInt
5     public void setSharedInt(int value) {
6         System.err.println(Thread.currentThread().getName()
7             + " setting sharedInt to " + value);
8         sharedInt = value;
9     }
10    // unsynchronized method return sharedInt's value
11    public int getSharedInt() {
12        System.err.println(Thread.currentThread().getName()
13            + " retrieving sharedInt value " + sharedInt);
14        return sharedInt;
15    }
16 }
```

```
Output - JavaApplication5 (run) X
run:
ConsumeInteger retrieving sharedInt value -1
ConsumeInteger retrieving sharedInt value -1
ProduceInteger setting sharedInt to 1
ConsumeInteger retrieving sharedInt value 1
ConsumeInteger retrieving sharedInt value 1
ConsumeInteger retrieving sharedInt value 1
ProduceInteger setting sharedInt to 2
ProduceInteger setting sharedInt to 3
ConsumeInteger retrieving sharedInt value 3
ConsumeInteger retrieving sharedInt value 3
ConsumeInteger retrieving sharedInt value 3
ProduceInteger setting sharedInt to 4
ProduceInteger setting sharedInt to 5
ConsumeInteger retrieving sharedInt value 5
ProduceInteger setting sharedInt to 6
ConsumeInteger retrieving sharedInt value 6
ProduceInteger setting sharedInt to 7
ConsumeInteger retrieving sharedInt value 7
ProduceInteger setting sharedInt to 8
ProduceInteger setting sharedInt to 9
ConsumeInteger retrieving sharedInt value 9
ProduceInteger setting sharedInt to 10
ProduceInteger finished producing values
Terminating ProduceInteger
```

1.9 Producer/consumer relationship with Synchronization

The application below demonstrates a producer and a consumer accessing a shared cell of memory with synchronization so that the consumer only consumes after the producer produces a value.

Classes **ProduceInteger.java** (Class1), **ConsumeInteger.java** (Class2)

and **SharedCell.java** (Class4) are identical to the previous classes except that they use the new class **HoldIntegerSynchronized.java** (Class3) in this example. For separation purpose we renamed as **ProduceInteger.java**, **ConsumeInteger1.java** and **SharedCell1.java**

Class **HoldIntegerSynchronized.java** (Class3) contains two instance variables—**sharedInt** (line 6) and **writable** (line 7). Also, method **setSharedInt** (lines 12–39) and method **getSharedInt** (lines 44–70) are now **synchronized** methods. Objects of class **HoldIntegerSynchronized** have monitors, because **HoldIntegerSynchronized** contains **synchronized** methods.

Instance variable **writable** is known as the monitor's **condition variable**—is a **boolean** used by methods **setSharedInt** and **getSharedInt** of class **HoldIntegerSynchronized**.

If **writable** is **true**, **setSharedInt** can place a value into variable **sharedInt**, because the variable currently does not contain information. However, this means **getSharedInt** currently cannot read the value of **sharedInt**. If **writable** is **false**, **getSharedInt** can read a value from variable **sharedInt** because the variable currently does contain information. However, this means **setSharedInt** currently cannot place a value into **sharedInt**.

Class: ProduceInteger1.java

```
1 package ThreadImplementation;
2 public class ProduceInteger1 extends Thread {
3     private HoldIntegerSynchronized sharedObject;
4     // initialize ProduceInteger thread object
5     public ProduceInteger1(HoldIntegerSynchronized shared) {
6         super("ProduceInteger");
7         sharedObject = shared;
8     }
9     // ProduceInteger thread loops 10 times and calls
10    // sharedObject's setSharedInt method each time
11    public void run() {
12        for (int count = 1; count <= 10; count++) {
13            // sleep for a random interval
14            try {
15                Thread.sleep((int) (Math.random() * 3000));
16            } // process InterruptedException during sleep
17            catch (InterruptedException exception) {
18                System.err.println(exception.toString());
19            }
20            // call sharedObject method from this
21            // thread of execution
22            sharedObject.setSharedInt(count);
23        }
24        System.err.println(
25            getName() + " finished producing values"
26            + "\nTerminating " + getName());
27    }
28 }
```

1.10 Producer/consumer relationship: The Circular Buffer

The program of thread with synchronization does access the shared data correctly, but it may not perform optimally. Because the threads are running asynchronously, we cannot predict their relative speeds. If the producer wants to produce faster than the consumer can consume, it cannot do so. To

enable the producer to continue producing, we can use a circular buffer that has enough extra cells to handle the “extra” production. The program of **a to d** below demonstrates a producer and a consumer accessing a circular buffer (in this case, a shared array of five cells) with synchronization so that the consumer only consumes a value when there are one or more values in the array, and the producer only produces a value when there are one or more available cells in the array. This program is implemented as a windowed application that sends its output to a **JTextArea**.

Class **SharedCell**’s constructor creates

the **HoldIntegerSynchronized**, **ProduceInteger** and **ConsumeInteger** objects.

The **HoldIntegerSynchronized** object **sharedObject**’s constructor receives a reference to a **JTextArea** object in which the program’s output will appear.

1.11 Daemon Threads

Daemon threads are low-priority threads whose only role is to provide services to user threads. A daemon thread is a thread that runs for the benefit of other threads. Unlike conventional user threads (i.e., any non-daemon thread in a program), daemon threads do not prevent a program from terminating. The garbage collector is a daemon thread. Nondaemon threads are conventional user threads or threads such as the event-dispatch thread used to process GUI events. We designate a thread as a daemon with the method call.

setDaemon(True);

A **false** argument means that the thread is not a daemon thread. A program can include a mixture of daemon threads and nond daemon threads. When only daemon threads remain in a program, the program exits. If a thread is to be a daemon, it must be set as such before its **start** method is called or an **IllegalThreadStateException** is thrown. Method **isDaemon** returns **true** if a thread is a daemon thread and **false** otherwise.

Creating a Daemon Thread: set a thread to be a daemon thread, all we need to do is to call `Thread.setDaemon()`. In this example, we’ll use the new thread class which extends the *Thread* class:

```
NewThread daemonThread = new NewThread();
daemonThread.setDaemon(true);
daemonThread.start();
```


Any thread inherits the daemon status of the thread that created it. Since the main thread is a user thread, any thread that is created inside the main method is by default a user thread. The method `setDaemon()` can only be called after the *Thread* object has been created and the thread has not been started. An attempt to call `setDaemon()` while a thread is running will throw an *IllegalThreadStateException*:

Sample Program

```
public class DeamonTreadExample {
    public static void main(String[] args) throws InterruptedException {
        Thread t = new Thread(() -> {
            int count = 0;
            while (true) {
                count++;
                try {
                    System.out.println("inside try"+ count);
                    Thread.currentThread().sleep(1000);
                } catch (InterruptedException e) {
                    // TODO Auto-generated catch block
                    e.printStackTrace();
                } finally {
                    System.out.println("finally executed"+ count);
                }
            }
        });
        t.setDaemon(true);
        t.start();

        Thread.currentThread().sleep(10000);
        System.out.println("main thread exited");
    }
}
```

Output

```
inside try1
finally executed1
inside try2
finally executed2
inside try3
finally executed3
inside try4
finally executed4
inside try5
finally executed5
inside try6
finally executed6
inside try7
finally executed7
inside try8
finally executed8
inside try9
finally executed9
inside try10
main thread exited
```

1.12 Runnable Interfaces

Until now, we extended class **Thread** to create new classes that support multithreading. We overrode the run method to specify the tasks to be performed concurrently. However, if we want multithreading support in a class that already extends a class other than **Thread**, we must implement the **Runnable** interface in that class, because Java does not allow a class to extend more than one class at a time. Java **Runnable** is an interface used to execute code on a concurrent thread. It is an interface that is implemented by any class if we want that the instances of that class should be executed by a thread.

Class **Thread** itself implements the **Runnable** interface (package **java.lang**) as expressed in the class header.

```
public class Thread extends Object implements Runnable
```

Implementing the **Runnable** interface in a class enables a program to manipulate objects of that class as **Runnable** objects. As with deriving from the **Thread** class, the code that controls the thread is placed in method **run**. A program that uses a **Runnable** object creates a **Thread** object and associates the **Runnable** object with that **Thread**. Class **Thread** provides four constructors that can receive references to **Runnable** objects as arguments. For example, the constructor

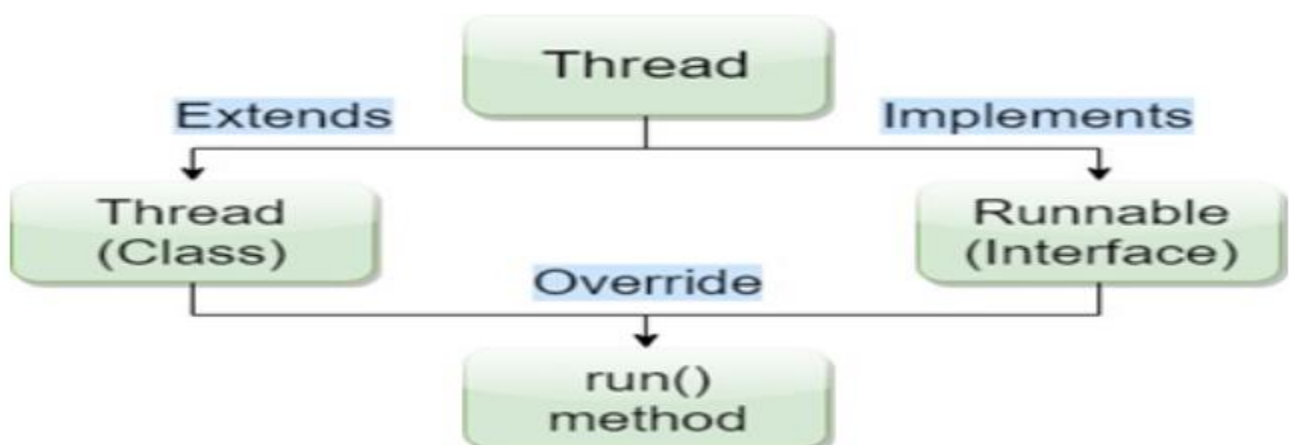
```
public Thread( Runnable runnableObject )
```

registers the method **run** of **runnableObject** as the method to be invoked when the thread begins execution. The constructor

```
public Thread( Runnable runnableObject )
```

constructs a **Thread** with the name **threadName** and registers the method **run** of its **runnableObject** argument as the method to be invoked when the thread begins execution. As always, the thread object's **start** method must be called to begin the thread's execution.

A simple thread example using runnable



```

1  package ThreadImplementation;
2  public class ThreadRunnableExampleClass implements Runnable{
3      @Override
4      public void run() {
5          System.out.println("Thread has ended");
6      }
7      public static void main(String[] args) {
8          ThreadRunnableExampleClass ex = new ThreadRunnableExampleClass();
9          Thread t1= new Thread(ex);
10         t1.start();
11         System.out.println("Hi");
12     }
13 }

```

Output

Output - JavaApplication5 (run) X	
	run:
	Hi
	Thread has ended
	BUILD SUCCESSFUL (total time: 0 seconds)

1.13 Chapter Review Questions

1. What is the difference between processes and threads?
2. Why does Java's multithreading capability enable you to write more efficient programs?
3. Explain different ways of creating a thread. Which one would you prefer and why?
4. Explain the available thread states at a high level.
5. Multithreading is supported by the class and the interface.
6. When creating a runnable object, why might you want to extend Thread rather than implement Runnable?
7. What is the effect of adding the synchronized keyword to a method?
8. The **wait()** and **notify()** methods are used to perform
9. Why can't you use **suspend()**, **resume()**, and **stop()** for new programs?
10. What does **isAlive()** return?
11. What is the difference between a synchronized method and a synchronized block?
12. How does thread synchronization occur inside a monitor? What levels of synchronization can you apply?
13. What's a deadlock?
14. How do you ensure that N threads can access N resources without deadlock?

Chapter Two

2 Networking

2.1 Introduction

Networking is a massive and complex topic. Java provides a rich complement of networking capabilities and will likely be used as an implementation vehicle in computer networking. Java's networking capabilities are grouped into several packages. The fundamental networking capabilities are defined by classes and interfaces of package **java.net**, through which Java offers socket-based communications that enable applications to view networking as streams of data. The classes and interfaces of package **java.net** also offer packet-based communications that enable individual packets of information to be transmitted—this is commonly used to transmit audio and video over the Internet.

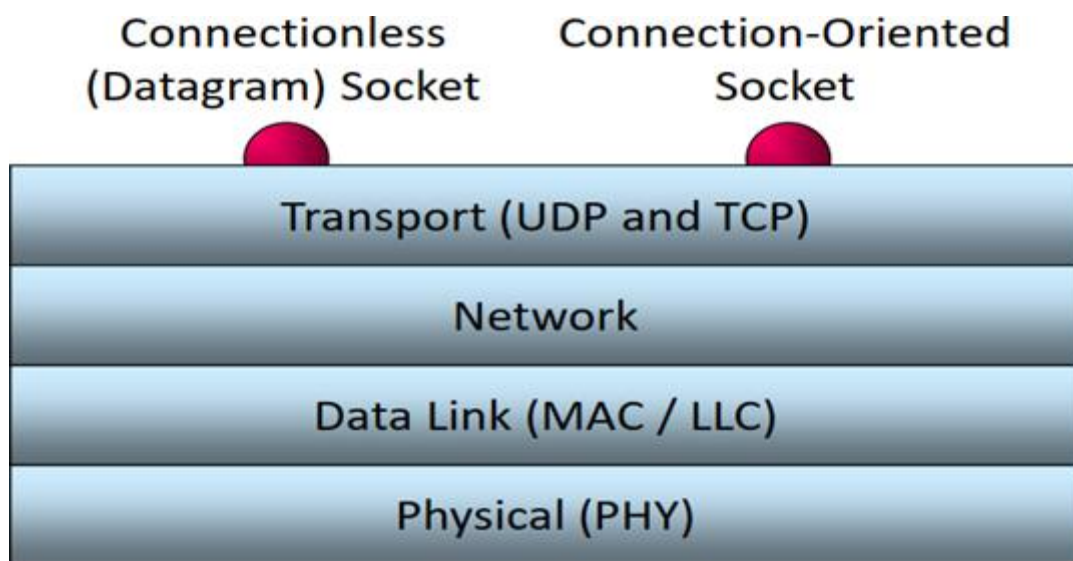
In this chapter, we show how to create and manipulate sockets and how to communicate with packets of data. Our discussion of networking focuses on both sides of a client-server relationship. The client requests that some action be performed, and the server performs the action and responds to the client. A common implementation of the request-response model is between World Wide Web browsers and World Wide Web servers. When a user selects a Web site to browse through a browser (the client application), a request is sent to the appropriate Web server (the server application). The server normally responds to the client by sending an appropriate HTML Web page.

Java provides stream sockets and datagram sockets. With stream sockets, a process establishes a connection to another process. While the connection is in place, data flows between the processes in continuous streams. Stream sockets are said to provide a connection-oriented service. The protocol used for transmission is the popular TCP (Transmission Control Protocol). With datagram sockets, individual packets of information are transmitted. This is not the right protocol for everyday users, because, unlike TCP, the protocol used—UDP, the User Datagram Protocol—is a connectionless service, and it does not guarantee that packets arrive in any particular order. In fact, packets can be lost, can be duplicated, and can even arrive out of sequence. So, with UDP, significant extra programming is required on the user's part to deal with these problems (if the user chooses to do so). UDP is most appropriate for network applications that do not require the error checking and reliability of TCP.



2.2 Sockets

Normally, a server runs on a specific computer and has a socket that is bound to a specific port number. The server just waits, listening to the socket for a client to make a connection request. Socket in Java is used for communication between the applications that are running on different **Java Runtime Environment (JRE)**. It is an interface to the network protocol stack typically at the transport layer. It can be either connection-oriented or connectionless.



Socket programming is a way of connecting two nodes on a network to communicate with each other. One *socket* (node) listens on a particular port at an IP, while the other *socket* reaches out to the other in order to form a connection. The client knows the hostname of the machine on which the server is running and the port number on which the server is listening. To make a connection request, the client tries to engage with the server on the server's machine and port. The client also needs to identify itself to the server so it binds to a local port number that it will use during this connection. This is usually assigned by the system

If everything goes well, the server accepts the connection. Upon acceptance, the server gets a new socket bound to the same local port and also has its remote endpoint set to the address and port of the client. It needs a new socket so that it can continue to listen to the original socket for connection requests while tending to the needs of the connected client.



On the client-side, if the connection is accepted, a socket is successfully created and the client can use the socket to communicate with the server. The client and server can now communicate by writing to or reading from their sockets. On the whole, a socket is a way to establish a connection between a client and a server.

Client

-  Create a TCP socket
-  Communicate
-  Close the connection

Server

-  Create a TCP socket
-  Repeatedly:
 -  Accept new connection
 -  Communicate
 -  Close the connection

Definition: A *socket* is one end-point of a two-way communication link between two programs (process) running on the network. A socket is bound to a port number so that the TCP/IP layer can identify the application to that data is destined to be sent. An endpoint is a combination of an **IP address and a port number**.

The client in socket programming has to know two information about the server:

1. **IP Address of Server, and**
2. **Port number.**

Every TCP connection can be uniquely identified by its two endpoints. That way you can have multiple connections between your host and the server.

A socket is a half association

{ protocol, local-address, local-port }

Example: Socket("10.198.7.20", 1234)

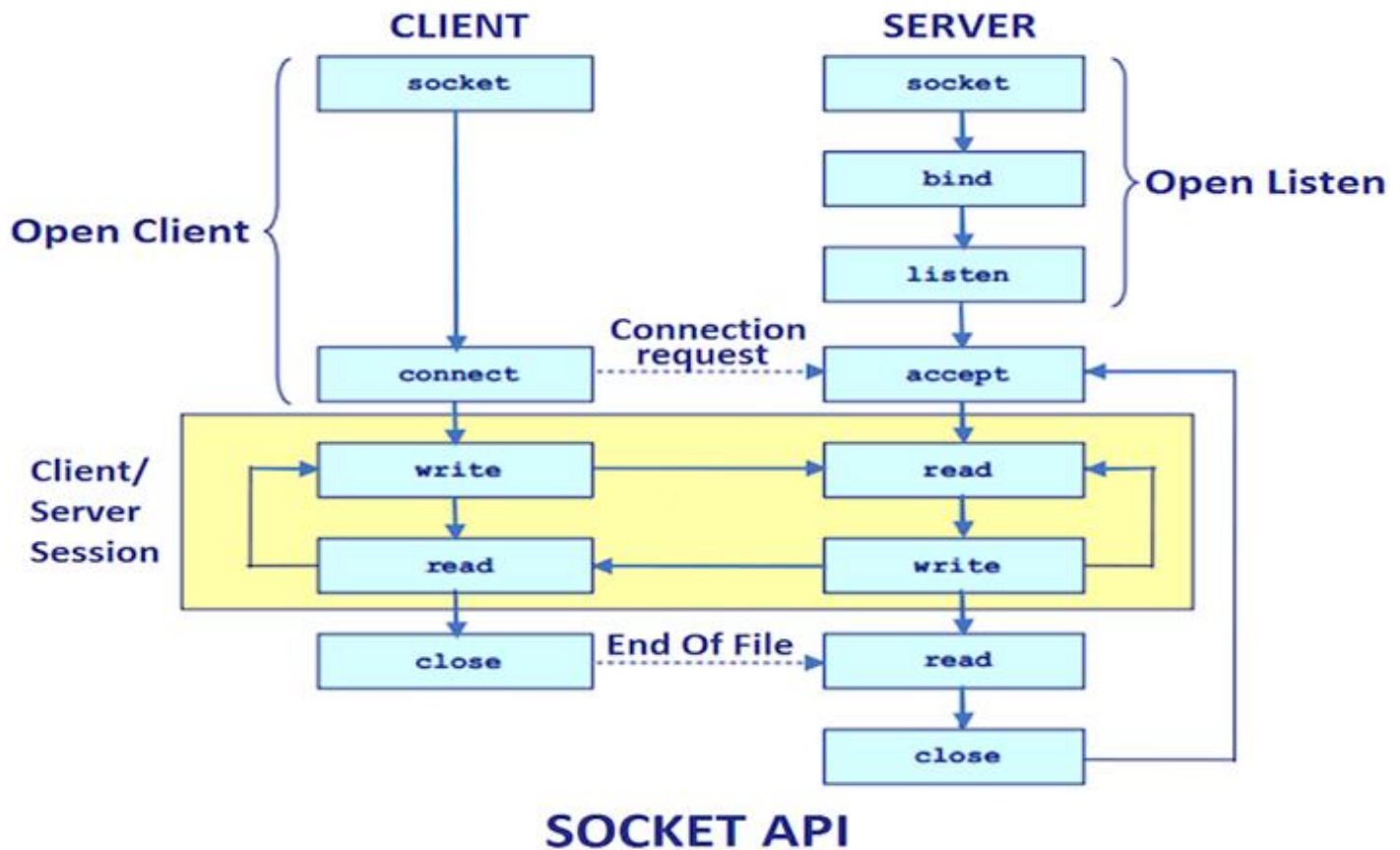
- Server_name: www.wcu.edu.et. The IP number represents this.
- A communication association is identified by two-half associations

{ protocol, local-address, local-port, remote-address, remote-port }

The server forms the listener *socket* while the client reaches out to the server. Socket and ServerSocket classes are used for connection-oriented socket programming. A **socket** is bound to a port number so that the TCP layer can identify the application that data is destined to be sent to. The package in the Java platform provides a class, Socket that implements one side of a two-way connection between your Java program and another program on the network. The class sits on top of a platform-dependent implementation, hiding the details of any particular system from your Java program.

The java.net package provides two classes:

1. **Socket Class:** Implements the client side of a two-way connection between your Java program and another program on the network by using **java.net.Socket**.
 - Socket classes are used to represent the connection between a client program and a server program.
 - The Socket class sits on top of a platform-dependent implementation, hiding the details of any particular system from your Java program. They are used to create a client Socket.



2.3 Manipulating a File on a Web Server

An application developed in java can use various kinds of mechanisms to manipulate the files from its directories. For our example purpose, we developed a java application that uses Swing GUI component **JEditorPane** (from package **javax.swing**) to display the contents of a file on a Web server. The user inputs the URI in the JTextField at the top of the window, and the program displays the corresponding document (if it exists) in the JEditorPane. Class JEditorPane is able to render both plain text and HTML-formatted text, so this application acts as a simple Web browser. The application also demonstrates how to process HyperlinkEvents when the user clicks a hyperlink in the HTML document. The screen captures illustrate that the JEditorPane can display both simple text (the first screen) and HTML text (the second screen). The techniques are shown in this example also can be used in applets. However, applets are allowed to read files only on the server from which the applet was downloaded.

```

2  // This program uses a JEditorPane to display the
3  // contents of a file on a Web server.
4
5  // Java core packages
6  import java.awt.*;
7  import java.awt.event.*;
8  import java.net.*;
9  import java.io.*;
10
11 // Java extension packages
12 import javax.swing.*;
13 import javax.swing.event.*;
14
15 public class ReadServerFile extends JFrame {
16     private JTextField enterField;
17     private JEditorPane contentsArea;

```

```

18
19 // set up GUI
20 public ReadServerFile()
21 {
22     super( "Simple Web Browser" );
23
24     Container container = getContentPane();
25
26     // create enterField and register its listener
27     enterField = new JTextField( "Enter file URL here" );
28
29     enterField.addActionListener(
30
31         new ActionListener() {
32
33             // get document specified by user
34             public void actionPerformed( ActionEvent event )
35             {
36                 getPage( event.getActionCommand() );
37             }
38
39         } // end anonymous inner class
40
41     ); // end call to addActionListener
42
43     container.add( enterField, BorderLayout.NORTH );
44

```

```

45 // create contentsArea and register HyperlinkEvent listener
46 contentsArea = new JEditorPane();
47 contentsArea.setEditable( false );
48
49 contentsArea.addHyperlinkListener(
50
51     new HyperlinkListener() {
52
53         // if user clicked hyperlink, go to specified page
54         public void hyperlinkUpdate( HyperlinkEvent event )
55         {
56             if ( event.getEventType() ==
57                 HyperlinkEvent.EventType.ACTIVATED )
58                 getThePage( event.getURL().toString() );
59         }
60     } // end anonymous inner class
61
62 ); // end call to addHyperlinkListener
63
64 container.add( new JScrollPane( contentsArea ),
65               BorderLayout.CENTER );
66
67
68 setSize( 400, 300 );
69 setVisible( true );
70 }

```

```

71
72 // load document; change mouse cursor to indicate stat
73 private void getThePage( String location )
74 {
75     // change mouse cursor to WAIT_CURSOR
76     setCursor( Cursor.getPredefinedCursor(
77         Cursor.WAIT_CURSOR ) );
78
79     // load document into contentsArea and display loca
80     // enterField
81     try {
82         contentsArea.setPage( location );
83         enterField.setText( location );
84     }
85
86     // process problems loading document
87     catch ( IOException ioException ) {
88         JOptionPane.showMessageDialog( this,
89             "Error retrieving specified URL",
90             "Bad URL", JOptionPane.ERROR_MESSAGE );
91     }
92
93     setCursor( Cursor.getPredefinedCursor(
94         Cursor.DEFAULT_CURSOR ) );
95 }
96
97 // begin application execution
98 public static void main( String args[] )
99 {
100     ReadServerFile application = new ReadServerFile();

```



```

101
102     application.setDefaultCloseOperation(
103         JFrame.EXIT_ON_CLOSE );
104     }
105
106 } // end class ReadServerFile

```



2.4 Establishing a Simple Server Using Stream Sockets

Step 1:- create a `ServerSocket` object. A call to the `ServerSocket` constructor such as

```
ServerSocket server = new ServerSocket( port, queueLength );
```

This **registers** an available **port number** and specifies a maximum number of clients that can wait to connect to the server (i.e., the **queueLength**). The port number is used by clients to the located server application on the server computer. This often is called the handshake **point**. If the queue is full, the server refuses client connections. The preceding statement establishes the port where the server waits for connections from clients (a process known as **binding the server to the port**). Each client will ask to connect to the server on this **port**. Programs manage each client connection with a `Socket` object. After binding the server to a port with a `ServerSocket`.

Step 2:- the server listens indefinitely (or blocks) for an attempt by a client to connect. To listen for a client, the program calls `ServerSocket` method to accept, as in

```
Socket connection = server.accept();
```

This statement returns a `Socket` object when a connection with a client is established. The `accept()` method is called by the server to validate an incoming request to the socket. It is called the *communication* phase.

Step 3:- Get the **`OutputStream`** and **`InputStream`** objects that enable the server to communicate with the client by sending and receiving bytes. The server sends information to the client via an **`OutputStream`** object. The server receives information from the client via an **`InputStream`** object. To obtain the streams, the server invokes method **`getOutputStream`** on the `Socket` to get a reference to the **`OutputStream`** associated with the `Socket` and invokes method **`getInputStream`** on the `Socket` to get a reference to the **`InputStream`** associated with the `Socket`.

The **`OutputStream`** and **`InputStream`** objects can be used to send or receive individual bytes or sets of bytes with the **`OutputStream`** method `write` and the **`InputStream`** method `read`, respectively. Often it is useful to send or receive values of primitive data types (such as `int` and `double`) or `Serializable` class data types (such as `String`) rather than sending bytes.

For Example:

```
ObjectInputStream input =  
    new ObjectInputStream( connection.getInputStream()  
  
ObjectOutputStream output =  
    new ObjectOutputStream( connection.getOutputStream()
```

The beauty of establishing these relationships is that whatever the server writes to the

`ObjectOutputStream` is sent via the **`OutputStream`** and is available at the client's **`InputStream`**

and whatever the client writes to its **`OutputStream`** (with a corresponding **`ObjectOutputStream`**) is available via the server's **`InputStream`**.

Step 4:- The processing phase, in which the server and the client communicate via the **`InputStream`** and **`OutputStream`** objects.

Step 5:- When the transmission is complete, the server closes the connection by invoking the `close` method on the `Socket` and on the corresponding streams.

2.5 Establishing a Simple Client Using Stream Sockets

Establishing a simple client in Java requires four steps.

Step 1:-Create a Socket to connect to the server. The Socket constructor establishes the connection to the server. For example, the statement:

```
Socket connection = new Socket( serverAddress, port );
```

uses the Socket constructor with two arguments—the server’s **Internet address** (serverAddress) and the port number. If the connection attempt is successful, this statement returns a Socket. A connection attempt that fails throws an instance of a subclass of IOException, so many programs simply catch IOException. An UnknownHostException occurs when a server address indicated by a client cannot be resolved. A ConnectException is thrown when an error occurs while attempting to connect to a server.

Step 2:-, Client uses Socket methods **getInputStream** and **getOutputStream** to obtain references to the Socket’s **InputStream** and **OutputStream**. As we mentioned in the preceding section, often it is useful to send or receive values of primitive data types (such as int and double) or class data types (such as String and Employee) rather than sending bytes. If the server is sending information in the form of actual data types, the client should receive the information in the same format. Thus, if the server sends values with an **ObjectOutputStream**, the client should read those values with an **ObjectInputStream**.

Step 3:- The processing phase in which the client and the server communicate via the **InputStream** and **OutputStream** objects.

Step 4:-, The client closes the connection when the transmission is complete by invoking the close method on the Socket and the corresponding streams. When processing information sent by a server, the client must determine when the server is finished sending information so the client can call close to close the Socket connection. For example, the InputStream method read returns the value –1 when it detects end-of-stream (also called EOF—end-of-file). If an ObjectInputStream is used to read information from the server, an EOFException occurs when the client attempts to read a value from a stream on which end-of-stream is detected.

2.6 Client/Server Interaction with Stream Socket Connection

The applications of stream sockets to demonstrate a simple client/server chat application. The server waits for a client connection attempt. When a client application connects to the server, the

server application sends a String object (remember that Strings are Serializable) indicating that the connection was successful to the client. Then the client displays the message. Both the client and the server applications contain JTextFields, which allow the user to type a message and send it to the other application. When the client or the server sends the String “TERMINATE”, the connection between the client and the server terminates. Then the server waits for the next client to connect.

2.7 Connectionless Client/Server Interaction with Datagram

We have been discussing **connection-oriented, streams-based transmission**. Now we consider **connectionless transmission with datagrams**. Connection-oriented transmission is like the telephone system in which you dial and are given a **connection** to the telephone of the person with whom you wish to communicate. The connection is maintained for the duration of your phone call, even when you are not talking.

Connectionless transmission with **datagrams** is more like the way mail is carried via the postal service. If a large message will not fit in one envelope, you break it into separate message pieces that you place in separate, sequentially numbered envelopes. Each of the letters is then mailed at the same time. The letters could arrive in order, out of order or not at all (although the last case is rare, it does happen). The person at the receiving end reassembles the message pieces into sequential order before attempting to make sense of the message.

If your message is small enough to fit in one envelope, you do not have to worry about the “out-of-sequence” problem, but it is still possible that your message might not arrive. One difference between datagrams and postal mail is that duplicates of datagrams can arrive on the receiving computer.

In the Client application, the user types a message into a JTextField and presses **Enter**. The program converts the message into a byte array and places it in a datagram packet that is sent to the server. The Server program receives the packet and displays the information in the packet, then **echoes** the packet back to the client.

When the client receives the packet, the client displays the information in the packet. In this example, the Client and Server classes are implemented similarly

Class Server defines two DatagramPackets that the server uses to send and receive information and one DatagramSocket that sends and receives these packets. The constructor for class Server (lines 20–41) creates the graphical user interface where the packets of information will be displayed. Next

the constructor creates the `DatagramSocket` in a try block. Line 32 uses the `DatagramSocket` constructor that takes an integer port number argument (5000) to bind the server to a port where the server can receive packets from clients. Clients sending packets to this Server specify port 5000 in the packets they send. The `DatagramSocket` constructor throws a `SocketException` if it fails to bind the `DatagramSocket` to a port.

*Specifying a port that is already in use or specifying an invalid port number when creating a **DatagramSocket** results in a **BindException**.*

DatagramSocket Server Program: **Server.java**

```
2 // Set up a Server that will receive packets from a
3 // client and send packets to a client.
4
5 // Java core packages
6 import java.io.*;
7 import java.net.*;
8 import java.awt.*;
9 import java.awt.event.*;
10
```

```

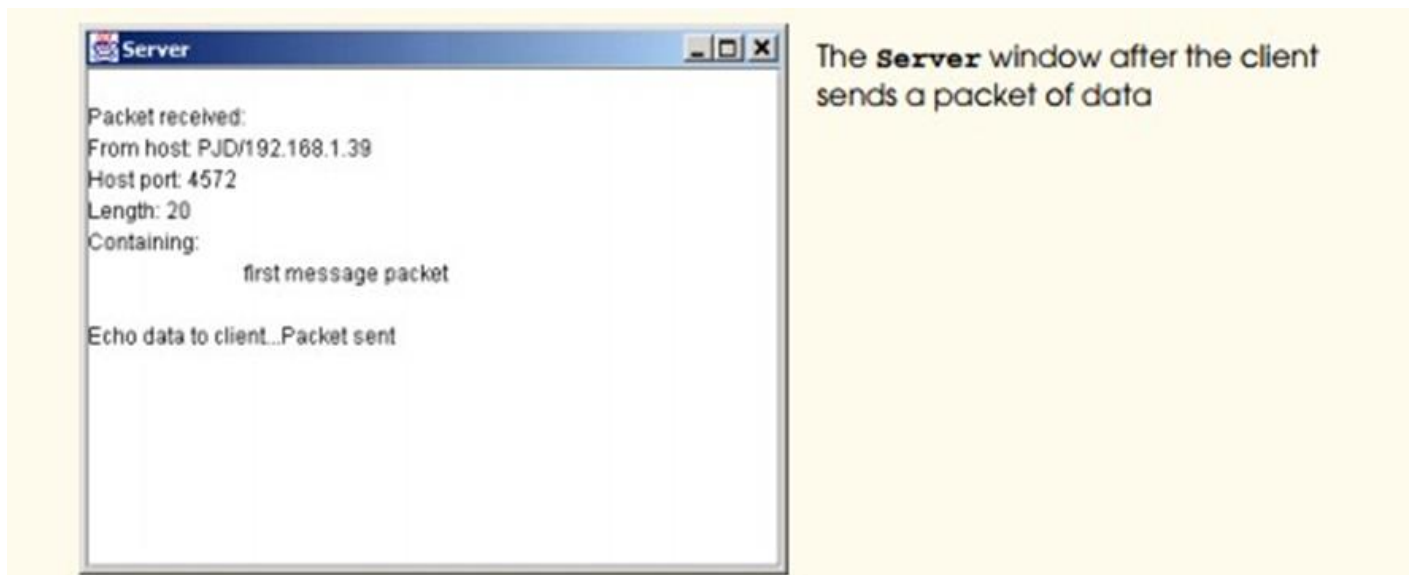
11 // Java extension packages
12 import javax.swing.*;
13
14 public class Server extends JFrame {
15     private JTextArea displayArea;
16     private DatagramPacket sendPacket, receivePacket;
17     private DatagramSocket socket;
18
19     // set up GUI and DatagramSocket
20     public Server()
21     {
22         super( "Server" );
23
24         displayArea = new JTextArea();
25         getContentPane().add( new JScrollPane( displayArea ),
26             BorderLayout.CENTER );
27         setSize( 400, 300 );
28         setVisible( true );
29
30         // create DatagramSocket for sending and receiving packets
31         try {
32             socket = new DatagramSocket( 5000 );
33         }
34
35         // process problems creating DatagramSocket
36         catch( SocketException socketException ) {
37             socketException.printStackTrace();
38             System.exit( 1 );
39         }
40
41     } // end Server constructor
42
43     // wait for packets to arrive, then display data and echo
44     // packet to client
45
46     public void waitForPackets()
47     {
48         // loop forever
49         while ( true ) {
50
51             // receive packet, display contents, echo to client
52             try {
53
54                 // set up packet
55                 byte data[] = new byte[ 100 ];
56                 receivePacket =
57                     new DatagramPacket( data, data.length );
58
59                 // wait for packet
60                 socket.receive( receivePacket );
61
62                 // process packet
63                 displayPacket();

```

```

63
64         // echo information from packet back to client
65         sendPacketToClient();
66     }
67
68     // process problems manipulating packet
69     catch( IOException ioException ) {
70         displayArea.append( ioException.toString() + "\n" );
71         ioException.printStackTrace();
72     }
73
74     } // end while
75
76 } // end method waitForPackets
77
78 // display packet contents
79 private void displayPacket()
80 {
81     displayArea.append( "\nPacket received:" +
82         "\nFrom host: " + receivePacket.getAddress() +
83         "\nHost port: " + receivePacket.getPort() +
84         "\nLength: " + receivePacket.getLength() +
85         "\nContaining:\n\t" +
86         new String( receivePacket.getData(), 0,
87             receivePacket.getLength() ) );
88 }
89
90 // echo packet to client
91 private void sendPacketToClient() throws IOException
92 {
93     displayArea.append( "\n\nEcho data to client..." );
94
95     // create packet to send
96
97     sendPacket = new DatagramPacket( receivePacket.getData(),
98         receivePacket.getLength(), receivePacket.getAddress(),
99         receivePacket.getPort() );
100
101     // send packet
102     socket.send( sendPacket );
103
104     displayArea.append( "Packet sent\n" );
105     displayArea.setCaretPosition(
106         displayArea.getText().length() );
107 }
108
109 // execute application
110 public static void main( String args[] )
111 {
112     Server application = new Server();
113
114     application.setDefaultCloseOperation(
115         JFrame.EXIT_ON_CLOSE );
116
117     application.waitForPackets();
118 }
119 } // end class Server

```



Demonstrating the server side of connectionless client/server computing with datagrams **Server** method **waitForPackets** (lines 45–76) uses an infinite loop to wait for packets to arrive at the **Server**. Lines 54–56 create a **DatagramPacket** in which a received packet of information can be stored. The **DatagramPacket** constructor for this purpose receives two arguments—a **byte** array containing the data and the length of the **byte** array.

2.8 Security and the Network

As much as we look forward to writing a great variety of powerful network-based applications, our efforts may be crimped by limitations imposed on Java because of security concerns. Many Web browsers, such as Netscape **Communicator** and Microsoft **Internet Explorer**, by default prohibit Java applets from doing file processing on the machines on which they execute. Think about it. A Java applet is designed to be sent to your browser via an HTML document that could be downloaded from any Web server in the world. Often you will know very little about the sources of Java applets that will execute on your system.

To allow these applets free rein with your files could be disastrous. A more subtle situation occurs with limiting the machines to which executing applets can connect. To build truly collaborative applications, we would ideally like to have our applets communicate with machines almost anywhere. The Java security manager in a Web browser often restricts an applet so that it can communicate only with the machine from which it was originally downloaded. These restrictions might seem too harsh. However, the Java Security API now provides capabilities for signed applets that will enable browsers to determine whether an applet is downloaded from a **trusted source**. In cases where an applet is trusted, the applet can be given additional access to the computer on which the applet is executing.

2.9 Chapter Review Questions

1. What is Socket?
2. Difference between TCP and UDP protocols?
3. How does the TCP handshake works?
4. What are the key steps in reading from a URL connection?
5. What are the key steps in writing to a URL connection?
6. Write a TCP client/server system in which the client program sends two numbers to a server program that returns the sum of the two numbers.
7. Write a UDP client/server system in which the client program sends a number between 1 and 10 and the server program returns the corresponding element in an array of strings.

CHAPTER THREE

3 Java Database connectivity with JDBC

3.1 Introduction

A **database** is a collection of related data. By data, we mean known facts that can be recorded and that have implicit meaning. For example, the names, telephone numbers, and addresses of the people you know. A DB is designed, built, and populated with data for a specific purpose. Database systems are everywhere. Your social security information is stored in a database by the government. If you shop online, your purchase information is stored in a database by the company. If you attend a university, your academic information is stored in a database by the university. A **database management system (DBMS)** is a collection of programs that enables users to create and maintain a database. The DBMS is a general-purpose software system that facilitates the processes of defining, constructing, manipulating, and sharing databases among various users and applications. Generally, a database system not only stores data, but they also provide means of accessing, updating,

manipulating, and analyzing data. Your social security information is updated periodically, and you can register in courses online. Database systems play an important role in society and in commerce. This chapter introduces database systems, SQL, and how to develop database applications using Java.

JDBC stands for **Java Database Connectivity**, which is a standard Java API for database-independent connectivity between the Java programming language and a wide range of databases.

The JDBC library includes APIs for each of the tasks commonly associated with database usage:

- Making a connection to a database
- Creating SQL or MySQL statements
- Executing that SQL or MySQL queries in the database
- Viewing & Modifying the resulting records

The JDBC is a package, much like other Java packages such as `java.awt`. It's not currently a part of the standard Java Developer's Kit (JDK) distribution, but it is scheduled to be included as a standard part of the general Java API as the `java.sql` package. Fundamentally, JDBC is a specification that provides a complete set of interfaces that allows for portable access to an underlying database.

Java can be used to write different types of executables, such as:

- Java Applications
- Java Applets
- Java Servlets
- Java ServerPages (JSPs)
- Enterprise JavaBeans (EJBs)

All of these different executables are able to use a JDBC driver to access a database and take advantage of the stored data. JDBC provides the same capabilities as ODBC, allowing Java programs to contain database-independent code.

Software Architectures

The first generation of client-server architectures is called two-tiered. It contains two active components: the client, which requests data, and the server, which delivers data. Basically, the application's processing is done separately for database queries and updates, and for user interface presentations. Usually, the network binds the back end to the front end, although both tiers could be present on the same hardware. For example, hundreds of thousands of airline seat reservation applications can connect to a central DBMS to request, insert, or modify data. While the clients process the graphics and data entry validation, the DBMS does all the data processing. Actually, it is inadvisable to overload the database engine with data processing that is irrelevant to the server, thus some processing usually also happens on the clients. The typical client-server architecture is shown in the Figure below:

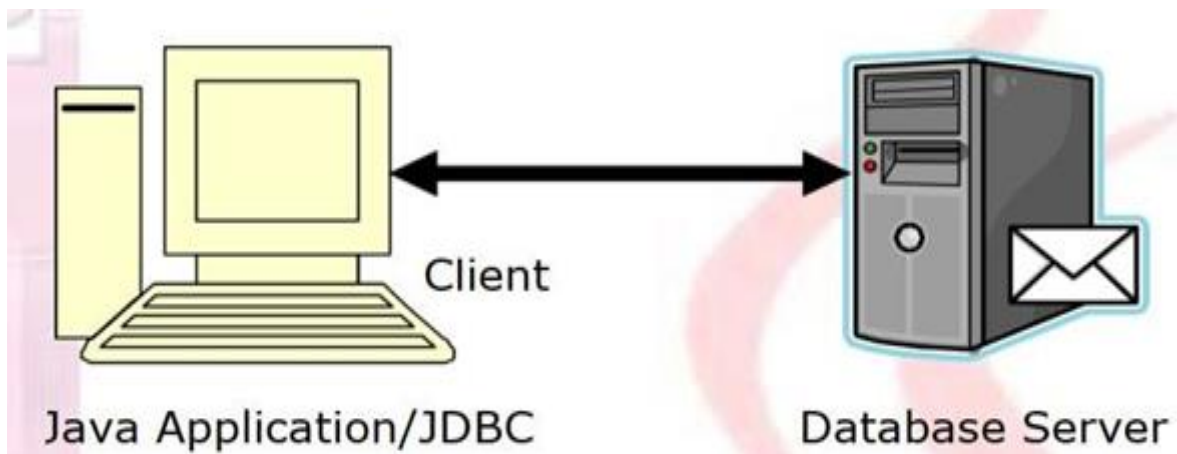


Figure 3

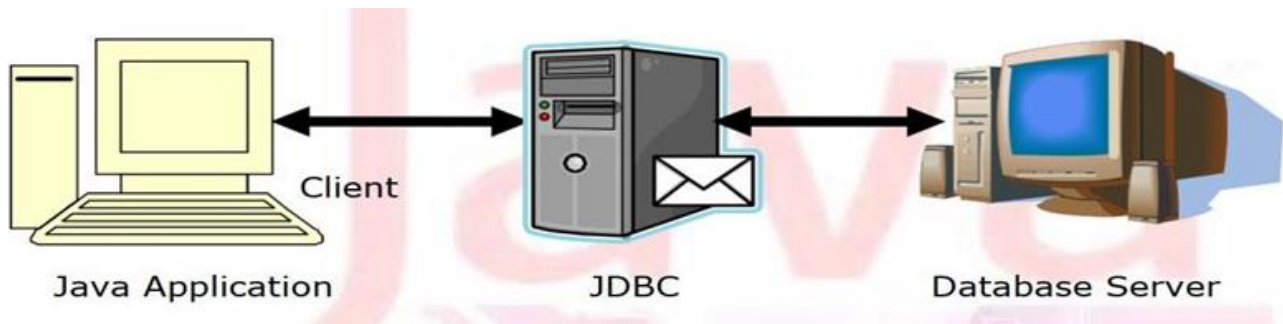
Two-tier client-server architecture

The two tiers are often called as **Application layer** includes JDBC drivers, business logic, and user interfaces whereas the second layer i.e. Database layer consists of the **RDBMS** server.

Advantages:

- It is simple in design.
- Client-side scripting offloads work onto the client
- Fat client.
- It is inflexible.

Although the two-tiered architecture is common, another design is starting to appear more frequently. To avoid embedding the application's logic at both the database side and the client side, a third software tier may be inserted. In three-tiered architectures, most of the business logic is frozen in the middle tier. In this architecture, when the business activity or business rules change, only the middleware must be modified. The figure below illustrates the three-tier architecture.

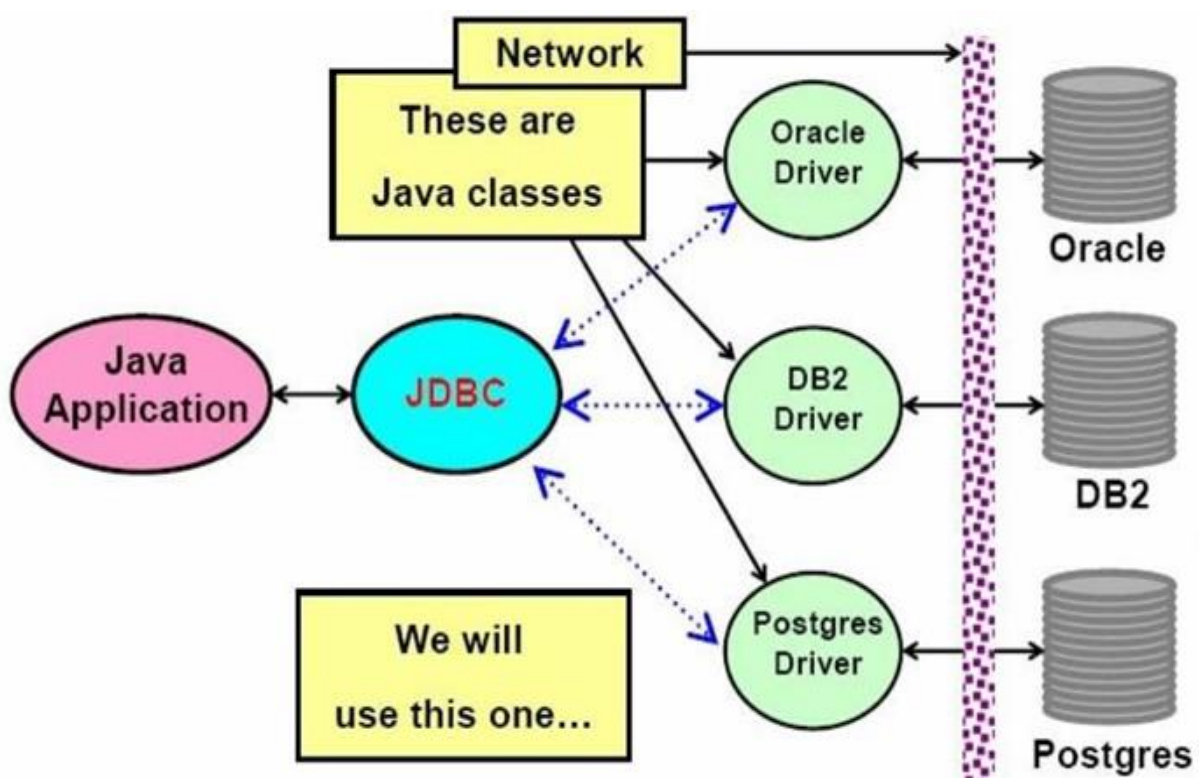


Advantages:

- Flexible: It can change one part without affecting others.
- It can connect to different databases without changing code.
- Specialization: presentation / business logic / data management.
- It can cache queries.
- It can implement proxies and firewalls.

Drawbacks:

- Higher complexity
- Higher maintenance
- Lower network efficiency
- More parts to configure (and buy)



The JDBC Architecture

3.2 Relational Database Systems

A **database system** consists of a database, the software that stores and manages data in the database, and the application programs that present data and enable the user to interact with the database system. It is shown as follows:

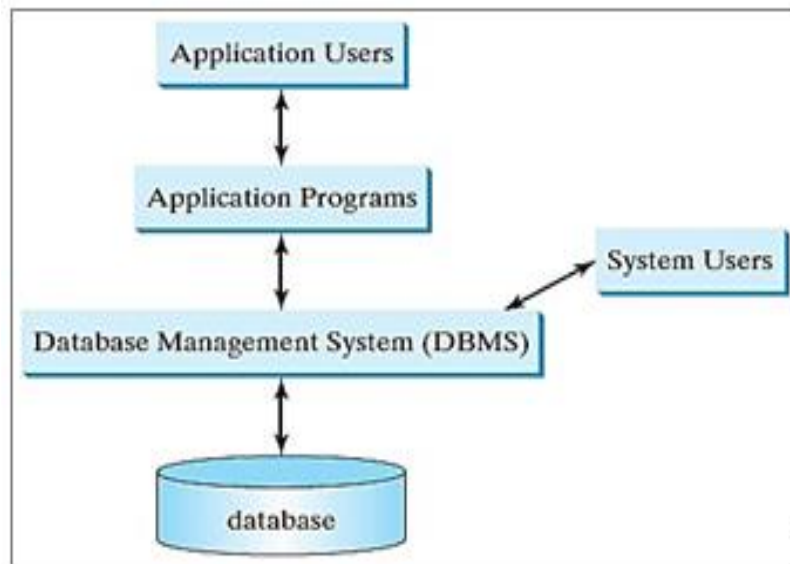
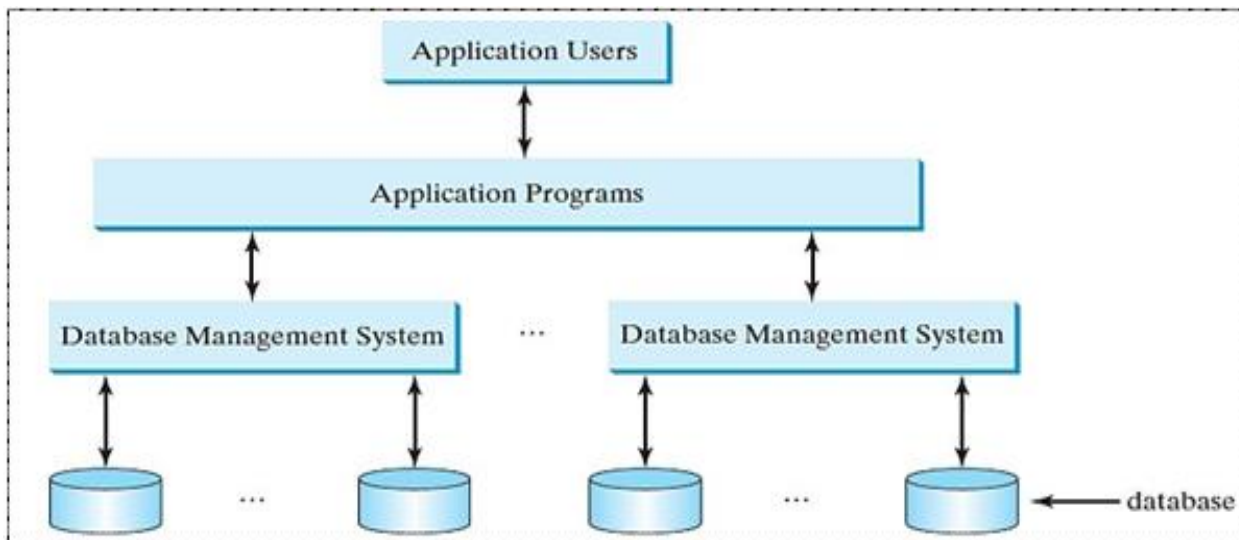


Figure above a database system consists of data, database management software, and application programs.

A **database** is a repository of data that forms information. When you purchase a database system, such as MySQL, Oracle, IBM, Microsoft SQL, or Sybase, from a software vendor, you actually purchase the software comprising a DBMS. DBMSs are designed for use by professional programmers and are not suitable for ordinary customers. Application programs are built on top of the DBMS for customers to access and update the database. Thus, application programs can be viewed as interfaces between the database system and its users. Application programs may be standalone GUI applications or Web applications and may access several different database systems in the network.

Most of today's database systems are relational database systems. They are based on the relational data model, which has three key components: **structure**, **integrity**, and **language**. Structure defines the representation of the data. Integrity imposes constraints on the data. Language provides the means for accessing and manipulating data.



3.3 Creating Database and Manipulating Data

3.3.1 SQL

Structured Query Language (SQL) is the **language** for defining tables and integrity constraints and for accessing and manipulating data. SQL (pronounced “S-Q-L” or “*sequel*”) is the universal language for accessing relational database systems. Application programs may allow users to access a database without directly using SQL, but these applications themselves must use SQL to access the database. SQL can be used on MySQL, Oracle, Sybase, IBM DB2, IBM Informix, Borland Interbase, MS Access, or any other relational database system.

3.3.2 Creating a User Account on MySQL

Assume that you have installed MySQL 5 or above with the default or if any MySQL is installed before on your computer, use another port number in the range of 3306 – 3309 configurations. Create a user named “root” with the password “root”. You can perform the administrative tasks using the MySQL command line.

Creating a UserName and Password:

Syntax:

```
create user 'UserName'@'localhost' identified by 'Password';
```

Example:

```
Create user 'oop' @'localhost' identified by 'it2';
```

ð Granting the User:

Syntax:

```
grant select, insert, update, delete, create, drop, execute, references on *.* to 'oop' @'localhost';
```

3.4 Creating Database

Syntax:

```
create database DatabaseName;
```

Example:

```
create database oopjava;
```

- To use the created Database:

Syntax:

```
use DatabaseName; // Example: use oopjava;
```

3.4.1.1 Creating and Dropping Tables

Tables are the essential objects in a database. To create a table, use the create table statement to specify a table name, attributes, and types.

To create a Table:

Syntax:

create table TableName (attributes DataTypes);

Example:

```
create table Geometry (geometryID int, geometryType varchar(20), primary key (geometryID));
```

Check if Tables is created or not:

Syntax:

```
describe TableName;
```

```
example: describe Geometry;
```

Dropping Table –If a table is no longer needed, it can be dropped permanently.

Syntax:

```
drop table TableName; // Example: drop table Geometry;
```

If a table to be dropped is referenced by other tables, you have to drop the other tables first.

3.4.1.2 Simple Insert, Update, and Delete

Once a table is created, you can insert data into it. You can also update and delete records.

- Insert values into Table:

Syntax:

```
insert into TableName [(attributes)](values);
```

Example:

```
insert into Geometry(geoID, geoType) values(100, 'Circle');
```

- Update values of a Table:

Syntax:

```
update TableName set column1 = newValue1 [, ...,] [where condition];
```

Example:

```
update Geometry set geoType = 'Rectangle';
```

- Delete values of a Table:

Syntax:

```
delete [from] TableName [where condition];
```

Example:

```
delete from Geometry where geoID = 100;
```

3.4.1.3 Simple Queries

To retrieve information from tables, use a **select** statement. The **select** clause lists the columns to be selected. The **from** clause refers to the tables involved in the query. The optional **where** clause specifies the conditions for the selected rows.

Syntax:

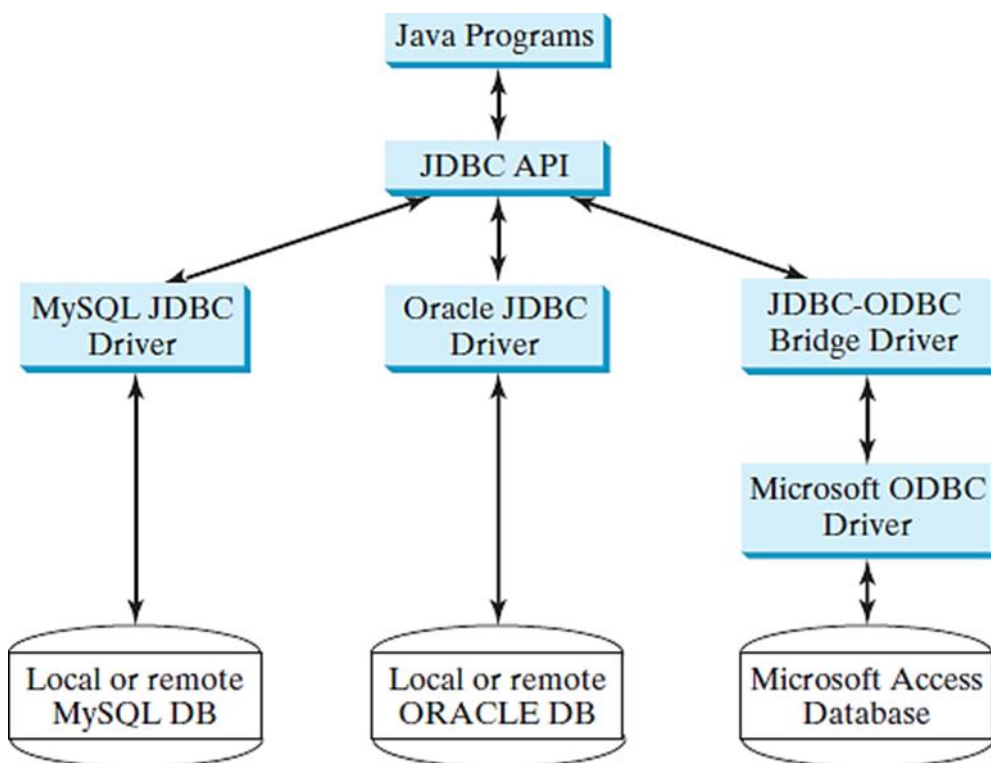
```
select attributes from TableName(list) [where condition];
```

Example:

```
select * from Geometry;
```

3.5 JDBC Drivers

JDBC drivers implement the defined interfaces in the JDBC API, for interacting with your database server. For example, using **JDBC drivers** enable you to open database connections and interact with them by sending SQL or database commands and then receiving results with Java. Following is the architectural diagram, which shows the location of the driver manager with respect to the JDBC drivers and the Java application:



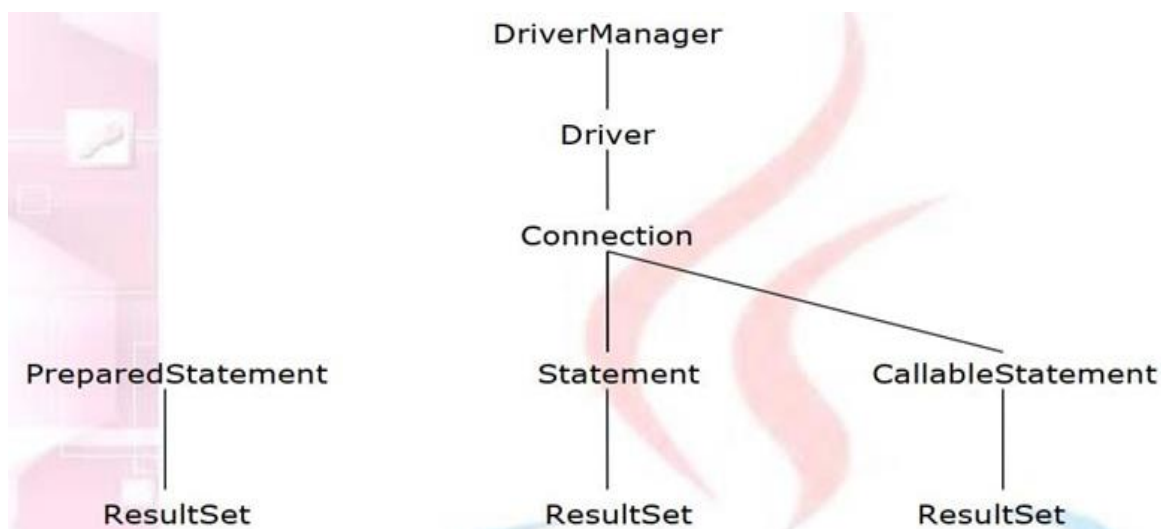
Java programs access and manipulate databases through JDBC drivers

3.6 The JDBC API

The JDBC API is a set of Java interfaces and classes used to write Java programs for accessing and manipulating relational databases. Since a *JDBC driver* serves as the *interface* to facilitate

communications between *JDBC* and a proprietary *database*, JDBC drivers are database specific and are normally provided by the database vendors. You need MySQL JDBC drivers to access the MySQL database, and Oracle JDBC drivers to access the Oracle database. For the Access database, use the JDBC-ODBC bridge driver included in JDK. ODBC is a technology developed by Microsoft for accessing databases on the Windows platform. An ODBC driver is preinstalled on Windows. The JDBC-ODBC bridge driver allows a Java program to access any ODBC data source.

The JDBC API is contained in two packages named `java.sql` and `javax.sql`. The `java.sql` package contains core Java objects of JDBC API. There are two distinct layers within the JDBC API: the application layer, which database application developers use, and the driver layer which the drivers' vendors implement. The connection between application and driver layers is illustrated in the figure below:



There are four main interfaces that every driver layer must implement and one class that bridges the Application and driver layers. These are `Driver`, `Connection`, `Statement` and `ResultSet`. The `Driver` interface implementation is where the connection to the database is made. In most applications, `Driver` is accessed through `DriverManager` class.

3.6.1 Common JDBC Components

The JDBC API provides the following interfaces and classes:

- **DriverManager** : This class manages a list of database drivers. Matches connection requests from the java application with the proper database driver using communication subprotocol. The first driver that recognizes a certain subprotocol under JDBC will be used to establish a database Connection.

- **Driver:** This interface handles the communications with the database server. You will interact directly with Driver objects very rarely. Instead, you use DriverManager objects, which manages objects of this type. It also abstracts the details associated with working with Driver objects. The **Java.sql** package that ships with JDK, contains various classes with their behaviors defined and their actual implementations are done in third-party drivers. Third party vendors implements the **java.sql.Driver** interface in their database driver.
- **Connection:** This interface with all methods for contacting a database. The connection object represents communication context, i.e., all communication with database is through connection object only.
- **Statement:** You use objects created from this interface to submit the SQL statements to the database. Some derived interfaces accept parameters in addition to executing stored procedures.
- **ResultSet:** These objects hold data retrieved from a database after you execute an SQL query using Statement objects. It acts as an iterator to allow you to move through its data.

SQLException: This class handles any errors that occur in a database application.

4 Developing Database Applications Using JDBC

The JDBC API is a Java application program interface to generic SQL databases that enables Java developers to develop DBMS-independent Java applications using a uniform interface. The JDBC API consists of classes and interfaces for establishing connections with databases, sending SQL statements to databases, processing the results of the SQL statements, and obtaining database metadata. Four key interfaces are needed to develop any database application using Java: **Driver**, **Connection**, **Statement**, and **ResultSet**. These interfaces define a framework for generic SQL database access. The JDBC API defines these interfaces. The JDBC driver vendors provide an implementation for them. Programmers use the interfaces. The relationship between these interfaces is shown in the figure below.

4.1.1 Creating JDBC Application

There are the following six steps involved in building a JDBC application:

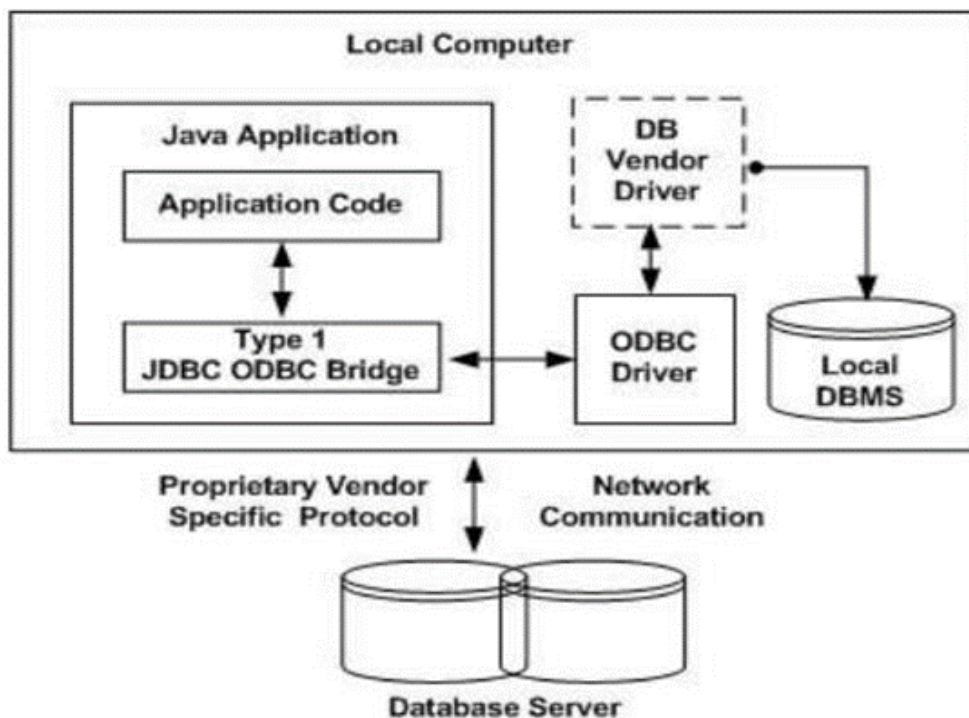
- Import the packages. Requires that you include the packages containing the JDBC classes needed for database programming. Most often, using **import java.sql.*** will suffice.
- Register the JDBC driver. Requires that you initialize a driver so you can open a communications channel with the database.

- Open a connection. Requires using the `DriverManager.getConnection()` method to create a `Connection` object, which represents a physical connection with the database.
- Execute a query. Requires using an object of type `Statement` for building and submitting an SQL statement to the database.
- Extract data from the result set. Requires that you use the appropriate `ResultSet.getXXX()` method to retrieve the data from the result set.
- Clean up the environment. Requires explicitly closing all database resources versus relying on the JVM's garbage collection.

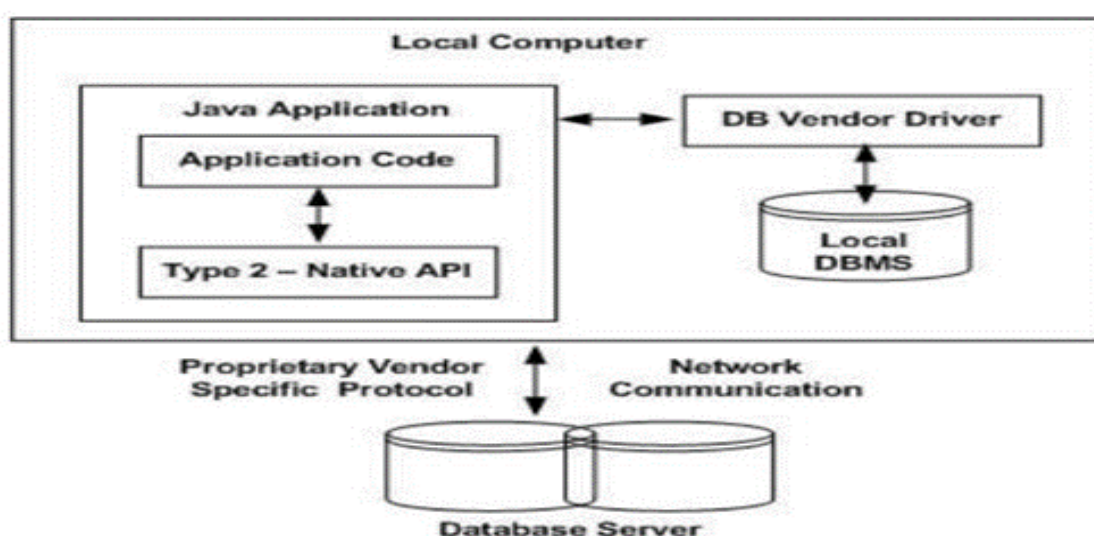
4.2 JDBC Drivers Types

JDBC driver implementations vary because of the wide variety of operating systems and hardware platforms in which Java operates. Sun has divided the implementation types into four categories, Types 1, 2, 3, and 4, which is explained below:

- **Type 1: JDBC-ODBC Bridge Driver:** In a Type 1 driver, a JDBC bridge is used to access ODBC drivers installed on each client machine. Using ODBC requires configuring on your system a Data Source Name (DSN) that represents the target database. When Java first came out, this was a useful driver because most databases only supported ODBC access but now this type of driver is recommended only for experimental use or when no other alternative is available.



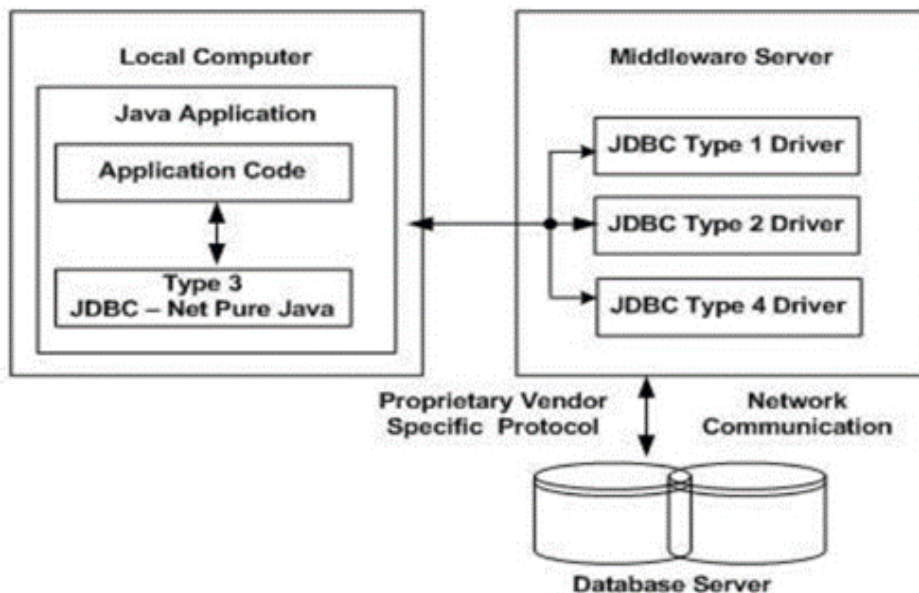
- The JDBC-ODBC bridge that comes with JDK 1.2 is a good example of this kind of driver. **Type 2: JDBC-Native API:** In a Type 2 driver, JDBC API calls are converted into native C/C++ API calls which are unique to the database. These drivers typically provided by the database vendors and used in the same manner as the JDBC-ODBC Bridge, the vendor- specific driver must be installed on each client machine. If we change the Database we have to change the native API as it is specific to a database and they are mostly obsolete now but you may realize some speed increase with a Type 2 driver because it eliminates ODBC's overhead.



The Oracle Call Interface (OCI) driver is an example of a Type 2 driver.

- Type 3: JDBC-Net pure Java: In a Type 3 driver, a three-tier approach is used to accessing databases. The JDBC clients use standard network sockets to communicate with a middleware

application server. The socket information is then translated by the middleware application server into the call format required by the DBMS and forwarded to the database server. This kind of driver is extremely flexible since it requires no code installed on the client and a single driver can actually provide access to multiple databases.



4.3 Chapter Review Questions

1. What is JDBC?
2. Explain the role of Driver in JDBC.
3. What is the purpose Class.forName method?
4. What is the advantage of PreparedStatement over Statement?
5. What does Connection pooling mean?
6. Create the tables Course, and Student with at least 4 (four) columns using an application program.
 1. Insert at least 10 data into the tables.
 2. List all courses with at least four credit hours.
 3. List all students whose last names contain the letter e two times.
 4. List all students whose birthdays are null.
7. What are the advantages of developing database applications using Java?

8. Describe the following JDBC interfaces: Driver, Connection, Statement, and ResultSet.
9. How do you load a JDBC driver? What are the driver classes for MySQL, Access, and Oracle?
10. How do you create a database connection? What are the URLs for MySQL, Access, and Oracle?
11. How do you create a Statement and execute an SQL statement?
12. How do you retrieve values in a ResultSet?
13. Describe PreparedStatement. How do you create instances of PreparedStatement?
14. How do you execute a PreparedStatement? How do you set parameter values in a PreparedStatement?

Chapter Four

5 SERVLETS

5.1 Introduction

A **servlet** is a Java programming language class that is used to extend the capabilities of servers that host applications accessed by means of a request-response programming model. Although servlets can respond to any type of request, they are commonly used to extend the applications hosted by web servers. For such applications, Java Servlet technology defines HTTP-specific servlet classes.

The `javax.servlet` and `javax.servlet.http` packages provide interfaces and classes for writing servlets. All servlets must implement the `Servlet` interface, which defines life-cycle methods. When implementing a generic service, you can use or extend the `GenericServlet` class provided with the Java Servlet API. The `HttpServlet` class provides methods, such as `doGet` and `doPost`, for handling HTTP-specific services. This chapter focuses on writing servlets that generate responses to HTTP requests.

5.2 Servlet Overview and Architecture

Servlets are grouped under the Advanced Java tree that are used to create dynamic web applications. Servlets are *robust in nature, well scalable* and are primarily used in developing *server-side applications*. If we go a little back in time, we would be able to witness that before the introduction of servlets, *CGI (Common Gateway Interface)* were used. Amid several indigenous tasks that a servlet is capable of doing, dynamically performing client requests and responses are most common. Other tasks that a servlet can do effectively are:

- Can easily manage/control the application flow.
- Suitable to implement business logic.
- Can effectively balance the load at the server-side.
- Easily generate dynamic web content.

Talking about the types of servlets, there are primarily two types, namely:

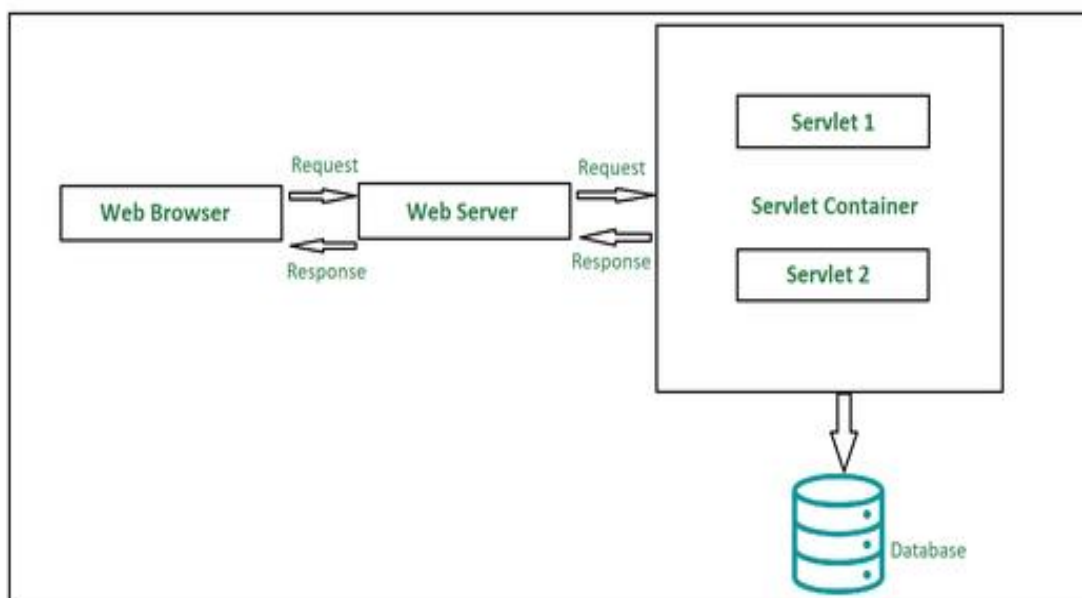
1. Generic Servlets
2. HTTP Servlets

There are three potential ways in which we can employ to create a servlet:

1. Implementing Servlet Interface
2. Extending Generic Servlet
3. Extending HTTP Servlet

5.2.1 Components of Servlet Architecture

Below is the high level architecture diagram of servlet. Let's see in brief, how does each component add to the working of a servlet.



Servlet Architecture

1. Client

The client shown in the architecture above is primarily working as a medium who is sending out HTTP requests over to the web server and again processing the response it

gets back from the server. As we can see in the diagram, our client here is the web browser.

2. Web Server

Primary job of a web server is to process the requests and responses that a user sends over time and maintain how a web user would be able to access the files that has been hosted over the server. The server we are talking about here is a software which manages access to a centralized resource or service in a network.. There are precisely two types of web servers:

1. Static web server
2. Dynamic web server

3. Web Container

Web container is another typical component in servlet architecture which is responsible for communicating with the servlets. Two prime tasks of a web container are:

- Managing the servlet lifecycle
- URL mapping

Web container sits at the server-side managing and handling all the requests that are coming in either from the servlets or from some JSP pages or potentially any other file system.

5.2.2 How does a Servlet Request flow?

Every servlet should override the following 3 methods namely:

1. `init()`
2. `service()`
3. `destroy()`

These methods are used to process the request from the user.

Following are the steps in which a request flows through a servlet which can be observed in the architecture diagram:

- The client sends over a request.
- The request is accepted by the web server and forwarded to the web container.
- In order to obtain the servlet's address, the web container traces *web.xml* file corresponding to the request *URL pattern*.
- By the time above process takes place, the servlet should have been instantiated and initialized. If the servlet has not been instantiated and initialized, *init()* method is invoked to serve the purpose.
- By passing *ServletRequest* and *Response* object, *public service()* method is called by the container.

- In the next step, the *ServletRequest* and *ServletResponse* objects are type-casted to *HttpServletRequest* and *HttpServletResponse* objects by the *public service()* method.
- Now *protected service()* method is called by the *public service()* method.
- The *protected service()* method dispatches the request to the correct handler method based on the type of request.
- When servlet container shuts down, it unloads all the servlets and calls *destroy()* method for each initialized servlets.

Advantages

- Prime functionality of a servlet is that they are independent of server configuration and they are pretty much compatible with any of the web servers
- Servlets are also protocol-independent supporting *FTP*, *HTTP*, *SMTP*, etc. protocols to the fullest.
- Until destroyed manually, servlets can be retained in the memory helping process several requests over time. Also, once a database connection is established, it can facilitate process several requests for a database in the very same database session.
- Servlets inherit Java's property of portability and hence are compatible with nearly any web server.
- Servlets are first converted into byte codes and then executed, which helps in increasing the processing time.

Disadvantages

- Designing a servlet can be pretty laborious.
- Exceptions need to be handled while designing a servlet since they are not thread-safe.
- Developers may need additional skills to program a servlet.

As we already know Servlets are portable (platform/server independent) in nature and hence are a better option if we talk in terms of other scripting languages. They process the requests and responses dynamically. Whenever we are developing a web application where we need to coordinate with different existing protocols, servlets are preferred over other means because of its capability to support various protocols. At last we can descend to a conclusion that employing a servlet could potentially be most suitable while developing a web application.

5.3 Life Cycle of a Servlet

The entire life cycle of a Servlet is managed by the **Servlet container** which uses the **javax.servlet.Servlet** interface to understand the Servlet object and manage it. So, before creating a Servlet object, let's first understand the life cycle of the Servlet object which is actually understanding how the Servlet container manages the Servlet object.

Stages of the Servlet Life Cycle: The Servlet life cycle mainly goes through four stages,

- Loading a Servlet
- Initializing the Servlet
- Request handling
- Destroying the Servlet.

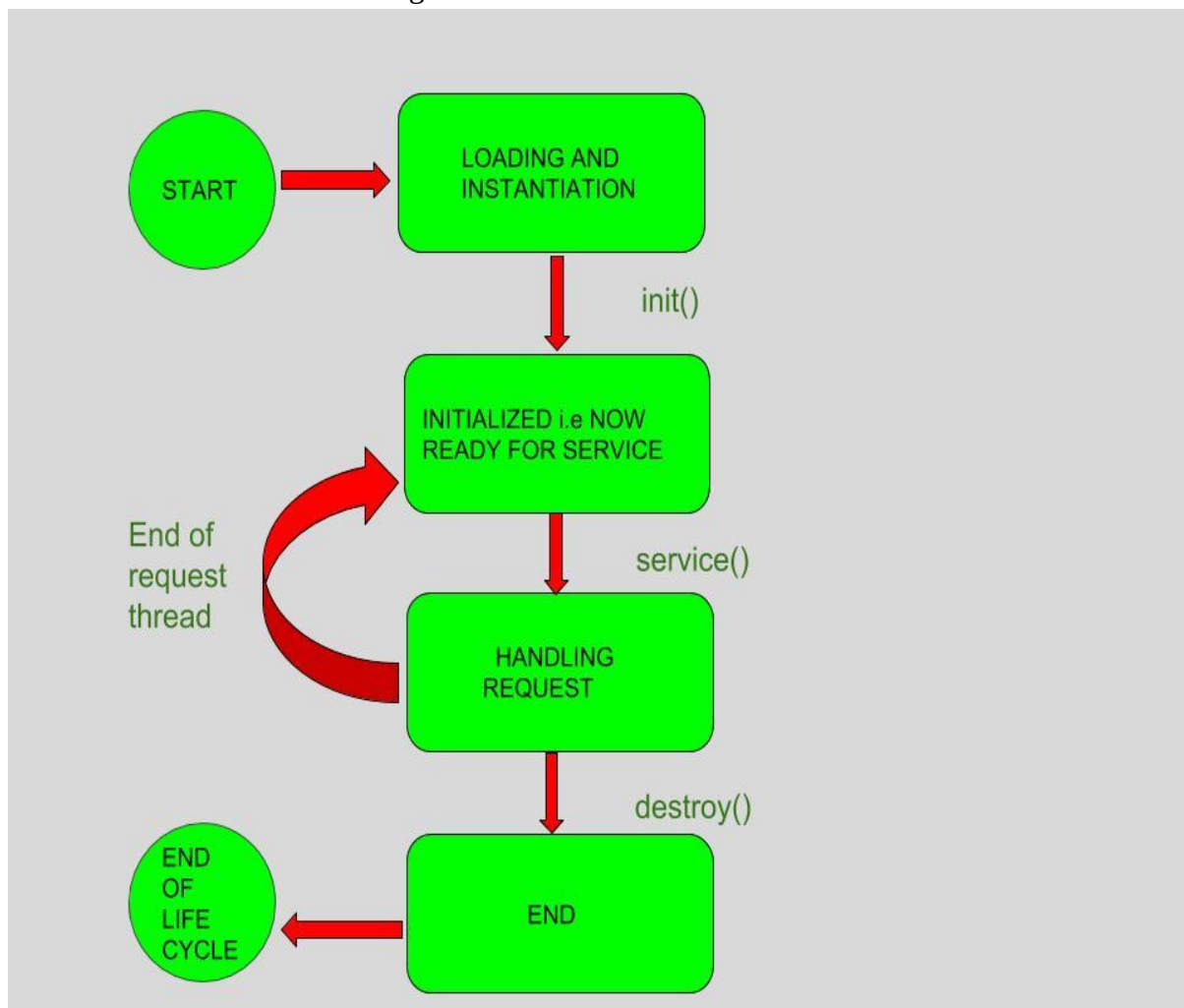
Let's look at each of these stages in details:

1. **Loading a Servlet:** The first stage of the Servlet lifecycle involves loading and initializing the Servlet by the Servlet container. The Web container or Servlet Container can load the Servlet at either of the following two stages :

- Initializing the context, on configuring the Servlet with a zero or positive integer value.
- If the Servlet is not preceding stage, it may delay the loading process until the Web container determines that this Servlet is needed to service a request.

The Servlet container performs two operations in this stage :

- **Loading** : Loads the Servlet class.
- **Instantiation** : Creates an instance of the Servlet. To create a new instance of the Servlet, the container uses the no-argument constructor.



2. **Initializing a Servlet:** After the Servlet is instantiated successfully, the Servlet container initializes the instantiated Servlet object. The container initializes the Servlet object by invoking the **Servlet.init(ServletConfig)** method which accepts ServletConfig object reference as parameter.

The Servlet container invokes the **Servlet.init(ServletConfig)** method only once, immediately after the **Servlet.init(ServletConfig)** object is instantiated successfully. This method is used to initialize the resources, such as JDBC datasource.

Now, if the Servlet fails to initialize, then it informs the Servlet container by throwing the **ServletException** or **UnavailableException**.

3. **Handling request:** After initialization, the Servlet instance is ready to serve the client requests. The Servlet container performs the following operations when the Servlet instance is located to service a request :

- It creates the **ServletRequest** and **ServletResponse** objects. In this case, if this is a HTTP request, then the Web container creates **HttpServletRequest** and **HttpServletResponse** objects which are subtypes of the **ServletRequest** and **ServletResponse** objects respectively.
- After creating the request and response objects it invokes the **Servlet.service(ServletRequest, ServletResponse)** method by passing the request and response objects.

The **service()** method while processing the request may throw the **ServletException** or **UnavailableException** or **IOException**.

4. **Destroying a Servlet:** When a Servlet container decides to destroy the Servlet, it performs the following operations,

- It allows all the threads currently running in the service method of the Servlet instance to complete their jobs and get released.
- After currently running threads have completed their jobs, the Servlet container calls the **destroy()** method on the Servlet instance.

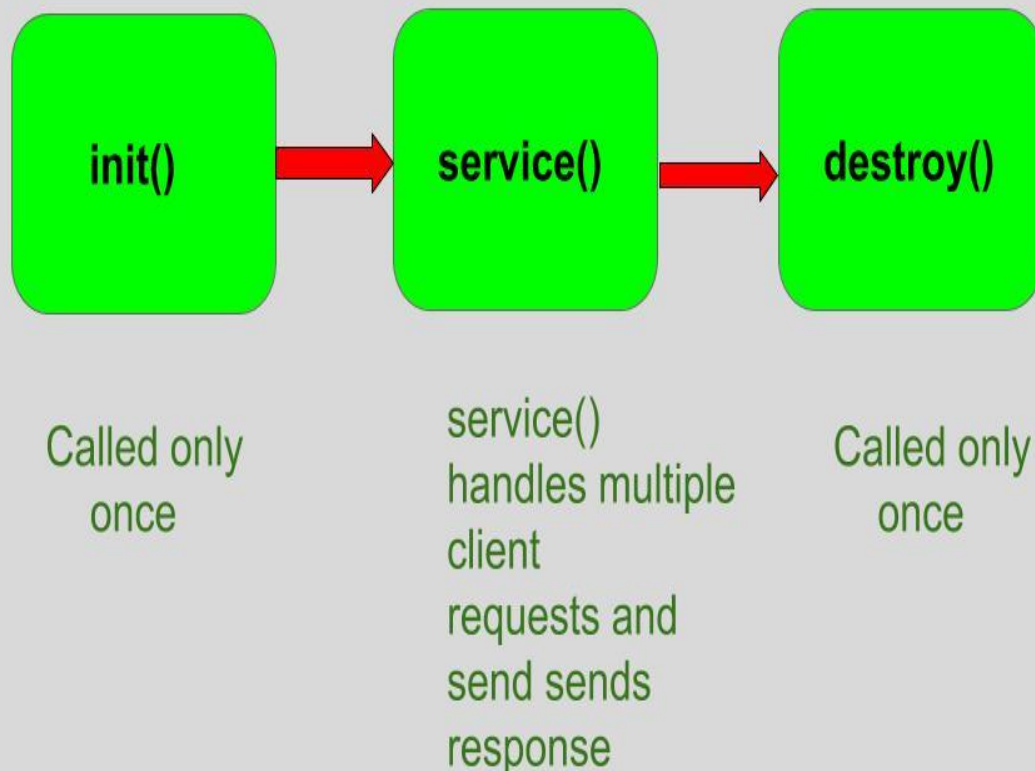
After the **destroy()** method is executed, the Servlet container releases all the references of this Servlet instance so that it becomes eligible for garbage collection.

5.3.1 Servlet Life Cycle Methods

There are three life cycle methods of a Servlet :

- **init()**
- **service()**
- **destroy()**

Life cycle methods of a Servlet



Let's look at each of these methods in details:

init() method: The **Servlet.init()** method is called by the Servlet container to indicate that this Servlet instance is instantiated successfully and is about to put into service.

//init() method

```
public class MyServlet implements Servlet{
    public void init(ServletConfig config) throws
    ServletException {
        //initialization code
    }
}
```

service() method: The **service()** method of the Servlet is invoked to inform the Servlet about the client requests.

- This method uses **ServletRequest** object to collect the data requested by the client.
- This method uses **ServletResponse** object to generate the output content.

// service() method

```
public class MyServlet implements Servlet{  
    public void service(ServletRequest res, ServletResponse  
res)  
        throws ServletException, IOException {  
        // request handling code  
    }  
    // rest of code  
}
```

2. **destroy() method:** The **destroy()** method runs only once during the lifetime of a Servlet and signals the end of the Servlet instance.

//destroy() method

```
public void destroy()
```

As soon as the **destroy()** method is activated, the Servlet container releases the Servlet instance.

Servlet Life Cycle:

Servlet life cycle can be defined as the stages through which the servlet passes from its creation to its destruction.

The servlet life cycle consists these stages:

- Servlet is borned
- Servlet is initialized
- Servlet is ready to service
- Servlet is servicing
- Servlet is not ready to service
- Servlet is destroyed

5.3.2 Handling Servlet Life-Cycle Events

You can monitor and react to events in a servlet's life cycle by defining listener objects whose methods get invoked when life-cycle events occur. To use these listener objects you must define and specify the listener class.

5.4 Defining the Listener Class

You define a listener class as an implementation of a listener interface. Table 4-1 lists the events that can be monitored and the corresponding interface that must be implemented. When a listener method is invoked, it is passed an event that contains information appropriate to the event. For example, the methods in the `HttpSessionListener` interface are passed an `HttpSessionEvent`, which contains an `HttpSession`.

Table 4-1 Servlet Life-Cycle Events

Object	Event	Listener Interface and Event Class
Web context (see Accessing the Web Context)	Initialization and destruction	<code>javax.servlet.ServletContextListener</code> and <code>ServletContextEvent</code>
	Attribute added, removed, or replaced	<code>javax.servlet.ServletContextAttributeListener</code> and <code>ServletContextAttributeEvent</code>
Session (See Maintaining Client State)	Creation, invalidation, activation, passivation, and timeout	<code>javax.servlet.http.HttpSessionListener</code> , <code>javax.servlet.http.HttpSessionActivationListener</code> , and <code>HttpSessionEvent</code>
	Attribute added, removed, or replaced	<code>javax.servlet.http.HttpSessionAttributeListener</code> and <code>HttpSessionBindingEvent</code>
Request	A servlet request has started being processed by web components	<code>javax.servlet.ServletRequestListener</code> and <code>ServletRequestEvent</code>
	Attribute added, removed, or replaced	<code>javax.servlet.ServletRequestAttributeListener</code> and <code>ServletRequestAttributeEvent</code>

The methods retrieve the web context object from `ServletContextEvent` and then store (and remove) the objects as servlet context attributes.

```
import database.BookDBAO;
import javax.servlet.*;
import util.Counter;
import javax.ejb.*;
import javax.persistence.*;
```

```
public final class ContextListener
    implements ServletContextListener {
    private ServletContext context = null;

    @PersistenceUnit
    EntityManagerFactory emf;

    public void contextInitialized(ServletContextEvent event) {
        context = event.getServletContext();
        try {
            BookDBAO bookDB = new BookDBAO(emf);
            context.setAttribute("bookDB", bookDB);
        } catch (Exception ex) {
```

```

        System.out.println(
            "Couldn't create database: " + ex.getMessage());
    }
    Counter counter = new Counter();
    context.setAttribute("hitCounter", counter);
    counter = new Counter();
    context.setAttribute("orderCounter", counter);
}

public void contextDestroyed(ServletContextEvent event) {
    context = event.getServletContext();
    BookDBAO bookDB = context.getAttribute("bookDB");
    bookDB.remove();
    context.removeAttribute("bookDB");
    context.removeAttribute("hitCounter");
    context.removeAttribute("orderCounter");
}
}

```

5.4.1 Specifying Event Listener Classes

You specify an event listener class using the `listener` element of the deployment descriptor. Review The Example Servlets for information on how to specify the `ContextListener` listener class.

You can specify an event listener using the deployment descriptor editor of NetBeans IDE by doing the following:

1. Expand your application's project node.
2. Expand the project's Web Pages and WEB-INF nodes.
3. Double-click `web.xml`.
4. Click General at the top of the `web.xml` editor.
5. Expand the Web Application Listeners node.
6. Click Add.
7. In the Add Listener dialog, click Browse to locate the listener class.
8. Click OK.

5.5 Handling Servlet Errors

Any number of exceptions can occur when a servlet executes. When an exception occurs, the web container generates a default page containing the message

A Servlet Exception Has Occurred

But you can also specify that the container should return a specific error page for a given exception. Review the deployment descriptor file included with the example to learn how to map the exceptions `exception.BookNotFound`, `exception.BooksNotFound`, and `exception.OrderException` returned by the Duke's Bookstore application to `errorpage.html`.

Life cycle methods

Life cycle methods are those methods which are used to control the life cycle of the servlet. These methods are called in specific order during the servlets's entire life cycle.

The class **Servlet** provides the methods to control and supervise the life cycle of servlet. There are three life cycle methods in the Servlet interface. There are as follows:

- **init() method :**

1. A servlet's life begins here .
2. This method is called only once to load the servlet. Since it is called only once in its lifetime, therefore "connected architecture" code is written inside it because we only want once to get connected with the database.

Now Question Arises is that:-

Q. Why can't we write connected architecture code inside the constructor, since constructor also run only once in its entire life?

Ans. Suppose if the connection doesn't get established, then we can throw an exception from init() and the rest of the steps stop executing. But in the constructor we can't use, throw in its prototype otherwise it is an error.

3. This method receives only one parameter, i.e **ServletConfig** object.
4. This method has the possibility to throw the ServletException.
5. Once the servlet is initialized, it is ready to handle the client request.
6. The prototype for the init() method:

```
public void init(ServletConfig con) throws ServletException{ }
```

where **con** is ServletConfig object

NOTE:- In programs of servlet, we use non parameterized version of **init()**.

- **Now, Question Arises is that:-**

Q. Why it is recommended to use the non parameterized version of init() instead of parameterized version as seen above?

Ans. To answer this, we have to go into detail. Think like developers, i.e there must be some valid reason for this and the answer will blow your mind. Coming to answer:

APPROACH 1

Whenever the lifecycle method of a servlet starts executing, i.e when public void init(ServletConfig con) throws ServletException gets call then our class public void init(ServletConfig con) throws ServletException gets called but we have to run the code which initializes servlet config object which is written inside "HttpServlet" method public void init(ServletConfig con) throws ServletException, i.e:

Coding of HttpServlet class be like:

```
public void init(ServletConfig con) throws ServletException
```

```

{
    //code to initialise ServletConfig object
    init();    //This HttpServlet has 2 init() one which is
              //parameterized and the other one is non
              //parameterized. But this non parameterized version of
              //init() has a blank body.
    //So this call is useless.
}

```

Now see the coding of our class

```

public void init(ServletConfig con) throws ServletException
{
    super.init(con);    //Since, our class init() will run first, but
                       //to run HttpServlet init() // have used super keyword. And
                       //Database connectivity code will be their }
}

```

NOTE:- As we can see, total 3 init() calls we have to make. First init() gets called of our class then of HttpServlet class then non parameterized version of HttpServlet class. But now, we will achieve the same thing with less number of calls:

APPROACH 2

Coding of HttpServlet parametrized and non parameterized versions of init() will remain the same. **But in our class instead of overriding parameterized version of init(), we will override non parameterized version of init().** Let's see the coding of our class non parameterized version of init():

```

public void init() throws ServletException
{
    //database connectivity code
}

```

NOTE: Since this method public void init() throws ServletException, we have override from HttpServlet class whose coding is like:

```

public void init() throws ServletException
{
    //empty body
}

```

Since its body is blank, therefore it is known as “**Helper method**” as it is used for overriding purpose. Now, as the servlet starts executing its methods, it will call the parameterized version of `init()`. Since we have not to override the parameterized version, therefore it will give a call to the `HttpServlet` parameterized version of `init()`. Since coding of a parameterized version of `init()` of `HttpServlet` is as same as above, therefore, from there on it will call `init()` (i.e non parameterized version of `init()`). It will give a call to our class non parameterized version of `init()` and the code continues.

Now, as you can see, total number of `init()` calls are 2 which is less than the first approach. Therefore, execution time is less in 2nd approach and less headache for CPU for maintaining stack and its speed increases as compared to 1st approach.

Therefore, it is highly recommended to override non parameterized version of `init()`. Although both will run but due to efficiency first approach is rarely used and also in first approach we have to use `super` keyword too. Therefore in below mentioned program, we have override non parameterized version of `init()`.

- **Service() method :**

1. The `service()` method is the most important method to perform that provides the connection between client and server.
2. The web server calls the `service()` method to handle requests coming from the client(web browsers) and to send response back to the client.
3. This method determines the type of `Http` request (`GET`, `POST`, `PUT`, `DELETE`, etc.) .
4. This method also calls various other methods such as `doGet()`, `doPost()`, `doPut()`, `doDelete()`, etc. as required.
5. This method accepts two parameters.
6. The prototype for this method:
7. `public void service(ServletRequest req, ServletResponse resp)`

throws `ServletException`, `IOException` { }

where:

- **req** is the `ServletRequest` object which encapsulates the connection from client to server
- **resp** is the `ServletResponse` object which encapsulates the connection from server back to the client

- **Destroy() method :**

1. The `destroy()` method is called only once.
2. It is called at the end of the life cycle of the servlet.
3. This method performs various tasks such as closing connection with the database, releasing memory allocated to the servlet, releasing resources that are allocated to the servlet and other cleanup activities.
4. When this method is called, the garbage collector comes into action.
5. The prototype for this method is:

```
public void destroy() { // Finalization code...}
```

Below is a sample program to illustrate Servlet in Java:

```
// Java program to show servlet example

// Importing required Java libraries

import java.io.*;

import javax.servlet.*;

import javax.servlet.http.*;

    // Extend HttpServlet class

public class AdvanceJavaConcepts extends HttpServlet

{

    private String output;

    // Initializing servlet

    public void init() throws ServletException

    {

        output = "Advance Java Concepts";

    }

    // Requesting and printing the output

    public void doGet(HttpServletRequest req,

                        HttpServletResponse resp)

        throws ServletException, IOException

    {
```



```

        resp.setContentType("text/html");

        PrintWriter out = resp.getWriter();

        out.println(output);

    }

    public void destroy()

    {

        System.out.println("Over");

    }

}

```

5.6 Handling HTTP get Request

5.6.1 Handling HTTP get Requests Containing Data

When requesting a document or resource from a Web server, it is possible to supply data as part of the request. The servlet `WelcomeServlet2` in below Example responds to an HTTP `get` request that contains a name supplied by the user. The servlet uses the name as part of the response to the client.

Example. `WelcomeServlet2` responds to a `get` request containing data.

```

1 // Example 26.12: WelcomeServlet2.java
2 // Processing HTTP get requests containing data.
3 import javax.servlet.ServletException;
4 import javax.servlet.http.HttpServlet;
5 import javax.servlet.http.HttpServletRequest;
6 import javax.servlet.http.HttpServletResponse;
7 import java.io.IOException;

```

```

8 import java.io.PrintWriter;
9
10 public class WelcomeServlet2 extends HttpServlet
11 {
12 // process "get" request from client
13 protected void doGet( HttpServletRequest request,
14 HttpServletResponse response )
15 throws ServletException, IOException
16 {
17 String firstName = request.getParameter( "firstname" );
18
19 response.setContentType( "text/html" );
20 PrintWriter out = response.getWriter();
21
22 // send XHTML document to client
23
24 // start XHTML document
25 out.println( "<?xml version = \"1.0\"?>" );
26
27 out.printf( "%s%s%s" , "<!DOCTYPE html PUBLIC" ,
28 " \"-//W3C//DTD XHTML 1.0 Strict//EN\"",
29 " \"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd\">\n" );
30
31 out.println( "<html xmlns = \"http://www.w3.org/1999/xhtml\">" );
32
33 // head section of document
34 out.println( "<head>" );
35 out.println(
36 "<title>Processing get requests with data</title>" );
37 out.println( "</head>" );
38
39 // body section of document
40 out.println( "<body>" );
41 out.println( "<h1>Hello " + firstName + ",<br />" );
42 out.println( "Welcome to Servlets!</h1>" );
43 out.println( "</body>" );
44
45 // end XHTML document
46 out.println( "</html>" );
47 out.close(); // close stream to complete the page
48 } // end method doGet
49 } // end class WelcomeServlet2

```

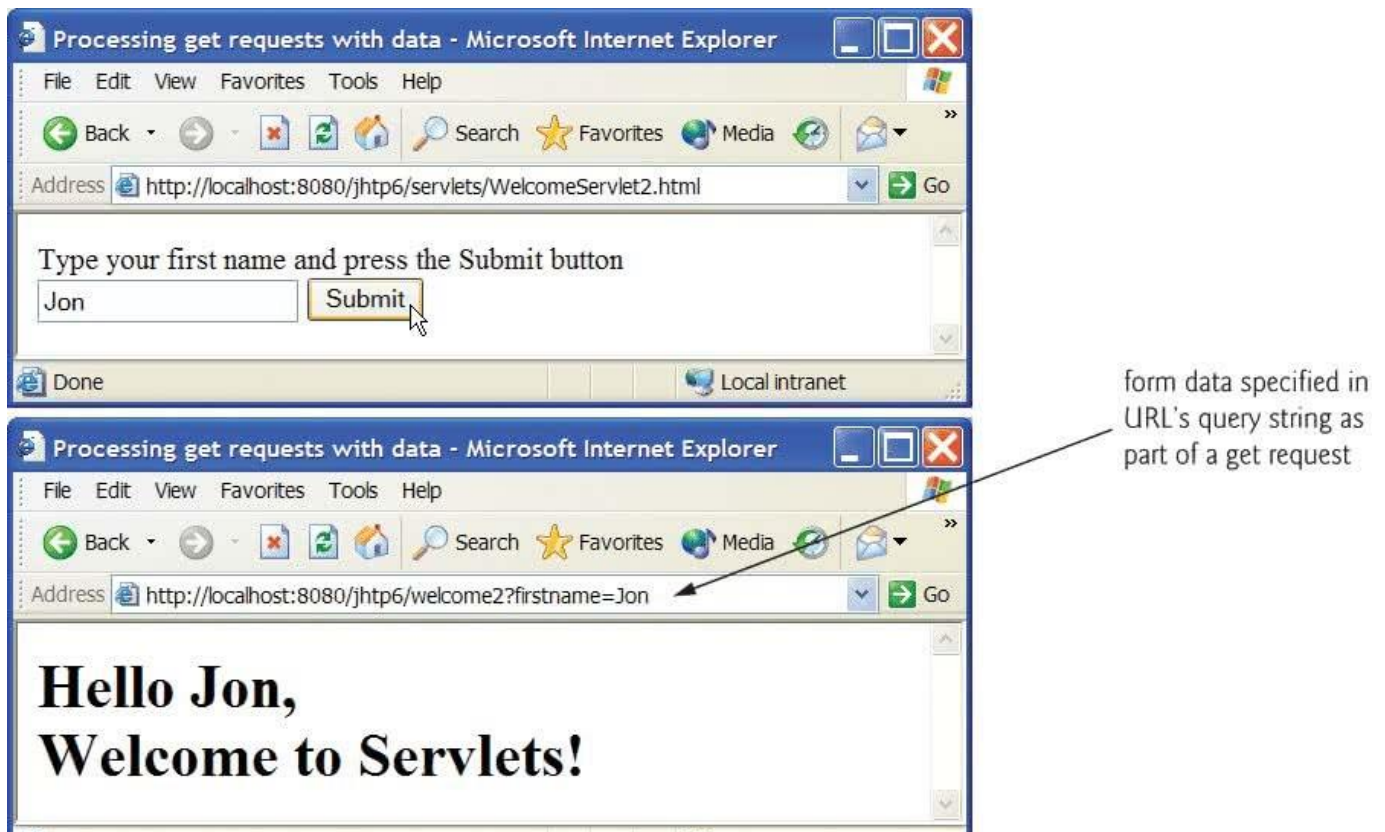
Parameters are passed as name-value pairs in a `get` request. Line 17 demonstrates how to obtain information that was passed to the servlet as part of the client request. The `request` object's `getParameter` method accepts the parameter name as an argument and returns the corresponding `String` value, or `null` if the parameter is not part of the request. Note that parameter names are case sensitive and must match exactly. Line 41 uses the result of line 17 as part of the response to the client.

The `WelcomeServlet2.html` document above Example provides a `form` in which the user can input a name in the text `input` element `firstname` (line 16) and click the Submit button to invoke `WelcomeServlet2`. When the user presses the Submit button, the values of the `input` elements are placed in name-value pairs as part of the request to the server. In the second screen capture in below example, note that the browser appended

to the end of the `action` URL. The `?` separates the query string (i.e., the data passed as part of the `get` request) from the rest of the URL in a `get` request. The name-value pairs are passed with the name and the value separated by `=`. If there is more than one namevalue pair, each pair is separated by `&`.

Example HTML document in which the form's `action` invokes `WelcomeServlet2` through the alias `welcome2` specified in `web.xml`.

```
1 <?xml version = "1.0"?>
2 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
3 "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
4
5 <!-- Example 26.13: WelcomeServlet2.html -->
6
7 <html xmlns = "http://www.w3.org/1999/xhtml">
8 <head>
9 <title>Processing get requests with data </title>
10 </head>
11
12 <body>
13 <form action = "/jhttp6/welcome2" method = "get" >
14 <p><label>
15 Type your first name and press the Submit button
16 <br /><input type = "text" name = "firstname" />
17 <input type = "submit" value = "Submit" />
18 </p></label>
19 </form>
20 </body>
21 </html>
```



Once again, we use our `jhttp6` context root to demonstrate the servlet of above example. Place `WelcomeServlet2.html` in the `servlets` directory created in . Place `WelcomeServlet2.class` in the `classes` subdirectory of `WEB-INF` in the `jhttp6` context root. Remember that classes in a package must be placed in the appropriate package directory structure. Then edit the `web.xml` deployment descriptor in the `WEB-INF` directory to include the information specified in above example This table contains the information for the `servlet` and `servlet-mapping` elements that you will add to the `web.xml` deployment descriptor. You should not type the italic text into the deployment descriptor. Restart Tomcat and type the following URL in your Web browser:

Example Deployment descriptor information for servlet **WelcomeServlet2**.

Descriptor element	Value
<code>servlet element</code>	
<code>servlet-name</code>	<code>welcome2</code>
<code>Description</code>	<code>Handling HTTP get requests with data.</code>
<code>servlet-class</code>	<code>WelcomeServlet2</code>
<code>servlet-mapping element</code>	
<code>servlet-name</code>	<code>welcome2</code>
<code>url-pattern</code>	<code>/welcome2</code>

Type your name in the text field of the Web page, then click Submit to invoke the servlet. Once again, note that the `get` request could have been typed directly into the browser's Address or Location field as follows:

5.6.2 Handling HTTP post request

An HTTP post request is frequently used to send data from an HTML form to a server-side form handler that processes the data. For example, when you respond to a Web-based survey, a post request normally supplies the information you type in the form to the Web server.

Browsers often cache (save on disk) Web pages so they can quickly reload them. If there are no changes between the last version stored in the cache and the current version on the Web, this helps speed up your browsing experience. The browser first asks the server whether the document has changed or expired since the date the file was cached. If not, the browser loads the document from the cache. Thus, the browser minimizes the amount of data that must be downloaded for you to view a Web page. Browsers typically do not cache the server's response to a `post` request, because the next `post` might not return the same result. For example, in a survey, many users could visit the same Web page and respond to a question. The survey results could then be displayed for the user. Each new response changes the overall results of the survey.

When you use a Web-based search engine, the browser normally supplies the information you specify in an HTML form to the search engine with a `get` request. The search engine performs the search, then returns the results to you as a Web page. Such pages are often cached by the browser in case you perform the same search again. As with `post` requests, `get` requests can supply parameters as part of the request to the Web server.

The `WelcomeServlet3` servlet in blow Example is identical to the servlet in blow Example, except that it defines a `doPost` method (lines 1348) rather than a `doGet` method to respond to post requests. The default functionality of `doPost` is to indicate a "Method not supported" error. We override this method to provide custom `post` request processing. Method `doPost` accepts the same two arguments as `doGet`: an object that implements interface `HttpServletRequest` to represent the client's request and an object that implements interface `HttpServletResponse` to represent the servlet's response. As with `doGet`, method `doPost` throws a `ServletException` if it is unable to handle a client's request and throws an `IOException` if a problem occurs during stream processing.

Example **WelcomeServlet3** responds to a **post** request containing data.

```
1 // Example: WelcomeServlet3.java
2 // Processing post requests containing data.
3 import javax.servlet.ServletException;
4 import javax.servlet.http.HttpServlet;
5 import javax.servlet.http.HttpServletRequest;
6 import javax.servlet.http.HttpServletResponse;
7 import java.io.IOException;
8 import java.io.PrintWriter;
9
10 public class WelcomeServlet3 extends HttpServlet
11 {
12     // process "post" request from client
13     protected void doPost( HttpServletRequest request,
14         HttpServletResponse response )
15         throws ServletException, IOException
16     {
17         String firstName = request.getParameter( "firstname" );
18
19         response.setContentType( "text/html" );
20         PrintWriter out = response.getWriter();
21
22         // send XHTML page to client
23
24         // start XHTML document
25         out.println( "<?xml version = \"1.0\"?>" );
26
27         out.printf( "%s%s%s" , "<!DOCTYPE html PUBLIC" ,
28             " \"-//W3C//DTD XHTML 1.0 Strict//EN\"",
29             "\"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd\">\n" );
30
31         out.println( "<html xmlns = \"http://www.w3.org/1999/xhtml\">" );
32
33         // head section of document
34         out.println( "<head>" );
35         out.println(
36             "<title>Processing post requests with data</title>" );
37         out.println( "</head>" );
38
39         // body section of document
40         out.println( "<body>" );
41         out.println( "<h1>Hello " + firstName + ",<br />" );
42         out.println( "Welcome to Servlets!</h1>" );
43         out.println( "</body>" );
44
45         // end XHTML document
46         out.println( "</html>" );
47         out.close(); // close stream to complete the page
48     } // end method doPost
49 } // end class WelcomeServlet3
```

`WelcomeServlet3.html` Example above provides a `form` in which the user can input a name in the text `input` element `firstname`, then click the Submit button to invoke `WelcomeServlet3`. When the user presses the Submit button, the values of the `input` elements are sent to the server as part of the request. However, note that the values are not appended to the request URL. The form's `method` in this example is `post`, but note that a `post` request cannot be typed into the browser's Address or Location field, and users cannot bookmark `post` requests in their browsers.

We use our `jhttp6` context root to demonstrate the servlet in above Example. Place `WelcomeServlet3.html` in the `servlets` directory created in . Place `WelcomeServlet3.class` in the `classes` subdirectory of `WEB-INF` in the `jhttp6` context root. Then, using the information specified in above Example , edit the `web.xml` deployment descriptor in the `WEB-INF` directory. Restart Tomcat and type the following URL in your Web browser:

```
http://localhost:8080/jhttp6/servlets/WelcomeServlet3.html
```

Type your name in the text field of the Web page, then click Submit to invoke the servlet.

Example 26.16. HTML document in which the form's `action` invokes `WelcomeServlet3` through the alias `welcome3` specified in `web.xml`.

```
1 <?xml version = "1.0"?>
2 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
3 "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
4
5 <!-- Example 26.16: WelcomeServlet3.html -->
6
7 <html xmlns = "http://www.w3.org/1999/xhtml">
8 <head>
9 <title>Handling an HTTP Post Request with Data</title>
10 </head>
11
12 <body>
13 <form action = "/jhttp6/welcome3" method = "post" >
14 <p><label>
15 Type your first name and press the Submit button
16 <br /><input type = "text" name = "firstname" />
17 <input type = "submit" value = "Submit" />
18 </label></p>
19 </form>
20 </body>
21 </html>
```




Example 26.17. Deployment descriptor information for servlet **WelcomeServlet3**.

Descriptor element	Value
<code>servlet element</code>	
<code>servlet-name</code>	<code>welcome3</code>
<code>description</code>	Handling HTTP post requests with data.
<code>servlet-class</code>	<code>WelcomeServlet3</code>
<code>servlet-mapping element</code>	
<code>servlet-name</code>	<code>welcome3</code>
<code>url-pattern</code>	<code>/welcome3</code>

5.7 Redirecting Requests to Other Resources

5.7.1 REDIRECTING REQUESTS TO OTHER RESOURCES

Sometimes it is useful to redirect a request to a different resource. For example, a servlet could determine the type of the client browser and redirect the request to a Web page that was designed specifically for that browser. This technique is also used to redirect browsers to an error page when handling a request fails. The `RedirectServlet` in [Example 26.18](#) receives a `page` parameter as part of a `get` request, then uses that parameter to redirect the request to a different resource.

Example 26.18. Redirecting requests to other resources.

```
1 // Example 26.18: RedirectServlet.java
2 // Redirecting a user to a different Web page.
3 import javax.servlet.ServletException;
4 import javax.servlet.http.HttpServlet;
5 import javax.servlet.http.HttpServletRequest;
6 import javax.servlet.http.HttpServletResponse;
7 import java.io.IOException;
8 import java.io.PrintWriter;
9
10 public class RedirectServlet extends HttpServlet
11 {
12     // process "get" request from client
13     protected void doGet( HttpServletRequest request,
14         HttpServletResponse response )
15         throws ServletException, IOException
16     {
17         String location = request.getParameter( "page" );
18
19         if ( location != null )
20         {
21             if ( location.equals( "deitel" ) )
22                 response.sendRedirect( "http://www.deitel.com" );
23             else if ( location.equals( "welcome1" ) )
24                 response.sendRedirect( "welcome1" );
25         } // end if
26
27         // code that executes only if this servlet
28         // does not redirect the user to another page
29         response.setContentType( "text/html" );
30         PrintWriter out = response.getWriter();
31
32         // start XHTML document
33         out.println( "<?xml version = \"1.0\"?>" );
34
35         out.printf( "%s%s%s" , "<!DOCTYPE html PUBLIC" ,
36             " \"-//W3C//DTD XHTML 1.0 Strict//EN\"",
37             " \"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd\">\n" );
38
39         out.println(
```

```

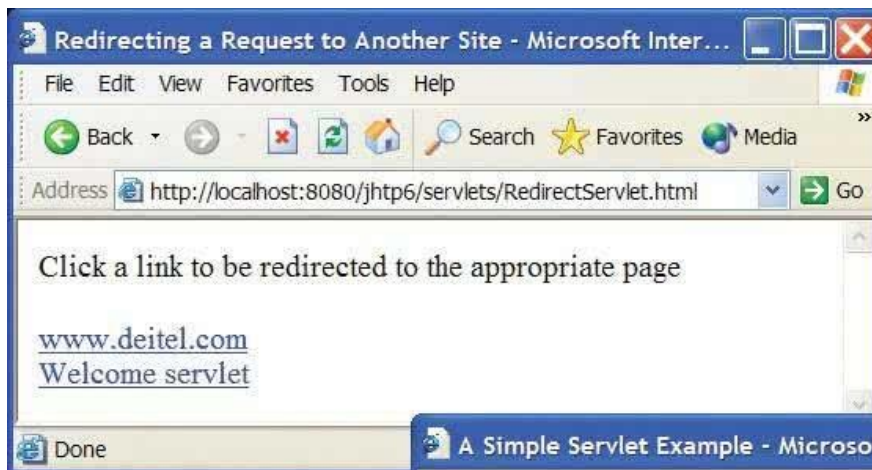
40 "<html xmlns = \"http://www.w3.org/1999/xhtml\">" );
41
42 // head section of document
43 out.println( "<head>" );
44 out.println( "<title>Invalid page</title>" );
45 out.println( "</head>" );
46
47 // body section of document
48 out.println( "<body>" );
49 out.println( "<h1>Invalid page requested</h1>" );
50 out.println( "<p><a href = " +
51 "\"servlets/RedirectServlet.html\">" );
52 out.println( "Click here to choose again</a></p>" );
53 out.println( "</body>" );
54
55 // end XHTML document
56 out.println( "</html>" );
57 out.close(); // close stream to complete the page
58 } // end method doGet
59 } // end class RedirectServlet

```

Line 17 obtains the `page` parameter from the request. If the value returned is not `null`, the nested `if... else if` statement at lines 21-24 determines whether the value is either `"deitel"` or `"welcome1"`. If the value is `"deitel"`, the `response` object's `sendRedirect` method (line 22) redirects the request to www.deitel.com. If the value is `"welcome1"`, line 24 redirects the request to the servlet of Example blow. Note that line 24 does not explicitly specify the `jhttp6` context root for our Web application. When a servlet uses a relative path to reference another static or dynamic resource, the servlet assumes the same base URL and context root as the one that invoked the servlet unless a complete URL is specified for the resource. Once method `sendRedirect` executes, processing of the original request by the `RedirectServlet` terminates. If method `sendRedirect` is not called, the remainder of method `doGet` outputs a Web page indicating that an invalid request was made. The page allows the user to try again by returning to the XHTML document of Example blow. Note that one of the redirects is sent to a static XHTML Web page and the other is sent to a servlet.

Example **RedirectServlet.html** document to demonstrate redirecting requests to other resources.

```
1 <?xml version = "1.0" ?>
2 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
3 "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
4
5 <!-- Example 26.19: RedirectServlet.html -->
6
7 <html xmlns = "http://www.w3.org/1999/xhtml">
8 <head>
9 <title>Redirecting a Request to Another Site</title>
10 </head>
11
12 <body>
13 <p>Click a link to be redirected to the appropriate page</p>
14 <p>
15 <a href = "/jhttp6/redirect?page=deitel">
16 www.deitel.com</a><br />
17 <a href = "/jhttp6/redirect?page=welcome1">
18 Welcome servlet</a>
19 </p>
20 </body>
21 </html>
```



Example Deployment descriptor information for servlet **RedirectServlet**.

Descriptor element	Value
servlet element	
servlet-name	Redirect
Description	Redirecting to static Web pages and other servlets
servlet-class	RedirectServlet
servlet-mapping element	
servlet-name	Redirect
url-pattern	/redirect

When redirecting requests, the request parameters from the original request are passed as parameters to the new request. Additional request parameters can also be passed. For example, the URL passed to `sendRedirect` could contain name-value pairs. New parameters are added to the existing parameters. A new parameter with the same name as an existing parameter takes precedence over the original value. However, all the values are still passed. In this case, the complete set of values for a given parameter name can be obtained by calling method `getParameterValues` from interface `HttpServletRequest`. This method accepts the parameter name as an argument and returns an array of strings containing the parameter values in most-recent-to-least-recent-order.

5.8 Multi-Tier Applications: Using JDBC from a Servlet

Servlets can communicate with databases via JDBC. Many of today's applications are **three-tier distributed applications**, consisting of a user interface, **business logic** and a database. The user interface in such an application is often created using HTML or XHTML (as shown in this chapter). HTML and XHTML are preferred in systems where portability is a concern. Using the networking provided by the browser, the user interface can communicate with the middle-tier business logic. The middle tier can then access the database to manipulate the data. The three tiers can reside on separate computers that are connected to a network.

In multitier architectures, Web servers often are used in the middle tier. Server-side components, such as servlets, execute in an application server alongside the Web server. These components provide the business logic that manipulates data from databases and communicates with client Web browsers. Servlets, through JDBC, can interact with popular database systems. Developers use SQL for queries, and JDBC drivers handle the specifics of interacting with each database system.

Example Multitier Web-based survey using XHTML, servlets and JDBC.

```
1 // Example 26.21: SurveyServlet.java
2 // A Web-based survey that uses JDBC from a servlet.
3 package com.deitel.jhttp6.servlets;
4
5 import java.io.PrintWriter;
6 import java.io.IOException;
7 import java.sql.Connection;
8 import java.sql.DriverManager;
9 import java.sql.Statement;
10 import java.sql.ResultSet;
11 import java.sql.SQLException;
12 import javax.servlet.ServletConfig;
13 import javax.servlet.ServletException;
14 import javax.servlet.UnavailableException;
15 import javax.servlet.http.HttpServlet;
16 import javax.servlet.http.HttpServletRequest;
17 import javax.servlet.http.HttpServletResponse;
18
19 public class SurveyServlet extends HttpServlet
20 {
21     private Connection connection;
22     private Statement statement;
23
24     // set up database connection and create SQL statement
25     public void init( ServletConfig config ) throws ServletException
26     {
27         // attempt database connection and create Statements
28         try
29         {
30             Class.forName( config.getInitParameter( "databaseDriver" ) );
31             connection = DriverManager.getConnection(
32                 config.getInitParameter( "databaseName" ) );
33             config.getInitParameter( "username" ),
34             config.getInitParameter( "password" ) );
35
36             // create Statement to query database
37             statement = connection.createStatement();
38         } // end try
39         // for any exception throw an UnavailableException to
40         // indicate that the servlet is not currently available
41         catch ( Exception exception )
42         {
43             exception.printStackTrace();
44             throw new UnavailableException(exception.getMessage());
45         } // end catch
46     } // end method init
47
48     // process survey response
49     protected void doPost( HttpServletRequest request,
50         HttpServletResponse response )
51     throws ServletException, IOException
```

```

52 {
53 // set up response to client
54 response.setContentType( "text/html" );
55 PrintWriter out = response.getWriter();
56
57 // start XHTML document
58 out.println( "<?xml version = \"1.0\"?>" );
59
60 out.printf( "%s%s%s" , "<!DOCTYPE html PUBLIC" ,
61 " \"-//W3C//DTD XHTML 1.0 Strict//EN\"",
62 " \"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd\">\n" );
63
64 out.println(
65 "<html xmlns = \"http://www.w3.org/1999/xhtml\">" );
66
67 // head section of document
68 out.println( "<head>" );
69
70 // read current survey response
71 int value =
72 Integer.parseInt( request.getParameter( "animal" ) );
73 String sql;
74
75 // attempt to process a vote and display current results
76 try
77 {
78 // update total for current survey response
79 sql = "UPDATE surveyresults SET votes = votes + 1 " +
80 "WHERE id = " + value;
81 statement.executeUpdate( sql );
82
83 // get total of all survey responses
84 sql = "SELECT sum( votes ) FROM surveyresults" ;
85 ResultSet totalRS = statement.executeQuery( sql );
86 totalRS.next(); // position to first record
87 int total = totalRS.getInt( 1 );
88
89 // get results
90 sql = "SELECT surveyoption, votes, id FROM surveyresults " +
91 "ORDER BY id";
92 ResultSet resultsRS = statement.executeQuery( sql );
93 out.println( "<title>Thank you!</title>" );
94 out.println( "</head>" );
95
96 out.println( "<body>" );
97 out.println( "<p>Thank you for participating." );
98 out.println( "<br />Results:</p><pre>" );
99
100 // process results
101 int votes;
102
103 while ( resultsRS.next() )
104 {
105 out.print( resultsRS.getString( 1 ) );
106 out.print( ": " );

```

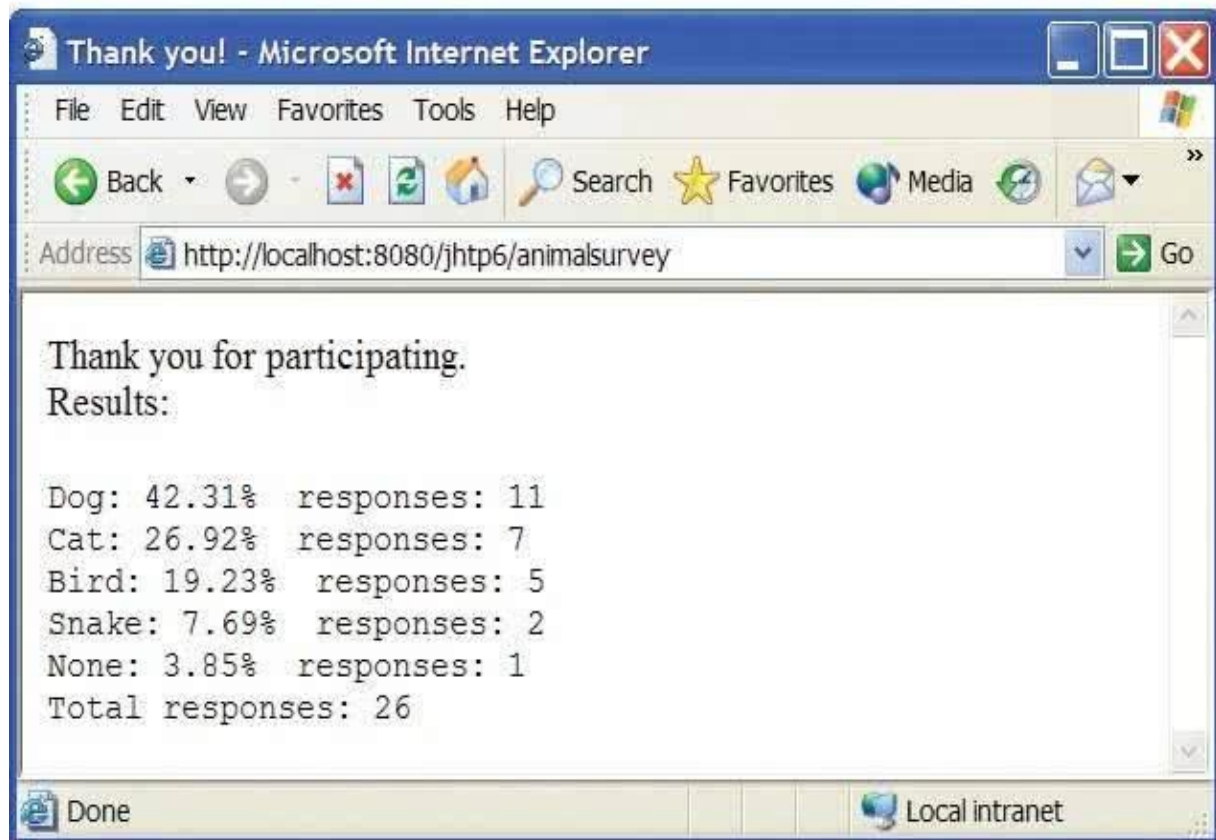
```

107 votes = resultsRS.getInt( 2 );
108 out.printf( "%.2f", ( double ) votes / total * 100 );
109 out.print( "% responses: " );
110 out.println( votes );
111 } // end while
112
113 resultsRS.close();
114
115 out.print( "Total responses: " );
116 out.print( total );
117
118 // end XHTML document
119 out.println( "</pre></body></html>" );
120 out.close();
121 } // end try
122 // if database exception occurs, return error page
123 catch ( SQLException sqlException )
124 {
125 sqlException.printStackTrace();
126 out.println( "<title>Error</title>" );
127 out.println( "</head>" );
128 out.println( "<body><p>Database error occurred. " );
129 out.println( "Try again later.</p></body></html>" );
130 out.close();
131 } // end catch
132 } // end method doPost
133
134 // close SQL statements and database when servlet terminates
135 public void destroy()
136 {
137 // attempt to close statements and database connection
138 try
139 {
140 statement.close();
141 connection.close();
142 } // end try
143 // handle database exceptions by returning error to client
144 catch ( SQLException sqlException )
145 {
146 sqlException.printStackTrace();
147 } // end catch
148 } // end method destroy
149 } // end class SurveyServlet

```

Example **Survey.html** document that allows users to submit survey responses to **SurveyServlet**.

```
1 <?xml version = "1.0" ?>
2 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
3 "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
4
5 <!-- Example 26.22: Survey.html -->
6
7 <html xmlns = "http://www.w3.org/1999/xhtml">
8 <head>
9 <title>Survey</title>
10 </head>
11
12 <body>
13 <form method = "post" action = "/jhttp6/animalsurvey">
14 <p>What is your favorite pet?</p>
15 <p>
16 <input type = "radio" name = "animal"
17 value = "1" />Dog<br />
18 <input type = "radio" name = "animal"
19 value = "2" />Cat<br />
20 <input type = "radio" name = "animal"
21 value = "3" />Bird<br />
22 <input type = "radio" name = "animal"
23 value = "4" />Snake<br />
24 <input type = "radio" name = "animal"
25 value = "5" checked = "checked" /> None
26 </p>
27 <p><input type = "submit" value = "Submit" /></p>
28 </form>
29 </body>
30 </html>
```

Example Deployment descriptor information for servlet **SurveyServlet**.

Descriptor element	Value
servlet element	
servlet-name	animalsurvey
Description	Connecting to a database from a servlet.
servlet-class	com.deitel.jhttp6.servlets.SurveyServlet
init-param	
param-name	databaseDriver
param-value	com.mysql.jdbc.Driver
init-param	
param-name	databaseName
param-value	jdbc:mysql://localhost/animalsurvey
init-param	
param-name	username
param-value	jhttp6
init-param	
param-name	password
param-value	jhttp6
servlet-mapping element	
servlet-name	animalsurvey
url-pattern	/animalsurvey

5.9 Chapter Review Questions

1. What is a Servlet?
2. Explain the architecture of a Servlet.
3. What is the difference between an Applet and a Servlet?
4. What is the difference between GenericServlet and HttpServlet?
5. Explain the life cycle of a Servlet.
6. What is the difference between doGet() and doPost() ?
7. What is meant by a Web Application?
8. What is a Server Side Include (SSI)?
9. What is Servlet Chaining?
10. How do you find out what client machine is making a request to your servlet?
11. What is the structure of the HTTP response?
12. What is a cookie? What is the difference between session and cookie?
13. Which protocol will be used by the browser and servlet to communicate?
14. What is HTTP Tunneling?
15. What's the difference between sendRedirect and forward methods?
16. What is URL Encoding and URL Decoding?
17. Write a Servlet program that creates a form and inserts the values of the form into the database.
18. Write a Servlet program that Uploads file into a database.

CHAPTER FIVE

6 Java Server Pages

6.1 Introduction

Java Server Pages (JSP) is a programming tool on the application server side that supports platform-independent and dynamic methods to construct Web-based applications. Much as Servlet technology does, the JSP method provides a web application. It can be considered an expansion of Servlet because it offers more features than servlet. Since we can differentiate design and development, the JSP pages are simpler to manage than Servlet.

HTML tags and JSP tags are present in Java Server Pages. To access enterprise servers, Java Server Pages has an approach to the entire community of Java APIs, including the JDBC API. This tutorial will walk you to the path of building your own web application in convenient and simple steps using Java Server Pages.

6.2 Java Server Pages Overview

6.2.1 Java Server Pages

JavaServer Pages (JSP) is a Web page development technology that supports dynamic content. This allows programmers to use specific JSP tags to insert Java code into HTML pages. A part of JavaServer Pages is a type of Java servlet designed to perform the function of a Java web application user interface.

JSPs are written as text files by Web developers that incorporate Html or XHTML script, XML components, and embedded JSP actions and commands. For many purposes, JSP tags may be used, such as downloading data from the database or recording user interests, accessing modules of JavaBeans, transferring power between sites, exchanging information among queries, etc.

6.2.2 Why Use JSP?

JavaServer Pages often serve the same purpose as programs implemented using the Common Gateway Interface (CGI). But JSP offers several advantages in comparison with the CGI.

- Performance is significantly better because JSP allows embedding Dynamic Elements in HTML Pages itself instead of having separate CGI files.
- JSP are always compiled before they are processed by the server unlike CGI/Perl which requires the server to load an interpreter and the target script each time the page is requested.
- JavaServer Pages are built on top of the Java Servlets API, so like Servlets, JSP also has access to all the powerful Enterprise Java APIs, including JDBC, JNDI, EJB, JAXP, etc.
- JSP pages can be used in combination with servlets that handle the business logic, the model supported by Java servlet template engines. Finally, JSP is an integral part of Java EE, a complete platform for enterprise class applications. This means that JSP can play a part in the simplest applications to the most complex and demanding.

Advantages of JSP

Following is the list of other advantages of using JSP over other technologies:

vs. Active Server Pages (ASP) The advantages of JSP are twofold. First, the dynamic part is written in Java, not Visual Basic or other MS specific language, so it is more powerful and easier to use. Second, it is portable to other operating systems and non-Microsoft Web servers.

vs. Pure Servlets It is more convenient to write (and to modify!) regular HTML than to have plenty of `println` statements that generate the HTML.

vs. Server-Side Includes (SSI) SSI is really only intended for simple inclusions, not for "real" programs that use form data, make database connections, and the like.

vs. JavaScript **JavaScript** can generate HTML dynamically on the client but can hardly interact with the web server to perform complex tasks like database access and image processing etc.

vs. Static HTML Regular HTML, of course, cannot contain dynamic information.

6.3 JSP Architecture

The web server needs a JSP engine(container) to process JSP pages. The JSP container is liable for intercepting requests for JSP pages. Here we have used Apache which has a built-in JSP container to support JSP page development. A JSP container works with the online server to supply the services a JSP needs and the runtime environment also. It knows the way to understand the special elements that are a part of JSPs.

There are two basic ways of using JSP technology. They are the client/server (page-centric) 2-tier approach and therefore the N-tier approach (dispatcher).

6.3.1 The Page-Centric Approach

Applications built employing a client-server (2-tier) approach contains one or more application programs running on client machines and connecting to a server-based application to figure. With the arrival of Servlets technology, 2-tier applications could even be developed using Java programming language. This model allows JSPs or Servlets direct access to some resources such as databases or legacy applications to service a client's request. The JSP page is where the incoming request is intercepted, processed and therefore the response sent back to the client. JSPs differ from Servlets during this scenario by providing clean code, separating code from the content by placing data access in EJBs. Even though this model makes application development easier, it doesn't proportion well for an outsized number of simultaneous clients because it entails a big amount of request processing to be performed and every request must establish or share a potentially scarce/expensive connection to the resource in question.

Page – View:

This basic architecture involves direct request invocations to a server page with embedded Java code and markup tags which dynamically generate output for substitution within the HTML. This approach has been blessed variety of advantages. It is very straight forward and is a low-overhead approach from a development perspective. All the Java code could also be embedded within the HTML, so changes are confined to a really limited area, reducing complexity drastically.

The big trade-off here is within the level of sophistication. As the scale of the system grows, some limitations begin to surface, such as bloating of business logic code in the page instead of factoring forward to a mediating Servlet or Factoring back to a worker bean. It is an incontrovertible fact that utilizing Servlet and helper beans helps to separate developer roles more cleanly and improves the potential for code reuse.

Page-view with bean:

This pattern is employed when the above architecture becomes too cluttered with business-related code and data access code. The Java code representing the business logic and straightforward data storage implementation within the previous model moves from the JSP to the JavaBean worker. This refactoring leaves away cleaner JSP with limited Java code, which may be comfortably owned by a private during a web-production role since it encapsulates mostly markup tags.

The Dispatcher Approach

In this approach, a Servlet or JSP acts as a mediator or controller, delegating requests to JSP pages and JavaBeans. There are three different architectures. They are mediator-view, mediator-composite view, and repair to workers.

In an N-tier application, the server-side of the architecture is choppy into multiple tiers. In this case, the appliance consists of multiple tiers, where the center tier, the JSP, interacts with the rear end resources via another object or EJBs component. The Enterprise JavaBeans server and the EJB provide managed access to resource sharing and performance issues of the 2-tier approach.

The first step is N-tiered application design should be identifying the correct objects and their interaction and the second step is identifying the JSPs or Servlets. These are divided into two categories:

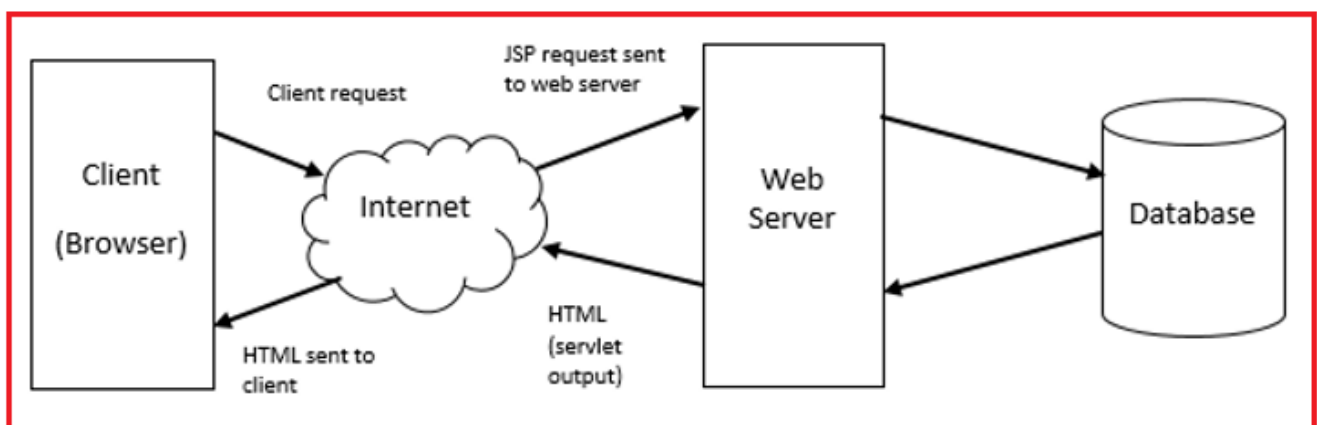
1. Front end JSPs or Servlets manage application flow and business logic evaluation. They act to some extent to intercept the HTTP requests coming from the users. They provide one entry point to an application, simplifying security management and making the application state easier to take care of.
2. Presentation JSPs or Servlets generate HTML or XML with their main purpose in life being the presentation of dynamic content. They contain the only presentation and rendering logic.

These categories resemble the Modal-View design pattern, where the front-end components are that the model and therefore the presentation component the view. In this approach, JSPs are wont to generate the presentation layer and either JSPs or Servlets to perform process-intensive tasks. The front-end component acts because the controller and is responsible for the request processing and therefore the creation of any beans or objects employed by the presentation JSP, as well as for deciding, counting on the user's actions, which JSP to forward this request to. There is no processing logic within the presentation JSP itself and it simply liable for retrieving any objects or beans which will are previously created by the Servlet and extracting the dynamic content for insertion within static templates.

6.4 JSP Processing and its flow

JSP stands for Java Server Pages. We can develop a web application with dynamic Input Screens and dynamic output screens by using JSP. The current version of JSP is JSP2.2. JSP pages are automatically compiled by the Server, such as Tomcat and Weblogic. Web pages created by using JSP are portable and can be used easily across multiple platforms and WebServers without making any changes. One of the most important benefits of JSP is the separation of business logic from the presentation logic. JSP pages are often accessed directly as an easy HTML page.

1. JSP is actually a powerful scripting language (interpreted language) executed on the server-side (like CGI, PHP, ASP.) and not on the client-side (unlike scripts written in JavaScript or Java applets which run in the browser of the user connected to a site).
2. JSPs are integrated into a web page in HTML using special tags that will notify the web server that the code included within these tags are to be interpreted. The result (HTML codes) is going to be returned to the client browser.
3. Java Server Pages are part of a 3-tier architecture: where a server supporting the Java, server Pages (generally referred to as application server) will act as a mediator between the client browser and a database (generally mentioned as a data server). JSP provides the necessary elements for the connection to the database management system and allows the manipulation of data through SQL.



A page using Java Server Pages is executed during the query, by a JSP engine (generally running with an internet Server or an application server). The JSP model springs from the one used for Java servlets (JSP is indeed how to write down servlets). It is a Java class derived from Servlet class, making use of using `doXXX()` to return an HTTP response. When a user calls a JSP page, the server calls the JSP engine which creates a Java source code from the JSP script and compiles the class to provide a compiled file (with the .class extension). Note that the JSP engine checks if the date of the .jsp file corresponds to the .class file. The JSP engine will convert and compile the category, as long as the JSP script has been updated. Thus, the very fact that the compilation only takes place when the JSP script is updated, makes JSP, one among the fastest technologies to make dynamic pages. JSPs are used to create servlets, by including specific tags in the JSP code. In this way, they supply a quick technology to make dynamic pages.

6.5 Applications and Advantages of JSP

Standard HTML does not contain dynamic data, whereas JSP does. JSP's benefits are fairly straightforward. First of all, the dynamic component is implemented in Java, not Visual Basic or any platform-specific language, so it is smoother and simpler to use. Lastly, it is platform-independent. If we compare JSP and SSI, Server Side Includes is intended only for basic inclusions, not for actual systems that use form data, build links to databases, and etc.

Writing and modifying standard HTML is more practical than creating lots of HTML-generating `println` statements to generate HTML. JavaScript can generate HTML dynamically on the client but can hardly interact with the webserver to perform complex tasks like database access and image processing etc.

6.6 Setting up the environment for JSP

Taking you to step 1 of the ladder of setting up the environment for JSP.

1. Set up the Java Development Kit

This move includes installing a Java Software Development Kit (SDK) implementation and properly setting up the PATH environment variable. You may access SDK from the Web site of Oracle (Java SE Downloads).

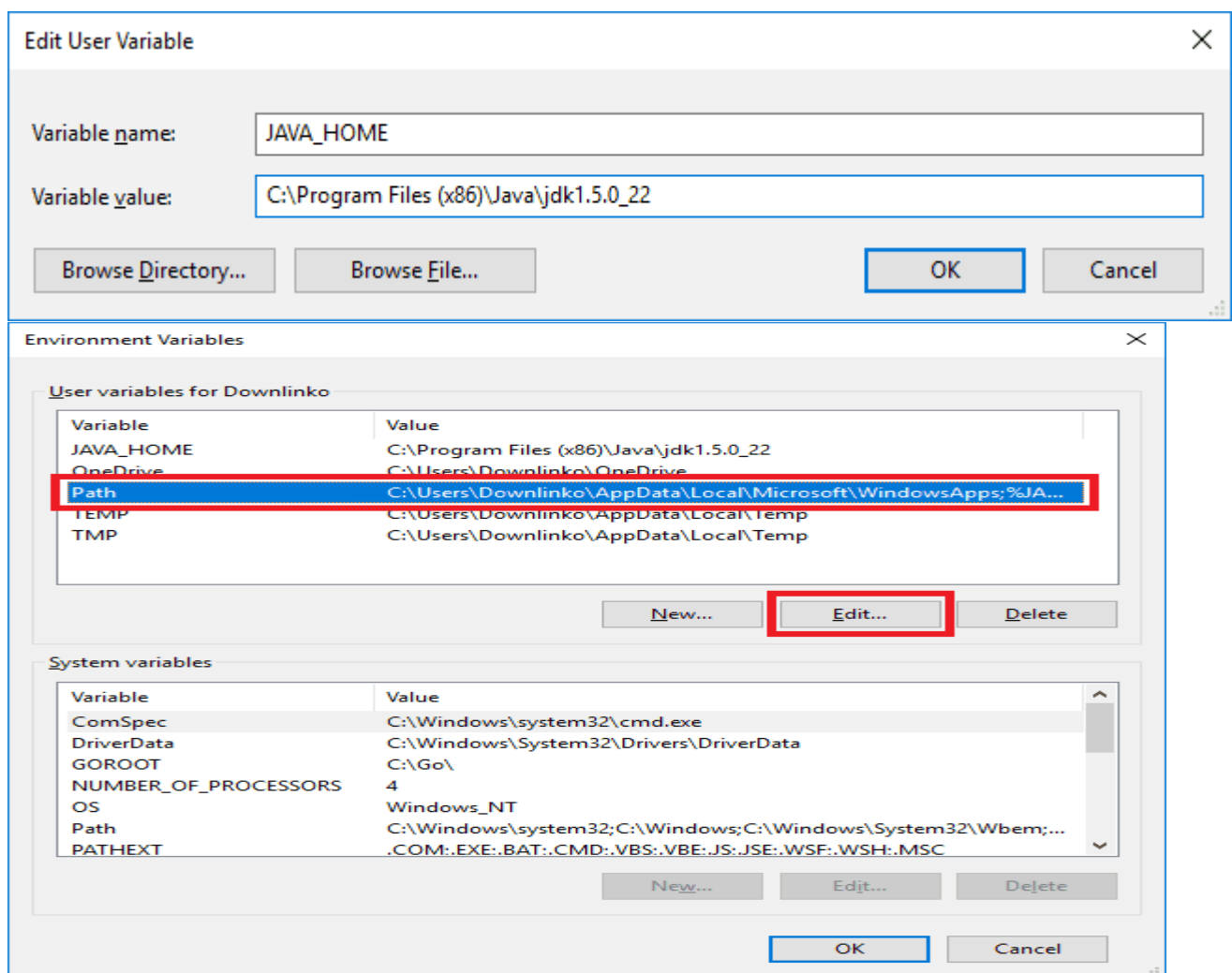
Follow the instructions provided for installing and configuring the setup after downloading your Java. Finally, set the `Path` and `JAVA_HOME` environment to refer to the directory with `java` and `javac`; usually, `java` installs `dir/bin` and `java` install `dir`.

Either you need to apply the following line to your C:\autoexec.bat file if you are running Windows and downloading the SDK in **C:\jdk1.5.0_22**:

```
set PATH = C:\jdk1.5.0_22\bin; %PATH%
```

```
set JAVA_HOME = C:\jdk1.5.0_22
```

Or you could also right-click on **My Computer Properties Advanced Environment Variables**. You'll then change the input PATH and press the OK button.



On Unix, if the SDK is installed in **/usr/local/jdk1.5.0_22** and you use the C shell, you will put the following into your **.cshrc** file: `setenv PATH /usr/local/jdk1.5.0_22/bin:$PATH` `setenv JAVA_HOME /usr/local/jdk1.5.0_22`

Scroll to know the second step.

2. Configuring a Web Server- Tomcat

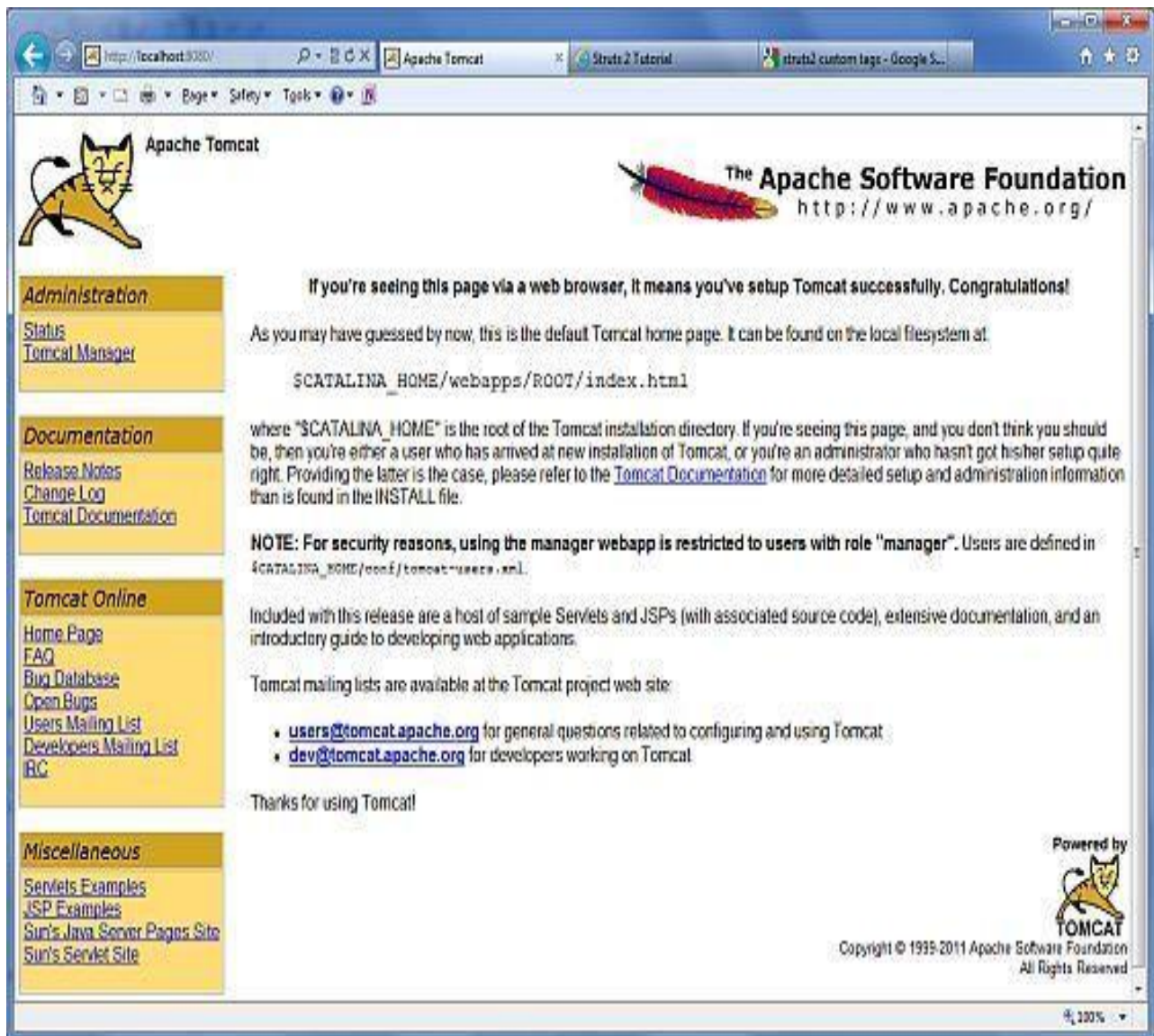
A variety of Web servers facilitating the creation of JavaServer Pages and Servlets are available on the market. Tomcat is among the servers which are free to download. Apache Tomcat is a free software JavaServer Pages and Servlet technology deployment software that can act as a standalone server for JSP and Servlets testing and can be combined with the Apache Web Server. Here are the measures on your computer to set up Tomcat:

- Download the fresh Tomcat update from <https://tomcat.apache.org/>.
- Unpack the binary distribution into a decent setting until you have downloaded the installation.

You can launch Tomcat by running the following commands on a Windows machine:

The default web applications that are included with Tomcat will be accessible after a promising startup by visiting <http://localhost:8080/>.

On successful installation, you'll see this page:



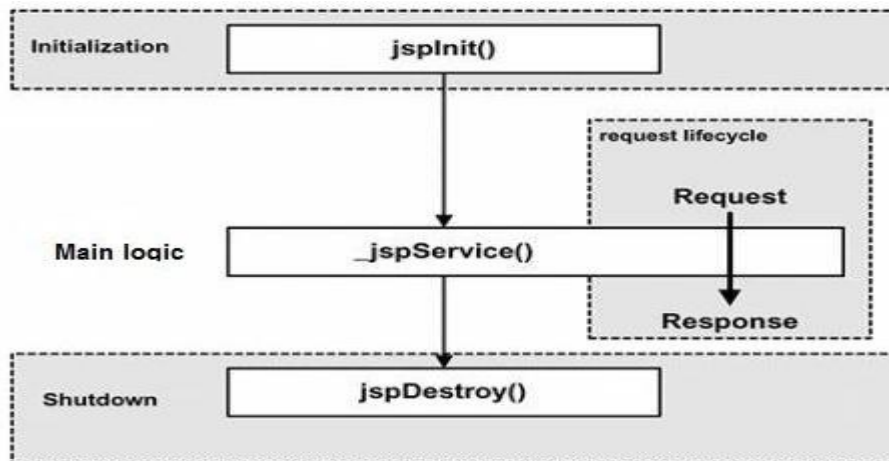
6.7 LIFE CYCLE

A JSP life cycle can be defined as the entire process from its creation till the destruction which is similar to a servlet life cycle with an additional step which is required to compile a JSP into servlet.

The following are the paths followed by a JSP

- Compilation
- Initialization
- Execution

- Cleanup



JSP Compilation:

When a browser asks for a JSP, the JSP engine first checks to see whether it needs to compile the page. If the page has never been compiled, or if the JSP has been modified since it was last compiled, the JSP engine compiles the page.

The compilation process involves three steps:

- Parsing the JSP.
- Turning the JSP into a servlet.
- Compiling the servlet.

JSP Initialization

When a container loads a JSP it invokes the `jspInit()` method before servicing any requests. If you need to perform JSP-specific initialization, override the `jspInit()` method:

```
public void jspInit(){
    // Initialization code...
}
```

Typically initialization is performed only once and as with the servlet `init` method, you generally initialize database connections, open files, and create lookup tables in the `jspInit` method.

JSP Execution

This phase of the JSP life cycle represents all interactions with requests until the JSP is destroyed. Whenever a browser requests a JSP and the page has been loaded and initialized, the JSP engine invokes the `_jspService()` method in the JSP.

The `jspService()` method takes an **HttpServletRequest** and an **HttpServletResponse** as its parameters as follows:

```
void _jspService(HttpServletRequest request,
    HttpServletResponse response)
{
    // Service handling code...
}
```

The `_jspService()` method of a JSP is invoked once per a request and is responsible for generating the response for that request and this method is also responsible for generating responses to all seven of the HTTP methods ie. GET, POST, DELETE etc

JSP Cleanup

The destruction phase of the JSP life cycle represents when a JSP is being removed from use by a container. The **`jspDestroy()`** method is the JSP equivalent of the destroy method for servlets. Override `jspDestroy` when you need to perform any cleanup, such as releasing database connections or closing open files.

The `jspDestroy()` method has the following form:

```
public void jspDestroy()
{
    // Your cleanup code goes here.
}
```

JSP Elements:

- JSP Comments
- JSP Scriptlets
- JSP Expression
- JSP Directives
- JSP Declaration

JSP Comments

Since JSP is built on top of HTML, we can write comments in JSP file like html comments as

`<-- This is HTML Comment -->` These comments are sent to the client and we can look it with view source option of browsers.

We can put comments in JSP files as: `<%-- This is JSP Comment--%>` This comment is suitable for developers to provide code level comments because these are not sent in the client response.

JSP Scriptlets

Scriptlet tags are the easiest way to put java code in a JSP page. A scriptlet tag starts with `<%` and ends with `%>`. Any code written inside the scriptlet tags goes into the `_jspService()` method.

For example

```
<%  
  
Date d = new Date(); System.out.println("Current  
Date="+d);  
  
%>
```

JSP Expression

Since most of the times we print dynamic data in JSP page using `out.print()` method, there is a shortcut to do this through JSP Expressions. JSP Expression starts with `<%=` and ends with `%>`. `<% out.print("Pankaj"); %>` can be written using JSP Expression as `<%= "Pankaj" %>`.

Notice that anything between `<%= %>` is sent as parameter to `out.print()` method. Also notice that scriptlets can contain multiple java statements and always ends with semicolon (;) but expression doesn't end with semicolon.

JSP Directives

JSP Directives are used to give special instructions to the container while JSP page is getting translated to servlet source code. JSP directives start

with `<%@` and ends with `%>`

For example, in above JSP Example, I am using *page* directive to instruct container JSP translator to import the Date class.

JSP Declaration

JSP Declarations are used to declare member methods and variables of servlet class.

JSP Declarations start with `<%!` and ends with `%>`.

For example we can create an int variable in JSP at class level as `<%!public static int count=0; %>`

JSP Directives

A JSP directive affects the overall structure of the servlet class. It usually has the following form:

```
<%@ directive attribute="value" %>
```

There are three types of directive tag:

Directive	Description
<code><%@ page ... %></code>	Defines page-dependent attributes, such as scripting language, error page, and buffering requirements.
<code><%@ include ... %></code>	Includes a file during the translation phase.
<code><%@ taglib ... %></code>	Declares a tag library, containing custom actions, used in the page

JSP Implicit Objects

JSP supports nine automatically defined variables, which are also called implicit objects. These variables are:

Objects	Description
request	This is the HttpServletRequest object associated with therequest.
response	This is the HttpServletResponse object associated with theresponse to the client.
out	This is the PrintWriter object used to send output to the client.
session	This is the HttpSession object associated with the request.
application	This is the ServletContext object associated with applicationcontext.
config	This is the ServletConfig object associated with the page.
pageContext	This encapsulates use of server-specific features like higher performance JspWriters .
page	This is simply a synonym for this , and is used to call the methods defined by the translated servlet class.
Exception	The Exception object allows the exception data to be accessed by designated JSP.

JSP Action Tags:

JSP actions use constructs in XML syntax to control the behavior of the servlet engine. You can dynamically insert a file, reuse JavaBeans components, forward the user to another page, or generate HTML for the Java plugin.

There is only one syntax for the Action element, as it conforms to the XML standard:

```
<jsp:action_name attribute="value" />
```

Action elements are basically predefined functions and there are following JSP actions available:

Syntax	Purpose
jsp:include	Includes a file at the time the page is requested
jsp:useBean	Finds or instantiates a JavaBean
jsp:setProperty	Sets the property of a JavaBean
jsp:getProperty	Inserts the property of a JavaBean into the output
jsp:forward	Forwards the requester to a new page
jsp:plugin	Generates browser-specific code that makes an OBJECT or EMBED tag for the Java plugin
jsp:element	Defines XML elements dynamically.
jsp:attribute	Defines dynamically defined XML element's attribute.
jsp:body	Defines dynamically defined XML element's body.
jsp:text	Use to write template text in JSP pages and documents.

To give an example for a JSP code, first we are going to print the text "Hello Hiox". Try the the following syntax code.

```
<html>
<body>
<!-- This is the JSP file-->
<%
out.println ("Hello HIOX");
%>
```

Example :

Result :

Hello HIOX

JSTL

JSTL (JSP Standard Tag Library)

The JSP Standard Tag Library (JSTL) represents a set of tags to simplify the JSP development.

Advantage of JSTL

1. **Fast Development** JSTL provides many tags that simplifies the JSP.
 2. **Code Reusability** We can use the JSTL tags in various pages.
- No need to use scriptlet tag** It avoids the use of scriptlet tag.

JSTL Tags

There JSTL mainly provides 5 types of tags:

Tag Name	Description
Core tags	The JSTL core tag provide variable support, URL management, flow control etc. The url for the core tag is http://java.sun.com/jsp/jstl/core . The prefix of core tag is c .
Function tags	The functions tags provide support for string manipulation and string length. The url for the functions tags is http://java.sun.com/jsp/jstl/functions and prefix is fn .
Formatting tags	The Formatting tags provide support for message formatting, number and date formatting etc. The url for the Formatting tags is http://java.sun.com/jsp/jstl/fmt and prefix is fmt .
XML tags	The xml sql tags provide flow control, transformation etc. The url forthe xml tags is http://java.sun.com/jsp/jstl/xml and prefix is x .
SQL tags	The JSTL sql tags provide SQL support. The url for the sql tagsis http://java.sun.com/jsp/jstl/sql and prefix is sql .

AJAX

AJAX stands for **A**synchronous **J**avaScript and **X**ML. AJAX is a new technique for creating better, faster, and more interactive web applications with the help of XML, HTML, CSS, and Java Script.

- Ajax uses XHTML for content, CSS for presentation, along with Document Object Model and JavaScript for dynamic content display.
- Conventional web applications transmit information to and from the sever using synchronous requests. It means you fill out a form, hit submit, and get directed to a new page with new information from the server.
- With AJAX, when you hit submit, JavaScript will make a request to the server, interpret the results, and update the current screen. In the purest sense, the user would never know that anything was even transmitted to the server.
- XML is commonly used as the format for receiving server data, although any format, including plain text, can be used.
- AJAX is a web browser technology independent of web server software.
- A user can continue to use the application while the client program requests information from the server in the background.
- Intuitive and natural user interaction. Clicking is not required, mouse movement is a sufficient event trigger.
- Data-driven as opposed to page-driven

6.8 Chapter Review Exercises

1. Write a program to display a “Hello World” message in the Web browser. In addition, display the host name and Session Id. Write JSP code using HTML code.
2. Write a program using JSP to calculate the sum of numbers from 1 to 25.
3. Write a program that uses the `response.setHeader()` method to display the date. In addition, set the response header attribute and utilize the JSP expression tag.
4. Write a program to display the session ID, creation time, and the last accessed time of the Web page. Using `session.getID`, `session.getCreationTime()`, and `session.getLastAccessedTime()`.
5. Write a program using the `request.getParameter()` method to enter the Name and Password of a user and display the output on another JSP page.

Course Title: Event-Driven Programming

Course Code: Itec2052

Compiled By Tizita Obsa

Review by

Birhanesh Fikre

EVENT DRIVEN PROGRAMMING

Course description

This is a course in event-driven programming building on prior programming experience. The course covers topics in control arrays, exception handling, and the use of properties, controls, and multiple forms. Introduces database manipulation and database controls, including use of database linking and programming applications to display, edit, and update databases by use of the data access object.

This course uses advanced programming tools such as Visual Studio .Net.

COURSE OBJECTIVE

Upon successful completion of the course, the student should be able to:

-  Demonstrate knowledge of a high-level object oriented programming language.
-  Demonstrate use of data types in program and systems development.
-  Demonstrate knowledge of event-driven programming and systems development
-  Develop well-organized, block-structured, easily read programs
-  Demonstrate ability to design graphical user interfaces (GUI).
-  Demonstrate ability to code arithmetic instructions, conditional statements, repetition structures, sub procedures, and control arrays in programs.
-  Demonstrate ability to develop programming applications to manipulate databases (including query, display, edit, update functions).
-  Demonstrate ability to develop integrated multiform programs.
-  Develop appropriate testing procedures and documentation for programs
-  To acquaint the student with computer problem solving using a visual development environment through the assignment of a variety of problems requiring solution development, program implementation, and documentation.

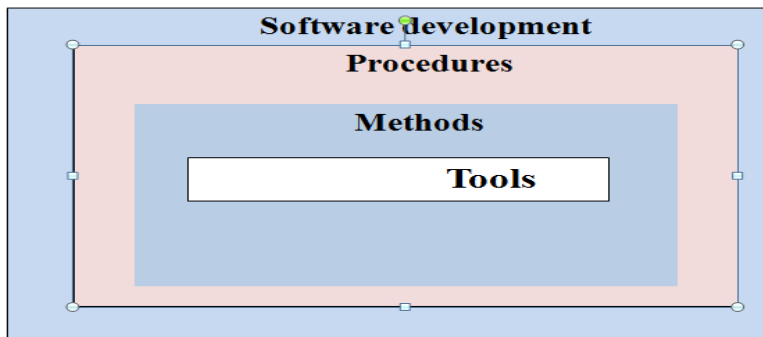
Chapter one:

1. Introduction

1.1. Introduction to Software Development

For developing software, software engineers use various tools, methods and procedures. These tools, methods, and procedures are used to control the process of software development in order to build high quality software. the method provide “how to perform task” for building software while the tools provide automated or semi-automated support for the method. They are used in all the stages of software development process.

The software development procedures integrate the methods and tools together.



❖ **We can divide the software development process in to different stages, these are**

- Planning identifying the problem
- Analysis determining the input ,process and output of the program or determining program requirements
- Design depicting the blue print of the program mainly using flow chart and algorithm
- Development writing the actual code using programming language and other development tools
- Maintenance r& testing finding errors and correcting errors or bugs which is also called debugging

1.2. Software Development Approaches(paradigms)

There are three basic software development approaches

- Procedural
 - The program specifies the sequence of all operations step-by-step. The program logic determines the order of the instruction to be executed. Ex. C, Basic, Cobol, Fortran. Etc
- Object oriented
 - The whole system is viewed developed as collection of objects.
 - An **Object** is a real world entity, which has distinguishable property from other Objects. Ex. person, account, table, form, text box, label.. etc

- **Properties** are the attributes or nature of the Object, such as name, color, size.. etc.
 - **Methods** are actions associated with objects. Ex. move, resize, print, read, clear ..etc.
 - Example for Object Oriented Language is C++, VB, Java.
- Event driven programming(approaches)
 - The programs do not follow sequential logic, instead, the user can press keys and click on various buttons and boxes in a window.
 - Each user action can cause an Event to occur, which triggers a procedure that programmer have written. Ex. VB.

Remark:

There is a difference between event and event procedures

- An **Event** is the program's response to an action taken by the user. When user takes action an event is occurred, which enables the appropriate procedure to be executed.

Ex. Command button click is one event

- The **Event Procedure** is the group of commands or instructions to be executed when some event occurs.

Ex. Set of actions to be carried over when the command button is clicked

1.3. Rapid Application Development(RAD)

RAD is the principle of developing computer applications quickly with drag and drop techniques without worrying to write codes particularly for the user interface of the application.

A good tool to develop RAD application is visual Basic which has a readymade tools that helps to develop the application user interface quickly.

1.4. Software Development Principles

Software development has the following objectives and principles

- Satisfy user requirements
- Be easy to understand by users
- Be easy to operate
- Be easy to modify or change
- Be easy to expandable
- Have a good user interface
- Handle the error and exceptions
- Have appropriate security controls against miss -use of data and application

Chapter two:

2. Introduction to .NET

2.1. The .NET Platform and Its Architecture

It is a virtual machine that provides a common platform to run an application that was built using the different language such as c#, vb.net, visual basic etc.

It is also used to create a form based, console based, mobile and web based application or services that are available in Microsoft environment.

Simply put the .NET framework has a run time environment, the common language run time that is responsible for executing .net code on the windows operating system.

It abstracts (hides the complexity) code execution and provides different services like memory management, security, exception handling.

The .NET framework architecture is a set of software components designed to support the development of desktop, web and mobile applications.

The .net framework consists of libraries that developers can use to create their programs

2.2. Base Class Library

The base class library is a simple run time library for modern programming languages.

It serves as the standard for run time library for the language c# as well as one of the common language interface (CLI) standard libraries.

The .net framework class library is the collection of classes, namespaces, interface and value types that are used for .net applications.

The two major components of .NET framework are the common Language Run Time (CLR) and the .NET framework class library.

2.3. Common Language Runtime

The CLR is the execution engine that handles running applications. The class library provides a set of APIs and types for common functionality

2.4. Uses of .NET Platform in Application Development

.NET framework class library is the collection of classes, namespaces, interface and value types that are used for .NET applications.

2.5. Introduction to Microsoft Visual Studio 2010

Visual Basic provides a familiar, easy to understand and modern object-oriented programming language designed for developing applications for the .NET Framework.

It also provides an Integrated Development Environment (IDE) with a collection of development tools, debuggers, and visual designers. Visual Basic 2010 improves readability of code, simplifies programming and deployment with the Microsoft Office APIs (Application Program Interface), allows easier interoperability with dynamic languages, and increases support for event-driven development and quick code navigation.

Event is an action taken by user such as mouse click, key press, message from operating system or SMS.



A programming paradigm in which the flow of program execution is determined by *events*.

For example a user action such as a *mouse click*, *key press*, or a *message* from the operating system or another program.

Virtually all object-oriented and visual languages support event-driven programming.

Visual Basic, C#, Visual C++ and Java are examples of such languages.

Integrated Development Environment (IDE) such as Visual studio provides much of the code for detecting events automatically when a new application is created.

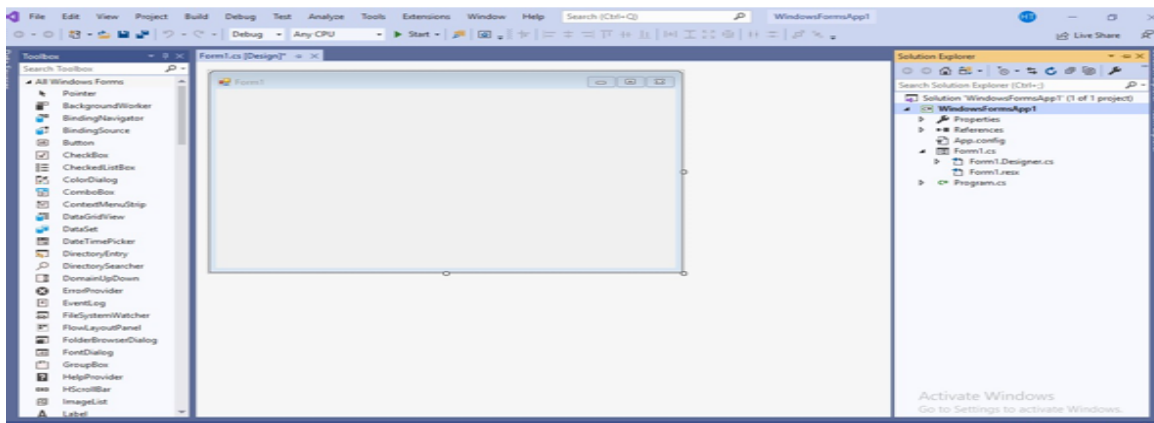
Visual Studio is the Integrated Development Environment in which developers work when creating programs in one of many languages, including C#, for the .NET Framework.

It is used to create console and graphical user interface (GUI) applications along with Windows Forms or WPF (Windows Presentation Foundation) applications, web applications, and web services in both native code together with managed code for all platforms supported by Microsoft window.

The programmer can therefore concentrate on issues such as interface design, which involves adding controls such as command **buttons**, **text boxes**, and **labels** to standard *forms*.

Once the user interface is substantially complete, the programmer can add event-handling code to each control as required.

Visual Studio IDE



2.6. SDI and MDI Forms

SDI

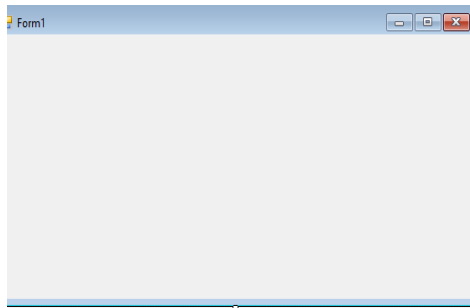
Single Document Interface (SDI) application allow only one open document frame window at a time where as MDI allows multiple document frame windows to be open in the same instance of an application.

MDI

Multiple Document Interface (MDI) applications enable you to display multiple document at the same time, with each document displayed in its own window.

MDI application often has a window menu item with sub menu for switching between or documents.

In MDI there is a single parent window and a number of child window. Word and excel are examples of MDI applications.



2.7. Form and Controls

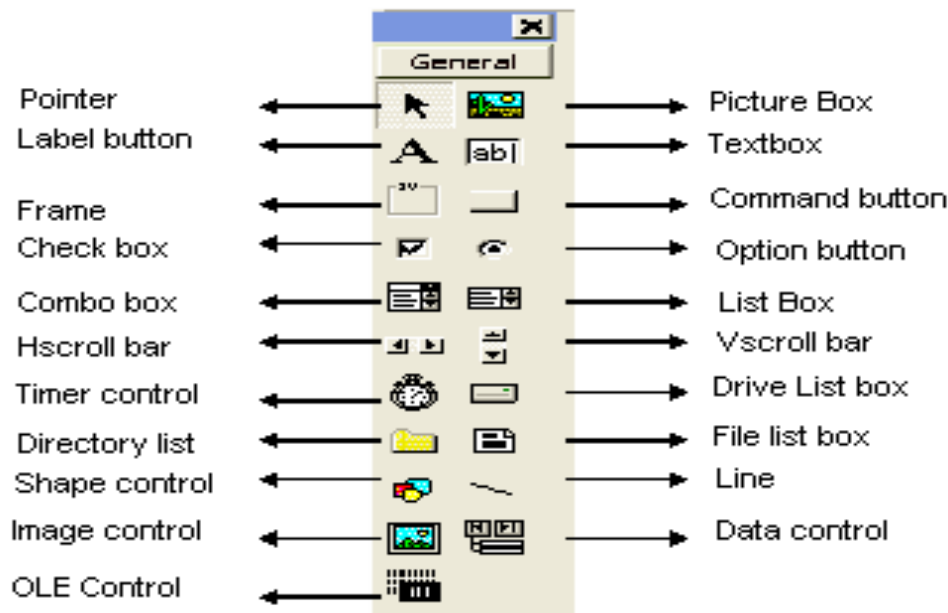
Visual basic form is the container for all the controls that make up the user interface.

Vb.Net provides a huge variety of controls that help you to create rich user interface.

Common controls in vb.NET

Working with Controls

- ✚ Controls are added to the Form from the Toolbox. Each control has a set of properties, and a set of event procedures associated with it. The list of controls is given below.



Displaying output data (Label, Textbox)

Label Control

- ✚ The Label control is used to display text that can't be changed directly by the user.
- ✚ Labels are more commonly used to place captions against other controls or store calculated values.
- ✚ The three-letter mnemonic for a Label is lbl (eg. lblPrompt).
- ✚ The default property of a Label is Caption. The Caption property may be set at design-time using the Properties Window, and changed at run-time.

lblPrompt.Caption = "Changed at run-time"

TextBox Control

- ✚ The TextBox is used to display text that may be edited directly by the user.
- ✚ The three-letter mnemonic for a TextBox is txt (eg. txtData).
- ✚ The default event for a TextBox is "Change", and the default property is "Text".
- ✚ To allow text on more than one line, the Multiline Property must be set to True. If the ScrollBars Property doesn't allow a Horizontal ScrollBar, then the text will wrap.
- ✚ The length of the Text property is limited to 2048 characters for a TextBox, and about 32K if the MultiLine property is set to True.

Entering input (Textbox),

- ✚ Input data is generally entered through a Text box. Typically, the user enters a string from the keyboard when the program is executed.

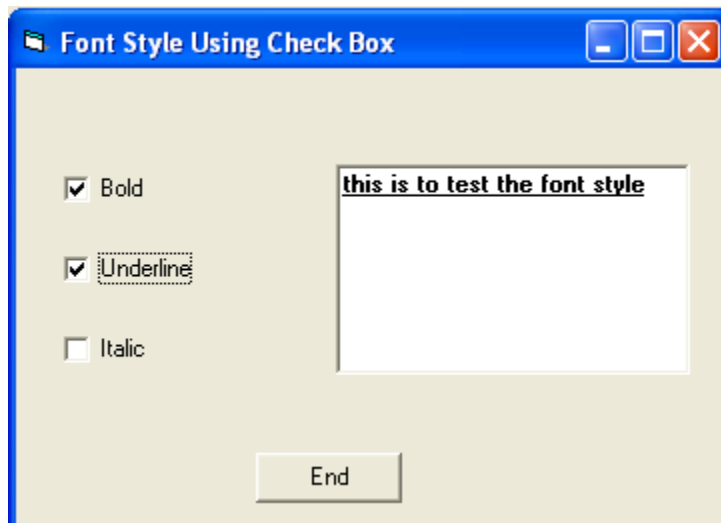
- ✚ This string is automatically assigned to the text box's text property. If the string represents a number, it can be converted to an actual numerical value by means of the **Val** function.

Selecting multiple features (Check box)

CheckBox Control

- ✚ The CheckBox control is used to give the user a choice of yes/no multiple choice options.
- ✚ The three-letter mnemonic for a CheckBox is chk (eg. chkHobbies).
- ✚ The "Value" property may be used to determine whether an item has been selected - a value of 1 indicates true, and a value of 0 indicates false

Example: Changing the font style using check box controls



Coding

```
Private Sub Chkbold_Click()  
If Chkbold.Value = 1 Then  
    Text1.FontBold = True  
  
Else  
    Text1.FontBold = False  
End If  
End Sub
```

```
Private Sub chkitalic_Click()  
If chkitalic.Value = 1 Then  
    Text1.FontItalic = True  
  
Else
```

```

        Text1.FontItalic = False
    End If
End Sub

Private Sub chkunder_Click()
    If chkunder.Value = 1 Then
        Text1.FontUnderline = True

    Else
        Text1.FontUnderline = False
    End If

End Sub

```

Selecting Exclusive alternatives (Opting button) and Frames

OptionButton Control

- ✚ The OptionButton control is used to group options where the user can only select only one.
- ✚ As only one item may be selected OptionButtons are grouped in containers such as the Form, Frame or PictureBox.
- ✚ The three-letter mnemonic for an OptionButton is opt (eg. optPaymentMethod).
- ✚ The "Value" property may be used to determine whether an item has been selected - either True or False.

Example : Temperature Conversion using Option buttons

Coding

```

Private Sub cmdcalculate_Click()
Dim vin, vout As Single
vin = Val(Text1.Text)

If optfahtocel.Value = True Then
    vout = 1.8 * vin + 32

Else
    vout = (5 / 9) * (vin - 32)
End If
Text2.Text = Str(vout)
End Sub

Private Sub optceltofah_Click()
If optceltofah.Value = True Then
    Label1.Caption = "Enter the Celcius value"
    Text1.Text = ""
    Label2.Caption = "The equivalent Fahrenheit value is "
    Text2.Text = ""
End If
End Sub

Private Sub optfahtocel_Click()
If optfahtocel.Value = True Then
    Label1.Caption = "Enter the Fahrenheit value"
    Text1.Text = ""
    Label2.Caption = "The equivalent Celcius value is "
    Text2.Text = ""
End If
End Sub

```

Frame Control

- ✚ The Frame control is used to group controls and provides a means of sub-dividing the Form visually.
- ✚ The three-letter mnemonic for a Frame is fra (eg. fraPaymentMethod).
- ✚ The default property is Caption, which can be used to give context to the grouping (eg. Payment Method).
- ✚ Controls should be drawn within the Frame in order to be associated with the Frame. When the control is associated, moving the Frame also moves all of the associated controls.
- ✚ When option buttons are used, only one may be selected on the Form. Option Buttons placed in a Frame are associated with the Frame, and are treated as a new group.

Selecting from a list (Combo Boxes and List boxes)

ComboBox Control

- ✚ The ComboBox is a combination of a TextBox and a ListBox control.
- ✚ Items may be added to the list at design-time using the "List" property (<CTRL> + <ENTER> takes you to the next line to add a new item).
- ✚ Items may be added to the list at run-time using the AddItem method, and removed at run-time using the RemoveItem method (see below for details).
- ✚ The "Clear" method removes all items from the list.
- ✚ The three-letter mnemonic for a ComboBox is cbo (eg. cboArtist).
- ✚ The "Style" property determines which type of ComboBox is displayed from a choice of three.

ComboBox Style Property		
Value	Constant	Description
0	vbComboDropDown	A TextBox with an arrow to produce a dropdown list
1	vbComboSimple	A TextBox with a ListBox always displayed beneath
2	vbComboDrop-DownList	Restricts items in the TextBox to those in the list

- ✚ The following example produces a MessageBox when a new item has been selected from the list.

```

Private Sub cboArtist_Click()
    MsgBox cboArtist.Text, vbInformation, "Selected Item"
End Sub

```

The AddItem Method

- ✚ The AddItem method is used to add items to either a ComboBox or a ListBox.
- ✚ The AddItem method has no effect if the ComboBox or ListBox is bound to a Data control.
- ✚ The AddItem method takes two parameters, "name", and "Index", where Index is an optional parameter.
- ✚ The following adds, "Garbage", to the end of the list, or to its correct sorted position of the Sorted Property has been set to True.
`cboArtist.AddItem "Garbage"`
- ✚ The following example adds the item, "Blur", to position 3 within the list. The Sorted Property must be set to False if you specify a position.
`cboArtist.AddItem "Blur", 3`
- ✚ The following example loads three artists into a list at run-time.

```
Private Sub Form_Load()  
    cboArtist.AddItem "Gorillaz"  
    cboArtist.AddItem "JJ72"  
    cboArtist.AddItem "Radiohead"  
End Sub
```

The RemoveItem Method

- ✚ The RemoveItem method is used to remove items from either a ComboBox or a ListBox.
- ✚ The RemoveItem method takes one parameter to indicate which item is to be removed.
`cboArtist.RemoveItem 0`
removes the first item from the list.
- ✚ The RemoveItem method has no effect if the ComboBox or ListBox is bound to a Data control.
- ✚ The individual items may be accessed using their index number to identify their position in the list. The following would display the fourth item in the list.

```
MsgBox cboArtist.List(3), vbInformation, "Artist"
```


ListBox Control

- ✚ The ListBox displays a list of items that may be selected by the user.
- ✚ If the number of items in the list exceeds the size of the list box, scroll bars are added automatically.
- ✚ Items may be added to the list at design-time using the "List" property (<CTRL> + <ENTER>) takes you to the next line to add a new item.
- ✚ Items may be added to the list at run-time using the AddItem method, and removed at run-time using the RemoveItem method. Both AddItem, and RemoveItem work in the same way as Combo box control.
- ✚ The "Clear" method removes all items from the list.
- ✚ The three-letter mnemonic for a ListBox is lst (eg. lstArtist).
- ✚ The "Style" property may be used to add a CheckBox to each item in the list.

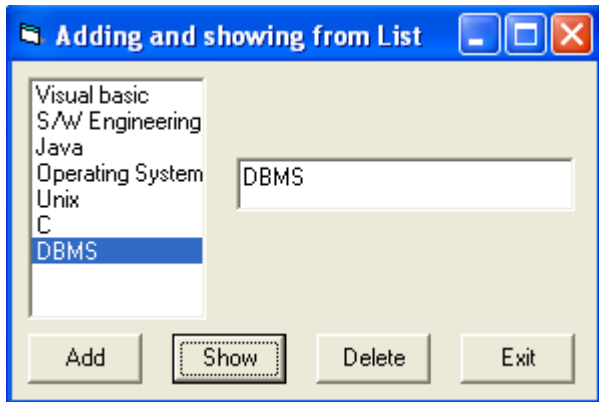
ListBox Style Property		
Value	Constant	Description
0	vbListBoxStandard	A standard list
1	vbListBoxCheckbox	A check box is added to each item in the list

- ✚ The following example produces a MessageBox when a new item has been selected from the list.

```
Private Sub lstArtist_Click()  
    Dim strMsg As String  
    strMsg = lstArtist.List(lstArtist.ListIndex)  
    MsgBox strMsg, vbInformation, "Selected Item"  
End Sub
```

- ✚ Like the ComboBox, individual items may be accessed using their **index number** to identify their position in the list.
- ✚ The **ListIndex** property maybe used to determine which element in the list currently has focus.

Example : Adding and showing items in the list box



Coding :

```

Private Sub cmdadd_Click()
    If (Text1.Text <> "") Then
        List1.AddItem (Text1.Text)
        Text1.Text = ""
    Else
        MsgBox "Enter an item to be entered in list"
    End If
    Text1.SetFocus
End Sub

Private Sub cmddel_Click()
    Dim choice As Integer
    If List1.ListIndex <> -1 Then
        choice = MsgBox("Are you sure, to delete the item?", vbOKCancel +
            vbQuestion, "Delete Item")
        If choice = 1 Then
            List1.RemoveItem (List1.ListIndex)
        End If
    Else
        MsgBox "Select an item"
    End If
End Sub

```

```

End If
End Sub

Private Sub cmdexit_Click()
End
End Sub

Private Sub cmdshow_Click()
Text1.Text = List1.List(List1.ListIndex)
End Sub

Private Sub Text1_KeyPress(KeyAscii As Integer)
If KeyAscii = 13 And Text1.Text <> "" Then
    List1.AddItem (Text1.Text)
    Text1.Text = ""
    Text1.SetFocus
End If
End Sub

```

Pointer, Line control, Command button

The Pointer

- ✚ The Pointer is not a control, but is used to interact with the controls on the form, allowing you to move and resize them.
- ✚ The Pointer is selected by default. When a control is added to the form, the focus reverts back to the Pointer.

Line Control

- ✚ The Line control is used to display a line on a form.
- ✚ The line is visible even when the form's AutoRedraw property is set to False. The X1, X2, Y1 and Y2 properties may be used to move and resize the line at run-time.
- ✚ The three-letter mnemonic for a Line is lin (eg. linDiagonal).
- ✚ The following example positions a line on the form from the top left corner to the bottom right corner.

```

Private Sub Form_Load()
    ' Top left corner
    linDiagonal.X1 = 0

```

```

linDiagonal.Y1 = 0

' Bottom right corner

linDiagonal.X2 = Me.ScaleWidth

linDiagonal.Y2 = Me.ScaleHeight

End Sub

```

Command Button Control

- ✚ The CommandButton is used by the user to invoke some action.
- ✚ The three-letter mnemonic for a CommandButton is cmd (eg. cmdQuit).
- ✚ The default event for a CommandButton is "Click".
- ✚ The following example uses a CommandButton to end the program.

```

Private Sub cmdQuit_Click()

    Dim response As Integer

    response = MsgBox("Are you sure", vbQuestion + vbYesNo, "Quit")

    If response = vbYes Then

        End

    End If

End Sub

```

To display images(Picture, Image controls)

PictureBox Control

- ✚ The PictureBox is used to display images or act as a container to other controls.
- ✚ The three-letter mnemonic for a PictureBox is pic (eg. picFace).
- ✚ The main event for a PictureBox is the Click event.
- ✚ Pictures are loaded into the PictureBox using the LoadPicture function.
- ✚ The following example loads a picture.

```

Dim strFileName As String

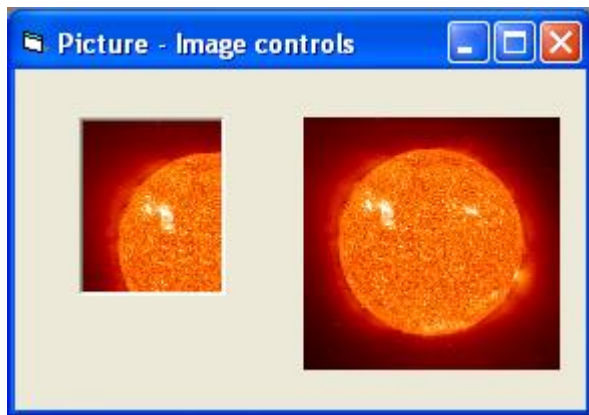
strFileName = "c:\images\face.gif"

picFace.Picture = LoadPicture(strFileName)

```

Image Control

- ✚ The Image control is used to display a picture.
- ✚ It doesn't contain as many properties, methods and events as a PictureBox but repaints faster as it uses fewer system resources.
- ✚ The three-letter mnemonic for an Image is img (eg. imgFace).
- ✚ The following example loads a picture into an Image control.
`imgFace.Picture = LoadPicture("c:\images\face.gif")`
- ✚ Both the controls are similar but the image control has stretch property. The stretch property to make the picture enlarge to fill the control.



Shape control

- ✚ The Shape control is used to display a graphical shape.
- ✚ The shape is set at design-time, and the actual shape is determined by the Shape property and may be changed at run-time.

Shape Property		
Value	Constant	Description
0	vbShapeRectangle	Rectangle
1	vbShapeSquare	Square
2	vbShapeOval	Oval
3	vbShapeCircle	Circle

4	vbShapeRoundedRectangle	RoundedRectangle
5	vbShapeRoundedSquare	RoundedSquare

- ✚ The three-letter mnemonic for a Shape is shp (eg. shpOutline).
- ✚ The following example changes a shape to an oval.

shpOutline.Shape = vbShapeOval

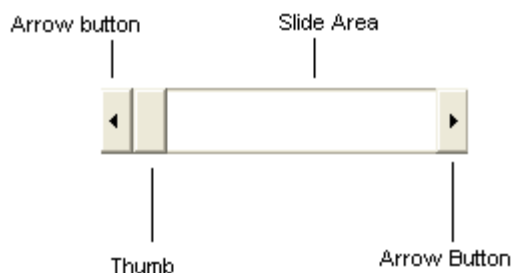
Creating Timed events (Timer control), Scroll bars

Timer Control

- ✚ The Timer control is visible at design-time, but not shown at run-time.
- ✚ It is used for background processing at intervals specified by the Interval property.
- ✚ The Interval property takes an integer in the range 0 to 65,535.
- ✚ The Interval is measured in milliseconds, therefore a value of 1000 equals an interval of one second. An Interval of 0 disables the Timer control in VB 6.
- ✚ To disable the Timer, set the Enabled Property to False. Setting it to True will enable it again.
- ✚ The three-letter mnemonic for a Timer is tmr (eg. tmrMove).
- ✚ The default event for a Timer is the Timer event.

HScroll and VScroll Control

- ✚ Scroll bars provide a convenient way of navigating through large amounts of information or providing a visual indication of a value.
- ✚ The Min and Max properties allows you to specify the minimum and maximum value of the scroll bar.
- ✚ The Value property indicates the position of the thumb of the scrollbar in relation to the Max and Min values.
- ✚ The SmallChange property specifies how much to increment or decrement the value if you click on the arrow, and the LargeChange property specifies how much to increment or decrement the value if you click on the track of the scroll bar.



- Visual Basic offers two scroll bars, a horizontal scroll bar (HScroll), and a vertical scroll bar (VScroll)
- A VScrollBar is placed vertically on the form.
- The three-letter mnemonic for a VScrollBar is vsb (eg. vsbVolume).
- A HScrollBar is placed horizontally on the form.
- The three-letter mnemonic for a HScrollBar is hsb (eg. hsbVolume).
- The default event for a scroll bar is the Change event.
- The following example uses a Label to show the exact value of the scroll bar.

```
Private Sub hsbVolume_Change()
    lblVolume = hsbVolume.Value
End Sub
```

Drive list, Directory and File list controls

DriveListBox Control

- The DriveListBox allows the user to select a valid drive at run-time. This control is usually synchronised with the DirListBox and the FileListBox.
- The three-letter mnemonic for a DriveListBox is drv (eg. drvList).
- The default event for a DriveListBox is the Change event.
- As the drive may not be ready to use, it is a good idea to use error trapping if synchronised with the DirListBox and FileListBox.

DirListBox Control

- The DirListBox displays a list of folders. This control is usually synchronised with the DriveListBox and the FileListBox.
- The three-letter mnemonic for a DirListBox is dir (eg. dirList). The default event for a DirListBox is the Change event.

FileListBox Control

- The FileListBox displays a list of files. This control is usually synchronised with the DriveListBox and the DirListBox.
- The three-letter mnemonic for a FileListBox is fil (eg. filList). The default event for a filListBox is the Click event.
- The following properties are available for the FileListBox.

FileListBox Properties

Property	Description
Archive	A boolean value to indicate whether archive attributes are displayed (default: True).
Hidden	A boolean value to indicate whether hidden attributes are displayed (default: False).
Multiselect	An integer value to indicate whether multiple selection is allowed. The default is 0, meaning mutliselction is not allowed. A value of 1 indicates simple multi-selection. Multiple items are toggled on and off with a mouse click. A value of 2 indicates extended multi-selection. The Shift and Control keys may be used to select multiple items.
Pattern	A string that specifies the files to be displayed (eg. *.html).
ReadOnly	A boolean value to indicate whether read-only attributes are displayed (default: True)
System	A boolean value to indicate whether system attributes are displayed (default: False).

DriveListBox, DirListBox, and FileListBox Example

- ✚ This example synchronises a DriveListBox, DirListBox and FileListBox. To try the example, add a DriveListBox and name it drvList, a DirListBox and name it dirList, and a FileListBox and name it filList.

```

Private Sub dirList_Change()
    filList.Path = dirList.Path
End Sub

Private Sub drvList_Change()
    On Error GoTo driveError
    dirList.Path = drvList.Drive
    Exit Sub
driveError:
    MsgBox Err.Description, vbExclamation, "Drive Error"
End Sub

Private Sub filList_Click()

```



```

Dim strFileName As String

strFileName = filList.List(filList.ListIndex)

MsgBox strFileName, vbInformation, "Selected File"

End Sub

```

Data Control

- ✚ The Data control allows data-aware controls to be bound to Fields within a recordset, providing an efficient solution for accessing a database without writing any code.
- ✚ The Caption property will be the name displayed on the Data control.
- ✚ The DatabaseName property is used to store the location and name of the database.
- ✚ The RecordSource property is used to determine the name of the table or stored procedure in the database.
- ✚ The three-letter mnemonic for a Data control is dta (eg. dtaQuestion).

OLE Control

Object Linking and Embedding (OLE) Control

- ✚ The Object Linking and Embedding (OLE) control allows you to insert an insertable object into your application.
- ✚ When you add the control to your form, you will be given a list of insertable objects that can be created from new on your machine (eg. Paint Shop Pro) or you can select an object from an existing file such as a Word document.
- ✚ The three-letter mnemonic for an OLE control is ole.

Assigning Properties collectively (The with block)

- ✚ When assigning values to several properties within the same Object at run time, it is convenient to use a With block.
- ✚ This construct allows the Object name to be specified only once, followed by each of the property assignments.

Example

```

With label1
    . caption = "Welcome"
    . Font.Size = 12
End with
With text1
    . Text = "hello"
    . Backcolor = &H8000000A
    . BordrStyle = 0
    . Font.Size = 10
End with

```

Chapter three

3. Object-Oriented Fundamentals in VB.NET

3.1. Language Fundamentals

What Are Forms and Controls?

A form(window) is where We put all the things that people interact with as they use Our program. Those things you put on the form are controls, which enable the people who use your program to do things, such as enter text and click buttons.

What Are Intrinsic Controls?

- The intrinsic controls are available whenever we use Visual Basic.
- It is built in we can't replace or remove.
- During design time, you can access them from the Toolbox.

Some of these are

- ✓ Label
- ✓ Text box
- ✓ List box
- ✓ Combo box
- ✓ Button
- ✓ Radio button and so on

Working with properties, methods & events

- An object is a type of user interface element we create on a Visual Basic form by using a toolbox control.
- In fact, in Visual Basic, the form itself is an object.
- Every Visual Basic control consists of three important elements

3.2. Variables and Data Types

Variables and Constants

Variable

- Variables are like mail boxes in the post office. The contents of the variables changes every now and then, just like the mail boxes.
- In term of VB, variables are areas allocated by the computer memory to hold data. Like the mail boxes, each variable must be given a name.
- To name a variable in Visual Basic, you have to follow a set of rules.
The following are the rules when naming the variables in Visual Basic
 - It must be less than 255 characters
 - No spacing is allowed
 - It must not begin with a number
 - Period is not permitted

Examples of valid and invalid variable names

Valid Name	Invalid Name
My_Car	My.Car
This year	1NewBoy
Long_Name_Can_beUSE	He&HisFather *& is not acceptable

Declaring Variables

In Visual Basic, one needs to declare the variables before using them by assigning names and data types. They are normally declared in the genaral section of the codes' windows using the Dim statement.

The format is as follows:

Dim variableName as DataType

Examples

```
Dim password As String
Dim firstnum As Integer
Dim doDate As Date
```

You may also combine them in one line , separating each variable with a comma, as follows:

```
Dim password As String, yourName As String, firstnum As Integer,.....
```

- If data type is not specified, VB will automatically declares the variable as a Variant
- For string declaration, there are two possible format, one for the variable-length string and another for the fixed-length string.

- For the variable-length string, just use the same format as above.
- However, for the fixed-length string, you have to use the format as shown below:

Dim VariableName as String * n,

where n defines the number of characters the string can hold.

Example

Dim yourName as String * 10

Assigning Values to Variables

- After declaring various variables using the Dim statements, we can assign values to those variables. The general format of an assignment is

Variable=Expression

- The variable can be a declared variable or a control property value. The expression could be a mathematical expression, a number, a string, a boolean value(true or false) and etc. The following are some examples:

firstNumber	=	100
secondNumber	=	firstNumber-99
username	=	"John Lyan"
userpass.Text	=	password
Label1.Visible	=	True
Command1.Visible	=	false
Label4.Caption	=	textbox1.Text
ThirdNumber	=	Val(usernum1.Text)
total = firstNumber + secondNumber+ThirdNumber		

Constants

- A Constant value never changes. Constants are declared with the Const keyword.

Const deadline As Date = #1/20/2002#

- Once the constant has been assigned a value, the value can't be changed. Attempting to re-assign a value will result in a run-time error.

Scope and Duration

- The scope and duration of a variable is determined by the keywords Dim, Redim, Static, Public or Private. Whilst declarations can appear anywhere within the body of code, it is good programming practice to declare all variables at the beginning.

Scope of Variable	
Declaration Type	Scope and Duration
Dim	Used to declare variables in procedures. Each time the procedure is called, the data stored in variables declared with this keyword are reset (eg. an Integer data type will be reset to zero).
Static	Used to declare variables in procedures. Each time the procedure is called, the data stored in variables declared with this keyword are retained.
Public	Used to declare variables within a module. The variable is available to all procedures in the project.
Private	Used to declare variables within a module. The variable is only available to procedures within the project.
ReDim	The ReDim keyword is used to re-dimension an array.

Data types and Data declarations

- ❖ There are many types of data we come across in our daily life. For example, we need to handle data such as names, addresses, money, date, stock quotes, statistics and etc everyday.
- ❖ Similarly In Visual Basic, we are also going to deal with these kinds of data. However, to be more systematic, VB divides data into different types.

Numeric Data

- ❖ Numeric data are data that consists of numbers, which can be computed mathematically with various standard operators such as add, minus, multiply, divide and so on.
- ❖ In Visual Basic, the numeric data are divided into 7 types.

Type	Storage	Range of Values
Byte	1 byte	0 to 255
Integer	2 bytes	-32,768 to 32,767
Long	4 bytes	-2,147,483,648 to 2,147,483,648

Single	4 bytes	-3.402823E+38 to -1.401298E-45 for negative values 1.401298E-45 to 3.402823E+38 for positive values.
Double	8 bytes	-1.79769313486232e+308 to -4.94065645841247E-324 for negative values 4.94065645841247E-324 to 1.79769313486232e+308 for positive values.
Currency	8 bytes	-922,337,203,685,477.5808 to 922,337,203,685,477.5807
Decimal	12 bytes	+/- 79,228,162,514,264,337,593,543,950,335 if no decimal is use +/- 7.9228162514264337593543950335 (28 decimal places).

Non-numeric Data Types

Data Type	Storage	Range
String(fixed length)	Length of string	1 to 65,400 characters
String(variable length)	Length + 10 bytes	0 to 2 billion characters
Date	8 bytes	January 1, 100 to December 31, 9999
Boolean	2 bytes	True or False
Object	4 bytes	Any embedded object
Variant(numeric)	16 bytes	Any value as large as Double
Variant(text)	Length+22 bytes	Same as variable-length string

Suffixes for Literals

- ❖ Literals are values that you assign to a data. In some cases, we need to add a suffix behind a literal so that VB can handle the calculation more accurately. For example, we can use num=1.3089# for a Double type data.
- ❖ Some of the suffixes are displayed in Table

Suffix	Data Type
&	Long
!	Single
#	Double

@	Currency
---	----------

- ❖ In addition, we need to enclose string literals within two quotations and date and time literals within two # sign. Strings can contain any characters, including numbers. The following are few examples:

```

memberName      ="Turban,                John."
TelNumber       =                "1800-900-888-777"
LastDay         =                #31-Dec-00#
ExpTime = #12:00 am#

```

Operators, Hierarchy of operators

- ✚ In order to compute inputs from users and to generate results, we need to use various mathematical operators. In Visual Basic, except for + and -, the symbols for the operators are different from normal mathematical operators, as shown in Table

Operator	Mathematical function	Example
^	Exponential	2^4=16
*	Multiplication	4*3=12
/	Division	5/2=2.5
Mod	Modulus(return the remainder from an integer division)	15 Mod 4=3
\	Integer Division(discards the decimal places)	5\2=2
+ or &	String concatenation	"Visual"&"Basic"="Visual Basic"

6.8.1.1

6.8.1.2

6.8.1.3 Relational Operators

- ✚ Relational operators are very powerful tools, they let the VB program compare data values and then decide what action to take.

Operator	Meaning
=	Equal to
>	More than
<	Less Than
>=	More than and equal
<=	Less than and equal
<>	Not Equal to

- You can also compare strings with the above operators. However, there are certain rules to follows: Upper case letters are less than lowercase letters, "A"<"B"<"C"<"D".....<"Z" and number are less than letters.

Logical Operators

- In addition to relational operators, there are a few logical operators which offer added power to the VB programs.

Operator	Meaning
And	Both sides must be true
Or	One side or other must be true
Not	Negates truth

Arithmetic expressions

- The Arithmetic expression $a / b * c$ is equivalent to the mathematical expression $(a/b)c$, since the operations are carried out from left to right.
- The arithmetic expression $b^2 - 4 * a * c$ is equivalent to $b^2 - 4ac$. In this case, the quantity b^2 formed initially, followed by the product $4 * a * c$. The subtraction is performed last, resulting in the final numerical quantity $(b^2) - (4 * a * c)$
- The algebraic expression $2(x_1 + 3x_2)$ must be written in VB as $2 * (x1 + 3 * x2)$

String Expressions

- Numerical operations cannot be performed on string constants or string variables.
- Strings and String variables can be concatenated. & or + sign can be used for concatenation.
- Suppose the string variables str1 and str2 have been assigned the following values
Str1 = "Hello "

Str2 = "welcome"

Then the string expression

Str1 & " " & str2 & " mydear" (or) str1 + " " + str2 + " mydear"

Will cause the three individual strings to be concatenated, resulting in the single string

Hello welcome mydear

Program Comments

.....

- Comments provide a convenient means to document a program
- A Comment consists of a single apostrophe ('), followed by a textual message.
- Comments can be inserted anywhere in a Visual Basic program, they have no effect during the program execution.

' Program to calculate the roots of a quadratic equation

x1 = (-b + root) / (2 * a) ' calculate the first root

x2 = (-b - root) / (2 * a) ' calculate the second root

.....

- The entire first line is a comment, which serves as a program heading. The last two lines each have a comment attached at the end of an executable statement.

Library functions

Visual Basic contains numerous Library Functions that provide a quick and easy way to carry out many mathematical operations, manipulate strings and perform various logical operations.

A library function is accessed simply by stating its name, followed by whatever information must be supplied to the function called arguments, enclosed in parentheses.

.....

Function	Application	Description
Abs	$y = \text{Abs}(x)$	Return the absolute value of x ; $y = x $.
CDB1, CInt, CSng, CStr, CVar, etc.	$y = \text{CInt}(x)$	Convert x to the appropriate data type (CDB1 converts to double, CInt to integer, CSng to single, etc.).
Chr	$y = \text{Chr}(x)$	Return the character whose numerically encoded value is x . For example, in the ASCII character set, $\text{Chr}(65) = \text{"A"}$.
Cos	$y = \text{Cos}(x)$	Return the cosine of x (x must be in radians).
Date	$y = \text{Date}$	Return the current system date.
Exp	$y = \text{Exp}(x)$	Return the value of e to the x power; $y = e^x$.
Format	$y = \text{Format}(x, \text{"fmt str"})$	Return the value of x in a format designated by "fmt str" (format string). Note that the format string may take on several different forms.
Int	$y = \text{Int}(x)$	Return the largest integer that algebraically does not exceed x . For example, $\text{Int}(-1.9) = -2$.
Lcase	$y = \text{Lcase}(x)$	Return the lowercase equivalent of x .
Left	$y = \text{Left}(x, n)$	Return the leftmost n characters of the string x .
Len	$y = \text{Len}(x)$	Return the length (number of characters) of x .
Log	$y = \text{Log}(x)$	Return the natural logarithm of x ; $y = \log_e(x)$, $x > 0$.
Mid	$y = \text{Mid}(x, n1, n2)$	Return the middle $n2$ characters of the string x , beginning with character number $n1$.
Right	$y = \text{Right}(x, n)$	Return the rightmost n characters of the string x .
Rnd	$y = \text{Rnd}$	Return a random number, uniformly distributed within the interval $0 \leq y < 1$.
Sgn	$y = \text{Sgn}(x)$	Determine the sign of x ; ($y = +1$ if x is positive, $y = 0$ if $x = 0$, and $y = -1$ if x is negative).
Sin	$y = \text{Sin}(x)$	Return the sine of x (x must be in radians).
Sqr	$y = \text{Sqr}(x)$	Return the square root of x ; $y = \sqrt{x}$, $x > 0$.
Str	$y = \text{Str}(x)$	Return a string whose characters comprise the value of x . For example, $\text{Str}(-2.50) = \text{"-2.50"}$.
Tan	$y = \text{Tan}(x)$	Return the tangent of x (x must be in radians).
Time	$y = \text{Time}$	Return the current system time.
Ucase	$y = \text{Ucase}(x)$	Return the uppercase equivalent of x .
Val	$y = \text{Val}(x)$	Return a numeric value corresponding to the string x , providing x has the appearance of a number. For example, $\text{Val}(\text{"-2.50"}) = -2.5$.

3.3. Control Flow

control flow (Branching and Looping statements)

Branching

If, If-Then-Else, Nested if statement

Select case Statement

Looping

For-Next, Do-Loop, While-Wend

Exit for, Exit Do and Stop statements

❖ Branching

Visual Basic procedures can test conditions and then, depending on the results of that test, perform different operations. The decision structures that Visual Basic supports include:

If...Then
If...Then...Else
Nested If
Elseif ladder
Select Case

➤ **IF... Then**

The IF condition allows you to execute a certain section of code in one particular instance.

The General syntax for If .. then

If conditions Then
VB expressions
End If

- Then keyword must be on the same line
- End if and Elseif must appear alone on a line
- Elseif is one word, where End if is two words

The condition / comparison made on similar typed values

The result of the condition is 0, if the condition fails 1 or any numeric values are treated as true

Comparing two strings begins with the leftmost char and proceeds one char at a time from left to right, when the char is not same, comparison is terminated

The strings are compared according to the ascii values

Y and y are not same, hence strings can be checked with Ucase and Lcase functions

Compound conditions are the conditions joined by logical operators

Ex If name = " Jose" and age < 20 then

➤ **If ..Then ...Else**

To effectively control the VB program flow, we shall use If...Then...Else statement together with the relational operators and logical operators.

The general format for the if...then...else statement is

If conditions Then
VB expressions
Else
VB expressions
End If

Any If..Then..Else statement must end with End If.

Sometime it is not necessary to use Else.

➤ **Elseif Ladder**

More than two alternatives can be incorporated with this form, if any one condition is met the control will exit the if structure.

```
If mark >= 75 Then
    Print "A- Grade"
Elseif mark >=60 and mark< 74 Then
    Print "B-grade"
Elseif mark >=45 and mark < 60 then
    Print "C-grade"
Else
    Print "D=grade"
End If
```

Select Case

Testing a variable or expression for multiple values the case structure provides a flexible and powerful solution

If you have a lot of conditional statements, using If..Then..Else could be very messy. For multiple conditional statements, it is better to use Select Case

The format is :

Select Case expression

Case							value1
	Block	of	one	or	more	VB	statements
Case							value2
	Block	of	one	or	more	VB	Statements
Case							value3
	Block	of	one	or	more	VB	statements
Case							value4
				.			.
Case							Else
	Block of one or more VB Statements						
End Select							

The data type specified in expression must match that of Case values.

the CASE structure is basically a whole set of IF statements put together.

Values list that we want to match may be numeric, string constants, variables, range of values, relational condition or combination of these.

There is no limit to the number of statements that can follow a case statement

When using relational operator, the keyword IS must be used

Case else part is executed when none of the values specified has a match

The CASE Structure is a much better alternative to nested IF conditions.

Example 1

Private	Sub	cmdOk_Click()
Select Case txtAge.Text		

```

Case 7
  MsgBox "Grade 1"
Case 8
  MsgBox "Grade 2"
Case 9
  MsgBox "Grade 3"
Case 10
  MsgBox "Grade 4"
Case 11, 12, 13
  MsgBox "Grade 5"
Case 14, 15, 16
  MsgBox "Grade 8"
Case Else
  MsgBox "Not in school"
End
End Sub
Select

```

Example 2

' Examination Grades

```

Dim name As String
Private Sub Compute_Click( )
grade=txtgrade.Text
Select Case name
Case
result.Caption="APPLE"
"Apple"
Case
result.Caption="BALL"
"Ball"
Case
result.Caption="CAT"
"Cat",
"pussy"
Case
result.Caption="Pass"
else
End Select
End sub

```

Example 3

```

Dim mark Sub As Single
Private Compute_Click()
'Examination Marks
mark = mrk.Text
Select Case mark

```

Case	Is	>=	85
	Label..Caption = "Excellence"		
Case	Is	>=	70
	Label..Caption = "Good"		
Case Is >= 60			
Label..Caption = "Above Average"			
Case Is >= 50			
Label..Caption = "Average"			
Case Else			
Label..Caption = "Need to work harder"			
End Select			

When using relational operator, the keyword IS must be used

Note we use the keyword **IS** here to impose the conditions. This is generally used for numeric data.

Example-4: Example-3 could be rewritten as follows:

Dim	mark	As	Single
Private	Sub		Compute_Click()
'Examination Marks			
mark	=		mrk.Text
Select Case mark			
Case	0	to	49
Label..Caption =	"Need	to	work
Case	50	to	59
Label..Caption =			"Average"
Case 60 to 69	Label..Caption = "Above Average"		
Case 70 to 84			
Label..Caption = "Good"			
Case Else			
Label..Caption = "Excellence"			
End Select			

Looping

- Looping is the process of repeating a series of instructions.
- The looping statements are used when a block of code is required to be executed continually until a condition is met.
- Pre-condition loops allow for a condition to be tested at the start of the loop. This type of loop is used when you only want to execute the block of code should the condition be true.

- Post-condition loops allow for a condition to be tested at the end of the loop. This type of loop is used when you want to execute the block of code at least once. This is useful for validation.
- Sometimes the number of iterations required are known in advance. In this case a counted loop may be used. Visual Basic provides a For ... Next construct to handle counted loops.
- An optional Step size may be added after the end value, should you wish the increment to be anything other than 1.
- The step size may be positive, negative, integer or fractional. Negative Step values are used to count backwards.

For...Next Loop

- For.. Next loop can be used to repeat the statements in a loop for a specific number of times

The

format

is:

For loopindex = initial value to final value [Step increment] One or more VB statements
--

- The counter variable is also called loop index, must be a numeric value
- The counter variable must be initialized, incremented and the counter must be tested.
- The initial and final value can be constants, variables, numeric expressions

Example

```
(a)  For counter=1 to 10
      display.Text=counter
      .....
    Next
```

```
(b)  For counter=1 to 1000 step 10
      .....
      .....
    Next
```

) Counting Down using a FOR Loop

```
Private Sub cmdCountDown_Click()  
    Dim A As Integer  
    For A = 10 To 1 Step -1  
        MsgBox "Countdown: " & A & ".", vbInformation, "Countdown"  
    Next  
End Sub
```

Do Loop

The format of Pretest loop statements

a)

```
Do While condition  
    Block of one or more VB statements  
Loop
```

b)

```
Do Until condition  
    Block of one or more VB statements  
Loop
```

The format of Post test loop statements

c)

```
Do  
    Block of one or more VB statements  
Loop While condition
```


d)

Do
 Block of one or more VB statements Loop Until
condition

Example 1

Do while counter <=1000

 num.Text=counter

 counter =counter+1

Loop

- The above example will keep on adding until counter >1000.
- The above example can be rewritten as

Do

 counter=counter+1

Loop until counter>1000

num.Text=counter

While Wend

- VB supports While-Wend structure in addition to Do loop. This loop continues to execute as long as the Logical expression is true.
- This loop is similar to Do while loop, the logical expression is tested in the beginning of the each pass through the loop.

The general syntax is

While logical expression

Executable statements

Example

```
Sum = 0
Count = 1
While count <=10
    Sum = sum + count
    Count = count + 1
Wend
```

3.4. Methods and Their Types

Methods are simply member procedures built into the class .

In vb.net there are two types of methods function and sub procedure .

In vb.net the function is a separate group of codes that are used to perform a specific task when the defined functions called.

Procedures are used to access or manipulate the characteristics of an object or a variable.

3.5. Events

An event is a signal that informs an application that some thing is important has occurred. For example when a user clicks control on a form the form can raise a clk event and call procedures that handles the event .

Event also allows separate tasks to communicate

3.6. Classes and Object

Object is an encapsulation of data along with functions that act upon that data.where as class is a blue print for objects.a class is a user defined type that describes what a certain type of object will look like

3.7. Inheritance and Overloading Implementation

Over loading allows several functions definition for the same name ,distinguished primarily through different argument types ; it is typically resolved at compile time .

Inheritance allows subclass to define more special version of the same function ; it is typically resolved at run time.

3.8. Classes Versus Components

Class provide the basic low level blue print of an object interface while component adds some functionality to it.

A component is a class that implement the system.

Generally speaking , a class is a software component that defines and implements one or more interfaces.

3.9. Array

- An array is a set of variables, represented by a single name. The individual variables are called "elements", and are identified by index numbers.
- In Visual Basic there are two types of arrays; fixed-size arrays that always remain the same size, and dynamic arrays whose size can change at run-time. You declare a visual basic array using the *Dim* statement. The following are examples of fixed arrays declared in Visual Basic:

```
Dim MyIntArray(10) As Integer
Dim MySingleArray(3 to 5) As Single
```

- The first example declares an array of (2-byte) integer values with 10 elements. The range of the array index can be 0 to 10 or 1 to 10, depending on the default value indicated with the *Option Base* statement.
- The second example declares an array of single values with 3 elements. The indexes for this second array go from 3 to 5.
- The dimensions of a fixed-size array cannot be changed; they remain the same throughout the scope of the array.

Option Base

- By default, the base index of an array is zero. The Option Base statement allows you to specify whether you want arrays to start at zero or one by default, and has the following syntax.

```
Option Base [1|0]
```

- If you want to index arrays starting at one, place the Option Base statement in General Declarations before any declarations. The following example Would have exactly 7 elements indexed from 1 to 7.

```
Option Base 1
Dim weekDay(7) As String
```

- Visual Basic actually gives you one more element than you required, so the above specifies 8 elements ranging from 0 to 7 using the default Option Base of zero. A similar declaration in C/C++ would give 7 elements indexed 0 to 6.

The Array Function

- The Array Function is used to populate a Variant data type with an array of values.

```
Dim weekDay As Variant
weekDay = Array("Sunday", "Monday", "Tuesday", _
```

```
"Wednesday", "Thursday", _  
"Friday", "Saturday")
```

Accessing an Element

- The individual element is printed using the index number to reference it. Using the above array, the following example would print Sunday on the current form.

```
Me.Print weekDay(0)
```

Similarly, you can assign values to the elements.

```
weekDay(0) = "Sunday"
```

Specifying a Range

- You may also specify minimum and maximum index numbers using the "To" keyword. This is preferable to using Option Base, as it allows other arrays to use an Index of zero. The following example declares an array with 11 elements, indexed 1999 to 2009.

```
Dim surveyYear(1999 To 2009) As Integer
```

To access an element, use the appropriate index number.

```
surveyYear(2002) = 6243
```

- The boundaries of an array may be found using the UBound and LBound functions.

```
For counter = LBound(surveyYear) To UBound(surveyYear)
```

```
    ' Process surveyYear(counter)
```

```
Next Counter
```

- If the Range of surveyYear is changed, the code will not need to be altered using the LBound and UBound functions.

Multi-Dimensional Arrays

- Visual Basic also allows multidimensional arrays. For example the statement

```
Dim a(1 to 5,1 to 5) As Double
```

Declares a square array of double precision values with 5 rows and 5 columns.

- Theoretically you can declare arrays with as many as thirty-two dimensions, but it would be hard to find a reason for doing such a thing. Normally your arrays will not have more than three or four dimensions.
- You declare a Visual Basic dynamic array by omitting its dimensions in the declaration statement:

```
Dim MyDynArray() As Long
```

- You must then redimension the array with a *ReDim* statement before you use it. For example:

```
ReDim MyDynArray(10)
```

- This will allocate 10 entries for array *MyDynArray*. You can redimension an array multiple times, so the following section of code is valid:

```
Dim B() as long
...
ReDim B(4)
'Do something with array B
ReDim B(10)
'Do something else with array B
```

- Each time you execute the *ReDim* statement, all the values currently stored in the array are lost. Visual Basic resets the values to the *Empty* value (for Variant arrays), to zero (for numeric arrays), to a zero-length string (for string arrays), or to *Nothing* (for arrays of objects).
- This is useful when you want to prepare the array for new data, or when you want to shrink the size of the array to take up minimal memory. Sometimes you may want to change the size of the array without losing the data in the array. You can do this by using *ReDim* with the *Preserve* keyword. For example, you can enlarge an array by one element without losing the values of the existing elements using the *UBound* function to refer to the upper bound:

```
ReDim Preserve MyDynArray(UBound(MyDynArray) + 1)
```

- Only the upper bound of the last dimension in a multidimensional array can be changed when you use the *Preserve* keyword; if you change any of the other dimensions, or the lower bound of the last dimension, a run-time error occurs. Thus, you can use code like this:

```
ReDim Preserve Matrix(10, UBound(Matrix, 2) + 1)
```

But you cannot use this code:

```
ReDim Preserve Matrix(UBound(Matrix, 1) + 1, 10)
```

- In Visual Basic, you can declare arrays of any standard types, including strings, variants, objects and User Defined Types (UDT). Arrays created in Visual Basic are always Safe Arrays.
- In Visual Basic you can also define procedures that receive array arguments. Array arguments can only be declared and passed by reference (*ByRef*).
- Arrays may have more than one dimension. Each dimension is separated by a comma and may be specified as a range. The following example declares a simple two dimension 4 by 4 grid (remember you get one more element than you specify).

```
Dim grid(3, 3) As Integer
```

- Combinations of the two index numbers identify the different elements.

'Place a '1' in the top left corner of the grid

```
grid(0, 0) = 1
```

'Place a '1' in the bottom right corner of the grid

```
grid(3, 3) = 1
```

Control Arrays

- A control array is an array of similar controls with the same name identified by their index number.
- The controls must all be of the same type and have the same name but all other properties may be changed for each individual control.
- The easiest way to create a control array is by copying the control with and pasting it.
- Visual Basic asks if you would like to create a control array if the Index property is not set.
- Controls may be added dynamically at run-time using the "Load" method. To do this, place a single control on the form and set its Index property to zero. You can then dynamically load further controls of that type.

```
Load picNode(1)
```

```
picNode(1).Visible = True
```

- Controls may be dynamically removed using the Unload function.

```
Unload picNode(1)
```

- Say you have a form with 100 command buttons and they all accomplish pretty much the same task -- they print their caption to a label control. Now what would be better; to draw 100 command buttons and name them btnPrint1 through btnPrint100, or should you find a better way to do it? In this scenario you would undoubtedly use a control array.

- A control array is exactly what the name implies -- an array of controls. So instead of 100 different names, you would have btnPrint(1) through btnPrint(100). We'll talk about how to set up a control array.
- In a control array, there is one control which the other controls take after when they are created. Each control in a control array also has an index so we can determine which one is to be used. You assign the index of each control by giving the control a value in its .index property. If a control has a value in its .index property, then it is assumed by the program to belong to a control array.
- Every control in the control array also has the same name, and they are distinguished as different controls through their index property.
- Let's set up a program with 5 command buttons that are part of a control array. When you click on one of the 5 buttons a notice will be displayed in a label indicating which button has been clicked.

```
button    .name=btnControl .index=0    .caption="Button &1"
button    .name=btnControl .index=1    .caption="Button &2"
button    .name=btnControl .index=2    .caption="Button &3"
button    .name=btnControl .index=3    .caption="Button &4"
button    .name=btnControl .index=4    .caption="Button &5"
label     .name=lblNotice  .caption=""
```

- Notice that the indexes start from 0 and go to 4, but the captions indicate the button's number is one greater than its index. We'll take this into account when we code the controls. All the controls in a control array share the same code, and each event of a control in a control array is passed the control's index. So you can use this index argument in your code, which we'll do here:

```
Sub btnControl_Click (Index As Integer)
    lblNotice = "You selected button " & Str$(Index + 1)
End Sub
```

Collections

- Collections are similar to an array, it stores related items. The advantage of a collection over an array is that the collection lets to access its items via a key.
- To use a collection the syntax is
Dim collname as new Collection
- The collection object provides three methods and one property
 - **Add method** Adds items to the collection
 - **Remove method** deletes an item from the collection by index or key
 - **Item method** Returns an item by index or by key
 - **Count property** Returns the number of items in the collection
- **Add method**

To add new item to the collection. The syntax is
Collection.Add value, key, before, after

For example, to add the temperature for the city of Jersey to the temperature collection
Temperatures.Add 76, "Jersey"

The number 76 is the value to be stored (temperature), and the string "jersey" is the new item's key.

To place the new item in a specific location in the array, one of the arguments before or after (but not both) can be specified.

➤ **Remove method**

The remove method removes an item from a collection. The index argument can be either the position of the item to be deleted or the item's key.

Temperatures.Remove "Jersey" or
Temperatures.Remove 4

➤ **Item method**

The item method returns the value of an item in the collection.

For example,

N = Temperatures.Item("jersey") or
N = Temperatures.Item(4)

The item method is the default method for collection object, hence it can be omitted as

N = Temperatures("jersey")

➤ **Count property**

The count property returns the number of items in the collection of items in the collection. To find the how many cities have been entered so far in the temperatures collection, use the following statement

Temperatures.Count

Example

Lets implement a collection for storing city names and temperatures. Start a new project and add the declaration in the form

Dim temperatures as new Collection

To add values to the collection, add this code in the form load event

```
Private sub Form_load()  
    Temperatures.Add 76, "Atlanta"  
    Temperatures.Add 85, "Los Angeles"  
    Temperatures.Add 97, "Las vegas"  
    Temperatures.Add 66, "Seattle"  
End Sub
```


Next, create a command button, set its caption property to “ Show city temperature”, and enter the following code in its click event

```
Private sub command1_click()  
On Error go to Noitem  
    City = inputbox(“What city”)  
    Temp = Temperatures.Item(city)  
    MsgBox temp  
    Exit sub  
Noitem  
    MsgBox “This city was not found in the catalog”  
End Sub
```

Add another command button to the form, set its caption to “show average Temperature” add this code

```
Private sub command2_click()  
    For each city in Temperature  
        Total = total + city  
        Debug.print city  
    Next  
    Avgtemperature = total / temperatures.count  
    MsgBox Avgtemperature  
End sub
```

The print statement displays each element in the immediate window. The name of the loop variable is city, which refers to the value of the current item, not the city name. in this case, it's the temperature in the current city.

7 Chapter four

4. Flow control &Exception Handling

4.1. Introducing Exception Handling

An exception is a response to an exceptional circumstance that arises while a program is running such as an attempt to divide a number by zero.

Exception provide a way to transfer control from one part of a program to another .

Vb.net exception handling is built up on four key words such as try, catch, throw ,and finally.

4.2. Structured Exception Handling

the keyword throw can be written inside the try block.
When an exception is detected ,it is thrown using the throw statement which is written inside the try block.

```
    try
    {
-
-
-
        throw                exception

    }
```

A catch block is defined like follow

```
    Catch( )
    {
--
-
    }
```

```
-  
-  
}
```

The keyword `catch` catches the exception thrown by the `throw` statement in `try` block and handle it.

4.3. Implementing Exception Handling

```
Dim x as integer  
Dim y as integer  
X=textbox1.text  
Y=textbox2.text  
    try  
    {  
If(y< >0) then  
    Textbox3,text=x/y  
    else  
    {  
        throw(y)  
    }  
Catch(integer n)  
    {  
        MsgBox("there is an exception division by zero the second number is zero")  
    }  
}
```

Chapter five

5. Manipulating Files

5.1. How to open a Text File

Files (Sequential and Random files)

Up until we are only creating programs that could accept data at runtime, when a program is terminated, the data also disappear. Is it possible to save data accepted by a VB program into a storage device, such as a hardisk or diskette, or even CDRW? The answer is possible.

In this chapter, we will learn how to create files by writing them into a storage device and then retrieve the data by reading the contents of the files using customized VB programs.

Open(Creating) files

To create a file, use the following command

Open "fileName" For Output As #fileNumber

Each file created must have a file name and a file number for identification. As for file name, you must also specify the path where the file will reside.

For example

Open "E:\sample.DOCX" For Output As #1

will create a text file by the name of sample.txt in the My Document folder. The accompanying file number is 1. If you wish to create and save the file in A drive, simply change the path, as follows"

Open "E:\sample.DOCX" For Output As #1

Sample Program : Creating a text file

```
Private Sub create_Click()  
Dim intMsg As String  
Dim StudentName As String  
Open "E:\sample.DOCX" For Output As #1
```

```

intMsg = MsgBox("File sample.DOCX opened")
StudentName = InputBox("Enter the student Name")
Print #1, StudentName
intMsg = MsgBox("Writing a" & StudentName & " to sample.DOCX ")
Close #1
intMsg = MsgBox("File sample.DOCX closed")
End Sub

```

The above program will create a file E:\sample.DOCX in the 'E:' folder and ready to receive input from users. Any data input by users will be saved in this text file.

5.2. Read a file line by line in VB .NET

Reading files

To read a file created, you can use the input # statement. However, we can only read the file according to the format when it was written.

You have to open the file according to its file number and the variable that hold the data. We also need to declare the variable using the DIM command.

Sample Program: Reading file

```

Private Sub Reading_Click()
Dim variable1 As String

Open "E:\sample.DOCX" For Input As #1
Input #1, variable1
TextBox1.Text = variable1
Close #1
End Sub

```

This program will open the sample.txt file and display its contents in the Text1 textbox.

5.3. Write to a Text File

 Read About How To How To Write to a Text File

5.4. How to Copy, Move and Delete a File

READING ASSIGNMENT

Read About How To How To Copy, Move And Delete A File .

Chapter six

6. Database Programming

6.1. The ADO.NET Architecture

ADO stands for Active Data object .ADO .net is one of the Microsoft datata access technologies using which we can communicate with different data sources.

ADO.NET Offers a link between front end interfaces and the back end database.

The ADO.NET architecture can be divided into components :the data source and the client application.

ActiveX Data Objects (ADO)

- £ To use ADO, you must set the appropriate references by selecting, Project, References, and making sure there's a tick against Microsoft ActiveX Data Object Library. OLE DB provides a set of system-level COM interfaces that provide a method for retrieving data regardless of its format. ADO provides a language independent Automation model to provide access to the OLE DB services. The ADO object model comprises the following seven objects.

ADO Objects	
Object	Description
Command	The Command object defines the command to be given to a data source. This may be either a SQL statement or an invocation of a stored procedure.
Connection	The Command object defines the command to be given to a data source. This may be either a SQL statement or an invocation of a stored procedure.
Connection	
Error	The Error object may be used if the OLE DB provider has implemented it. In its absence, errors are raised in the Visual Basic Err object.

Field	The Field object represents a column in a Recordset object.
Parameter	The Parameter object is implemented as a collection and stores the parameters to be used by a Command object.
Property	The Property object may be used by the Connection, Command, Recordset, and Field objects. Each of the objects has a set of unique properties that either describe or control the behaviour of the object. The properties may either be built-in, or added to the Properties collection by the data provider.
Recordset	The Recordset object contains a set of records retrieved from the Connection object.

Cursors

- £ The Recordset object fetches data using Cursors, which are used to identify the current position in the result set, and what row will be returned next. The Recordset object supports four cursor types.

Recordset Cursors		
Cursor	Constant	Description
Command	adOpenDynamic	The result set reflects subsequent changes, additions and deletions by other users. There are no limitations on how the result set may be navigated.
Forward-only	adOpenForwardOnly	The result set reflects subsequent changes, additions and deletions by other users. The result set may only be navigated in a forward direction, and may improve performance if you make a single pass through the result set. If no cursor type is specified, this cursor is used by default.
Keyset-Driven	adOpenKeyset	The result set reflects subsequent modifications to data, but not records added by other users. This cursor prevents access to records deleted by other users. There are no limitations on how the result set may be navigated.
Static	adOpenStatic	Returns a static set of records that cannot be modified. Modifications, additions, and deletions to data by other users will not be seen with this type of cursor. There are no limitations on how the result set may

		be navigated.
--	--	---------------

- £ The cursor type is set with the CursorType property of the Recordset object.
adoRS.CursorType = adOpenDynamic

Opening a Recordset

- £ The Open method is used to retrieve a recordset from the database. The method takes four optional parameters, the recordset source, the active connection, the cursor type, the lock type, and options. The following table describes the parameters.

Open Method Parameters								
Parameter	Description							
Source	The source may be a Command object variable, a SQL statement, a table, or a stored procedure. In the absence of this parameter, the recordset will be opened with the value of the Source property of the Recordset object.							
ActiveConnection	The ActiveConnection property may be either a valid Connection object variable, or a Connction string. In the absence of this parameter, the recordset will use the ConnectionString property of the Recordset object.							
CursorType	Determines the type of cursor to use with the Recordset, where the cursor type constants are adOpenDynamic, dOpenForwardOnly, adOpenKeyset, or adOpenStatic. In the absence of this parameter, the recordset will use the value of the CursorType property of the Recordset object. If no value is specified as either a parameter, or in the CursorType property, adOpenForwardOnly is used by default.							
LockType	Determines how to implement concurrent sessions. The following constants may be used: <table><tr><td colspan="2">LockType Constants</td></tr><tr><td>LockType</td><td>Description</td></tr><tr><td>adLockBatchOptimistic</td><td>Uses an optimisic lock for bacth updates, as opposed to immediate updates.</td></tr></table>		LockType Constants		LockType	Description	adLockBatchOptimistic	Uses an optimisic lock for bacth updates, as opposed to immediate updates.
LockType Constants								
LockType	Description							
adLockBatchOptimistic	Uses an optimisic lock for bacth updates, as opposed to immediate updates.							

	adLockOptimistic	Only locks a record when an Update method is called.
	adLockPessimistic	Usually locks a record immediately upon editing.
	adLockReadOnly	No locking is used as the records are read only. If no value exists, adLockReadOnly is used by default.
	In the absence of this parameter, the recordset will be locked using the value in the LockType property of the Recordset object.	
Options	A constant that indicates how the provider should evaluate the Source argument if it represents something other than a Command object.	
	LockType Constants	
	CommandType	Description
	adCmdText	Specifies that the command should be interpreted as text.
	adCmdTable	Specifies that the command should use a SQL statement to retrieve all records from the table specified in Source.
	adCmdTableDirect	Specifies that the command should retrieve all records from the table specified in Source directly.
	adCmdStoredProc	Specifies that the value of Source is a stored procedure.
	adCmdUnknown	Specifies the type of command in Source is unknown.
	adCommandFile	Specifies that the Recordset should be restored from the file named in Source.
	adExecuteAsync	Specifies that the Source should be executed asynchronously.
	adFetchAsync	After the initial quantity specified in the CacheSize property is fetched, remaining rows should be fetched asynchronously.

- £ The following may be used to open a recordset without using any of the properties of the Recordset object.

```
adoRS.Open strSQL, strConn, adOpenForwardOnly, adLockReadOnly, adCmdText
```

- £ The following example uses the Properties to determine how the recordset should be opened, and calls the Open method with no parameters.

```
Dim adoConn As New ADODB.Connection
Dim adoRS As New ADODB.Recordset
Dim strConn As String
strConn = "Provider=Microsoft.Jet.OLEDB.4.0;Data Source="
strConn = strConn & "c:\db\music.mdb"
adoConn.ConnectionString = strConn
adoConn.Open
adoRS.Source = "SELECT * FROM CD"
adoRS.CursorType = adOpenForwardOnly
adoRS.ActiveConnection = adoConn
adoRS.Open
Do While Not adoRS.EOF
    Debug.Print adoRS.Fields("Title").Value
    adoRS.MoveNext
Loop
adoRS.Close
Set adoRS = Nothing
adoConn.Close
Set adoConn = Nothing
```

Navigating Through a Recordset

- £ The Move method moves to a record in a database specified by a row number. An optional parameter may be specified to move relative to a bookmark. The following example moves to the fourth record in the Recordset.
adoRS.Move 4

MoveFirst

- £ The MoveFirst method moves to the first record in the Recordset.
adoRS.MoveFirst

MoveLast

- £ The MoveLast method moves to the last record in the Recordset.
adoRS.MoveLast

MoveNext

- £ The MoveNext method moves to the next record in the Recordset. The EOF property should be checked to prevent an error occurring. EOF is set to True if the current record is after the last record in the Recordset.
adoRS.MoveNext

If adoRS.EOF Then
 adoRS.MovePrevious
 MsgBox "At last record", vbInformation, "Database"
End If

MovePrevious

- £ The MovePrevious method moves to the previous record in the Recordset. The BOF property should be checked to prevent an error occurring. BOF is set to True if the current record is before the first record in the Recordset.
adoRS.MovePrevious

If adoRS.BOF Then
 adoRS.Recordset.MoveNext

```
MsgBox "At first record", vbInformation, "Database"  
End If
```

Adding Records

- £ The AddNew method adds a new record to the end of the Recordset.
adoRS.AddNew

Updating Records

- £ The Update method saves the contents of the edit buffer to a specified Recordset object. Use after changing data in the current record.
 response = MsgBox("Update this record", vbYesNo + vbQuestion, "Update")
 If response = vbYes Then
 adoRS.Update
 Else
 adoRS.CancelUpdate
 End If

Structured Query Language

- The Structured Query Language (SQL - pronounced "sequel") is a very simple language used to manipulate relational databases. You can use SQL to retrieve, insert, update and delete records from relational databases. Clauses may be added to restrict the records that are effected.
 - The Select Query
 - The Insert Statement
 - The Update Statement
 - The Delete Statement

The SQL Select Query

- The select query is used to retrieve records from a database. The keywords used in a select query are summarised in the following table.

Select Keywords	
Keyword	Description
SELECT	Retrieves fields from one or more tables.
FROM	Tables containing the fields.
WHERE	Criteria to restrict the records returned.
GROUP BY	Determines how the records should be grouped.
HAVING	Used with GROUP BY to specify the criteria for the grouped records.
ORDER BY	Criteria for ordering the records.

- The simplest select query is to retrieve all records from a single table. For example, to retrieve all records from the CD table in the music database:

```
SELECT * FROM CD
```

- To select specific fields from a table, replace the (*) with a comma-separated list of field names. For example, to select the Artist and Title from the CD table:

```
SELECT Artist, Title FROM CD
```

- To limit the records returned according to some criteria, use the WHERE clause. For example, to return all Artists whose name starts with an 'S':

```
SELECT * FROM CD WHERE Artist LIKE 'S*'
```

- The condition in the WHERE clause can contain the operators <, >, <=, >=, =, <> and LIKE. Multiple clauses are joined together using the logical AND and OR operators. The results of a query are ordered using the ORDER BY clause. The results may be sorted in ascending order using the ASC keyword or descending order using the DESC keyword. For example, to list the artists in ascending order by name:

```
SELECT * FROM CD ORDER BY Artist ASC
```

- To list the artists in descending order by name:

```
SELECT * FROM CD ORDER BY Artist DESC
```

- The DISTINCT keyword may be used to retrieve unique values from fields where duplicate data may exist. The following example returns a list of unique surnames.

```
SELECT DISTINCT surname FROM customer ORDER BY surname
```

The SQL Insert Statement

- The insert statement is used to insert records into a table. The keywords used in an insert statement are summarised in the following table.

Insert Keywords	
Keyword	Description
INSERT	Inserts data into a table.
INTO	Specifies the table name to insert the data.
VALUES	The data for the fields in the table.

- The following example inserts the data "Gorillaz" and "Debut Album" into the "Band" and "Title" fields of the "CD" table.
INSERT INTO CD ([Band], [Title]) VALUES ('Gorillaz', 'Debut Album')
- The next example uses ADO to insert a record into a database.

```
Dim adoConn As New ADODB.Connection
Dim adoCmd As New ADODB.Command
Dim strConn As String, strSQL As String
' Open a Connection object
strConn = "Provider=Microsoft.Jet.OLEDB.4.0;Data Source="
strConn = strConn & "c:\db\music.mdb"
adoConn.ConnectionString = strConn
adoConn.Open
' Define a query string
strSQL = "INSERT INTO Programming ([Band], [Title]) VALUES ("
strSQL = strSQL & txtBand.Text & ", "
strSQL = strSQL & txtTitle.Text & ")"
```

```

' Set up the Command object
adoCmd.CommandText = strSQL
adoCmd.CommandType = adCmdText
adoCmd.ActiveConnection = adoConn
adoCmd.Execute

' Tidy up
Set adoCmd = Nothing
adoConn.Close
Set adoConn = Nothing

```

- Further records may be added to the database using the INSERT Statement with different values.

The SQL Update Statement

- The update statement is used to update records in a database. The keywords used in an update statement are summarised in the following table.

Update Keywords	
Keyword	Description
UPDATE	Updates data in one or more tables.
SET	Specifies the field names to be updates. If the fields belong to more than one table, the table name should be specified before the field name. (eg. CD.Artist).
WHERE	Criteria to restrict the records being updated.

- The following example updates all records in the "CD" table where the Band is "JJ73" to "JJ72".
UPDATE CD SET [Band] = 'JJ72' WHERE [Band] = 'JJ73'

The next example uses ADO to update a record in the database.

```

Dim adoConn As New ADODB.Connection
Dim adoCmd As New ADODB.Command

```

```

Dim strConn As String, strSQL As String
' Open a Connection object
strConn = "Provider=Microsoft.Jet.OLEDB.4.0;Data Source="
strConn = strConn & "c:\db\client.mdb"
adoConn.ConnectionString = strConn
adoConn.Open
' Define a query string
strSQL = "UPDATE customer SET [email]="
strSQL = strSQL & txtEmail.Text
strSQL = strSQL & " WHERE [CustNo]=" & CInt(txtCustNo.Text)
' Set up the Command object
adoCmd.CommandText = strSQL
adoCmd.CommandType = adCmdText
adoCmd.ActiveConnection = adoConn
adoCmd.Execute
' Tidy up
Set adoCmd = Nothing
adoConn.Close
Set adoConn = Nothing

```

The SQL Delete Statement

- The delete statement is used to delete records in a database. The keywords used in a delete statement are summarised in the following table.

Delete Keywords	
Keyword	Description
DELETE	Deletes the records in one or more tables.
FROM	Tables containing the records.

WHERE	Criteria to restrict the records deleted.
-------	---

- The following example deletes the records from the "CD" table where the Band and Title are passed from a Form.
- The following example deletes the record in the "CD" table where the Band is "Trash" and the title of the album is "Deserve To Go".

```
DELETE FROM CD WHERE [Band]='Trash' AND [Title]='Deserve To Go'
```

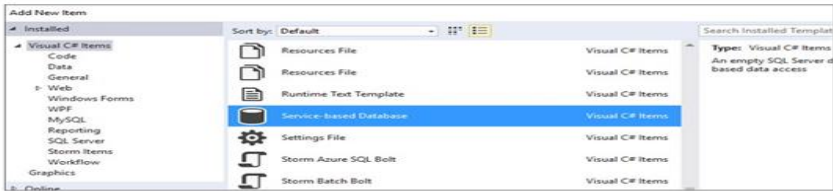
- The next example uses ADO to delete a set of records from a database.

```
Dim adoConn As New ADODB.Connection
Dim adoCmd As New ADODB.Command
Dim strConn As String, strSQL As String
' Open a Connection object
strConn = "Provider=Microsoft.Jet.OLEDB.4.0;Data Source="
strConn = strConn & "c:\db\active.mdb"
adoConn.ConnectionString = strConn
adoConn.Open
' Define a query string
strSQL = "DELETE FROM ActiveUsers WHERE [Session]='Expired'"
' Set up the Command object
adoCmd.CommandText = strSQL
adoCmd.CommandType = adCmdText
adoCmd.ActiveConnection = adoConn
adoCmd.Execute
' Tidy up
Set adoCmd = Nothing
adoConn.Close
Set adoConn = Nothing
```

Example :to connect to SQL database

Create a project and a local database file.

1. Create a new **Windows Forms App** project and name it **SampleDatabaseWalkthrough**.
2. On the menu bar, select **Project > Add New Item**.
3. In the list of item templates, scroll down and select **Service-based Database**.



4. Name the database **SampleDatabase**, and then click **Add**.

Add a data source

1. If the **Data Sources** window isn't open, open it by pressing **Shift+Alt+D** or selecting **View > Other Windows > Data Sources** on the menu bar.
2. In the **Data Sources** window, select **Add New Data Source**.



The **Data Source Configuration Wizard** opens.

3. On the **Choose a Data Source Type** page, choose **Database** and then choose **Next**.
4. On the **Choose a Database Model** page, choose **Next** to accept the default (Dataset).
5. On the **Choose Your Data Connection** page, select the **SampleDatabase.mdf** file in the drop-down list, and then choose **Next**.
6. On the **Save the Connection String to the Application Configuration File** page, choose **Next**.
7. On the **Choose your Database Objects** page, you'll see a message that says the database

View properties of the data connection

You can view the connection string for the *SampleDatabase.mdf* file by opening the Properties window of the data connection:

- Select **View > SQL Server Object Explorer** to open the **SQL Server Object Explorer** window. Expand **(localdb)\MSSQLLocalDB > Databases**, and then right-click on *SampleDatabase.mdf* and select **Properties**.
- Alternatively, you can select **View > Server Explorer**, if that window isn't already open. Open the Properties window by expanding the **Data Connections** node, right-clicking on *SampleDatabase.mdf*, and then selecting **Properties**.

1. In **Server Explorer**, expand the **Data Connections** node, and then expand the **SampleDatabase.mdf** node.

If you can't expand the Data Connections node, or the SampleDatabase.mdf connection is not listed, select the **Connect to Database** button in the Server Explorer toolbar. In the **Add Connection** dialog box, make sure that **Microsoft SQL Server Database File** is selected under **Data source**, and then browse to and select the SampleDatabase.mdf file. Finish adding the connection by selecting **OK**.

2. Right-click on **Tables** and select **Add New Table**.

The Table Designer opens and shows a grid with one default row, which represents a single column in the table that you're creating. By adding rows to the grid, you'll add columns in the table.

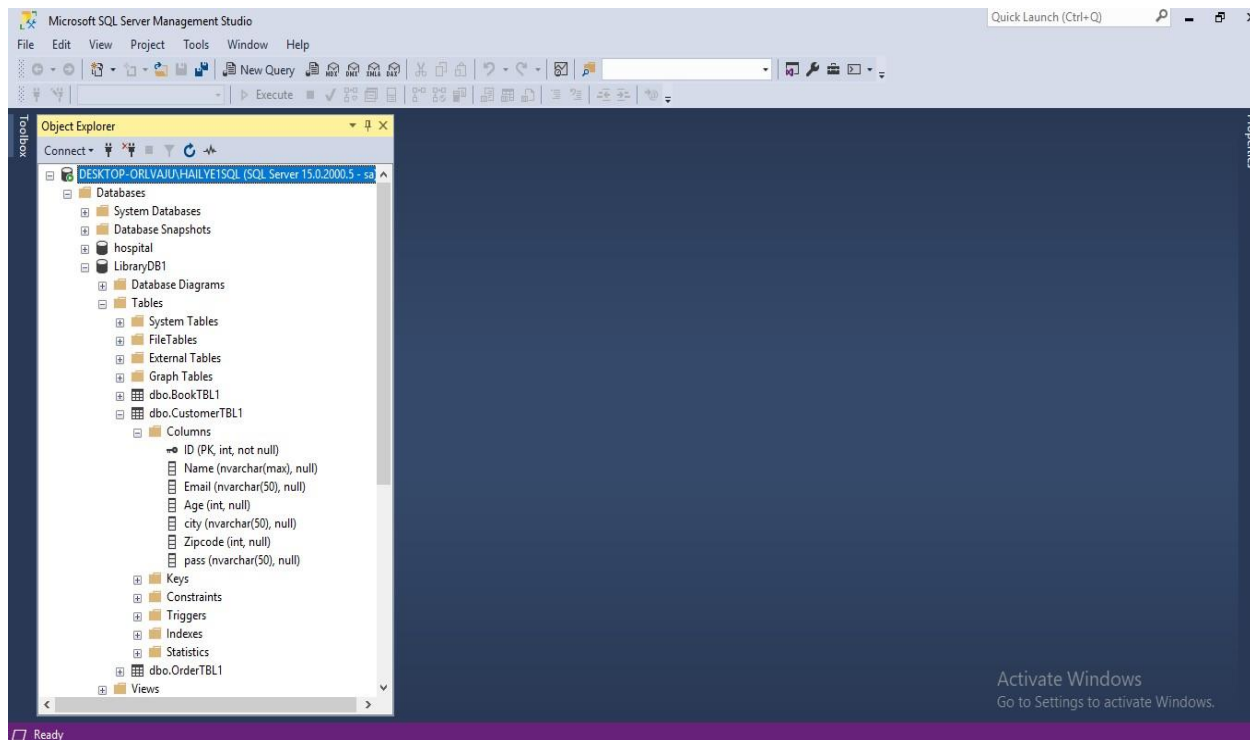
3. In the grid, add a row for each of the following entries:

Column name	Data type	Allow nulls
CustomerID	nchar(5)	False (cleared)
CompanyName	nvarchar(50)	False (cleared)
ContactName	nvarchar (50)	True (selected)
Phone	nvarchar (24)	True (selected)

4. Right-click on the `CustomerID` row, and then select **Set Primary Key**.
5. Right-click on the default row (`Id`), and then select **Delete**.
6. Name the Customers table by updating the first line in the script pane to match the following sample:

SQL	Copy
<pre>CREATE TABLE [dbo].[Customers]</pre>	

Database connection using SQL & .NET

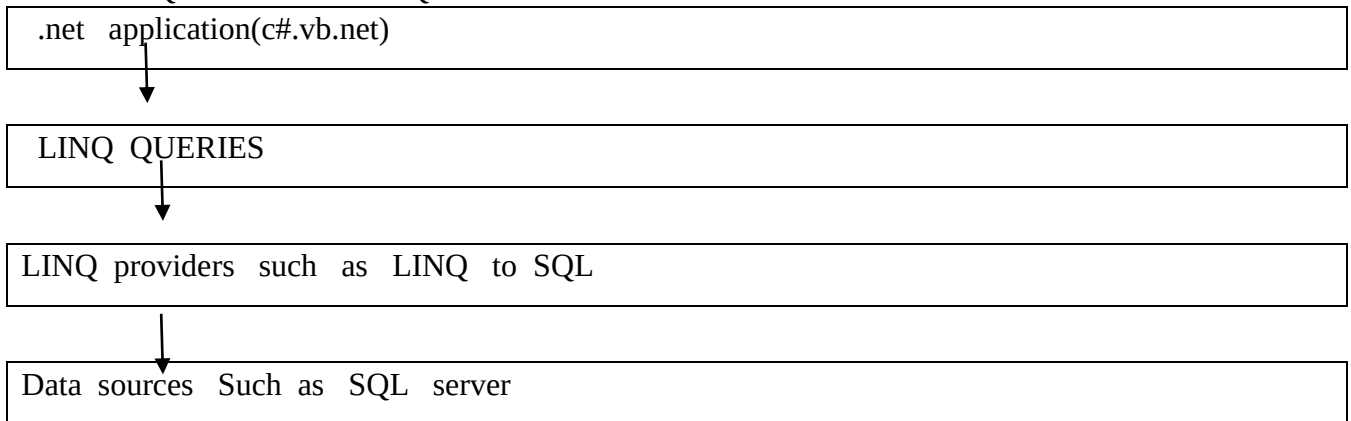


6.2. LINQ Architecture

Language Integrated Query (LINQ) is the name for a set of technologies based on the integration of query capabilities directly in to vb.net languages.

LINQ comes in two flavours the query syntax and method syntax.

How LINQ works or LINQ architecture



6.3. The .NET Data providers

7.1.1.1 Basic Code for the Data Control

- ☞ Although browsing and editing require no code, you must write code to add and delete records. Furthermore, you should add a couple lines of code in your *Form_Load* event to avoid an empty Recordset.
- ☞ A problem that has plagued VB.net from the original database version is how it handles "no current record". There are several situations where your "form" can be blank. It appears to be an empty record or a record in which you can enter data, but it isn't! There's no current record displayed. Information you type won't be saved, and trying a "delete" will crash your program. It's important to avoid those situations.
- ☞ Before VB.NET, when adding a record, if you didn't immediately *UpdateRecord*, the user could crash the program by clicking Delete. They changed that in VB.net. Now the program doesn't crash.... instead you wind up deleting some *other* record!
- ☞ Here is safe code for adding and deleting. The only issue you may have is if your database has required fields. Before you use the *UpdateRecord* command, you should assign default data to those fields. Otherwise, you must use an alternate method for adding a new record.

6.4. Working with the common .NET Data providers

A .NET framework data provider is used for connecting to a database, executing commands and retrieving results.

6.5. The Dataset Component

7.1.1.2 The Data Control's RecordSource and RecordSet Properties

The *RecordSource* tells where the data will come from. You can think of the *Recordset* property as the actual data that you've requested using the *RecordSource*.

The *RecordSource* can be the name of a table in a database, the name of an Access database query, or the text of a SQL query. If you change the *RecordSource* at run-time, you must use the method `Data1.Refresh` to actually go out and get the data and assign it to the *RecordSet*.

Once you have a *RecordSet*, you can use the following methods in your program:

Record Set Methods

`Data1.RecordSet.AddNew`

adds a new record to the recordset

`Data1.RecordSet.Delete`

deletes the current record

`Data1.RecordSet.MoveNext`

moves to the next record in the recordset

`Data1.RecordSet.MovePrevious`

moves to the previous record

`Data1.RecordSet.MoveFirst`

moves to the first record

`Data1.RecordSet.MoveLast`

moves to the last record

`Data1.UpdateRecord`

saves a changed record. This is also done automatically when you use any of the Move methods. With VB6, this is also done automatically when the control is unloaded (when your application exits).

RecordSet Properties

Data1.RecordSet.EOF

True if *MoveNext* has moved us past the last record

Data1.RecordSet.BOF

True if *MovePrevious* has moved us in front of the first record

Data1.RecordSet.RecordCount

The number of records in the *RecordSet*. This is accurate with small *RecordSets*. For large *RecordSets* you may have to perform a *MoveLast* to get this actual value.

Data1.RecordSet.AbsolutePosition

The number of the current record in the *RecordSet*. Starts at zero. This property does not appear to be reliable in V.NET. I put code to display this in the Data Control's Reposition event, and sometimes it worked, sometimes it gave odd results.

Data1.RecordSet! <i>fieldname</i>

The contents of a specific field in the current record of the *RecordSet*. Replace *fieldname* with the valid name of a field.

6.6. Using the Data Grid View for database access

Grid View

1.  Start a new project and put a Data Control on the form
2. Set the *DatabaseName* property of Data1
3. Set the *RecordSource* property
4. From the *Project* menu, choose *Components*. Check either the *Microsoft FlexGrid* control or the *Microsoft DataBound Grid* control. The *FlexGrid* is read-only, but it has features such as data-pivoting. The *DataBound Grid* control allows the user to edit its fields.
5.  Put a *DBGrid* or *FlexGrid* control on your form. Make it fairly large.
6. Set the *DataSource* property of the grid to Data1
7. Run your program

The code below assumes you have Command Buttons for Add and Delete.

```
Private Sub cmdAdd_Click()  
    ' Add a blank record and save it so it is the current record. It gets put at the end.  
    Data1.RecordSet.AddNew  
    ' Put code here if you have required fields in your database. Assign default values.  
    Data1.UpdateRecord  
    Data1.Recordset.MoveLast  
End Sub
```

```

Private Sub cmdDelete_Click()
    Data1.RecordSet.Delete
    ' Now attempt to move forward to a valid record:
    Data1.RecordSet.MoveNext
    ' But if we just deleted the Last record, we need to move back instead:
    If Data1.RecordSet.EOF Then
        Data1.RecordSet.MovePrevious
        ' Now check to see if we've moved off the front...
        ' if so, we've just deleted the ONLY record! So add a new one...
        If Data1.RecordSet.BOF Then
            Data1.RecordSet.AddNew
            Data1.UpdateRecord
            Data1.Recordset.MoveLast
        End If
    End If
End Sub

Private Sub Form_Unload()
    ' add the following to Form_Unload to make sure any last changes are saved:
    Data1.UpdateRecord
End Sub

```

- ☞ This "UpdateRecord" in the Form_Unload event may not be necessary in VB6. In earlier versions it was needed, but the VB.NET Data Control appears to automatically update the last record when you exit the program.
- ☞ If you have an Exit button, or a File menu with the Exit option (you should!), you should use the line "Unload Me" to exit your program. In earlier programs it did not matter if we used the other choice: "End". Now it does.
- ☞ The following code for *Form_Load* does that. It assumes a database file named "MYDATA.MDB". Use whatever is appropriate. The one other check made by *Form_Load* is that the *RecordSet* is not empty. If it is, a new record is added.

```

Private Sub Form_Load()
    Dim DBPath as String
    DBPath = App.Path
    If Right(DBPath,1) <> "\" Then DBPath = DBPath & "\"
    Data1.DatabaseName = DBPath & "mydata.mdb"
    Data1.Refresh

    If Data1.RecordSet.EOF Then
        Data1.RecordSet.AddNew
        Data1.UpdateRecord
        Data1.Recordset.MoveLast
    End If
End Sub

```


7.1.1.3 More Database Programming Tips

Accessing data directly, without a bound control:

```
Dim s as String
```

```
' assign the value of a ZipCode field to s:
```

```
s = Data1.Recordset!ZipCode
```

Looping through the whole recordset:

```
' Perform some operation on every record in the recordset
```

```
Data1.Recordset.MoveFirst
```

```
Do While Data1.Recordset.EOF = False
```

```
    ' Insert your code here to do whatever to this record:
```

```
Data1.Recordset.MoveNext
```

```
Loop
```

7.1.2 Brief introduction to the usages of SQL data bases

- ☞ There are many ways to work with databases in Visual Basic, and I would think you have at least glanced at the Data control. I will not even mention the Data control further in this text, since it is so easy to use and too limited to be interesting for a professional developer. (Ok, there are some exceptions to this.)
- ☞ What I will teach you to use in this text is DAO (Data Access Objects). You will get familiar with opening a database and retrieving/adding/deleting/updating records from tables.

7.1.3 Database Object

- ☞ The first thing you must do in your application is to open a database where your tables are stored. You need to declare a variable to hold your database in order to do this. This is done with:

```
Dim dbMyDB As Database
```

- ☞ This gives you a variable/object that can hold a reference to your database. To open a simple Access database named "MyDatabase.mdb", do this:

```
Set dbMyDB = OpenDatabase("MyDatabase.mdb")
```

- ☞ *You should really specify the complete path to the db, but if your current directory is the directory where the database is situated, this will work.*

- ☞ So, now you have opened a database. This won't give you any data. What you need to do is open a table in the database. You're not limited to open a single table; sometimes you have two or more tables that are related to each other and linked together with foreign keys, and there are ways to handle this to. But in this "Visual Basic - Database Primer" I will only show you how to open a single table.

7.1.4 [RecordSet Object](#)

- ☞ Visual Basic uses an object called RecordSet to hold your table. To declare such an object and to open the table, do this:

```
Dim rsMyRS As RecordSet
```

```
Set rsMyRS = dbMyDB.OpenRecordSet("MyTable", dbOpenDynaset)
```

- ☞ What happened there? Well, I declared a RecordSet object and used the Database object's OpenRecordSet method to open a table of type Dynaset.
- ☞ You can open a RecordSet in several modes. VB's online help file explains the different modes and what they are for.
- ☞ The Dynaset mode is the mode I use mostly. It gives you a RecordSet that you can add, delete and modify records in.

7.1.5 [Accessing records](#)

- ☞ Now that we have opened a table (referred to as RecordSet from now on) we want to access the records in it. The RecordSet object allows us to move in it by using the methods MoveFirst, MoveNext, MovePrevious, MoveLast (among others). I will use some of these to fill up a list box with the records of our RecordSet.
- ☞ To get this example to work, make a database (with Access) called "MyDatabase.mdb" with the table "MyTable" in it. This table should have the fields "ID" of type "Counter" that you set to be the primary key, the field "Name" of type Text and a field "Phone" of type Text. Add some records to it. Put a list box on a form and call it "lstRecords".

```
Dim dbMyDB As Database
```

```
Dim rsMyRS As RecordSet
```

```
Private Sub Form_Load()
```

```
Set dbMyDB = OpenDatabase("MyDatabase.mdb")
```

```
Set rsMyRS = dbMyDB.OpenRecordSet("MyTable", dbOpenDynaset)
```

```
If Not rsMyRS.EOF Then rsMyRS.MoveFirst
```

```
Do While Not rsMyRS.EOF
```

```
    lstRecords.AddItem rsMyRS!Name
```

```
    lstRecords.ItemData(lstRecords.NewIndex) = rsMyRS!ID
```

```
    rsMyRS.MoveNext
```

```
Loop
```

End Sub

- ☞ This will make the list box fill up with your records when the form loads. I have introduced some new concepts with this example. We have already covered the first part where we open the table. The line that says `If Not rsMyRS.EOF Then rsMyRS.Move First` tells the program to move to the first record in case there are any records at all. The EOF is a Boolean property that is true if the current record is the last. It is also true if there are no records in the Record Set.
- ☞ Then we make the program add the "Name" field of all records to the list box by adding the current records field "Name" and moving to the next record. You ask for a field of a Record Set by putting a ! Between the name of the Record Set object and the name of the field. The while loop checks to see if there are more records to add.

7.1.6 Searching the RecordSet

- ☞ Put a text box somewhere on the form and call it "txtPhone". Then copy the following code to the project.

```
Private Sub lstRecords_Click()
```

```
    rsMyRS.FindFirst "ID=" & Str(lstRecords.ItemData(lstRecords.ListIndex))  
    txtPhone.Text = rsMyRS!Phone
```

```
End Sub
```

- ☞ This will display the phone number of the selected person when clicking in the list box. It uses the Find First method of the Record Set object. This takes a string parameter that is like what is after WHERE in a SQL expression. You state the field that you want to search in (here "ID"), then the evaluation criteria (here "=") and last the value to search for (here the Item Data of the selected item in the list box).
- ☞ So what we did was to search for the record with the "ID" field value that was the same as the Item Data property of the selected item in the list box. Then we show the value of the "Phone" field in the text box.

7.1.7 Updating the Database

- ☞ You will probably want to be able to update some value of some field when doing database programming. This is done with Edit and Update. We will try to change the value of the "Phone" field by editing the text in the text box and clicking a button.
- ☞ Put a command button on the form and name it "cmdUpdate". Then copy the following code to the project.

```
Private Sub cmdUpdate_Click()
```

```
    rsMyRS.Edit  
    rsMyRS!Phone = txtPhone.Text  
    rsMyRS.Update
```

End Sub

- ☞ Could it be that simple? Yes. This changes the phone number of our selected person. Or to put it technically: This changes the value of the "Phone" field of our current record. Imagine the current record being a set of boxes, with a field in each box. The Edit method takes the lid off all of the boxes and Update puts them back on. When we write `rsMyRS!Phone = txtPhone.Text` we replace the content of the "Phone" box with the content in the text box.

7.1.8 Deleting and Adding records

7.1.9 Deleting

- ☞ Deleting records couldn't be simpler. To delete the current record you just invoke the Delete method of the Record Set object. We will put this feature in our little project. Make one more command button named "cmdDelete" and the following code will do the work of deleting our currently selected person.

```
Private Sub cmdDelete_Click()

    rsMyRS.Delete
    lstRecords.RemoveItem lstRecords.ListIndex

End Sub
```

- ☞ The first statement deletes the record and the second removes the list box entry.

7.1.10 Adding

- ☞ Adding records is much like updating, except you use AddNew instead of Edit. Let's add one more command button to our application. Let's call it...errh...let me see...yea! "cmdNew" =). Here is the code that adds a new record.

```
Private Sub cmdNew_Click()

    rsMyRS.AddNew
    rsMyRS!Name = "A New Person"
    lstRecords.AddItem rsMyRS!Name
    lstRecords.ItemData(lstRecords.NewIndex) = rsMyRS!ID
    rsMyRS!Phone = "Person's Phone Number"
    rsMyRS.Update

End Sub
```

- ☞ The Add New method takes a set of new boxes and adds them to our Record Set. We then put some new values in them and close the lids with Update. As you can see we never stated any value for "ID", but as you remember, this is a field of type "Counter" which automatically gets a

SYSTEM ANALYSIS AND DESIGN

NOTES

Department of Information Technology



Compiled by
Mesele Gebre
Revied by
Aklilu Eliays

UNIT - I

1.1 Definition of System

A **system** is an interrelated set of business procedures (or components) used within one business unit, working together for some purpose. For example, a system in the payroll department keeps track of checks, whereas an inventory system keeps track of supplies. The two systems are separate. A system has nine characteristics, seven of which are shown in the following figure. A detailed explanation of each characteristic follows, but from the figure you can see that a system exists within a larger world, an environment. A boundary separates the system from its environment. The system takes input from outside, processes it, and sends the resulting output back to its environment. The arrows in the figure show this interaction between the system and the world outside of it.

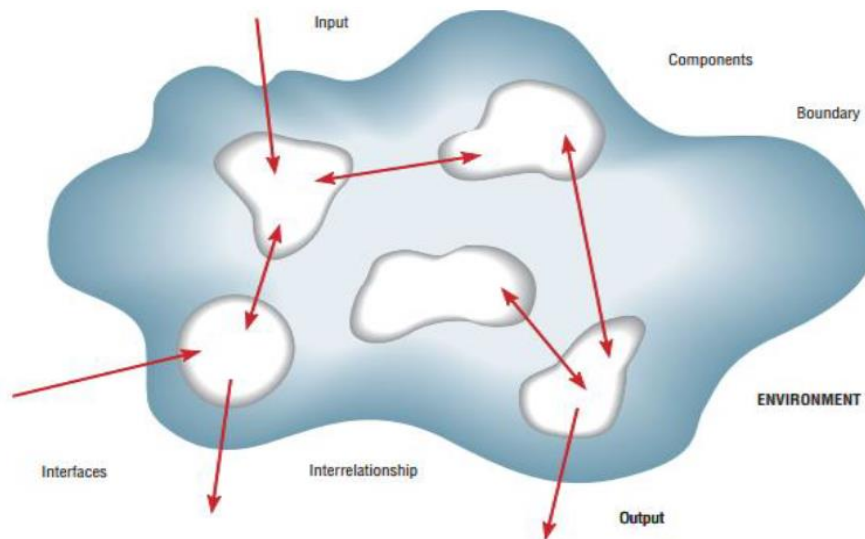


Figure 1: System Characteristics

The nine system characteristics include:

- Components
- Interrelated components

- Boundary
- Purpose
- Environment
- Interfaces
- Constraints
- Input
- Output

A system is made up of components. A component is either an irreducible part or an aggregate of parts, also called a subsystem. The simple concept of a component is very powerful. For example, just as with an automobile or a stereo system, with proper design, we can repair or upgrade the system by changing individual components without having to make changes throughout the entire system. The components are interrelated; that is, the function of one is somehow tied to the functions of the others. For example, the work of one component, such as producing a daily report of customer orders received, may not progress successfully until the work of another component is finished, such as sorting customer orders by date of receipt. A system has a boundary, within which all of its components are contained and which establishes the limits of a system, separating it from other systems. Components within the boundary can be changed, whereas systems outside the boundary cannot be changed. All of the components work together to achieve some overall purpose for the larger system: the system's reason for existing.

A system exists within an environment—everything outside the system's boundary that influences the system. For example, the environment of a state university includes prospective students, foundations and funding agencies, and the news media. Usually the system interacts with its environment. A university interacts with prospective students by having open houses and recruiting from local high schools. An information system interacts with its environment by receiving data (raw facts) and information (data processed in a useful format). The points at which the system meets its environment are called interfaces; an interface also occurs between subsystems.

In its functioning, a system must face constraints—the limits (in terms of capacity, speed, or capabilities) to what it can do and how it can achieve its purpose within its environment. Some of these constraints are imposed inside the system (e.g., a limited number of staff available), and others are imposed by the environment (e.g., due dates or regulations).

A system takes input from its environment in order to function. People, for example, take in food, oxygen, and water from the environment as input. You are constrained from breathing fresh air if you're in an elevator with someone who is smoking.

Finally, a system returns output to its environment as a result of its functioning and thus achieves its purpose. The system is constrained if electrical power is cut.

Systems analysts need to know several other important systems concepts such as:

- Decomposition
- Modularity

- Coupling
- Cohesion

Decomposition is the process of breaking down a system into its smaller components. These components may themselves be systems (subsystems) and can be broken down into their components as well. How does decomposition aid understanding of a system? It results in smaller and less complex pieces that are easier to understand than larger, complicated pieces. Decomposing a system also allows us to focus on one particular part of a system, making it easier to think of how to modify that one part independently of the entire system. Decomposition is a technique that allows the systems analyst to:

- Break a system into small, manageable, and understandable subsystems
- Focus attention on one area (subsystem) at a time, without interference from other areas
- Concentrate on the part of the system pertinent to a particular group of users, without confusing users with unnecessary details
- Build different parts of the system at independent times and have the help of different analysts

The following figure shows the decomposition of a portable MP3 player. Decomposing the system into subsystems reveals the system's inner workings. You can decompose an MP3 player into at least three separate physical subsystems.

One subsystem, the battery, supplies the power for the entire system to operate. A second physical subsystem, the storage system, is made up of a hard drive that stores thousands of MP3 recordings.

The third subsystem, the control subsystem, consists of a printed circuit board (PCB), with various chips attached, that controls the entire recording, playback, and access functions. Breaking the subsystems down into their components reveals even more about the inner workings of the system and greatly enhances our understanding of how the overall system works.

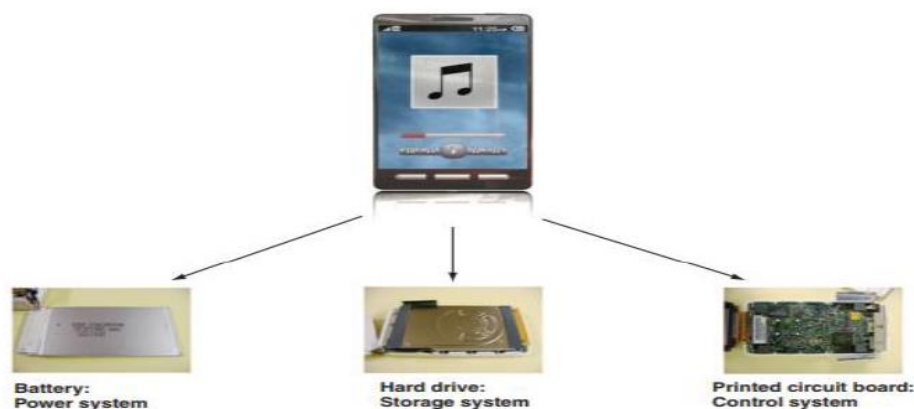


Figure 2: System Decomposition

Modularity is a direct result of decomposition. It refers to dividing a system into chunks or modules of a relatively uniform size. Modules can represent a system simply, making it easier to understand and easier to redesign and rebuild. For example, each of the separate subsystem modules for the MP3 player in the above figure shows how decomposition makes it easier to understand the overall system.

Coupling means that subsystems are dependent on each other. Subsystems should be as independent as possible. If one subsystem fails and other subsystems are highly dependent on it, the others will either fail themselves or have problems functioning. Looking at the above figure, we would say the components of a portable MP3 player are tightly coupled. The best example is the control system, made up of the printed circuit board and its chips. Every function the MP3 player can perform is enabled by the board and the chips. A failure in one part of the circuit board would typically lead to replacing the entire board rather than attempting to isolate the problem on the board and fix it. Even though repairing a circuit board in an MP3 player is certainly possible, it is typically not cost-effective; the cost of the labor expended to diagnose and fix the problem may be worth more than the value of the circuit board itself.

In a home stereo system, the components are loosely coupled because the subsystems, such as the speakers, the amplifier, the receiver, and the CD player, are all physically separate and function independently. If the amplifier in a home stereo system fails, only the amplifier needs to be repaired.

Cohesion is the extent to which a subsystem performs a single function. In the MP3 player example, supplying power is a single function.

What is System Analysis and Design?

System analysis and design is a method used by companies ranging from IBM to PepsiCo to Sony to create and maintain information systems that perform basic business functions such as keeping track of customer names and addresses, processing orders, and paying employees. The main goal of systems analysis and design is to improve organizational systems, typically through applying software that can help employees accomplish key business tasks more easily and efficiently. As a systems analyst, you will be at the center of developing this software. The analysis and design of information systems are based on:

- Your understanding of the organization's objectives, structure, and processes
- Your knowledge of how to exploit information technology for advantage

1.2 System Development Life Cycle

Organizations use a standard set of steps, called a systems development methodology, to develop and support their information systems. Like many processes, the development of information systems often follows a life cycle. For example, a commercial product, such as a Nike sneaker or a Honda car, follows a life cycle: It is created, tested, and introduced to the market. Its sales increase, peak, and decline. Finally, the product is removed from the market and is replaced by something else.

The system development life cycle (SDLC) is a common methodology for systems development in many organizations. It marks the phases or steps of information systems development: Someone has an idea for an information system and what it should do. The organization that will use the system decides to devote the necessary resources to acquiring it. A careful study is done of how the organization currently handles the work the system will support. Professionals develop a strategy for designing the new system, which is then either built or purchased.

Once complete, the system is installed in the organization, and after proper training, the users begin to incorporate the new system into their daily work. Every organization uses a slightly different life-cycle to model these steps, with anywhere from three to almost twenty identifiable phases. In this module, we highlight five SDLC steps:

- Planning and selection
- Analysis
- Design
- Implementation
- Maintenance (see the figure).

Although any life cycle appears at first glance to be a sequentially ordered set of phases, it actually is not. The specific steps and their sequence are meant to be adapted as required for a project. For example, in any given SDLC phase, the project can return to an earlier phase, if necessary. Similarly, if a commercial product does not perform well just after its introduction, it may be temporarily removed from the market and improved before being reintroduced. In the systems development life cycle, it is also possible to complete some activities in one phase in parallel with some activities of another phase. Sometimes the life cycle is iterative; that is, phases are repeated as required until an acceptable system is found. Some systems analysts consider the life cycle to be a spiral, in which we constantly cycle through the phases at different levels of detail.

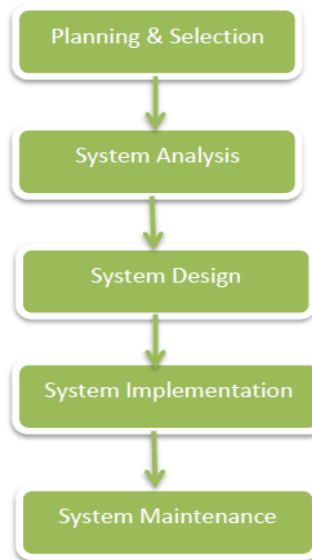


Figure 3: SDLC Phases

Phase 1: System Planning and Selection

The first phase in the SDLC, system planning and selection has two primary activities. First, someone identifies the need for a new or enhanced system. Information needs of the organization are examined, and projects to meet these needs are identified. The organization's information system needs may result from:

- ⌘ Requests to deal with problems in current procedures
- ⌘ The desire to perform additional tasks
- ⌘ The realization that information technology could be used to capitalize on an existing opportunity

The systems analyst prioritizes and translates the needs into a written plan for the information systems department, including a schedule for developing new major systems. Requests for new systems spring from users who need new or enhanced systems. During the systems planning and selection phase, an organization determines whether resources should be devoted to the development or enhancement of each information system under consideration. A feasibility study is conducted before the second phase of the SDLC to determine the economic and organizational impact of the system.

The second task in the systems planning and selection phase is to investigate the system and determine the proposed system's scope. The team of systems analysts then produces a specific plan for the proposed project for the team to follow.

This baseline project plan customizes the standardized SDLC and specifies the time and resources needed for its execution. The formal definition of a project is based on the likelihood that the organization is able to develop a system that will solve the problem or exploit the opportunity and determine whether the costs of developing the system outweigh the possible benefits. The final presentation to the organization's management of the plan for proceeding with the subsequent project phases is usually made by the project leader and other team members.

Phase 2: System Analysis

The second phase of the systems development life cycle is system analysis. During this phase, the analyst thoroughly studies the organization's current procedures and the information systems used to perform tasks such as general ledger, shipping, order entry, machine scheduling, and payroll. Analysis has several subphases. The first subphase involves determining the requirements of the system. In this subphase, you and other analysts work with users to determine what the users want from a proposed system. This subphase involves a careful study of any current systems, manual and computerized, that might be replaced or enhanced as part of this project. Next, you study the requirements and structure them according to their interrelationships, eliminating any redundancies.

As part of structuring, you generate alternative initial designs to match the requirements. Then you compare these alternatives to determine which best meets the requirements within the cost, labor, and technical levels the organization is willing to commit to the development process. The output of the analysis phase is a description of the alternative solution recommended by the analysis team. Once the recommendation is accepted by the organization, you can make plans to acquire any hardware and system software necessary to build or operate the system as proposed.

Phase 3: System Design

The third phase of the SDLC is called system design. During systems design, analysts convert the description of the recommended alternative solution into logical and then physical system specifications. You must design all aspects of the system from input and output screens to reports, databases, and computer processes.

Logical design is not tied to any specific hardware and systems software platform. Theoretically, the system you design could be implemented on any hardware and systems software. Logical design concentrates on the business aspects of the system; that is, how the system will impact the functional units within the organization.

In physical design, you turn the logical design into physical, or technical, specifications. For example, you must convert diagrams that map the origin, flow, and processing of data in a system into a structured systems design that can then be broken down into smaller and smaller units for conversion to instructions written in a programming language.

You design the various parts of the system to perform the physical operations necessary to facilitate data capture, processing, and information output. During physical design, the analyst team decides which programming languages the computer instructions will be written in, which database systems and file structures will be used for the data, and which hardware platform, operating system, and network environment the system will run under. These decisions finalize the hardware and software plans initiated at the end of the analysis phase. Now you can acquire any new technology not already present in the organization. The final product of the design phase is the physical system specifications, presented in a form, such as a diagram or written report, ready to be turned over to programmers and other system builders for construction.

Phase 4: System Implementation

The fourth phase of the SDLC is a two-step process: systems implementation and operation. During systems implementation and operation, you turn system specifications into a working system that is tested and then put into use. Implementation includes coding, testing, and installation. During coding, programmers write the programs that make up the system. During testing, programmers and analysts test individual programs and the entire system in order to find and correct errors.

During installation, the new system becomes a part of the daily activities of the organization. Application software is installed, or loaded, on existing or new hardware; then users are introduced to the new system and trained. Begin planning for both testing and installation as early as the project planning and selection phase, because they both require extensive analysis in order to develop exactly the right approach. Systems implementation activities also include initial user support such as the finalization of documentation, training programs, and ongoing user assistance.

Note that documentation and training programs are finalized during implementation; documentation is produced throughout the life cycle, and training (and education) occurs from the inception of a project. Systems implementation can continue for as long as the system exists because ongoing user support is also part of implementation. Despite the best efforts of analysts, managers, and programmers, however, installation is not always a simple process. Many well-designed systems have failed because the installation process was faulty. Note that even a well-designed system can fail if implementation is not well managed.

The second part of the fourth phase of the SDLC is operation. While a system is operating in an organization, users sometimes find problems with how it works and often think of improvements. During operation, programmers make the changes that users ask for and modify the system to reflect changing business conditions. These changes are necessary to keep the system running and useful. The amount of time and effort devoted to system enhancements during operation depends a great deal on the performance of the previous phases of the life cycle. Inevitably, the time comes when an information system is no longer performing as desired, when the costs of keeping a system running become prohibitive, or when an organization's needs have changed substantially. Such problems indicate that it is time to begin designing the system's replacement, thereby completing the loop and starting the life cycle over again.

The SDLC is a highly linked set of phases whose products feed the activities in subsequent phases. The following table summarizes the outputs or products of each phase based on the preceding descriptions. The subsequent sections on the SDLC phases discuss the products of each phase and how they are developed.

Phase	Products, Outputs, or Deliverables
Systems planning and selection	Priorities for systems and projects
	Architecture for data, networks, hardware, and IS management
	Detailed work plan for selected project
	Specification of system scope
Systems analysis	System justification or business case
	Description of current system
	General recommendation on how to fix, enhance, or replace current system
	Explanation of alternative systems and justification for chosen alternative
Systems design	Acquisition plan for new technology
	Detailed specifications of all system elements
Systems implementation and operation	Code
	Documentation
	Training procedures and support capabilities
	New versions or releases of software with associated updates to documentation, training, and support

Table 1: Outputs of SDLC Phases

Throughout the systems development life cycle, the systems development project itself needs to be carefully planned and managed. The larger the systems project, the greater the need for project management.

Phase 5: System Maintenance (The students shall discuss as a class activity, refer chapter 3)

1.3 Alternative Approaches to System Development

Prototyping, computer-aided software engineering (CASE) tools, joint application design (JAD), rapid application development (RAD), participatory design (PD), and the use of Agile Methodologies represent different approaches that streamline and improve the systems analysis and design process from different perspectives.

1.3.1 Prototyping

Designing and building a scaled-down but working version of a desired system is known as prototyping. A prototype can be developed with a CASE tool, a software product that automates steps in the systems development life cycle. CASE tools make prototyping easier and more creative by supporting the design of screens and reports and other parts of a system interface. CASE tools also support many of the diagramming techniques you will learn, such as data-flow diagrams and entity-relationship diagrams.

The analyst works with users to determine the initial or basic requirements for the system. The analyst then quickly builds a prototype.

When the prototype is completed, the users work with it and tell the analyst what they like and do not like about it. The analyst uses this feedback to improve the prototype and takes the new version back to the users. This iterative process continues until the users are relatively satisfied with what they have seen. The key advantages of the prototyping technique are: (1) it involves

the user in analysis and design, and (2) it captures requirements in concrete, rather than verbal or abstract, form. The following figure illustrates prototyping.

1.3.2 Computer-Aided Software Engineering (CASE) Tools

Computer-aided software engineering (CASE) refers to automated software tools used by systems analysts to develop information systems. These tools can be used to automate or support activities throughout the systems development process with the objective of increasing productivity and improving the overall quality of systems. CASE helps provide an engineering-type discipline to software development and to the automation of the entire software life-cycle process, sometimes with a single family of integrated software tools.

In general, CASE assists systems builders in managing the complexities of information system projects and helps ensure that high-quality systems are constructed on time and within budget.

The general types of CASE tools include:

- Diagramming tools that enable system process, data, and control structures to be represented graphically.
- Computer display and report generators that help prototype how systems look and feel to users.
- Display (or form) and report generators also make it easier for the systems analyst to identify data requirements and relationships.
- Analysis tools that automatically check for incomplete, inconsistent, or incorrect specifications in diagrams, forms, and reports.
- A central repository that enables the integrated storage of specifications, diagrams, reports, and project management information.
- Documentation generators that help produce both technical and user documentation in standard formats.
- Code generators that enable the automatic generation of program and database definition code directly from the design documents, diagrams, forms, and reports.

1.3.3 Joint Application Design

In the late 1970s, systems development personnel at IBM developed a new process for collecting information system requirements and reviewing system designs. The process is called joint application design (JAD). The idea behind JAD is to structure the requirements determination phase of analysis and the reviews that occur as part of the design. Users, managers, and systems developers are brought together for a series of intensive structured meetings run by a JAD session leader. By gathering the people directly affected by an IS in one room at the same time to work together to agree on system requirements and design details, time and organizational resources are better managed. Group members are more likely to develop a shared understanding of what the IS is supposed to do.

1.3.4 Rapid Application Development

Prototyping, CASE, and JAD are key tools that support rapid application development (RAD).

The fundamental principle of any RAD methodology is to delay producing detailed system design documents until after user requirements are clear. The prototype serves as the working description of needs. RAD involves gaining user acceptance of the interface and developing key system capabilities as quickly as possible. RAD is widely used by consulting firms. It is also used as an in-house methodology by firms such as the Boeing Company. RAD sacrifices computer efficiency for gains in human efficiency in rapidly building and rebuilding working systems.

On the other hand, RAD methodologies can overlook important systems development principles, which may result in problems with systems developed this way.

RAD grew out of the convergence of two trends: the increased speed and turbulence of doing business in the late 1980s and early 1990s, and the ready availability of high-powered computer-based tools to support systems development and easy maintenance. As the conditions of doing business in a changing, competitive global environment became more turbulent, management in many organizations began to question whether it made sense to wait two to three years to develop systems that would be obsolete upon completion. On the other hand, CASE tools and prototyping software were diffusing throughout organizations, making it relatively easy for end users to see what their systems would look like before they were completed. Why not use these tools to address the problems of developing systems more productively in a rapidly changing business environment? So RAD was born. The same phases followed in the SDLC are also followed in RAD, but the phases are combined to produce a more streamlined development technique. Planning and design phases in RAD are shortened by focusing work on system functional and user interface requirements at the expense of detailed business analysis and concern for system performance issues. Also, usually RAD looks at the system being developed in isolation from other systems, thus eliminating the time-consuming activities of coordinating with existing standards and systems during design and development. The emphasis in RAD is generally less on the sequence and structure of processes in the life cycle and more on doing different tasks in parallel with each other and on using prototyping extensively. Notice also, that the iteration in the RAD life cycle is limited to the design and development phases, which is where the bulk of the work in a RAD approach takes place.

1.3.5 Agile Methodologies

Many other approaches to systems analysis and design have been developed over the years. These approaches include extreme Programming, the Crystal family of methodologies, Adaptive Software Development, Scrum, and Feature Driven Development.

Agile Methodologies share three key principles: (1) a focus on adaptive rather than predictive methodologies, (2) a focus on people rather than roles, and (3) a self-adaptive process. Adopting an adaptive rather than predictive methodology refers to the observation that engineering based methodologies work best when the process and product are predictive.

Software tends not to be as predictive as, say, a bridge, especially in today's turbulent business environment.

More adaptive methodologies are needed, then, and the Agile Methodologies are based on the ability to adapt quickly. The focus on people rather than roles is also a criticism of engineering-based techniques, where people became interchangeable.

An Agile approach views people as talented individuals, not people filling roles, each of whom has unique talents to bring to a development project. Finally, Agile Methodologies promote a self-adaptive software development process. As the methodologies are applied, they should also be adapted by a particular development team working on a particular project in a particular context. No single monolithic methodology effectively fits all developers on all projects at all times.

1.4 Managing Projects

Project management is an important aspect of the development of information systems and a critical skill for a systems analyst. The focus of project management is to ensure that system development projects meet customer expectations and are delivered within budget and time constraints.

The project manager is a systems analyst with a diverse set of skills— management, leadership, technical, conflict management, and customer relationship—who is responsible for initiating, planning, executing, and closing down a project. Creating and implementing successful projects requires managing resources, activities, and tasks needed to complete the information systems project. A project is a planned undertaking of a series of related activities, having a beginning and an end, to reach an objective.

In general, systems development projects are undertaken for two primary reasons: to take advantage of business opportunities and to solve business problems. Taking advantage of an opportunity might mean providing an innovative service to customers through the creation of a new system. Solving a business problem could involve modifying how an existing system processes data so that more accurate or timely information is provided to users.

Once a potential project has been identified, an organization must determine the resources required for its completion by analyzing the scope of the project and determining the probability of successful completion.

After getting this information, the organization can then determine whether taking advantage of an opportunity or solving a particular problem is feasible within time and resource constraints. If deemed feasible, a more detailed project analysis is then conducted.

Project management process, involves four phases:

- Initiating the project
- Planning the project
- Executing the project
- Closing down the project

1.4.1 Initiating the Project

During project initiation the project manager performs several activities that assess the size,

scope, and complexity of the project, and establishes procedures to support subsequent activities. Depending on the project, some initiation activities may be unnecessary and some may be more involved. The types of activities you will perform when initiating a project are summarized in the following table.

<i>Project Initiation</i>	
1.	Establishing the Project Initiation Team
2.	Establishing a Relationship with the Customer
3.	Establishing the Project Initiation Plan
4.	Establishing Management Procedures
5.	Establishing the Project Management Environment and Project Workbook
6.	Developing the Project Charter

Table 2: Project Initiation Activities

1. Establishing the project initiation team: This activity involves organizing an initial core of project team members to assist in accomplishing the project initiation activities.
2. Establishing a relationship with the customer: A thorough understanding of your customer builds stronger partnerships and higher levels of trust.
3. Establishing the project initiation plan: This step defines the activities required to organize the initiation team while it is working to define the scope of the project.
4. Establishing management procedures: Successful projects require the development of effective management procedures.
5. Establishing the project management environment and project workbook: The focus of this activity is to collect and organize the tools that you will use while managing the project and to construct the project workbook.
For example, most diagrams, charts, and system descriptions provide much of the project workbook contents. Thus, the project workbook serves as a repository for all project correspondence, inputs, outputs, deliverables, procedures, and standards established by the project team.
6. Developing the project charter: The project charter is a short (typically one page), high-level document prepared for the customer that describes what the project will deliver and outlines many of the key elements of the project. A project charter can vary in the amount of detail it contains, but often includes the following elements:
 - ⌘ Project title and date of authorization
 - ⌘ Project manager name and contact information
 - ⌘ Customer name and contact information
 - ⌘ Project start and completion dates
 - ⌘ Project description and objectives
 - ⌘ Key assumptions or approach
 - ⌘ Key stakeholders, roles, responsibilities and signatures

The project charter ensures that both you and your customer gain a common understanding of the project. It is also a useful communication tool; it helps to announce to the organization that a particular project has been chosen for development. Project initiation is complete once these six activities have been performed. Before moving on to the next phase of the project, the work performed during project initiation is reviewed at a meeting attended by management, customers, and project team members. An outcome of this meeting is a decision to continue the project, modify it, or abandon it.

1.4.2 Planning the Project

The next step in the project management process is project planning. Project planning involves defining clear, discrete activities and the work needed to complete each activity within a single project. It often requires you to make numerous assumptions about the availability of resources such as hardware, software, and personnel. It is much easier to plan nearer-term activities than those occurring in the longer term.

In actual fact, you often have to construct longer-term plans that are more general in scope and nearer-term plans that are more detailed.

The repetitive nature of the project management process requires that plans be constantly monitored throughout the project and periodically updated (usually after each phase) based upon the most recent information. As with the project initiation process, varied and numerous activities must be performed during project planning. The types of activities that you can perform during project planning are summarized in the table below.

Project Planning	
1.	Describing Project Scope, Alternatives, and Feasibility
2.	Dividing the Project into Manageable Tasks
3.	Estimating Resources and Creating a Resource Plan
4.	Developing a Preliminary Schedule
5.	Developing a Communication Plan
6.	Determining Project Standards and Procedures
7.	Identifying and Assessing Risk
8.	Creating a Preliminary Budget
9.	Developing a Project Scope Statement
10.	Setting a Baseline Project Plan

Table 3: Project Planning Activities

1. Describing project scope, alternatives, and feasibility: The purpose of this activity is to understand the content and complexity of the project. During this activity, you should reach agreement on the following questions:
 - What problem or opportunity does the project address?
 - What are the quantifiable results to be achieved?

- What needs to be done?
- How will success be measured?
- How will we know when we are finished?

After defining the scope of the project, your next objective is to identify and document general alternative solutions for the current business problem or opportunity. You must then assess the feasibility of each alternative solution and choose which to consider during subsequent SDLC phases. In some instances, off-the-shelf software can be found.

It is also important that any unique problems, constraints, and assumptions about the project be clearly stated.

2. Dividing the project into manageable tasks: This activity is critical during the project planning process. Here, you must divide the entire project into manageable tasks and then logically order them to ensure a smooth evolution between tasks. The definition of tasks and their sequence is referred to as the work breakdown structure (WBS). Some tasks may be performed in parallel, whereas others must follow one another sequentially. Task sequence depends on which tasks produce deliverables needed in other tasks, when critical resources are available, the constraints placed on the project by the client, and the process outlined in the SDLC.

For example, suppose that you are working on a new development project and need to collect system requirements by interviewing users of the new system and reviewing reports they currently use to do their job. A work breakdown for these activities is represented in a Gantt chart in table below. A Gantt chart is a graphical representation of a project that shows each task as a horizontal bar whose length is proportional to its time for completion. Different colors, shades, or shapes can be used to highlight each kind of task.

Gantt charts do not show how tasks must be ordered (precedence) but simply show when an activity should begin and end. In the table, the task duration is shown in the second column, in days, and necessary prior tasks are noted in the third column as predecessors. Most project management software tools support a broad range of task durations, including minutes, hours, days, weeks, and months. The SDLC consists of several phases, which you need to break down into activities. Creating a work breakdown structure requires that you decompose phases into activities—summary tasks—and activities into specific tasks. For example, the following table shows that the activity Interviewing consists of three tasks: design interview form, schedule appointments, and conduct interviews. Defining tasks in too much detail will make the management of the project unnecessarily complex.

What are the characteristics of a task?

A task:

- Can be done by one person or a well-defined group
- Has a single and identifiable deliverable (the task, however, is the process of creating the deliverable)

- Has a known method or technique
- Has well-accepted predecessor and successor steps
- Is measurable so that percent completed can be determined

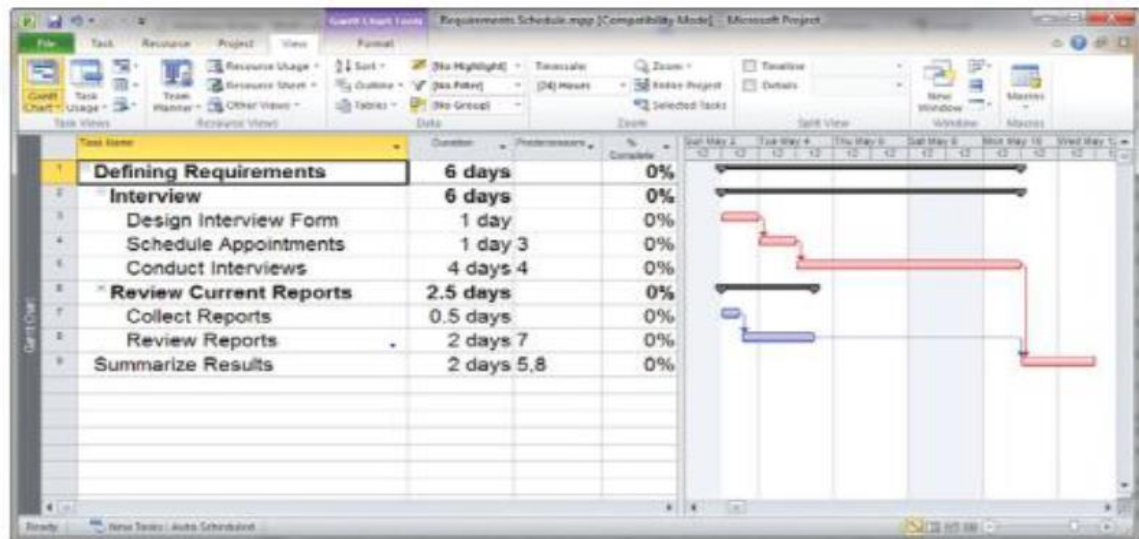


Figure 5: Gantt Chart

3. Estimating resources and creating a resource plan: The goal of this activity is to estimate resource requirements for each project activity and use this information to create a project resource plan. The resource plan helps assemble and deploy resources in the most effective manner. For example, you would not want to bring additional programmers onto the project at a rate faster than you could prepare work for them. Project managers use a variety of tools to assist in making estimates of project size and costs. The most widely used method is called COCOMO (CONstructive COst MOdel), which uses parameters that were derived from prior projects of differing complexity.

COCOMO uses these different parameters to predict human resource requirements for basic, intermediate, and complex systems. People are the most important and expensive part of project resource planning.

Project time estimates for task completion and overall system quality are significantly influenced by the assignment of people to tasks. It is important to give people tasks that allow them to learn new skills. It is equally important to make sure that project members are not in —over their heads or working on a task that is not well suited to their skills. Resource estimates may need to be revised based upon the skills of the actual person (or people) assigned to a particular activity. One approach to assigning tasks is to assign a single task type (or only a few task types) to each worker for the duration of the project. For example, you could assign one worker to create all computer displays and another to create all system reports. Such specialization ensures that both workers become efficient at their own particular tasks. A worker may become bored if the task is too specialized or is long in duration, so you could assign workers to a wider variety of tasks. However, this approach may lead to lowered task efficiency. A middle ground would be to make assignments with a balance of both specialization and task variety. Assignments depend upon the

size of the development project and the skills of the project team. Regardless of the manner in which you assign tasks, make sure that each team member works only on one task at a time.

4. Developing a preliminary schedule: During this activity, you use the information on tasks and resource availability to assign time estimates to each activity in the work breakdown structure. These time estimates will allow you to create target starting and ending dates for the project. Target dates can be revisited and modified until a schedule produced is acceptable to the customer. Determining an acceptable schedule may require that you find additional or different resources or that the scope of the project be changed. The schedule may be represented as a Gantt chart, or as a Network diagram. A Network diagram is a graphical depiction of project tasks and their interrelationships. As with a Gantt chart, each type of task can be highlighted by different features on the Network diagram.

The distinguishing feature of a Network diagram is that the ordering of tasks is shown by connecting tasks—depicted as rectangles or ovals—with its predecessor and successor tasks.

5. Developing a communication plan: The goal of this activity is to outline the communication procedures among management, project team members, and the customer.

The communication plan includes when and how written and oral reports will be provided by the team, how team members will coordinate work, what messages will be sent to announce the project to interested parties, and what kinds of information will be shared with vendors and external contractors involved with the project. It is important that free and open communication occurs among all parties, with respect for proprietary information and confidentiality with the customer.

6. Determining project standards and procedures: During this activity, you specify how various deliverables are produced and tested by you and your project team. For example, the team must decide on which tools to use, how the standard SDLC might be modified, which SDLC methods will be used, documentation styles (e.g., type fonts and margins for user manuals), how team members will report the status of their assigned activities, and terminology. Setting project standards and procedures for work acceptance is a way to ensure the development of a high-quality system. Also, it is much easier to train new team members when clear standards are in place. Organizational standards for project management and conduct make the determination of individual project standards easier and the interchange or sharing of personnel among different projects feasible.
7. Identifying and assessing risk: The goal of this activity is to identify sources of project risk and to estimate the consequences of those risks. Risks might arise from the use of new technology, prospective users' resistance to change, availability of critical resources, competitive reactions or changes in regulatory actions due to the construction of a system, or team member inexperience with technology or the business area. You should continually try to identify and assess project risk.
8. Creating a preliminary budget: During this phase, you need to create a preliminary budget that outlines the planned expenses and revenues associated with your project. The project justification will demonstrate that the benefits are worth these costs. This analysis shows

net present value calculations of the project's benefits and costs, as well as a return on investment and cash flow analysis.

9. Developing a project scope statement: An important activity that occurs near the end of the project planning phase is the development of the project scope statement. Developed primarily for the customer, this document outlines work that will be done and clearly describes what the project will deliver.

The project scope statement is useful to make sure that you, the customer, and other project team members have a clear understanding of the intended project size, duration, and outcomes.

10. Setting a baseline project plan: Once the entire prior project planning activities has been completed, you will be able to develop a baseline project plan. This baseline plan provides an estimate of the project's tasks and resource requirements and is used to guide the next project phase— execution. As new information is acquired during project execution, the baseline plan will continue to be updated. At the end of the project planning phase, a review of the baseline project plan is conducted to double-check all the information in the plan. As with the project initiation phase, it may be necessary to modify the plan, which means returning to prior project planning activities before proceeding.

1.4.3 Executing the Project

Project execution puts the baseline project plan into action. Within the context of the SDLC, project execution occurs primarily during the analysis, design, and implementation phases. These activities are summarized in the following table.

Project Execution	
1.	Executing the Baseline Project Plan
2.	Monitoring Project Progress Against the Baseline Project Plan
3.	Managing Changes to the Baseline Project Plan
4.	Maintaining the Project Workbook
5.	Communicating the Project Status

Table 4: Project Execution Activities

1. Executing the baseline project plan: As project manager, you oversee the execution of the baseline plan: You initiate the execution of project activities, acquire and assign resources; orient and train new team members, keep the project on schedule, and ensure the quality of project deliverables. This formidable task is made much easier through the use of sound project management techniques.

For example, as tasks are completed during a project, they can be marked as completed on the project schedule. You are responsible for initiating new team members by

providing them with the resources they need and helping them assimilate into the team. You may want to plan social events, regular team project status meetings, team-level reviews of project deliverables, and other group events to mold the group into an effective team.

2. **Monitoring project progress against the baseline project plan:** While you execute the baseline project plan, you should monitor your progress.
If the project gets ahead of (or behind) schedule, you may have to adjust resources, activities, and budgets. Monitoring project activities can result in modifications to the current plan. Measuring the time and effort expended on each activity helps you improve the accuracy of estimations for future projects. It is possible with project schedule charts, like Gantt, to show progress against a plan; and it is easy with Network diagrams to understand the ramifications of delays in an activity. Monitoring progress also means that the team leader must evaluate and appraise each team member, occasionally change work assignments or request changes in personnel, and provide feedback to the employee's supervisor.
3. **Managing changes to the baseline project plan:** You will encounter pressure to make changes to the baseline plan. The request to make changes should explain why changes are desired and describe all possible impacts on prior and subsequent activities, project resources, and the overall project schedule.
4. **Maintaining the project workbook:** As in all project phases, maintaining complete records of all project events is necessary. The workbook provides the documentation new team members require to assimilate project tasks quickly. It explains why design decisions were made and is a primary source of information for producing all project reports.
5. **Communicating the project status:** The project manager is responsible for keeping all team members—system developers, managers, and customers—abreast of the project status. Clear communication is required to create a shared understanding of the activities and goals of the project; such an understanding ensures better coordination of activities. This means that the entire project plan should be shared with the entire project team, and any revisions to the plan should be communicated to all interested parties so that everyone understands how the plan is evolving. Procedures for communicating project activities vary from formal meetings to informal hallway discussions. Some procedures are useful for informing others of project status, others for resolving issues, and others for keeping permanent records of information and events.
Two types of information are routinely exchanged throughout the project: (1) work results, or the outcomes of the various tasks and activities that are performed to complete the project, and (2) the project plan, which is the formal comprehensive document used to execute the project. The project plan contains numerous items including the project charter, project schedule, budgets, risk plan, and so on.
The ease with which the project can be managed is significantly influenced by the quality of prior project phases. If you develop a high-quality project plan, it is much more likely that the project will be successfully executed.

1.4.4 Closing down the Project

The focus of project closedown is to bring the project to an end. Projects can conclude with a natural or unnatural termination. A natural termination occurs when the requirements of the project have been met—the project has been completed and is a success. An unnatural

termination occurs when the project is stopped before completion. Several events can cause an unnatural termination of a project. For example, it may be learned that the assumption used to guide the project proved to be false, or that the performance of the system or development group was somehow inadequate, or that the requirements are no longer relevant or valid in the customer's business environment. The most likely reasons for the unnatural termination of a project relate to running out of time or money, or both. Regardless of the project termination outcome, several activities must be performed: closing down the project, conducting post project reviews, and closing the customer contract. Within the context of the SDLC, project closedown occurs after the implementation phase. The system maintenance phase typically represents an ongoing series of projects, each needing to be individually managed. The following table summarizes the project closedown activities.

<i>Project Closedown</i>	
1.	Closing Down the Project
2.	Conducting Postproject Reviews
3.	Closing the Customer Contract

Table 5: Project Closedown Activities

1. Closing down the project: During closedown, you perform several diverse activities. For example, if you have several team members working with you, project completion may signify job and assignment changes for some members. You will likely be required to assess each team member and provide an appraisal for personnel files and salary determination. You may also want to provide career advice to team members, write letters to superiors praising special accomplishments of team members, and send thank-you letters to those who helped but were not team members. As project manager, you must be prepared to handle possible negative personnel issues, such as job termination, especially if the project was not successful. When closing down the project, it is also important to notify all interested parties that the project has been completed and to finalize all project documentation and financial records so that a final review of the project can be conducted. You should also celebrate the accomplishments of the team. The goal is to celebrate the team's effort in bringing a difficult task to a successful conclusion.
2. Conducting post project reviews: Once you have closed down the project, final reviews of the project should be conducted with management and customers. The objective of these reviews is to determine the strengths and weaknesses of project deliverables, the processes used to create them, and the project management process. It is important that everyone understands what went right and what went wrong, in order to improve the process for the next project. Remember, the systems development methodology adopted by an organization is a living guideline that must undergo continual improvement.

3. Closing the customer contract: The focus of this final activity is to ensure that all contractual terms of the project have been met. A project governed by a contractual agreement is typically not completed until agreed to by both parties, often in writing. Thus, it is paramount that you gain agreement from your customer that all contractual obligations have been met and that further work is either their responsibility or covered under another system service request or contract. Closedown is an important activity. A project is not complete until it is closed, and it is at close down that projects are deemed a success or failure. Completion also signifies the chance to begin a new project and apply

Activity	Description	Skill
Leadership	Influencing the activities of others toward the attainment of a common goal through the use of intelligence, personality, and abilities	Communication; liaison between management, users, and developers; assigning activities; monitoring progress
Management	Getting projects completed through the effective utilization of resources	Defining and sequencing activities; communicating expectations; assigning resources to activities; monitoring outcomes
Customer relations	Working closely with customers to ensure project deliverables meet expectations	Interpreting system requests and specifications; site preparation and user training; contact point for customers
Technical problem solving	Designing and sequencing activities to attain project goals	Interpreting system requests and specifications; defining activities and their sequence; making trade-offs between alternative solutions; designing solutions to problems
Conflict management	Managing conflict within a project team to ensure that conflict is not too high or too low	Problem solving; smoothing out personality differences; compromising; goal setting
Team management	Managing the project team for effective team performance	Communication within and between teams; peer evaluations; conflict resolution; team building; self-management
Risk and change management	Identifying, assessing, and managing the risks and day-to-day changes that occur during a project	Environmental scanning; risk and opportunity identification and assessment; forecasting; resource redeployment

Table 6: Common Activities and Skills of a Project Manager

what you have learned.

Representing Project Plans

Project scheduling and management requires that time, costs, and resources be controlled. Resources are any person, group of people, piece of equipment, or material used in accomplishing an activity. Network diagramming is a critical path scheduling technique used for controlling resources. A critical path refers to a sequence of task activities whose order and durations directly affect the completion date of a project. A Network diagram is one of the most

widely used and best-known scheduling methods. A major strength of Network diagramming is its ability to represent how completion times vary for activities. Because of this, it is more often used than Gantt charts to manage projects such as information systems development where variability in the duration of activities is the norm.

Network diagrams are composed of circles or rectangles representing activities and connecting arrows showing required work flows, as illustrated in the following figure.

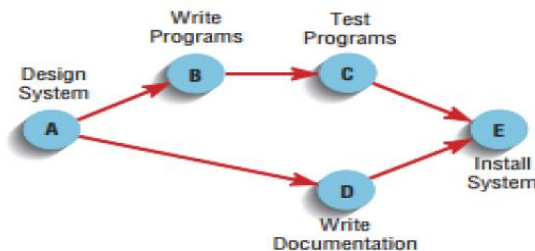


Figure 6: Network Diagram

Calculating Expected Time Durations Using PERT

One of the most difficult and most error-prone activities when constructing a project schedule is the determination of the time duration for each task within a work breakdown structure. It is particularly problematic to make these estimates when a high degree of complexity and uncertainty characterize a task. PERT (program evaluation review technique) is a technique that uses optimistic, pessimistic, and realistic time estimates to calculate the expected time for a particular task. This technique helps you obtain a better time estimate when you are uncertain as to how much time a task will require to be completed. The optimistic (o) and pessimistic (p) times reflect the minimum and maximum possible periods of time for an activity to be completed. The realistic time (r), or most likely time, reflects the project manager's —best guess— of the amount of time the activity will require for completion.

Once each of these estimates is made for an activity, an expected completion time (ET) can be calculated for that activity. Because the expected completion time should be closer to the realistic time (r), the realistic time is typically weighted 4 times more than the optimistic (o) and pessimistic (p) times. Once you add these values together, it must be divided by 6 to determine the ET. This equation is shown in the following formula:

$$ET = \frac{o + 4r + p}{6}$$

Where; ET = expected time for the completion for an activity

o = optimistic completion time for an activity

r = realistic completion time for an activity

p = pessimistic completion time for an activity

For example, suppose that your instructor asked you to calculate an expected time for the completion of an upcoming programming assignment. For this assignment, you estimate an optimistic time of 2 hours, a pessimistic time of 8 hours, and a most likely time of 6 hours. Using PERT, the expected time for completing this assignment is 5.67 hours.

1.5 Systems Planning and Selection

1.5.1 Identifying and Selecting Systems Development Projects

The first activity of the systems planning and selection phase of the SDLC is project identification and selection. During this activity, a senior manager, a business group identifies and assesses all possible systems development projects that a business unit could undertake. Next, those projects deemed most likely to yield significant organizational benefits, given available resources, are selected. Organizations vary in their approach to identifying and selecting projects. In some organizations, project identification and selection is a formal process in which projects are outcomes of a larger overall planning process. For example, a large organization may follow a formal project identification process that involves rigorously comparing all competing projects. Alternatively, a small organization may use informal project selection processes that allow the highest-ranking manager to select projects independently or allow individual business units to decide on projects after agreeing on funding. Requests for information systems development can come from three key sources, as depicted in the following figure:

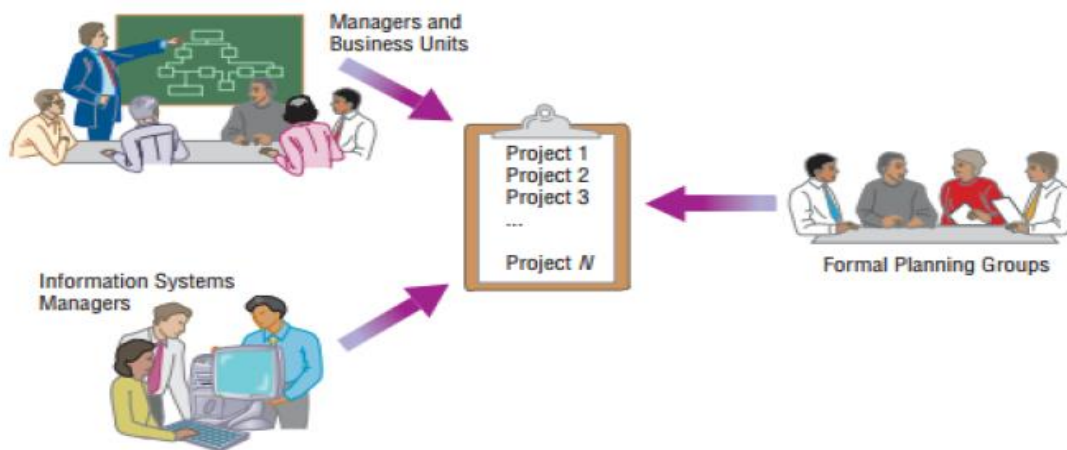


Figure 7: System Development Request Sources

1. Managers and business units: those who want to replace or extend an existing system in order to gain needed information or to provide a new service to customers.
2. Information systems managers: those who want to make a system more efficient, less costly to operate, or want to move a system to a new operating environment.
3. Formal planning groups: those who want to improve an existing system in order to help the organization meet its corporate objectives, such as providing better customer service.

Process of Identifying and Selecting Systems Development Projects

Project identification and selection consists of three primary activities: identifying potential projects, classifying and ranking projects, and selecting projects for development.

1. Identifying potential systems development projects: Organizations vary as to how they identify projects. This process can be performed by:
 - ⌘ A key member of top management, either the CEO of a small or medium-size organization or a senior executive in a larger organization.
 - ⌘ A steering committee composed of a cross section of managers with an interest in systems.
 - ⌘ User departments, in which either the head of the requesting unit or a committee from the requesting department decides which projects to submit (as a systems analyst, you will help users prepare such requests).
 - ⌘ The development group or a senior manager.

Each identification method has strengths and weaknesses. For example, projects identified by top management have a strategic organizational focus. Alternatively, projects identified by steering committees reflect the diversity of the committee and therefore have a cross-functional focus. Projects identified by individual departments or business units have a narrow, tactical focus. The development group identifies projects based on the ease with which existing hardware and systems will integrate with the proposed project. Other factors, such as project cost, duration, complexity, and risk, also influence the people who identify a project. Of all the possible project sources, those identified by top management and steering committees most often reflect the broader needs of the organization. These groups have a better understanding of overall business objectives and constraints.

Projects identified by top management or by a diverse steering committee are therefore referred to as coming from a top-down source. Projects identified by a functional manager, a business unit, or the information systems development group are often designed for a particular business need within a given business unit and may not reflect the overall objectives of the organization.

It's not that projects identified by individual managers, business units, or the IS development group are deficient, but rather that they may not consider broader organizational issues. Project initiatives stemming from managers, business units, or the development group are referred to as coming from a bottom-up source. As a systems analyst, you provide ongoing support for users of these types of projects and are involved early in the life cycle. You help managers describe their information needs and the reasons for doing the project. These descriptions are evaluated in selecting which projects will be approved to move into the project initiation and planning activities.

In sum, projects are identified by both top-down and bottom-up initiatives. The formality of identifying and selecting projects can vary substantially across organizations. Because limited resources preclude the development of all proposed systems, most organizations have some process of classifying and ranking each project's merit. Those projects deemed to be inconsistent

with overall organizational objectives, redundant in functionality to some existing system, or unnecessary will not be considered.

2. Classifying and ranking systems development projects: Assessing the merit of potential projects is the second major activity in the project identification and selection phase. As with project identification, classifying and ranking projects can be performed by top managers, a steering committee, business units, or the IS development group. The criteria used to assign the merit of a given project can vary based on the size of the organization.

The following table summarizes the criteria commonly used to evaluate and rank projects.

Evaluation Criteria	Description
Value chain analysis	Extent to which activities add value and costs when developing products and/or services; information systems projects providing the greatest overall benefits will be given priority over those with fewer benefits
Strategic alignment	Extent to which the project is viewed as helping the organization achieve its strategic objectives and long-term goals
Potential benefits	Extent to which the project is viewed as improving profits, customer service, etc., and the duration of these benefits
Resource availability	Amount and type of resources the project requires and their availability
Project size/duration	Number of individuals and the length of time needed to complete the project
Technical difficulty/risks	Level of technical difficulty to complete the project successfully within given time and resource constraints

Table 7: Evaluation Criteria to Rank Projects

Selecting systems development projects: The selection of projects is the final activity in the project identification and selection phase. The short- and long-term projects most likely to achieve business objectives are considered. As business conditions change over time, the relative importance of any single project may substantially change. Thus, the identification and selection of projects is an important and ongoing activity. Numerous factors must be considered when selecting a project, as illustrated in the following figure:

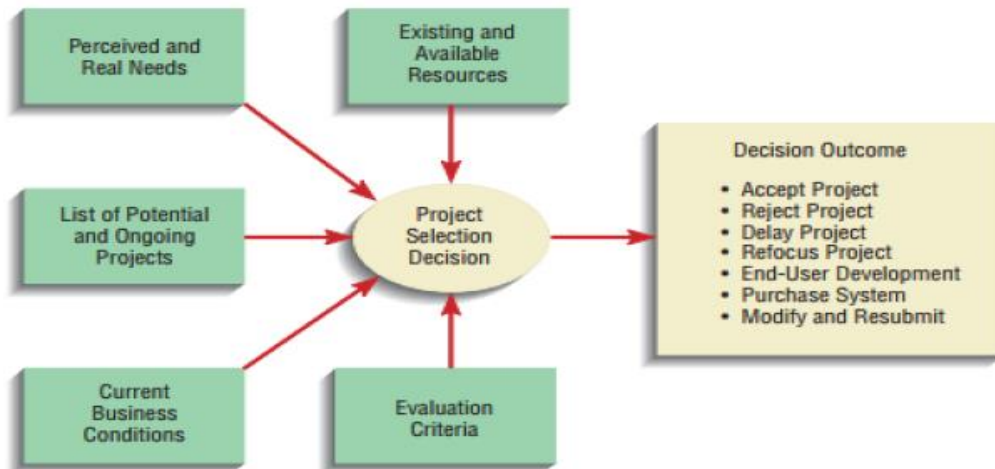


Figure 8: Factors while Project Selection

This decision-making process can lead to numerous outcomes. Of course, projects can be accepted or rejected. Acceptance of a project usually means that funding to conduct the next SDLC activity has been approved. Rejection means that the project will no longer be considered for development. However, projects may also be conditionally accepted; projects may be accepted pending the approval or availability of needed resources or the demonstration that a particularly difficult aspect of the system can be developed.

Projects may also be returned to the original requesters who are told to develop or purchase the requested system themselves. Finally, the requesters of a project may be asked to modify and resubmit their request after making suggested changes or clarifications.

Deliverables and Outcomes

The primary deliverable, or end product, from the project identification and selection phase is a schedule of specific systems development projects. These projects come from both top-down and bottom-up sources, and once selected they move into the second activity within this SDLC phase—project initiation and planning.

An outcome of this activity is the assurance that people in the organization gave careful consideration to project selection and clearly understood how each project could help the organization reach its objectives. Because of the principle of incremental commitment, a selected project does not necessarily result in a working system. Incremental commitment means that after each subsequent SDLC activity, you, other members of the project team, and organization officials will reassess your project. This reassessment will determine whether the business conditions have changed or whether a more detailed understanding of a system's costs, benefits, and risks would suggest that the project is not as worthy as previously thought.

1.5.2 Initiating and Planning Systems Development Projects

Many activities performed during initiation and planning could also be completed during the next phase of the SDLC—systems analysis. Proper and insightful project initiation and planning, including determining project scope and identifying project activities, can reduce the time

needed to complete later project phases, including systems analysis. For example, a careful feasibility analysis conducted during initiation and planning could lead to rejecting a project and saving a considerable expenditure of resources.

The actual amount of time expended will be affected by the size and complexity of the project as well as by the experience of your organization in building similar systems. A rule of thumb is that between 10 and 20 percent of the entire development effort should be expended on initiation and planning. In other words, you should not be reluctant to spend considerable time and energy early in the project's life in order to fully understand the motivation for the requested system.

The objective of project initiation and planning is to transform a vague system request document into a tangible project description.

Effective communication among the systems analysts, users, and management is crucial to the creation of a meaningful project plan. Getting all parties to agree on the direction of a project may be difficult for cross-department projects when different parties have different business objectives. Projects at large, complex organizations require systems analysts to take more time to analyze both the current and proposed systems.

Process of Initiating and Planning Systems Development Projects

As its name implies, two major activities occur during project initiation and project planning. Project initiation focuses on activities that will help organize a team to conduct project planning. During initiation, one or more analysts are assigned to work with a customer to establish work standards and communication procedures. The following table summarizes six activities performed during project initiation.

- Establishing the project initiation team
- Establishing a relationship with the customer
- Establishing the project initiation plan
- Establishing management procedures
- Establishing the project management environment and project workbook
- Developing the project charter

Table 8: Project Initiation Activities

The second activity, project planning, focuses on defining clear, discrete tasks and the work needed to complete each task. The objective of the project planning process is to produce two documents: a baseline project plan (BPP) and the project scope statement (PSS).

The BPP becomes the foundation for the remainder of the development project. It is an internal document used by the development team but not shared with customers. The PSS, produced by the project team, clearly outlines the objectives of the project for the customer. As with the project initiation process, the size, scope, and complexity of a project dictate the comprehensiveness of the project planning process and the resulting documents. Further,

numerous assumptions about resource availability and potential problems will have to be made. The following table lists the activities performed during project planning.

- Describing the project scope, alternatives, and feasibility
- Dividing the project into manageable tasks
- Estimating resources and creating a resource plan
- Developing a preliminary schedule
- Developing a communication plan
- Determining project standards and procedures
- Identifying and assessing risk
- Creating a preliminary budget
- Developing a project scope statement
- Setting a baseline project plan

Table 9: Activities during Project Planning

Deliverables and Outcomes

The major outcomes and deliverables from project initiation and planning are the baseline project plan and the project scope statement. The baseline project plan (BPP) contains all information collected and analyzed during the project initiation and planning activity. The plan contains the best estimate of the project's scope, benefits, costs, risks, and resource requirements given the current understanding of the project. The BPP specifies detailed project activities for the next life cycle phase—systems analysis—and provides less detail for subsequent project phases (because these depend on the results of the analysis phase). Similarly, benefits, costs, risks, and resource requirements will become more specific and quantifiable as the project progresses. The project selection committee uses the BPP to help decide whether to continue, redirect, or cancel a project. If selected, the BPP becomes the foundation document for all subsequent SDLC activities; however, it is updated as new information is learned during subsequent SDLC activities.

1.5.3 Assessing Project Feasibility

Information systems projects have budgets and deadlines. Assessing project feasibility is a required task that can be a large undertaking because it requires you, as a systems analyst, to evaluate a wide range of factors. Although the specifics of a given project will dictate which factors are most important, most feasibility factors fall into the following six categories:

- Economic
- Operational
- Technical
- Schedule
- Legal and contractual
- Political

The analysis of these six factors forms the business case that justifies the expenditure of resources on the project.

Assessing Economic Feasibility

A study of economic feasibility is required for the baseline project plan. The purpose for assessing economic feasibility is to identify the financial benefits and costs associated with the development project. Economic feasibility is often referred to as cost-benefit analysis. During project initiation and planning, it will be impossible for you to define precisely all benefits and costs related to a particular project. Yet, it is important that you identify and quantify benefits and costs, or it will be impossible for you to conduct a sound economic analysis and determine whether one project is more feasible than another.

An information system can provide many benefits to an organization. For example, a new or renovated IS can automate monotonous jobs, reduce errors, provide innovative services to customers and suppliers, and improve organizational efficiency, speed, flexibility, and morale. These benefits are both tangible and intangible. A tangible benefit is an item that can be measured in dollars, birr and with certainty. Examples of tangible benefits include reduced personnel expenses, lower transaction costs, or higher profit margins. It is important to note that not all tangible benefits can be easily quantified. For example, a tangible benefit that allows a company to perform a task 50 percent of the time may be difficult to quantify in terms of hard dollar savings. Most tangible benefits fit in one or more of the following categories:

- Cost reduction and avoidance
- Error reduction
- Increased flexibility
- Increased speed of activity
- Improvement of management planning and control
- Opening new markets and increasing sales opportunities

Intangible benefits refer to items that cannot be easily measured in dollars, birr or with certainty.

Intangible benefits may have direct organizational benefits, such as the improvement of employee morale, or they may have broader societal implications, such as the reduction of waste creation or resource consumption. Potential tangible benefits may have to be considered intangible during project initiation and planning because you may not be able to quantify them in dollars or with certainty at this stage in the life cycle. During later stages, such intangibles can become tangible benefits as you better understand the ramifications of the system you are designing.

After determining project benefits, project costs must be identified. An information system can have both tangible and intangible costs. A tangible cost refers to an item that you can easily measure in dollars and with certainty.

From a systems development perspective, tangible costs include items such as hardware costs, labor costs, and operational costs from employee training and building renovations.

Alternatively, an intangible cost refers to an item that you cannot easily measure in terms of dollars or with certainty. Intangible costs can include loss of customer goodwill, employee morale, or operational inefficiency. Besides tangible and intangible costs, you can distinguish system-related development costs as either one-time or recurring. A one-time cost refers to a cost associated with project initiation and development and the start-up of the system. These costs typically encompass the following activities:

- System development
- New hardware and software purchases
- User training
- Site preparation
- Data or system conversion

When conducting an economic cost-benefit analysis, you should create a worksheet for capturing these expenses.

A recurring cost refers to a cost resulting from the ongoing evolution and use of the system. Examples of these costs typically include:

- Application software maintenance
- Incremental data storage expense
- Incremental communications
- New software and hardware leases
- Consumable supplies and other expenses (e.g., paper, forms, datacenter personnel)

Most techniques used to determine economic feasibility encompass the concept of the time value of money (TVM). TVM refers to comparing present cash outlays to future expected returns. Furthermore, benefits from systems development will likely occur sometime in the future.

Assessing Operational Feasibility

Operational feasibility is the process of examining the likelihood that the project will attain its desired objectives.

The goal of this study is to understand the degree to which the proposed system will likely solve the business problems or take advantage of the opportunities outlined in the system service request or project identification study. In other words, assessing operational feasibility requires that you gain a clear understanding of how a system will fit into the current day-to-day operations of the organization.

Assessing Technical Feasibility

The goal of technical feasibility is to understand the development organization's ability to construct the proposed system. This analysis should include an assessment of the development group's understanding of the possible target hardware, software, and operating environments to be used, as well as, system size, complexity, and the group's experience with similar systems.

Assessing Schedule Feasibility

Schedule feasibility considers the likelihood that all potential time frames and completion date

schedules can be met and that meeting these dates will be sufficient for dealing with the needs of the organization. For example, a system may have to be operational by a government-imposed deadline by a particular point in the business cycle (such as the beginning of the season when new products are introduced), or at least by the time a competitor is expected to introduce a similar system.

Legal and Contractual Feasibility

Assessing legal and contractual feasibility requires that you gain an understanding of any potential legal and contractual ramifications due to the construction of the system.

Considerations might include copyright or nondisclosure infringements, labor laws, antitrust legislation (which might limit the creation of systems to share data with other organizations), foreign trade regulations (e.g., some countries limit access to employee data by foreign corporations), and financial reporting standards as well as current or pending contractual obligations. Typically, legal and contractual feasibility is a greater consideration if your organization has historically used an outside organization for specific systems or services that you now are considering handling yourself.

Political Feasibility

Assessing political feasibility involves understanding how key stakeholders within the organization view the proposed system. Because an information system may affect the distribution of information within the organization, and thus the distribution of power, the construction of a system can have political ramifications. Those stakeholders not supporting the project may take steps to block, disrupt, or change the project's intended focus.

1.5.4 Building Baseline Project Plan

All the information collected during project initiation and planning is collected and organized into a document called the baseline project plan. An outline of a baseline project plan contains four major sections:

- Introduction
- System description
- Feasibility assessment
- Management issues

The purpose of the introduction is to provide a brief overview of the entire document and outline a recommended course of action for the project. The introduction is often limited to only a few pages. Although it is sequenced as the first section of the BPP, it is often the final section to be written. It is only after performing most of the project planning activities that a clear overview and recommendation can be created. One initial activity that should be performed is the definition of the project scope, its range, which is an important part of the BPP's introduction section.

A **project scope statement** is a short document prepared primarily for the customer to clearly describe what the project will deliver and outline generally at a high level all the work required for completing the project. It is therefore a useful communication tool. The project scope statement ensures that both you and your customer gain a common understanding of the project

size, duration, and outcomes. The project scope statement is an easy document to create because it typically consists of a high-level summary of the baseline project plan (BPP) information.

The second section of the BPP is the system description, in which you outline possible alternative solutions to the one deemed most appropriate for the given situation. Note that this description is at a high level, mostly narrative in form.

In the third section of the BPP, feasibility assessment, the systems analyst outlines project costs and benefits and technical difficulties.

The final section of the BPP, management issues, outlines the concerns that management has about the project. It will be a short section if the proposed project is going to be conducted exactly as prescribed by the organization's standard systems development methodology. Most projects, however, have some unique characteristics that require minor to major deviation from the standard methodology. In the team configuration and management portion, you identify the types of people to work on the project, who will be responsible for which tasks, and how work will be supervised and reviewed. In the communication plan portion, you explain how the user will be kept informed about project progress, such as periodic review meetings or even a newsletter, and which mechanisms will be used to foster sharing of ideas among team members, such as a computer-based conference facility.

1.6 System Requirement Structuring

The amount of information gathered during requirements determination could be huge, especially if the scope of the system under development is broad. The time required to collect and structure a great deal of information can be extensive and, because it involves so much human effort, quite expensive.

1.6.1 Process Modeling

Process modeling involves graphically representing the processes, or actions, that capture, manipulate, store, and distribute data between a system and its environment and among components within a system. A common form of a process model is a data-flow diagram (**DFD**). A data-flow diagram is a graphic representation that illustrates the movement of data between external entities and the processes and data stores within a system.

Although several different tools have been developed for process modeling, we focus solely on data-flow diagrams because they are useful tools for process modeling.

Data-flow diagramming is one of several structured analysis techniques used to increase software development productivity. Although not all organizations use each structured analysis technique, collectively, these techniques, like dataflow diagrams, have had a significant impact on the quality of the systems development process.

The analysis team begins the process of structuring requirements with an abundance of information gathered during requirements determination. As part of structuring, you and the other team members must organize the information into a meaningful representation of the information system that exists and of the requirements desired in a replacement system. In addition to modeling the processing elements of an information system and transformation of

data in the system, you must also model the structure of data within the system. Analysts use both process and data models to establish the specification of an information system. With a supporting tool, such as a CASE tool, process and data models can also provide the basis for the automatic generation of an information system.

Deliverables and Outcomes

In structured analysis, the primary deliverables from process modeling are a set of coherent, interrelated data-flow diagrams. The following table lists the progression of deliverables that result from studying and documenting a system’s process.

1. Context DFD
2. DFDs of current physical system
3. DFDs of new logical system
4. Thorough descriptions of each DFD component

Table 10: Deliverables for Process Modeling

First, a context data-flow diagram shows the scope of the system, indicating which elements are inside and outside the system. Second, data-flow diagrams of the current system specify which people and technologies are used in which processes to move and transform data, accepting inputs and producing outputs. The detail of these diagrams allows analysts to understand the current system and eventually to determine how to convert the current system into its replacement.

Third, technology-independent, or logical, data-flow diagrams show the dataflow, structure, and functional requirements of the new system. Finally, entries for all of the objects in all diagrams are included in the project dictionary or CASE repository.

This logical progression of deliverables helps you to understand the existing system. You can then reduce this system into its essential elements to show the way in which the new system should meet its information processing requirements, as they were identified during requirements determination. In later steps in the systems development life cycle, you and other project team members make decisions on exactly how the new system will deliver these new requirements in specific manual and automated functions. Because requirements determination and structuring are often parallel steps, data-flow diagrams evolve from the more general to the more detailed as current and replacement systems are better understood.

Even though data-flow diagrams remain popular tools for process modeling and can significantly increase software development productivity, they are not used in all systems development methodologies. Some methodologies, such as rapid application development (RAD), do not model processes separately at all.

Instead, RAD builds processes—the work or actions that transform data so that they can be stored or distributed—into the prototypes created as the core of its development life cycle.

However, even if you never formally use data-flow diagrams in your professional career, they remain a part of systems development's history.

DFDs illustrate important concepts about the movement of data between manual and automated steps and are a way to depict work flow in an organization. DFDs continue to benefit information systems professionals as tools for both analysis and communication.

1.6.2 Data-Flow Diagram Mechanics

Data-flow diagrams are versatile diagramming tools. With only four symbols, data-flow diagrams can represent both physical and logical information systems. The four symbols used in DFDs represent data flows, data stores, processes, and sources/sinks (or external entities). The set of four symbols we use in this module was developed by Gane and Sarson (1979) and is illustrated in the following figure.

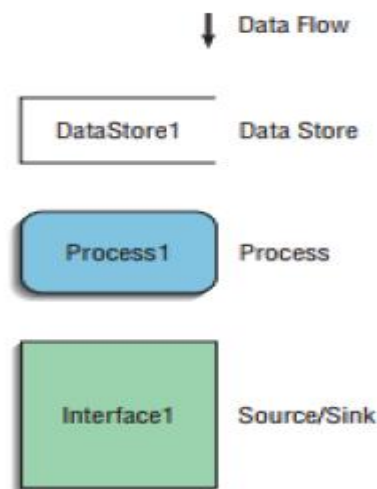


Figure 9: Symbols Used in DFDs

A data flow is data that are in motion and moving as a unit from one place in a system to another. A data flow could represent data on a customer order form or a payroll check. It could also represent the results of a query to a database, the contents of a printed report, or data on a data-entry computer display form. A data flow can be composed of many individual pieces of data that are generated at the same time and that flow together to common destinations. A data store is data at rest. A data store may represent one of many different physical locations for data, including a file folder, one or more computer-based file(s), or a notebook.

To understand data movement and handling in a system, the physical configuration is not really important. A data store might contain data about customers, students, customer orders, or supplier invoices.

A process is the work or actions performed on data so that they are transformed, stored, or distributed. When modeling the data processing of a system, it doesn't matter whether a process

is performed manually or by a computer.

Finally, a source/sink is the origin and/or destination of the data. Source/sinks are sometimes referred to as external entities because they are outside the system. Once processed, data or information leave the system and go to some other place.

Definitions and Symbols

Among the DFD symbols presented in the above figure, a data flow is depicted as an arrow. The arrow is labeled with a meaningful name for the data in motion; for example, customer order, sales receipt, or paycheck.

The name represents the aggregation of all the individual elements of data moving as part of one packet, that is, all the data moving together at the same time. A rectangle or square is used for sources/sinks, and its name states what the external agent is, such as customer, teller, Environmental Protection Agency (EPA) office, or inventory control system. The symbol for a process is a rectangle with rounded corners. Inside the rectangle are written both the number of the process and a name, which indicates what the process does.

For example, the process may generate paychecks, calculate overtime pay, or compute grade-point average. The symbol for a data store is a rectangle with the right vertical line missing. Its label includes the number of the data store (e.g., D1 or D2) and a meaningful label, such as student file, transcripts, or roster of classes.

Developing DFDs: An Example

The main processes in the DFD represent the major functions of the system, and these major functions correspond to such actions as the following:

- ⌘ Capturing data from different sources (Process 1.0)
- ⌘ Maintaining data stores (Processes 2.0 and 3.0)
- ⌘ Producing and distributing data to different sinks (Process 4.0)
- ⌘ High-level descriptions of data transformation operations (Process 1.0)

We see that the system in the following figure begins with an order from a customer. In the first process, labeled —1.0, we see that the customer order is processed. The results are four streams or flows of data: (1) The food order is transmitted to the kitchen, (2) the customer order is transformed into a list of goods sold, (3) the customer order is transformed into inventory data, and (4) the process generates a receipt for the customer.

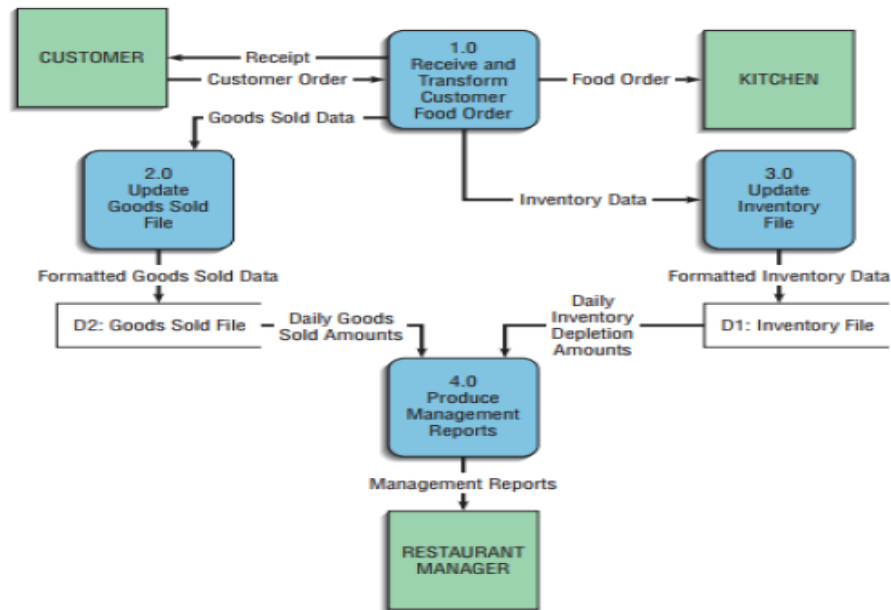


Figure 10: DFD Example

Let's trace the flow of the data: First, the data labeled Goods Sold go to Process 2.0, Update Goods Sold File. The output for this process is labeled Formatted Goods Sold Data. This output updates a data store labeled Goods Sold File. If the customer order were for two cheeseburgers, one order of fries, and a large soft drink, each of these categories of goods sold in the data store would be incremented appropriately. The Daily Goods Sold Amounts are then used as input to Process 4.0, Produce Management Reports.

Similarly, the remaining data flow generated by Process 1.0, called Inventory Data, serves as input for Process 3.0, Update Inventory File. This process updates the Inventory File data store, based on the inventory that would have been used to create the customer order.

Data-Flow Diagram Rules

You must follow a set of rules when drawing data-flow diagrams. These rules, listed in the following table allow you to evaluate DFDs for correctness.

Process

- A. No process can have only outputs. It is making data from nothing (a miracle). If an object has only outputs, then it must be a source.
- B. No process can have only inputs (a black hole). If an object has only inputs, then it must be a sink.
- C. A process has a verb-phrase label.

Data Store

- D. Data cannot move directly from one data store to another data store. Data must be moved by a process.
- E. Data cannot move directly from an outside source to a data store. Data must be moved by a process that receives data from the source and places the data into the data store.
- F. Data cannot move directly to an outside sink from a data store. Data must be moved by a process.
- G. A data store has a noun-phrase label.

Source/Sink

- H. Data cannot move directly from a source to a sink. They must be moved by a process if the data are of any concern to our system. Otherwise, the data flow is not shown on the DFD.
- I. A source/sink has a noun-phrase label.

Data Flow

- J. A data flow has only one direction of flow between symbols. It may flow in both directions between a process and a data store to show a read before an update. The latter is usually indicated, however, by two separate arrows because the read and update usually happen at different times.
- K. A fork in a data flow means that exactly the same data go from a common location to two or more different processes, data stores, or sources/sinks (it usually indicates different copies of the same data going to different locations).
- L. A join in a data flow means that exactly the same data come from any of two or more different processes, data stores, or sources/sinks to a common location.
- M. A data flow cannot go directly back to the same process it leaves. At least one other process must handle the data flow, produce some other data flow, and return the original data flow to the beginning process.
- N. A data flow to a data store means update (delete or change).
- O. A data flow from a data store means retrieve or use.
- P. A data flow has a noun-phrase label. More than one data-flow noun phrase can appear on a single arrow as long as all of the flows on the same arrow move together as one package.

Source: Based on J. Celko, "1. Data Flow Diagrams," *Computer Language* 4 (January 1987), 41–43.

Table 11: DFD Rules

Besides the rules in table, two DFD guidelines apply most of the time:

- ⌘ The inputs to a process are different from the outputs of that process: The reason is that processes, to have a purpose, typically transform inputs into outputs, rather than simply passing the data through without some manipulation. The same input may go in and out of a process, but the process also produces other new data flows that are the result of manipulating the inputs.
- ⌘ Objects on a DFD have unique names: Every process has a unique name. There is no reason to have two processes with the same name.

To keep a DFD uncluttered, however, you may repeat data stores and sources/sinks. When two arrows have the same data-flow name, you must be careful that these flows are exactly the same. It is a mistake to reuse the same data-flow name when two packets of data are almost the same but not identical. Because a data-flow name represents a specific set of data, another data flow that has even one more or one less piece of data must be given a different, unique name.

Decomposition of DFDs

In the food-ordering system, we started with a high-level context diagram. After drawing the diagram, we saw that the larger system consisted of four processes. The act of going from a single system to four component processes is called (functional) decomposition. Functional decomposition is a repetitive process of breaking the description or perspective of a system down into finer and finer detail. This process creates a set of hierarchically related charts in which one process on a given chart is explained in greater detail on another chart. For the food-ordering system, we broke down or decomposed the larger system into four processes. Each of those processes (or subsystems) is also a candidate for decomposition. Each process may consist of

several subprocesses. Each subprocess may also be broken down into smaller units. Decomposition continues until no subprocess can logically be broken down any further. The lowest level of DFDs is called a primitive DFD. Let's continue with food-ordering system to see how a level-0 DFD can be further decomposed. The first process in the above food ordering figure, called Receive and Transform Customer Food Order, transforms a customer's verbal food order (e.g., —Give me two cheeseburgers, one small order of fries, and one large orange soda)) into four different outputs. Process 1.0 is a good candidate process for decomposition. Think about all of the different tasks that Process 1.0 has to perform: (1) Receive a customer order, (2) transform the entered order into a printed receipt for the customer, (3) transform the order into a form meaningful for the kitchen's system, (4) transform the order into goods sold data, and (5) transform the order into inventory data. At least these five logically separate functions occur in Process 1.0. We can represent the decomposition of Process 1.0 as another DFD, as shown in the following figure.

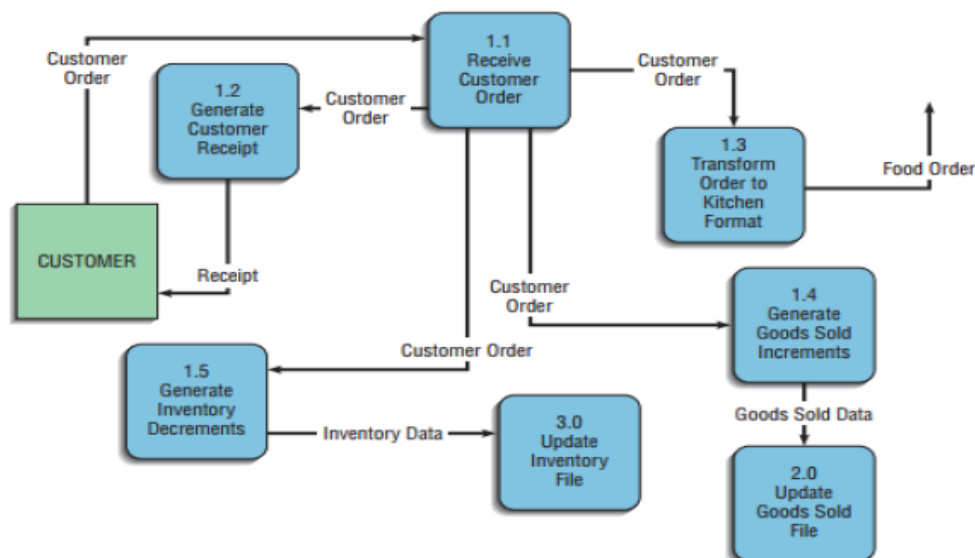


Figure 11: DFD Decomposition

Note that each of the five processes in the figure is labeled as subprocesses of Process 1.0: Process 1.1, Process 1.2, and so on. Also note that, just as with the other data-flow diagrams we have looked at, each of the processes and data flows are named. No sources or sinks are represented. The context and level-0 diagrams show the sources and sinks. The data-flow diagram in the figure is called a level-1 diagram. If we should decide to decompose Processes 2.0, 3.0, or 4.0 in a similar manner, the DFDs we create would also be called level-1 diagrams. In general, a level-n diagram is a DFD that is generated from n nested decompositions from a level-0 diagram.

Processes 2.0 and 3.0 perform similar functions in that they both use data input to update data stores. Because updating a data store is a singular logical function, neither of these processes needs to be decomposed further. We can, on the other hand, decompose Process 4.0, Produce Management Reports, into at least three subprocesses: Access Goods Sold and Inventory Data,

Aggregate Goods Sold and Inventory Data, and Prepare Management Reports. The decomposition of Process 4.0 is shown in the level-1 diagram as follows:

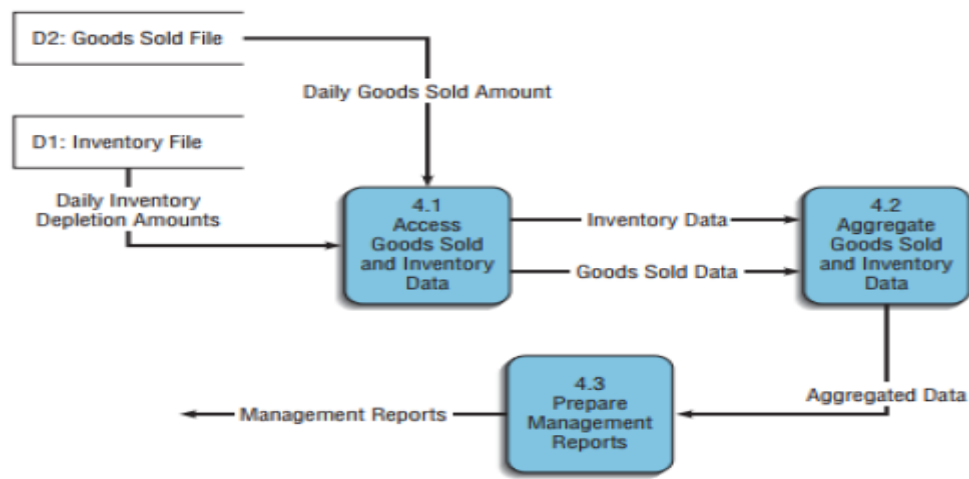


Figure 12: DFD Decomposition Level -1

To continue with the decomposition of food-ordering system, we examine each of the subprocesses identified in the two level-1 diagrams, one for Process 1.0 and one for Process 4.0. To further decompose any of these subprocesses, we would create a level-2 diagram showing that decomposition. For example, if we decided to further decompose Process 4.3 in the above figure, we would create a diagram that looks something like the following:

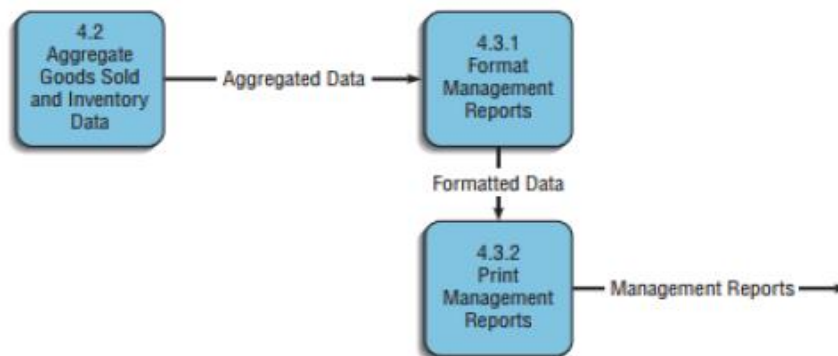


Figure 13: DFD Decomposition Level - 2

Again, notice how the sub processes are labeled. Just as the labels for processes must follow numbering rules for clear communication, process names should also be clear, yet concise. Typically, process names begin with an action verb, such as receive, calculate, transform, generate, or produce. Often process names are the same as the verbs used in many computer programming languages. Examples include merge, sort, read, write, and print. Process names should capture the essential action of the process in just a few words, yet be

descriptive enough of the action of the process so that anyone reading the name gets a good idea of what the process does.

Many times, students just learning DFDs will use the names of people who perform the process or the department in which the process is performed as the process name. This practice is not especially useful, because we are more interested in the action the process represents than the person performing it or the place where it occurs.

Primitive DFDs

One of the more difficult decisions you need to make when drawing DFDs is when to stop decomposing processes. One rule is to stop drawing when you have reached the lowest logical level; however, it is not always easy to know what the lowest logical level is. Other more concrete rules for when to stop decomposing are:

- ⌘ When you have reduced each process to a single decision or calculation or to a single database operation, such as retrieve, update, create, delete, or read
- ⌘ When each data store represents data about a single entity, such as a customer, employee, product, or order
- ⌘ When the system user does not care to see any more detail, or when you and other analysts have documented sufficient detail to do subsequent systems development tasks
- ⌘ When every data flow does not need to be split further to show that different data are handled in various ways
- ⌘ When you believe that you have shown each business form or transaction, computer online display, and report as a single data flow (e.g., often means that each system display and report title corresponds to the name of an individual data flow)
- ⌘ When you believe a separate process is shown for each choice on all lowest-level menu options

By the time you stop decomposing DFDs, a DFD can become quite detailed. Seemingly simple actions, such as generating an invoice, may pull information from several entities and may also return different results depending on the specific situation.

For example, the final form of an invoice may be based on the type of customer (which would determine such things as discount rate), where the customer lives (which would determine such things as sales tax), and how the goods are shipped (which would determine such things as the shipping and handling charges). At the lowest-level DFD, called a primitive DFD, all of these conditions would have to be met.

1.6.3 Logic Modeling

Although data-flow diagrams are good for identifying processes, they do not show the logic inside the processes. Even the processes on the primitive-level data-flow diagrams do not show the most fundamental processing steps. Just what occurs within a process? How are the input data converted to the output information? Because data-flow diagrams are not really designed to show the detailed logic of processes

Logic modeling involves representing the internal structure and functionality of the processes represented on data-flow diagrams. These processes appear on DFDs as little more than black boxes, in that we cannot tell from only their names precisely what they do and how they do it.

Yet, the structure and functionality of a system's processes are a key element of any information system. Processes must be clearly described before they can be translated into a programming language.

We introduce you to a common method for modeling system logic: Decision tables allow you to represent in a tabular format a set of conditions and the actions that follow from them. When several conditions and several possible actions can occur, decision tables help you keep track of the possibilities in a clear and concise manner.

Creating diagrams of process logic is not an end in itself. Rather, these diagrams are created ultimately to serve as part of a clear and thorough explanation of the system's specifications. These specifications are used to explain the system requirements to developers, whether people or automated code generators. Users, analysts, and programmers use logic diagrams throughout analysis to incrementally specify a shared understanding of requirements. Logic diagrams do not take into account specific programming languages or development environments.

Such diagrams may be discussed during JAD sessions or project review meetings. Alternatively, system prototypes generated from such diagrams may be reviewed, and requested changes to a prototype will be implemented by changing logic diagrams and generating a new prototype from a CASE tool or other code generator.

Logic Modeling with Decision Tables

A decision table is a diagram of process logic where the logic is reasonably complicated. All of the possible choices and the conditions the choices depend on are represented in tabular form, as illustrated in the decision table in the following figure. The decision table in figure models the logic of a generic payroll system. The three parts to the table include the condition stubs, the action stubs, and the rules. The condition stubs contain the various conditions that apply in the situation the table is modeling. In the figure, two condition stubs correspond to employee type and hours worked. Employee type has two values: —S, which stands for salaried, and —H, which stands for hourly. Hours worked have three values: less than 40, exactly 40, and more than 40.

	Conditions/ Courses of Action	Rules					
		1	2	3	4	5	6
Condition Stubs	Employee type	S	H	S	H	S	H
	Hours worked	< 40	< 40	40	40	> 40	> 40
Action Stubs	Pay base salary	X		X		X	
	Calculate hourly wage		X		X		X
	Calculate overtime						X
	Produce Absence Report		X				

Figure 14: Decision Table

The action stubs contain all the possible courses of action that result from combining values of the condition stubs. Four possible courses of action are indicated in this table: pay base salary, calculate hourly wage, calculate overtime, and produce Absence Report. You can see that not all actions are triggered by all combinations of conditions. Instead, specific combinations trigger specific actions. The part of the table that links conditions to actions is the section that contains the rules.

To read the rules, start by reading the values of the conditions as specified in the first column: Employee type is —S, I or salaried, and hours worked are less than 40. When both of these conditions occur, the payroll system is to pay the base salary.

In the next column, the values are ‘H I’ and ‘40,’ meaning an hourly worker who worked fewer than 40 hours. In such a situation, the payroll system calculates the hourly wage and makes an entry in the Absence Report. Rule 3 addresses the situation when a salaried employee works exactly 40 hours. The system pays the base salary, as was the case for rule 1. For an hourly worker who has worked exactly 40 hours, rule 4 calculates the hourly wage. Rule 5 pays the base salary for salaried employees who work more than 40 hours.

Rule 5 has the same action as rules 1 and 3 and governs behavior with regard to salaried employees. The number of hours worked does not affect the outcome for rules 1, 3, or 5. For these rules, hours worked is an indifferent condition, in that its value does not affect the action taken. Rule 6 calculates hourly pay and overtime for an hourly worker who has worked more than 40 hours.

In constructing these decision tables, we have actually followed a set of basic procedures, as follows:

- ⌘ Name the conditions and the values each condition can assume. Determine all of the conditions that are relevant to your problem, and then determine all of the values each condition can take. For some conditions, the values will be simply —yes I or —no I (called a limited entry). For others, such as the conditions in the above figure, the conditions may have more values (called an extended entry).

- ⌘ Name all possible actions that can occur. The purpose of creating decision tables is to determine the proper course of action given a particular set of conditions.

- ⌘ List all possible rules. When you first create a decision table, you have to create an exhaustive set of rules. Every possible combination of conditions must be represented.

It may turn out that some of the resulting rules are redundant or make no sense, but these determinations should be made only after you have listed every rule so that no possibility is overlooked. To determine the number of rules, multiply the number of values for each condition by the number of values for every other condition. In the above figure, we have two conditions, one with two values and one with three, so we need $2 * 3$, or 6, rules. If we added a third condition with three values, we would need $2 * 3 * 3$, or 18, rules.

- ⌘ Define the actions for each rule. Now that all possible rules have been identified, provide an action for each rule. In our example, we were able to figure out what each action should be and whether all of the actions made sense.

If an action doesn't make sense, you may want to create an —impossible row in the action stubs in the table to keep track of impossible actions. If you can't tell what the system ought to do in that situation, place question marks in the action stub spaces for that particular rule.

UNIT - II

2.1 Designing Databases

File and database design occurs in two steps. You begin by developing a logical database model, which describes data using a notation that corresponds to a data organization used by a database management system. This system software is responsible for storing, retrieving, and protecting data (such as Microsoft Access, Oracle, or SQL Server). The most common style for a logical database model is the relational database model. Once you develop a clear and precise logical database model, you are ready to prescribe the technical specifications for computer files and databases in which to store the data ultimately.

2.1.1 Logical Database Design

You typically do logical and physical database design in parallel with other systems design steps. Thus, you collect the detailed specifications of data necessary for logical database design as you design system inputs and outputs. Logical database design can be driven from the form and report layouts.

You study data elements on these system inputs and outputs and identify interrelationships among the data. As with conceptual data modeling, the work of all systems development team members is coordinated and shared through the project dictionary or repository. The designs for logical databases and system inputs and outputs are then used in physical design activities to specify to computer programmers, database administrators, network managers, and others how to implement the new information system.

We assume for this text that the design of computer programs and distributed information processing and data networks are topics of other courses, so we concentrate on the aspect of physical design most often undertaken by a systems analyst—physical file and database design.

Process of Database Design

Database modeling and design activities occur in all phases of the systems development process. In this section we discuss methods that help you finalize logical and physical database designs during the design phase.

In logical database design you use a process called normalization, which is a way to build a data model that has the properties of simplicity, no redundancy, and minimal maintenance.

Four steps are keys to logical database design:

- Develop a logical data model for each known user interface (form and report) for the application, using normalization principles.
- Combine normalized data requirements from all user interfaces into one consolidated logical database model; this step is called view integration.

- Translate the conceptual E-R data model for the application, developed without explicit consideration of specific user interfaces, into normalized data requirements.
- Compare the consolidated logical database design with the translated E-R model and produce, through view integration, one final logical database model for the application. During physical database design, you use the results of these four key logical database design steps. You also consider definitions of each attribute; descriptions of where and when data are entered, retrieved, deleted, and updated; expectations for response time and data integrity; and descriptions of the file and database technologies to be used.

Deliverables and Outcomes

During logical database design, you must account for every data element on a system input or output—form or report—and on the E-R model. Each data element (like customer name, product description, or purchase price) must be a piece of raw data kept in the system's database, or in the case of a data element on a system output, the element can be derived from data in the database.

The following figures illustrate the outcomes from the four-step logical database design process. Figures (A and B) contain two sample system outputs for a customer order processing system at Pine Valley Furniture. A description of the associated database requirements, in the form of what we call normalized relations, is listed below each output diagram. Each relation (think of a relation as a table with rows and columns) is named, and its attributes (columns) are listed within parentheses.

The primary key attribute—that attribute whose value is unique across all occurrences of the relation—is indicated by an underline, and an attribute of a relation that is the primary key of another relation is indicated by a dashed underline.

A

HIGHEST VOLUME CUSTOMER

ENTER PRODUCT ID.:	M128
START DATE:	11/01/2012
END DATE:	12/31/2012

CUSTOMER ID.:	1256
NAME:	Commonwealth Builder
VOLUME:	30

Figure 15: Outcomes of Logical Database Design

This inquiry screen shows the customer with the largest volume total sales of a specified product during an indicated time period.

Relations: CUSTOMER(Customer_ID,Name)

ORDER(Order_Number,Customer_ID,Order_Date) PRODUCT(Product_ID) LINE

ITEM(Order_Number,Product_ID,Order_Quantity)

B

BACKLOG SUMMARY REPORT		PAGE 1
11/30/2012		
<u>PRODUCT ID</u>	<u>BACKLOG</u>	
	<u>QUANTITY</u>	
B381	0	
B975	0	
B985	6	
E125	30	
⋮		
M128	2	
⋮		

Figure 16: Outcomes of Logical Database Design

This report shows the unit volume of each product that has been ordered less than amount shipped through the specified date.

Relations: PRODUCT(Product_ID)

LINE ITEM(Product_ID,Order_Number,Order_Quantity)

ORDER(Order_Number,Order_Date)

SHIPMENT(Product_ID,Invoice_Number,Ship_Quantity)

INVOICE(Invoice_Number,Invoice_Date,Order_Number)

2.1.2 Relational Database Model

Many different database models are in use and are the basis for database technologies. Although hierarchical and network models have been popular in the past, they are not often used today for new information systems. Object oriented database models are emerging but are still not common. The vast majority of information systems today use the relational database model. The relational database model represents data in the form of related tables or relations. A relation is a named, two-dimensional table of data. Each relation (or table) consists of a set of named columns and an arbitrary number of unnamed rows. Each column in a relation corresponds to an attribute of that relation. Each row of a relation corresponds to a record that contains data values for an entity. Below figure shows an example of a relation named EMPLOYEE1.

EMPLOYEE1

<u>Emp_ID</u>	Name	Dept	Salary
100	Margaret Simpson	Marketing	42,000
140	Allen Beeton	Accounting	39,000
110	Chris Lucero	Info Systems	41,500
190	Lorenzo Davis	Finance	38,000
150	Susan Martin	Marketing	38,500

Figure 17: Relational Database

This relation contains the following attributes describing employees: Emp_ID, Name, Dept, and Salary. The table contains five sample rows, corresponding to five employees.

You can express the structure of a relation by a shorthand notation in which the name of the relation is followed (in parentheses) by the names of the attributes in the relation.

The identifier attribute (called the primary key of the relation) is underlined. For example, you would express EMPLOYEE1 as follows: Employee (Emp_ID, Name, Dept, Salary)

Well-Structured Relations

What constitutes a well-structured relation (or table)? Intuitively, a well-structured relation contains a minimum amount of redundancy and allows users to insert, modify, and delete the rows in a table without errors or inconsistencies. EMPLOYEE1 (the above figure) is such a relation. Each row of the table contains data describing one employee, and any modification to an employee's data (such as a change in salary) is confined to one row of the table.

In contrast, EMPLOYEE2 (the below figure) contains data about employees and the courses they have completed. Each row in this table is unique for the combination of Emp_ID and Course, which becomes the primary key for the table. It is not a well-structured relation, however. If you examine the sample data in the table, you notice a considerable amount of redundancy. For example, the Emp_ID, Name, Dept, and Salary values appear in two separate rows for employees 100, 110, and 150. Consequently, if the salary for employee 100 changes, we must record this fact in two rows (or more, for some employees). The problem with this relation is that it contains data about two entities: EMPLOYEE and COURSE.

EMPLOYEE2

<u>Emp_ID</u>	Name	Dept	Salary	<u>Course</u>	Date_Completed
100	Margaret Simpson	Marketing	42,000	SPSS	6/19/2012
100	Margaret Simpson	Marketing	42,000	Surveys	10/7/2012
140	Alan Beeton	Accounting	39,000	Tax Acc	12/8/2012
110	Chris Lucero	Info Systems	41,500	SPSS	1/12/2012
110	Chris Lucero	Info Systems	41,500	C++	4/22/2012
190	Lorenzo Davis	Finance	38,000	Investments	5/7/2012
150	Susan Martin	Marketing	38,500	SPSS	6/19/2012
150	Susan Martin	Marketing	38,500	TQM	8/12/2012

Figure 18: Relational Database

You will learn to use principles of normalization to divide EMPLOYEE2 into two relations. One of the resulting relations is EMPLOYEE1 (previous figure). The other we will call EMP COURSE, which appears with sample data in the following figure. The primary key of this relation is the combination of Emp_ID and Course (we emphasize this by underlining the column names for these attributes).

EMP COURSE

<u>Emp_ID</u>	<u>Course</u>	Date_Completed
100	SPSS	6/19/2012
100	Surveys	10/7/2012
140	Tax Acc	12/8/2012
110	SPSS	1/22/2012
110	C++	4/22/2012
190	Investments	5/7/2012
150	SPSS	6/19/2012
150	TQM	8/12/2012

Figure 19: Relational Database

2.1.3 Physical Database Design

Designing physical databases requires certain information that should have been collected and produced during prior SDLC phases. This information includes:

- Normalized relations, including volume estimates
- Definitions of each attribute
- Descriptions of where and when data are used: entered, retrieved, deleted, and updated (including frequencies)
- Expectations or requirements for response time and data integrity

- Descriptions of the technologies used for implementing the database so that the range of required decisions and choices for each is known

Normalized relations are, of course, the result of logical database design. Statistics on the number of rows in each table, as well as the other information listed may have been collected during requirements determination in systems analysis. If not, these items need to be discovered to proceed with database design. We take a bottom-up approach to reviewing physical database design. Thus, we begin the physical design phase by addressing the design of physical fields for each attribute in a logical data model.

Designing Fields

A field is the smallest unit of application data recognized by system software, such as a programming language or database management system. An attribute from a logical database model may be represented by several fields. For example, a student name attribute in a normalized student relation might be represented as three fields: last name, first name, and middle initial. Each field requires a separate definition when the application system is implemented.

In general, you will represent each attribute from each normalized relation as one or more fields. The basic decisions you must make in specifying each field concern the type of data (or storage type) used to represent the field and data integrity controls for the field.

Choosing Data Types

A data type is a coding scheme recognized by system software for representing organizational data. The bit pattern of the coding scheme is usually immaterial to you, but the space to store data and the speed required to access data are of consequence in the physical file and database design.

The specific file or database management software you use with your system will dictate which choices are available to you. Selecting a data type balances four objectives that will vary in degree of importance for different applications:

- Minimize storage space
- Represent all possible values of the field
- Improve data integrity for the field
- Support all data manipulations desired on the field

You want to choose a data type for a field that minimizes space, represents every possible legitimate value for the associated attribute, and allows the data to be manipulated as needed. For example, suppose a —quantity sold field can be represented by a Number data type. You would select a length for this field that would handle the maximum value, plus some room for growth of the business. Further, the Number data type will restrict users from entering inappropriate values (text), but it does allow negative numbers (if this is a problem, application code or form design may be required to restrict the values to positive). Be careful—the data type must be suitable for the life of the application; otherwise, maintenance will be required.

Choose data types for future needs by anticipating growth. Also, be careful that date arithmetic can be done so that dates can be subtracted or time periods can be added to or subtracted from a date.

Example: Designing Database for furniture web store

The first step taken when designing the database for the Web Store was to review the conceptual data model—the entity-relationship diagram—developed during the analysis phase of the SDLC (see the following table). Given that the diagram contained no associative entities or many-to-many relationships, we began by identifying four distinct entity types that we named:

- CUSTOMER
- ORDER
- INVENTORY
- SHOPPING_CART

Once reacquainted with the conceptual data model, we examined the lists of attributes for each entity.

We noted that three types of customers were identified during conceptual data modeling, namely: corporate customers, home-office customers, and student customers. Yet, all were simply referred to as a —customer.¶ Nonetheless, because each type of customer had some unique information (attributes) that other types of customers did not, we created three additional entity types, or subtypes, of customers:

- CORPORATE
- HOME_OFFICE
- STUDENT

The following table lists the common and unique information about each customer type.

Common Information about ALL Customer Types		
Corporate Customer	Home-Office Customer	Student Customer
Customer ID	Customer ID	Customer ID
Address	Address	Address
Phone	Phone	Phone
E-mail	E-mail	E-mail
Unique Information about EACH Customer Type		
Corporate Customer	Home-Office Customer	Student Customer
Corporate name	Customer name	Customer name
Shipping method	Corporate name	School
Buyer name	Fax	
Fax		
*Having multiple "types" of an entity, with some sharing common attributes and each having unique attributes, is modeled in E-R diagrams as a subclass entity and is commonly referred to as an "is a" relationship (e.g., a customer is a corporate customer, a customer is a homeoffice customer, or a customer is a student customer). Please see a comprehensive database management text such as Hoffer, Ramesh, and Topi (2011) for more information on subclass entities and "is a" relationships.		

Table 12: Customer Types

As the above table implies, four separate relations are needed to keep track of customer information without having anomalies. The CUSTOMER relation is used to capture common attributes, whereas the additional relations are used to capture information unique to each distinct customer type. In order to identify the type of customer within the CUSTOMER relation easily, a Customer_Type attribute is added to the CUSTOMER relation. Thus, the CUSTOMER relation consists of:

CUSTOMER(Customer_ID, Address, Phone, E-mail, Customer_Type).

In order to link the CUSTOMER relation to each of the separate customer types—CORPORATE, HOME_OFFICE, and STUDENT—they all share the same primary key, Customer_ID, in addition to the attributes unique to each, which results in the following relations:

CORPORATE(Customer_ID, Corporate_Name, Shipping_Method, Buyer_Name, Fax)

HOME_OFFICE(Customer_ID, Customer_Name, Corporate_Name, Fax)

STUDENT(Customer_ID, Customer_Name, School).

In addition to identifying all the attributes for customers, we also identified the attributes for the other entity types. The results of this investigation are summarized in the following table.

Order	Inventory	Shopping_Cart
<u>Order_ID</u> (primary key)	<u>Inventory_ID</u> (primary key)	<u>Cart_ID</u> (primary key)
<u>Invoice_ID</u> (foreign key)	Name	<u>Customer_ID</u> (foreign key)
<u>Customer_ID</u> (foreign key)	Description	<u>Inventory_ID</u> (foreign key)
Return_Code	Size	Material
Order_Status	Weight	Color
	Materials	Quantity
	Colors	
	Price	
	Lead_Time	

Table 13: Physical Database

Much of the order-related information is captured and tracked within Purchasing Fulfillment System.

Therefore, the ORDER relation does not need to track all the details of the order because the Purchasing Fulfillment System produces a detailed invoice that contains all order details, such as the list of ordered products, materials used, colors, quantities, and other such information. In order to access this invoice information, a foreign key, Invoice_ID, is included in the ORDER relation. Additionally, to easily identify which orders belong to a specific customer, the Customer_ID attribute is also included in ORDER. Two additional attributes, Return_Code and

Order_Status, are also included in ORDER. The Return_Code is used to track the return of an order more easily—or a product within an order—whereas Order_Status is a code used to represent the state of an order as it moves through the purchasing fulfillment process. These attributes result in the following ORDER relation: ORDER(Order_ID, Invoice_ID, Customer_ID, Return_Code, Order_Status). In the INVENTORY entity, two attributes—Materials and Colors—could take on multiple values but were represented as single attributes. For example, Materials represents the range of materials from which a particular inventory item could be constructed. Likewise, Colors is used to represent the range of possible product colors. Finally, in addition to Cart_ID, each shopping cart contains the Customer_ID and Inventory_ID attributes so that each item in a cart can be linked to a particular inventory item and to a specific customer. In other words, both the Customer_ID and Inventory_ID attributes are foreign keys in the SHOPPING_CART relation. Recall that the SHOPPING_CART is temporary and is kept only while a customer is shopping.

When a customer actually places the order, the ORDER relation is created and the line items for the order—the items in the shopping cart—are moved to the Purchase Fulfillment System and stored as part of an invoice. Because we also need to know the selected material, color, and quantity of each item in the SHOPPING_CART, these attributes are included in this relation, which results in the following:

SHOPPING_CART(Cart_ID, Customer_ID, Inventory_ID, Material, Color, Quantity)

2.1.4 Normalization

In relational database, normalization is the process of restructuring the logical data model of a database to eliminate redundancy, organize data efficiently, and reduce repeating data and to reduce the potential for anomalies during data operations.

Data normalization also may improve data consistency and simplify future extension of the logical data model. The formal classifications used for describing a relational database's level of normalization are called normal forms.

The goal of normalization is to create a set of relational tables that are free of redundant data and that can be consistently and correctly modified. This means that all tables in a relational database should be in the third normal form (3NF). A relational table is in 3NF if and only if all non-key columns are (a) mutually independent and (b) fully dependent upon the primary key. Mutual independence means that no non-key column is dependent upon any combination of the other columns.

The first two normal forms are intermediate steps to achieve the goal of having all tables in 3NF. In order to better understand the 2NF and higher forms, it is necessary to understand the concepts of functional dependencies.

Functional Dependencies

The concept of functional dependencies is the basis for the first three normal forms. A column, Y, of the relational table R is said to be functionally dependent upon column X of R if and only if each value of X in R is associated with precisely one value of Y at any given time. Saying that

column Y is functionally dependent upon X is the same as saying the values of column X identify the values of column Y. If column X is a primary key, then all columns in the relational table R must be functionally dependent upon X.

$R.x \longrightarrow R.y$, which can be read as in the relational table named R, column x functionally determines (identifies) column y.

Full functional dependency applies to tables with composite keys. Column Y in relational table R is fully functional on X of R if it is functionally dependent on X and not functionally dependent upon any subset of X. Full functional dependence means that when a primary key is composite, made of two or more columns, then the other columns must be identified by the entire key and not just some of the columns that make up the key.

First Normal Form (1NF)

A relational table, by definition, is in first normal form, if all values of the columns are atomic. That is, they contain no repeating values. The following figure shows the table PRODUCT in 1NF.

PRODUCT

SId	Status	City	PId	Quantity
S1	2	Dire Dawa	P1	300
S1	2	Dire Dawa	P2	400
S1	2	Dire Dawa	P3	900
S2	5	Harar	P1	700
S2	5	Harar	P2	600
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.

Table 14: 1NF

Although the table PRODUCT is in 1NF it contains redundant data. For example, information about the supplier's location and the location's status has to be repeated for every part supplied. Redundancy causes what are called update anomalies. Update anomalies are problems that arise when information is inserted, deleted, or updated. For example, the following anomalies could occur in PRODUCT:

⌘INSERT. The fact that a certain supplier (S2) is located in a particular city (Addis Ababa) cannot be added until they supplied a product.

⌘DELETE. If a row is deleted, then not only is the information about quantity and product is lost but also information about the supplier.

⌘UPDATE. If supplier S1 moved from Dire Dawa to Bahir Dar, then three rows would have to be updated with this new information.

Second Normal Form (2NF)

A relation to be in its second normal form, every non-key column must be dependent upon the entire primary key. PRODUCT is in 1NF but not in 2NF because status and city are functionally dependent upon only on the column Sid of the composite key (Sid, Pid). This can be illustrated by listing the functional dependencies in the table:

Sid	—> city, status
city	—> status
(Sid,Pid)	—>qty

The process for transforming a 1NF table to 2NF is:

- Identify any determinants other than the composite key, and the columns they determine.
- Create and name a new table for each determinant and the unique columns it determines.
- Move the determined columns from the original table to the new table. The determinate becomes the primary key of the new table.
- Delete the columns you just moved from the original table except for the determinate which will serve as a foreign key.

To transform PRODUCT into 2NF we move the columns Sid, status, and city to a new table called SUPPLIER. The column Sid becomes the primary key of this new table. The results are shown below:

SUPPLIER

Sid	Status	city
S1	2	Dire Dawa
S2	5	Harar

PRODUCT

Pid	Sid	quantity
P1	S1	300
P1	S2	700
P2	S1	400
P2	S2	600
P3	S1	900

Table 15: Tables in 2NF

Tables in 2NF but not in 3NF still contain modification anomalies. In the example of SUPPLIER, they are:

⊖INSERT. The fact that a particular city has a certain status (DIRE DAWA has a status of 2) cannot be inserted until there is a supplier in the city.

⊖DELETE. Deleting any row in SUPPLIER destroys the status information about the city as well as the association between supplier and city.

Third Normal Form (3NF)

The third normal form requires that all columns in a relational table are dependent only upon the primary key. A more formal definition is:

A relational table is in third normal form (3NF) if it is already in 2NF and every non-key column is non-transitively dependent upon its primary key. In other words, all non-key attributes are functionally dependent only upon the primary key.

Table PRODUCT is already in 3NF. The non-key column, qty, is fully dependent upon the primary key (Sid,Pid). SUPPLIER is in 2NF but not in 3NF because it contains a transitive dependency. A transitive dependency occurs when a non-key column that is a determinant of the primary key is the determinate of other columns. The concept of a transitive dependency can be illustrated by showing the functional dependencies in SUPPLIER:

Sid $\rightarrow$ status

Sid $\rightarrow$ city

city $\rightarrow$ status

Note that status is determined both by the primary key Sid and the non-key column city. The process of transforming a table into 3NF is:

- Identify any determinants, other the primary key, and the columns they determine.
- Create and name a new table for each determinant and the unique columns it determines.
- Move the determined columns from the original table to the new table. The determinate becomes the primary key of the new table.
- Delete the columns you just moved from the original table except for the determinate which will serve as a foreign key.

To transform SUPPLIER into 3NF, we create a new table called CITY_STATUS and move the columns city and status into it. Status is deleted from the original table; city is left behind to serve as a foreign key to CITY_STATUS. The results are shown below.

SUPPLIER

Sid	city
S1	Dire Dawa
S2	Harar

CITY

City	Status
Dire Dawa	2
Harar	5

Table 16: Tables in 3NF

The result of putting the original table into 3NF has created three tables.

Advantages of Third Normal Form:

The advantage of having relational tables in 3NF is that it eliminates redundant data which in turn saves space and reduces manipulation anomalies. For example, the improvements to our sample database are:

- **INSERT:** Facts about the status of a city can be added even though there is not supplier in that city. Likewise, facts about new suppliers can be added even though they have not yet supplied parts.
- **DELETE:** Information about parts supplied can be deleted without destroying information about a supplier or a city.
- **UPDATE:** Changing the location of a supplier or the status of a city requires modifying only one row.

2.1.5 Designing Human (User) Interface

Analysts must complete two important activities in the systems design phase: designing the human interface and designing databases.

In this chapter, you learn guidelines to follow when designing the human computer interface. In

the first section, we describe the process of designing forms and reports and provide guidance on the deliverables produced during this process. Properly formatted segments of information are the building blocks for designing all forms and reports. We present guidelines for formatting information and for designing interfaces and dialogues. Next, we show you a method for representing human-computer dialogues called dialogue diagram. Finally, we close the chapter by examining various human-computer interface design issues for Internet-based applications.

Designing Forms and Reports

System inputs and outputs—forms and reports—are produced at the end of the systems analysis phase of the SDLC. During systems analysis, however, you may not have been concerned with the precise appearance of forms and reports. Instead, you focused on which forms and reports needed to exist and the content they needed to contain. You may have distributed to users the prototypes of forms and reports that emerged during analysis as a way to confirm requirements. Forms and reports are integrally related to the DFD diagrams developed during requirements structuring. For example, every input form is associated with a data flow entering a process on a DFD, and every output form or report is a data flow produced by a process on a DFD. Therefore, the contents of a form or report correspond to the data elements contained in the associated data flow. Further, the data on all forms and reports must consist of data elements in data stores for the application or else is computed from these data elements. A form is a business document containing some predefined data and often includes some areas where additional data are to be filled in. Most forms have a stylized format and are usually not in simple rows and columns. Examples of business forms are product order forms, employment applications, and class registration sheets.

Traditionally, forms have been displayed on a paper medium, but today, video display technology allows us to duplicate the layout of almost any printed form, including an organizational logo or any graphic, on a video display terminal. Forms on a video display may be used for data display or data entry. Additional examples of forms are an electronic spreadsheet, computer sign-in or menu, and an automated teller machine (ATM) transaction layout.

On the Internet, form interaction is the standard method of gathering and displaying information when consumers order products, request product information, or query account status.

A report is a business document containing only predefined data; it is a passive document used solely for reading or viewing. Examples of reports are invoices, weekly sales summaries by region and salesperson, and a pie chart of population by age categories. We usually think of a report as printed on paper, but it may be printed to a computer file, a visual display screen, or some other medium such as microfilm. Often a report has rows and columns of data, but a report may consist of any format—for example, mailing labels. Frequently, the differences between a form and a report are subtle. A report is only for reading and often contains data about multiple unrelated records in a computer file. On the other hand, a form typically contains data from only one record or is, at least, based on one record, such as data about one customer, one order, or one student.

Process of Designing Forms and Reports

Designing forms and reports is a user-focused activity that typically follows a prototyping

approach. First, you must gain an understanding of the intended user and task objectives during the requirements determination process. During this process, the intended user must answer several questions that attempt to answer who, what, when, where, and how related to the creation of all forms or reports, as listed in the following table. Gaining an understanding of these questions is a required first step in the creation of any form or report.

1. Who will use the form or report?
2. What is the purpose of the form or report?
3. When is the form or report needed and used?
4. Where does the form or report need to be delivered and used?
5. How many people need to use or view the form or report?

Table 17: Fundamental Questions When Designing Forms and Reports

Understanding the skills and abilities of the users helps you create an effective design. Are your users experienced computer users or novices? What is their educational level, business background, and task-relevant knowledge? Answers to these questions provide guidance for both the format and the content of your designs. Also, what is the purpose of the form or report? What task will users be performing, and what information is needed to complete this task? Other questions are also important to consider. Where will the users be when performing this task? Will users have access to online systems or will they be in the field? How many people will need to use this form or report?

If, for example, a report is being produced for a single user, the design requirements and usability assessment will be relatively simple. A design for a larger audience, however, may need to go through a more extensive requirements collection and usability assessment process. After collecting the initial requirements, you structure and refine this information into an initial prototype. Structuring and refining the requirements are completed without assistance from the users, although you may occasionally need to contact users to clarify some issue overlooked during analysis.

Finally, you ask users to review and evaluate the prototype; then they may accept the design or request that changes be made. If changes are needed, repeat the construction evaluate-refinement cycle until the design is accepted. Usually, several repetitions of this cycle occur during the design of a single form or report. As with any prototyping process, you should make sure that these iterations occur rapidly in order to gain the greatest benefit from this design approach. The initial prototype may be constructed in numerous environments, including Visual Basic, Java, or HTML. The following figure shows an example of data input form as designed in

Microsoft's Visual Basic.Net. Note the variety of fonts, sizes, and highlighting that was used. Online graphical tools for designing forms and reports are rapidly becoming the standard in most professional development organizations.

The screenshot shows a Windows-style application window titled "Customer Information Entry". In the top right corner, it displays "Today: 11-OCT-12". The main content area is titled "Customer Information" and contains a form with the following fields: "Customer Number:" with a dropdown menu showing "1273"; "Name:" with a text box containing "Contemporary Designs"; "Address:" with a text box containing "123 Oak Street"; "City:" with a text box containing "Austin"; "State:" with a text box containing "TX"; and "Zip:" with a text box containing "28384". At the bottom of the form are three buttons: "Save", "Help", and "Exit".

Figure 20: Example of Data Input Form

Deliverables and Outcomes

Each SDLC activity helps you to construct a system. In order to move from phase to phase, each activity produces some type of deliverable that is used in a later activity.

For example, within the systems planning and selection phase of the SDLC, the baseline project plan serves as input to many subsequent SDLC activities. In the case of designing forms and reports, design specifications are the major deliverables and are inputs to the system implementation and operation phase. Design specifications have three sections:

- Narrative overview
- Sample design
- Testing and usability assessment

Formatting Forms and Reports

A wide variety of information can be provided to users of information systems, ranging from text to video to audio. As technology continues to evolve, a greater variety of data types will be used. A definitive set of rules for delivering every type of information to users has yet to be defined because these rules are continuously evolving along with the rapid changes in technology.

Research conducted by computer scientists on human-computer interaction has provided numerous general guidelines for formatting information. Many of these guidelines undoubtedly will apply to the formatting of all evolving information types on yet-to-be-determined devices. Keep in mind that designing usable forms and reports requires your active interaction with users. If this single and fundamental activity occurs, you will likely create effective designs.

Guideline	Description
Use meaningful titles	Clear and specific titles describing content and use of form or report Revision date or code to distinguish a form or report from prior versions Current date that identifies when the form or report was generated Valid date that identifies on what date (or time) the data in the form or report were accurate
Include meaningful information	Only needed information displayed Information provided in a usable manner without modification
Balance the layout	Information balanced on the screen or page Adequate spacing and margins used All data and entry fields clearly labeled
Design an easy navigation system	Clearly show how to move forward and backward Clearly show where you are (e.g., page 1 of 3) Notify user of the last page of a multipage sequence

Table 18: Guidelines for Designing Forms and Reports

These guidelines reflect some of the general truths of formatting most types of information. The differences between a well-designed form or report and a poorly designed one often will be obvious. For example, the following figure (figure A) shows a poorly designed form for viewing account balance for a given customer. Whereas (figure B) is a better design, incorporating several general guidelines listed in the above table.

Figure 21: Poor Design and Better Design of Forms

Design A: Poor Design

Annotations:

- Vague title
- Difficult to read: information is packed too tightly
- No navigation information
- No summary of account activity

Design B: Better Design

Annotations:

- Easy to read: clear, balanced layout
- Clear title
- Summary of account information
- Clear navigation information

Table 1: Customer Information (Design A)

CUSTOMER INFORMATION	
CUSTOMER NO:	1273
NAME:	CONTEMPORARY DESIGNS
ADDRESS:	123 OAK ST.
CITY-STATE-ZIP:	AUSTIN, TX 78788
YTD-PURCHASE:	47,285.00
CREDIT LIMIT:	10,000.00
YTD-PAYMENTS:	42,556.85
DISCOUNT %:	5.0
PURCHASE:	21-JAN-12 22,000.00
PAYMENT:	21-JAN-12 13,000.00
PURCHASE:	02-MAR-12 10,000.00
PAYMENT:	02-MAR-12 15,500.00
PAYMENT:	23-MAY-12 5,000.00
PURCHASE:	12-JUL-12 9,285.00
PAYMENT:	12-JUL-12 3,785.00
PAYMENT:	21-SEP-12 6,371.85
STATUS:	ACTIVE

Table 2: Transaction Summary (Design B)

DATE	PURCHASE	PAYMENT	CURRENT BALANCE
01-Jan-12			0.00
21-Jan-12	(22,000.00)		(22,000.00)
21-Jan-12		13,000.00	(9,000.00)
02-Mar-12	(10,000.00)		(19,000.00)
02-Mar-12		15,500.00	(3,500.00)
23-May-12		5,000.00	(1,500.00)
12-Jul-12	(9,285.00)		(10,785.00)
12-Jul-12		3,785.00	(7,000.00)
21-Sep-12		6,371.85	(6,628.15)
YTD SUMMARY	(47,285.00)	42,556.85	(4,628.15)

Figure 21: Poor Design and Better Design of Forms

The first major difference between the two forms has to do with the title. The title in figure (A) (Customer Information) is ambiguous, whereas the title in figure (B) (Detail Customer Account Information) clearly and specifically describes the contents of the form. The form in figure (B) also includes the date (October 11, 2012) the form was generated so that, if printed, it will be clear to the reader when this occurred.

Figure (A) displays the account status and customer address, information that is extraneous to viewing the current account balance, which is the intent of the form and provides information that is not in the most useful format for the user. For example, figure (A) provides all customer data, as well as account transactions and a summary of year-to-date purchases and payments.

The form does not, however, provide the current outstanding balance of the account, leaving the reader to perform a manual calculation. The layout of information between the two forms also varies in balance and information density.

Gaining an understanding of the skills of the intended system users and the tasks they will be performing is invaluable when constructing a form or report. By following these general guidelines, your chances of creating effective forms and reports will be enhanced.

In the following sections, we discuss specific guidelines for highlighting information, displaying text, and presenting numeric tables and lists.

Highlighting Information

As display technologies continue to improve, a greater variety of methods will be available to highlight information. The following table lists the most commonly used methods for highlighting information.

Blinking and audible tones
Color differences
Intensity differences
Size differences
Font differences
Reverse video
Boxing
Underlining
All capital letters
Offsetting the position of nonstandard information

Table 19: Commonly Used Methods For Highlighting Information

Given this vast array of options, it is important to consider how highlighting can be used to enhance an output without being a distraction.

In general, highlighting should be used sparingly to draw the user to or away from certain information and to group together related information. In several situations, highlighting can be a valuable technique for conveying special information:

- Notifying users of errors in data entry or processing
- Providing warnings to users regarding possible problems, such as unusual data values or an unavailable device
- Drawing attention to keywords, commands, high-priority messages, and data that have changed or gone outside normal operating ranges

Highlighting techniques can be used singularly or in tandem, depending upon the level of emphasis desired by the designer. The figure below shows a form where several types of highlighting are used. In this example, columns clarify different categories of data; capital letters and different fonts distinguish labels from actual data; and bolding is used to draw attention to important data.



Figure 22: Information Highlighting Example

Highlighting should be used conservatively. For example, blinking and audible tones should be used only to highlight critical information requiring the user's immediate response. Once a response is made, these highlights should be turned off.

Additionally, highlighting methods should be consistently selected and used based upon the level of importance of the emphasized information. It is also important to examine how a particular highlighting method appears on all possible output devices that could be used with the system. For example, some color combinations may convey appropriate information on one display configuration but wash out and reduce legibility on another.

Recent advances in the development of graphical operating environments such as Windows, Mac OS, or Linux provide designers with some standard highlighting guidelines. However, because these guidelines are continuously evolving, they are often quite vague and leave a great deal of control in the hands of the systems developer.

To realize the benefits of using standard graphical operating environments—such as reduced user training time and interoperability among systems—you must be disciplined in how you use highlighting.

Displaying Text

In business-related systems, textual output is becoming increasingly important as text-based applications, such as electronic mail, blogs, and information services (e.g., Dow Jones Industrial Average stock index), are more widely used. The display and formatting of system help screens, which often contain lengthy textual descriptions and examples, is one example of textual data

that can benefit from following the simple guidelines that have emerged from systems design research. These guidelines appear in the following table.

Case	Display text in mixed upper- and lowercase and use conventional punctuation.
Spacing	Use double spacing if space permits. If not, place a blank line between paragraphs.
Justification	Left-justify text and leave a ragged right margin.
Hyphenation	Do not hyphenate words between lines.
Abbreviations	Use abbreviations and acronyms only when they are widely understood by users and are significantly shorter than the full text.

Table 20: Text Displaying Guidelines

When displaying textual information, you should also be careful not to hyphenate words between lines or use obscure abbreviations and acronyms. Users may not know whether the hyphen is a significant character if it is used to continue words across lines.

Information and terminology that are not widely understood by the intended users may significantly influence the usability of the system. Thus, you should use abbreviations and acronyms only if they are significantly shorter than the full text and are commonly known by the intended system users.

Designing Tables and Lists

Unlike textual information, where context and meaning are derived through reading, the context and meaning of tables and lists are derived from the format of the information. Consequently, the usability of information displayed in tables and alphanumeric lists is likely to be much more influenced by effective layout than most other types of information display. As with the display of textual information, tables and lists can also be greatly enhanced by following a few simple guidelines. These are summarized in the following table.

Guideline	Description
Use meaningful labels	All columns and rows should have meaningful labels. Labels should be separated from other information by using highlighting. Redisplay labels when the data extend beyond a single screen or page.
Format columns, rows, and text	Sort in a meaningful order (e.g., ascending, descending, or alphabetical). Place a blank line between every five rows in long columns. Similar information displayed in multiple columns should be sorted vertically (i.e., read from top to bottom, not left to right). Columns should have at least two spaces between them. Allow white space on printed reports for user to write notes. Use a single typeface, except for emphasis. Use same family of typefaces within and across displays and reports. Avoid overly fancy fonts.
Format numeric, textual, and alphanumeric data	Right-justify <i>numeric data</i> and align columns by decimal points or other delimiter. Left-justify <i>textual data</i>. Use short line length, usually 30 to 40 characters per line (this guideline is what newspapers use, and it is easy to speed-read). Break long sequences of <i>alphanumeric data</i> into small groups of three to four characters each.

Table 21: Guidelines for Designing Tables and Lists

When you design the display of numeric information, you must determine whether a table or a graph should be used. In general, tables are best when the user's task involves finding an individual data value from a larger data set, whereas line and bar graphs are more appropriate for analyzing data changes over time.

Designing Interfaces and Dialogues

Interface and dialogue design focuses on how information is provided to and captured from users. Dialogues are analogous to a conversation between two people. The grammatical rules followed by each person during a conversation are analogous to the human-computer interface. The design of interfaces and dialogues involves defining the manner in which humans and computers exchange information. A good human-computer interface provides a uniform structure for finding, viewing, and invoking the different components of a system.

Process of Designing Interfaces and Dialogues

Similar to designing forms and reports, the process of designing interfaces and dialogues is a user-focused activity. You follow a prototyping methodology of iteratively collecting information, constructing a prototype, assessing usability, and making refinements. To design usable interfaces and dialogues, you must answer the same who, what, when, where, and how questions used to guide the design of forms and reports. Thus, this process parallels that of designing forms and reports.

Deliverables and Outcomes

The deliverable and outcome from system interface and dialogue design is the creation of a design specification. This specification is similar to the specification produced for form and report designs—with one exception. Recall that the design specification for forms and reports had three sections:

- ⌘ Narrative overview
- ⌘ Sample design
- ⌘ Testing and usability assessment

For interface and dialogue designs, one additional subsection is included: a section outlining the dialogue sequence—the ways a user can move from one display to another. Later in this section you will learn how to design a dialogue sequence by using dialogue diagramming. An outline for a design specification for interfaces and dialogues is shown in the following figure.

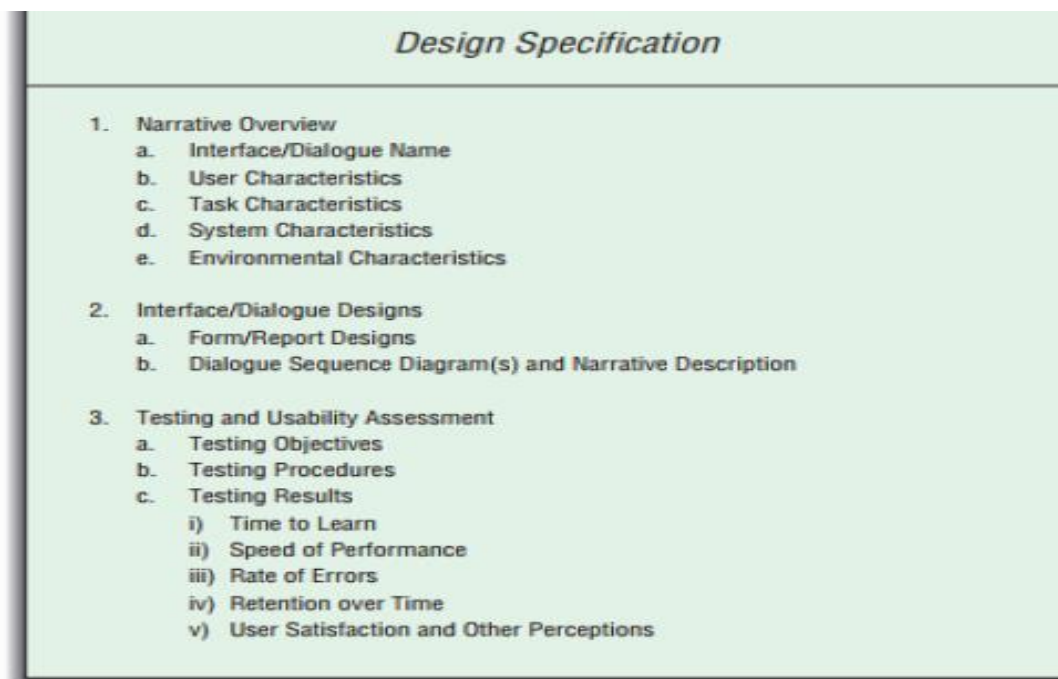


Figure 23: Design Specifications for Interfaces and Dialogues

Designing Interfaces

In this section we discuss the design of interface layouts. This discussion provides guidelines for structuring and controlling data-entry fields, providing feedback, and designing online help. Effective interface design requires you to gain a thorough understanding of each of these concepts.

Designing Layouts

In many organizations, data are often first recorded on paper-based forms and then later recorded within application systems. When designing layouts to record or display information on paper-based forms, try to make both as similar as possible. Additionally, data-entry displays should be

consistently formatted across applications to speed data entry and reduce errors. The below figure shows an example.

Customer Order Report Today: 11-OCT-12
Order Number: 913-A36-98

Customer Information

Customer Number: 1273
Name: Contemporary Designs
Address: 123 Oak Street
City: Austin
State: TX
Zip: 28384

PRODUCT NUMBER	DESCRIPTION	QUANTITY ORDERED	UNIT PRICE	TOTAL PRICE
M125	Bookcase	4	200.00	800.00
B381	Cabinet	2	150.00	300.00
B210	Table	1	500.00	500.00
G200	Deluxe Chair	8	400.00	3,200.00

TOTAL ORDER AMOUNT 4,800.00
5% DISCOUNT 240.00
TOTAL AMOUNT DUE 4560.00

Buttons: Help, Print (password required), Select Customer or Exit

Figure 24: Designing Layouts

The design of between-field navigation is another item to consider when designing the layout of computer-based forms. Because you can control the sequence for users to move between fields, standard screen navigation should flow from left-to-right and top-to-bottom. When designing the navigation procedures within your system, flexibility and consistency are primary concerns. Users should be able to move freely forward and backward or to any desired data-entry fields. Users should be able to navigate each form in the same way or in as similar a manner as possible. Additionally, data should not usually be permanently saved by the system until the user makes an explicit request to do so. This option allows the user to abandon a data-entry screen, back up, or move forward without adversely impacting the contents of the permanent data.

Consistency extends to the selection of keys and commands. Assign each key or command

Cursor-Control Capabilities

- Move the cursor forward to the next data field.
- Move the cursor backward to the previous data field.
- Move the cursor to the first, last, or some other designated data field.
- Move the cursor forward one character in a field.
- Move the cursor backward one character in a field.

Editing Capabilities

- Delete the character to the left of the cursor.
- Delete the character under the cursor.
- Delete the whole field.
- Delete data from the whole form (empty the form).

Exit Capabilities

- Transmit the screen to the application program.
- Move to another screen/form.
- Confirm the saving of edits or go to another screen/form.

Help Capabilities

- Get help on a data field.
- Get help on a full screen/form.

Source: Based on J. S. Dumas (1988). *Designing User Interfaces for Software*. Upper Saddle River, NJ: Prentice Hall.

Figure 25: Checklist for Validating Usability of User Interface

only one function.

This assignment should be consistent throughout the entire system and across systems, if possible. Depending upon the application, various types of functional capabilities will be required to provide smooth navigation and data entry. For example, a good interface design provides a consistent way for moving the cursor to different places on the form, editing characters and fields, moving among form displays, and obtaining help. These functions may be provided by keystrokes, mouse, menu, or function keys

Structuring Data Entry

You should consider several guidelines when structuring data-entry fields on a form. The first is simple, yet, is often violated by designers. To minimize data-entry errors and user frustration, never require the user to enter information that is already available within the system or information that can be easily computed by the system.

For example, never require the user to enter the current date and time, because each of these values can be easily retrieved from the computer system's internal calendar and clock.

By allowing the system to do these tasks, the user simply confirms that the calendar and clock are working properly.

Other guidelines are equally important. For example, suppose that a bank customer is repaying a loan on a fixed schedule with equal monthly payments. Each month when a payment is sent to the bank, a clerk needs to record that the payment has been received into a loan-processing system. Within such a system, default values for fields should be provided whenever appropriate, which allows the clerk to enter specific data into the system only when the customer pays more or less than the scheduled amount. In all other cases, the clerk simply verifies that the check is for the default amount provided by the system and presses a single key to confirm the receipt of payment.

When entering data, do not require the user to specify the dimensional units of a particular value, for example, whether an amount is in dollars or a weight is in tons. Use field formatting and the data-entry prompt to make clear the type of data being requested. In other words, place a caption describing the data to be entered adjacent to each data field so that the user knows what type of data is being requested. As with the display of information, all data entered onto a form should automatically justify in a standard format (e.g., date, time, money).

Controlling Data Input

One objective of interface design is to reduce data-entry errors. As data are entered into an information system, steps must be taken to ensure that the input is valid. As a systems analyst, you must anticipate the types of errors users may make and design features into the system's interfaces to avoid, detect, and correct data-entry mistakes. Several types of data errors are summarized in the following table. Data errors can occur from appending extra data onto a field, truncating characters off a field, transcribing the wrong characters into a field, or transposing one or more characters within a field.

Data Error	Description
Appending	Adding additional characters to a field
Truncating	Losing characters from a field
Transcripting	Entering invalid data into a field
Transposing	Reversing the sequence of one or more characters in a field

Table 22: Types of Data Errors

Systems designers have developed numerous tests and techniques for detecting invalid data before saving or transmission, thus improving the likelihood that data will be valid. The below table summarizes these techniques.

These tests and techniques are often incorporated into both data-entry screens and when data are transferred from one computer to another.

Validation Test	Description
Class or composition	Test to ensure that data are of proper type (e.g., all numeric, all alphabetic, alphanumeric)
Combinations	Test to see that value combinations of two or more data fields are appropriate or make sense (e.g., does the quantity sold make sense given the type of product?)
Expected values	Test to see whether data are what is expected (e.g., match with existing customer names, payment amount, etc.)
Missing data	Test for existence of data items in all fields of a record (e.g., is there a quantity field on each line item of a customer order?)
Pictures/templates	Test to ensure that data conform to a standard format (e.g., are hyphens in the right places for a student ID number?)
Range	Test to ensure data are within a proper range of values (e.g., is a student's grade-point average between 0 and 4.0?)
Reasonableness	Test to ensure data are reasonable for situation (e.g., pay rate for a specific type of employee)
Self-checking digits	Technique by which extra digits, derived using a standard formula (see Figure 8-15), are added to a numeric field before transmission and checked after transmission
Size	Test for too few or too many characters (e.g., is social security number exactly nine digits?)
Values	Test to make sure values come from a set of standard values (e.g., two-letter state codes)

Table 23: Techniques Used by Systems Designers to Detect Data Error: Transmission

Correcting erroneous data is much easier to accomplish before it is permanently stored in a system. Online systems can notify a user of input problems as data are being entered. When data are processed online as events occur, it is much less likely that data-validity errors will occur and not be caught. In an online system, most problems can be easily identified and resolved before permanently saving data to a storage device using many of the techniques

described in the above table. However, in systems where data inputs are stored and entered (or transferred) in batches, the identification and notification of errors are more difficult.

Batch processing systems can, however, reject invalid inputs and store them in a log file for later resolution. Most of the straightforward tests and techniques shown in the above table are widely used. Some can be handled by data-management technologies, such as a database management system (DBMS), to ensure that they are applied for all data maintenance operations. If a DBMS cannot perform these tests, then you must design the tests into program modules.

In addition to validating the data values entered into a system, controls must be established to verify that all input records are correctly entered and processed only once. A common method used to enhance the validity of entering batches of data records is to create an audit trail of the entire sequence of data entry, processing, and storage. In such an audit trail, the actual sequence, count, time, source location, and human operator are recorded in a separate transaction log in the event of a data input or processing error. If an error occurs, corrections can be made by reviewing the contents of the log. Detailed logs of data inputs not only are useful for resolving batch data-entry errors and system audits, but also serve as a powerful method for performing backup and recovery operations in the case of a catastrophic system failure. Audit trail is a record of the sequence of data entries and the date of those entries.

Providing Feedback

When you talk with friends, you expect them to give you feedback by nodding and replying to your questions and comments. Without feedback, you would be concerned that they were not listening. Similarly, when designing system interfaces, providing appropriate feedback is an easy way to make a user's interaction more enjoyable; not providing feedback is a sure way to frustrate and confuse. System feedback can consist of three types:

- Status information
- Prompting cues
- Error and warning messages

Status Information: Providing status information is a simple technique for keeping users informed of what is going on within a system. For example, relevant status information, such as displaying the current customer name or time, placing appropriate titles on a menu or screen, and identifying the number of screens following the current one (e.g., Screen 1 of 3), all provide needed feedback to the user. Providing status information during processing operations is especially important if the operation takes longer than a second or two.

For example, when opening a file, you might display, —Please wait while I open the file, or when performing a large calculation, flash the message —Working . . . to the user. Further, it is important to tell the user that besides working, the system has accepted the user's input and the input was in the correct form. Sometimes it is important to give the user a chance to obtain more feedback. For example, a function key could toggle between showing a —Working . . . message and giving more specific information as each intermediate step is accomplished. Providing status information reassures users that nothing is wrong and makes them feel in command of the system, not vice versa.

Prompting Cues: A second feedback method is to display prompting cues. When prompting the

user for information or action, it is useful to be specific in your request. For example, suppose a system prompted users with the following request: READY FOR INPUT: _ with such a prompt, the designer assumes that the user knows exactly what to enter. A better design would be specific in its request, possibly providing an example, default values, or formatting information. An improved prompting request might be as follows: Enter the customer account number (123-456-7):_

Error and Warning Messages: A final method available to you for providing system feedback is using error and warning messages. Following a few simple guidelines can greatly improve the usefulness of these messages. First, make messages specific and free of error codes and jargon. Additionally, messages should never scold the user but attempt to guide the user toward a resolution. For example, a message might say, —No customer record found for that customer ID. Please verify that digits were not transposed.¶ Messages should be in user, not computer, terms. Terms such as end of file, disk I/O error, or write protected may be too technical and not helpful for many users. Multiple messages can be useful so that a user can get more detailed explanations if wanted or needed.

Also, make sure error messages appear in roughly the same format and placement each time so that they are recognized as error messages and not as some other information.

Providing Help

Designing a help system is one of the most important interface design issues you will face. When designing help, you need to put yourself in the user's place. When accessing help, the user likely does not know what to do next, does not understand what is being requested, or does not know how the requested information needs to be formatted.

A user requesting help is much like a ship in distress, sending an SOS. Our first guideline, simplify, suggests that help messages should be short, to the point, and use words that users can understand. The second guideline, organize, means that the information in help messages should be easy for users to absorb. Long paragraphs of text are often difficult for people to understand. A better design organizes lengthy information in a manner easier for users to digest through the use of bulleted and ordered lists. Finally, it is often useful to explicitly show users how to perform an operation and the outcomes of procedural steps.

Many systems are also designed so that users can vary the level of detail provided. Help may be provided at the system level, screen or form level, and individual field level. The ability to provide field-level help is often referred to as context-sensitive help. For some applications, providing context-sensitive help for all system options is a tremendous undertaking that is virtually a project in itself. If you do decide to design an extensive help system with many levels of detail, you must be sure that you know exactly what the user needs help with, or your efforts may confuse users more than help them.

After leaving a help screen, users should always return back to where they were prior to requesting help. If you follow these simple guidelines, you will likely design a highly usable help system.

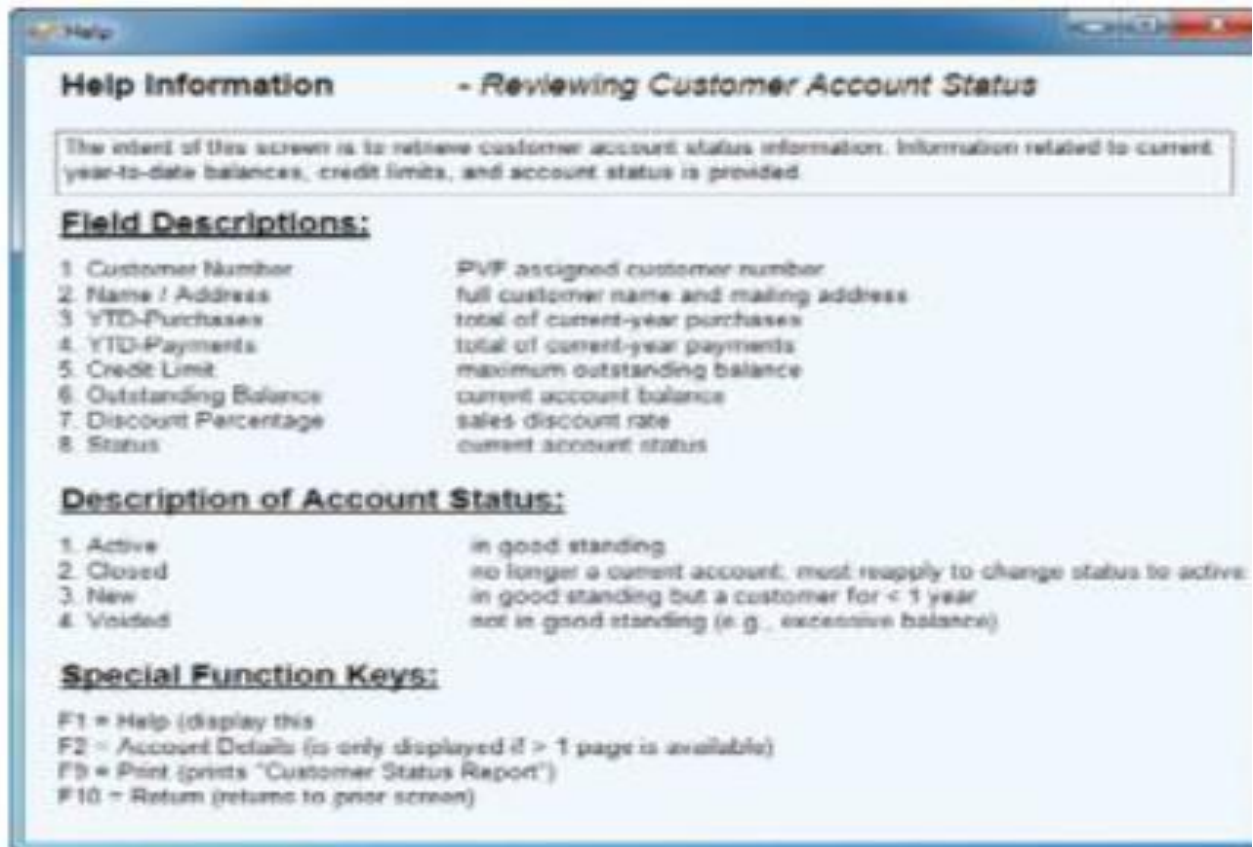


Figure 26: Designing Help Screens

As with the construction of menus, many programming environments provide powerful tools for designing system help. For example, Microsoft's HTML Help SDK allows you to construct hypertext-based help systems quickly. In this environment, you use a text editor to construct help pages that can be easily linked to other pages containing related or more specific information. Linkages are created by embedding special characters into the text document that make words hypertext buttons—that is, direct linkages—to additional information. The HTML Help SDK transforms the text document into a hypertext document. For example, the following figure

shows a hypertext-based help screen from Microsoft.



Figure 27: Designing Help Screens

Hypertext-based help systems have become the standard environment for most commercial operating environments for two primary reasons. First, standardizing system help across applications eases user training. Second, hypertext allows users to selectively access the level of help they need, making it easier to provide effective help for both novice and experienced users within the same system.

Designing Dialogues

The process of designing the overall sequences that users follow to interact with an information system is called dialogue design. A dialogue is the sequence in which information is displayed to and obtained from a user. As with other design processes, designing dialogues is a three-step process:

- ⌘ Designing the dialogue sequence
- ⌘ Building a prototype
- ⌘ Assessing usability

Designing the Dialogue Sequence

Your first step in dialogue design is to define the sequence. In other words, you must have a clear understanding of the user, task, technological, and environmental characteristics when designing dialogues.

As a designer, once you understand how a user wishes to use a system, you can then transform these activities into a formal dialogue specification. A method for designing and representing dialogues is dialogue diagramming. Dialogue diagrams, illustrated in the following figure, have only one symbol, a box with three sections; each box represents one display (which might be a

full screen or a specific form or window) within a dialogue. The three sections of the box are used as follows:

- Top: Contains a unique display reference number used by other displays for referencing it
- Middle: Contains the name or description of the display
- Bottom: Contains display reference numbers that can be accessed from the current display

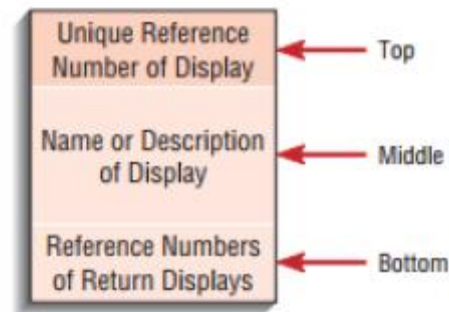


Figure 28: Dialogue Sequence

All lines connecting the boxes within dialogue diagrams are assumed to be bidirectional and, thus, do not need arrowheads to indicate direction.

With this capability, users are allowed to always move forward and backward between adjacent displays. If you desire only unidirectional flows within a dialogue, arrowheads should be placed at one end of the line.

Within a dialogue diagram, you can easily represent the sequencing of displays, the selection of one display over another, or the repeated use of a single display (e.g., a data-entry display).

These three concepts—sequence, selection, and iteration—are illustrated in the below figure.

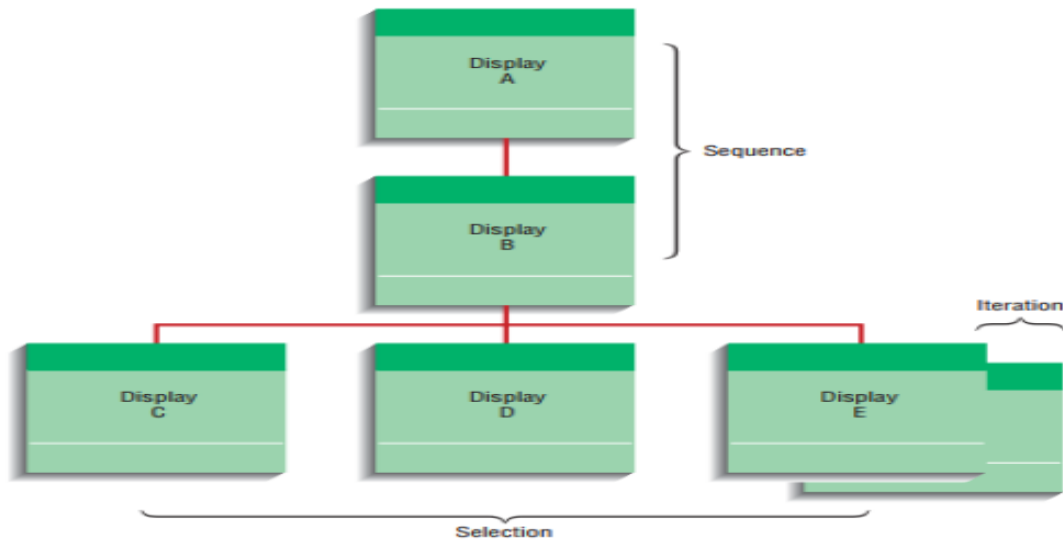


Figure 29: Dialogue Sequence

Example

The below figure shows a partial dialogue diagram for processing the marketing manager's request. In this diagram, the analyst placed the request to view year-to-date customer information within the context of the overall Customer Information System. The user must first gain access to the system through a log-on procedure (item 0). If log-on is successful, a main menu is displayed that has four items (item 1). Once the user selects the Individual Customer Information display (item 2), control is transferred to the Select Customer display (item 2.1). After a customer is selected, the user is presented with an option to view customer information four different ways (item 2.1.1).

Once the user views the customer's year-to-date transaction activity (item 2.1.1.2), the system will allow the user to back up to select a different customer or back up to the main menu (see bottom of item 2.1.1.2).

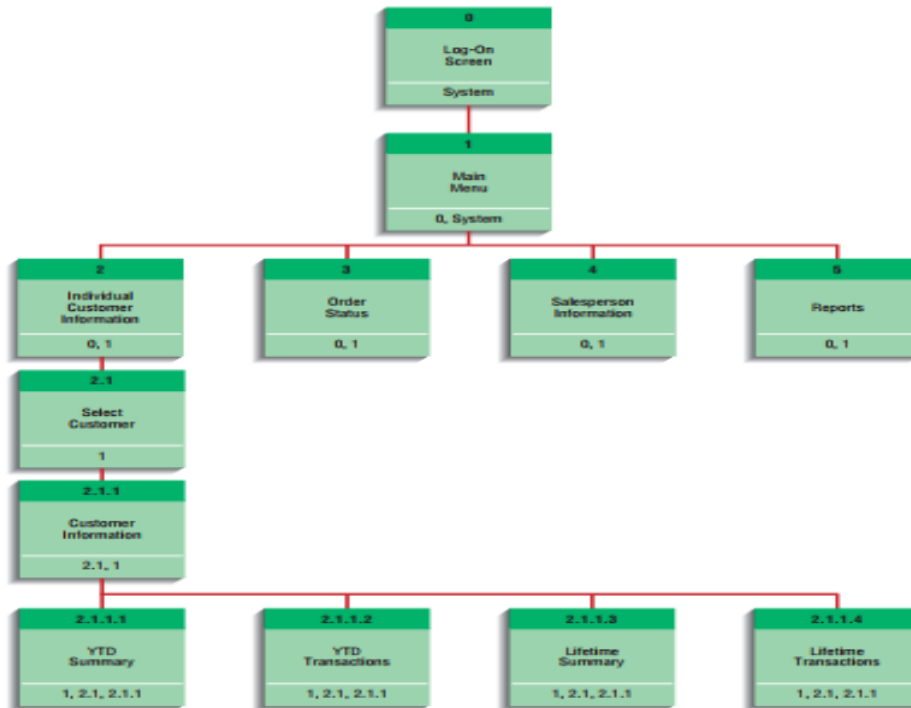


Figure 30: User Interface Flow Diagram

Building Prototypes and Assessing Usability

Building dialogue prototypes and assessing usability are often optional activities. Some systems may be simple and straightforward. Others may be more complex but are extensions to existing systems where dialogue and display standards have already been established. In either case, you may not be required to build prototypes and do a formal assessment. However, for many other systems, it is critical that you build prototype displays and then assess the dialogue; developing a prototype can pay numerous dividends later in the systems development life cycle (e.g., it may be easier to implement a system or train users on a system they have already seen and used).

Several tools called —Prototypers or —Demo Builders[¶] allow you to design displays quickly and show how an interface will work within a full system. These demo systems allow users to enter data and move through displays as if they were using the actual system. Such activities are useful not only for showing how an interface will look and feel but also for assessing usability and performing user training long before actual systems are completed.

UNIT - III

3.1 System Implementation

System implementation and maintenance is made up of seven major activities:

- Coding
- Testing
- Installation
- Documentation
- Training
- Support
- Maintenance

The purpose of these steps is to convert the final physical system specifications into working and reliable software and hardware, document the work that has been done, and provide help for current and future users and caretakers of the system. Usage of the system leads to changes, so during maintenance, users and others submit maintenance requests; requests are transformed into specific changes to the system; the system is redesigned to accept the changes; and the changes are implemented.

These steps are often done by other project team members besides analysts, although analysts may do some programming and other steps. Often a separate analyst and developer team from those who developed the original system is responsible for testing, documenting, training, and maintenance activities. In any case, analysts are responsible for ensuring that all of these various activities are properly planned and executed. We briefly discuss these activities in three groups:

1. Activities that lead to the system going into operation—coding, testing, and installation
2. Activities those are necessary for successful system operation—documenting the system and training and supporting users
3. Activities that are ongoing and needed to keep the system working and up to date—maintenance

Processes of Coding, Testing, and Installation

Coding, as we mentioned before, is the process through which the physical design specifications created by the design team are turned into working computer code by the programming team. Depending on the size and complexity of the system, coding can be an involved, intensive activity. Once coding has begun, the testing process can begin and proceed in parallel. As each program module is produced, it can be tested individually, then as part of a larger program, and then as part of a larger system. You learn about the different strategies for testing later in the chapter. We should emphasize that although testing is done during implementation, you must

begin planning for testing earlier in the project. Planning involves determining what needs to be tested and collecting test data. These activities are often done during the analysis phase, because testing requirements are related to system requirements.

Installation is the process during which the current system is replaced by the new system. It includes conversion of existing data, software, documentation, and work procedures to those consistent with the new system. Users must give up the old ways of doing their jobs, whether manual or automated, and adjust to accomplishing the same tasks with the new system. Users will sometimes resist these changes, and you must help them adjust. However, you cannot control all the dynamics of user-system interaction involved in the installation process.

Deliverables and Outcomes from Coding, Testing, and Installation

The following table shows the deliverables from the coding, testing, and installation processes. The most obvious outcome is the code itself, but just as important as the code is documentation of the code. Modern programming languages, such as Visual Basic, are said to be largely self-documenting. When standard naming and program design conventions are used, the code itself spells out much about the program's logic, the meaning of data and variables, and the locations where data are accessed and output. But even well-documented codes can be mysterious to maintenance programmers who must maintain the system for years after the original system was written and the original programmers have moved on to other jobs. Therefore, clear, complete documentation for all individual modules and programs is crucial to the system's continued smooth operation.

Action	Deliverable
Coding	Code
	Program documentation
Testing	Test scenarios (test plan) and test data
	Results of program and system testing
Installation	User guides
	User training plan
	Installation and conversion plan
	Hardware and software installation schedule
	Data conversion plan
	Site and facility remodeling plan

Table 24: Deliverables and Outcomes from Coding, Testing, and Installation

Increasingly, automated tools are used to maintain the documentation needed by systems professionals. The results of program and system testing are important deliverables from the testing process because they document the tests, as well as the test results. For example, what type of test was conducted? What test data were used? How did the system handle the test? The answers to these questions can provide important information for system maintenance as changes

require retesting, and similar testing procedures will be used during the maintenance process. The next two deliverables, user guides and the user training plan, result from the installation process. User guides provide information on how to use the new system, and the training plan is a strategy for training users so they can quickly learn the new system. The development of the training plan probably began earlier in the project, and some training on the concepts behind the new system may have already taken place. During the early stages of implementation, the training plans are finalized and training on the use of the system begins. Similarly, the installation plan lays out a strategy for moving from the old system to the new. Installation includes installing the system (hardware and software) at central and user sites. The installation plan answers such questions as when and where the new system will be installed, what people and resources are required, which data will be converted and cleansed, and how long the installation process will take. It is not enough that the system is installed; users must actually use it.

As an analyst, your job is to ensure that all of these deliverables are produced and done well, whether by you or by others.

Coding, testing, and installation work may be done by IS professionals in your organization, by contractors, hardware designers, and, increasingly, by users. The extent of your responsibilities will vary according to the size and standards of the organization you work for, but your ultimate role includes ensuring that all the coding, testing, and installation work leads to a system that meets the specifications developed in earlier project phases.

Processes of Documenting the System, Training Users, and Supporting Users

Although the process of documentation proceeds throughout the life cycle, it receives formal attention now, because once the system is installed, the analysis team's involvement in system development usually ceases. As the team is getting ready to move on to new projects, you and the other analysts need to prepare documents that reveal all of the important information you have learned about this system during its development and implementation. The two audiences for this final documentation are (1) the information systems personnel who will maintain the system throughout its productive life, and (2) the people who will use the system as part of their daily lives.

Larger organizations also tend to provide training and support to computer users throughout the organization, sometimes as part of a corporate university. Some of the training and support is directed to off-the-shelf software packages. For example, it is common to find courses on Microsoft Windows and Office in organization-wide training facilities. Analysts typically work with corporate trainers to provide training and support tailored to particular computer applications they have helped develop.

Centralized information system training facilities tend to have specialized staff that can help with training and support issues. In smaller organizations that cannot afford to have well-staffed centralized training and support facilities, fellow users are the best source of training and support that users have, whether the software is customized or off the shelf.

Deliverables and Outcomes from Documenting the System, Training Users, and Supporting Users

The following table shows the deliverables from documenting the system, training users, and supporting users. User documentation can be paper based, but it should also include computer-based modules. For modern information systems, this documentation includes any online help designed as part of the system interface.

The development team should think through the user training process: Who should be trained? How much training is adequate for each training audience? What do different types of users need to learn during training? The training plan should be supplemented by actual training modules, or at least outlines of such modules, that at a minimum address the three questions stated previously. Finally, the development team should also deliver a user support plan that addresses such issues as how users will be able to find help once the information system has become integrated into the organization. The development team should consider a multitude of support mechanisms and modes of delivery. Each deliverable is addressed in more detail later in the chapter.

Documentation	User Training Modules
System documentation	Training materials
User documentation	Computer-based training aids
User Training Plan	User Support Plan
Classes	Help desk
Tutorials	Online help
	Bulletin boards and other support mechanisms

Table 25: Deliverables from Documentation, User Training and Support

Process of Maintaining Systems

Throughout this module, we have drawn the systems development life cycle as a circle where one phase leads to the next, with overlap and feedback loops. This means that the process of maintaining an information system is the process of returning to the beginning of the SDLC and repeating development steps, focusing on the needs for system change, until the change is implemented.

Four major activities occur within maintenance:

1. Obtaining maintenance requests
2. Transforming requests into changes
3. Designing changes
4. Implementing changes

3.1.1 Software Testing

Testing software begins earlier in the systems development life cycle, even though many of the actual testing activities are carried out during implementation. During analysis, you develop an overall test plan. During design, you develop a unit test plan, an integration test plan, and a system test plan. During implementation, these various plans are put into effect, and the actual testing is performed. The purpose of these written test plans is to improve communication among all the people involved in testing the application software. The plan specifies what each person's role will be during testing. The test plans also serve as checklists you can use to determine whether all testing steps have been completed. The overall test plan is not just a single document but is a collection of documents. Each of the component documents represents a complete test plan for one part of the system or for a particular type of test. Some organizations have specially trained personnel who supervise and support testing. Testing managers are responsible for developing test plans, establishing testing standards, integrating testing and development activities in the life cycle, and ensuring that test plans are completed. Testing specialists help develop test plans, create test cases and scenarios, execute the actual tests, and analyze and report test results.

Types of Testing

Software application testing is an umbrella term that covers several types of tests. Tests can be done with or without executing the code, and they may be manual or automated. Using this framework, we can categorize types of tests as shown in the following table.

	Manual	Automated
Without code execution	Inspections	Syntax checking
With code execution	Walkthroughs	Unit testing
	Desk checking	Integration testing
		System testing
		Stub testing

Table 26: Types of Testing

Let's examine each type of test in turn. Inspections are formal group activities in which participants manually examine code for occurrences of well-known errors. Syntax, grammar, and some other routine errors can be checked in early stages of coding by automated inspection software, so manual inspection checks are used for more subtle errors.

Code inspection participants compare the code they are examining to a checklist of well-known errors for that particular language.

Exactly what the code does is not investigated in an inspection. Code inspections have been used by organizations to detect from 60 to 90 percent of all software defects, as well as to provide programmers with feedback that enables them to avoid making the same types of errors in future

work. The inspection process can also be used to ensure that design specifications are accomplished.

Unlike inspections, what the code does is an important question in a walkthrough. Using structured walkthroughs is an effective method of detecting errors in code. Structured walkthroughs can be used to review many systems development deliverables, including design specifications and code. Whereas specification walkthroughs tend to be formal reviews, code walkthroughs tend to be informal. Informality makes programmers less apprehensive of criticism and, thus, helps increase the frequency of walkthroughs. Code walkthroughs should be done frequently when the pieces of work reviewed are relatively small and before the work is formally tested. If walkthroughs are not held until the entire program is tested, the programmer will have already spent too much time looking for errors that the programming team could have found much more quickly. Further, the longer a program goes without being subjected to a walkthrough, the more defensive the programmer becomes when the code is reviewed. Although each organization that uses walkthroughs conducts them differently, you can follow a basic structure that works well.

It should be stressed that the purpose of a walkthrough is to detect errors, not to correct them. It is the programmer's job to correct the errors uncovered in a walkthrough. Sometimes it can be difficult for the reviewers to refrain from suggesting ways to fix the problems they find in the code, but increased experience with the process can help change reviewers' behavior.

What the code does is also important in desk checking, an informal process where the programmer or someone else who understands the logic of the program works through the code with a paper and pencil. The programmer executes each instruction, using test cases that may or may not be written down. In one sense, the reviewer acts as the computer, mentally checking each step and its results for the entire set of computer instructions.

Syntax checking is typically done by a compiler. Errors in syntax are uncovered, but the code is not executed. For the other three automated techniques, the code is executed.

The first such technique is **unit testing**, sometimes called module or functional testing. In unit testing, each module (roughly a section of code that performs a single function) is tested alone in an attempt to discover any errors that may exist in the module's code. Yet, because modules coexist and work with other modules in programs and systems, they must be tested together in larger groups. Combining modules and testing them is called integration testing. Integration testing is gradual. First you test the highest level, or coordinating module, and only one of its subordinate modules. The process assumes a typical structure for a program, with one highest-level, or main, module and various subordinate modules referenced from the main module. Each subordinate module may have a set of modules subordinate to it, and so on, similar to an organization chart. Next, you continue testing subsequent modules at the same level until all subordinate to the highest-level module have been successfully tested together. Once the program has been tested successfully with the high-level module and all of its immediate subordinate modules, you add modules from the next level one at a time. Again, you move forward only when tests are successfully completed. If an error occurs, the process stops, the error is identified and corrected, and the test is redone. The process repeats until the entire program—all modules at all levels—is successfully integrated and tested with no errors.

System testing is a similar process, but instead of integrating modules into programs for testing, you integrate programs into systems. System testing follows the same incremental logic that integration testing does.

Under both integration and system testing, not only do individual modules and programs get tested many times, so do the interfaces between modules and programs. Current practice (as previously outlined) calls for a top-down approach to writing and testing modules. Under a top-down approach, the coordinating module is written first. Then the modules at the next level are written, followed by the modules at the next level, and so on, until all of the modules in the system are done. Each module is tested as it is written. Because top-level modules contain many calls to subordinate modules, you may wonder how they can be tested if the lower-level modules haven't been written yet. The answer is **stub testing**. Stubs are two or three lines of code written by a programmer to stand in for the missing modules. During testing, the coordinating module calls the stub instead of the subordinate module. The stub accepts control and then returns it to the coordinating module.

System testing is more than simply expanded integration testing, where you are testing the interfaces between programs in a system rather than testing the interfaces between modules in a program. System testing is also intended to demonstrate whether a system meets its objectives. The system test is typically conducted by information systems personnel led by the project team leader, although it can also be conducted by users under the guidance of information systems personnel.

Testing Process

Up to this point, we have talked about an overall test plan and seven different types of tests for software applications. We haven't said much about the process of testing itself. Two important things to remember about testing information systems are:

- The purpose of testing is confirming that the system satisfies requirements.
- Testing must be planned.

Testing is not haphazard. You must pay attention to many different aspects of a system, such as response time, response to extreme data values, response to no input, response to heavy volumes of input, and so on. You must test anything (within resource constraints) that could go wrong or be wrong with a system.

At a minimum, you should test the most frequently used parts of the system and as many other paths through the system as time permits.

Planning gives analysts and programmers an opportunity to think through all the potential problem areas, list these areas, and develop ways to test for problems. As indicated previously, one part of a test plan is creating a set of test cases, each of which must be carefully documented. A test case is a specific scenario of transactions, queries, or navigation paths that represent a typical, critical, or abnormal use of the system. A test case should be repeatable so that it can be

rerun as new versions of the software are tested. This is important for all codes, whether written in-house, developed by a contractor, or purchased. Test cases need to determine that new software works with other existing software with which it must share data. Even though analysts often do not do the testing, systems analysts, because of their intimate knowledge of applications, often make up or find test data. The people who create the test cases should not be the same people who coded and tested the system. In addition to a description of each test case, there must also be a summary of the test results, with an emphasis on how the actual results differed from the expected results.

The testing summary will indicate why the results were different and what, if anything, should be done to change the software. Further, this summary will then suggest the need for retesting, possibly introducing new tests necessary to discover the source of the differences. One important reason to keep such a thorough description of test cases and results is so that testing can be repeated for each revision of an application. Although new versions of a system may necessitate new test data to validate new features of the application, previous test data usually can and should be reused. Results from use of the test data with prior versions are compared to new versions to show that changes have not introduced new errors and that the behavior of the system, including response time, is no worse.

Testing often requires a great deal of labor. Manual code reviews can be time-consuming and tedious work; and, most importantly, are not always the best solution. As such, special-purpose testing software, called a testing harness, has been developed for a variety of environments to help designers automatically review the quality of their code. In many situations, a testing harness greatly enhances the testing process because they can automatically expand the scope of the tests beyond the current development platform, as well as be run every time with each new version of the software. For instance, with the testing harness called NUnit, an open-source unit testing framework for .NET, a developer can answer questions such as: How stable is the code? Does the code follow standard rules?

Will the code work across multiple platforms? When deploying large scale, multiplatform projects, automatic code review systems have become a necessity.

Acceptance Testing

Once the system tests have been satisfactorily completed, the system is ready for acceptance testing, which is testing the system in the environment where it will eventually be used.

Acceptance refers to the fact that users typically sign off on the system and —accept it once they are satisfied with it. The purpose of acceptance testing is for users to determine whether the system meets their requirements. The extent of acceptance testing will vary with the organization and with the system in question. The most complete acceptance testing will include alpha testing, (also called mock client testing), where simulated but typical data are used for system testing; beta testing, in which live data are used in the users' real working environment; and a system audit conducted by the organization's internal auditors or by members of the quality assurance group.

During alpha testing, the entire system is implemented in a test environment to discover whether

the system is overtly destructive to itself or to the rest of the environment. The types of tests performed during alpha testing include the following:

- Recovery testing. Forces the software (or environment) to fail in order to verify that recovery is properly performed.
- Security testing. Verifies that protection mechanisms built into the system will protect it from improper penetration.
- Stress testing. Tries to break the system (e.g., what happens when a record is written to the database with incomplete information or what happens under extreme online transaction loads or with a large number of concurrent users).
- Performance testing. Determines how the system performs on the range of possible environments in which it may be used (e.g., different hardware configurations, networks, operating systems); often the goal is to have the system perform with similar response time and other performance measures in each environment.

In beta testing, a subset of the intended users runs the system in their own environments using their own data. The intent of the beta test is to determine whether the software, documentation, technical support, and training activities work as intended. In essence, beta testing can be viewed as a rehearsal of the installation phase. Problems uncovered in alpha and beta testing in any of these areas must be corrected before users can accept the system.

3.1.2 Installation

The process of moving from the current information system to the new one is called installation. All employees who use a system, regardless of whether they were consulted during the development process or not, must give up their reliance on the current system and begin to rely on the new system. Four different approaches to installation have emerged over the years:

- Direct
- Parallel
- Single location
- Phased

The approach (or combination) an organization decides to use will depend on the scope and complexity of the change associated with the new system and the organization's risk aversion. In practice, you will rarely choose a single strategy to the exclusion of all others; most installations will rely on a combination of two or more approaches. For example, if you choose a single location strategy, you have to decide how installation will proceed there and at subsequent sites. Will it be direct, parallel, or phased?

Characteristics	Positive Aspects	Hazards/Risks
Direct Installation • Abrupt • "Cold turkey"	• Low cost • High interest in making installation a success • May be the only possible approach if new and existing systems cannot coexist in some form	• Operational errors have direct impact on users and organization • It may take too long to restore old system, if necessary • Time-consuming, and benefits may be delayed until whole system is installed
Parallel Installation • Old and new systems coexist • Safe	• New systems can be checked against old systems • Impact of operational errors are minimized because old system is also processing all data	• Not all aspects of new systems can be compared to old system • Very expensive because of duplication of effort to run and maintain two systems • Can be confusing to users • May be a delay until benefits result • May not be feasible because of costs or system size
Single Location Installation • Pilot approach • Middle-of-the-road approach • May involve a series of single location installations • Each location may be branch office, factory, or department	• Learning can occur and problems fixed by concentrating on one site • Limits potential harm and costs from system errors or failure to selected pilot sites • Can use early success to convince others to convert to new system	• Burden on IS staff to maintain old and new systems • If different sites require data sharing, extra programs need to be written to "bridge" the two systems • Some parts of organization get benefits earlier than other parts
Phased Installation • Staged, incremental, gradual, based on system functional components • Similar to bringing system out via multiple releases	• Allows for system development also to be phased • Limits potential harm and costs from system error or failure to certain business activities/functions • Risk spread over time • Some benefits can be achieved early • Each phase is small and more manageable	• Old and new systems must be able to work together and share data, which likely will require extra programming to "bridge" the two systems • Conversion is constant and may extend over a long period, causing frustration and confusion for users

Table 27: Approaches to System Installation

Each installation strategy involves converting not only software but also data and (potentially) hardware, documentation, work methods, job descriptions, offices and other facilities, training materials, business forms, and other aspects of the system.

Of special interest in the installation process is the conversion of data. Because existing systems usually contain data required by the new system, current data must be made error-free, unloaded from current files, combined with new data, and loaded into new files.

Data may need to be reformatted to be consistent with more advanced data types supported by newer technology used to build the new system. New data fields may have to be entered in large quantities so that every record copied from the current system has all the new fields populated. Manual tasks, such as taking a physical inventory, may need to be done in order to validate data before they are transferred to the new files. The total data conversion process can be tedious. Furthermore, this process may require that current systems be shut off while the data are extracted so that updates to old data, which would contaminate the extract process, cannot occur. Any decision that requires the current system to be shut down, in whole or in part, before the

replacement system is in place must be done with care. Typically, off-hours are used for installations that require a lapse in system support. Whether a lapse in service is required or not, the installation schedule should be announced to users well in advance to let them plan their work schedules around outages in service and periods when their system support might be erratic. Successful installation steps should also be announced, and special procedures put in place so that users can easily inform you of problems they encounter during installation periods. You should also plan for emergency staff to be available in case of system failure so that business operations can be recovered and made operational as quickly as possible. More than just a computer system is involved—you are also changing how people do their jobs and how the organization operates.

3.1.3 Documenting the System

In one sense, every information systems development project is unique and will generate its own unique documentation. In another sense, though, system development projects are probably more alike than they are different. Each project shares a similar systems development life cycle, which dictates that certain activities be undertaken and that each of those activities be documented.

We can simplify the situation by dividing documentation into two basic types: system documentation and user documentation. System documentation records detailed information about a system's design specifications, its internal workings, and its functionality. User documentation is written or other visual information about how an application system works and how to use it.

System documentation is intended primarily for maintenance programmers, user documentation is intended mainly for users. An organization should have definitive standards on system documentation. These standards may include the outline for the project dictionary and specific pieces of documentation within it. Standards for user documentation are not as explicit.

User Documentation

User documentation consists of written or other visual information about an application system, how it works, and how to use it.

Types of user documentation include a *reference guide*, a *quick reference guide*, *user's guide*, *release description*, *system administrator's guide*, and *acceptance sign-off*.

Most online reference guides allow you to search by topic area or by typing in the first few letters of your keyword. Reference guides are great for specific information but are not as good for the broader picture of how to perform all the steps required for a given task. The quick reference guide provides essential information about operating a system in a short, concise format.

Where computer resources are shared and many users perform similar tasks on the same machines (as with airline reservation or mail-order-catalog clerks), quick reference guides are often printed on index cards or as small books and mounted on or near the computer terminal. The purpose of a reference guide is to provide information on how users can use computer systems to perform specific tasks. The information in a user's guide is typically ordered by how often tasks are performed and how complex they are.

Increasingly, software vendors are using Web sites to provide additional user-guide content. Web-based documentation allows the vendor to provide more up-to-date reference material without issuing new software CDs. Because most software is reissued as new features are added, a release description contains information about a new system release, including a list of complete documentation for the new release, features and enhancements, known problems and how they have been dealt with in the new release, and information about installation. A system administrator's guide is intended primarily for a particular type of user—those who will install and administer a new system—and contains information about the network on which the system will run, software interfaces for peripherals such as printers, troubleshooting, and setting up user accounts. Finally, the acceptance sign-off allows users to test for proper system installation and then signify their acceptance of the new system and its documentation with their signatures.

Preparing User Documentation

User documentation, regardless of its content or intended audience, is now most often delivered online in hypertext format. In all its forms, user documentation is an investment that reduces training and consultation costs. As a future analyst, you need to consider the source of documentation, its quality, and whether its focus is on the information system's functionality or on the tasks the system can be used to perform. The traditional source of user and system documentation has been the organization's information systems department. Until recently, the bulk of this documentation was system documentation, intended for analysts, programmers, and those who must maintain the system.

In today's end-user information systems environment, users interact directly with many computing resources; users have many options or querying capabilities from which to choose when using a system; and users are able to develop many local applications themselves. Analysts often serve as consultants for these local end-user applications.

For end-user applications, the nature and purpose of documentation has changed from documentation intended for the maintenance programmer to documentation for the end user. Application oriented documentation, whose purpose is to increase user understanding and utilization of the organization's computing resources, has also come to be important. Although some of this user-oriented documentation continues to be supplied by the information systems department, much of it now originates with vendors and with users themselves.

3.1.4 Training and Supporting Users

Training and support are critical for the success of an information system. As the person responsible for the new system, you and other analysts on the project team must ensure that high-quality training and support are available. Training and support help people adequately use computer systems to do their primary work. Without proper training and the opportunity to ask questions and gain assistance/consultation when needed, users will misuse, underuse, or not use the information system you develop. Although training and support can be talked about as if they are two separate things, in organizational practice, the distinction between the two is not all that clear, as the two sometimes overlap. After all, both deal with learning about computing.

It is clear that support mechanisms are also a good way to provide training, especially for intermittent users of a system. Intermittent or occasional system users are not interested in, nor

would they profit from, typical user training methods. Intermittent users must be provided with —point-of-need support, specific answers to specific questions at the time the answers are needed. A variety of mechanisms, such as the system interface itself and online help facilities can be designed to provide both training and support at the same time.

Training System Users

Many organizations tend to underinvest in computer skills training. It is true that some organizations institutionalize high levels of information system training, but many others offer no systematic training at all. Many studies show that training users to be effective with the systems they have now can be a cost effective way to increase productivity, even more so than installing hardware and software upgrades.

The type of necessary training will vary by type of system and expertise of users. The list of potential topics from which you must determine whether training will be useful include the following:

- Use of the system (e.g., how to enter a class registration request)
- General computer concepts (e.g., computer files and how to copy them)
- Information system concepts (e.g., batch processing)
- Organizational concepts (e.g., FIFO inventory accounting)
- System management (e.g., how to request changes to a system)
- System installation (e.g., how to reconcile current and new systems during phased installation)

As you can see from this partial list, many topics go beyond simply how to use the new system. It may be necessary for you to develop training for users in other areas so that they will be ready, conceptually and psychologically, to use the new system. Some training, such as concept training, should begin early in the project because this training can assist in convincing users of the need for system and organizational change. Each element of training can be delivered in a variety of ways.

The following table lists the most common training methods used by information system departments.

Method of Training

Formal courses—several people taught at the same time

Resident expert

E-learning/distance learning

Blended learning (combination of instructor-led training and e-learning)

Software help components

External sources, such as vendors

Figure 31: Training Methods

The most common delivery method for corporate training remains traditional instructor-led classroom training. Many times users turn to the resident expert and to fellow users for training. Users are more likely to turn to local experts for help than to the organization's technical support staff, because the local expert understands both the users' primary work and the computer systems they use. Given their dependence on fellow users for training, it should not be surprising that end users describe their most common mode of computer training as —self-training.¶ One conclusion from the experience with user training methods is that an effective strategy for training on a new system is first to train a few key users and then to organize training programs and support mechanisms that involve these users to provide further training, both formal and on-demand. Often, training is most effective if you customize it to particular user groups, and the lead trainers from these groups are in the best position for this task.

Increasingly, corporations are turning to e-learning as a key delivery mode for training. Although the term e-learning is not precisely defined, it generally means the same thing as distance learning, i.e., a formalized learning system designed to be carried out remotely, using computer-based electronic communication. You may have taken a distance learning course at your school, or you may have experience in on-campus classes with some of the dominant software packages used in e-learning, such as WebCT, Blackboard, or Desire2Learn. E-learning courses can be delivered over the Internet or over company intranets. Such courses can be purchased from vendors or prepared by the corporation's in-house training staff. E-learning is relatively inexpensive, compared to traditional classroom training, and it has the additional advantage of being available anytime from just about anywhere.

Students can also learn at their own pace. E-learning systems can make available several different elements that enhance the learning experience, including simulations, online access to mentors and experts, e-books, net meetings, and video on-demand. Another trend in corporate training is blended learning, the combining of e-learning with instructor-led classroom training. A recent survey reported that over 80 percent of respondents were using e-learning or blended learning to train their employees. Half of the respondents in the study believed that e-learning would become the dominant training delivery method in their organizations by 2010. Another training method listed in the above table is software help components. One common type is

called an electronic performance support system (EPSS). Electronic performance support systems are online help systems that go beyond simply providing help—they embed training directly into a software package. An EPSS may take on one or more forms: It can be an online tutorial, provide hypertext-based access to context-sensitive reference material, or consist of an expert system shell that acts as a coach. The main idea behind the development of an EPSS is that the user never has to leave the application to get the benefits of training. Users learn a new system or unfamiliar features at their own pace and on their own machines, without having to lose work time to attend remote group-training sessions. Furthermore, this learning is on-demand when the user is most motivated to learn, because the user has a task to do. EPSS is sometimes referred to as “just-in-time knowledge.”

Supporting System Users

Historically, computing support for users has been provided in one of a few forms: paper, online versions of paper-based support, as well as help desks provided by vendors or resident experts within the same organization. As we stated earlier, support, whatever its form, has often been inadequate for users' needs. Yet users consider support to be extremely important. As computing spreads throughout organizations, especially with the advent of personal computers, the need for support increased as more and more employees came to rely on computing to do their jobs. As organizations moved to client/server architectures, their need for support increased even more, and organizations relied more and more on vendor support. This increased need for support comes in part from the lack of standards governing client/server products and the resulting need to make equipment and software from different vendors compatible.

Automated Issue Tracking Bugs and change requests in any software release are inevitable. Previously, we talked about using a system service request, a standard form for requesting systems development work. To automate this process, many organizations are deploying a Web-based issue tracking system to provide a systematic way to log, track, and assign system bugs and change requests to developers.

Automating Support As vendors have primarily shifted their offerings from expensive mainframe packages to inexpensive off-the-shelf software; they have found that they can no longer bear the cost of providing support for free. Most vendors now charge for support, and many have instituted toll-free numbers and other automated support mechanisms or sell customers unlimited support for a given monthly or annual charge. Common methods for automating support include online support forums (on private Web sites) and voice-response systems. Online support forums provide users access to information on new releases, bugs, and tips for more effective usage. Voice-response systems allow users to navigate option menus that lead to prerecorded messages about usage, problems, and workarounds. Organizations have established similar support mechanisms for systems developed or purchased by the organization. Internal e-mail and instant messaging can be used to support such capabilities within an organization.

Providing Support through a Help Desk Whether assisted by vendors or going it alone, the center of support activities for a specific information system in many organizations is the help desk. A help desk is an information systems department function, staffed by IS personnel. The

help desk is the first place users should call when they need assistance with an information system. The help desk staff either deals with the users' questions or refers the users to the most appropriate person. Today, help desks are increasingly common as management comes to appreciate the special combination of technical skills and people skills needed to make good help desk staffers. Many software packages exist to automate the record keeping for a help desk. Records must be kept on each user contact, the content of the question or problem, and the status and resolution of the problem. Help desk managers use the software to track problems with different information systems, assess help desk personnel efficiency and effectiveness, and identify users who require training. Help desk personnel need to be good at communicating with users, by listening to their problems and intelligently communicating potential solutions. These personnel also need to understand the technology they are helping users with. It is crucial, however, that help desk personnel know when new systems and releases are being implemented and when users are being trained for new systems. Help desk personnel themselves should be well trained on new systems. One sure recipe for disaster is to train users on new systems but not train the help desk personnel these same users will turn to for their support needs.

3.2 System Maintenance

A significant portion of an organization's budget for information systems does not go to the development of new systems but to the maintenance of existing systems. We describe various types of maintenance, factors influencing the complexity and cost of maintenance, alternatives for managing maintenance, and the role of automated development tools during maintenance. Given that maintenance activities consume the majority of information systems-related expenditures, gaining an understanding of these topics will yield numerous benefits to your career as an information system professional.

3.2.1 Types of Maintenance

By maintenance, we mean the fixing or enhancing of an information system. You can perform several types of maintenance on an information system. Corrective maintenance refers to changes made to repair defects in the design, coding, or implementation of the system. For example, if you purchase a new home, corrective maintenance would involve repairs made to things that had never worked as designed, such as a faulty electrical outlet or misaligned door. Most corrective maintenance problems surface soon after installation. When corrective maintenance problems arise, they are typically urgent and need to be resolved to curtail possible interruptions in normal business activities. Some corrective maintenance is due to incompatibilities between the new system and other information systems with which it must exchange data. Corrective maintenance adds little or no value to the organization; it simply focuses on removing defects from an existing system without adding new functionality. Adaptive maintenance involves making changes to an information system to evolve its functionality to changing business needs or to migrate it to a different operating environment. Within a home, adaptive maintenance might be adding storm windows to improve its energy efficiency. Adaptive maintenance is usually less urgent than corrective maintenance because business and technical changes typically occur over some period of time. Contrary to corrective maintenance,

adaptive maintenance is generally a small part of an organization's maintenance effort but does add value to the organization.

Perfective maintenance involves making enhancements to improve processing performance, interface usability, or to add desired, but not necessarily required, system features (—bells and whistles). In our home example, perfective maintenance would be adding a new room. Many system professionals feel that perfective maintenance is not really maintenance but new development.

Preventive maintenance involves changes made to a system to reduce the chance of future system failure. An example of preventive maintenance might be to increase the number of records that a system can process far beyond what is currently needed. In our home example, preventive maintenance could be painting the exterior to better protect the home from severe weather conditions. As with adaptive maintenance, both perfective and preventive maintenance are typically a much lower priority than corrective maintenance. Adaptive, perfective, and **preventive maintenance** activities can lead to corrective maintenance activities if not carefully designed and implemented.

3.2.2 Cost of Maintenance

Information systems maintenance costs are a significant expenditure. For some organizations, as much 60–80 percent of their information systems budget is allocated to maintenance activities. This proportion has risen from roughly 50 percent twenty years ago because many organizations have accumulated more and more old systems that require more and more maintenance. It means that you must understand the factors influencing the maintainability of systems. Maintainability is the ease with which software can be understood, corrected, adapted, and enhanced. Systems with low maintainability result in uncontrollable maintenance expenses. Numerous factors influence the maintainability of a system. These factors, or cost elements, determine the extent to which a system has high or low maintainability. Of these factors, three are most significant: number of latent defects, number of customers, and documentation quality. The others—personnel, tools, and software structure—have noticeable, but less, influence.

- **Latent defects:** This is the number of unknown errors existing in the system after it is installed. Because corrective maintenance accounts for most maintenance activity, the number of latent defects in a system influences most of the costs associated with maintaining a system.
- **Number of customers for a given system:** In general, the greater the number of customers, the greater the maintenance costs. For example, if a system has only one customer, problem and change requests will come from only one source. Also, training, reporting errors, and support will be simpler. Maintenance requests are less likely to be contradictory or incompatible.
- **Quality of system documentation:** Without quality documentation, maintenance effort can increase exponentially. Quality documentation makes it easier to find code that needs to be changed and to understand how the code needs to be changed. Good documentation also explains why a system does what it does and why alternatives were not feasible, which saves wasted maintenance efforts.

- Maintenance personnel: In some organizations, the best programmers are assigned to maintenance. Highly-skilled programmers are needed because the maintenance programmer is typically not the original programmer and must quickly understand and carefully change the software.
- Tools: Tools that can automatically produce system documentation where none exists can also lower maintenance costs.
Also, tools that can automatically generate new code based on system specification changes can dramatically reduce maintenance time and costs.
- Well-structured programs: Well-designed programs are easier to understand and fix.

3.2.3 Measuring Maintenance Effectiveness

Because maintenance can be so costly, it is important to measure its effectiveness. To measure effectiveness, you must measure these factors:

- Number of failures
- Time between each failure
- Type of failure

Measuring the number of and time between failures will provide you with the basis to calculate a widely-used measure of system quality. This measure is referred to as the mean time between failures (MTBF). As its name implies, the MTBF measure shows the average length of time between the identification of one system failure until the next. Over time, you should expect the MTBF value to increase rapidly after a few months of use (and corrective maintenance) of the system. If the MTBF does not rapidly increase over time, it will be a signal to management that major problems exist within the system that is not being adequately resolved through the maintenance process.

A more revealing method of measurement is to examine the failures that are occurring. Over time, logging the types of failures provides a clear picture of where, when, and how failures occur. For example, knowing that a system repeatedly fails logging new account information to the database when a particular customer is using the system can provide invaluable information to the maintenance personnel. Were the users adequately trained? Is there something unique about this user? Is there something unique about an installation that is causing the failure? What activities were being performed when the system failed? Tracking the types of failures also provides important management information for future projects. For example, if a higher frequency of errors occurs when a particular development environment is used, such information can help guide personnel assignments, training courses, or the avoidance of a particular package, language, or environment during future development.

The primary lesson here is that without measuring and tracking maintenance activities, you cannot gain the knowledge to improve or know how well you are doing relative to the past. To manage effectively and to improve continuously, you must measure and assess performance over time.

UNIT - IV

4.1 OO Concepts from Structured Point of View

4.1.1 Objects and Classes

The OO paradigm is based on building systems from items called objects. An object is any person, place, thing, event, concept, screen, or report. A class generalizes/represents a collection of similar objects and is effectively a template from which to create objects. In a university system, Sarah is a student object, she attends several seminar objects, and she is working on a degree object. In a banking system, Sarah is a customer object. She has a checking account object from which she bounces rubber-check objects. In an inventory control system, every inventory item is an object, every delivery is an object, and every customer is an object. In the real world, you have objects; therefore, you need them as a concept to reflect your problem space accurately. However, in the real world, objects are often similar to other kinds of objects. Students share similar qualities (they do the same sort of things; they are described in the same sort of way), courses share similar qualities, inventory items share similar qualities, bank accounts share similar qualities, and so on. While you could model (and program) every object, that is a lot of work. I prefer to define what it is to be a student once, define course once, define inventory item once, and define bank account once, and so on. That is why you need the concept of a class.

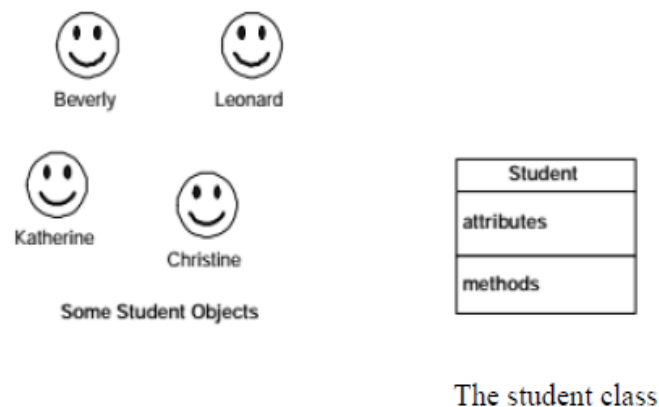


Figure 32: Objects and Class

The above figure depicts how we have student objects and how we model the class Student. It also shows the standard notations to model a class.

Classes are typically modeled by using a rectangle that lists its attributes and methods. Listing

the attributes and methods can be quite helpful. It enables readers of your class models to gain a better understanding of your design at a single glance.

Class names are typically singular nouns. The name of a class should be one or two words, usually a noun, and should accurately describe the class using common business terminology. If you are having trouble naming a class, either you need to understand it better or it might be several classes you have mistakenly combined. You should model classes with names like Student, Professor, and Course, not Students, People Who Teach Seminars. Think of it like this: in the real world, you would say —I am a student, not —I am a students. Class can also represent concepts that are not nouns, like the process of checking out a book from a library. When object-oriented software is running, objects are instantiated (created/defined) from classes. We say an object is an instance of a class and we instantiate those objects from classes.

4.1.2 Attributes and Methods

Classes have responsibilities, the things they know and do. Attributes are the things classes know; methods are the things classes do. The object-oriented paradigm is based on the concepts that systems should be built out of objects, and that objects have both data and functionality. Attributes define the data, while methods define the functionality.

When you define a class, you must define the attributes it has, as well as its methods. The definition of an attribute is straightforward. You define its name, perhaps its type (whether it is a number, a string, or a date, and so forth). The definition of a method is simpler: you define the logic for it, just as you would code for a function or a procedure. An important implication is that methods do one of two things: either they return a value and/or they do something of value; that is, they have a side effect.

In the following figure, we are using the three-section class notation in this case: the top section for the name, the middle section to list the attributes, and the bottom section to list the methods. It depicts two types of attributes: instance attributes, which are applicable to a single object, and static attributes, which are applicable to all instances of a single class. Attributes are underlined, instance attributes are not. For example, name is an instance attribute of the class Student.

Each individual student has a name; for example, one student may have the name —Smith, John, whereas another student may have the name —Jones, Sally. It could even happen that two individual students may have the same name, such as —Smith, John although they are, in fact, two different people. On the other hand, nextStudentNumber is a static attribute (also referred to as a class attribute) that is applicable to the class Student, not specifically to individual instances. This attribute is used to store the value of the next student number to be assigned to a student: when a new student joins the school, his or her student number is set to the current value of nextStudentNumber, which is then incremented to ensure all students have unique student numbers.

Similarly, there is the concept of instance methods and static/class methods: instance methods operate on a single instance, whereas static methods operate potentially on all instances of a

single class. In the following figure, you see that Student has instance methods called enrollInSeminar and dropSeminar, things an individual student would do. It also has the static method findByName, which supports the behavior of searching for students whose names meet specified search criteria, a method that operates on all instances of the class.

Example:

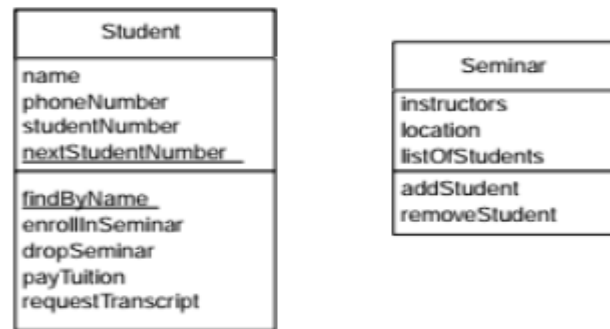


Figure 33: Student and Seminar Classes

4.2 Abstraction, Encapsulation and Information Hiding

Instead of saying we determined what a class knows and does, we say we ‘abstracted’ the class. Instead of saying we designed how the class will accomplish these things; we say we ‘encapsulated’ them. Instead of saying we designed the class well by restricting access to its attributes, we say we have ‘hidden’ the information.

4.2.1 Abstraction

The world is a complicated place. To deal with that complexity we form abstractions of the things in it. For example, consider the abstraction of a person. From the point of view of a university, it needs to know the person’s name, address, telephone number, social security number, and educational background. From the point of view of the police, they need to know a person’s name, address, phone number, weight, height, hair color, eye color, and so on. It is still the same person, just a different abstraction, depending on the application at hand.

Abstraction is an analysis issue that deals with what a class knows or does. Your abstraction should include the responsibilities, the attributes, and the methods of interest to your application—and ignore the rest. That is why the abstraction of a student would include the person’s name and address, but probably not his or her height and weight. OO systems abstract only what they need to solve the problem at hand.

People often say abstraction is the act of painting a clear box around something; you are identifying what it does and does not do. Some people will also say that abstraction is the act of defining the interface to something. Either way, you are defining what the class knows and does.

4.2.2 Encapsulation

Although the act of abstraction tells us that we need to store a student's name and address, as well as be able to enroll students in seminars, it does not tell us how we are going to do this. Encapsulation deals with the issue of how you intend to modularize the features of a system. In the object-oriented world, you modularize systems into classes, which, in turn, are modularized into methods and attributes. We say that we encapsulate behavior into a class or we encapsulate functionality into a method.

Encapsulation is a design issue that deals with how functionality is compartmentalized within a system. You should not have to know how something is implemented to be able to use it. The implication of encapsulation is that you can build anything anyway you want, and then you can later change the implementation and it will not affect other components within the system (as long as the interface to that component did not change).

People often say encapsulation is the act of painting the box black: you are defining how something is going to be done, but you are not telling the rest of the world how you are going to do it. In other words you are hiding the details of the implementation of an item from the users of that item. For example, consider your bank. How does it keep track of your account information, on a mainframe, a mini, or a PC? What database does it use? What operating system? It does not matter, because it has encapsulated the way in which it performs account services. You just walk up to a teller and initiate whatever transactions you want. By hiding the details of the way it has implemented accounts, your bank is free to change that implementation at any time, and it should not affect the way services are provided to you.

4.2.3 Information Hiding

To make your applications maintainable, you want to restrict access to data attributes and some methods. The basic idea is this: if one class wants information about another class, it should have to ask for it, instead of taking it. When you think about it, this is exactly the way the real world works. If you want to learn somebody's name, what would you do? Would you ask the person for his name, or would you steal his wallet and look at his ID? By restricting access to attributes, you prevent other programmers from writing highly coupled code. When code is highly coupled, a change in one part of the code forces you to make a change in another, and then another, and so on. Coupling is described in detail in the next Section.

4.3 Inheritance

Similarities often exist between different classes. Two or more classes often share the same attributes and/or the same methods. Because you do not want to have to write the same code repeatedly, you want a mechanism that takes advantage of these similarities. Inheritance is that mechanism. Inheritance models —is all, —is kind of, and —is like relationships, enabling you to reuse existing data and code easily. For example, students have names, addresses, and telephone numbers, and they drive vehicles. At the same time, professors also have names, addresses, and telephone numbers, and they drive vehicles. Without a doubt, you could develop

the classes for student and professor and get them both running. In fact, you could even develop the class Student first and, once it is running, make a copy of it, call it Professor, and make the necessary modifications. While this is straightforward to do, it is not perfect. What if there was an error in the original code for student? Now you must fix the error in two places, which is twice the work.

What would happen if you needed to change the way you handled names (say, you go from a length of 30 to a length of 40)? Now you would have to make the same change in two places again, which is a lot of dull, boring, tedious work (and costly). Would it not be nice if you had only one copy of the code to develop and maintain? This is exactly what inheritance is all about. With inheritance, you define a new class that encapsulates the similarities between students and professors. This new class would have the attributes name, address, and phoneNumber, and the method driveVehicle. Because you need to name all our classes, you need to ask yourself what this collection of data and functionality describes. In this case, the name Person is fitting. Once you have the class Person defined, you then make Student and Professor inherit from it. You would say Person is the superclass of both Student and Professor, and Student and Professor are the subclasses of Person. Everything that a superclass knows or does, the subclass knows or does free without writing extra code. Actually, for this example, you would need to write two lines of code, one saying Student is a subclass of Person, and another saying Professor is a subclass of Person; therefore, it is almost free. Because Person has a name, address, and telephone number, both Student and Professor also have those attributes. Because Person has the ability to drive a vehicle, so do the classes Student and Professor.

4.3.1 Modeling Inheritance

The following figure depicts the modeling notation for inheritance, a line with a closed arrowhead. The way you would read the diagram is —B inherits from A. In other words, B is a direct subclass of A and A is the direct superclass of B.

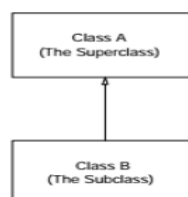


Figure 34: Modeling Notation for Inheritance

The below figure presents how you would model the Person inheritance class hierarchy, often simply called a class hierarchy. Notice how the name of the Person class is in italics, indicating it is abstract, whereas Professor and Student are concrete classes. Abstract and concrete classes are discussed in the following Section.

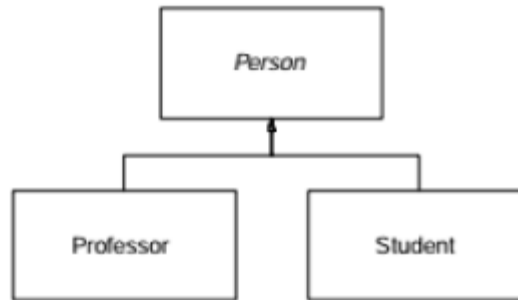


Figure 35: Modeling Inheritance

4.3.2 Single and Multiple Inheritance

When a class inherits from only one other class, we call this single inheritance. When a class inherits from two or more other classes, we call this multiple inheritance. Remember this: the subclass inherits all the attributes and methods of its superclass(es). Not all languages support multiple inheritance. C++ is one of the few languages that does, whereas languages such as Java, Smalltalk, and C# do not. The point to be made is if your target implementation language does not support multiple inheritance, then you should not use it when you are modeling.

In the below figure, you see several similarities between airplanes and cars. They both have a number of passengers, a maximum fuel level, and they can either increase or decrease their speed. To take advantage of these similarities, you could create a new class called *Vehicle* and have **Airplane** and **Car** inherit from it. We say the classes *Vehicle*, **Airplane**, and **Car** form an inheritance hierarchy, also called a class hierarchy. The topmost class in a class hierarchy (in this case *Vehicle*) is called the root or root class. Notice how there is the method **Turn** for **Car** and **Bank** for **Airplane**. Turning and banking are exactly the same thing. You could have defined a **Turn** method in *Vehicle*, and had **Airplane** and **Car** inherit it (then you would remove **Bank** and **Turn** from the subclasses). This would imply that you would require users of airplanes (probably pilots) to change the terminology they use to work with airplanes. Realistically, this would not work. A better solution would be to define **Turn** in *Vehicle* and have the method **Bank** invokes it as needed.

In the following diagram, the class *Vehicle* is marked as abstract (the name is in italics; in previous versions of the UML you could also indicate with the constraint {abstract}), whereas **Airplane** and **Car** are not. We say *Vehicle* is an abstract class, whereas **Airplane** and **Car** are both concrete classes. The main difference between abstract classes and concrete classes is that objects are instantiated (created) from concrete classes, but not from abstract classes. For example, in your problem domain, you have airplanes and cars, but you do not have anything that is just a vehicle (if something is not an airplane or a car, you are not interested in it). This means your software will instantiate airplane and car objects, but will never create vehicle

objects. Abstract classes are modeled when you need to create a class that implements common features from two or more classes.

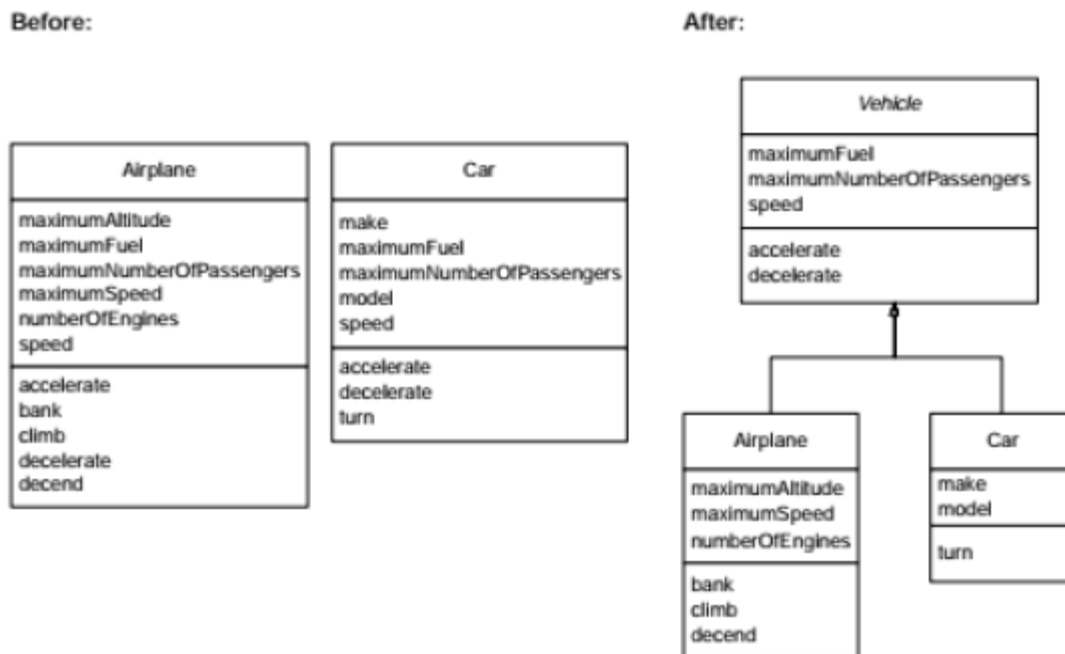


Figure 36: Creating Vehicle Class Hierarchy

In the before picture of the following diagram, you might want to create a new class called Dragon. You already have the classes Bird and Lizard. A dragon is like a bird because they both fly. A dragon is also like a lizard because they both have claws and scales. Because dragons have the features of both birds and lizards, in the after picture we have the class Dragon inheriting from both Bird and Lizard. This is an example of an ‘is like’ relationship: a dragon is like a bird and a dragon is (also) like a lizard.

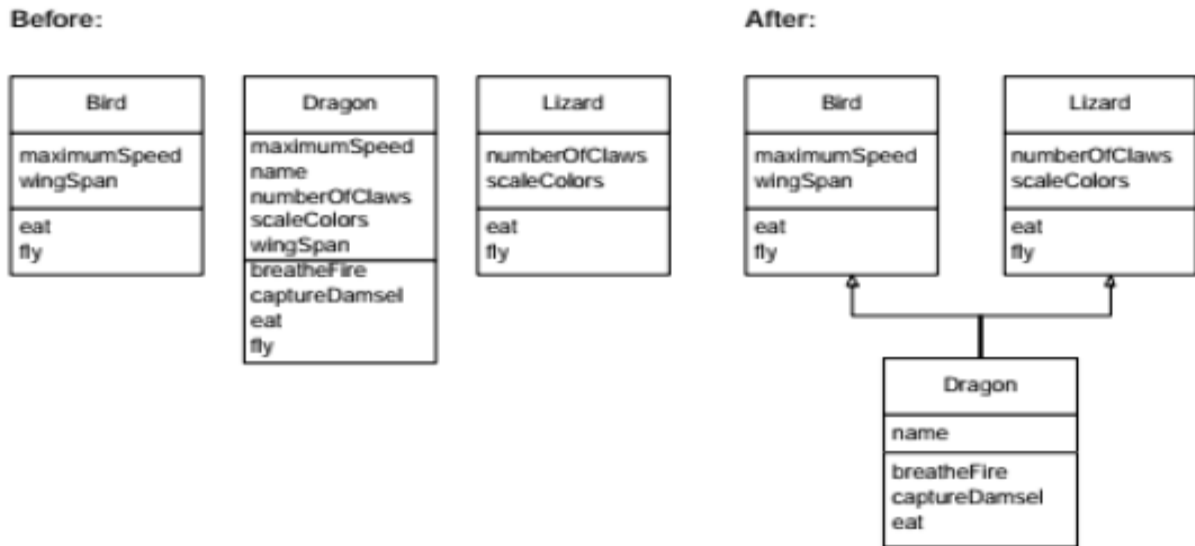


Figure 37: Example of Multiple Inheritance

Notice how we listed the method eat for Dragon. Although all three types of creatures eat, they all eat in different ways. Birds eat bird seed, lizards eat bugs, and dragons eat knights in shining armor. Because the way dragons eat is different than the way either birds or lizards eat, I needed to redefine, or override, the definition of the eat method.

he general idea is a subclass will need to override the definition of either an attribute (occasionally) or a method whenever it uses that data, or performs that method in a manner different than that of its superclass.

4.4 Persistence

Persistence focuses on the issue of how to make objects available for future use of your software. In other words, how to save objects to permanent storage. To make an object persistent, you must save the values of its attributes to permanent storage (such as a relational database or a file) as well as any information needed to maintain the relationships (aggregation, inheritance, and association) with which it is involved. In other words you need to save the appropriate properties to permanent storage. In addition to saving objects, persistence is also concerned with their retrieval and deletion. From a development point of view, there are two types of objects: persistent objects that stick around and transient objects that do not. For example, a Customer is a persistent class. You want to save customer objects into some sort of permanent storage so you can work with them again in the future. A customer editing screen, however, is a transient object.

Your application creates the customer-editing screen object, displays it, and then gets rid of it once the user is done editing the data for the customer with whom he or she is currently dealing.

4.5 Associations

An association is a persistent relationship between two or more classes or objects. When you model associations in class diagrams, you show them as a thin line connecting two classes, as you see in the below figure.



Examples:

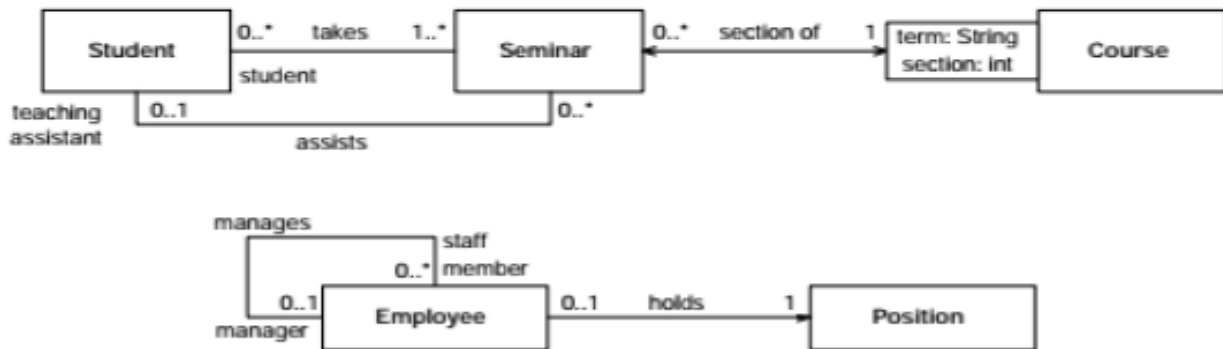


Figure 38: Associations

Associations can become quite complex; consequently, you can depict several things about them on your diagrams. The above Figures show the common items to model for an association:

- **Directionality:** The open arrowheads indicate the directionality of the association. When there is one arrowhead the association is unidirectional: it can be traversed in one direction only (in the direction of the arrow). When there is either zero or two arrowheads the association is bi-directional: it can be traversed in both directions. Although you should indicate both arrowheads it is common practice to drop the arrowheads from bi-directional associations.
- **Label:** The label, which is optional, is typically one or two words describing the association. Reading one class, the label, and then the other class should produce a sentence fragment that describes the relationship, e.g., Professor teaches Seminars. Avoid generic labels like —hasl or —communicates withl as much as possible.
- **Multiplicity:** The multiplicity of the association is labeled on either end of the line, one multiplicity indicator for each direction (below table summarizes the potential multiplicity indicators you can use).

Indicator	Meaning
0..1	Zero or one
1	One only
0..*	Zero or more
1..*	One or more
n	Only n (where $n > 1$)
0..n	Zero to n (where $n > 1$)
1..n	One to n (where $n > 1$)

Table 28: Multiplicity indicators

Here is how you would read the associations:

- A student takes one or more seminars
- A seminar is taken by zero or more students
- A student, as a teaching assistant, may assist in zero or more seminars
- A seminar may have zero or one student who acts as a teaching assistant
- A seminar is a section of one course
- A course has zero or more sections
- An employee holds one position
- A position may be held by one employee
- An employee may be managed by one other employee, their manager (the company president is the only one without a manager)
- An employee manages zero or more employees (some employees do not have any staff members)
 - Role: The role is the context that an object takes within the association— may also be indicated at each end of the association.
 - Qualifier: A qualifier is a value that selects an object from the set of related objects across an association. Qualifiers are optional and in practice are rarely modeled.

Several important lessons are contained in the earlier above figure. First, you see it is possible to have more than one association between two classes: the classes Student and Seminar have the takes and assists associations between them.

You are interested in two relationships between these two classes for your university information system; therefore, you need to model both associations.

Second, it is valid that the same class may be involved with both ends of an association; something called a recursive association or a self-association. A perfect example of this is the manages association that the Employee class has with itself. The way you read this association is that any given employee may have several other employees he or she manages, and that one other employee may, in turn, manage them.

Third, sometimes the direction of an association is one way, as you have seen in the figure, with the holds association between Employee and Position. The implication is employee objects know

the position object they hold, but the position object does not need to know what employee holds it. This is called a unidirectional association, an association that is traversed in only one direction. If you do not have a requirement to traverse an association in both directions, for example, position objects do not have a need to collaborate with employee objects, and then you should use a unidirectional association.

4.6 Aggregation

Sometimes an object is made up of other objects. For example, an airplane is made up of a fuselage, wings, engines, landing gear, flaps, and so on. A delivery shipment contains one or more packages. A project team consists of two or more employees. These are all examples of the concept of aggregation, which represents —is part of relationships. An engine is part of a plane, a package is part of a shipment, and an employee is part of a team.

Aggregation is simply a type of association; therefore, you still need to model the multiplicity and roles, just as you would with associations. Not indicating the multiplicity of the whole end of an aggregation is permissible as shown below:

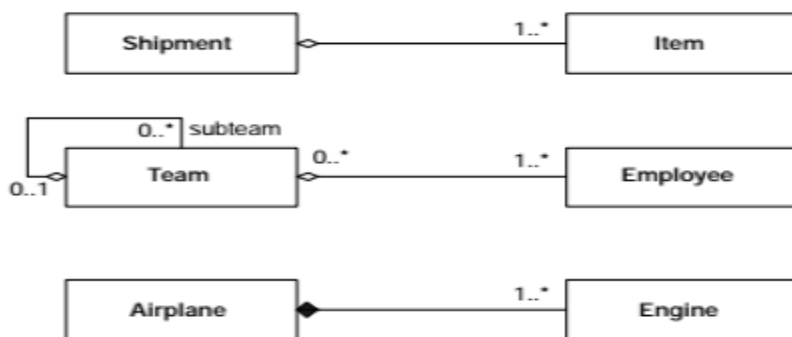


Figure 39: Examples of Aggregation

For example, in the figure above, multiplicity is not indicated for either shipments or airplanes; in these cases it is assumed the multiplicity is 1. Just as associations are two-way streets, so is aggregation. Furthermore, the aggregation is read in a similar manner:

- An item is part of one and only one shipment
- A shipment is composed of one or more items
- An engine is part of one and only one airplane
- An airplane has one or more engines
- An employee may be part of one or more teams
- A team is made up of one or more employees
- Any given team may be part of a larger team
- A team may be made up of smaller subteams

4.7 Collaboration

Classes often need to work together to fulfill their responsibilities. Actually, it is typically the objects and the instances of the classes that are working together. Collaboration occurs between objects when one object asks another for information or to do something. For example, an airplane collaborates with its engines to fly. For the plane to go faster, the engines must go faster. When the plane needs to slow down, the engines must slow down. If the airplane did not collaborate with its engines, it would be unable to fly. Objects collaborate with one another by sending each other messages. A message is either a request to do something or a request for information. Messages are modeled in sequence and collaboration diagrams.

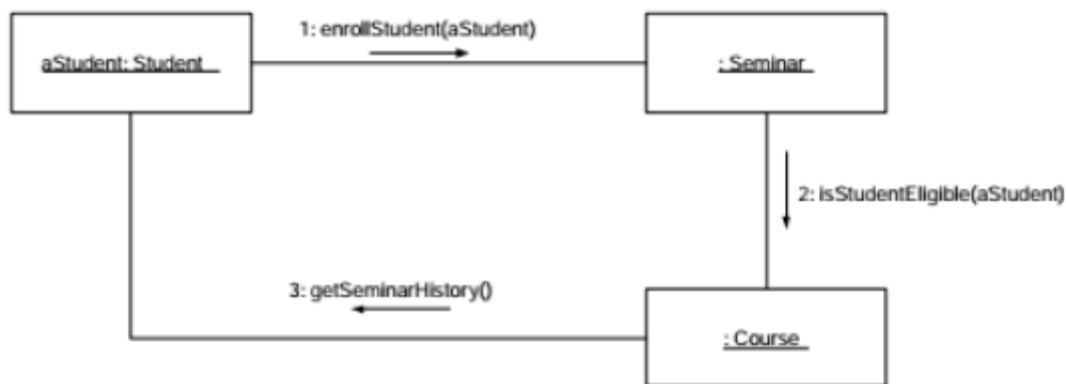


Figure 40: Collaboration Diagram

4.8 Coupling

Coupling is a measure of how two items, such as classes or methods, are interrelated. When one class depends on another class, we say they are coupled. When one class interacts with another class, but does not know any of the implementation details of the other class, we say they are loosely coupled. When one class relies on the implementation (that is, it directly accesses the data attributes of the other), we say they are tightly coupled.

Previously we discussed the example of how `Student` could implement the `enroll` method: it could directly access the attribute `listOfStudents` in `Seminar`, or it could send `Seminar` objects a message asking it to enroll the student in the seminar. Directly accessing and updating the attribute `listOfStudents` might save a few CPU cycles and run a little quicker, but as soon as the implementation of that attribute changes, you would need to modify the code in `Student`. As you saw, this was not very good. The basic problem is when two classes are tightly coupled; a change in one often requires a change in the other. This, in turn, could require a change in another class, and then another, and then another, and so on. Tight coupling is one of the main reasons such a large maintenance burden exists. What should be a simple maintenance change can often create

months of work, if it can be done at all. It is amazing how much code is out there that nobody is willing to touch because they are afraid of breaking it.

4.9 Cohesion

Cohesion is a measure of how much an item, such as a class or method, makes sense. A good measure of the cohesiveness of something is how long it takes to describe it in one sentence: the longer it takes, the less cohesive it likely is. You want to design methods and classes that are highly cohesive. In other words, it should be very clear what a method or class is all about. A method is highly cohesive if it does one thing and one thing only. For example, in the class `Student` you would have methods to enroll a student in a seminar and to drop a student from a seminar. Both of these methods do one thing and one thing only. You could write one method to do both these functions, perhaps called `changeSeminarStatus`. The problem with this solution is the code for this method would be more complex than the code for the separate `enrollInSeminar` or `dropSeminar` methods. This means your software would be harder to understand and, hence, harder to maintain. Remember that you want to reduce the maintenance burden, not increase it.

A highly cohesive class represents one type of object and only one type of object. For example, for the university information system we model professors, not employees. While a professor is, indeed, an employee, they are very different from other kinds of employees. For example, professors do different things than do janitors, who do different things than do secretaries, who do different things than do registrars, and so on. We could easily write a generic `Employee` class that is able to handle all the functionality performed by every type of employee working for the university. However, this class would quickly become cumbersome and difficult to maintain. A better solution would be to define an inheritance hierarchy made up of the classes `Professor`, `Janitor`, `Secretary`, `Registrar`, and so on. Because many similarities exist between these classes, you would create a new abstract class called `Employee`, which would inherit from `Person`. The other classes, including `Professor`, would now inherit from `Employee`. The advantage of this is each class represents one type of object. If there are ever any changes that need to be made with respect to janitors, you can go right to the class `Janitor` and make them. You do not need to worry about affecting the code for professors. In fact, you do not even need to know anything about professors at all.

5. Polymorphism

An individual object may be one of several types. For example, a John Smith object may be a student, a registrar, or even a professor. Should it matter to other objects in the system what type of person John is? It would significantly reduce the development effort if other objects in the system could treat people objects the same way and not need to have separate sections of code for each type. The concept of polymorphism says you can treat instances of various classes the

same way within your system. The implication is you can send a message to an object without first knowing what type it is and the object will still do —the right thing, at least from its point of view.

5.1 Polymorphism at University

Consider a slightly more realistic example of polymorphism by exploring the design of how the university handles the hiring of new staff, depicted in the following figure. There is a standard process for hiring staff at the university: once a person is hired, she is added to the university pension plan and an employee card is created for her. When a professor is hired at the university, the same process is followed.

If the hire method has been implemented in the Employee class, it would implement the behavior needed to add the person into the university pension plan and print an employee card for them. The hire method has been overridden in the Professor class. Presumably, it would invoke the hire method in the Employee class because that behavior is still applicable for professors.

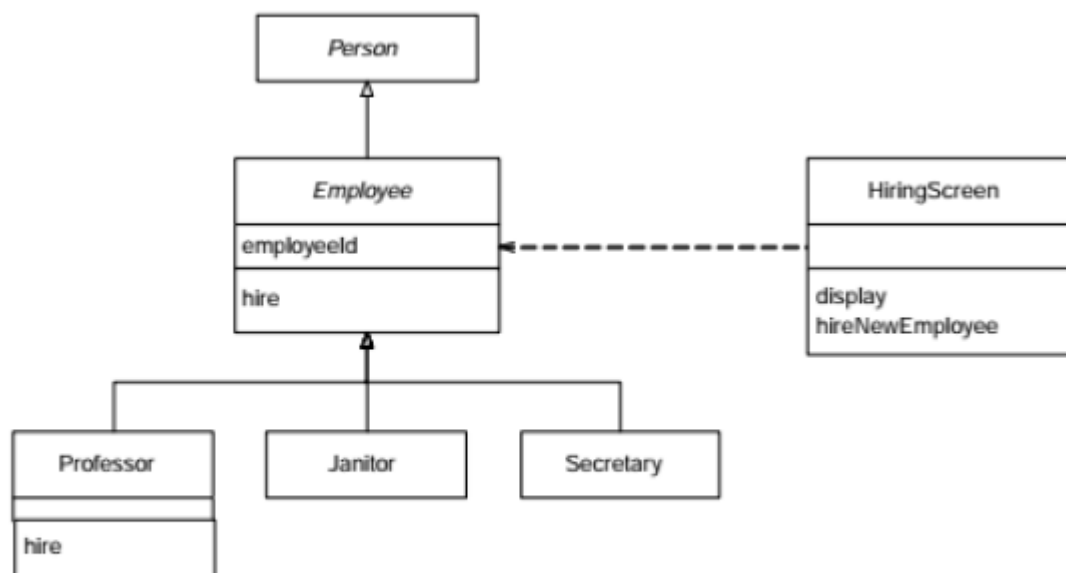


Figure 41: A Partial View of Polymorphism at University Human Resource System

By being able to send the message hire to any kind of employee, there is not the need for a complicated set of IF or CASE statements in the hireNewEmployee method of the screen object. This method does not need to send a hireProfessor message to professor objects, hireJanitor to janitor objects, and so on. It just sends hire to any type of employee and the object will do the right thing. As a result, you can add new types of employees (perhaps Registrar) and you do not need to change the screen object at all. In other words, the class is loosely coupled to the employee class hierarchy, enabling you to extend your system easily.

6. Interfaces

An interface is the definition of a collection of one or more methods, and zero or more attributes. Interfaces ideally define a cohesive set of behaviors. Interfaces are implemented by classes and components. To implement an interface, a class or component must include the methods defined by the interface. For example, the below figure indicates the class Student implements the Serializable interface and the Searchable interface. To implement the Searchable interface, Student must include a method called find, which takes criteria as a parameter.

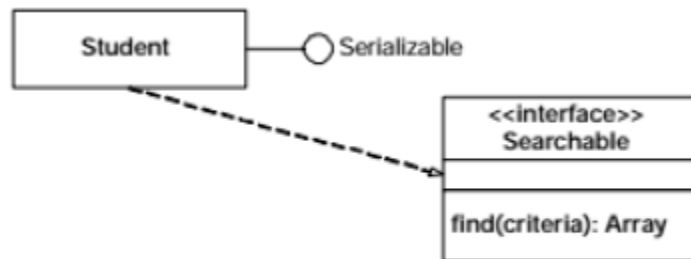


Figure 42: Interface Implementation Notation

Interfaces are a powerful method for ensuring loose coupling. They allow a class to participate in a common set of functionality without another class having to know anything about it except that it supports that interface. A GUI object, for example, could present a list of students that meet a set of criteria without knowing anything about the Student class except that it implements the Searchable interface and the name of a single method to get a presentation name, which could perhaps be part of a Description interface. That same GUI could perform the exact same action with professors if the Professor class also implemented those two interfaces. If the Seminar class implemented those interfaces, the GUI could display them, too, and anything else that implemented those interfaces. The GUI knows nothing about these classes except that they implement two interfaces; this promotes loose coupling between the GUI and those classes.

7. Components

A component is a modular, extensible unit of independent deployment that has contractually specified interface(s) and explicitly defined dependencies, if any. Ideally, components should be modular, extensible, and open. Modularity implies a component contains everything it needs to fulfill its responsibilities; extensibility implies a component can be enhanced to fulfill more responsibilities than it was originally intended to, and open implies it can operate on several platforms and interact with other components through a single programming interface. Components, like classes, implement interfaces. A component's interfaces define its access points. Components are typically implemented as collections of classes, ideally classes that form a cohesive subset of your overall systems. Components are typically heavyweights and could even be thought of as large classes or even subsystems. For example, a database could be a

component or the collection of business/domain classes that implement the behaviors required to implement people within your application could be a component.

Component diagrams (Object Management Group 2003) show the software components that make up a larger piece of software, their interfaces, and their interrelationships.

The following figure shows an example of a component diagram being used to model the business architecture for the university. The boxes represent components—in this case either the user interfaces that people use to interact with the university systems, business components such as Facility, or technical components such as the Security component or the database.

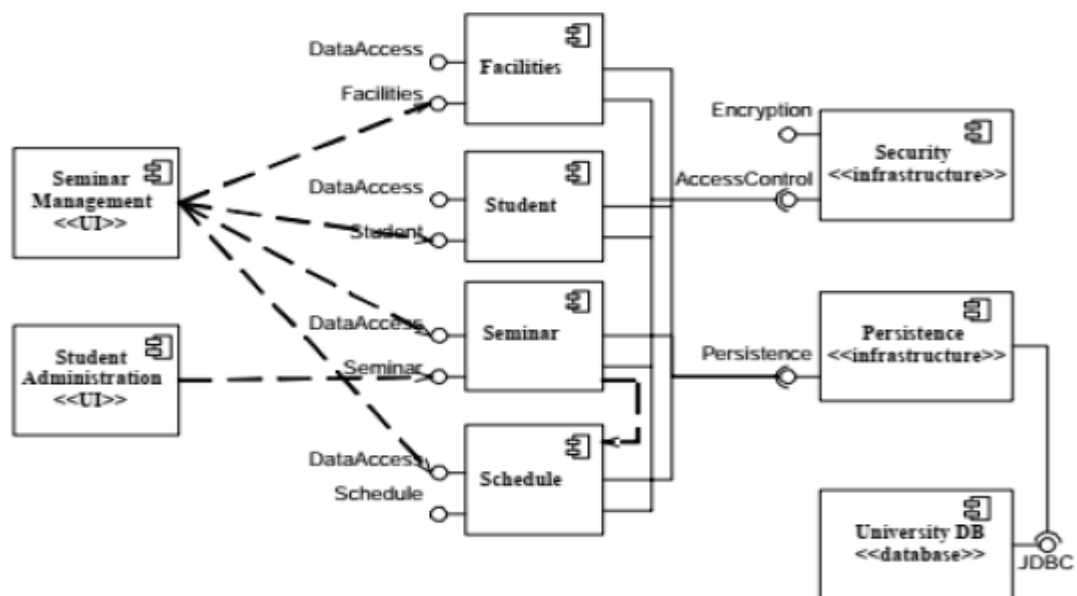


Figure 43: Component Diagram

In the figure you see that the UI components have dependencies on the interfaces of the business components. By making them dependent on the interfaces and not the components themselves you make it possible to replace the components with different implementations as long as the new version implements the given interfaces. This reduces coupling. Similarly it is possible for business components to be dependent on each other; for example, the Seminar component is coupled to the Schedule component.

8. Patterns

Doesn't it always seem as if you are solving the same problems repeatedly? If you personally have not solved a given problem before, then chances are pretty good you could find somebody who had tackled the same or, at least, a similar problem in the past.

Sometimes the problem you are working on is simple, sometimes it is complex, but usually it has been worked on before. Wouldn't it be nice to be able to find a solution easily, or at least a partial solution, to your problem? Think how much time and effort could be saved if you had access to a library of solutions to common system development problems. This is what patterns are all about.

A pattern is a solution to a common problem taking relevant forces into account, effectively supporting the reuse of proven techniques and approaches of other developers. Several flavors of patterns exist, including analysis patterns, design patterns, and process patterns. Analysis patterns describe a solution to common problems found in the analysis/business domain of an application, design patterns describe a solution to common problems found in the design of systems, and process patterns address software process related issues. Analysis patterns, design patterns and process patterns are discussed in the coming chapters.

9. UML Diagrams

Although there is far more to modeling than just the UML, as you will see throughout this module, the reality is that the UML defines the standard modeling artifacts when it comes to object technology. There are three classifications of UML diagrams:

- ⌘ Behavior diagrams. This is a type of diagram that depicts behavioral features of a system or business process. This includes activity, state machine, and use case diagrams as well as the interaction diagrams.

- ⌘ Interaction diagrams. This is a subset of behavior diagrams that emphasize object interactions. This includes collaboration, sequence diagrams.

- ⌘ Structure diagrams. This is a type of diagram that depicts the static elements of a specification that are irrespective of time. This includes class, component, deployment, and package diagrams.

UNIT - V

5.1 Structured Vs Object Oriented Paradigm

5.1.1 Structured Paradigm

The structured paradigm is a development strategy based on the concept that a system should be separated into two parts: Data and functions/procedures (modeled using a process model). Using the structured approach, you develop applications in which data is separated from behavior in both the design model and in the system implementation (that is, the program).

Example: Consider the design of an information system for a university. Taking the structured approach, you would define the layout of a data initially as a separate system and the design of a program to access that data as another. The programs have the ability to change the data states.

5.1.2 Object Oriented Paradigm

The main concept behind the object oriented paradigm is that instead of defining systems as two separate parts (data and functions), system is defined as a collection of interacting objects. It describes and builds a system that consists of objects. An object oriented system comprises a number of software objects that interact to achieve the system objective.

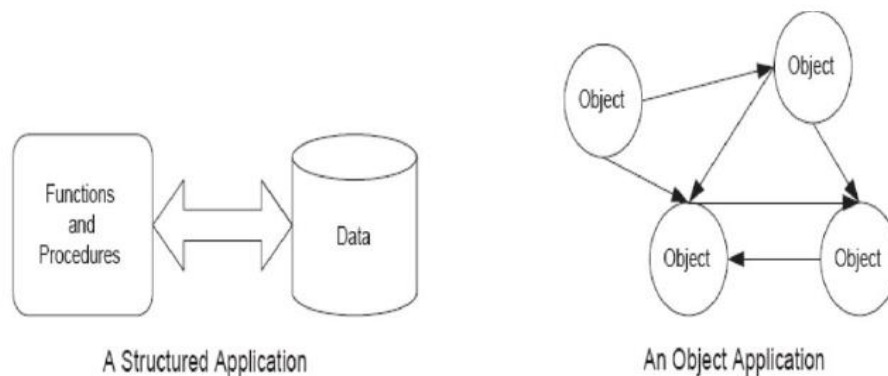


Figure 44: Structured and Object Oriented Paradigms

5.2 The Potential Benefits of Object Orientation

5.2.1 Increased Reusability

The object oriented paradigm provides opportunities for reuse through the concepts of inheritance, polymorphism, encapsulation, coupling and cohesion. OO paradigm provides more opportunities for reuse than the structured paradigm.

5.2.2 Increased Extensibility

Because classes have both data and functionality, when you add new features to the system you need to make changes in one place – the class.

5.2.3 Improved Quality

Quality systems are on time, on budget and meet or exceed the expectations of their users. Improved quality comes from increased participation of users in systems development. OO systems development techniques provide greater opportunity for users to participate in the development process.

5.2.4 Financial Benefits

Reusability, extensibility, and improved quality are all technical benefits. Object orientation enables you to build systems better, faster and cheaper. The benefits OO are realized throughout the entire development life cycle, not just programming.

5.2.5 Increased Chance of Project Success

A project is successful if it is on time, on budget and meets the needs of the users. Users are expert at business and they are the only ones who can tell you what they need. You need to know the right question to ask, know the business very well. You need models that communicate the required information and that users understand. You need to work closely with users; Time invested in defining requirements and modeling pays off in the long run.

5.2.6 Reduce Maintenance Burdon

Software organizations currently spend significant resources (80%) maintaining and operating software, and because of the long waiting list of work to be done, it takes significant time to get new projects started.

These two problems are respectively called the maintenance Burdon and the application backlog. These are problems that object orientation can help you to overcome.

5.3 The Potential Drawbacks of Object Orientation

Nothing is perfect including object orientation. While many benefits exist to OO, they come at a price:

- OO requires greater concentration on requirements analysis and design
 - You cannot build a system that meets users' needs unless you know what those needs are (you need to do requirements)
 - You cannot built a system unless you know how it all fit together (you need to do analysis and design)
 - But this fact is often ignored by many developers
- Developers must closely work with users

- Users are the experts but they have their own jobs to do (busy)
- OO requires a complete change in the mindset on the part of individuals
 - They should understand the benefits of OO
- OO is just more than programming
- Many OO benefits are long term
 - OO truly pays off when you extend and enhance your system
- OO demands up front investments in training education and tools
 - Organizations must train and educate their development staff.
 - Development tools and magazines
- OO techniques do not guarantee you will build the right system
 - While OO increases the probability of project success, it still depends on the ability of individuals involved.
 - Developers, users, managers must be working together to have a good working atmosphere
- OO necessitates increased testing
 - OO is typically iterative in nature, and probably developing complex system using the objects, the end result is you need to spend more time in testing.
- OO is only part of the solution
 - You still need CASE tools
 - Need to perform quality assurance (QA)

5.4 Object Standards

Object orientation today becomes the significant part of the software development. Objects are the primary enabling technology for components. It defines requirements for the information, geometry, behavior, and presentation of objects, to give reassurance of quality that will enable greater collaboration and efficient information exchange across construction industry for the system. By standardizing objects, you can consistently use, compare, analyze, and share information to make informed decisions quickly and confidently. The standard is paramount, not just for some objects as we can create objects to a common data set.

Since the lack of structure and data consistency recognized across all objects, object standard is required to achieve data consistency and better structure with recognizable set of criteria, to which all objects can be created.

Basic standards of object for analysis include:

- Identifying objects
- Identifying the object relationships
- Identifying the attributes
- Identifying services
- Defining object life cycles

5.5 The Object Orientation Software Process

Software process is a set of project phases, stages, methods, techniques, and practices that people employ to develop and maintain software and its associated artifacts. It enables organizations to increase productivity when developing software.

Any modern object-oriented approach to develop software process must have objectives, a focus of activity over the workflows, and incremental deliverables. Each of the phases is described as follows:

5.5.1 Inception

In this phase, a business case is made for the proposed system. This includes feasibility analysis that should answer questions such as the following:

- Do we have the technical capability to build it?
- If we build it, will it provide business value?
- If we build it, will it be used by the organization?

To answer these questions, the development team performs work related primarily to the business modeling, requirements, and analysis workflows.

This implies that the design, implementation, and test workflows also could be involved. The primary deliverables from the inception phase are:

- A vision document that sets the scope of the project, identifies the primary requirements and constraints, sets up an initial project plan, and describes the feasibility and risks associated with the project
- The adoption of the necessary environment to develop the system

5.5.2 Elaboration

The analysis and design workflows are the primary focus during this phase. This phase continues with developing the vision document, including finalizing the business case, revising the risk assessment, and completing a project plan in sufficient detail to allow the stakeholders to be able to agree with constructing the actual final system.

It deals with gathering the requirements, building the UML structural and behavioral models of the problem domain, and detailing the how the problem domain models fit into the evolving system architecture. The primary deliverables of this phase include:

- The UML structure and behavior diagrams
- An executable of a baseline version of the evolving information system

5.5.3 Construction

The construction phase, as expected by its name, is heavily focused on programming the evolving information system. As such, it is primarily concerned with the implementation workflow. However, the requirements, analysis, and design workflows also are involved with this phase. It is during this phase that missing requirements are uncovered, and the analysis and design models are finally completed. At times, an iteration may have to be rolled back. The primary deliverable of this phase is an implementation of the system that can be released for beta

and acceptance testing.

5.5.4 Transition

Like the construction phase, the transition phase addresses aspects typically associated with the implementation phase of SDLC approach. Its primary focus is on the testing and deployment workflows. Depending on the results from the testing workflow, it is possible that some redesign and programming activities on the design and implementation workflows could be necessary, but they should be minimal at this point in time. From a managerial perspective, the project management, configuration and change management, and environment are involved. The other deliverables include user manuals, a plan to support the users, and a plan for upgrading the information system in the future.

5.6 Software Development Process Models

5.6.1 Waterfall Model

The simplest process model is the waterfall model, which states that the phases are organized in a linear order. In this model, a project begins with feasibility analysis. Upon successfully demonstrating the feasibility of a project, the requirements analysis and project planning begins. The design starts after the requirements analysis is complete, and coding begins after the design is complete. Once the programming is completed, the code is integrated and testing is done. Upon successful completion of testing, the system is installed. After this, the regular operation and maintenance of the system takes place. The requirements analysis phase is mentioned as “analysis and planning.” Planning is a critical activity in software development. A good plan is based on the requirements of the system and should be done before later phases begin. Linear ordering of activities has some important consequences. First, to clearly identify the end of a phase and the beginning of the next. This is usually done by some verification and validation means that will ensure that the output of a phase is consistent with its input (which is the output of the previous phase), and that the output of the phase is consistent with the overall requirements of the system. The consequence of the need for certification is that each phase must have some defined output that can be evaluated and certified. That is, when the activities of a phase are completed, there should be some product that is produced by that phase. The outputs of the earlier phases are often called work products and are usually in the form of documents like the requirements document or design document.

1. Waterfall Model

- Also called Linear Sequential Model/Classic Life Cycle Model.

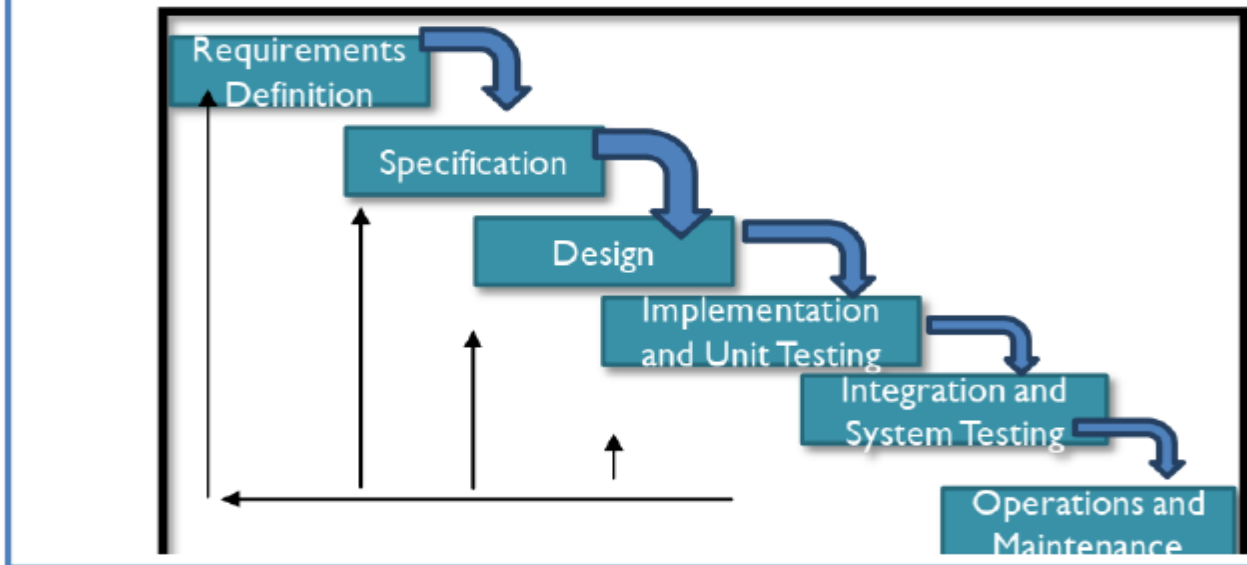


Figure 45: Waterfall Model

Advantages:

- One of the main advantages of this model is its simplicity.
- It is conceptually straightforward and divides the large task of building a software system into a series of cleanly divided phases, each phase dealing with a separate logical concern.
- It is also easy to administer in a contractual setup as each phase is completed and its work product produced, some amount of money is given by the customer to the developing organization.

Limitations:

- It assumes that the requirements of a system can be frozen before the design begins. But for new systems, determining the requirements is difficult as the user does not even know the requirements.
- Freezing the requirements usually requires choosing the hardware, may become obsolete over a period of time.
- The entire software is delivered in one shot at the end. This entails heavy risks, as the user does not know until the very end what they are getting.

5.6.2 Prototyping Model

The goal of a prototyping-based development process is that instead of freezing the requirements before any design or coding can proceed. This prototype is developed based on the currently known requirements.

Development of the prototype obviously undergoes design, coding, and testing, but each of these phases is not done very formally or thoroughly.

By using this prototype, the client can get an actual feel of the system; because the interactions with the prototype can enable the client to, better understand the requirements of the desired system. Prototyping is an attractive idea for complicated and large systems for which there is no manual process or existing system to help determine the requirements. In both situations, the risks associated with the projects are being reduced through the use of prototyping.

After the prototype has been developed, the end users and clients are given an opportunity to use the prototype. Based on their experience, they provide feedback to the developers. Based on the feedback, the prototype is modified to incorporate some of the suggested changes that can be done easily, and then the users and the clients are again allowed to use the system. Based on the feedback, the initial requirements are modified to produce the final requirements specification, which is then used to develop the production quality system.

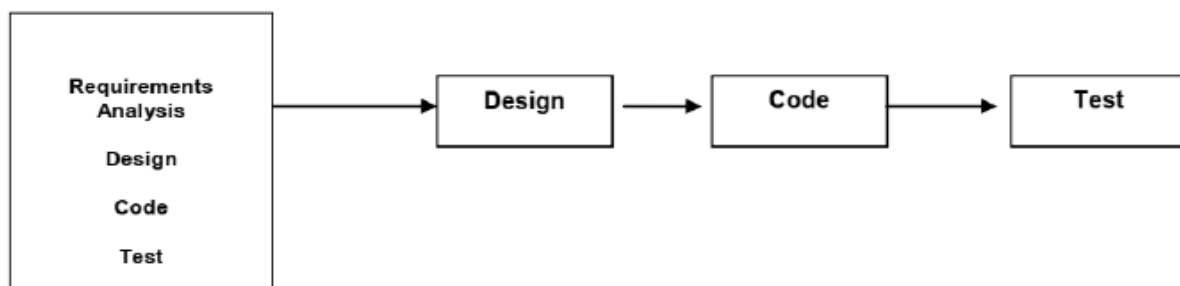


Figure 46: Prototyping Model

The focus of the development is to include those features that are not properly understood. And the development approach is with the focus on quick development rather than quality.

Advantages:

- Users are actively involved in the development
- It provides a better system to users, as users have natural tendency to change their mind in specifying requirements and this method of developing systems supports this user tendency.
- Errors can be detected much earlier as the system is made side by side.

Limitations:

- This Model Leads to ‘implementing and then repairing’ way of building systems.
- This may increase the complexity of the system as scope of the system may expand beyond original plans.
- Cost of implementing this method for larger or complex systems is more.

5.6.3 Iterative Model

The iterative development process model tries to combine the benefits of both prototyping and

the waterfall model. The basic idea is that the software should be developed in increments, each increment adding some functional capability to the system until the full system is implemented. At each step, extensions and design modifications can be made. An advantage of this approach is that it can result in better testing because testing each increment is likely to be easier than testing the entire system as in the waterfall model. The prototyping, the increments provide feedback to the client that is useful for determining the final requirements of the system.

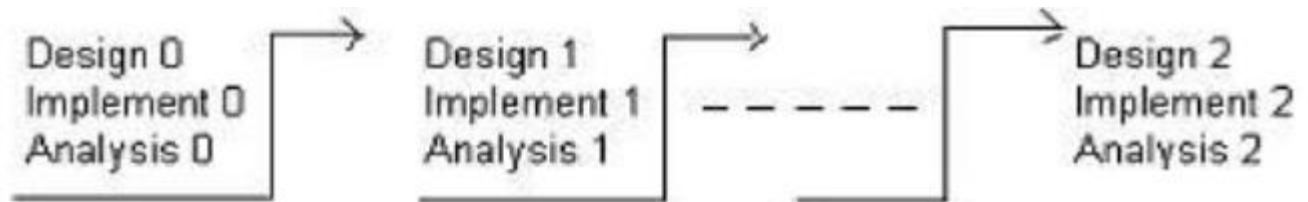


Figure 47: Iterative Model

5.6.4 Spiral Model

The spiral model is another iterative model that has been proposed. As the name suggests, the activities in this model can be organized like a spiral that has many cycles as shown below:

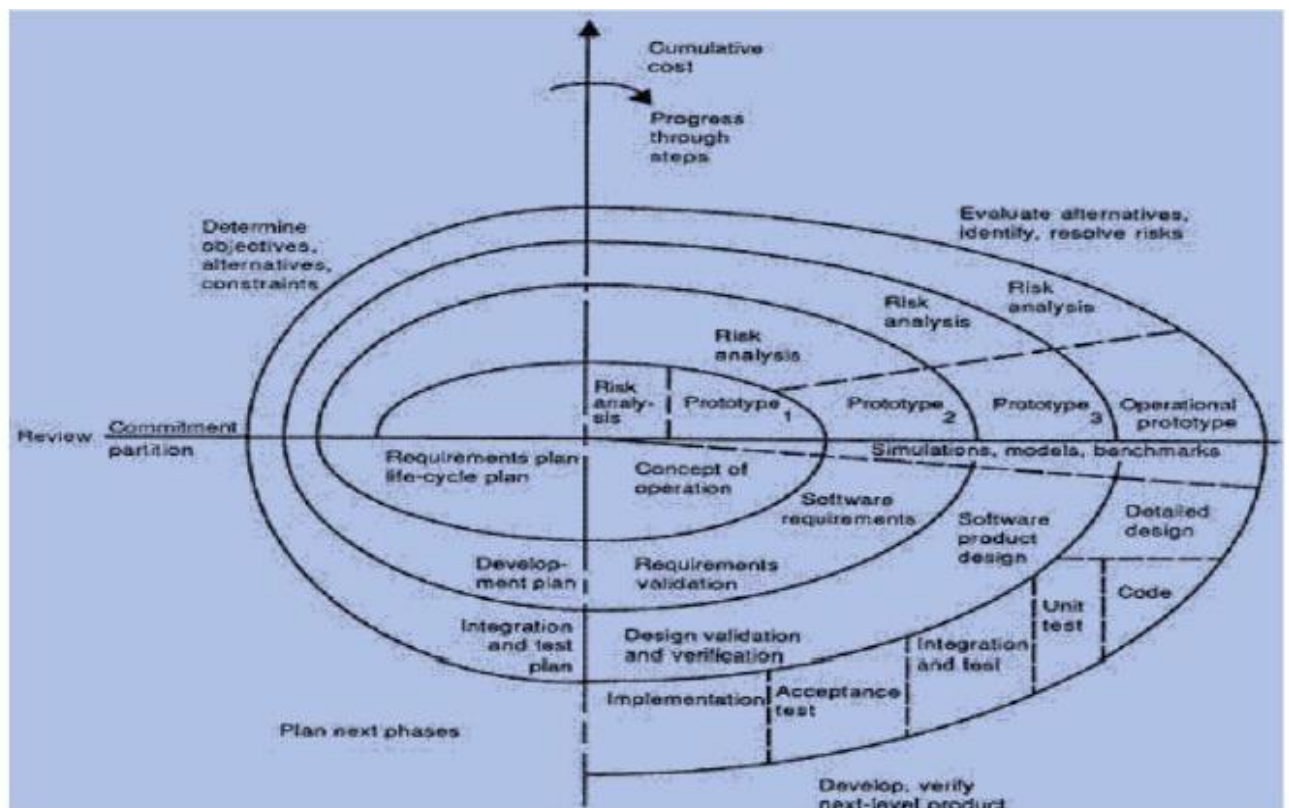


Figure 48: Spiral Model

Each cycle in the spiral begins with the identification of objectives for that cycle, the different alternatives that are possible for achieving the objectives, and the constraints that exist. The focus of evaluation in this step is based on the risk perception for the project. The next step is to develop strategies that resolve the uncertainties and risks.

UNIT - VI

6.1 Overview of Requirements Gathering

Requirements identify the objective of a product. It is features of system or system function used to fulfill system purpose. It focuses on customer's needs and problem, not on solutions. Before anything is done to develop a product, we must identify what the system must do. Therefore, understanding what requirements are, what they are not, and how best to identify them is crucial to the success of the system. Development of a system must follow a set of practices, procedures, rules, and techniques called methodology.

A requirement gathering is also called requirements elicitation. It is a process of collecting the user needs to solve a problem or issues and to achieve an objective. If the requirement gathering is not done properly/ completely, all the subsequent phase is incomplete, no matter how best the design, until and unless requirements are complete. So, we should carefully plan and carry out the requirements gathering with a systematic approach. Gathering user requirements is the first step in software development. Types of requirements can be categorized as follows:

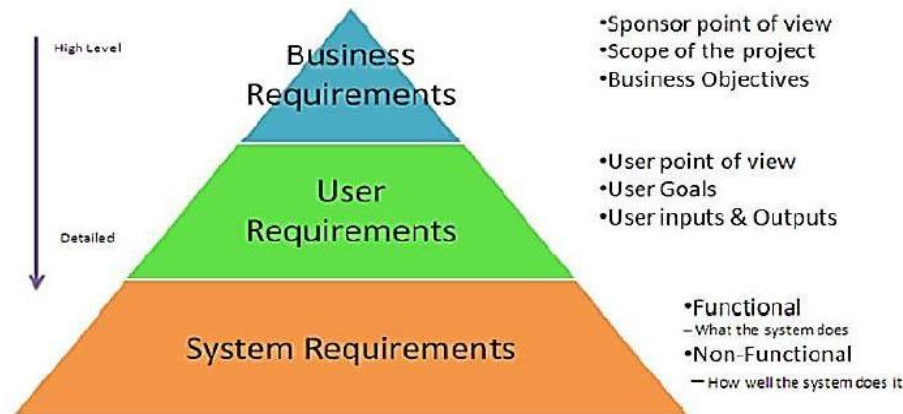


Figure 49: Types of Requirements

6.1.1 Functional Requirements

It specifies the software functionality that the developers must build into the product to enable users to accomplish their tasks, thereby satisfying the business requirements. It is a state what the system must do. In fact, it is usually stated by using the “shall” statement.

Example:

- The website shall notify the administrator via email when a user register with it

6.1.2 Non – Functional Requirements

Non – functional requirements are Constraints or standards that the system must have or comply with. It defines the system's quality characteristics. Non – functional requirements describe aspects of the system that are not directly related to the functional behavior of the system. Nonfunctional requirements include a broad variety of requirements that apply to many different aspects of the system, from usability to performance. These requirements are discussed below:

- **Usability:** is the ease with which a user can learn to operate, prepare inputs for, and interpret outputs of a system or component. Usability requirements include, for example, conventions adopted by the user interface, the scope of online help, and the level of user documentation. Often, clients address usability issues by requiring the developer to follow user interface guidelines on color schemes, logos, and fonts.
- **Reliability:** is the ability of a system or component to perform its required functions under stated conditions for a specified period of time. Reliability requirements include, for example, an acceptable mean time to failure and the ability to detect specified faults or to withstand specified security attacks.
- **Robustness:** the degree to which a system or component can function correctly in the presence of invalid inputs or stressful environment conditions.
- **Safety:** a measure of the absence of catastrophic consequences to the system.
- **Performance:** requirements are concerned with quantifiable attributes of the system, such as response time (how quickly the system reacts to a user input).
- **Supportability:** requirements are concerned with the ease of changes to the system after deployment, including for example, adaptability (the ability to change the system to deal with additional application domain concepts), maintainability (the ability to change the system to deal with new technology or to fix defects), and internationalization (the ability to change the system to deal with additional international conventions, such as languages, units, and number formats).
- **Portability:** (the ease with which a system or component can be transferred from one hardware or software environment to another).

6.2 Putting Together Requirements Gathering Team

- Choosing Good Subject-Matter Experts – is a person who has special skills or knowledge on a particular job or topic.
- Choosing Good Facilitators – is to make easy or ease a process.
- Choosing Good Scribes – a scribe is a person who copies manuscripts or a pointed instrument used for marking where something should be cut.

6.3 Fundamental Requirements Gathering Techniques

6.3.1 Interview

Interviewing is one of the primary ways analysts gather information about an information systems project. Early in a project, an analyst may spend a large amount of time interviewing people about their work, the information they use to do it, and the types of information processing that might supplement their work. Others are interviewed to understand organizational direction, policies, and expectations that managers have of the units they supervise. During interviewing, you gather facts, opinions, and speculation and observe body language, emotions, and other signs of what people want and how they assess current systems. Interviewing someone effectively can be done in many ways, and no one method is necessarily better than another. Some guidelines to keep in mind when you interview are summarized in the following table and are discussed next.

Guidelines	What Is Involved
Plan the interview	Prepare interviewee by making an appointment and explaining the purpose of the interview. Prepare a checklist, an agenda, and questions.
Be neutral	Avoid asking leading questions.
Listen and take notes	Give your undivided attention to the interviewee and take notes or tape-record the interview (if permission is granted).
Review notes	Review your notes within forty-eight hours of the meeting. If you discover follow-up questions or need additional information, contact the interviewee.
Seek diverse views	Interview a wide range of people, including potential users and managers.

Table 29: Interview Guidelines

First, prepare thoroughly before the interview. Set up an appointment at a time and for duration that is convenient for the interviewee. The general nature of the interview should be explained to the interviewee in advance. You may ask the interviewee to think about specific questions or issues, or to review certain documentation to prepare for the interview. Spend some time thinking about what you need to find out, and write down your questions. Do not assume that you can anticipate all possible questions. You want the interview to be natural and, to some degree, you want to direct the interview spontaneously as you discover what expertise the interviewee brings to the session. Prepare an interview guide or checklist so that you know in which sequence to ask your questions and how much time to spend in each area of the interview. The checklist might include some probing questions to ask as follow-up if you receive certain anticipated responses. You can, to some extent, integrate your interview guide with the notes you take during the interview. The first page of the sample interview guide contains a general outline of the interview. Besides basic information on who is being interviewed and when, list major objectives for the interview. These objectives typically cover the most important data you need to collect, a list of issues on

which you need to seek agreement (e.g., content for certain system reports), and which areas you need to explore.

Also, include reminder notes to yourself on key information about the interviewee (e.g., job history, known positions taken on issues, and role with current system). This information helps you to be personal, shows that you consider the interviewee important, and may assist in interpreting some answers.

Also included is an agenda with approximate time limits for different sections of the interview. You may not follow the time limits precisely, but the schedule helps you cover all areas during the time the interviewee is available. Space is also allotted for general observations that do not fit under specific questions and for notes taken during the interview about topics skipped or issues raised that could not be resolved.

On subsequent pages, list specific questions. The sample form in the following figure includes space for taking notes on these questions. Because the interviewee may provide information you were not expecting, you may not follow the guide in sequence. You can, however, check off questions you have asked and write reminders to yourself to return to or skip other questions as the interview takes place.

Interview Outline	
Interviewee: <i>Name of person being interviewed</i>	Interviewer: <i>Name of person leading interview</i>
Location/Medium: <i>Office, conference room, or phone number</i>	Appointment Date: Start Time: End Time:
Objectives: <i>What data to collect</i> <i>On what to gain agreement</i> <i>What areas to explore</i>	Reminders: <i>Background/experience of interviewee</i> <i>Known opinions of interviewee</i>
Agenda: Introduction Background on Project Overview of Interview Topics to Be Covered Permission to Tape Record Topic 1 Questions Topic 2 Questions ... Summary of Major Points Questions from Interviewee Closing	Approximate Time: 1 minute 2 minutes 1 minute 5 minutes 7 minutes ... 2 minutes 5 minutes 1 minute
General Observations: Interviewee seemed busy—probably need to call in a few days for follow-up questions because he gave only short answers. PC was turned off—probably not a regular PC user.	
Unresolved Issues, Topics Not Covered: He needs to look up sales figures from 2010. He raised the issue of how to handle returned goods, but we did not have time to discuss.	
Interviewee:	Date:
Questions:	Notes:
<i>When to ask question, if conditional</i> Question: 1 Have you used the current sales tracking system? If so, how often? If yes, go to Question 2	<i>Answer</i> Yes, I ask for a report on my product line weekly. <i>Observations</i> Seemed anxious—may be overestimating usage frequency
Question: 2 What do you like least about this system?	<i>Answer</i> Sales are shown in units, not dollars. <i>Observations</i> System can show sales in dollars, but user does not know this.

Figure 50: Interview Outlines

Interview Questions

You need to decide on the mix and sequence of open-ended and closed-ended questions to use. Open-ended questions are usually used to probe for information when you cannot anticipate all possible responses or when you do not know the precise question to ask. The person being interviewed is encouraged to talk about whatever interests him or her within the general bounds of the question. An example is, —What would you say is the best thing about the information system you currently use to do your job? or —List the three most frequently used menu options. You must react quickly to answers and determine whether any follow-up questions are needed for clarification or elaboration. Sometimes body language will suggest that a user has given an incomplete answer or is reluctant to provide certain information. If so, a follow-up question might result in more information.

One advantage of open-ended questions is that previously unknown information can surface. You can then continue exploring along unexpected lines of inquiry to reveal even more new information.

Open-ended questions also often put the interviewees at ease because they are able to respond in their own words using their own structure. Open-ended questions give interviewees more of a sense of involvement and control in the interview.

A major disadvantage of open-ended questions is the length of time it can take for the questions to be answered. They also can be difficult to summarize.

Closed-ended questions provide a range of answers from which the interviewee may choose. Here is an example: Which of the following would you say is the one best thing about the information system you currently use to do your job (pick only one)?

- Having easy access to all of the data you need
- The system's response time
- The ability to run the system concurrently with other applications

Closed-ended questions work well when the major answers to questions are well known. Another plus is that interviews based on closed-ended questions do not necessarily require a large time commitment—more topics can be covered.

Closed-ended questions can also be an easy way to begin an interview and to determine which line of open-ended questions to pursue. You can include an —other option to encourage the interviewee to add unexpected responses.

A major disadvantage of closed-ended questions is that useful information that does not quite fit the defined answers may be overlooked as the respondent tries to make a choice instead of providing his or her best answer. Like objective questions on an examination, closed-ended questions can follow several forms, including these choices:

- True or false
- Multiple choice (with only one response or selecting all relevant choices)
- Rating a response or idea on some scale, say, from bad to good or strongly agree to strongly disagree (each point on the scale should have a clear and consistent meaning to each person, and there is usually a neutral point in the middle of the scale)
- Ranking items in order of importance

Interview Guidelines

First, with either open- or closed-ended questions, do not phrase a question in a way that implies a right or wrong answer. Respondents must feel free to state their true opinions and perspectives and trust that their ideas will be considered. Avoid questions such as —Should the system continue to provide the ability to override the default value, even though most users now do not like the feature?‖ because such wording predefines a socially acceptable answer.

Second, listen carefully to what is being said. Take careful notes or, if possible, record the interview on a tape recorder (be sure to ask permission first!). The answers may contain extremely important information for the project. Also, this may be your only chance to get information from this particular person. If you run out of time and still need more information from the person you are talking to, ask to schedule a follow-up interview.

Third, once the interview is over, go back to your office and key in your notes within forty-eight hours with a word processing program such as Microsoft Word. For numerical data, you can use a spreadsheet program such as Microsoft Excel. If you recorded the interview, use the recording to verify your notes. After forty-eight hours, your memory of the interview will fade quickly. As you type and organize your notes, write down any additional questions that might arise from lapses in your notes or ambiguous information. Separate facts from your opinions and interpretations. Make a list of unclear points that need clarification.

Call the person you interviewed and get answers to these new questions. Use the phone call as an opportunity to verify the accuracy of your notes. You may also want to send a written copy of your notes to the person you interviewed to check your notes for accuracy. Finally, make sure to thank the person for his or her time. You may need to talk to your respondent again.

If the interviewee will be a user of your system or is involved in some other way in the system's success, you want to leave a good impression.

Fourth, be careful during the interview not to set expectations about the new or replacement system unless you are sure these features will be part of the delivered system. Let the interviewee know that there are many steps to the project. Many people will have to be interviewed. Choices will have to be made from among many technically possible alternatives.

Let respondents know that their ideas will be carefully considered. Because of the repetitive nature of the systems development process, however, it is premature to say now exactly what the ultimate system will or will not do.

Fifth, seek a variety of perspectives from the interviews. Talk to several different people:

potential users of the system, users of other systems that might be affected by this new system, managers and superiors, information systems staff, and others. Encourage people to think about current problems and opportunities and what new information services might better serve the organization. You want to understand all possible perspectives so that later you will have information on which to base a recommendation or design decision that everyone can accept.

6.3.2 Direct Observation

Interviewing involves getting people to recall and convey information they have about organizational processes and the information systems that support them. People, however, are not always reliable, even when they try to be and say what they think is the truth. As odd as it may sound, people often do not have a completely accurate appreciation of what they do or how they do it, especially when infrequent events, issues from the past, or issues for which people have considerable passion are involved. Because people cannot always be trusted to interpret and report their own actions reliably, you can supplement what people tell you by watching what they do in work situations.

For example, one possible view of how a hypothetical manager does her job is that a manager carefully plans her activities, works long and consistently on solving problems, and controls the pace of her work.

A manager might tell you that is how she spends her day. Several studies have shown, however, that a manager's day is actually punctuated by many, many interruptions.

Managers work in a fragmented manner, focusing on a problem or a communication for only a short time before they are interrupted by phone calls or visits from subordinates and other managers. An information system designed to fit the work environment described by our hypothetical manager would not effectively support the actual work environment in which that manager finds herself.

The intent behind obtaining system records and direct observation is to obtain more firsthand and objective measures of employee interaction with information systems. In some cases, behavioral measures will more accurately reflect reality than what employees themselves believe. In other cases, the behavioral information will substantiate what employees have told you directly.

Although observation and obtaining objective measures are desirable ways to collect pertinent information, such methods are not always possible in real organizational settings. Thus, these methods are not totally unbiased, just as no one data-gathering method is unbiased. For example, observation can cause people to change their normal operating behavior. Employees who know they are being observed may be nervous and make more mistakes than normal. On the other hand, employees under observation may follow exact procedures more carefully than they typically do. They may work faster or slower than normal. Because observation typically cannot be continuous, you receive only a snapshot image of the person or task you observe. Such a view may not include important events or activities.

Due to time constraints, you observe for only a limited time, a limited number of people, and a limited number of sites. Observation yields only a small segment of data from a possibly vast variety of data sources. Exactly which people or sites to observe is a difficult selection problem. You want to pick both typical and atypical people and sites and observe during normal and abnormal conditions and times to receive the richest possible data from observation.

6.3.3 Analyzing Documents

As previously noted, interviewing people who use a system every day or who have an interest in a system is an effective way to gather information about current and future systems. Observing current system users is a more direct way of seeing how an existing system operates.

Both interviewing and observing have limitations. Methods for determining system requirements can be enhanced by examining system and organizational documentation to discover more details about current systems and the organization they support.

What can the analysis of documents tell you about the requirements for a new system? In documents you can find information about:

- Problems with existing systems (e.g., missing information or redundant steps)
- Opportunities to meet new needs if only certain information or information processing were available (e.g., analysis of sales based on customer type)
- Organizational direction that can influence information system requirements (e.g., trying to link customers and suppliers more closely to the organization)
- Titles and names of key individuals who have an interest in relevant existing systems (e.g., the name of a sales manager who has led a study of buying behavior of key customers)
- Values of the organization or individuals who can help determine priorities for different capabilities desired by different users (e.g., maintaining market share even if it means lower short-term profits)
- Special information-processing circumstances that occur irregularly that may not be identified by any other requirements determination technique (e.g., special handling needed for a few large-volume customers who require use of customized customer ordering procedures)
- The reason why current systems are designed as they are, which can suggest features left out of current software that may now be feasible and desirable (e.g., data about a customer's purchase of competitors' products not available when the current system was designed; these data now available from several sources)
- Data, rules for processing data, and principles by which the organization operates that must be enforced by the information system (e.g., each customer assigned exactly one sales department staff member as primary contact if customer has any questions)

One type of useful document is a written work procedure for an individual or a work group. The procedure describes how a particular job or task is performed, including data and information used and created in the process of performing the job.

A second type of document useful to systems analysts is a business form. Forms are used for all types of business functions, from recording an order to acknowledging the payment of a bill to indicating what goods have been shipped. Forms are important for understanding a system because they explicitly indicate what data flow in or out of a system.

A printed form may correspond to a computer display that the system will generate for someone to enter and maintain data or to display data to online users. The most useful forms contain actual organizational data that allow you to determine the data characteristics actually used by the application. The ways in which people use forms change over time, and data that were needed when a form was designed may no longer be required.

A third type of useful document is a report generated by current systems. As the primary output for some types of systems, a report enables you to work backward from the information on the report to the data that must have been necessary to generate it.

If the current system is computer based, a fourth set of useful documents is one that describes the current information systems—how they were designed and how they work. Several different types of documents fit this description, everything from flowcharts to data dictionaries to user manuals. An analyst who has access to such documents is fortunate because many in-house-developed information systems lack complete documentation. Analysis of organizational documents and observation, along with interviewing and distributing questionnaires, are the methods used most for gathering system requirements.

Deliverables and Outcomes

The primary deliverables from requirements gathering are the types of information gathered during the determination process. The information can take many forms: transcripts of interviews; notes from observation and analysis of documents; sets of forms, reports, job descriptions, and other documents; and computer-generated output such as system prototypes. In short, anything that the analysis team collects as part of determining system requirements is included in these deliverables.

These deliverables contain the information you need for systems analysis. In addition, you need to understand the following components of an organization:

- The business objectives that drive what and how work is done
- The information people need to do their jobs
- The data handled within the organization to support the jobs
- When, how, and by whom or what the data are moved, transformed, and stored
- The sequence and other dependencies among different data-handling activities
- The rules governing how data are handled and processed

- Policies and guidelines that describe the nature of the business, the market, and the environment in which it operates

6.3.4 Questionnaires

One of the challenges with interviews is that you will only get the information that the person is consciously aware of. Sometimes there are latent requirements and features that are better obtained through questionnaires. In questionnaires, you basically prepare some questions to be answered and filled on papers so that you would obtain the information you need in gathering information.

6.4 Essential Use Case Modeling

It is a diagram of all the use-cases, actors and use case-actor association used to describe a particular system. It is semantically closed abstraction of a subject system. A use case model comprises zero or more use case diagrams, although most have at least one diagram, and one or more use-case specifications (often simply called use cases). Use case diagrams are one of the standard Unified Modeling Language (UML) artifacts. There are two basic types of use case models:

- Essential use case models
- System use case models

Essential use case model often referred to as a task case model or an abstract use case model models a technology independent view of your behavioral requirements, whereas system use case models also known as concrete use case models or detailed use case models, model your analysis of your behavioral requirements, describing in detail how users will work with your system, including references to its user-interface aspects.

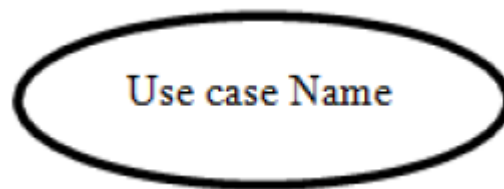
Essential use-case modeling is:

- A fundamental aspect of usage-centered designs, an approach to software development.
- Intended to capture the essence of problems through technology-free, idealized, and abstract descriptions.
- More flexible, leaving open more options and more readily accommodating changes in technology.
- More robust than concrete representations, simply because they are more likely to remain valid in the face of both changing requirements and changes in the technology of implementation.
- Ideal artifacts to capture the requirements for your system.

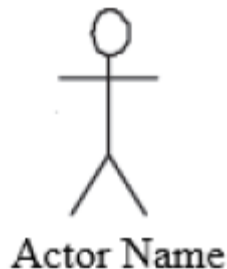
Therefore, when you are using an essential use case modeling, you develop a use case

diagram, identify essential use cases, and identify potential actors that interact with your system. The use case diagrams depict the core elements:

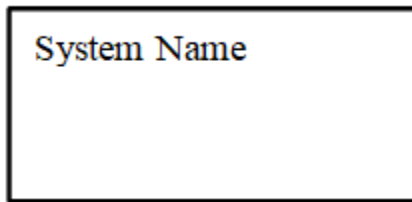
- **Use Case(s)** – describes a sequence of actions that provide something of measurable value to an actor. It describes the typical ways (or cases) of using the system. Each use case expresses the goal of the actors involved and describes the task that the system, with the assistance of appropriate actors, will perform. You can get the idea of use case's goal simply by observing its name and associations. It is drawn as a horizontal ellipse as follows:



- **Actor(s)** – is a person, organization, or external system that plays a role in one or more interactions with your system. A list of actors would be a list of all the different roles that people or other systems could play while interacting with the system. Actors in other term we can say users. Few systems operate without interacting with other systems, and many systems interact only with other systems. Actors are outside the system and usually outside the control of the system. Actors are drawn as stick figures as shown below:

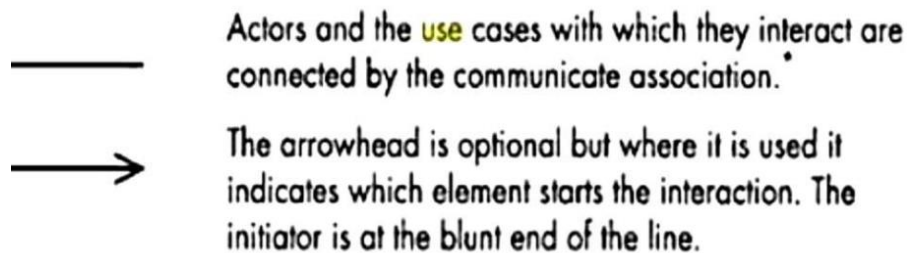


- **System Boundary (optional)** – is used to indicate the scope of your system. It is drawn by a rectangle around the use cases. Anything within the box represents functionality that is in scope, and anything outside the box is not. The symbol looks the following:



Use case diagram also depicts the core relationships:

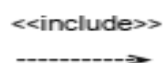
- **Association(s)** – it is a relationship between actors and use cases. It is indicated in use case diagrams by solid lines. It exists whenever an actor is involved with an interaction described by a use case. It can also exist between use cases and even between actors, although this is typically an issue for system use case models. A use case has at most one communicates association to a specific actor and an actor has at most one communicates association to a specific use case, no matter how many interactions there are. It is modeled as lines connecting use cases and actors to one another, with an optional arrowhead on one end of the line. The arrowhead is often used to indicate the direction of the initial invocation of the relationship and/or to indicate the primary actor within the use case. It is shown below:



- **Extend:** a relationship from the extension use case to a base use case specifying how the behavior of extension use case can be inserted into the behavior defined for the base use case. It is represented as follows:



- **Include:** a relationship from a base use case to inclusion use case specifying how the behavior of the inclusion use case can be inserted into the behavior defined for the base use case. It is represented as follows:



- **Generalization:** a taxonomic relationship between a ~~more general use~~ case and a more specific use case. It is represented as follows:

Example 1:

Students are enrolling in courses with the potential help of registrars. Professors input the marks students earn on assignments and registrars authorize the distribution of transcripts (report cards) to students. Note how for some use cases there is more than one actor involved. Moreover, note how some associations have arrowheads any given use case association will have a zero or one arrowhead.

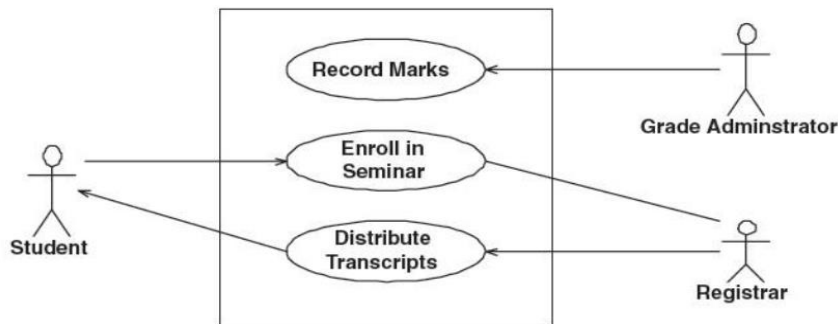


Figure 51: Use Case Diagram

Actors are always involved with at least one-use case and are always drawn on the outside edges of a use case diagram.

You should ask how the actors interact with the system to identify an initial set of use cases.

Then, on the diagram, you connect the actors with the use cases with which they are involved.

There are several interesting things to note about the use cases:

- No time ordering is indicated between use cases.
- Actors are usually involved in many use cases.
- Use cases are not functions.
- Use cases should be functionally cohesive.
- Each use case should be temporally cohesive.
- Use cases should describe something of business value.
- Repetitive actions need not be expressed within a single use case.

6.4.1 Identifying Actors

An actor represents anything or anyone that interfaces with the system. This may include people (not just the end user), external systems, and other organizations.

Actors are always external to the system being modeled; they are never part of the system. To help find actors in your system, you should ask yourself the following questions:

- Who is the main customer of your system?
- Who obtains, supply, use, remove and provides information from this or to the system?
- Who installs, operates and shut-down the system?
- What other systems interact with this system?
- Does anything happen automatically at a preset time?
- Where does the system get information?

Example:

If the person employed as the registrar at a university is also taking courses, he or she may play the roles of both Student and Registrar. This is perfectly valid from a use case point of view. To describe an actor, you want to give it a name that accurately reflects its role within your model. Actor names are usually singular nouns, such as Grade Administrator, Customer, and Payment Processor.

Identifying Use Cases

To identify use cases, is to identify potential services by asking your stakeholders the following questions from the point of view of the actors:

- What are users in this role trying to accomplish?
- To fulfill this role, what do users need to be able to do?
- What are the main tasks of users in this role?
- What information do users in this role need to examine, create, or change?
- What do users in this role need to be informed of by the system?
- What do users in this role need to inform the system about?

Example:

From the point of view of the Student actor, you may discover that students

- Enroll in, attend, drop, fail, and pass seminars.
- Need a list of available seminars.
- Need to determine basic information about a seminar, such as its description and its prerequisites.
- Obtain a copy of their transcript, their course schedules, and the fees due.
- Pay fees, pay late charges, receive reimbursements for dropped and cancelled courses, receive grants, and receive student loans.

6.5 Domain Modeling With CRC

A class responsibility collaborator (CRC) model is a collection of standard index cards that have been divided into three sections, as depicted in figure below:

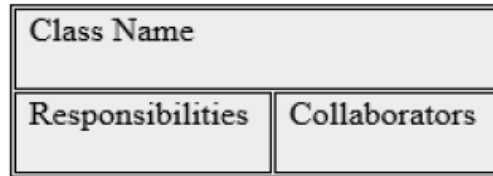


Figure 53: Layout of CRC Cards

A class represents a collection of similar objects, a responsibility is something that a class knows or does, and a collaborator is another class that a class interacts with to fulfill its responsibilities.

Example:

In a university system, classes would represent students, tenured professors, and seminars. The CRC models are an incredibly effective tool for conceptual modeling as well as for detailed design. The name of the class appears across the top of a CRC card and is typically a singular noun or singular noun phrase, such as Student, Professor, or Seminar. You use singular names because each class represents a generalized version of a singular object. The information about a student describes a single person, not a group of people. Therefore, it makes sense to use the name Student and not Students. Class names should also be simple.

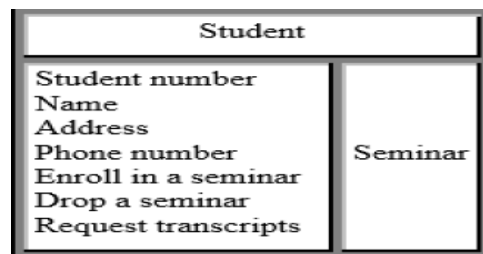


Figure 54: CRC Example

A responsibility is anything that a class knows or does. For instance, Students have names, addresses, and phone numbers. A student knows these things. Students also enroll in seminars, drop seminars, and request transcripts. A student does these things. The things a class knows and does constitute its responsibilities. Important: A class is able to change the values of the things it knows, but it is unable to change the values of what other classes know. Sometimes a class has a responsibility to fulfill, but not have enough information to do it. As you see in figure above, students enroll in seminars. To do this, a student needs to know whether a spot is available in the seminar and, if so, he then needs to be added to the seminar. However, students only have information about themselves (their names and so forth), and not about seminars. What the student needs to do is collaborate/interact with the card labeled Seminar to sign up for a seminar. Therefore, Seminar is included in the list of collaborators of Student.

Collaboration takes one of two forms: A request for information or a request to do something. For example, the card Student requests an indication from the card Seminar whether a space is available, a request for information.

Student then requests to be added to the Seminar, a request to do something. Another way to perform this logic, however, would have been to have Student simply request Seminar to enroll himself into itself.

Then have Seminar do the work of determining whether a seat is available. If the seat is available, then enroll the student; if not, and then inform the student that he was not enrolled.

So how do you create CRC models? Iteratively perform the following steps:

⌘Find Classes

⌘Find Responsibility – you should ask yourself what a class does. Example: Transcripts must be able to calculate their average marks and rooms must be able to indicate when they are available

⌘Define Collaborators – collaboration must occur when a class needs information it does not have. Similarly, collaboration occurs when a class needs to modify information it does not have because any given class can update only the information it knows. This implies that if it needs to have information updated in another class, then it must ask that class to update it.

⌘Move the cards around – CRC modeling is typically performed by a group of people around a large desk. As the CRC cards are created, they are placed on the desk, so everyone can see them. Two cards that collaborate with one another should be placed close together on the table, whereas two cards that do not collaborate should be placed far apart. Furthermore, the more two cards collaborate; the closer they should be on the desk. By having cards that collaborate with one another close together, it is easier to understand the relationships between classes.

More CRC Examples:

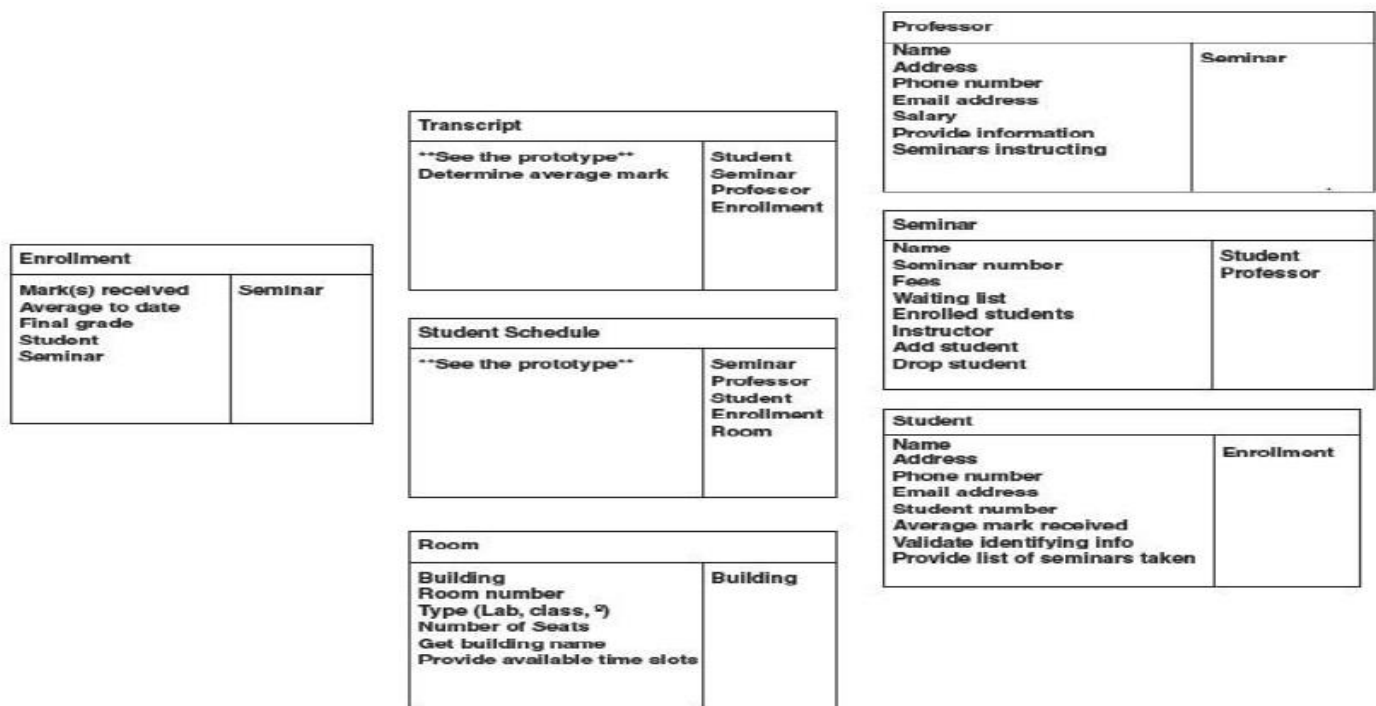


Figure 55: More CRC Examples

6.5.1 Advantages of CRC Cards

- The experts do the analysis
- User participation increased
- Breaks down communication barriers
- Simple and straightforward
- Portable
- Transition – ease of transition from process orientation to object-orientation.

6.5.2 Disadvantages of CRC Cards

- Inefficient for large systems
- Limited details
- Hard to get users together.
- CRC cards are limited.

6.6 Developing Supplementary Specification

It is a RUP (Rational Unified Process) document that contains requirements not contained directly in your use cases. This often includes business rules, technical requirements, and constraints. In short, it is a container into which you place other requirements. It is not model in their own right but instead is the documentation of models.

It consists of:

- **Business Rule (BR):** It defines or constrains one aspect of your business that is intended to assert business structure or influence the behavior of your business. It often focuses on access control issues. Some BR focus on the policies of your organization; perhaps the university policy is to expel for one year anyone who fails more than two courses in the same semester.

Example:

- Professors are allowed to input and modify the marks of the students taking the seminars they instruct, but not the marks of students in other seminars.
- How to convert a percentage mark (for example, 91 percent) that a student receives in a seminar into a letter grade (for example, A-).

It has a unique identifier. By convention use the format of BR#, but you are free to set your own numbering approach.

The unique identifier enables you to refer easily to business rules in other development artifacts, such as class models and use cases. There are at least three sections of a business rules:

- Name** – the name should give you a good idea about the topic of the business rule.
- Description** – the description defines the rule exactly. Although I used text to describe this rule it is quite common to see diagrams such as flow charts or UML activity diagrams used to

describe an algorithm.

c. **Example** (optional) – an example of the rule is presented to help clarify it.

d. **Source** (optional) – the source of the rule is indicated so it may be verified (it is quite common that the source of a rule is a person, often one of your project stakeholders, or a team of people).

e. **Related Rules** (optional) – a list of related business rules, if any, is provided to support traceability between rules.

f. **Revision History** (optional) – an indication of the date a change was made, the person who made the change, and a description of the change.

Example:

♂	BR001 - Tenured professors may administer student grades.
♂	BR002 - Teaching assistants who have been granted authority by a tenured professor may administer student grades.
♂	BR003 - Table to convert between numeric grades and letter grades.
♂	BR004 - All master's degree programs must include the development of a thesis.

- **Constraints:** It is a restriction on the degree of freedom you have in providing a solution. It is effectively global requirements, such as limited development resources or a decision by senior management that restricts the way you develop a system. It can be economic, political, technical, or environmental and pertain to your project resources, schedule, target environment, or to the system itself.

Example:

The potential constraints for the university system

♂	C24 - the system will work on our existing technical infrastructure - no new technologies will be introduced.
♂	C56 - the system will only use the data contained in the existing corporate database.
♂	C73 - the system shall be available 99.99 percent of the time for any 24-hour period.
♂	C76 - all master's degree programs must include the development of a thesis.

6.7 Identifying Change Cases

Change cases are used to describe new potential requirements for a system or modifications to existing requirements. You describe the potential change to your existing requirements, indicate the likeliness of that change occurring, and indicate the potential impact of that change.

Example:

Consider an existing University

♂ **Change case:** The University will open a new campus.
♂ **Likelihood:** Certain. It has been announced that a new campus will be opened in two years across town.
♂ **Impact:** Large. Students will be able to register in classes at either campus. Some instructors will teach at both campuses. Some departments, such as Information Technology and Computer Science, are slated to move their entire programs to the new campus. The likelihood is great that most students will want to schedule courses at only one of the two campuses, so we will need to make this easy to support.

The name of a change case should describe the potential change itself. Change cases are often the result of brainstorming with your project stakeholders. Good questions to consider include:

- How can the business change?
- What is the long-term vision for our organization?
- What technology can change?
- What legislation can change?
- What is your competition doing?
- What systems will we need to interact with?
- Who else might use the system and how?

UNIT - VII

7.1 Requirements Validation Vs Requirements Verification

7.1.1 Requirements Validation

- Checking a work product against higher-level work products or authorities that frame this particular product.
- It is a heterogeneous process based on application of a great variety of independent techniques.
- It is nothing more than checking whether the analysts have understood the stakeholders' intention correctly and have not introduced any errors when writing the specification.
- Requirements are validated by stakeholders.

7.1.2 Requirements Verification

- Checking a work product against some standards and conditions (specification) imposed on this type of product and the process of its development to allow them to be used effectively to guide further work.
- Requirements are verified by the analysts mainly
- It is shown graphically as follows:

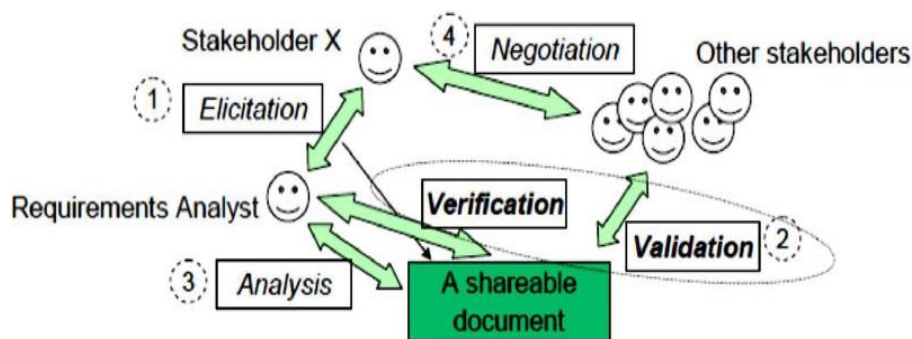


Figure 56: Requirements Verification and validation

- A stakeholder is a party that has an interest in a company and can either affect or be affected by the business.

- Requirements elicitation is the practice of researching and discovering the requirements of a system from users, customers, and other stakeholders.
- Negotiation is an open process for two parties to find an acceptable solution to a complicated conflict.

7.1.3 Characteristics of Requirements Validation and Verification

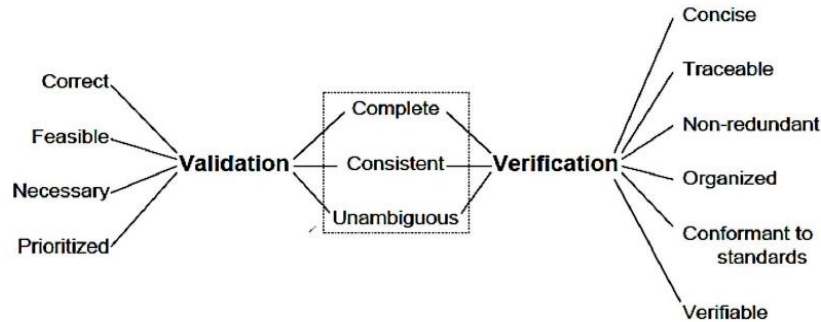


Figure 57: Characteristics of Requirements Validation and Verification

7.1.4 Requirements Validation Techniques

The most common validation technique that can be performed manually is called review. Three major types of reviews can be differentiated:

- **Prototypes** – A prototype allows the stakeholders to try out the requirements for the system and experience them thereby: develop the prototype (tool support), Training of the stakeholders, Observation of prototype usage, and Collect issues.
- **Inspections** – A static analysis technique that relies on visual examination of development products to detect errors, violations of development standards, and other problems.
- **Walk-throughs** – is a meeting where you gather all of your stakeholders together and walk- through the requirements documentation, page-by-page, line-by-line, to ensure that the document represents everyone's complete understanding of what is to be accomplished in this particular project.

7.2 Testing Early and Often

Testing is the process of evaluating a system or its components with the intent to find that whether it satisfies the specified requirements or not. It is executing a system in order to identify any gaps, errors, or missing requirements in contrary to the actual desire or requirements.

Who Tests?



Figure 58: Who Tests?

There are mainly three reasons why we should start testing early and often:

- With early and regularly tests we have a much higher chance of catching up with our due dates. If we start early, we have more time to resolve discovered defects, and if we test more often, we have a better chance of catching more defects faster, and we are going to have more time to fix them.
- Testing early and often saves us much effort. Testing early and often can help us to catch errors in very first stages of development. If we cannot or don't find defects soon as we move on in the software development process, it is going to get more complicated and harder for us to discover the defects and we have to put a lot of effort and time to dive into the code and figure it out.
- It is easier to get back on track. If we start testing soon and often, we can catch the defects fast and in the early stages so if we have to (want to) change our design and approaches to address the defect we have more time and fewer complications ahead of us compare to our options in the later stages.

7.2.1 Methods of Testing

- **Black Box Testing:** Testing without having any knowledge of the interior workings of the applications. It is also called blind testing or closed box testing. The tester does not have access to the source code.
- **White Box Testing:** Detailed investigation of internal logic and structure of the code. It is also called glass testing or open box testing. The tester needs to possess knowledge of the internal working of the code.

7.3 Use Case Scenario Testing

It is simply called test documentation. A use case represents the actions that are required to enable or abandon a goal. A use case has multiple —paths that can be taken by any user at any

one time.

Scenario describes of how one or more people or organizations interact with the system.

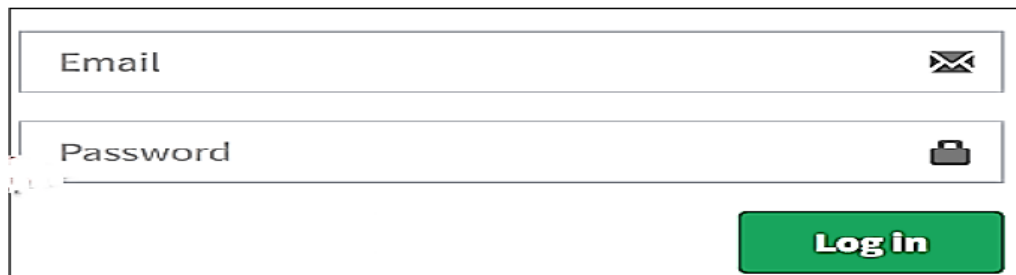
Scenario describes the steps, events, and/or actions which occur during the interaction. A use-case scenario is a single path through the use-case.

A Test Scenario is defined as any functionality that can be tested. It is also called test condition or test possibility. As a tester, you may put yourself in the end user's shoes and figure out the real- world scenarios and use cases of the application under test.

Use case testing is defined as a software testing technique, that helps identify test cases that cover the entire system, on a transaction by transaction basis from start to the finishing point.

Example 1:

Suppose in a use case for —Login functionality, to access a Gmail of a Web Application. An actor is represented by “A” and system by “S”. The UI is shown as follows:



The image shows a login form with two input fields. The first field is labeled 'Email' and has an envelope icon on the right. The second field is labeled 'Password' and has a lock icon on the right. Below these fields is a green button with the text 'Log In' in white.

Figure 59: Use Case Scenario Testing

The basic main test cases are:

Main Success Scenario	Step	Description
A:Actor S:System	1	A: Enter Agent Name & Password
	2	S: Validate Password
	3	S: Allow Account Access
Extensions	2a	<u>Password not valid</u> S: Display Message and ask for re-try 4 times
	2b	<u>Password not valid 4 times</u> S: Close Application

Explanation of the test cases:

- In the first step, the Actor enters email and password for end-to-end scenario of login functionality.

- In the next step, the system will validate the password.
- Next, if the password is correct, the access will be granted.
- There can be an extension of this use case. In case password is not valid system will display a message and ask for re-try four times.
- If Password, not valid four times system will ban the IP address.

Example 2:

Suppose in a use-case for —Login functionality of an ATM, an actor is represented by “A” and system by “S”. Here below, the test cases and description:

Main Success Scenario A: Actor S: System	Step	Description
	1	A: Inserts card
	2	S: Validates card and asks for PIN
	3	A: Enters PIN
	4	S: Validates PIN
	5	S: Allows access to account
Extensions	2a	Card not valid S: Display message and reject card
	4a	PIN not valid S: Display message and ask for re-try (twice)
	4b	PIN invalid 3 times S: Eat card and exit

Figure 60: Use Case Scenario Testing

7.3.1 Advantages of Use Case Scenario Testing

- It helps to capture the functional requirements of a system.
- It is traceable.
- It can serve as the basis for the estimating, scheduling, and validating effort.
- It can evolve at each iteration from a method of capturing requirements, to development guidelines to programmers, to a test case and finally into user documentation.

7.3.2 Disadvantages of Use Case Scenario Testing

- Poor identification of structure and flow.
- Time-consuming to generate.
- Limited software tool support.
- Still poor integration with established methods.

UNIT - VIII

8.1 Introduction

Your requirements model, although affective for understanding what your users want to have built, is not as effective at understanding what will be built. Object-oriented analysis techniques, such as system use-case modeling, sequence diagramming, class modeling, activity diagramming, and user-interface prototyping are used to bridge the gap between requirements and system design. Requirements engineering focuses on understanding users and their usage, whereas analysis focuses on understanding what needs to be built. Therefore, the purpose of analysis is to understand what will be built which is an iterative process.

OO Analysis is the process that groups the items that interact with one another, typically by class, attributes, or operations, to create a model that accurately represent the intended purpose of the system as a whole.

8.2 System Use Case Modeling

During analysis, your main goal is to evolve your essential use case into system use cases. The main difference between an essential use case and a system use case is, in the system use case, you include high-level implementation decisions.

Example:

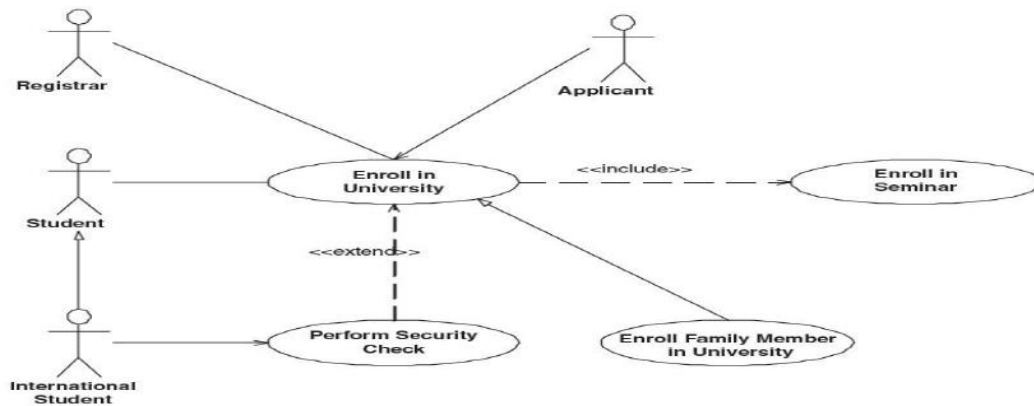


Figure 61: System Use Case Diagram

A system use case model is composed of a use case diagram and the accompanying documentation describing the use cases, actors, and associations.

8.3 Use Case Documentation (Use Case Description)

The use case has a basic course of action, which is the main start-to-finish path the user will follow.

When you document your use case, the following sections are included:

- Name- the name of the use case. The name should implicitly express the user's intent of the use case
- Description- several sentences summarizing the use case
- Actors [optional]-list of actors associated with the use case
- Status [optional] – an indication of the status of the use case like work in progress, ready for review, passed review, or failed review
- Preconditions- a list of conditions, if any, that must be met before a use case may be invoked
- Post conditions- a list of conditions, if any, that will be true once the use case finishes successfully
- Extension points [optional]- list of the points in a use case from which other use cases extend
- Include use cases [optional] a list of use cases this one includes
- Basic Course of Action- the main path of logic an actor follows through a use case. Often referred to as
 - happy path or the —main path, because it describes how the use case works when everything works as it normally should
- Alternative Course of Action- the infrequently used paths of logic in a use case a result of exceptions

There are two common styles exist for writing use cases:

a. Narrative Style – it is used to write the basic and alternative courses of action are written one step at a time.

Example:

Name: Enroll in Seminar

Identifier: UC 17

Description:

Enroll an existing student in a seminar for which she is eligible.

Preconditions:

The Student is registered at the University.

Post conditions:

The Student will be enrolled in the course she wants if she is eligible and room is available.

Basic Course of Action:

1. The use case begins when a student wants to enroll in a seminar.
2. The student inputs her name and student number into the system via *UI23 Security Login Screen*.
3. The system verifies the student is eligible to enroll in seminars at the university according to business rule *BR129 Determine Eligibility to Enroll*. [Alt Course A]
4. The system displays *UI32 Seminar Selection Screen*, which indicates the list of available seminars.
5. The student indicates the seminar in which she wants to enroll. [Alt Course B: The Student Decides Not to Enroll]
6. The system validates the student is eligible to enroll in the seminar according to the business rule *BR130 Determine Student Eligibility to Enroll in a Seminar*. [Alt Course C]

7. The system validates the seminar fits into the existing schedule of the student according to the business rule *BR143 Validate Student Seminar Schedule*.
8. The system calculates the fees for the seminar based on the fee published in the course catalog, applicable student fees, and applicable taxes. Apply business rules *BR 180 Calculate Student Fees* and *BR45 Calculate Taxes for Seminar*.
9. The system displays the fees via *UI33 Display Seminar Fees Screen*.
10. The system asks the student if she still wants to enroll in the seminar.
11. The student indicates she wants to enroll in the seminar.
12. The system enrolls the student in the seminar.
13. The system informs the student the enrollment was successful via *UI88 Seminar Enrollment Summary Screen*.
14. The system bills the student for the seminar, according to business rule *BR100 Bill Student for Seminar*.
15. The system asks the student if she wants a printed statement of the enrollment.
16. The student indicates she wants a printed statement.
17. The system prints the enrollment statement *UI89 Enrollment Summary Report*.
18. The use case ends when the student takes the printed statement.

Alternate Course A: The Student is Not Eligible to Enroll in Seminars.

A.3. The registrar determines the student is not eligible to enroll in seminars.

A.4. The registrar informs the student he is not eligible to enroll.

A.5. The use case ends.

Alternate Course B: The Student Decides Not to Enroll in an Available Seminar

B.5. The student views the list of seminars and does not see one in which he wants to enroll.

B.6. The use case ends.

Alternate Course C: The Student Does Not Have the Prerequisites

C.6. The registrar determines the student is not eligible to enroll in the seminar he chose.

C.7. The registrar informs the student he does not have the prerequisites.

C.8. The registrar informs the student of the prerequisites he needs.

C.9. The use case continues at Step 4 in the basic course of action.

b. Action-Response Style – it is used to present use case steps in columns, one column for each actor and a second column for the system.

Example:

Student

1. The student wants to enroll in a seminar.
2. The student inputs his name and student number into the system via "UI23 Security Login Screen."
5. The student indicates the seminar in which she wants to enroll.

System

3. The system verifies the student is eligible to enroll in seminars at the university, according to business rule "BR129 Determine Eligibility to Enroll."
4. The system displays "UI32 Seminar Selection Screen," which indicates the list of available seminars.

The advantage of the action-response style is, it is easier to see how actors interact with the system and how the system responds. The disadvantage is, it is little harder to understand the flow of logic of the use case.

8.4 Sequence Diagram

It is an interaction diagram that shows how the objects and classes involved in the scenario operate with one another and the sequence of messages exchanged.

It is used to model the logic of usage scenarios. A usage scenario is exactly what its name indicates – the description of a potential way your system used.

The logic of a usage scenario may be part of a use case. It may also be one entire pass through a use case or the logic contained in several use cases.

The boxes across the top of the diagram represent classifiers or their instances, typically use-cases, objects, classes, or actors. Because, you can send messages to both objects and classes, objects respond to messages through the invocation of an operation, and classes do so through the invocation of static operations, it makes sense to include both on sequence diagrams. Because actors initiate and take an active part in usage scenarios, they are also included in sequence diagrams. Objects have labels in the standard UML format “ObjectName: ClassName,” where ‘ObjectName’ is optional (objects that haven’t been given a name on the diagram are called anonymous objects). Classes have labels in the format ‘ClassName,’ with an indication of <> and actors have names in the format ‘ActorName’ with an indication of <> and for major user-interfaces should be stereotyped as <> and the use-case with <> stereotype.

The dashed lines (vertical lines) hanging from the boxes are called object lifelines representing the life span of the object during the scenario being modeled.

The long, thin boxes on the lifelines are method-invocation boxes indicating that processing being performed by the target object/ class to fulfill a message.

The X at the bottom of a method-invocation box is a UML convention to indicate that an object has been removed from memory, typically the result of receiving a message with the stereotype of <>.

Messages are indicated as labeled arrows, when the source and target of a message is an object or class the label is the signature of the method invoked in response to the message. However, if either the source or target is a human actor, the message is labeled with brief text describing the information being communicated.

Return values are optionally indicated as using a dashed arrow with a label indicating the return value. Messages fulfill the logic of the steps of the use-case, summarized down the left-hand side of the diagram or on the label.

Generally, there are basically five types of actions explicitly used in a UML sequence diagram.

a. create – is used to create an object. It tells a class to create an instance of itself.

b. call – invokes an operation on an object.

c. send – is used to send a message to an object.

d. return – is the return of a value to the caller, in response to a call action.

e. destroy – is used to destroy an object. Represented with X

Example:

The sequence of actions performed for ATM system. In this case, Client, and ATM are actors and Controller and Database are anonymous object of the concrete classes. The diagram is shown below:

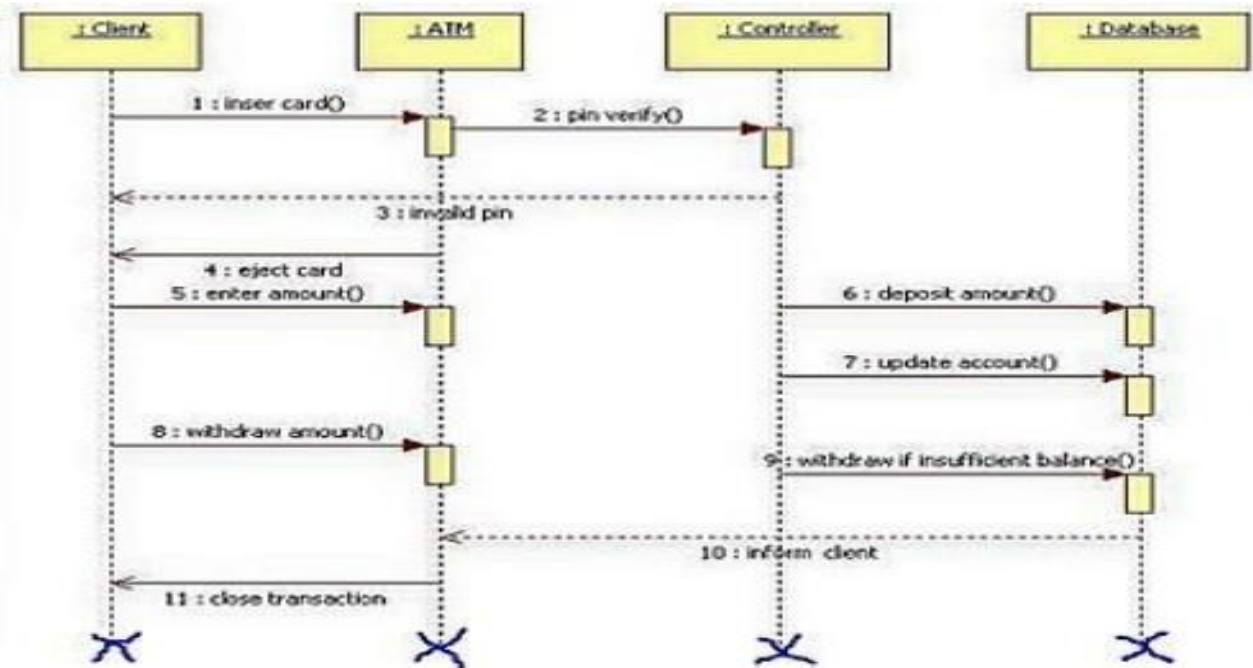


Figure 62: Sequence Diagram

To draw sequence diagram follow the following points:

- Identify the scope of the sequence diagram.
- List the use-case steps down the left-hand side.
- Introduce boxes for each actor.
- Introduce controller class (es).
- Introduce a box for each major UI element.
- Introduce a box for each included use-case.
- Identify appropriate messages for each use-case step.
- Add method-invocation box for each invocation of a method.
- Add destruction messages where appropriate.

8.5 Conceptual Modeling: Class Diagram

Class models are the mainstay of the OO analysis and design. Class models show the classes of the system, their interrelationships (including inheritance, aggregation, and association) and the operations and attributes of the classes. The conceptual models are used to depict your detailed understanding of the problem space and solution for your system.

The easiest way to begin conceptual modeling is to convert the CRC (as a base) directly to UML class diagram. While a CRC model provides an excellent overview of a system, it doesn't provide the details needed to actually build it.

For each card in the CRC model, you create a concrete class in the class diagram, with the exception of cards that represent actors (actors exist in the real world). The CRC model for the University represented here below:

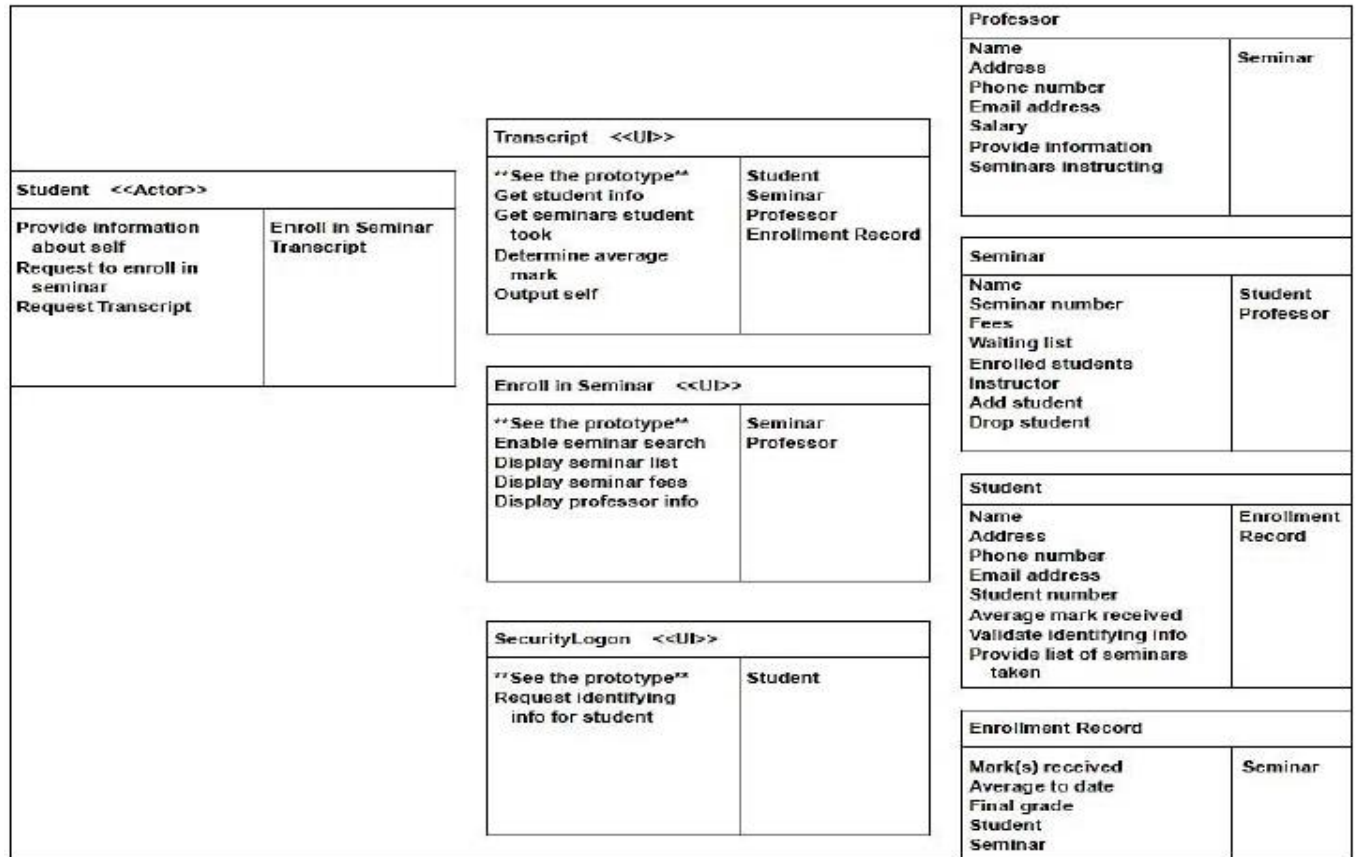


Figure 63: CRC Model of the University

Notice how the names stayed the same (spaces were removed from the names to follow the naming convention of class name). The collaborators on the CRC cards indicate the need for an association, aggregation, association, or dependency between classes.

Dependencies are modeled between UI classes and the business classes with which they collaborate because UI classes are transitory in nature, implying the associations they are involved with are transitory. The diagram consists of the association and dependency between classes. Implied the associations they are involved with is transitory, and hence should be modeled as dependencies. The corresponding class model of the CRC model:

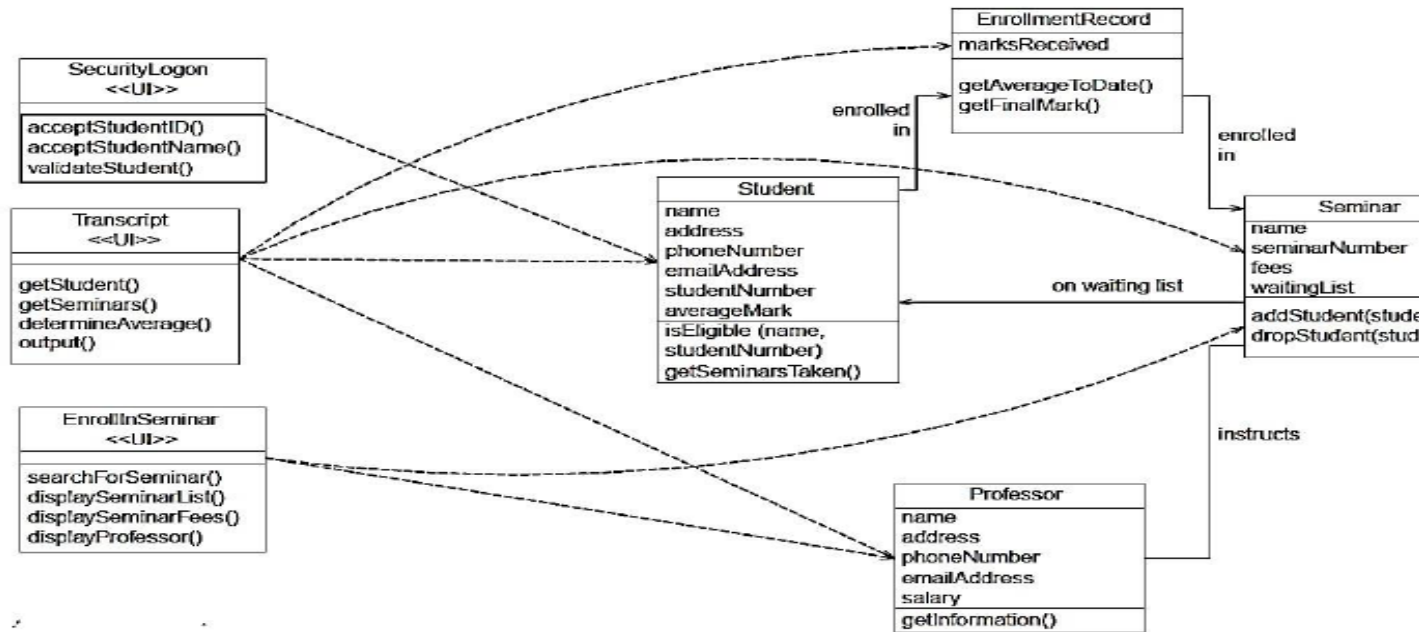


Figure 64: Class Diagram Based on the CRC Model

The class models contain a wealth of information and can be used for both analysis and design of systems. To create and evolve a class model, you need to model:

- Classes
- Methods or operations
- Attributes
- Associations
- Dependencies

A class is a representation of an object and is simply a template from which objects are created. Classes form the main building blocks of an object-oriented application. Classes are a collection of similar objects. Attributes are the information stored about an object (at least information temporarily maintained about an object), while methods are the things an object or class does. Here below, the concrete classes that made class diagrams:

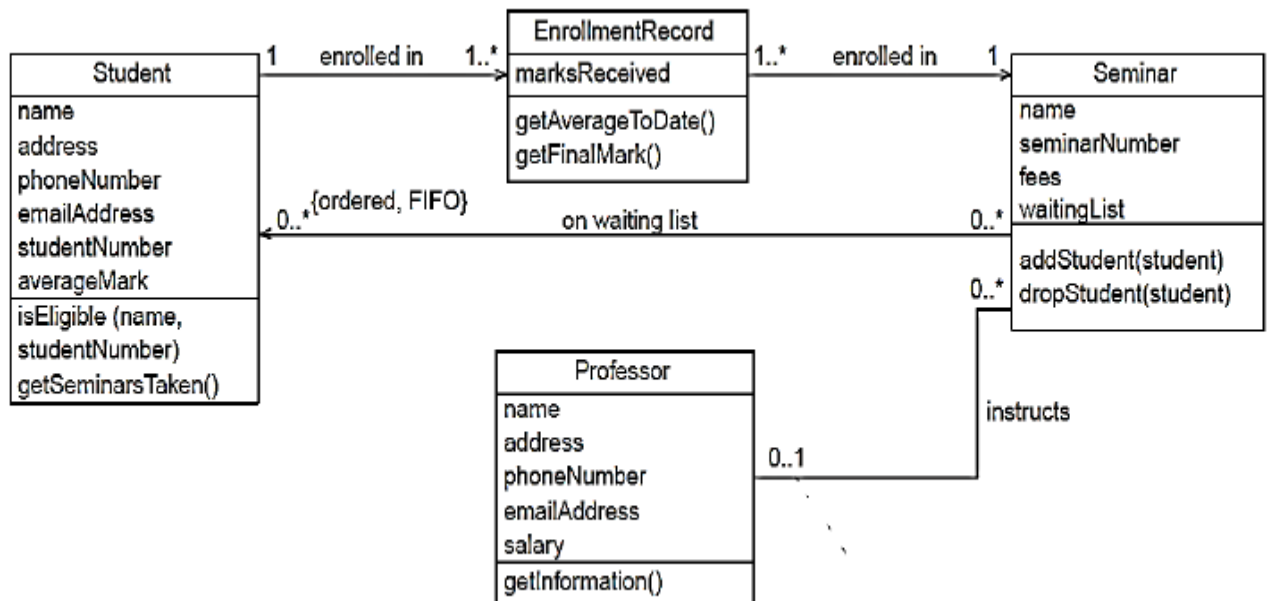


Figure 65: Class Diagram

8.6 Activity Diagram

Activity diagrams are used to document the logic of a single operation/ method, a single use case, or the flow of logic of a business process. In many ways, it is the object-oriented equivalent of flow charts and data-flow diagrams (DFD) from structured development.

Example:

The activity diagram for how someone new to the university would enroll for the first time:

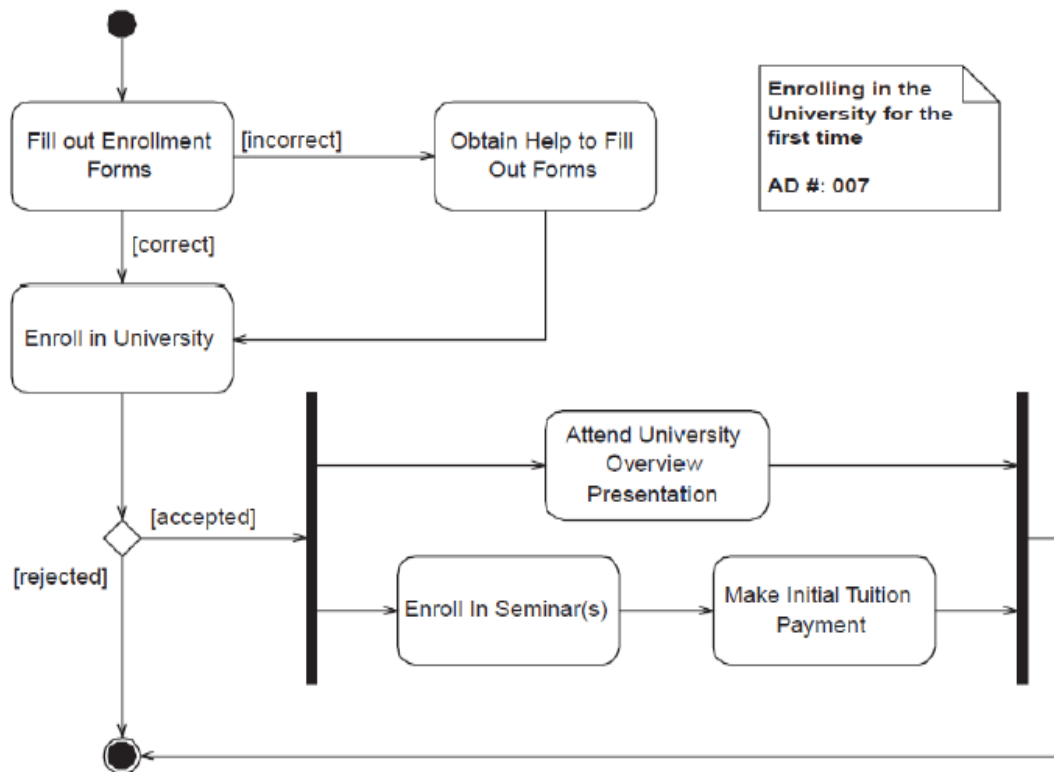


Figure 66: Activity Diagram

The filled circle represents the starting point of the activity diagram – effectively a placeholder – and the filled circle with a border represents the ending point.

The rounded rectangles represent processes or activities that are performed. The activities map reasonably closely to use-cases.

The diamond represents decision points. The decision point had only two possible outcomes. The arrows represent transitions between activities, modeling the flow order between the various activities.

The text on the arrows represent conditions that must be fulfilled to proceed along the transition and are always described using the format [condition].

The thick bars represent the start and end of potentially parallel processes – after you are successfully enrolled in the university, you must attend the mandatory overview presentation and others.

To draw activity diagram consider the following points:

- Identify the scope of the activity diagram. Begin by identifying what you are modeling. Is it a single use case? A portion of a use case? A business process that includes several use-cases? A single method of a class?

- Add start and end points. Every activity diagram has one starting point and one ending point, so you might as well add them right way.
- Add activities. If you are modeling a use-case, introduce an activity for each major step initiated by an actor.
- Add transitions from the activities. It is always to exit from the activity, even if it is simply to an ending point. Whenever there is more than one transition out of an activity, you must label each transition appropriately.
- Add decision points. Sometimes the logic of what you are modeling calls for a decision to be made.
- Identify opportunities for parallel activities. Two activities can occur in parallel when no direct r/ship exists between them and they must both occur before a third activity can.

8.7 User Interface Prototyping Evolving Sour Supplementary Specification

It is an iterative analysis technique in which users are actively involved in the mocking-up of the UI for a system. UI prototyping has two purposes:

1. It is an analysis technique because it enables you to explore the problem space your system addresses.
2. It enables you to explore the solution space of your system, at least from the point-of-view of its users, and provides a vehicle for you to communicate the possible UI design(s) of your system.

There are four high-level steps are in the UI prototyping process:

- a. Determine the needs of your users
- b. Build the prototype
- c. Evaluate the prototype
- d. Determine if you are finished.

It is the iterative steps of prototyping, and summed-up as follows:

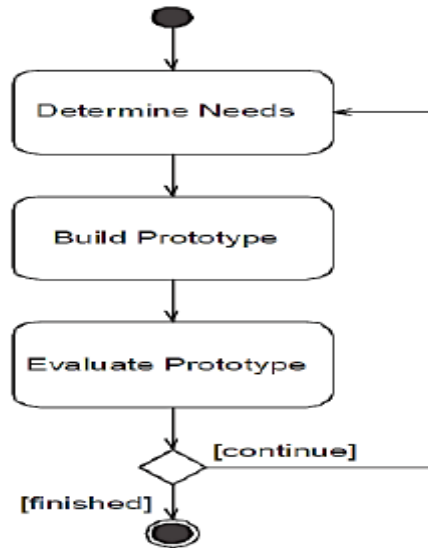


Figure 67: Steps of Prototyping

The following are prototyping Tips and Techniques:

- Work with the real users.
- Use a prototyping tool.
- Get your SMEs (Small-Medium Enterprise) to work with the prototype.
- Understand the underlying business.
- Don't spend a lot of time making the code good.
- Only prototype features that you can actually build.
- Get an interface expert to help you design it.

A good thing to understand about prototyping is to differentiate between the concepts of —WYSIWYG”, —What You See Is What You Get” and —WYSIWYN”, What You See Is What You Need”. During analysis, you will evolve your understanding of the contents of your supplementary specifications. This includes fleshing out the constraints, business rules and nonfunctional requirements.

8.8 Applying Analysis Patterns Effectively

Analysis pattern describe solutions to common problem found in the analysis/ business domain of a system. It is typically more specific than design patterns, because they describe a solution for a portion of a business domain. This doesn't mean an analysis pattern is applicable only to a single line of business, although it could be. The basic analysis patterns are:

- The Business Entity Analysis Pattern: The basic idea of this pattern is to separate the concepts of a business entity, such as a person or company, from the roles it fulfills.
Example:

Suppose Mr. Hosty may be a customer of your organization, as well as an employee. Furthermore, one day he may also sell services to your company, also making him a supplier. The person doesn't change, but the role(s) he has with your organization does, so you need to find a way to model this, which is what this pattern does.

Each business entity has one or more roles with your organization and each role has a range during which it was applicable (the —start|| and —end|| attributes). Each role implements the behavior specific to it.

- The Contact Point Analysis Pattern: It describes an approach for keeping track of the way your organization interacts with business entities. The basic idea behind this pattern is that surface addresses, email addresses, and phone numbers are really the same sort of thing – a means by which you can contact other business entities.

Some Potential Advantages of Patterns:

- Patterns increase developer productivity.
- Patterns describe proven solutions to common problems.
- Patterns increase the consistency between applications.
- Patterns are potentially better than reusable code.
- More and more patterns are being developed every day.

8.9 Organizing your Models with Packages

Packages are UML constructs that enable you to organize model elements into groups, making your UML diagrams simpler and easier to understand. Packages are depicted as file folders and can be used on any of the UML diagrams, although they are most common on use case diagrams and class diagrams because these models have tendency to grow.

UNIT - IX

9.1 Layering Your Models

9.1.1 State Chart modeling

A state chart diagram shows a state machine that depicts the control flow of an object from one state to another. A state machine portrays the sequences of states which an object undergoes due to events and their responses to events. State Chart Diagrams comprise of:

- States: Simple or Composite
- Transitions between states
- Events causing transitions
- Actions due to the events

State-chart diagrams are used for modeling objects which are reactive in nature. Example: In the Automated Trading House System, let us model Order as an object and trace its sequence. The following figure shows the corresponding state chart diagram:

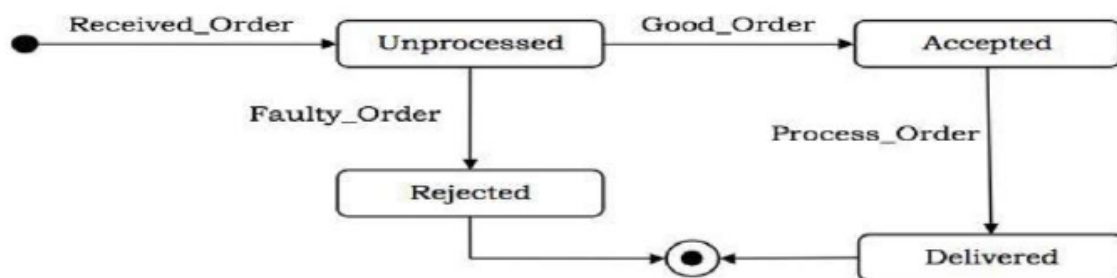


Figure 68: State Chart Diagram

9.1.2 Collaboration Modeling

Collaboration diagrams are interaction diagrams that illustrate the structure of the objects that send and receive messages. In these diagrams, the objects that participate in the interaction are shown using vertices. The links that connect the objects are used to send and receive messages. The message is shown as a labeled arrow. Sequence is shown by including a sequence number on the message.

A collaboration diagram is formed by:

- Placing the objects that participate in the interaction as the vertices in a graph
- Rendering the links that connect these objects as the arcs of this graph
- Adorning these links with the messages that the objects send and receive

Collaboration diagram has four key elements:

- Objects appear at the vertices of the graph
- Paths that indicate how one object is linked to another
- Sequence number to indicate the time order of a message
- Messages show the actions that objects perform on each other and on themselves

Example:

Collaboration diagram for the Automated Trading House System is illustrated in the figure below:

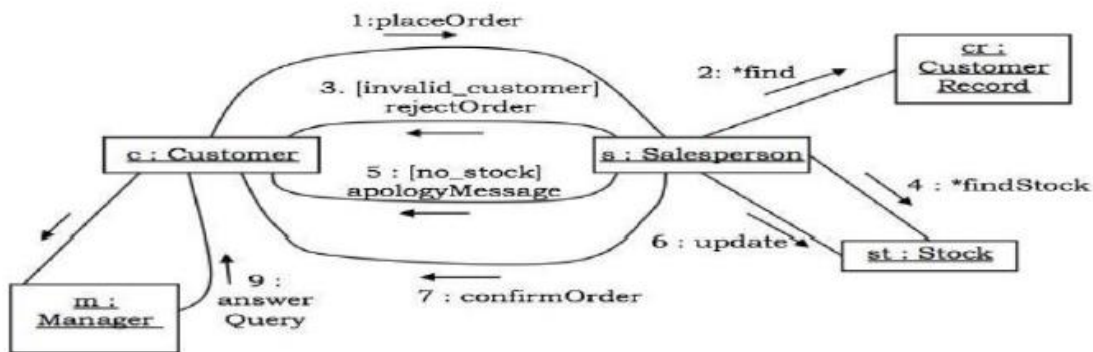


Figure 69: Collaboration Diagram

9.1.3 Component Modeling

Component diagrams show the organization and dependencies among a group of components. Component diagrams comprise of:

- Components
- Interfaces
- Relationships
- Packages and Subsystems (optional) Component diagrams are used for: Constructing systems through forward and reverse engineering. Modeling configuration management of source code files while developing a system using an object-oriented programming language.
- Representing schemas in modeling databases.
- Modeling behaviors of dynamic systems.

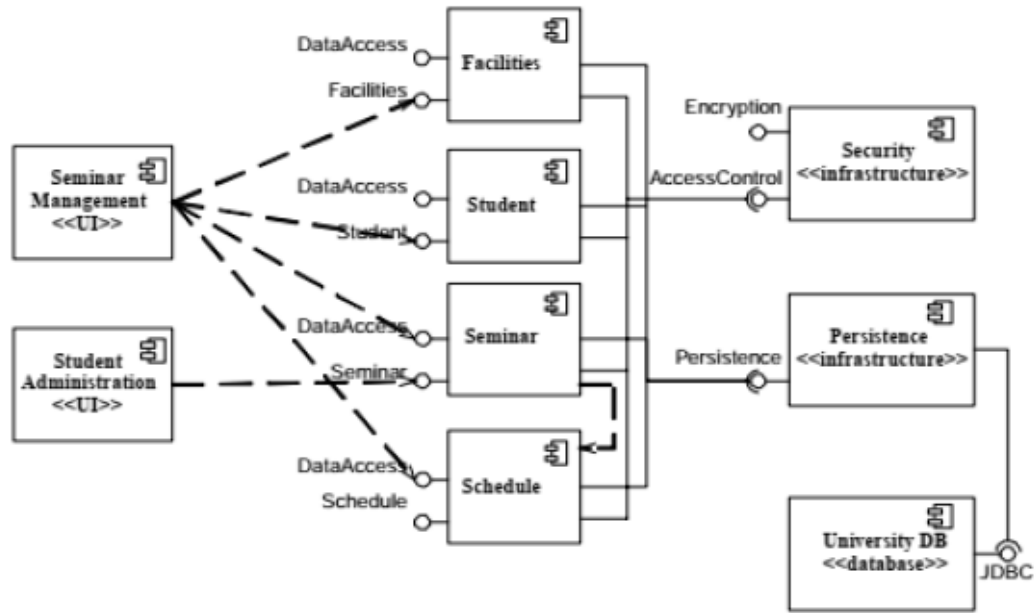


Figure 70: Component Diagram

9.1.4 Deployment Modeling

A deployment diagram puts emphasis on the configuration of runtime processing nodes and their components that live on them. They are commonly comprised of nodes and dependencies, or associations between the nodes. Deployment diagrams are used to:

- Model devices in embedded systems that typically comprise of software-intensive collection of hardware.
- Represent the topologies of client/server systems.
- Model fully distributed systems.

Example: The following figure shows the topology of a computer system that follows client/server architecture. The figure illustrates a node stereotyped as server that comprises of processors. The figure indicates that four or more servers are deployed at the system. Connected to the server are the client nodes, where each node represents a terminal device such as workstation, laptop, scanner, or printer.

The nodes are represented using icons that clearly depict the real-world equivalent as shown below:

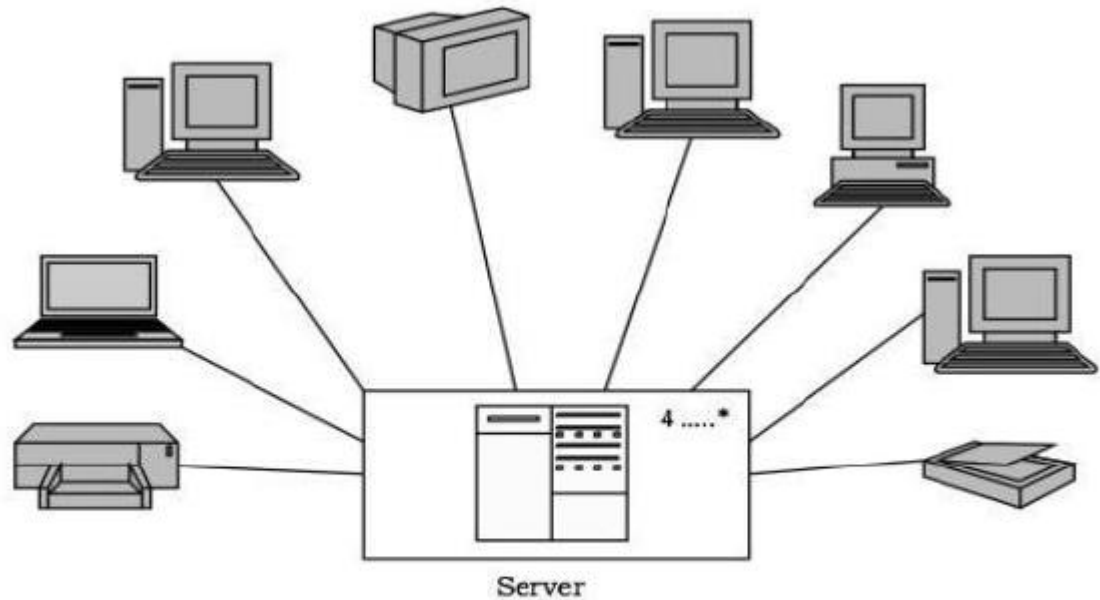


Figure 71: Deployment Diagram

9.1.5 Persistence Modeling

If a system wants to have the data irrespective of a single execution instance, there will be a need to have a facility to put the data permanently.

The persistent objects are often extracted from the application domain. For example, for a sales system you need to keep the customer, sales items, sales invoices, and similar domain objects persistently. One of the options to store data is through the use of relational databases which needs transformation of the class diagrams (from class diagrams to RDBs).

Rules to transform/convert class diagrams to the relational database:

1. Classes are converted/mapped to tables. The attributes become columns
2. Inheritance:
 - Map in to a single table(type attribute could be used to differentiate the two classes)
 - Map the sub classes in to tables
 - Map all classes in to tables
3. Mapping relationship:
 - One-to-one: Maintain through a foreign key in either of the table
 - One-to-many: Maintained through a foreign key on the many side

- Many-to-Many: Maintain it through another intermediate associate table

9.2 Applying Design Patterns Effectively

While designing applications, some commonly accepted solutions are adopted for some categories of problems. These are the patterns of design. A pattern can be defined as a documented set of building blocks that can be used in certain types of application development problems. Some commonly used design patterns are:

- Façade pattern
- Model view separation pattern
- Observer pattern
- Model view controller pattern
- Publish subscribe pattern
- Proxy pattern

References

1. Ireland, R., Hughes, B., West, B., Smith, N., & Shepherd, D. I. (2012). Project Management for It-Related Projects. BCS, The Chartered Institute.
2. Willcocks, L. P., & Sykes, R. (2000). Enterprise resource planning: the role of the CIO and it function in ERP. *Communications of the ACM*, 43(4), 32-38.
3. Mitchell, V. L., & Zmud, R. W. (1999). The effects of coupling IT and work process strategies in redesign projects. *Organization Science*, 10(4), 424-438.
4. A Guide to the Project Management Body of Knowledge, Project Management Institute, Pennsylvania, USA (2000)
5. “Information Technology Project Management” Kathy Schwalbe, International Student Edition, THOMSON Course Technology, 2003
6. Basics of Software Project Management, NIIT, Prentice-Hall India, 2004
7. Software Project Management in Practice, Pankaj Jalote, Pearson Education, 2002
8. Software Project Management, A Concise Study, S.A. Kelkar, Revised Edition, Prentice-Hall India, 2003
9. Joseph S. Valacich, Joey F. George, Jeffrey A. Hoffer, —Essentials of Systems Analysis and Design, 5th edition.
10. Scott W. Ambler, —The Object Primer, 2nd, 3rd editions.

